

ゲームグラフィックス特論

構築: Doxygen 1.15.0

1 ゲームグラフィックス特論の宿題用補助プログラム GLFW3 版.	1
2 OpenXR ラッパー (GgApp::OpenXR) マニュアル&使用方法解説	3
2.1 目次	3
2.2 1. 概要と特徴	3
2.2.1 主な特徴	4
2.3 2. クイックスタート (最小の VR プログラム)	4
2.4 3. API リファレンス	5
2.4.1 型定義・定数	5
2.4.2 初期化と終了処理	5
2.4.2.1 static OpenXR& initialize(const Window& window, Xr← ReferenceSpaceType spaceType = XR_REFERENCE_SPACE_← TYPE_STAGE)	5
2.4.2.2 void terminate()	5
2.4.3 フレーム描画ループ	5
2.4.3.1 bool begin()	5
2.4.3.2 void select(int eye)	5
2.4.3.3 void select(int eye, GLfloat* screen, GLfloat* position, GLfloat* orientation)	6
2.4.3.4 void commit(int eye)	6
2.4.3.5 bool submit(bool mirror = true)	6
2.4.4 カメラ・投影変換・視点情報	6
2.4.4.1 GgMatrix getProjectionMatrix(int eye, GLfloat zNear = 0.1f, GLfloat zFar = 100.0f) const	6
2.4.4.2 GgMatrix getViewMatrix(int eye) const	6
2.4.4.3 GgMatrix getPoseMatrix(int eye) const	6
2.4.4.4 GgVector getPosition(int eye) const	6
2.4.4.5 GgQuaternion getOrientation(int eye) const	7
2.4.4.6 const XrFovf& getFov(int eye) const	7
2.4.4.7 const XrPosef& getPose(int eye) const	7
2.4.4.8 GLsizei getWidth(int eye = 0) const/GLsizei getHeight(int eye = 0) const	7
2.4.4.9 GLfloat getAspect(int eye = 0) const	7
2.4.4.10 uint32_t getViewCount() const	7
2.4.5 コントローラトラッキング・姿勢	7
2.4.5.1 bool isTracked(int hand) const	7
2.4.5.2 GgMatrix getGripMatrix(int hand) const	7
2.4.5.3 GgMatrix getAimMatrix(int hand) const	7
2.4.5.4 GgVector getGripPosition(int hand) const / GgQuaternion getGripOrientation(int hand) const	7
2.4.5.5 GgVector getAimPosition(int hand) const / GgQuaternion getAimOrientation(int hand) const	7
2.4.6 コントローラーボタン・スティック入力・振動	7
2.4.6.1 float getTrigger(int hand) const	7

2.4.6.2	<code>float getGrip(int hand) const</code>	7
2.4.6.3	<code>std::array<float, 2> getThumbstick(int hand) const</code>	8
2.4.6.4	<code>bool getThumbstickClick(int hand) const</code>	8
2.4.6.5	<code>bool getPrimaryButton(int hand) const</code>	8
2.4.6.6	<code>bool getSecondaryButton(int hand) const</code>	8
2.4.6.7	<code>bool getMenuButton(int hand = Hand::Left) const</code>	8
2.4.6.8	<code>void applyHapticVibration(int hand, float duration← Seconds = 0.1f, float frequency = XR.FREQUENCY_UNSPECIFIED, float amplitude = 0.5f)</code>	8
2.4.7	参照空間 (空間座標系)	8
2.4.7.1	<code>XrReferenceSpaceType getReferenceSpaceType() const</code>	8
2.5.4	実践チュートリアル&使用例	8
2.5.1.4.1	基本的なステレオレンダリング	8
2.5.2.4.2	コントローラーの姿勢取得とモデル描画	9
2.5.3.4.3	スティック入力による移動とトリガー入力	9
2.5.4.4.4	床面基準 (STAGE) と着席基準 (LOCAL) の切り替え	9
2.5.5.4.5	PC ウィンドウへのミラー表示	9
2.6.5	旧 Oculus SDK (LibOVR) ラッパーからの移行ガイド	10
2.6.1	コードの対比 (Before / After)	10
2.6.1.1	[Before] 旧 API	10
2.6.1.2	[After] 最適化版 API	10
2.7.6	動作環境とセットアップ	11
2.7.1	必要な環境	11
2.7.2	プロジェクトへの組み込み	11
3	ggsample01	13
3.1	ゲームグラフィックス特論A 第1回 宿題	13
3.2	宿題プログラムの作成に必要な環境	13
3.3	宿題の提出	13
3.4	補足	13
3.5	宿題プログラム用補助プログラムについて	14
3.5.1	補助プログラムのドキュメント	14
3.5.2	補助プログラムの使い方	14
3.5.3	OpenXR を使う場合	15
3.5.4	Dear ImGui を使う場合	15
3.5.4.1	imconfig.h の変更点	16
4	名前空間索引	17
4.1	名前空間一覧	17
5	階層索引	19
5.1	クラス階層	19
6	クラス索引	21

6.1 クラス一覧	21
7 ファイル索引	23
7.1 ファイル一覧	23
8 名前空間詳解	25
8.1 gg 名前空間	25
8.1.1 詳解	27
8.1.2 列挙型詳解	28
8.1.2.1 BindingPoints	28
8.1.3 関数詳解	28
8.1.3.1 _ggError()	28
8.1.3.2 _ggFBOError()	28
8.1.3.3 ggArraysObj()	28
8.1.3.4 ggConjugate()	29
8.1.3.5 ggCreateComputeShader()	30
8.1.3.6 ggCreateNormalMap()	30
8.1.3.7 ggCreateShader()	31
8.1.3.8 ggCross() [1/2]	32
8.1.3.9 ggCross() [2/2]	32
8.1.3.10 ggDistance3() [1/2]	33
8.1.3.11 ggDistance3() [2/2]	33
8.1.3.12 ggDistance4() [1/2]	34
8.1.3.13 ggDistance4() [2/2]	35
8.1.3.14 ggDot3() [1/2]	36
8.1.3.15 ggDot3() [2/2]	36
8.1.3.16 ggDot4() [1/2]	37
8.1.3.17 ggDot4() [2/2]	37
8.1.3.18 ggElementsMesh()	38
8.1.3.19 ggElementsObj()	39
8.1.3.20 ggElementsSphere()	40
8.1.3.21 ggEllipse()	40
8.1.3.22 ggEulerQuaternion() [1/3]	42
8.1.3.23 ggEulerQuaternion() [2/3]	42
8.1.3.24 ggEulerQuaternion() [3/3]	43
8.1.3.25 ggFrustum()	44
8.1.3.26 ggIdentity()	45
8.1.3.27 ggIdentityQuaternion()	45
8.1.3.28 ggInit()	45
8.1.3.29 ggInvert() [1/2]	46
8.1.3.30 ggInvert() [2/2]	47
8.1.3.31 ggLength3() [1/2]	47
8.1.3.32 ggLength3() [2/2]	48

8.1.3.33 ggLength4() [1/2]	48
8.1.3.34 ggLength4() [2/2]	49
8.1.3.35 ggLoadComputeShader()	50
8.1.3.36 ggLoadHeight()	50
8.1.3.37 ggLoadImage()	51
8.1.3.38 ggLoadShader() [1/2]	52
8.1.3.39 ggLoadShader() [2/2]	52
8.1.3.40 ggLoadSimpleObj() [1/2]	53
8.1.3.41 ggLoadSimpleObj() [2/2]	54
8.1.3.42 ggLoadTexture()	54
8.1.3.43 ggLookat() [1/3]	55
8.1.3.44 ggLookat() [2/3]	56
8.1.3.45 ggLookat() [3/3]	56
8.1.3.46 ggMatrixQuaternion() [1/2]	57
8.1.3.47 ggMatrixQuaternion() [2/2]	58
8.1.3.48 ggNorm()	59
8.1.3.49 ggNormal()	59
8.1.3.50 ggNormalize()	59
8.1.3.51 ggNormalize3() [1/4]	60
8.1.3.52 ggNormalize3() [2/4]	60
8.1.3.53 ggNormalize3() [3/4]	61
8.1.3.54 ggNormalize3() [4/4]	62
8.1.3.55 ggNormalize4() [1/4]	62
8.1.3.56 ggNormalize4() [2/4]	63
8.1.3.57 ggNormalize4() [3/4]	63
8.1.3.58 ggNormalize4() [4/4]	64
8.1.3.59 ggOrthogonal()	64
8.1.3.60 ggPerspective()	65
8.1.3.61 ggPointsCube()	66
8.1.3.62 ggPointsSphere()	66
8.1.3.63 ggQuaternionMatrix()	67
8.1.3.64 ggQuaternionTransposeMatrix()	67
8.1.3.65 ggReadImage()	68
8.1.3.66 ggRectangle()	69
8.1.3.67 ggRotate() [1/5]	69
8.1.3.68 ggRotate() [2/5]	70
8.1.3.69 ggRotate() [3/5]	70
8.1.3.70 ggRotate() [4/5]	71
8.1.3.71 ggRotate() [5/5]	71
8.1.3.72 ggRotateQuaternion() [1/3]	72
8.1.3.73 ggRotateQuaternion() [2/3]	73
8.1.3.74 ggRotateQuaternion() [3/3]	73

8.1.3.75 ggRotateX()	74
8.1.3.76 ggRotateY()	74
8.1.3.77 ggRotateZ()	75
8.1.3.78 ggSaveColor()	75
8.1.3.79 ggSaveDepth()	76
8.1.3.80 ggSaveTga()	76
8.1.3.81 ggScale() [1/3]	77
8.1.3.82 ggScale() [2/3]	78
8.1.3.83 ggScale() [3/3]	78
8.1.3.84 ggSlerp() [1/4]	79
8.1.3.85 ggSlerp() [2/4]	79
8.1.3.86 ggSlerp() [3/4]	80
8.1.3.87 ggSlerp() [4/4]	81
8.1.3.88 ggTranslate() [1/3]	81
8.1.3.89 ggTranslate() [2/3]	82
8.1.3.90 ggTranslate() [3/3]	82
8.1.3.91 ggTranspose()	84
8.1.3.92 operator*()	84
8.1.3.93 operator+() [1/2]	86
8.1.3.94 operator+() [2/2]	86
8.1.3.95 operator-() [1/2]	86
8.1.3.96 operator-() [2/2]	87
8.1.3.97 operator/()	87
8.1.4 変数詳解	87
8.1.4.1 ggBufferAlignment	87
9 クラス詳解	89
9.1 Config クラス	89
9.1.1 詳解	89
9.1.2 構築子と解体子	89
9.1.2.1 Config() [1/2]	89
9.1.2.2 Config() [2/2]	90
9.1.2.3 ~Config()	90
9.1.3 関数詳解	90
9.1.3.1 getHeight()	90
9.1.3.2 getWidth()	91
9.1.3.3 load()	91
9.1.3.4 save()	92
9.1.4 フレンドと関連関数の詳解	93
9.1.4.1 Menu	93
9.2 GgApp クラス	93
9.2.1 詳解	94

9.2.2 構築子と解体子	94
9.2.2.1 GgApp() [1/3]	94
9.2.2.2 GgApp() [2/3]	94
9.2.2.3 GgApp() [3/3]	95
9.2.2.4 ~GgApp()	95
9.2.3 関数詳解	95
9.2.3.1 getUsername()	95
9.2.3.2 main()	96
9.2.3.3 operator=() [1/2]	97
9.2.3.4 operator=() [2/2]	98
9.3 gg::GgBuffer<T> クラステンプレート	98
9.3.1 詳解	99
9.3.2 構築子と解体子	99
9.3.2.1 GgBuffer() [1/3]	99
9.3.2.2 GgBuffer() [2/3]	100
9.3.2.3 GgBuffer() [3/3]	100
9.3.2.4 ~GgBuffer()	101
9.3.3 関数詳解	101
9.3.3.1 bind()	101
9.3.3.2 copy()	101
9.3.3.3 getBuffer()	102
9.3.3.4 getCount()	102
9.3.3.5 getStride()	103
9.3.3.6 getTarget()	103
9.3.3.7 map() [1/2]	103
9.3.3.8 map() [2/2]	104
9.3.3.9 operator=() [1/2]	104
9.3.3.10 operator=() [2/2]	106
9.3.3.11 read()	106
9.3.3.12 send()	107
9.3.3.13 unbind()	108
9.3.3.14 unmap()	108
9.4 gg::GgColorTexture クラス	108
9.4.1 詳解	108
9.4.2 構築子と解体子	108
9.4.2.1 GgColorTexture() [1/3]	108
9.4.2.2 GgColorTexture() [2/3]	109
9.4.2.3 GgColorTexture() [3/3]	109
9.4.2.4 ~GgColorTexture()	110
9.4.3 関数詳解	110
9.4.3.1 bind()	110
9.4.3.2 get()	110

9.4.3.3 getHeight()	110
9.4.3.4 getSize() [1/2]	110
9.4.3.5 getSize() [2/2]	111
9.4.3.6 getTexture()	111
9.4.3.7 getWidth()	111
9.4.3.8 load() [1/2]	111
9.4.3.9 load() [2/2]	112
9.4.3.10 operator bool()	112
9.4.3.11 operator"!()	113
9.4.3.12 unbind()	113
9.5 gg::GgElements クラス	113
9.5.1 詳解	115
9.5.2 構築子と解体子	115
9.5.2.1 GgElements() [1/2]	115
9.5.2.2 GgElements() [2/2]	115
9.5.2.3 ~GgElements()	116
9.5.3 関数詳解	116
9.5.3.1 draw()	116
9.5.3.2 getIndexBuffer()	117
9.5.3.3 getIndexCount()	117
9.5.3.4 load()	117
9.5.3.5 send()	118
9.6 gg::GgMatrix クラス	119
9.6.1 詳解	121
9.6.2 構築子と解体子	121
9.6.2.1 GgMatrix() [1/6]	121
9.6.2.2 GgMatrix() [2/6]	121
9.6.2.3 GgMatrix() [3/6]	122
9.6.2.4 GgMatrix() [4/6]	123
9.6.2.5 GgMatrix() [5/6]	123
9.6.2.6 GgMatrix() [6/6]	124
9.6.2.7 ~GgMatrix()	124
9.6.3 関数詳解	124
9.6.3.1 frustum()	124
9.6.3.2 get() [1/2]	125
9.6.3.3 get() [2/2]	126
9.6.3.4 invert()	127
9.6.3.5 loadFrustum()	127
9.6.3.6 loadIdentity()	128
9.6.3.7 loadInvert() [1/2]	128
9.6.3.8 loadInvert() [2/2]	129
9.6.3.9 loadLookat() [1/3]	129

9.6.3.10 loadLookat() [2/3]	131
9.6.3.11 loadLookat() [3/3]	132
9.6.3.12 loadNormal() [1/2]	133
9.6.3.13 loadNormal() [2/2]	134
9.6.3.14 loadOrthogonal()	134
9.6.3.15 loadPerspective()	135
9.6.3.16 loadRotate() [1/5]	135
9.6.3.17 loadRotate() [2/5]	137
9.6.3.18 loadRotate() [3/5]	138
9.6.3.19 loadRotate() [4/5]	138
9.6.3.20 loadRotate() [5/5]	139
9.6.3.21 loadRotateX()	140
9.6.3.22 loadRotateY()	141
9.6.3.23 loadRotateZ()	141
9.6.3.24 loadScale() [1/3]	142
9.6.3.25 loadScale() [2/3]	143
9.6.3.26 loadScale() [3/3]	143
9.6.3.27 loadTranslate() [1/3]	144
9.6.3.28 loadTranslate() [2/3]	145
9.6.3.29 loadTranslate() [3/3]	145
9.6.3.30 loadTranspose() [1/2]	146
9.6.3.31 loadTranspose() [2/2]	147
9.6.3.32 lookat() [1/3]	147
9.6.3.33 lookat() [2/3]	148
9.6.3.34 lookat() [3/3]	148
9.6.3.35 normal()	149
9.6.3.36 operator*() [1/3]	150
9.6.3.37 operator*() [2/3]	151
9.6.3.38 operator*() [3/3]	151
9.6.3.39 operator*=() [1/2]	152
9.6.3.40 operator*=() [2/2]	153
9.6.3.41 operator+() [1/2]	153
9.6.3.42 operator+() [2/2]	155
9.6.3.43 operator+=() [1/2]	156
9.6.3.44 operator+=() [2/2]	156
9.6.3.45 operator-() [1/2]	157
9.6.3.46 operator-() [2/2]	157
9.6.3.47 operator-=() [1/2]	158
9.6.3.48 operator-=() [2/2]	158
9.6.3.49 operator/() [1/2]	160
9.6.3.50 operator/() [2/2]	161
9.6.3.51 operator/=() [1/2]	161

9.6.3.52 operator/=() [2/2]	162
9.6.3.53 operator=() [1/3]	163
9.6.3.54 operator=() [2/3]	163
9.6.3.55 operator=() [3/3]	163
9.6.3.56 orthogonal()	164
9.6.3.57 perspective()	164
9.6.3.58 projection() [1/4]	165
9.6.3.59 projection() [2/4]	165
9.6.3.60 projection() [3/4]	166
9.6.3.61 projection() [4/4]	166
9.6.3.62 rotate() [1/5]	166
9.6.3.63 rotate() [2/5]	167
9.6.3.64 rotate() [3/5]	167
9.6.3.65 rotate() [4/5]	168
9.6.3.66 rotate() [5/5]	168
9.6.3.67 rotateX()	169
9.6.3.68 rotateY()	170
9.6.3.69 rotateZ()	170
9.6.3.70 scale() [1/3]	171
9.6.3.71 scale() [2/3]	171
9.6.3.72 scale() [3/3]	172
9.6.3.73 translate() [1/3]	173
9.6.3.74 translate() [2/3]	173
9.6.3.75 translate() [3/3]	174
9.6.3.76 transpose()	175
9.7 gg::GgNormalTexture クラス	176
9.7.1 詳解	176
9.7.2 構築子と解体子	176
9.7.2.1 GgNormalTexture() [1/3]	176
9.7.2.2 GgNormalTexture() [2/3]	176
9.7.2.3 GgNormalTexture() [3/3]	177
9.7.2.4 ~GgNormalTexture()	177
9.7.3 関数詳解	177
9.7.3.1 bind()	177
9.7.3.2 get()	178
9.7.3.3 getHeight()	178
9.7.3.4 getSize() [1/2]	178
9.7.3.5 getSize() [2/2]	178
9.7.3.6 getTexture()	178
9.7.3.7 getWidth()	178
9.7.3.8 load() [1/2]	179
9.7.3.9 load() [2/2]	179

9.7.3.10 operator bool()	180
9.7.3.11 operator"!()	180
9.7.3.12 unbind()	180
9.8 gg::GgPoints クラス	180
9.8.1 詳解	182
9.8.2 構築子と解体子	182
9.8.2.1 GgPoints() [1/2]	182
9.8.2.2 GgPoints() [2/2]	182
9.8.2.3 ~GgPoints()	183
9.8.3 関数詳解	183
9.8.3.1 draw()	183
9.8.3.2 getBuffer()	183
9.8.3.3 getCount()	184
9.8.3.4 load()	184
9.8.3.5 operator bool()	185
9.8.3.6 operator"!()	185
9.8.3.7 send()	185
9.9 gg::GgPointShader クラス	185
9.9.1 詳解	186
9.9.2 構築子と解体子	186
9.9.2.1 GgPointShader() [1/3]	186
9.9.2.2 GgPointShader() [2/3]	187
9.9.2.3 GgPointShader() [3/3]	188
9.9.2.4 ~GgPointShader()	188
9.9.3 関数詳解	188
9.9.3.1 get()	188
9.9.3.2 load() [1/2]	189
9.9.3.3 load() [2/2]	190
9.9.3.4 loadMatrix() [1/2]	190
9.9.3.5 loadMatrix() [2/2]	191
9.9.3.6 loadModelviewMatrix() [1/2]	192
9.9.3.7 loadModelviewMatrix() [2/2]	192
9.9.3.8 loadProjectionMatrix() [1/2]	193
9.9.3.9 loadProjectionMatrix() [2/2]	194
9.9.3.10 unuse()	194
9.9.3.11 use() [1/5]	194
9.9.3.12 use() [2/5]	195
9.9.3.13 use() [3/5]	196
9.9.3.14 use() [4/5]	196
9.9.3.15 use() [5/5]	197
9.10 gg::GgQuaternion クラス	197
9.10.1 詳解	201

9.10.2 構築子と解体子	201
9.10.2.1 GgQuaternion() [1/7]	201
9.10.2.2 GgQuaternion() [2/7]	201
9.10.2.3 GgQuaternion() [3/7]	202
9.10.2.4 GgQuaternion() [4/7]	202
9.10.2.5 GgQuaternion() [5/7]	203
9.10.2.6 GgQuaternion() [6/7]	203
9.10.2.7 GgQuaternion() [7/7]	204
9.10.2.8 ~GgQuaternion()	204
9.10.3 関数詳解	204
9.10.3.1 add() [1/4]	204
9.10.3.2 add() [2/4]	204
9.10.3.3 add() [3/4]	205
9.10.3.4 add() [4/4]	205
9.10.3.5 conjugate()	207
9.10.3.6 divide() [1/4]	208
9.10.3.7 divide() [2/4]	209
9.10.3.8 divide() [3/4]	209
9.10.3.9 divide() [4/4]	210
9.10.3.10 euler() [1/3]	210
9.10.3.11 euler() [2/3]	211
9.10.3.12 euler() [3/3]	212
9.10.3.13 get()	212
9.10.3.14 getConjugateMatrix() [1/3]	213
9.10.3.15 getConjugateMatrix() [2/3]	213
9.10.3.16 getConjugateMatrix() [3/3]	214
9.10.3.17 getMatrix() [1/3]	214
9.10.3.18 getMatrix() [2/3]	215
9.10.3.19 getMatrix() [3/3]	216
9.10.3.20 invert()	216
9.10.3.21 loadAdd() [1/4]	217
9.10.3.22 loadAdd() [2/4]	217
9.10.3.23 loadAdd() [3/4]	218
9.10.3.24 loadAdd() [4/4]	218
9.10.3.25 loadConjugate() [1/2]	219
9.10.3.26 loadConjugate() [2/2]	220
9.10.3.27 loadDivide() [1/4]	220
9.10.3.28 loadDivide() [2/4]	221
9.10.3.29 loadDivide() [3/4]	221
9.10.3.30 loadDivide() [4/4]	222
9.10.3.31 loadEuler() [1/2]	223
9.10.3.32 loadEuler() [2/2]	223

9.10.3.33 loadIdentity()	224
9.10.3.34 loadInvert() [1/2]	225
9.10.3.35 loadInvert() [2/2]	225
9.10.3.36 loadMatrix() [1/2]	226
9.10.3.37 loadMatrix() [2/2]	227
9.10.3.38 loadMultiply() [1/4]	228
9.10.3.39 loadMultiply() [2/4]	228
9.10.3.40 loadMultiply() [3/4]	228
9.10.3.41 loadMultiply() [4/4]	230
9.10.3.42 loadNormalize() [1/2]	231
9.10.3.43 loadNormalize() [2/2]	231
9.10.3.44 loadRotate() [1/3]	232
9.10.3.45 loadRotate() [2/3]	232
9.10.3.46 loadRotate() [3/3]	234
9.10.3.47 loadRotateX()	235
9.10.3.48 loadRotateY()	236
9.10.3.49 loadRotateZ()	236
9.10.3.50 loadSlerp() [1/4]	237
9.10.3.51 loadSlerp() [2/4]	238
9.10.3.52 loadSlerp() [3/4]	238
9.10.3.53 loadSlerp() [4/4]	239
9.10.3.54 loadSubtract() [1/4]	240
9.10.3.55 loadSubtract() [2/4]	240
9.10.3.56 loadSubtract() [3/4]	241
9.10.3.57 loadSubtract() [4/4]	241
9.10.3.58 multiply() [1/4]	242
9.10.3.59 multiply() [2/4]	243
9.10.3.60 multiply() [3/4]	243
9.10.3.61 multiply() [4/4]	244
9.10.3.62 norm()	244
9.10.3.63 normalize()	245
9.10.3.64 operator*() [1/3]	246
9.10.3.65 operator*() [2/3]	246
9.10.3.66 operator*() [3/3]	246
9.10.3.67 operator*=() [1/3]	247
9.10.3.68 operator*=() [2/3]	247
9.10.3.69 operator*=() [3/3]	248
9.10.3.70 operator+() [1/3]	248
9.10.3.71 operator+() [2/3]	248
9.10.3.72 operator+() [3/3]	249
9.10.3.73 operator+=() [1/3]	249
9.10.3.74 operator+=() [2/3]	249

9.10.3.75 operator+=()	[3/3]	250
9.10.3.76 operator-()	[1/3]	250
9.10.3.77 operator-()	[2/3]	250
9.10.3.78 operator-()	[3/3]	250
9.10.3.79 operator-=()	[1/3]	251
9.10.3.80 operator-=()	[2/3]	251
9.10.3.81 operator-=()	[3/3]	251
9.10.3.82 operator/()	[1/3]	252
9.10.3.83 operator/()	[2/3]	252
9.10.3.84 operator/()	[3/3]	253
9.10.3.85 operator/=()	[1/3]	253
9.10.3.86 operator/=()	[2/3]	253
9.10.3.87 operator/=()	[3/3]	254
9.10.3.88 operator=()	[1/4]	254
9.10.3.89 operator=()	[2/4]	255
9.10.3.90 operator=()	[3/4]	255
9.10.3.91 operator=()	[4/4]	256
9.10.3.92 rotate()	[1/3]	256
9.10.3.93 rotate()	[2/3]	257
9.10.3.94 rotate()	[3/3]	257
9.10.3.95 rotateX()		258
9.10.3.96 rotateY()		259
9.10.3.97 rotateZ()		259
9.10.3.98 slerp()	[1/2]	260
9.10.3.99 slerp()	[2/2]	260
9.10.3.100 subtract()	[1/4]	261
9.10.3.101 subtract()	[2/4]	261
9.10.3.102 subtract()	[3/4]	262
9.10.3.103 subtract()	[4/4]	262
9.11 gg::GgShader クラス		263
9.11.1 詳解		264
9.11.2 構築子と解体子		264
9.11.2.1 GgShader()	[1/4]	264
9.11.2.2 GgShader()	[2/4]	265
9.11.2.3 GgShader()	[3/4]	265
9.11.2.4 GgShader()	[4/4]	266
9.11.2.5 ~GgShader()		266
9.11.3 関数詳解		266
9.11.3.1 get()		266
9.11.3.2 operator=()		266
9.11.3.3 unuse()		267
9.11.3.4 use()		267

9.12 gg::GgShape クラス	267
9.12.1 詳解	268
9.12.2 構築子と解体子	268
9.12.2.1 GgShape()	268
9.12.2.2 ~GgShape()	268
9.12.3 関数詳解	268
9.12.3.1 draw()	268
9.12.3.2 get()	269
9.12.3.3 getMode()	269
9.12.3.4 operator bool()	270
9.12.3.5 operator"!()	270
9.12.3.6 setMode()	270
9.13 gg::GgSimpleObj クラス	271
9.13.1 詳解	271
9.13.2 構築子と解体子	271
9.13.2.1 GgSimpleObj()	271
9.13.2.2 ~GgSimpleObj()	271
9.13.3 関数詳解	272
9.13.3.1 draw()	272
9.13.3.2 get()	272
9.13.3.3 operator bool()	272
9.13.3.4 operator"!()	272
9.14 gg::GgSimpleShader クラス	273
9.14.1 詳解	274
9.14.2 構築子と解体子	274
9.14.2.1 GgSimpleShader() [1/4]	274
9.14.2.2 GgSimpleShader() [2/4]	275
9.14.2.3 GgSimpleShader() [3/4]	276
9.14.2.4 GgSimpleShader() [4/4]	276
9.14.2.5 ~GgSimpleShader()	277
9.14.3 関数詳解	277
9.14.3.1 load() [1/2]	277
9.14.3.2 load() [2/2]	277
9.14.3.3 loadMatrix() [1/4]	278
9.14.3.4 loadMatrix() [2/4]	279
9.14.3.5 loadMatrix() [3/4]	279
9.14.3.6 loadMatrix() [4/4]	280
9.14.3.7 loadModelviewMatrix() [1/4]	281
9.14.3.8 loadModelviewMatrix() [2/4]	281
9.14.3.9 loadModelviewMatrix() [3/4]	282
9.14.3.10 loadModelviewMatrix() [4/4]	282
9.14.3.11 operator=()	283

9.14.3.12 use() [1/13]	284
9.14.3.13 use() [2/13]	285
9.14.3.14 use() [3/13]	286
9.14.3.15 use() [4/13]	286
9.14.3.16 use() [5/13]	287
9.14.3.17 use() [6/13]	288
9.14.3.18 use() [7/13]	288
9.14.3.19 use() [8/13]	289
9.14.3.20 use() [9/13]	289
9.14.3.21 use() [10/13]	290
9.14.3.22 use() [11/13]	290
9.14.3.23 use() [12/13]	291
9.14.3.24 use() [13/13]	291
9.15 gg::GgTexture クラス	292
9.15.1 詳解	292
9.15.2 構築子と解体子	293
9.15.2.1 GgTexture() [1/3]	293
9.15.2.2 GgTexture() [2/3]	294
9.15.2.3 GgTexture() [3/3]	294
9.15.2.4 ~GgTexture()	295
9.15.3 関数詳解	295
9.15.3.1 bind()	295
9.15.3.2 getHeight()	295
9.15.3.3 getSize() [1/2]	295
9.15.3.4 getSize() [2/2]	296
9.15.3.5 getTexture()	296
9.15.3.6 getWidth()	296
9.15.3.7 operator=() [1/2]	297
9.15.3.8 operator=() [2/2]	297
9.15.3.9 swapRandB()	297
9.15.3.10 unbind()	298
9.16 gg::GgTrackball クラス	298
9.16.1 詳解	302
9.16.2 構築子と解体子	302
9.16.2.1 GgTrackball()	302
9.16.2.2 ~GgTrackball()	303
9.16.3 関数詳解	303
9.16.3.1 begin()	303
9.16.3.2 end()	303
9.16.3.3 get()	304
9.16.3.4 getMatrix()	304
9.16.3.5 getQuaternion()	304

9.16.3.6	getScale() [1/3]	305
9.16.3.7	getScale() [2/3]	305
9.16.3.8	getScale() [3/3]	305
9.16.3.9	getStart() [1/3]	305
9.16.3.10	getStart() [2/3]	305
9.16.3.11	getStart() [3/3]	306
9.16.3.12	motion()	306
9.16.3.13	operator=()	306
9.16.3.14	region() [1/2]	307
9.16.3.15	region() [2/2]	307
9.16.3.16	reset()	308
9.16.3.17	rotate()	309
9.17	gg::GgTriangles クラス	309
9.17.1	詳解	311
9.17.2	構築子と解体子	311
9.17.2.1	GgTriangles() [1/2]	311
9.17.2.2	GgTriangles() [2/2]	311
9.17.2.3	~GgTriangles()	312
9.17.3	関数詳解	312
9.17.3.1	draw()	312
9.17.3.2	getBuffer()	313
9.17.3.3	getCount()	313
9.17.3.4	load()	313
9.17.3.5	send()	314
9.18	gg::GgUniformBuffer< T > クラステンプレート	314
9.18.1	詳解	315
9.18.2	構築子と解体子	315
9.18.2.1	GgUniformBuffer() [1/3]	315
9.18.2.2	GgUniformBuffer() [2/3]	315
9.18.2.3	GgUniformBuffer() [3/3]	316
9.18.2.4	~GgUniformBuffer()	316
9.18.3	関数詳解	317
9.18.3.1	bind()	317
9.18.3.2	copy()	317
9.18.3.3	fill()	318
9.18.3.4	getBuffer()	319
9.18.3.5	getCount()	320
9.18.3.6	getStride()	320
9.18.3.7	getTarget()	321
9.18.3.8	load() [1/2]	321
9.18.3.9	load() [2/2]	322
9.18.3.10	map() [1/2]	323

9.18.3.11 map() [2/2]	323
9.18.3.12 read()	324
9.18.3.13 send()	325
9.18.3.14 unbind()	326
9.18.3.15 unmap()	326
9.19 gg::GgVector クラス	327
9.19.1 詳解	328
9.19.2 構築子と解体子	328
9.19.2.1 GgVector() [1/6]	328
9.19.2.2 GgVector() [2/6]	329
9.19.2.3 GgVector() [3/6]	330
9.19.2.4 GgVector() [4/6]	330
9.19.2.5 GgVector() [5/6]	331
9.19.2.6 GgVector() [6/6]	331
9.19.2.7 ~GgVector()	331
9.19.3 関数詳解	331
9.19.3.1 distance3()	331
9.19.3.2 distance4()	332
9.19.3.3 dot3()	332
9.19.3.4 dot4()	333
9.19.3.5 length3()	334
9.19.3.6 length4()	334
9.19.3.7 normalize3()	334
9.19.3.8 normalize4()	335
9.19.3.9 operator*() [1/2]	335
9.19.3.10 operator*() [2/2]	335
9.19.3.11 operator*=() [1/2]	336
9.19.3.12 operator*=() [2/2]	336
9.19.3.13 operator+() [1/2]	337
9.19.3.14 operator+() [2/2]	337
9.19.3.15 operator+=() [1/2]	338
9.19.3.16 operator+=() [2/2]	338
9.19.3.17 operator-() [1/2]	339
9.19.3.18 operator-() [2/2]	339
9.19.3.19 operator-=() [1/2]	340
9.19.3.20 operator-=() [2/2]	340
9.19.3.21 operator/() [1/2]	341
9.19.3.22 operator/() [2/2]	341
9.19.3.23 operator/=() [1/2]	342
9.19.3.24 operator/=() [2/2]	342
9.19.3.25 operator=() [1/2]	343
9.19.3.26 operator=() [2/2]	343

9.20 gg::GgVertex 構造体	344
9.20.1 詳解	344
9.20.2 構築子と解体子	344
9.20.2.1 GgVertex() [1/4]	344
9.20.2.2 GgVertex() [2/4]	345
9.20.2.3 GgVertex() [3/4]	345
9.20.2.4 GgVertex() [4/4]	345
9.20.3 メンバ詳解	346
9.20.3.1 normal	346
9.20.3.2 position	346
9.21 gg::GgVertexArray クラス	346
9.21.1 詳解	346
9.21.2 構築子と解体子	347
9.21.2.1 GgVertexArray() [1/3]	347
9.21.2.2 GgVertexArray() [2/3]	347
9.21.2.3 GgVertexArray() [3/3]	348
9.21.2.4 ~GgVertexArray()	348
9.21.3 関数詳解	348
9.21.3.1 bind()	348
9.21.3.2 get()	348
9.21.3.3 operator=()	348
9.22 gg::GgSimpleShader::Light 構造体	349
9.22.1 詳解	350
9.22.2 メンバ詳解	350
9.22.2.1 ambient	350
9.22.2.2 diffuse	350
9.22.2.3 position	350
9.22.2.4 specular	350
9.23 gg::GgSimpleShader::LightBuffer クラス	350
9.23.1 詳解	352
9.23.2 構築子と解体子	352
9.23.2.1 LightBuffer() [1/3]	352
9.23.2.2 LightBuffer() [2/3]	353
9.23.2.3 LightBuffer() [3/3]	353
9.23.2.4 ~LightBuffer()	354
9.23.3 関数詳解	354
9.23.3.1 load() [1/2]	354
9.23.3.2 load() [2/2]	355
9.23.3.3 loadAmbient() [1/3]	355
9.23.3.4 loadAmbient() [2/3]	356
9.23.3.5 loadAmbient() [3/3]	356
9.23.3.6 loadColor()	357

9.23.3.7 loadDiffuse() [1/3]	358
9.23.3.8 loadDiffuse() [2/3]	359
9.23.3.9 loadDiffuse() [3/3]	359
9.23.3.10 loadPosition() [1/4]	360
9.23.3.11 loadPosition() [2/4]	361
9.23.3.12 loadPosition() [3/4]	361
9.23.3.13 loadPosition() [4/4]	362
9.23.3.14 loadSpecular() [1/3]	363
9.23.3.15 loadSpecular() [2/3]	364
9.23.3.16 loadSpecular() [3/3]	364
9.23.3.17 select()	365
9.24 gg::GgSimpleShader::Material 構造体	366
9.24.1 詳解	367
9.24.2 メンバ詳解	367
9.24.2.1 ambient	367
9.24.2.2 diffuse	367
9.24.2.3 shininess	368
9.24.2.4 specular	368
9.25 gg::GgSimpleShader::MaterialBuffer クラス	368
9.25.1 詳解	369
9.25.2 構築子と解体子	370
9.25.2.1 MaterialBuffer() [1/3]	370
9.25.2.2 MaterialBuffer() [2/3]	371
9.25.2.3 MaterialBuffer() [3/3]	371
9.25.2.4 ~MaterialBuffer()	372
9.25.3 関数詳解	372
9.25.3.1 load() [1/2]	372
9.25.3.2 load() [2/2]	373
9.25.3.3 loadAmbient() [1/3]	373
9.25.3.4 loadAmbient() [2/3]	374
9.25.3.5 loadAmbient() [3/3]	375
9.25.3.6 loadAmbientAndDiffuse() [1/3]	376
9.25.3.7 loadAmbientAndDiffuse() [2/3]	376
9.25.3.8 loadAmbientAndDiffuse() [3/3]	377
9.25.3.9 loadDiffuse() [1/3]	378
9.25.3.10 loadDiffuse() [2/3]	379
9.25.3.11 loadDiffuse() [3/3]	379
9.25.3.12 loadShininess() [1/2]	380
9.25.3.13 loadShininess() [2/2]	381
9.25.3.14 loadSpecular() [1/3]	381
9.25.3.15 loadSpecular() [2/3]	382
9.25.3.16 loadSpecular() [3/3]	382

9.25.3.17 select()	383
9.26 Menu クラス	384
9.26.1 詳解	384
9.26.2 構築子と解体子	384
9.26.2.1 Menu() [1/3]	384
9.26.2.2 Menu() [2/3]	385
9.26.2.3 Menu() [3/3]	385
9.26.2.4 ~Menu()	385
9.26.3 関数詳解	386
9.26.3.1 draw()	386
9.26.3.2 getLight()	386
9.26.3.3 getModel()	386
9.26.3.4 getShader()	387
9.26.3.5 operator=()	387
9.26.4 フレンドと関連関数の詳解	387
9.26.4.1 Scene	387
9.27 GgApp::Window クラス	387
9.27.1 詳解	389
9.27.2 構築子と解体子	389
9.27.2.1 Window() [1/3]	389
9.27.2.2 Window() [2/3]	390
9.27.2.3 Window() [3/3]	390
9.27.2.4 ~Window()	391
9.27.3 関数詳解	391
9.27.3.1 get()	391
9.27.3.2 getAltArrow()	391
9.27.3.3 getAltArrowX()	392
9.27.3.4 getAltArrowY()	392
9.27.3.5 getArrow() [1/2]	393
9.27.3.6 getArrow() [2/2]	393
9.27.3.7 getArrowX()	394
9.27.3.8 getArrowY()	395
9.27.3.9 getAspect()	395
9.27.3.10 getControlArrow()	396
9.27.3.11 getControlArrowX()	396
9.27.3.12 getControlArrowY()	397
9.27.3.13 getFboHeight()	398
9.27.3.14 getFboSize() [1/2]	398
9.27.3.15 getFboSize() [2/2]	398
9.27.3.16 getFboWidth()	399
9.27.3.17 getHeight()	399
9.27.3.18 getKey()	400

9.27.3.19 getLastKey()	400
9.27.3.20 getMouse() [1/3]	400
9.27.3.21 getMouse() [2/3]	400
9.27.3.22 getMouse() [3/3]	400
9.27.3.23 getMouseX()	401
9.27.3.24 getMouseY()	401
9.27.3.25 getRotation()	401
9.27.3.26 getRotationMatrix()	401
9.27.3.27 getScrollMatrix()	402
9.27.3.28 getShiftArrow()	402
9.27.3.29 getShiftArrowX()	403
9.27.3.30 getShiftArrowY()	403
9.27.3.31 getSize() [1/2]	404
9.27.3.32 getSize() [2/2]	404
9.27.3.33 getTranslation()	404
9.27.3.34 getTranslationMatrix()	405
9.27.3.35 getUserPointer()	405
9.27.3.36 getWheel() [1/3]	405
9.27.3.37 getWheel() [2/3]	406
9.27.3.38 getWheel() [3/3]	406
9.27.3.39 getWheelX()	406
9.27.3.40 getWheelY()	406
9.27.3.41 getWidth()	406
9.27.3.42 operator bool()	407
9.27.3.43 operator=() [1/2]	407
9.27.3.44 operator=() [2/2]	408
9.27.3.45 reset()	408
9.27.3.46 resetRotation()	409
9.27.3.47 resetTranslation()	409
9.27.3.48 restoreViewport()	409
9.27.3.49 selectInterface()	409
9.27.3.50 setClose()	410
9.27.3.51 setKeyboardFunc()	410
9.27.3.52 setMouseFunc()	410
9.27.3.53 setResizeFunc()	411
9.27.3.54 setUserPointer()	411
9.27.3.55 setVelocity()	411
9.27.3.56 setWheelFunc()	412
9.27.3.57 shouldClose()	412
9.27.3.58 swapBuffers()	412
9.27.3.59 updateViewport()	413

10 ファイル詳解	415
10.1 Config.cpp ファイル	415
10.1.1 詳解	415
10.1.2 変数詳解	416
10.1.2.1 defaultLight	416
10.2 Config.cpp	416
10.3 Config.h ファイル	418
10.3.1 詳解	418
10.4 Config.h	419
10.5 gg.cpp ファイル	420
10.5.1 詳解	420
10.6 gg.cpp	420
10.7 gg.h ファイル	496
10.7.1 詳解	500
10.7.2 マクロ定義詳解	501
10.7.2.1 ggError	501
10.7.2.2 ggFBOError	501
10.7.3 型定義詳解	501
10.7.3.1 pathChar	501
10.7.3.2 pathString	501
10.7.4 関数詳解	501
10.7.4.1 TCharToUtf8()	501
10.7.4.2 Utf8ToTChar()	501
10.8 gg.h	502
10.9 GgApp.cpp ファイル	558
10.9.1 詳解	558
10.10 GgApp.cpp	559
10.11 GgApp.h ファイル	577
10.11.1 詳解	578
10.11.2 マクロ定義詳解	578
10.11.2.1 GG.BUTTON_COUNT	578
10.11.2.2 GG.INTERFACE_COUNT	578
10.11.2.3 GG.USE_IMGUI	578
10.12 GgApp.h	578
10.13 ggsample01.cpp ファイル	587
10.13.1 マクロ定義詳解	587
10.13.1.1 CONFIG_FILE	587
10.13.1.2 PROJECT_NAME	587
10.14 ggsample01.cpp	588
10.15 main.cpp ファイル	589
10.15.1 詳解	589
10.15.2 マクロ定義詳解	589

10.15.2.1 HEADER_STR	589
10.15.3 関数詳解	589
10.15.3.1 main()	589
10.16 main.cpp	590
10.17 Menu.cpp ファイル	591
10.17.1 詳解	591
10.18 Menu.cpp	592
10.19 Menu.h ファイル	593
10.19.1 詳解	594
10.20 Menu.h	594
10.21 OPENXR.md ファイル	595
10.22 parseconfig.h ファイル	595
10.22.1 詳解	596
10.22.2 関数詳解	597
10.22.2.1 getString() [1/3]	597
10.22.2.2 getString() [2/3]	597
10.22.2.3 getString() [3/3]	598
10.22.2.4 getValue() [1/4]	598
10.22.2.5 getValue() [2/4]	599
10.22.2.6 getValue() [3/4]	599
10.22.2.7 getValue() [4/4]	600
10.22.2.8 getVector()	600
10.22.2.9 setString() [1/3]	601
10.22.2.10 setString() [2/3]	601
10.22.2.11 setString() [3/3]	602
10.22.2.12 setValue() [1/4]	602
10.22.2.13 setValue() [2/4]	602
10.22.2.14 setValue() [3/4]	603
10.22.2.15 setValue() [4/4]	603
10.22.2.16 setVector()	604
10.23 parseconfig.h	604
10.24 README.md ファイル	608
Index	609

Chapter 1

ゲームグラフィックス特論の宿題用補助 プログラム **GLFW3** 版.

著作権所有

Copyright (c) 2011-2025 Kohe Tokoi. All Rights Reserved.

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies or substantial portions of the Software.

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL KOHE TOKOI BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Chapter 2

OpenXR ラッパー (GgApp::OpenXR) マニュアル&使用方法解説

本書は、ゲームグラフィックス特論用補助プログラム ([GgApp](#)) に組み込まれた **OpenXR** ラッパークラス (**GgApp::OpenXR**) の仕様、API リファレンス、および実践的な使用方法についての解説ドキュメントです。

2.1 目次

1. 概要と特徴
 2. クイックスタート (最小の VR プログラム)
 3. API リファレンス
 - 初期化と終了処理
 - フレーム描画ループ
 - カメラ・投影変換・視点情報
 - コントローラートラッキング・姿勢
 - コントローラーボタン・スティック入力・振動
 - 参照空間 (空間座標系)
 4. 実践チュートリアル&使用例
 - 4.1 基本的なステレオレンダリング
 - 4.2 コントローラーの姿勢取得とモデル描画
 - 4.3 スティック入力による移動とトリガー入力
 - 4.4 床面基準 (STAGE) と着席基準 (LOCAL) の切り替え
 - 4.5 PC ウィンドウへのミラー表示
 5. 旧 Oculus SDK (LibOVR) ラッパーからの移行ガイド
 6. 動作環境とセットアップ
-

2.2 1. 概要と特徴

GgApp::OpenXR は、業界標準の XR 規格である **OpenXR** を用いて、OpenGL による VR アプリケーションを極めてシンプルかつ直感的に開発できるように設計された C++ ラッパークラスです。

2.2.1 主な特徴

- **gg.h** 数学ライブラリとのシームレスな統合:
 - `getProjectionMatrix(eye, zNear, zFar)` を呼ぶだけで、HMD 特有の非対称視野角 (Off-Center Frustum) に対応した正確な透視投影変換行列 (GgMatrix) を直接取得できます。
 - `getViewMatrix(eye)` により、HMD の頭部トラッキング姿勢からワールド座標系に対するビュー変換行列 (GgMatrix) を 1 行で取得できます。
- 標準化されたコントローラー (入力・トラッキング) の統合サポート:
 - Meta Quest Touch、HTC Vive、Valve Index、Windows Mixed Reality などの主要な VR コントローラーを標準でサポート。
 - 手のグリップ姿勢 (`getGripMatrix`)、エイム (ポインティング) 姿勢 (`getAimMatrix`)、各種ボタン (Trigger, Grip, Thumbstick, Primary/Secondary, Menu) やハプティクス振動を即座に利用可能。
- シンプルなフレームライフサイクル:
 - `begin()` -> `select(eye)` -> `commit(eye)` -> `submit(mirror)` の分かりやすいループ構成。FBO バインド、スワップチェーン管理、ビューポート設定、sRGB カラースペース処理を内部で自動処理。
- 柔軟な空間座標系:
 - ルームスケール・床面基準の `XR_REFERENCE_SPACE_TYPE_STAGE` と、着席・頭部原点基準の `XR_REFERENCE_SPACE_TYPE_LOCAL` に両対応。

2.3 2. クイックスタート (最小の VR プログラム)

OpenXR を有効にするには、`#include "GgApp.h"` の前に `#define GG_USE_OPENXR` を定義します。

```
#define GG_USE_OPENXR
#include "GgApp.h"

int GgApp::main(int argc, const char* const* argv)
{
    // ミラー表示用のウィンドウを作成
    Window window("OpenXR Sample", 1280, 720);

    // OpenXR を初期化
    auto& openxr = OpenXR::initialize(window);

    // 物体の回転・配置行列
    const GgMatrix modelMatrix = ggTranslate(0.0f, 1.2f, -2.0f);

    // ウィンドウが開いている間ループ
    while (window)
    {
        // OpenXR のフレーム開始
        if (openxr.begin())
        {
            // 左右の目を順にレンダリング (通常 2 回)
            for (int eye = 0; eye < openxr.getViewCount(); ++eye)
            {
                // 描画対象の目を指定 (FBO のバインドとビューポート設定が自動で行われる)
                openxr.select(eye);

                // バッファの消去
                glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

                // 投影変換行列とビュー変換行列を直接取得
                const GgMatrix mp = openxr.getProjectionMatrix(eye, 0.1f, 100.0f);
                const GgMatrix mv = openxr.getViewMatrix(eye);

                //
                // ここでシェードに mp, mv * modelMatrix を渡してオブジェクトを描画
                //

                // 描画の完了 (スワップチェーンイメージの解放)
                openxr.commit(eye);
            }
        }
    }
}
```

```

    }

    // HMD フレームを転送し、ウィンドウへミラー表示
    openxr.submit(true);
}

// 終了処理
openxr.terminate();
return 0;
}

```

2.4 3. API リファレンス

クラス定義: `GgApp::OpenXR` (シングルトン)

2.4.1 型定義・定数

```

enum Hand
{
    Left   = 0,    // 左手
    Right  = 1,    // 右手
    Count  = 2     // 手の総数
};

```

2.4.2 初期化と終了処理

2.4.2.1 `static OpenXR& initialize(const Window& window, XrReferenceSpaceType spaceType = XR_REFERENCE_SPACE_TYPE_STAGE)`

OpenXR インスタンス、システム、OpenGL バインディング、セッション、参照空間、スワップチェーン、アクションセットを初期化します。

- 引数:
 - window: ミラー表示を行う `GgApp::Window` オブジェクト。
 - spaceType: 参照空間タイプ (`XR_REFERENCE_SPACE_TYPE_STAGE`: 床面基準、`XR_REFERENCE_SPACE_TYPE_LOCAL`: 頭部基準)。STAGE がサポートされない環境では自動的に LOCAL にフォールバックします。
- 戻り値: 初期化された OpenXR クラスの静的参照。

2.4.2.2 `void terminate()`

作成したすべての OpenXR リソース (セッション、スワップチェーン、スペース、アクション、FBO、デプステクスチャ) を安全に破棄します。

2.4.3 フレーム描画ループ

2.4.3.1 `bool begin()`

OpenXR イベントをポーリングし、セッション状態の遷移、フレーム同期 (`xrWaitFrame / xrBeginFrame`)、視点位置・コントローラー入力状態の同期 (`xrLocateViews / pollActions`) を行います。

- 戻り値: レンダリングを実行すべきフレームであれば `true`、描画が不要 (またはセッション非アクティブ) であれば `false`。

2.4.3.2 `void select(int eye)`

指定した目 (0: 左目、1: 右目) のレンダリング用 FBO にスワップチェーンイメージとデプステクスチャをアタッチ・バインドし、ビューポート (`glViewport`) を推奨解像度に自動設定します。

- 引数:
 - eye: 表示する目のインデックス (0 または 1)。

2.4.3.3 void select(int eye, GLfloat* screen, GLfloat* position, GLfloat* orientation)

旧 LibOVR 仕様からの移行用オーバーロードです。FBO バインドに加えて、視野のタンジェント境界値、視点位置、回転四元数を生配列に代入します。

2.4.3.4 void commit(int eye)

レンダリング完了後、FBO バインドを解除し、現在のスワップチェーンイメージを OpenXR ランタイムへ解放 (Release) します。

- 引数:
 - eye: 完了した目のインデックス (0 または 1)。

2.4.3.5 bool submit(bool mirror = true)

左右両眼のレンダリング結果を合成レイヤー (XrCompositionLayerProjection) として OpenXR ランタイムへ転送 (xrEndFrame) します。

- 引数:
 - mirror: true の場合、PC のウィンドウ (GgApp::Window) へ左目画像を glBlitFramebuffer でミラー転送します。
- 戻り値: 成功時に true。

2.4.4 カメラ・投影変換・視点情報

2.4.4.1 GgMatrix getProjectionMatrix(int eye, GLfloat zNear = 0.1f, GLfloat zFar = 100.0f) const

指定した目の視野角情報 (XrFovf) から、オフセンター視錐台に対応した透視投影変換行列を生成して返します。

- 引数:
 - eye: 目のインデックス (0: 左目, 1: 右目)。
 - zNear: 前方面の位置 (メートル、デフォルト 0.1f)。
 - zFar: 後方面の位置 (メートル、デフォルト 100.0f)。
- 戻り値: GgMatrix 型の透視投影変換行列。

2.4.4.2 GgMatrix getViewMatrix(int eye) const

トラッキングされた視点位置と頭部回転から、ワールド座標系から視点 (カメラ) 座標系への変換を行うビュー変換行列を計算して返します。

- 計算式: $V = R^{\text{top}} \cdot T(-p) = \text{conjugate}(q) \cdot \text{translate}(-p)$
- 戻り値: GgMatrix 型のビュー変換行列。

2.4.4.3 GgMatrix getPoseMatrix(int eye) const

ワールド座標系における各眼のローカル座標系変換行列 (カメラのモデル変換行列 / 位置と向き) を返します。

- 計算式: $M = T(p) \cdot R(q)$
- 戻り値: GgMatrix 型の姿勢行列。

2.4.4.4 GgVector getPosition(int eye) const

指定した目の視点位置 $(x, y, z, 1.0)$ を GgVector で返します。

2.4.4.5 GgQuaternion getOrientation(int eye) const

指定した目の回転四元数 (x, y, z, w) を GgQuaternion で返します。

2.4.4.6 const XrFovf& getFov(int eye) const

指定した目の OpenXR 視野角構造体 (angleLeft, angleRight, angleUp, angleDown [ラジアン]) の参照を返します。

2.4.4.7 const XrPosef& getPose(int eye) const

指定した目の OpenXR 姿勢構造体 (position, orientation) の参照を返します。

2.4.4.8 GLsizei getWidth(int eye = 0) const / GLsizei getHeight(int eye = 0) const

OpenXR ランタイムが推奨するレンダリング解像度の幅および高さをピクセル単位で返します。

2.4.4.9 GLfloat getAspect(int eye = 0) const

推奨解像度のアスペクト比 (幅 / 高さ) を返します。

2.4.4.10 uint32_t getViewCount() const

ビューの総数 (通常ステレオ表示で 2) を返します。

2.4.5 コントローラートラッキング・姿勢**2.4.5.1 bool isTracked(int hand) const**

指定した手 (Hand::Left または Hand::Right) のコントローラーが現在有効にトラッキングされているかを判定します。

2.4.5.2 GgMatrix getGripMatrix(int hand) const

コントローラーのグリップ位置・姿勢を表すモデル変換行列 (位置と回転) を返します。コントローラーの 3D モデルを描画する際の配置行列として使用できます。

2.4.5.3 GgMatrix getAimMatrix(int hand) const

コントローラーのポインティング (エイム) 方向を表すモデル変換行列を返します。レーザーポインターや銃の照準ベクトルを描画・計算する際に適しています。

2.4.5.4 GgVector getGripPosition(int hand) const / GgQuaternion getGripOrientation(int hand) const

グリップのワールド位置座標および回転四元数を取得します。

2.4.5.5 GgVector getAimPosition(int hand) const / GgQuaternion getAimOrientation(int hand) const

エイムのワールド位置座標および回転四元数を取得します。

2.4.6 コントローラーボタン・スティック入力・振動**2.4.6.1 float getTrigger(int hand) const**

人差し指トリガーのアナログ押し込み量 (0.0f 1.0f) を返します。

2.4.6.2 float getGrip(int hand) const

中指グリップ (スクイーズ) のアナログ押し込み量 (0.0f 1.0f) を返します。

2.4.6.3 std::array<float, 2> getThumbstick(int hand) const

アナログスティック（またはトラックパッド）の2次元入力値 \$(x, y)\$（各 $-1.0f$ $1.0f$ ）を返します。

2.4.6.4 bool getThumbstickClick(int hand) const

アナログスティックを押し込んだクリック状態（押されていればtrue）を返します。

2.4.6.5 bool getPrimaryButton(int hand) const

プライマリボタン（左手:X ボタン、右手:A ボタン）の押下状態を返します。

2.4.6.6 bool getSecondaryButton(int hand) const

セカンダリボタン（左手:Y ボタン、右手:B ボタン）の押下状態を返します。

2.4.6.7 bool getMenuButton(int hand = Hand::Left) const

メニューボタンの押下状態を返します。

2.4.6.8 void applyHapticVibration(int hand, float durationSeconds = 0.1f, float frequency = XR_FREQUENCY_UNSPECIFIED, float amplitude = 0.5f)

コントローラーに触覚フィードバック（振動）を出力します。

- 引数:
 - hand: 対象の手 (Hand::Left または Hand::Right)。
 - durationSeconds: 振動の持続時間（秒）。
 - frequency: 振動周波数（Hz、デフォルト XR_FREQUENCY_UNSPECIFIED）。
 - amplitude: 振動の強さ（ $0.0f$ $1.0f$ 、デフォルト $0.5f$ ）。

2.4.7 参照空間（空間座標系）**2.4.7.1 XrReferenceSpaceType getReferenceSpaceType() const**

現在有効な参照空間のタイプ（XR_REFERENCE_SPACE_TYPE_STAGE または XR_REFERENCE_SPACE_TYPE_LOCAL）を返します。

2.5 4. 実践チュートリアル & 使用例**2.5.1 4.1 基本的なステレオレンダリング**

```
#define GG_USE_OPENXR
#include "GgApp.h"

int GgApp::main(int argc, const char* const* argv)
{
    Window window("OpenXR Basic", 1280, 720);
    auto& openxr = OpenXR::initialize(window);

    // 描画設定
    glEnable(GL_DEPTH_TEST);
    glEnable(GL_CULL_FACE);

    while (window)
    {
        if (openxr.begin())
        {
            for (int eye = 0; eye < openxr.getViewCount(); ++eye)
            {
                openxr.select(eye);

                glClearColor(0.2f, 0.3f, 0.4f, 1.0f);
```

```

    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    // 透視投影行列 (Near: 0.05m, Far: 50.0m)
    const GgMatrix mp = openxr.getProjectionMatrix(eye, 0.05f, 50.0f);
    // 頭部姿勢に追従するビュー行列
    const GgMatrix mv = openxr.getViewMatrix(eye);

    // シーンを描画
    // myShader.use(mp, mv * myObjectMatrix);
    // myObject.draw();

    openxr.commit(eye);
}
openxr.submit(true);
}

openxr.terminate();
return 0;
}

```

2.5.2 4.2 コントローラーの姿勢取得とモデル描画

コントローラーの位置・向きに合わせて手のオブジェクトや武器モデルを配置・描画します。

```

for (int hand = 0; hand < OpenXR::Hand::Count; ++hand)
{
    if (openxr.isTracked(hand))
    {
        // コントローラーのグリップ位置・向きのモデル行列を取得
        const GgMatrix mg = openxr.getGripMatrix(hand);

        // ビュー変換と掛け合わせて描画
        handShader.use(mp, mv * mg, light);
        handModel.draw();
    }
}

```

2.5.3 4.3 スティック入力による移動とトリガー入力

左スティックでプレイヤーを前後左右に移動させ、右トリガーでオブジェクトを掴む・発射する例です。

```

// プレイヤーのワールド座標位置
static GgVector playerPos{ 0.0f, 0.0f, 0.0f, 1.0f };

// 左スティックの入力を取得
const auto stick = openxr.getThumbstick(OpenXR::Hand::Left);
const float moveSpeed = 0.05f;

// スティックの傾きに依じて移動
playerPos[0] += stick[0] * moveSpeed;
playerPos[2] -= stick[1] * moveSpeed; // 前後移動

// プレイヤーの平行移動行列
const GgMatrix playerMatrix = ggTranslate(playerPos[0], playerPos[1], playerPos[2]);

// 右トリガーの押し込みをチェック
if (openxr.getTrigger(OpenXR::Hand::Right) > 0.8f)
{
    // トリガーが深く引かれたときに右手に振動を与える
    openxr.applyHapticVibration(OpenXR::Hand::Right, 0.05f, XR_FREQUENCY_UNSPECIFIED, 0.8f);
}

// レンダリンググループ内でのビュー行列適用
const GgMatrix mv = openxr.getViewMatrix(eye) * playerMatrix.inverse();

```

2.5.4 4.4 床面基準 (STAGE) と着席基準 (LOCAL) の切り替え

```

// ルームスケール (床面が Y=0 となる座標系、立ち位置・歩行用)
auto& openxr = OpenXR::initialize(window, XR_REFERENCE_SPACE_TYPE_STAGE);

// または着席プレイ (HMD の起動位置が原点となる座標系、フライトシムやドライブゲーム用)
auto& openxr = OpenXR::initialize(window, XR_REFERENCE_SPACE_TYPE_LOCAL);

```

2.5.5 4.5 PC ウィンドウへのミラー表示

`openxr.submit(mirror)` の引数でミラー表示の有無を切り替えられます。

// PC 画面に HMD の左目視界をフルスクリーン／ウィンドウサイズに合わせて表示

```
openxr.submit(true);

// ミラー表示をオフにしてパフォーマンスを最優先する場合
openxr.submit(false);
```

2.6 5. 旧 Oculus SDK (LibOVR) ラッパーからの移行ガイド

旧 GgApp::Oculus (LibOVR) および旧 GgApp::OpenXR から、最適化版 API への対応表です。

処理内容	旧 API (LibOVR ラッパー互換)	最適化版 OpenXR API
初期化	Oculus↔ ::initialize(window)	OpenXR↔ ::initialize(window, spaceType)
目の指定とFBOバインド	openxr.select(eye, screen, pos, ori)	openxr.select(eye)
透視投影行列の生成	手動で gg↔ Frustum(screen[0]*n, ...)	openxr.getProjection↔ Matrix(eye, zNear, zFar)
ビュー行列の生成	手動で Gg↔ Quaternion(ori).get↔ ConjugateMatrix() * gg↔ Translate(-pos)	openxr.getViewMatrix(eye)
カメラ姿勢行列	手動で合成	openxr.getPoseMatrix(eye)
TimeWarp 設定	openxr.timewarp(mp) (空ダミー)	不要 (削除)
フレーム完了	openxr.commit(eye)	openxr.commit(eye)
フレーム転送	openxr.submit(mirror)	openxr.submit(mirror)
コントローラー姿勢	なし	openxr.getGrip↔ Matrix(hand), getAim↔ Matrix(hand)
ボタン・スティック入力	なし	getTrigger, getGrip, get↔ Thumbstick, getPrimary↔ Button など
ハプティクス振動	なし	openxr.applyHaptic↔ Vibration(hand, ...)

2.6.1 コードの対比 (Before / After)

2.6.1.1 [Before] 旧 API

```
Glfloor screen[4], position[3], orientation[4];
openxr.select(eye, screen, position, orientation);

// 面倒な手動計算が必要だった
const GgMatrix mp = ggFrustum(
    screen[0] * zNear, screen[1] * zNear,
    screen[2] * zNear, screen[3] * zNear,
    zNear, zFar);

const GgMatrix mv = GgQuaternion(orientation).getConjugateMatrix()
    * ggTranslate(-position[0], -position[1], -position[2]);

shader.use(mp, mv * model, light);
model.draw();
openxr.commit(eye);
```

2.6.1.2 [After] 最適化版 API

```
openxr.select(eye);

// 1行で直感的に取得可能！
```



```
const GgMatrix mp = openxr.getProjectionMatrix(eye, zNear, zFar);
const GgMatrix mv = openxr.getViewMatrix(eye);

shader.use(mp, mv * model, light);
model.draw();
openxr.commit(eye);
```

2.7 6. 動作環境とセットアップ

2.7.1 必要な環境

- OS:
 - Windows 10 / 11 (64bit)
 - Linux (Ubuntu / Linux Mint 等、X11 環境)
- VR ハードウェア:
 - Meta Quest 2 / 3 / 3S / Pro (Quest Link / Air Link / Virtual Desktop / Steam Link)
 - HTC Vive / Vive Pro / Focus 3
 - Valve Index
 - Windows Mixed Reality 各種 HMD
 - Bigscreen Beyond, Pico 4 等
- OpenXR ランタイム:
 - Meta Quest Link アプリ (Oculus ランタイムをアクティブに設定)
 - SteamVR (SteamVR 設定 -> OpenXR -> 「SteamVR をアクティブな OpenXR ランタイムとして設定」)
 - Monado (Linux 用オープンソースランタイム)

2.7.2 プロジェクトへの組み込み

1. ソースコード冒頭で OpenXR を有効化:

```
#define GG_USE_OPENXR
#include "GgApp.h"
```

2. OpenXR SDK のヘッダー (<openxr/openxr.h>, <openxr/openxr_platform.h>) およびローダーライブラリ (openxr_loader.lib または -lopenxr_loader) をプロジェクトにリンクします。

Chapter 3

ggsample01

3.1 ゲームグラフィックス特論A 第1回 宿題

OpenGL の開発環境を整備してください。

- 宿題のひな形は [GitHub](#) にあります (宿題のひな形で使っている [補助プログラムの解説](#))。
- 詳しくは [講義のスライド](#) を参照してください。

3.2 宿題プログラムの作成に必要な環境

- Linux Mint 22.1 (Xia), gcc 13.3.0 以降 (libglfw3-dev, libgtk-3-dev, pkg-config 必須)。
- macOS 14 (Sonoma), Xcode 15 以降 (Intel / Apple 両対応)。
- Windows 11 (Win32/x64), Visual Studio 2022 以降 (“C++ によるデスクトップ開発” 必須)。
- OpenGL 4.1 以降が実行できる環境 (対応した GPU を搭載したビデオカード や CPU) が必要です。

3.3 宿題の提出

- このプロジェクトをビルド・実行して表示されたウィンドウのスクリーンショットを Moodle にアップロードしてください。
- 気力があったら表示図形が回転するようにして (AI 使用可)、そのムービー (10MB 以下) を送ってください。

3.4 補足

このプログラムを実行すると、次のような図形が表示されます。

- macOS では Intel CPU と Apple Silicon の両方に対応した Universal Binary を作成するようにしています。
- Raspberry Pi 4B にも対応しました。make -f Makefile.rpi でビルドしてください。
- ひな形プログラムがコンパイル／実行できなかったら知らせてください。
- fork 推奨ですが解答を PR で受け取る**気力はありません**。

3.5 宿題プログラム用補助プログラムについて

ゲームグラフィックス特論 A/B で課す宿題プログラムでは、専用の補助プログラムを用意しています。これは以下の 5 つのファイルで構成されています。

- `gg.h / gg.cpp`
 - GLFW での利用を想定した OpenGL のローダとユーティリティ
- `GgApp.h / GgApp.cpp`
 - ウィンドウやマウス関連のユーザインタフェースを管理する GLFW のラッパー
- `main.cpp`
 - メインプログラム

GLFW は OpenGL や、その後継の Vulkan を使用したアプリケーションを作成するための、非常にコンパクトなフレームワークです。本当はこれだけで簡単にアプリケーションが作れるのですが、授業内容とはあまり関係のない処理を分離するために、屋上屋ながら**この授業専用の**フレームワークを用意しました。なお、`gg.h / gg.cpp` には OpenGL の拡張機能を使用可能にする機能を含んでいるので、別に GLEW や glad、GL3W などを導入する必要はありません。

また、この補助プログラムは Dear ImGui をサポートする機能を含んでいます。このプログラム (ggsample01) には、その使い方のサンプルコードを示すために、Dear ImGui のソースプログラムも含めています。日本語メニューの表示のために M+ FONTS の Mplus1-Regular.ttf もリポジトリに含めています。

このほか、この `GgApp.h / GgApp.cpp` には OpenXR をサポートする機能を組み込んでいます。Meta Quest 2 / 3 / 3S / Pro や HTC Vive、Valve Index 等の VR デバイスを接続しているなら、この補助プログラムを使って C++ だけで VR アプリケーションが作成できます。

3.5.1 補助プログラムのドキュメント

Doxygen で生成したドキュメントの [HTML 版](#) を `html` フォルダに、[PDF 版](#) を `pdf` フォルダに置いてあります。また、OpenXR の詳細なマニュアルおよび使用方法の解説は [OPENXR.md](#) を参照してください。

3.5.2 補助プログラムの使い方

補助プログラムを使用するには、最小限、GLFW が使える環境が必要です。ゲームグラフィックス特論 A/B の宿題のリポジトリには Windows 用および macOS 用にコンパイルしたライブラリファイル一式を含めていますので、これらについては宿題のために別に用意する必要はありません。Linux (および Raspberry Pi) では `libglfw3-dev`、`libgtk-3-dev`、および `pkg-config` パッケージをインストールしておいてください (% `sudo apt-get install libglfw3-dev libgtk-3-dev pkg-config`)。 `gg.h`、`gg.cpp`、`GgApp.h`、`GgApp.cpp` および `main.cpp` だけを使うときは、それぞれの環境で GLFW をインストールしておいてください。この補助プログラムを使用した最小のプログラムは、多分こんな感じになります。このソースファイルと同じところに `gg.h`、`gg.cpp`、`GgApp.h`、`GgApp.cpp`、および `main.cpp` を置き、`gg.cpp`、`GgApp.cpp`、`main.cpp` と一緒にコンパイルして、GLFW のライブラリファイルをリンクしてください。

```
#include "GgApp.h"

int GgApp::main(int argc, const char* const* argv)
{
    Window window;

    while (window)
    {
        //
        // ここで OpenGL による描画を行う
        //
    }
}
```

3.5.3 OpenXR を使う場合

すべての `#include "GgApp.h"` の前に、`#define GG_USE_OPENXR` を置いてください。

```
// ウィンドウ関連の処理
#define GG_USE_OPENXR
#include "GgApp.h"
```

あるいは、`GgApp.h` 中の `//#define GG_USE_OPENXR` のコメント (`//`) を削除してください。

```
// OpenXR を使うなら
#define GG_USE_OPENXR
```

OpenXR を使った VR アプリケーションの詳しい書き方、透視投影行列・ビュー変換行列の取得方法、コントローラーの姿勢取得やボタン入力、ハプティクス振動などの機能については、[OPENXR.md](#) を参照してください。

3.5.4 Dear ImGui を使う場合

すべての `#include "GgApp.h"` の前に、`#define GG_USE_IMGUI` を置いてください。

```
// ウィンドウ関連の処理
#define GG_USE_IMGUI
#include "GgApp.h"
```

あるいは、`GgApp.h` 中の `//#define GG_USE_IMGUI` のコメント (`//`) を削除してください

```
// Dear ImGui を使うなら
#define GG_USE_IMGUI
```

Dear ImGui のウィンドウの実際のレンダリング (`ImGui::RenderDrawData()`; の呼び出し) は `window.swapbuffers()` の中で行っているため、描画ループの中で Dear ImGui の API と OpenGL の API は混在していても構いません。

なお、Dear ImGui を有効にした場合は、Dear ImGui がマウスを使っているとき (`io.WantCaptureMouse == true`) に、`Window` クラスが保持しているマウスカーソルの位置を更新しないようにしています。また、Dear ImGui がキーボードを使っているとき (`io.WantCaptureKeyboard == true`) には、`GgApp::Window` クラスはキーボードのイベントを処理しないようにしています。

```
// ウィンドウ関連の処理
#define GG_USE_IMGUI
#include "GgApp.h"
```

```
int GgApp::main(int argc, const char* const* argv)
{
    // ウィンドウを作成する
    Window window("Window Title", 1280, 720);

    // ImGui の初期設定
    ImGui::StyleColorsDark();

    // 背景色を指定する
    glClearColor(1.0f, 1.0f, 1.0f, 0.0f);

    // ウィンドウが開いている間繰り返す
    while (window)
    {
        // ImGui のフレームに一つ目の ImGui のウィンドウを描く
        ImGui::Begin("Control panel");
        ImGui::Text("Frame rate: %6.2f fps", ImGui::GetIO().Framerate);
        if (ImGui::Button("Quit")) window.setClose();
        ImGui::End();

        // ウィンドウを消去する
        glClear(GL_COLOR_BUFFER_BIT);

        //
        // ここで OpenGL による描画を行う
        //

        // カラーバッファを入れ替えてイベントを取り出す
        window.swapBuffers();
    }
}
```

このソースファイルと Dear ImGui に含まれる以下のファイル、および `gg.h`、`gg.cpp`、`GgApp.h`、`GgApp.cpp` および `main.cpp` を同じところに置き、`gg.cpp`、`GgApp.cpp`、`main.cpp` と以下のうちの `*.cpp` ファイルと一緒にコンパイルして、GLFW のライブラリファイルをリンクしてください。

```
imconfig.h
imgui.h
imgui_impl_glfw.h
imgui_impl_opengl3.h
```

```
imgui_impl_opengl3_loader.h  
imgui_internal.h  
imstb_rectpack.h  
imstb_textedit.h  
imstb_truetype.h
```

```
imgui.cpp  
imgui_draw.cpp  
imgui_impl_glfw.cpp  
imgui_impl_opengl3.cpp  
imgui_tables.cpp  
imgui_widgets.cpp
```

3.5.4.1 imconfig.h の変更点

Dear ImGui はバージョン 1.86 から独自のローダを使用するようになったので、`IMGUI_IMPL_OPENGL_LOADER_CUSTOM` にこの授業オリジナルのローダ [gg.h](#) / [gg.cpp](#) を指定する必要はありません。ただし、Raspberry Pi では `imconfig.h` の**最後**で記号定数 `IMGUI_IMPL_OPENGL_ES3` を明示的に定義する必要があります。

```
// The Raspberry Pi needs to explicitly define the symbolic constant IMGUI_IMPL_OPENGL_ES3.  
#if defined(__RASPBERRY_PI__)  
# define IMGUI_IMPL_OPENGL_ES3  
#endif
```

Chapter 4

名前空間索引

4.1 名前空間一覧

全名前空間の一覧です。

99 25

Chapter 5

階層索引

5.1 クラス階層

クラス階層一覧です。大雑把に文字符号順で並べられています。

std::array	
gg::GgMatrix	119
gg::GgVector	327
gg::GgQuaternion	197
gg::GgTrackball	298
Config	89
GgApp	93
gg::GgBuffer< T >	98
gg::GgColorTexture	108
gg::GgNormalTexture	176
gg::GgPointShader	185
gg::GgSimpleShader	273
gg::GgShader	263
gg::GgShape	267
gg::GgPoints	180
gg::GgTriangles	309
gg::GgElements	113
gg::GgSimpleObj	271
gg::GgTexture	292
gg::GgUniformBuffer< T >	314
gg::GgUniformBuffer< Light >	314
gg::GgSimpleShader::LightBuffer	350
gg::GgUniformBuffer< Material >	314
gg::GgSimpleShader::MaterialBuffer	368
gg::GgVertex	344
gg::GgVertexArray	346
gg::GgSimpleShader::Light	349
gg::GgSimpleShader::Material	366
Menu	384
GgApp::Window	387

Chapter 6

クラス索引

6.1 クラス一覧

クラス・構造体・共用体・インターフェースの一覧です。

Config	89
GgApp	93
gg::GgBuffer< T >	98
gg::GgColorTexture	108
gg::GgElements	113
gg::GgMatrix	119
gg::GgNormalTexture	176
gg::GgPoints	180
gg::GgPointShader	185
gg::GgQuaternion	197
gg::GgShader	263
gg::GgShape	267
gg::GgSimpleObj	271
gg::GgSimpleShader	273
gg::GgTexture	292
gg::GgTrackball	298
gg::GgTriangles	309
gg::GgUniformBuffer< T >	314
gg::GgVector	327
gg::GgVertex	344
gg::GgVertexArray	346
gg::GgSimpleShader::Light	349
gg::GgSimpleShader::LightBuffer	350
gg::GgSimpleShader::Material	366
gg::GgSimpleShader::MaterialBuffer	368
Menu	384
GgApp::Window	387

Chapter 7

ファイル索引

7.1 ファイル一覧

ファイル一覧です。

Config.cpp	415
Config.h	418
gg.cpp	420
gg.h	496
GgApp.cpp	558
GgApp.h	577
ggsample01.cpp	587
main.cpp	589
Menu.cpp	591
Menu.h	593
parseconfig.h	595

Chapter 8

名前空間詳解

8.1 gg 名前空間

クラス

- class [GgVector](#)
- class [GgMatrix](#)
- class [GgQuaternion](#)
- class [GgTrackball](#)
- class [GgTexture](#)
- class [GgColorTexture](#)
- class [GgNormalTexture](#)
- class [GgBuffer](#)
- class [GgUniformBuffer](#)
- class [GgVertexArray](#)
- class [GgShape](#)
- class [GgPoints](#)
- struct [GgVertex](#)
- class [GgTriangles](#)
- class [GgElements](#)
- class [GgShader](#)
- class [GgPointShader](#)
- class [GgSimpleShader](#)
- class [GgSimpleObj](#)

列挙型

- enum [BindingPoints](#) { [LightBindingPoint](#) = 0 , [MaterialBindingPoint](#) }

関数

- void [ggInit](#) ()
- void [_ggError](#) (const std::string &name="", unsigned int line=0)
- void [_ggFBOError](#) (const std::string &name="", unsigned int line=0)
- void [ggCross](#) (GLfloat *c, const GLfloat *a, const GLfloat *b)
- GLfloat [ggDot3](#) (const GLfloat *a, const GLfloat *b)
- GLfloat [ggLength3](#) (const GLfloat *a)
- GLfloat [ggDistance3](#) (const GLfloat *a, const GLfloat *b)
- void [ggNormalize3](#) (const GLfloat *a, GLfloat *b)
- void [ggNormalize3](#) (GLfloat *a)
- GLfloat [ggDot4](#) (const GLfloat *a, const GLfloat *b)
- GLfloat [ggLength4](#) (const GLfloat *a)
- GLfloat [ggDistance4](#) (const GLfloat *a, const GLfloat *b)

- void `ggNormalize4` (const GLfloat *a, GLfloat *b)
- void `ggNormalize4` (GLfloat *a)
- const `GgVector` & `operator+` (const `GgVector` &v)
- `GgVector` `operator+` (GLfloat a, const `GgVector` &b)
- const `GgVector` `operator-` (const `GgVector` &v)
- `GgVector` `operator-` (GLfloat a, const `GgVector` &b)
- `GgVector` `operator*` (GLfloat a, const `GgVector` &b)
- `GgVector` `operator/` (GLfloat a, const `GgVector` &b)
- `GgVector` `ggCross` (const `GgVector` &a, const `GgVector` &b)
- GLfloat `ggDot3` (const `GgVector` &a, const `GgVector` &b)
- GLfloat `ggLength3` (const `GgVector` &a)
- GLfloat `ggDistance3` (const `GgVector` &a, const `GgVector` &b)
- `GgVector` `ggNormalize3` (const `GgVector` &a)
- void `ggNormalize3` (`GgVector` *a)
- GLfloat `ggDot4` (const `GgVector` &a, const `GgVector` &b)
- GLfloat `ggLength4` (const `GgVector` &a)
- GLfloat `ggDistance4` (const `GgVector` &a, const `GgVector` &b)
- `GgVector` `ggNormalize4` (const `GgVector` &a)
- void `ggNormalize4` (`GgVector` *a)
- `GgMatrix` `ggIdentity` ()
- `GgMatrix` `ggTranslate` (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f)
- `GgMatrix` `ggTranslate` (const GLfloat *t)
- `GgMatrix` `ggTranslate` (const `GgVector` &t)
- `GgMatrix` `ggScale` (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f)
- `GgMatrix` `ggScale` (const GLfloat *s)
- `GgMatrix` `ggScale` (const `GgVector` &s)
- `GgMatrix` `ggRotateX` (GLfloat a)
- `GgMatrix` `ggRotateY` (GLfloat a)
- `GgMatrix` `ggRotateZ` (GLfloat a)
- `GgMatrix` `ggRotate` (GLfloat x, GLfloat y, GLfloat z, GLfloat a)
- `GgMatrix` `ggRotate` (const GLfloat *r, GLfloat a)
- `GgMatrix` `ggRotate` (const `GgVector` &r, GLfloat a)
- `GgMatrix` `ggRotate` (const GLfloat *r)
- `GgMatrix` `ggRotate` (const `GgVector` &r)
- `GgMatrix` `ggLookat` (GLfloat ex, GLfloat ey, GLfloat ez, GLfloat tx, GLfloat ty, GLfloat tz, GLfloat ux, GLfloat uy, GLfloat uz)
- `GgMatrix` `ggLookat` (const GLfloat *e, const GLfloat *t, const GLfloat *u)
- `GgMatrix` `ggLookat` (const `GgVector` &e, const `GgVector` &t, const `GgVector` &u)
- `GgMatrix` `ggOrthogonal` (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar)
- `GgMatrix` `ggFrustum` (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar)
- `GgMatrix` `ggPerspective` (GLfloat fovy, GLfloat aspect, GLfloat zNear, GLfloat zFar)
- `GgMatrix` `ggTranspose` (const `GgMatrix` &m)
- `GgMatrix` `ggInvert` (const `GgMatrix` &m)
- `GgMatrix` `ggNormal` (const `GgMatrix` &m)
- `GgQuaternion` `ggIdentityQuaternion` ()
- `GgQuaternion` `ggMatrixQuaternion` (const GLfloat *a)
- `GgQuaternion` `ggMatrixQuaternion` (const `GgMatrix` &m)
- `GgMatrix` `ggQuaternionMatrix` (const `GgQuaternion` &q)
- `GgMatrix` `ggQuaternionTransposeMatrix` (const `GgQuaternion` &q)
- `GgQuaternion` `ggRotateQuaternion` (GLfloat x, GLfloat y, GLfloat z, GLfloat a)
- `GgQuaternion` `ggRotateQuaternion` (const GLfloat *v, GLfloat a)
- `GgQuaternion` `ggRotateQuaternion` (const GLfloat *v)
- `GgQuaternion` `ggEulerQuaternion` (GLfloat heading, GLfloat pitch, GLfloat roll)
- `GgQuaternion` `ggEulerQuaternion` (const GLfloat *e)
- `GgQuaternion` `ggEulerQuaternion` (const `GgVector` &e)

- [GgQuaternion ggSlerp](#) (const GLfloat *a, const GLfloat *b, GLfloat t)
- [GgQuaternion ggSlerp](#) (const [GgQuaternion](#) &q, const [GgQuaternion](#) &r, GLfloat t)
- [GgQuaternion ggSlerp](#) (const [GgQuaternion](#) &q, const GLfloat *a, GLfloat t)
- [GgQuaternion ggSlerp](#) (const GLfloat *a, const [GgQuaternion](#) &q, GLfloat t)
- GLfloat [ggNorm](#) (const [GgQuaternion](#) &q)
- [GgQuaternion ggNormalize](#) (const [GgQuaternion](#) &q)
- [GgQuaternion ggConjugate](#) (const [GgQuaternion](#) &q)
- [GgQuaternion ggInvert](#) (const [GgQuaternion](#) &q)
- bool [ggSaveTga](#) (const std::string &name, const void *buffer, unsigned int width, unsigned int height, unsigned int depth)
- bool [ggSaveColor](#) (const std::string &name)
- bool [ggSaveDepth](#) (const std::string &name)
- bool [ggReadImage](#) (const std::string &name, std::vector< GLubyte > &image, GLsizei *pWidth, GLsizei *pHeight, GLenum *pFormat)
- GLuint [ggLoadTexture](#) (const GLvoid *image, GLsizei width, GLsizei height, GLenum format=GL_RGB, GLenum type=GL_UNSIGNED_BYTE, GLenum internal=GL_RGB, GLenum wrap=GL_CLAMP_TO_EDGE, bool swizzle=false)
- GLuint [ggLoadImage](#) (const std::string &name, GLsizei *pWidth=nullptr, GLsizei *pHeight=nullptr, GLenum internal=GL_RGBA, GLenum wrap=GL_CLAMP_TO_EDGE)
- void [ggCreateNormalMap](#) (const GLubyte *hmap, GLsizei width, GLsizei height, GLenum format, GLfloat nz, GLenum internal, std::vector< [GgVector](#) > &nmap)
- GLuint [ggLoadHeight](#) (const std::string &name, GLfloat nz, GLsizei *pWidth=nullptr, GLsizei *pHeight=nullptr, GLenum internal=GL_RGBA)
- GLuint [ggCreateShader](#) (const std::string &vsr, const std::string &fsrc="", const std::string &gsr="", GLint nvarying=0, const char *const *varyings=nullptr, const std::string &vtext="vertex shader", const std::string &ftext="fragment shader", const std::string >ext="geometry shader")
- GLuint [ggLoadShader](#) (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- GLuint [ggLoadShader](#) (const std::array< std::string, 3 > &files, GLint nvarying=0, const char *const *varyings=nullptr)
- GLuint [ggCreateComputeShader](#) (const std::string &csr, const std::string &ctext="compute shader")
- GLuint [ggLoadComputeShader](#) (const std::string &comp)
- std::shared_ptr< [GgPoints](#) > [ggPointsCube](#) (GLsizei countv, GLfloat length=1.0f, GLfloat cx=0.0f, GLfloat cy=0.0f, GLfloat cz=0.0f)
- std::shared_ptr< [GgPoints](#) > [ggPointsSphere](#) (GLsizei countv, GLfloat radius=0.5f, GLfloat cx=0.0f, GLfloat cy=0.0f, GLfloat cz=0.0f)
- std::shared_ptr< [GgTriangles](#) > [ggRectangle](#) (GLfloat width=1.0f, GLfloat height=1.0f)
- std::shared_ptr< [GgTriangles](#) > [ggEllipse](#) (GLfloat width=1.0f, GLfloat height=1.0f, GLuint slices=16)
- std::shared_ptr< [GgTriangles](#) > [ggArraysObj](#) (const std::string &name, bool normalize=false)
- std::shared_ptr< [GgElements](#) > [ggElementsObj](#) (const std::string &name, bool normalize=false)
- std::shared_ptr< [GgElements](#) > [ggElementsMesh](#) (GLuint slices, GLuint stacks, const GLfloat(*pos)[3], const GLfloat(*norm)[3]=nullptr)
- std::shared_ptr< [GgElements](#) > [ggElementsSphere](#) (GLfloat radius=1.0f, int slices=16, int stacks=8)
- bool [ggLoadSimpleObj](#) (const std::string &name, std::vector< std::array< GLuint, 3 > > &group, std::vector< [GgSimpleShader::Material](#) > &material, std::vector< [GgVertex](#) > &vert, bool normalize=false)
- bool [ggLoadSimpleObj](#) (const std::string &name, std::vector< std::array< GLuint, 3 > > &group, std::vector< [GgSimpleShader::Material](#) > &material, std::vector< [GgVertex](#) > &vert, std::vector< GLuint > &face, bool normalize=false)

変数

- GLint [ggBufferAlignment](#)
使用している GPU のバッファアライメント。

8.1.1 詳解

ゲームグラフィックス特論の宿題用補助プログラムの名前空間

8.1.2 列挙型詳解

8.1.2.1 BindingPoints

enum `gg::BindingPoints`

光源と材質の uniform buffer object の結合ポイント.

列挙値

<code>LightBindingPoint</code>	光源の uniform buffer object の結合ポイント.
<code>MaterialBindingPoint</code>	材質の uniform buffer object の結合ポイント.

`gg.h` の 1355 行目に定義があります.

8.1.3 関数詳解

8.1.3.1 _ggError()

```
void gg::_ggError (
    const std::string & name = "",
    unsigned int line = 0) [extern]
```

OpenGL のエラーをチェックする.

引数

<i>name</i>	エラー発生時に標準エラー出力に出力する文字列, <code>nullptr</code> なら何も出力しない.
<i>line</i>	エラー発生時に標準エラー出力に出力する数値, 0 なら何も出力しない.

覚え書き

OpenGL の API を呼び出し直後に実行すればエラーのあるときにメッセージを表示する.

`gg.cpp` の 2609 行目に定義があります.

8.1.3.2 _ggFBOError()

```
void gg::_ggFBOError (
    const std::string & name = "",
    unsigned int line = 0) [extern]
```

FBO のエラーをチェックする.

引数

<i>name</i>	エラー発生時に標準エラー出力に出力する文字列, <code>nullptr</code> なら何も出力しない.
<i>line</i>	エラー発生時に標準エラー出力に出力する数値, 0 なら何も出力しない.

覚え書き

FBO の API を呼び出し直後に実行すればエラーのあるときにメッセージを表示する.

`gg.cpp` の 2653 行目に定義があります.

8.1.3.3 ggArraysObj()

```
std::shared_ptr< gg::GgTriangles > gg::ggArraysObj (
    const std::string & name,
    bool normalize = false) [extern]
```

Wavefront OBJ ファイルを読み込む (Arrays 形式)

引数

<i>name</i>	ファイル名.
<i>normalize</i>	true なら大きさを正規化.

戻り値

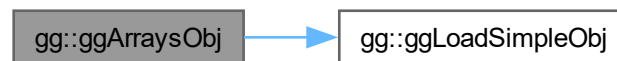
[GgTriangles](#) 型のポインタ.

覚え書き

三角形分割された Wavefront OBJ ファイルを読み込んで GgArrays 形式の三角形データを生成する.

[gg.cpp](#) の 5302 行目に定義があります。

呼び出し関係図:



8.1.3.4 ggConjugate()

```
GgQuaternion gg::ggConjugate (
    const GgQuaternion & q) [inline]
```

共役四元数を返す.

引数

<i>q</i>	GgQuaternion 型の四元数.
----------	-------------------------------------

戻り値

四元数 *q* の共役四元数.

[gg.h](#) の 4787 行目に定義があります。

呼び出し関係図:



8.1.3.5 ggCreateComputeShader()

```
GLuint gg::ggCreateComputeShader (
    const std::string & csrc,
    const std::string & ctext = "compute shader") [extern]
```

コンピュータシェーダのソースプログラムの文字列を読み込んでプログラムオブジェクトを作成する。

引数

<i>csrc</i>	コンピュータシェーダのソースプログラムの文字列.
<i>ctext</i>	コンピュータシェーダのコンパイル時のメッセージに追加する文字列.

戻り値

プログラムオブジェクトのプログラム名 (作成できなければ 0).

[gg.cpp](#) の 5073 行目に定義があります。

被呼び出し関係図:



8.1.3.6 ggCreateNormalMap()

```
void gg::ggCreateNormalMap (
    const GLubyte * hmap,
    GLsizei width,
    GLsizei height,
    GLenum format,
    GLfloat nz,
    GLenum internal,
    std::vector< GgVector > & nmap) [extern]
```

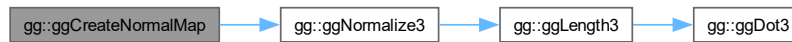
グレースケール画像 (8bit) から法線マップのデータを作成する。

引数

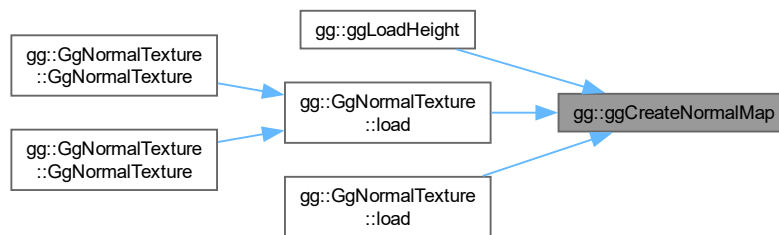
<i>hmap</i>	グレースケール画像のデータ.
<i>width</i>	高さマップのグレースケール画像 <i>hmap</i> の横の画素数.
<i>height</i>	高さマップのグレースケール画像 <i>hmap</i> の縦の画素数.
<i>format</i>	データの書式 (GL_RED, GL_RG, GL_RGB, GL_RGBA).
<i>nz</i>	法線の z 成分の割合.
<i>internal</i>	法線マップを格納するテクスチャの内部フォーマット.
<i>nmap</i>	法線マップを格納する vector.

[gg.cpp](#) の 3897 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.1.3.7 ggCreateShader()

```

GLuint gg::ggCreateShader (
    const std::string & vsrc,
    const std::string & fsrc = "",
    const std::string & gsrc = "",
    GLint nvarying = 0,
    const char *const * varyings = nullptr,
    const std::string & vtext = "vertex shader",
    const std::string & ftext = "fragment shader",
    const std::string & gtext = "geometry shader") [extern]
  
```

シェーダのソースプログラムの文字列を読み込んでプログラムオブジェクトを作成する。

引数

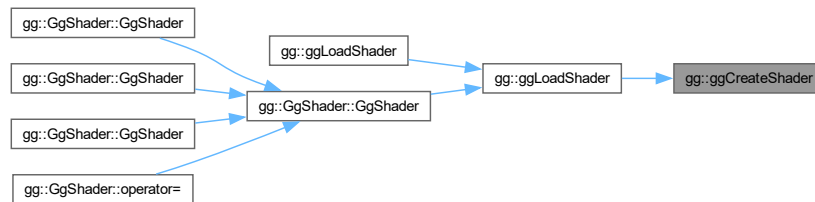
<i>vsrc</i>	バーテックスシェーダのソースプログラムの文字列.
<i>fsrc</i>	フラグメントシェーダのソースプログラムの文字列 (空文字列なら不使用).
<i>gsrc</i>	ジオメトリシェーダのソースプログラムの文字列 (空文字列なら不使用).
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (nullptr なら不使用).
<i>vtext</i>	バーテックスシェーダのコンパイル時のメッセージに追加する文字列.
<i>ftext</i>	フラグメントシェーダのコンパイル時のメッセージに追加する文字列.
<i>gtext</i>	ジオメトリシェーダのコンパイル時のメッセージに追加する文字列.

戻り値

プログラムオブジェクトのプログラム名 (作成できなければ 0).

gg.cpp の 4897 行目に定義があります。

被呼び出し関係図:



8.1.3.8 ggCross() [1/2]

```
GgVector gg::ggCross (
    const GgVector & a,
    const GgVector & b) [inline]
```

GgVector 型の 3 要素の外積.

引数

<i>a</i>	GgVector 型のベクトル.
<i>b</i>	GgVector 型のベクトル.

戻り値

a と *b* の外積.

覚え書き

戻り値の *w* (第4) 要素は 0.

gg.h の 2025 行目に定義があります。

呼び出し関係図:



8.1.3.9 ggCross() [2/2]

```
void gg::ggCross (
    GLfloat * c,
```

```
const GLfloat * a,
const GLfloat * b) [inline]
```

3 要素の外積.

引数

<i>a</i>	GLfloat 型の 3 要素の配列変数.
<i>b</i>	GLfloat 型の 3 要素の配列変数.
<i>c</i>	結果を格納する GLfloat 型の 3 要素の配列変数.

gg.h の 1435 行目に定義があります。
被呼び出し関係図:



8.1.3.10 ggDistance3() [1/2]

```
GLfloat gg::ggDistance3 (
    const GgVector & a,
    const GgVector & b) [inline]
```

GgVector 型の 3 要素の距離.

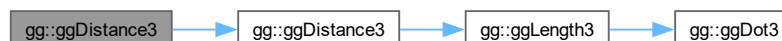
引数

<i>a</i>	GgVector 型のベクトル.
<i>b</i>	GgVector 型のベクトル.

戻り値

a と b の距離.

gg.h の 2062 行目に定義があります。
呼び出し関係図:



8.1.3.11 ggDistance3() [2/2]

```
GLfloat gg::ggDistance3 (
    const GLfloat * a,
    const GLfloat * b) [inline]
```

3 要素の距離.

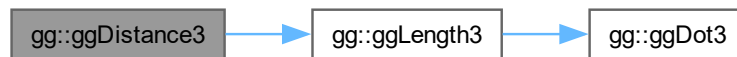
引数

<i>a</i>	GLfloat 型の 3 要素の配列変数.
<i>b</i>	GLfloat 型の 3 要素の配列変数.

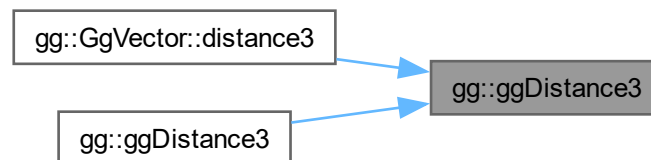
戻り値

a と *b* の距離.

[gg.h](#) の 1472 行目に定義があります。
呼び出し関係図:



被呼び出し関係図:



8.1.3.12 ggDistance4() [1/2]

```

GLfloat gg::ggDistance4 (
    const GgVector & a,
    const GgVector & b) [inline]
  
```

[GgVector](#) 型の 4 要素の距離.

引数

<i>a</i>	GgVector 型の変数.
<i>b</i>	GgVector 型の変数.

戻り値

2 つの [GgVector](#) `a`, `b` の距離.

[gg.h](#) の 2126 行目に定義があります。

呼び出し関係図:



8.1.3.13 ggDistance4() [2/2]

```

GLfloat gg::ggDistance4 (
    const GLfloat * a,
    const GLfloat * b) [inline]
  
```

GLfloat 型の 4 要素の距離.

引数

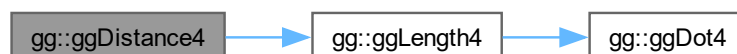
<i>a</i>	GLfloat 型の 4 要素の配列変数.
<i>b</i>	GLfloat 型の 4 要素の配列変数.

戻り値

`a` と `b` のそれぞれの 4 要素の距離.

[gg.h](#) の 1537 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.1.3.14 ggDot3() [1/2]

```
GLfloat gg::ggDot3 (
    const GgVector & a,
    const GgVector & b) [inline]
```

GgVector 型の 3 要素の内積.

引数

<i>a</i>	GgVector 型のベクトル.
<i>b</i>	GgVector 型のベクトル.

戻り値

a と *b* のそれぞれの 3 要素の内積.

[gg.h](#) の 2039 行目に定義があります。

呼び出し関係図:

**8.1.3.15 ggDot3() [2/2]**

```
GLfloat gg::ggDot3 (
    const GLfloat * a,
    const GLfloat * b) [inline]
```

3 要素の内積.

引数

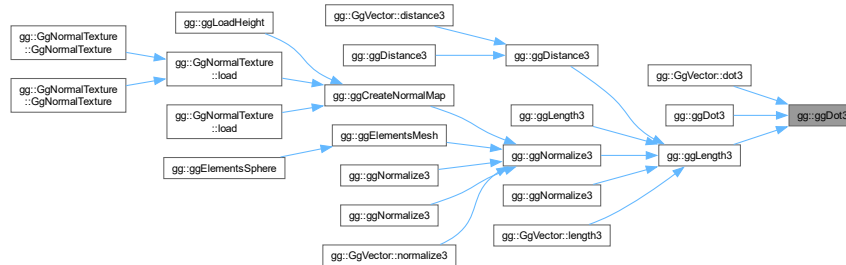
<i>a</i>	GLfloat 型の 3 要素の配列変数.
<i>b</i>	GLfloat 型の 3 要素の配列変数.

戻り値

a と b のそれぞれの 3 要素の内積.

gg.h の 1449 行目に定義があります。

被呼び出し関係図:



8.1.3.16 ggDot4() [1/2]

```
GLfloat gg::ggDot4 (
    const GgVector & a,
    const GgVector & b) [inline]
```

GgVector 型の 4 要素の内積.

引数

a	GgVector 型の変数.
b	GgVector 型の変数.

戻り値

a と b のそれぞれの 4 要素の内積.

gg.h の 2103 行目に定義があります。

呼び出し関係図:



8.1.3.17 ggDot4() [2/2]

```
GLfloat gg::ggDot4 (
    const GLfloat * a,
    const GLfloat * b) [inline]
```

GLfloat 型の 4 要素の内積

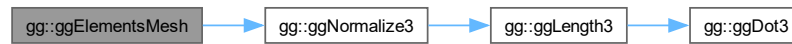
引数

覚え書き

メッシュ状に **GgElements** 形式の三角形データを生成する.

gg.cpp の 5336 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.1.3.19 ggElementsObj()

```
std::shared_ptr< gg::GgElements > gg::ggElementsObj (
    const std::string & name,
    bool normalize = false) [extern]
```

Wavefront OBJ ファイル を読み込む (Elements 形式).

引数

<i>name</i>	ファイル名.
<i>normalize</i>	true なら大きさを正規化.

戻り値

GgElements 型のポインタ.

覚え書き

三角形分割された Wavefront OBJ ファイル を読み込んで **GgElements** 形式の三角形データを生成する。

gg.cpp の 5318 行目に定義があります。
呼び出し関係図:



8.1.3.20 ggElementsSphere()

```
std::shared_ptr< gg::GgElements > gg::ggElementsSphere (
    GLfloat radius = 1.0f,
    int slices = 16,
    int stacks = 8) [extern]
```

球状に三角形データを生成する (Elements 形式)。

引数

<i>radius</i>	球の半径.
<i>slices</i>	球の経度方向の分割数.
<i>stacks</i>	球の緯度方向の分割数.

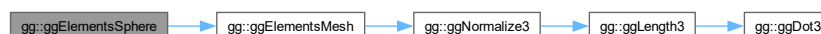
戻り値

GgElements 型のポインタ.

覚え書き

球状に **GgElements** 形式の三角形データを生成する。

gg.cpp の 5431 行目に定義があります。
呼び出し関係図:



8.1.3.21 ggEllipse()

```
std::shared_ptr< gg::GgTriangles > gg::ggEllipse (
    GLfloat width = 1.0f,
    GLfloat height = 1.0f,
    GLuint slices = 16) [extern]
```

楕円状に三角形を生成する.

引数

<i>width</i>	楕円の横幅.
<i>height</i>	楕円の高さ.
<i>slices</i>	楕円の分割数.

戻り値

[GgTriangles](#) 型のポインタ.

[gg.cpp](#) の 5277 行目に定義があります。

8.1.3.22 ggEulerQuaternion() [1/3]

```
GgQuaternion gg::ggEulerQuaternion (
    const GgVector & e) [inline]
```

オイラー角 (e[0], e[1], e[2]) で与えられた回転を表す四元数を返す.

引数

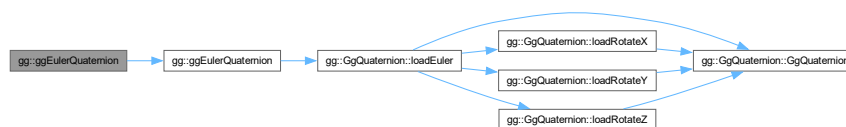
<i>e</i>	オイラー角を表す GgVector 型の変数 (heading, pitch, roll) の参照.
----------	--

戻り値

回転を表す四元数.

[gg.h](#) の 4701 行目に定義があります。

呼び出し関係図:



8.1.3.23 ggEulerQuaternion() [2/3]

```
GgQuaternion gg::ggEulerQuaternion (
    const GLfloat * e) [inline]
```

オイラー角 (e[0], e[1], e[2]) で与えられた回転を表す四元数を返す.

引数

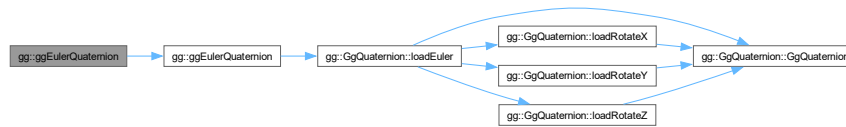
<i>e</i>	オイラー角を表す GLfloat 型の 3 要素の配列変数 (heading, pitch, roll).
----------	---

戻り値

回転を表す四元数.

gg.h の 4690 行目に定義があります。

呼び出し関係図:



8.1.3.24 ggEulerQuaternion() [3/3]

```
GgQuaternion gg::ggEulerQuaternion (
    GLfloat heading,
    GLfloat pitch,
    GLfloat roll) [inline]
```

オイラー角 (heading, pitch, roll) で与えられた回転を表す四元数を返す.

引数

<i>heading</i>	y 軸中心の回転角.
<i>pitch</i>	x 軸中心の回転角.
<i>roll</i>	z 軸中心の回転角.

戻り値

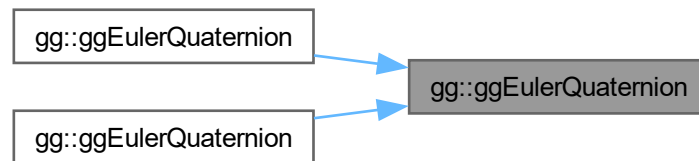
回転を表す四元数.

gg.h の 4678 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.1.3.25 ggFrustum()

```
GgMatrix gg::ggFrustum (
    GLfloat left,
    GLfloat right,
    GLfloat bottom,
    GLfloat top,
    GLfloat zNear,
    GLfloat zFar) [inline]
```

透視投影変換行列を返す。

引数

<i>left</i>	ウィンドウの左端の位置.
<i>right</i>	ウィンドウの右端の位置.
<i>bottom</i>	ウィンドウの下端の位置.
<i>top</i>	ウィンドウの上端の位置.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

求めた透視投影変換行列.

gg.h の 3419 行目に定義があります。

呼び出し関係図:



8.1.3.26 ggIdentity()

```
GgMatrix gg::ggIdentity () [inline]
```

単位行列を返す.

戻り値

単位行列.

gg.h の 3144 行目に定義があります。

呼び出し関係図:



8.1.3.27 ggIdentityQuaternion()

```
GgQuaternion gg::ggIdentityQuaternion () [inline]
```

単位四元数を返す.

戻り値

単位四元数.

gg.h の 4576 行目に定義があります。

呼び出し関係図:



8.1.3.28 ggInit()

```
void gg::ggInit () [extern]
```

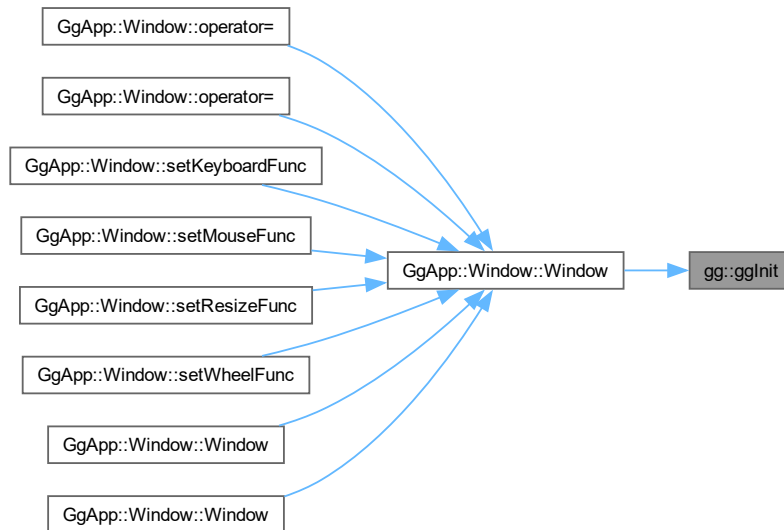
ゲームグラフィックス特論の都合にもとづく初期化を行う.

覚え書き

Windows において OpenGL 1.2 以降の API を有効化する.

gg.cpp の 1354 行目に定義があります。

被呼び出し関係図:



8.1.3.29 ggInvert() [1/2]

```
GgMatrix gg::ggInvert (
    const GgMatrix & m) [inline]
```

逆行列を返す.

引数

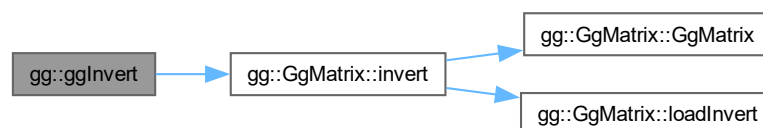
<i>m</i>	元の変換行列.
----------	---------

戻り値

m の逆行列.

gg.h の 3464 行目に定義があります。

呼び出し関係図:



8.1.3.30 ggInvert() [2/2]

```
GgQuaternion gg::ggInvert (
    const GgQuaternion & q) [inline]
```

四元数の逆元を求める。

引数

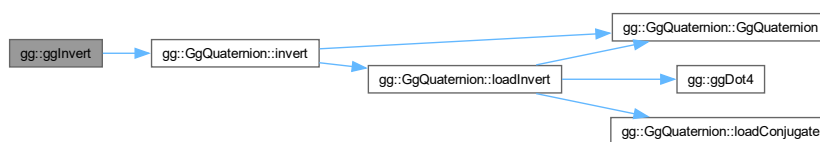
<i>q</i>	GgQuaternion 型の四元数.
----------	---------------------

戻り値

四元数 *q* の逆元.

gg.h の 4798 行目に定義があります。

呼び出し関係図:



8.1.3.31 ggLength3() [1/2]

```
GLfloat gg::ggLength3 (
    const GgVector & a) [inline]
```

GgVector 型の 3 要素の長さ。

引数

<i>a</i>	GgVector 型のベクトル.
----------	------------------

戻り値

a の 3 要素の長さ.

gg.h の 2050 行目に定義があります。

呼び出し関係図:



8.1.3.32 ggLength3() [2/2]

```
GLfloat gg::ggLength3 (
    const GLfloat * a) [inline]
```

3 要素の長さ.

引数

<i>a</i>	GLfloat 型の 3 要素の配列変数.
----------	-----------------------

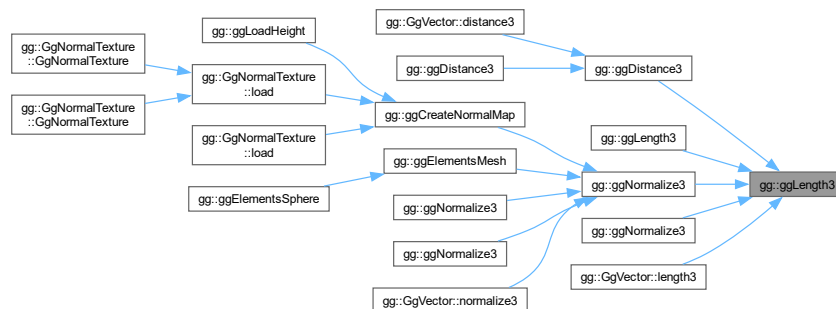
戻り値

a の 3 要素の長さ.

[gg.h](#) の 1460 行目に定義があります。
呼び出し関係図:



被呼び出し関係図:



8.1.3.33 ggLength4() [1/2]

```
GLfloat gg::ggLength4 (
    const GgVector & a) [inline]
```

[GgVector](#) 型の 4 要素の長さ.

引数

<i>a</i>	GgVector 型の変数.
----------	--------------------------------

8.1.3.35 ggLoadComputeShader()

```
GLuint gg::ggLoadComputeShader (
    const std::string & comp) [extern]
```

コンピュートシェーダのソースファイルを読み込んでプログラムオブジェクトを作成する.

引数

<i>comp</i>	コンピュートシェーダのソースファイル名.
-------------	----------------------

戻り値

プログラムオブジェクトのプログラム名 (作成できなければ 0).

[gg.cpp](#) の 5114 行目に定義があります。

呼び出し関係図:



8.1.3.36 ggLoadHeight()

```
GLuint gg::ggLoadHeight (
    const std::string & name,
    GLfloat nz,
    GLsizei * pWidth = nullptr,
    GLsizei * pHeight = nullptr,
    GLenum internal = GL_RGBA) [extern]
```

TGA 画像ファイルの高さマップを読み込んで法線マップのテクスチャを作成する.

引数

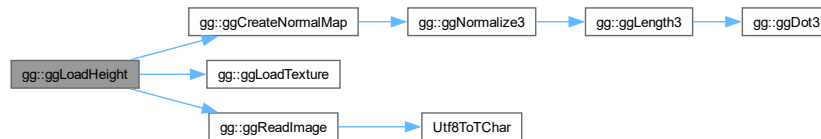
<i>name</i>	読み込むファイル名.
<i>nz</i>	法線の z 成分の割合.
<i>pWidth</i>	読みだした画像ファイルの横の画素数の格納先のポインタ (nullptr なら格納しない).
<i>pHeight</i>	読みだした画像ファイルの縦の画素数の格納先のポインタ (nullptr なら格納しない).
<i>internal</i>	glTexImage2D() に指定するテクスチャの内部フォーマット.

戻り値

テクスチャの作成に成功すればテクスチャ名, 失敗すれば 0.

gg.cpp の 3982 行目に定義があります。

呼び出し関係図:



8.1.3.37 ggLoadImage()

```

GLuint gg::ggLoadImage (
    const std::string & name,
    GLsizei * pWidth = nullptr,
    GLsizei * pHeight = nullptr,
    GLenum internal = GL_RGBA,
    GLenum wrap = GL_CLAMP_TO_EDGE) [extern]
  
```

テクスチャを作成して TGA フォーマットの画像ファイルを読み込む。

引数

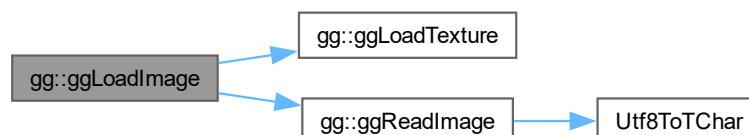
<i>name</i>	読み込むファイル名.
<i>pWidth</i>	読みだした画像ファイルの横の画素数の格納先のポインタ (nullptr なら格納しない).
<i>pHeight</i>	読みだした画像ファイルの縦の画素数の格納先のポインタ (nullptr なら格納しない).
<i>internal</i>	テクスチャの内部フォーマット, デフォルトは GL_RGBA. 0 なら外部フォーマットに合わせる.
<i>wrap</i>	テクスチャのラッピングモード, デフォルトは GL_CLAMP_TO_EDGE.

戻り値

テクスチャの作成に成功すればテクスチャ名, 失敗すれば 0.

gg.cpp の 3848 行目に定義があります。

呼び出し関係図:



8.1.3.38 ggLoadShader() [1/2]

```
GLuint gg::ggLoadShader (
    const std::array< std::string, 3 > & files,
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [inline]
```

シェーダのソースファイルを読み込んでプログラムオブジェクトを作成する。

引数

<i>files</i>	バーテックスシェーダのソースファイル名の配列.
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (nullptr なら不使用).

戻り値

プログラムオブジェクトのプログラム名 (作成できなければ 0).

[gg.h](#) の 5187 行目に定義があります。

呼び出し関係図:



8.1.3.39 ggLoadShader() [2/2]

```
GLuint gg::ggLoadShader (
    const std::string & vert,
    const std::string & frag = "",
    const std::string & geom = "",
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [extern]
```

シェーダのソースファイルを読み込んでプログラムオブジェクトを作成する。

引数

<i>vert</i>	バーテックスシェーダのソースファイル名.
<i>frag</i>	フラグメントシェーダのソースファイル名 (空文字列なら不使用).
<i>geom</i>	ジオメトリシェーダのソースファイル名 (空文字列なら不使用).
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (nullptr なら不使用).

戻り値

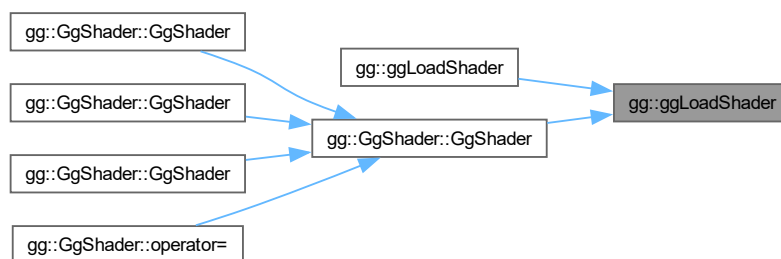
プログラムオブジェクトのプログラム名 (作成できなければ 0).

[gg.cpp](#) の 5040 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.1.3.40 ggLoadSimpleObj() [1/2]

```

bool gg::ggLoadSimpleObj (
    const std::string & name,
    std::vector< std::array< GLuint, 3 > > & group,
    std::vector< GgSimpleShader::Material > & material,
    std::vector< GgVertex > & vert,
    bool normalize = false) [extern]
  
```

三角形分割された OBJ ファイルと MTL ファイルを読み込む (Arrays 形式)

引数

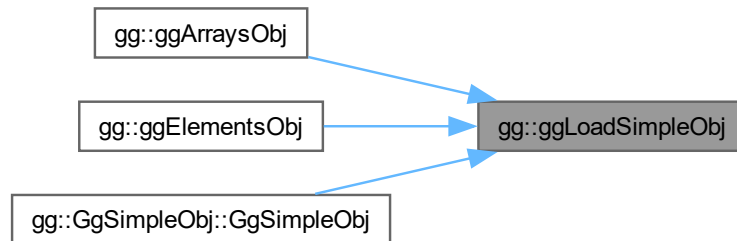
<i>name</i>	読み込む Wavefront OBJ ファイル名.
<i>group</i>	読み込んだデータのポリゴングループごとの最初の三角形の番号と三角形数・材質番号.
<i>material</i>	読み込んだデータのポリゴングループごとの <code>GgSimpleShader::Material</code> 型の材質.
<i>vert</i>	読み込んだデータの頂点属性.
<i>normalize</i>	<code>true</code> なら読み込んだデータの大きさを正規化する.

戻り値

ファイルの読み込みに成功したら `true`.

`gg.cpp` の 4649 行目に定義があります。

被呼び出し関係図:



8.1.3.41 ggLoadSimpleObj() [2/2]

```

bool gg::ggLoadSimpleObj (
    const std::string & name,
    std::vector< std::array< GLuint, 3 > > & group,
    std::vector< GgSimpleShader::Material > & material,
    std::vector< GgVertex > & vert,
    std::vector< GLuint > & face,
    bool normalize = false) [extern]
  
```

三角形分割された OBJ ファイルを読み込む (Elements 形式).

引数

<i>name</i>	読み込む Wavefront OBJ ファイル名.
<i>group</i>	読み込んだデータのポリゴングループごとの最初の三角形の番号と三角形数・材質番号.
<i>material</i>	読み込んだデータのポリゴングループごとの材質.
<i>vert</i>	読み込んだデータの頂点属性.
<i>face</i>	読み込んだデータの三角形の頂点インデックス.
<i>normalize</i>	true なら読み込んだデータの大きさを正規化する.

戻り値

ファイルの読み込みに成功したら `true`.

`gg.cpp` の 4738 行目に定義があります。

8.1.3.42 ggLoadTexture()

```

GLuint gg::ggLoadTexture (
    const GLvoid * image,
    GLsizei width,
    GLsizei height,
  
```

```

GGLenum format = GL_RGB,
GGLenum type = GL_UNSIGNED_BYTE,
GGLenum internal = GL_RGB,
GGLenum wrap = GL_CLAMP_TO_EDGE,
bool swizzle = false) [extern]

```

テクスチャを作成して確保して画像データをテクスチャとして読み込む。

引数

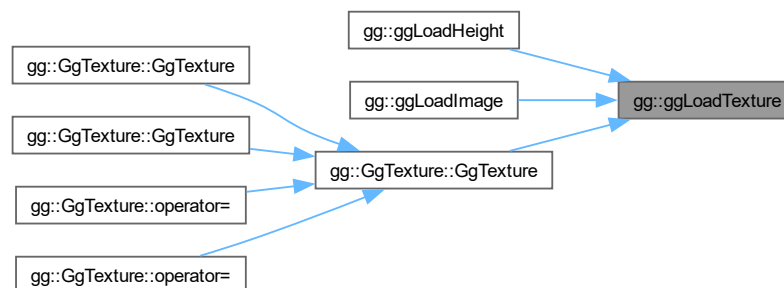
<i>image</i>	テクスチャとして読み込むデータ, nullptr ならテクスチャの作成のみを行う。
<i>width</i>	テクスチャとして読み込むデータ <i>image</i> の横の画素数。
<i>height</i>	テクスチャとして読み込むデータ <i>image</i> の縦の画素数。
<i>format</i>	<i>image</i> のフォーマット。
<i>type</i>	<i>image</i> のデータ型。
<i>internal</i>	テクスチャの内部フォーマット。
<i>wrap</i>	テクスチャのラッピングモード, デフォルトは GL_CLAMP_TO_EDGE。
<i>swizzle</i>	true ならテクスチャの赤と青を入れ替える, デフォルトは false。

戻り値

テクスチャの作成に成功すればテクスチャ名, 失敗すれば 0。

gg.cpp の 3797 行目に定義があります。

被呼び出し関係図:



8.1.3.43 ggLookat() [1/3]

```

GgMatrix gg::ggLookat (
    const GgVector & e,
    const GgVector & t,
    const GgVector & u) [inline]

```

ビュー変換行列を返す。

引数

<i>e</i>	視点の位置を格納した <code>GgVector</code> 型の変数。
<i>t</i>	目標点の位置を格納した <code>GgVector</code> 型の変数。

u	上方向のベクトルを格納した GgVector 型の変数.
-----	-------------------------------------

戻り値

求めたビュー変換行列.

gg.h の 3379 行目に定義があります。

呼び出し関係図:



8.1.3.44 ggLookat() [2/3]

```

GgMatrix gg::ggLookat (
    const GLfloat * e,
    const GLfloat * t,
    const GLfloat * u) [inline]
  
```

ビュー変換行列を返す.

引数

e	視点の位置を格納した GLfloat 型の 3 要素の配列変数.
t	目標点の位置を格納した GLfloat 型の 3 要素の配列変数.
u	上方向のベクトルを格納した GLfloat 型の 3 要素の配列変数.

戻り値

求めたビュー変換行列.

gg.h の 3361 行目に定義があります。

呼び出し関係図:



8.1.3.45 ggLookat() [3/3]

```

GgMatrix gg::ggLookat (
    GLfloat ex,
  
```

```

GLfloat ey,
GLfloat ez,
GLfloat tx,
GLfloat ty,
GLfloat tz,
GLfloat ux,
GLfloat uy,
GLfloat uz) [inline]

```

ビュー変換行列を返す。

引数

<i>ex</i>	視点の位置の x 座標値.
<i>ey</i>	視点の位置の y 座標値.
<i>ez</i>	視点の位置の z 座標値.
<i>tx</i>	目標点の位置の x 座標値.
<i>ty</i>	目標点の位置の y 座標値.
<i>tz</i>	目標点の位置の z 座標値.
<i>ux</i>	上方向のベクトルの x 成分.
<i>uy</i>	上方向のベクトルの y 成分.
<i>uz</i>	上方向のベクトルの z 成分.

戻り値

求めたビュー変換行列.

[gg.h](#) の 3343 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.1.3.46 ggMatrixQuaternion() [1/2]

[GgQuaternion](#) gg::ggMatrixQuaternion (

```
const GgMatrix & m) [inline]
```

回転の変換行列 **m** を表す四元数を返す.

引数

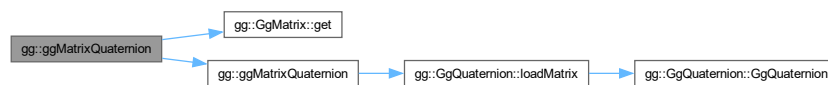
m	GgMatrix 型の変換行列.
----------	-------------------------

戻り値

m による回転の変換に相当する四元数.

gg.h の 4600 行目に定義があります。

呼び出し関係図:



8.1.3.47 ggMatrixQuaternion() [2/2]

```
GgQuaternion gg::ggMatrixQuaternion (
    const GLfloat * a) [inline]
```

回転の変換行列 **m** を表す四元数を返す.

引数

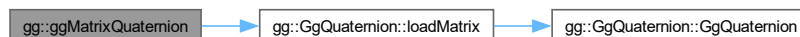
a	GLfloat 型の 16 要素の配列変数.
----------	-------------------------------

戻り値

a による回転の変換に相当する四元数.

gg.h の 4588 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.1.3.48 ggNorm()

```
GLfloat gg::ggNorm (
    const GgQuaternion & q) [inline]
```

四元数のノルムを返す。

引数

<i>q</i>	GgQuaternion 型の四元数.
----------	---------------------

戻り値

四元数 *q* のノルム.

gg.h の 4765 行目に定義があります。

呼び出し関係図:



8.1.3.49 ggNormal()

```
GgMatrix gg::ggNormal (
    const GgMatrix & m) [inline]
```

法線変換行列を返す。

引数

<i>m</i>	元の変換行列.
----------	---------

戻り値

m の法線変換行列.

gg.h の 3475 行目に定義があります。

呼び出し関係図:



8.1.3.50 ggNormalize()

```
GgQuaternion gg::ggNormalize (
    const GgQuaternion & q) [inline]
```

正規化した四元数を返す。

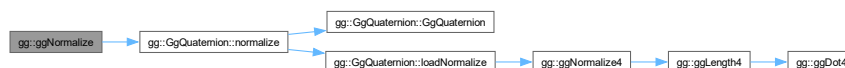
引数

<i>q</i>	GgQuaternion 型の四元数.
----------	---------------------

戻り値

四元数 *q* を正規化した四元数.

gg.h の 4776 行目に定義があります。
呼び出し関係図:



8.1.3.51 ggNormalize3() [1/4]

```
GgVector gg::ggNormalize3 (
    const GgVector & a) [inline]
```

GgVector 型の 3 要素の正規化.

引数

<i>a</i>	GgVector 型のベクトル.
----------	------------------

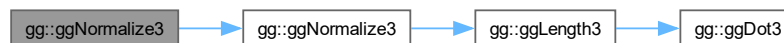
戻り値

a の 3 要素を正規化したもの.

覚え書き

戻り値の *w* (第4) 要素は操作しない.

gg.h の 2076 行目に定義があります。
呼び出し関係図:



8.1.3.52 ggNormalize3() [2/4]

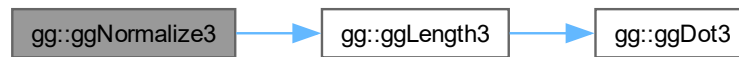
```
void gg::ggNormalize3 (
    const GLfloat * a,
    GLfloat * b) [inline]
```

3 要素の正規化.

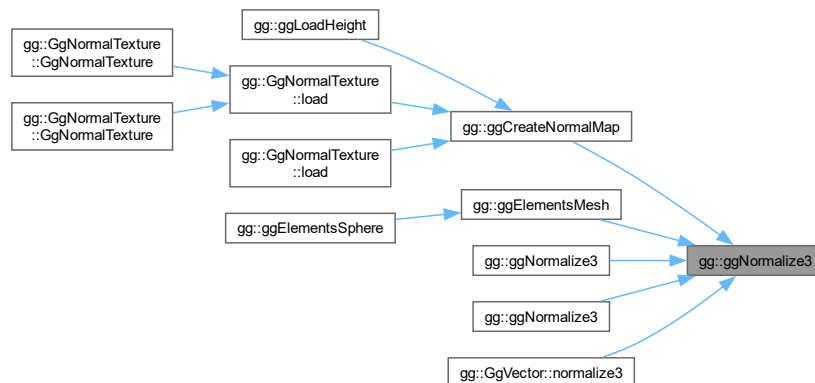
引数

<i>a</i>	GLfloat 型の 3 要素の配列変数.
<i>b</i>	<i>a</i> の 3 要素を正規化した結果を格納する GLfloat 型の 3 要素の配列変数.

gg.h の 1484 行目に定義があります。
呼び出し関係図:



被呼び出し関係図:



8.1.3.53 ggNormalize3() [3/4]

```
void gg::ggNormalize3 (
    GgVector * a) [inline]
```

GgVector 型の 3 要素の正規化.

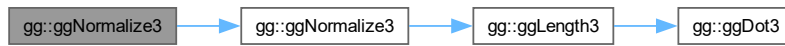
引数

<i>a</i>	GgVector 型の変数のポインタ.
----------	---------------------

覚え書き

`a` の `w` (第4) 要素は操作しない.

`gg.h` の 2091 行目に定義があります。
呼び出し関係図:



8.1.3.54 ggNormalize3() [4/4]

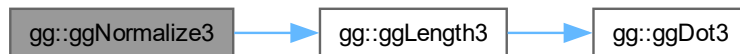
```
void gg::ggNormalize3 (
    GLfloat * a) [inline]
```

3 要素の正規化.

引数

<code>a</code>	GLfloat 型の 3 要素の配列変数.
----------------	-----------------------

`gg.h` の 1498 行目に定義があります。
呼び出し関係図:



8.1.3.55 ggNormalize4() [1/4]

```
GgVector gg::ggNormalize4 (
    const GgVector & a) [inline]
```

`GgVector` 型の 4 要素の正規化.

引数

<code>a</code>	<code>GgVector</code> 型の変数.
----------------	-----------------------------

戻り値

a の 4 要素を正規化した結果.

gg.h の 2137 行目に定義があります。

呼び出し関係図:



8.1.3.56 ggNormalize4() [2/4]

```
void gg::ggNormalize4 (
    const GLfloat * a,
    GLfloat * b) [inline]
```

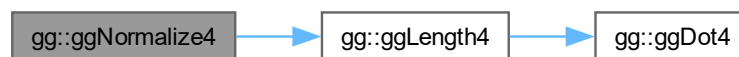
GLfloat 型の 4 要素の正規化.

引数

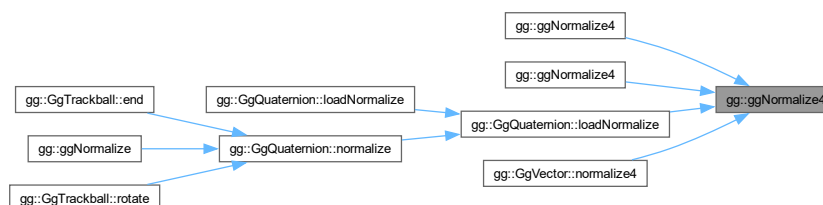
<i>a</i>	GLfloat 型の 4 要素の配列変数.
<i>b</i>	a を正規化した結果を格納する GLfloat 型の 4 要素の配列変数.

gg.h の 1549 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.1.3.57 ggNormalize4() [3/4]

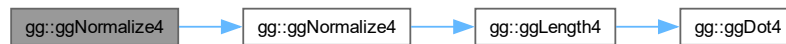
```
void gg::ggNormalize4 (
    GgVector * a) [inline]
```

GgVector 型の 4 要素の正規化.

引数

<i>a</i>	GLfloat 型の 4 要素の配列変数.
----------	-----------------------

gg.h の 2149 行目に定義があります。
呼び出し関係図:



8.1.3.58 ggNormalize4() [4/4]

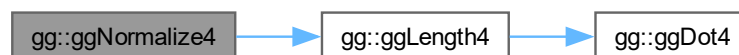
```
void gg::ggNormalize4 (
    GLfloat * a) [inline]
```

GLfloat 型の 4 要素の正規化.

引数

<i>a</i>	正規化する GLfloat 型の 4 要素の配列変数.
----------	-----------------------------

gg.h の 1564 行目に定義があります。
呼び出し関係図:



8.1.3.59 ggOrthogonal()

```
GgMatrix gg::ggOrthogonal (
    GLfloat left,
    GLfloat right,
    GLfloat bottom,
    GLfloat top,
    GLfloat zNear,
    GLfloat zFar) [inline]
```

直交投影変換行列を返す.

引数

<i>left</i>	ウィンドウの左端の位置.
<i>right</i>	ウィンドウの右端の位置.

<i>bottom</i>	ウィンドウの下端の位置.
<i>top</i>	ウィンドウの上端の位置.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

求めた直交投影変換行列.

[gg.h](#) の 3400 行目に定義があります。
呼び出し関係図:



8.1.3.60 ggPerspective()

```

GgMatrix gg::ggPerspective (
    GLfloat fovy,
    GLfloat aspect,
    GLfloat zNear,
    GLfloat zFar) [inline]
  
```

画角を指定して透視投影変換行列を返す.

引数

<i>fovy</i>	y 方向の画角.
<i>aspect</i>	縦横比.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

求めた透視投影変換行列.

[gg.h](#) の 3438 行目に定義があります。
呼び出し関係図:



被呼び出し関係図:



8.1.3.61 ggPointsCube()

```
std::shared_ptr< Gg::GgPoints > gg::ggPointsCube (
    GLsizei countv,
    GLfloat length = 1.0f,
    GLfloat cx = 0.0f,
    GLfloat cy = 0.0f,
    GLfloat cz = 0.0f) [extern]
```

点群を立方体状に生成する。

引数

<i>countv</i>	生成する点の数.
<i>length</i>	点群を生成する立方体の一辺の長さ.
<i>cx</i>	点群の中心の x 座標.
<i>cy</i>	点群の中心の y 座標.
<i>cz</i>	点群の中心の z 座標.

戻り値

[GgPoints](#) 型の ポインタ.

[gg.cpp](#) の 5201 行目に定義があります。

8.1.3.62 ggPointsSphere()

```
std::shared_ptr< Gg::GgPoints > gg::ggPointsSphere (
    GLsizei countv,
    GLfloat radius = 0.5f,
    GLfloat cx = 0.0f,
    GLfloat cy = 0.0f,
    GLfloat cz = 0.0f) [extern]
```

点群を球状に生成する。

引数

<i>countv</i>	生成する点の数.
<i>radius</i>	点群を生成する半径.
<i>cx</i>	点群の中心の x 座標.
<i>cy</i>	点群の中心の y 座標.

<code>cz</code>	点群の中心の z 座標.
-----------------	----------------

戻り値

`GgPoints` 型のポインタ.

`gg.cpp` の 5228 行目に定義があります.

8.1.3.63 `ggQuaternionMatrix()`

```
GgMatrix gg::ggQuaternionMatrix (
    const GgQuaternion & q) [inline]
```

四元数 q の回転の変換行列を返す.

引数

q	元の四元数.
-----	--------

戻り値

四元数 q が表す回転に相当する `GgMatrix` 型の変換行列.

`gg.h` の 4611 行目に定義があります.

呼び出し関係図:



8.1.3.64 `ggQuaternionTransposeMatrix()`

```
GgMatrix gg::ggQuaternionTransposeMatrix (
    const GgQuaternion & q) [inline]
```

四元数 q の回転の転置した変換行列を返す.

引数

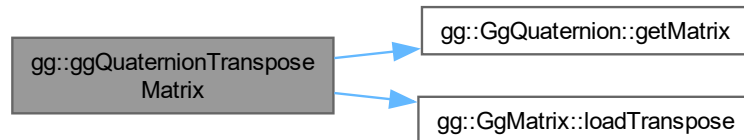
q	元の四元数.
-----	--------

戻り値

四元数 q が表す回転に相当する転置した `GgMatrix` 型の変換行列.

`gg.h` の 4624 行目に定義があります。

呼び出し関係図:



8.1.3.65 ggReadImage()

```
bool gg::ggReadImage (
    const std::string & name,
    std::vector< GLubyte > & image,
    GLsizei * pWidth,
    GLsizei * pHeight,
    GLenum * pFormat) [extern]
```

TGA ファイル (8/16/24/32bit) をメモリに読み込む.

引数

<i>name</i>	読み込むファイル名.
<i>image</i>	読み込んだデータを格納する vector.
<i>pWidth</i>	読み込んだ画像の横の画素数の格納先のポインタ, nullptr なら格納しない.
<i>pHeight</i>	読み込んだ画像の縦の画素数の格納先のポインタ, nullptr なら格納しない.
<i>pFormat</i>	読み込んだファイルの書式 (GL_RED, GL_RG, GL_RGB, GL_RGBA) の格納先のポインタ, nullptr なら格納しない.

戻り値

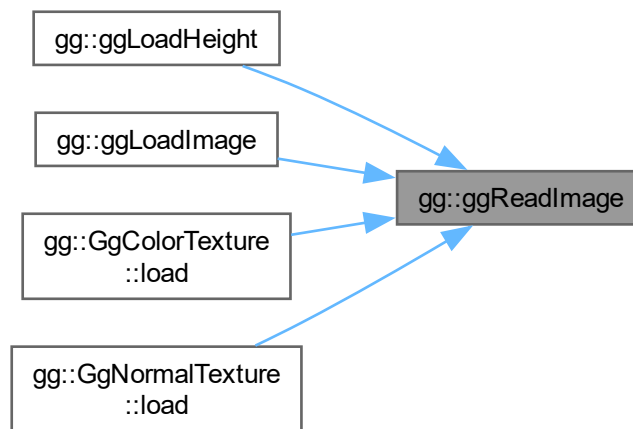
読み込みに成功すれば `true`, 失敗すれば `false`.

`gg.cpp` の 3682 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.1.3.66 ggRectangle()

```
std::shared_ptr< GgTriangles > gg::ggRectangle (
    GLfloat width = 1.0f,
    GLfloat height = 1.0f) [extern]
```

矩形状に 2 枚の三角形を生成する.

引数

<i>width</i>	矩形の横幅.
<i>height</i>	矩形の高さ.

戻り値

[GgTriangles](#) 型のポインタ.

[gg.cpp](#) の 5256 行目に定義があります。

8.1.3.67 ggRotate() [1/5]

```
GgMatrix gg::ggRotate (
    const GgVector & r) [inline]
```

r 方向のベクトルを軸とする回転の変換行列を乗じた結果を返す.

引数

<i>r</i>	回転軸のベクトルと回転角を表す GgVector 型の変数.
----------	--

戻り値

($r[0]$, $r[1]$, $r[2]$) を軸に $r[3]$ だけ回転する変換行列.

[gg.h](#) の 3323 行目に定義があります。

呼び出し関係図:



8.1.3.68 ggRotate() [2/5]

```
GgMatrix gg::ggRotate (
    const GgVector & r,
    GLfloat a) [inline]
```

r 方向のベクトルを軸とする回転の変換行列を乗じた結果を返す.

引数

r	回転軸のベクトルを表す GgVector 型の変数.
a	回転角.

戻り値

r を軸に a だけ回転する変換行列.

[gg.h](#) の 3299 行目に定義があります。

呼び出し関係図:



8.1.3.69 ggRotate() [3/5]

```
GgMatrix gg::ggRotate (
    const GLfloat * r) [inline]
```

r 方向のベクトルを軸とする回転の変換行列を乗じた結果を返す.

引数

<i>r</i>	回転軸のベクトルと回転角を表す GLfloat 型の 4 要素の配列変数.
----------	---------------------------------------

戻り値

(*r*[0], *r*[1], *r*[2]) を軸に *r*[3] だけ回転する変換行列.

gg.h の 3311 行目に定義があります.

呼び出し関係図:



8.1.3.70 ggRotate() [4/5]

```
GgMatrix gg::ggRotate (
    const GLfloat * r,
    GLfloat a) [inline]
```

r 方向のベクトルを軸とする回転の変換行列を乗じた結果を返す.

引数

<i>r</i>	回転軸のベクトルを表す GLfloat 型の 3 要素の配列変数.
<i>a</i>	回転角.

戻り値

r を軸に *a* だけ回転する変換行列.

gg.h の 3286 行目に定義があります.

呼び出し関係図:



8.1.3.71 ggRotate() [5/5]

```
GgMatrix gg::ggRotate (
    GLfloat x,
    GLfloat y,
```

```
GLfloat z,
GLfloat a) [inline]
```

(x, y, z) 方向のベクトルを軸とする回転の変換行列を乗じた結果を返す.

引数

<i>x</i>	回転軸の x 成分.
<i>y</i>	回転軸の y 成分.
<i>z</i>	回転軸の z 成分.
<i>a</i>	回転角.

戻り値

(x, y, z) を軸にさらに a 回転する変換行列.

[gg.h](#) の 3273 行目に定義があります。

呼び出し関係図:



8.1.3.72 ggRotateQuaternion() [1/3]

```
GgQuaternion gg::ggRotateQuaternion (
    const GLfloat * v) [inline]
```

(v[0], v[1], v[2]) を軸として角度 v[3] 回転する四元数を返す.

引数

<i>v</i>	軸ベクトルを表す GLfloat 型の 4 要素の配列変数.
----------	--------------------------------

戻り値

回転を表す四元数.

[gg.h](#) の 4665 行目に定義があります。

呼び出し関係図:



8.1.3.73 ggRotateQuaternion() [2/3]

```
GgQuaternion gg::ggRotateQuaternion (
    const GLfloat * v,
    GLfloat a) [inline]
```

(v[0], v[1], v[2]) を軸として角度 a 回転する四元数を返す.

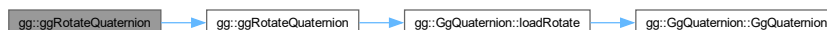
引数

<i>v</i>	軸ベクトルを表す GLfloat 型の 3 要素の配列変数.
<i>a</i>	回転角.

戻り値

回転を表す四元数.

gg.h の 4654 行目に定義があります。
呼び出し関係図:



8.1.3.74 ggRotateQuaternion() [3/3]

```
GgQuaternion gg::ggRotateQuaternion (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat a) [inline]
```

(x, y, z) を軸として角度 a 回転する四元数を返す.

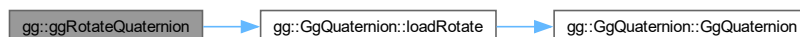
引数

<i>x</i>	軸ベクトルの x 成分.
<i>y</i>	軸ベクトルの y 成分.
<i>z</i>	軸ベクトルの z 成分.
<i>a</i>	回転角.

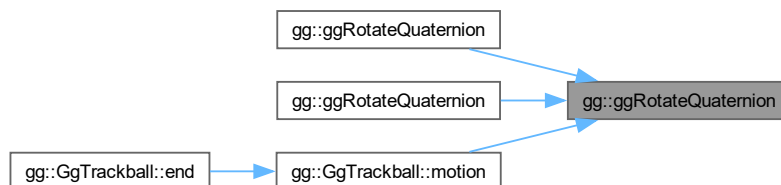
戻り値

回転を表す四元数.

gg.h の 4641 行目に定義があります。
呼び出し関係図:



被呼び出し関係図:



8.1.3.75 ggRotateX()

```
GgMatrix gg::ggRotateX (
    GLfloat a) [inline]
```

x 軸中心の回転の変換行列を返す.

引数

<i>a</i>	回転角.
----------	------

戻り値

x 軸中心に *a* だけ回転する変換行列.

[gg.h](#) の 3234 行目に定義があります。
呼び出し関係図:



8.1.3.76 ggRotateY()

```
GgMatrix gg::ggRotateY (
    GLfloat a) [inline]
```

y 軸中心の回転の変換行列を返す.

引数

<i>a</i>	回転角.
----------	------

戻り値

y 軸中心に a だけ回転する変換行列.

gg.h の 3246 行目に定義があります。

呼び出し関係図:



8.1.3.77 ggRotateZ()

```
GgMatrix gg::ggRotateZ (
    GLfloat a) [inline]
```

z 軸中心の回転の変換行列を返す.

引数

<i>a</i>	回転角.
----------	------

戻り値

z 軸中心に a だけ回転する変換行列.

gg.h の 3258 行目に定義があります。

呼び出し関係図:



8.1.3.78 ggSaveColor()

```
bool gg::ggSaveColor (
    const std::string & name) [extern]
```

カラーバッファの内容を TGA ファイルに保存する.

引数

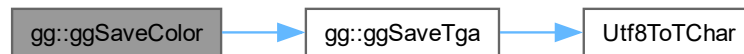
<i>name</i>	保存するファイル名.
-------------	------------

戻り値

保存に成功すれば `true`, 失敗すれば `false`.

`gg.cpp` の 3626 行目に定義があります。

呼び出し関係図:



8.1.3.79 ggSaveDepth()

```
bool gg::ggSaveDepth (
    const std::string & name) [extern]
```

デブスバッファの内容を TGA ファイルに保存する.

引数

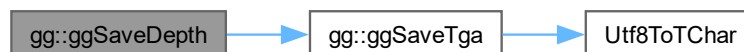
<i>name</i>	保存するファイル名.
-------------	------------

戻り値

保存に成功すれば `true`, 失敗すれば `false`.

`gg.cpp` の 3652 行目に定義があります。

呼び出し関係図:



8.1.3.80 ggSaveTga()

```
bool gg::ggSaveTga (
    const std::string & name,
    const void * buffer,
    unsigned int width,
    unsigned int height,
    unsigned int depth) [extern]
```

配列の内容を TGA ファイルに保存する.

引数

<i>name</i>	保存するファイル名.
<i>buffer</i>	画像データを格納した配列.
<i>width</i>	画像の横の画素数.
<i>height</i>	画像の縦の画素数.
<i>depth</i>	1画素のバイト数.

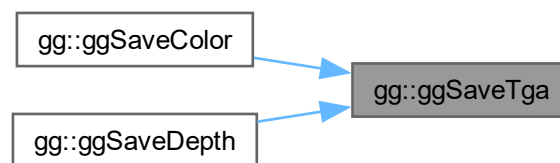
戻り値

保存に成功すれば `true`, 失敗すれば `false`.

`gg.cpp` の 3550 行目に定義があります。
呼び出し関係図:



被呼び出し関係図:



8.1.3.81 ggScale() [1/3]

```
GgMatrix gg::ggScale (  
    const GgVector & s) [inline]
```

拡大縮小の変換行列を返す.

引数

s	拡大率の <code>GgVector</code> 型の変数.
----------	----------------------------------

戻り値

拡大縮小の変換行列。

[gg.h](#) の 3222 行目に定義があります。

呼び出し関係図:



8.1.3.82 ggScale() [2/3]

```
GgMatrix gg::ggScale (
    const GLfloat * s) [inline]
```

拡大縮小の変換行列を返す。

引数

s	拡大率の GLfloat 型の 3 要素の配列変数 (x, y, z).
----------	--------------------------------------

戻り値

拡大縮小の変換行列。

[gg.h](#) の 3210 行目に定義があります。

呼び出し関係図:



8.1.3.83 ggScale() [3/3]

```
GgMatrix gg::ggScale (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w = 1.0f) [inline]
```

拡大縮小の変換行列を返す。

引数

<i>x</i>	x 方向の拡大率.
<i>y</i>	y 方向の拡大率.
<i>z</i>	z 方向の拡大率.
<i>w</i>	拡大率のスケールファクタ (= 1.0f).

戻り値

拡大縮小の変換行列.

gg.h の 3198 行目に定義があります。

呼び出し関係図:



8.1.3.84 ggSlerp() [1/4]

```
GgQuaternion gg::ggSlerp (
    const GgQuaternion & q,
    const GgQuaternion & r,
    GLfloat t) [inline]
```

二つの四元数の球面線形補間の結果を返す.

引数

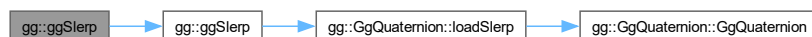
<i>q</i>	GgQuaternion 型の四元数.
<i>r</i>	GgQuaternion 型の四元数.
<i>t</i>	補間パラメータ.

戻り値

q, *r* を *t* で内分した四元数.

gg.h の 4728 行目に定義があります。

呼び出し関係図:



8.1.3.85 ggSlerp() [2/4]

```
GgQuaternion gg::ggSlerp (
    const GgQuaternion & q,
```

```
const GLfloat * a,
GLfloat t) [inline]
```

二つの四元数の球面線形補間の結果を返す.

引数

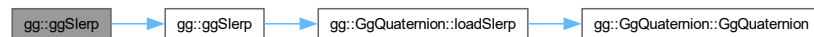
<i>q</i>	GgQuaternion 型の四元数.
<i>a</i>	四元数を格納した <code>GLfloat</code> 型の 4 要素の配列変数.
<i>t</i>	補間パラメータ.

戻り値

q, *a* を *t* で内分した四元数.

[gg.h](#) の 4741 行目に定義があります。

呼び出し関係図:



8.1.3.86 ggSlerp() [3/4]

```
GgQuaternion gg::ggSlerp (
    const GLfloat * a,
    const GgQuaternion & q,
    GLfloat t) [inline]
```

二つの四元数の球面線形補間の結果を返す.

引数

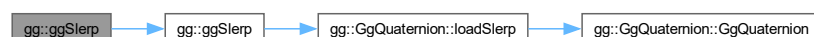
<i>a</i>	四元数を格納した <code>GLfloat</code> 型の 4 要素の配列変数.
<i>q</i>	GgQuaternion 型の四元数.
<i>t</i>	補間パラメータ.

戻り値

a, *q* を *t* で内分した四元数.

[gg.h](#) の 4754 行目に定義があります。

呼び出し関係図:



8.1.3.87 ggSlerp() [4/4]

```
GgQuaternion gg::ggSlerp (
    const GLfloat * a,
    const GLfloat * b,
    GLfloat t) [inline]
```

二つの四元数の球面線形補間の結果を返す.

引数

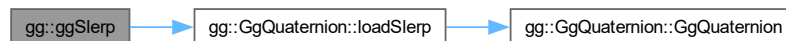
<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
<i>b</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
<i>t</i>	補間パラメータ.

戻り値

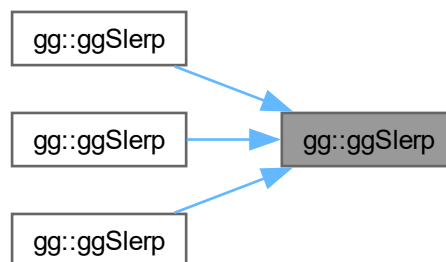
a, *b* を *t* で内分した四元数.

gg.h の 4714 行目に定義があります.

呼び出し関係図:



被呼び出し関係図:



8.1.3.88 ggTranslate() [1/3]

```
GgMatrix gg::ggTranslate (
    const GgVector & t) [inline]
```

平行移動の変換行列を返す.

引数

<i>t</i>	移動量の GgVector 型の変数.
----------	----------------------------

戻り値

平行移動の変換行列.

gg.h の 3183 行目に定義があります。
呼び出し関係図:



8.1.3.89 ggTranslate() [2/3]

```
GgMatrix gg::ggTranslate (
    const GLfloat * t) [inline]
```

平行移動の変換行列を返す.

引数

<i>t</i>	移動量の GLfloat 型の 3 要素の配列変数 (x, y, z).
----------	---

戻り値

平行移動の変換行列.

gg.h の 3171 行目に定義があります。
呼び出し関係図:



8.1.3.90 ggTranslate() [3/3]

```
GgMatrix gg::ggTranslate (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w = 1.0f) [inline]
```

平行移動の変換行列を返す.

引数

<i>x</i>	x 方向の移動量.
<i>y</i>	y 方向の移動量.
<i>z</i>	z 方向の移動量.
<i>w</i>	移動量のスケールファクタ (= 1.0f).

戻り値

平行移動の変換行列.

[gg.h](#) の 3159 行目に定義があります。

呼び出し関係図:



8.1.3.91 ggTranspose()

```
GgMatrix gg::ggTranspose (
    const GgMatrix & m) [inline]
```

転置行列を返す.

引数

<i>m</i>	元の変換行列.
----------	---------

戻り値

m の転置行列.

[gg.h](#) の 3453 行目に定義があります。

呼び出し関係図:



8.1.3.92 operator*()

```
GgVector gg::operator* (
    GLfloat a,
    const GgVector & b) [inline]
```

スカラーに `GgVector` 型の各要素を乗じた積を返す.

引数

<i>a</i>	GLfloat 型の値.
<i>b</i>	GgVector 型のベクトル.

戻り値

a に *b* の各要素を乗じた積.

gg.h の 1998 行目に定義があります。

8.1.3.93 operator+() [1/2]

```
const GgVector & gg::operator+ (
    const GgVector & v) [inline]
```

単項の + 演算子なので何もしない.

引数

<i>v</i>	GgVector 型のベクトル.
----------	------------------

戻り値

v の値.

gg.h の 1951 行目に定義があります。

8.1.3.94 operator+() [2/2]

```
GgVector gg::operator+ (
    GLfloat a,
    const GgVector & b) [inline]
```

スカラーに GgVector 型の各要素を足した和を返す.

引数

<i>a</i>	GLfloat 型の値.
<i>b</i>	GgVector 型のベクトル.

戻り値

a に *b* の各要素を足した和.

gg.h の 1963 行目に定義があります。

8.1.3.95 operator-() [1/2]

```
const GgVector gg::operator- (
    const GgVector & v) [inline]
```

符号の反転.

引数

<i>v</i>	GgVector 型のベクトル.
----------	------------------

戻り値

v の値の符号を反転した結果.

gg.h の 1974 行目に定義があります。

8.1.3.96 operator-() [2/2]

```
GgVector gg::operator- (
    GLfloat a,
    const GgVector & b) [inline]
```

スカラーから GgVector 型の各要素を引いた差を返す.

引数

<i>a</i>	GLfloat 型の値.
<i>b</i>	GgVector 型のベクトル.

戻り値

a から b の各要素を引いた差.

gg.h の 1986 行目に定義があります。

8.1.3.97 operator/()

```
GgVector gg::operator/ (
    GLfloat a,
    const GgVector & b) [inline]
```

スカラーを GgVector 型の各要素で割った商を返す.

引数

<i>a</i>	GLfloat 型の値.
<i>b</i>	GgVector 型のベクトル.

戻り値

a を b の各要素で割った商.

gg.h の 2010 行目に定義があります。

8.1.4 変数詳解

8.1.4.1 ggBufferAlignment

```
GLint gg::ggBufferAlignment [extern]
```

使用している GPU のバッファアライメント.
使用している GPU のバッファオブジェクトのアライメント, 初期化に取得される.

Chapter 9

クラス詳解

9.1 Config クラス

```
#include <Config.h>
```

公開メンバ関数

- [Config](#) ()
- [Config](#) (const std::string &filename)
- virtual [~Config](#) ()
- auto [getWidth](#) () const
- auto [getHeight](#) () const
- bool [load](#) (const std::string &filename)
- bool [save](#) (const std::string &filename) const

フレンド

- class [Menu](#)

9.1.1 詳解

構成データ

[Config.h](#) の 20 行目に定義があります。

9.1.2 構築子と解体子

9.1.2.1 Config() [1/2]

`Config::Config ()`

コンストラクタ

[Config.cpp](#) の 28 行目に定義があります。

被呼び出し関係図:



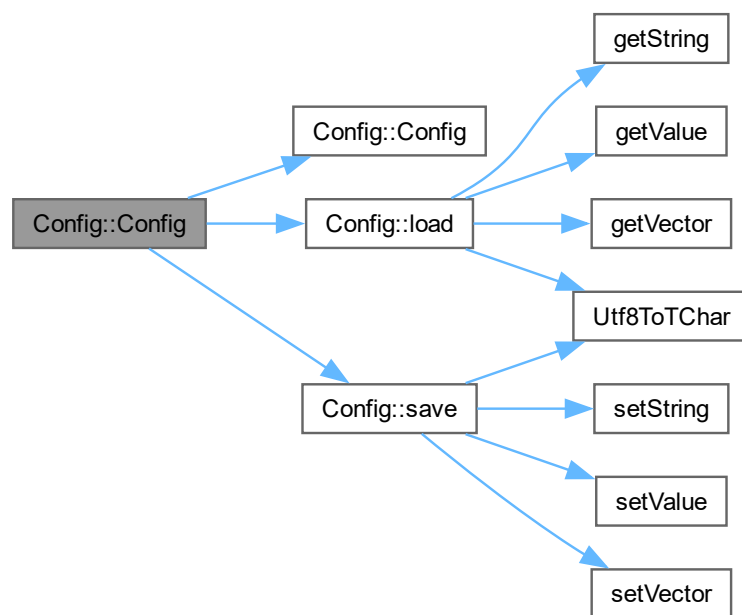
9.1.2.2 Config() [2/2]

```
Config::Config (
    const std::string & filename)
    ファイルから構成データを読み込むコンストラクタ
```

引数

<i>filename</i>	読み込む構成ファイル名
-----------------	-------------

[Config.cpp](#) の 45 行目に定義があります。
呼び出し関係図:



9.1.2.3 ~Config()

```
Config::~~Config () [virtual]
    デストラクタ
    Config.cpp の 55 行目に定義があります。
```

9.1.3 関数詳解

9.1.3.1 getHeight()

```
auto Config::getHeight () const [inline]
    ウィンドウの高さを得る
```

戻り値

ウィンドウの高さ

[Config.h](#) の 77 行目に定義があります。

被呼び出し関係図:



9.1.3.2 getWidth()

```
auto Config::getWidth () const [inline]
```

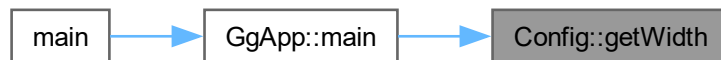
ウィンドウの横幅を得る

戻り値

ウィンドウの横幅

[Config.h](#) の 67 行目に定義があります。

被呼び出し関係図:



9.1.3.3 load()

```
bool Config::load (
```

`const std::string & filename`)

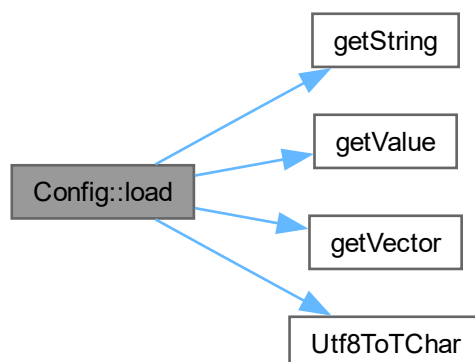
構成ファイルを読み込む

引数

<code>filename</code>	読み込む構成ファイル名
-----------------------	-------------

[Config.cpp](#) の 62 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.1.3.4 save()

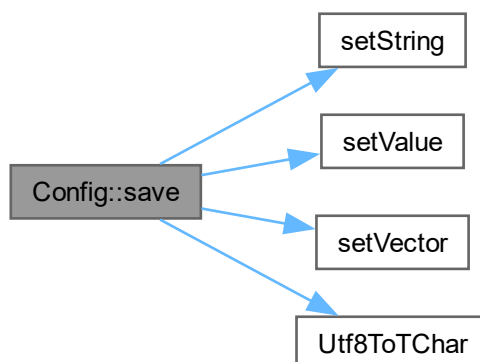
```
bool Config::save (  
    const std::string & filename) const  
構成ファイルを書き出す
```

引数

書き出す構成ファイル名	
-------------	--

[Config.cpp](#) の 115 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.1.4 フレンドと関連関数の詳解

9.1.4.1 Menu

```
friend class Menu [friend]
```

[Config.h](#) の 23 行目に定義があります。

このクラス詳解は次のファイルから抽出されました:

- [Config.h](#)
- [Config.cpp](#)

9.2 GgApp クラス

```
#include <GgApp.h>
```

クラス

- class [Window](#)

公開メンバ関数

- [GgApp](#) (int major=0, int minor=1)
- [GgApp](#) (const [GgApp](#) &w)=delete

- `GgApp (GgApp &&w)=default`
- `virtual ~GgApp ()`
- `GgApp & operator= (const GgApp &w)=delete`
- `GgApp & operator= (GgApp &&w)=default`
- `int main (int argc, const char *const *argv)`

静的公開メンバ関数

- `static std::string getUsername ()`

9.2.1 詳解

ゲームグラフィックス特論宿題アプリケーションクラス。
`GgApp.h` の 93 行目に定義があります。

9.2.2 構築子と解体子

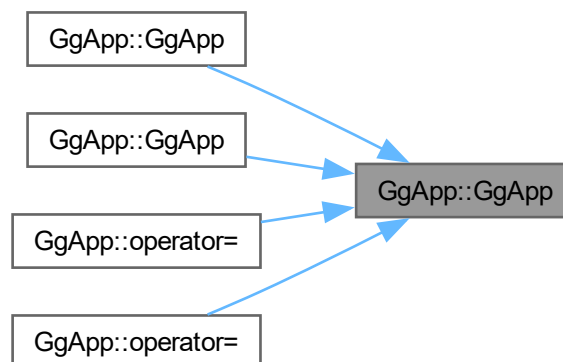
9.2.2.1 GgApp() [1/3]

```
GgApp::GgApp (
    int major = 0,
    int minor = 1)
コンストラクタ.
```

引数

<i>major</i>	使用する OpenGL の major 番号, 0 なら無指定.
<i>minor</i>	使用する OpenGL の minor 番号, major 番号が 0 なら無視.

`GgApp.cpp` の 49 行目に定義があります。
被呼び出し関係図:



9.2.2.2 GgApp() [2/3]

```
GgApp::GgApp (
    const GgApp & w) [delete]
コピーコンストラクタは使用しない.
```

引数

<i>w</i>	コピー元のオブジェクト.
----------	--------------

呼び出し関係図:



9.2.2.3 GgApp() [3/3]

```
GgApp::GgApp (
    GgApp && w) [default]
ムーブコンストラクタ.
```

引数

<i>w</i>	ムーブ元のオブジェクト.
----------	--------------

呼び出し関係図:



9.2.2.4 ~GgApp()

```
GgApp::~GgApp () [virtual]
デストラクタ.
GgApp.cpp の 91 行目に定義があります。
```

9.2.3 関数詳解

9.2.3.1 getUsername()

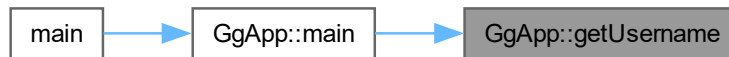
```
std::string GgApp::getUsername () [static]
ユーザ名を得る.
```

戻り値

ユーザ名の文字列.

[GgApp.cpp](#) の 1515 行目に定義があります。

被呼び出し関係図:



9.2.3.2 main()

```
int GgApp::main (  
    int argc,  
    const char *const * argv)  
アプリケーション本体.
```

引数

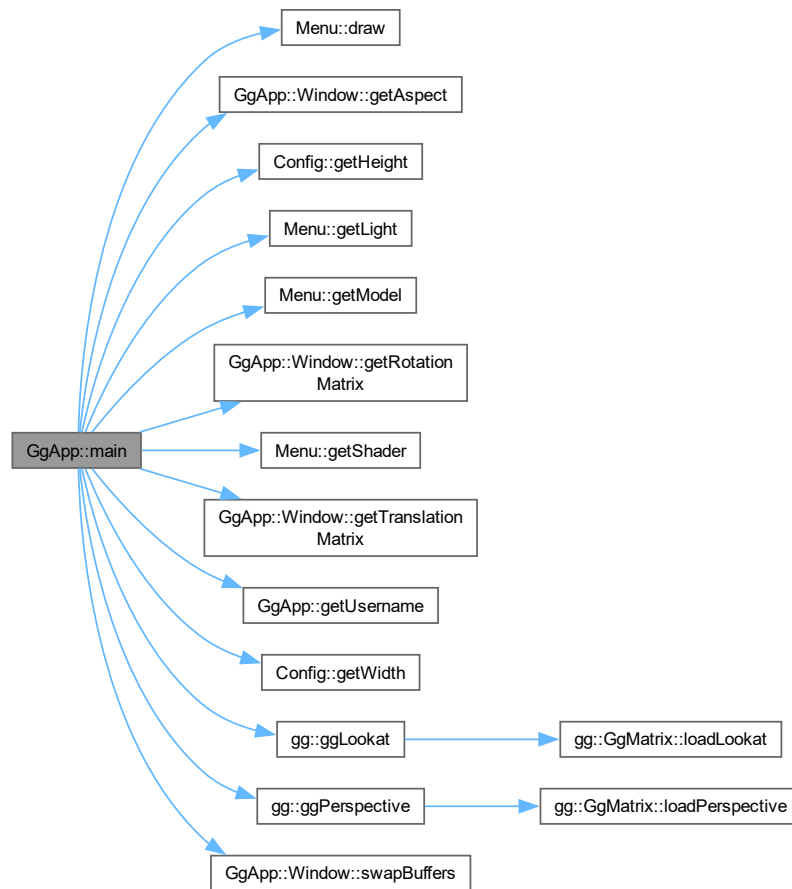
<i>argc</i>	コマンドライン引数の数.
<i>argv</i>	コマンドライン引数の配列.

戻り値

アプリケーションの終了ステータス.

[ggsample01.cpp](#) の 30 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.2.3.3 operator=() [1/2]

```
GgApp & GgApp::operator= (
    const GgApp & w) [delete]
```

代入演算子は使用しない.

引数

<i>w</i>	代入元のオブジェクト.
----------	-------------

戻り値

代入後のこのオブジェクトの参照.

呼び出し関係図:



9.2.3.4 operator=() [2/2]

```
GgApp & GgApp::operator= (
    GgApp && w) [default]
```

ムーブ代入演算子.

引数

<i>w</i>	ムーブ代入元のオブジェクト.
----------	----------------

戻り値

ムーブ代入後のこのオブジェクトの参照.

呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [GgApp.h](#)
- [GgApp.cpp](#)
- [ggsample01.cpp](#)

9.3 gg::GgBuffer< T > クラステンプレート

```
#include <gg.h>
```


公開メンバ関数

- `GgBuffer` (GLenum target, const T *data, GLsizei stride, GLsizei count, GLenum usage)
- `GgBuffer` (const GgBuffer< T > &buffer)=delete
- `GgBuffer` (GgBuffer< T > &&buffer)=default
- virtual `~GgBuffer` ()
- `GgBuffer< T > & operator=` (const GgBuffer< T > &buffer)=delete
- `GgBuffer< T > & operator=` (GgBuffer< T > &&buffer)=default
- const GLuint & `getTarget` () const
- GLsizeiptr `getStride` () const
- const GLsizei & `getCount` () const
- const GLuint & `getBuffer` () const
- void `bind` () const
- void `unbind` () const
- void * `map` () const
- void * `map` (GLint first, GLsizei count) const
- void `unmap` () const
- void `send` (const T *data, GLint first, GLsizei count) const
- void `read` (T *data, GLint first, GLsizei count) const
- void `copy` (GLuint src_buffer, GLint src_first=0, GLint dst_first=0, GLsizei count=0) const

9.3.1 詳解

```
template<typename T>
class gg::GgBuffer< T >
```

バッファオブジェクト.

覚え書き

頂点属性／頂点インデックス／ユニフォーム変数を格納するバッファオブジェクトの基底クラス.

`gg.h` の 5794 行目に定義があります。

9.3.2 構築子と解体子

9.3.2.1 GgBuffer() [1/3]

```
template<typename T>
gg::GgBuffer< T >::GgBuffer (
    GLenum target,
    const T * data,
    GLsizei stride,
    GLsizei count,
    GLenum usage) [inline]
```

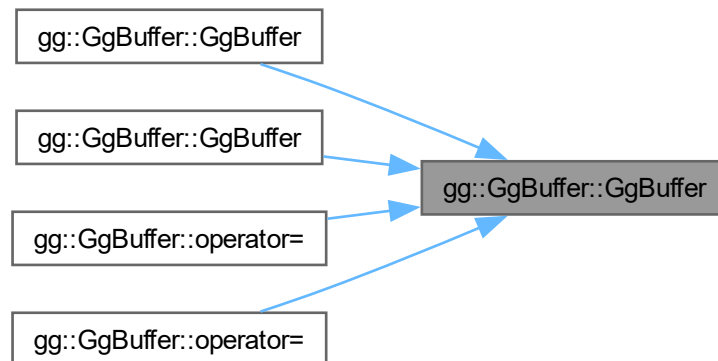
コンストラクタ.

引数

<i>target</i>	バッファオブジェクトのターゲット.
<i>data</i>	データが格納されている領域の先頭のポインタ (nullptr ならデータを転送しない).
<i>count</i>	データの数.
<i>stride</i>	データの間隔.
<i>usage</i>	バッファオブジェクトの使い方.

`gg.h` の 5819 行目に定義があります。

被呼び出し関係図:



9.3.2.2 GgBuffer() [2/3]

```
template<typename T>
gg::GgBuffer< T >::GgBuffer (
    const GgBuffer< T > & buffer) [delete]
コピーコンストラクタは使用しない.
```

引数

<i>buffer</i>	コピー元のバッファ.
---------------	------------

呼び出し関係図:



9.3.2.3 GgBuffer() [3/3]

```
template<typename T>
gg::GgBuffer< T >::GgBuffer (
    GgBuffer< T > && buffer) [default]
ムーブコンストラクタ.
```

引数

<i>buffer</i>	ムーブ元のバッファ.
---------------	------------

呼び出し関係図:



9.3.2.4 ~GgBuffer()

```
template<typename T>
virtual gg::GgBuffer< T >::~~GgBuffer () [inline], [virtual]
デストラクタ.
```

gg.h の 5853 行目に定義があります。

9.3.3 関数詳解

9.3.3.1 bind()

```
template<typename T>
void gg::GgBuffer< T >::bind () const [inline]
バッファオブジェクトを結合する.
```

gg.h の 5919 行目に定義があります。

9.3.3.2 copy()

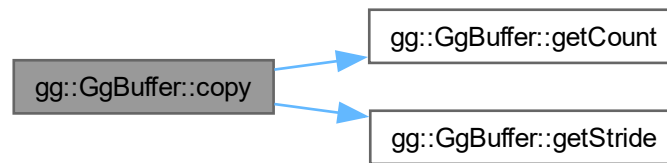
```
template<typename T>
void gg::GgBuffer< T >::copy (
    GLuint src_buffer,
    GLint src_first = 0,
    GLint dst_first = 0,
    GLsizei count = 0) const [inline]
別のバッファオブジェクトからデータを複写する.
```

引数

<i>src_buffer</i>	複写元のバッファオブジェクト名.
<i>src_first</i>	複写元 (<i>src_buffer</i>) の先頭のデータの位置.
<i>dst_first</i>	複写先 (getBuffer()) の先頭のデータの位置.
<i>count</i>	複写するデータの数 (0 ならバッファオブジェクト全体).

gg.h の 6024 行目に定義があります。

呼び出し関係図:



9.3.3.3 `getBuffer()`

```
template<typename T>
const GLuint & gg::GgBuffer< T >::getBuffer () const [inline]
```

バッファオブジェクト名を取り出す。

戻り値

このバッファオブジェクト名。

[gg.h](#) の 5911 行目に定義があります。

9.3.3.4 `getCount()`

```
template<typename T>
const GLsizei & gg::GgBuffer< T >::getCount () const [inline]
```

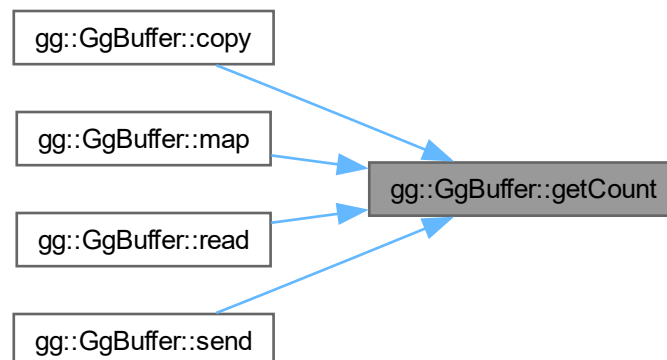
バッファオブジェクトが保持するデータの数を取り出す。

戻り値

このバッファオブジェクトが保持するデータの数。

[gg.h](#) の 5901 行目に定義があります。

被呼び出し関係図:



9.3.3.5 getStride()

```
template<typename T>
```

```
GLsizeiptr gg::GgBuffer< T >::getStride () const [inline]
```

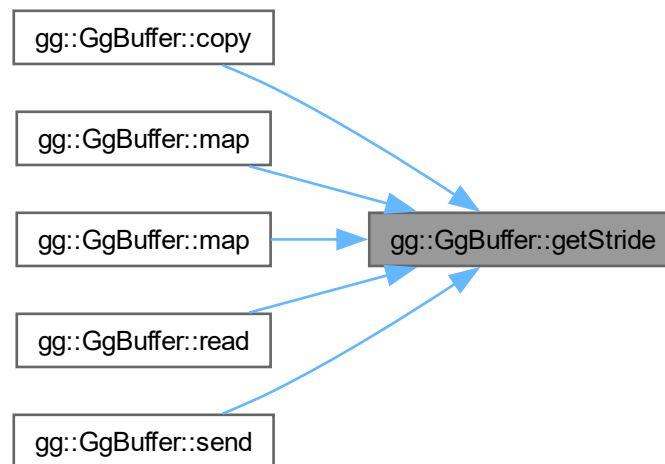
バッファオブジェクトのアライメントを考慮したデータの間隔を取り出す。

戻り値

このバッファオブジェクトのデータの間隔。

gg.h の 5891 行目に定義があります。

被呼び出し関係図:



9.3.3.6 getTarget()

```
template<typename T>
```

```
const GLuint & gg::GgBuffer< T >::getTarget () const [inline]
```

バッファオブジェクトのターゲットを取り出す。

戻り値

このバッファオブジェクトのターゲット。

gg.h の 5881 行目に定義があります。

9.3.3.7 map() [1/2]

```
template<typename T>
```

```
void * gg::GgBuffer< T >::map () const [inline]
```

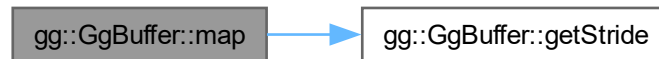
バッファオブジェクトをマップする。

戻り値

マップしたメモリの先頭のポインタ.

gg.h の 5937 行目に定義があります。

呼び出し関係図:



9.3.3.8 map() [2/2]

```
template<typename T>
void * gg::GgBuffer< T >::map (
    GLint first,
    GLsizei count) const [inline]
```

バッファオブジェクトの指定した範囲をマップする.

引数

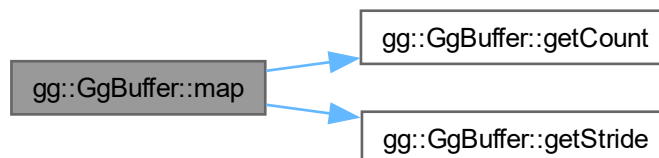
<i>first</i>	マップする範囲のバッファオブジェクトの先頭からの位置.
<i>count</i>	マップするデータの数 (0 ならバッファオブジェクト全体).

戻り値

マップしたメモリの先頭のポインタ.

gg.h の 5954 行目に定義があります。

呼び出し関係図:



9.3.3.9 operator=() [1/2]

```
template<typename T>
GgBuffer< T > & gg::GgBuffer< T >::operator= (
    const GgBuffer< T > & buffer) [delete]
```

代入演算子は使用しない.

引数

<i>buffer</i>	代入元のバッファ.
---------------	-----------

戻り値

代入後のこのバッファの参照.

呼び出し関係図:



9.3.3.10 operator=() [2/2]

```
template<typename T>
GgBuffer< T > & gg::GgBuffer< T >::operator= (
    GgBuffer< T > && buffer) [default]
ムーブ代入演算子.
```

引数

<i>buffer</i>	ムーブ代入元のバッファ.
---------------	--------------

戻り値

ムーブ代入後のこのバッファの参照.

呼び出し関係図:



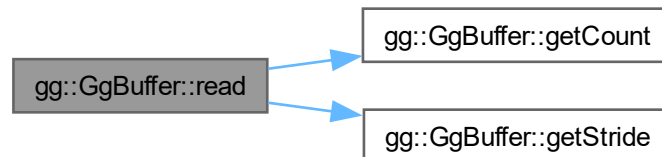
9.3.3.11 read()

```
template<typename T>
void gg::GgBuffer< T >::read (
    T * data,
    GLint first,
    GLsizei count) const [inline]
バッファオブジェクトのデータから抽出する.
```

引数

<i>data</i>	抽出先の領域の先頭のポインタ.
<i>first</i>	抽出元のバッファオブジェクトの取り出すデータの領域の先頭の要素番号.
<i>count</i>	抽出するデータの数 (0 ならバッファオブジェクト全体).

gg.h の 5997 行目に定義があります。
呼び出し関係図:



9.3.3.12 send()

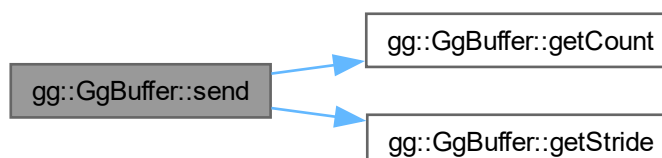
```
template<typename T>
void gg::GgBuffer< T >::send (
    const T * data,
    GLint first,
    GLsizei count) const [inline]
```

すでに確保したバッファオブジェクトにデータを転送する.

引数

<i>data</i>	転送元のデータが格納されている領域の先頭のポインタ.
<i>first</i>	転送先のバッファオブジェクトの先頭の要素番号.
<i>count</i>	転送するデータの数 (0 ならバッファオブジェクト全体).

gg.h の 5979 行目に定義があります。
呼び出し関係図:



9.3.3.13 unbind()

```
template<typename T>
void gg::GgBuffer< T >::unbind () const [inline]
バッファオブジェクトを解放する.
gg.h の 5927 行目に定義があります。
```

9.3.3.14 unmap()

```
template<typename T>
void gg::GgBuffer< T >::unmap () const [inline]
バッファオブジェクトをアンマップする.
gg.h の 5967 行目に定義があります.
このクラス詳解は次のファイルから抽出されました:
```

- [gg.h](#)

9.4 gg::GgColorTexture クラス

```
#include <gg.h>
```

公開メンバ関数

- [GgColorTexture](#) ()
- [GgColorTexture](#) (const GLvoid *image, GLsizei width, GLsizei height, GLenum format=GL_RGB, GLenum type=GL_UNSIGNED_BYTE, GLenum internal=GL_RGB, GLenum wrap=GL_CLAMP_TO_EDGE, bool swizzle=true)
- [GgColorTexture](#) (const std::string &name, GLenum internal=0, GLenum wrap=GL_CLAMP_TO_EDGE)
- virtual [~GgColorTexture](#) ()
- void [load](#) (const GLvoid *image, GLsizei width, GLsizei height, GLenum format=GL_RGB, GLenum type=GL_UNSIGNED_BYTE, GLenum internal=GL_RGB, GLenum wrap=GL_CLAMP_TO_EDGE, bool swizzle=true)
- void [load](#) (const std::string &name, GLenum internal=0, GLenum wrap=GL_CLAMP_TO_EDGE)
- [operator bool](#) () const noexcept
- bool [operator!](#) () const noexcept
- const [GgTexture](#) * [get](#) () const
- GLuint [getTexture](#) () const
- GLsizei [getWidth](#) () const
- GLsizei [getHeight](#) () const
- void [getSize](#) (GLsizei *size) const
- const GLsizei * [getSize](#) () const
- void [bind](#) () const
- void [unbind](#) () const

9.4.1 詳解

カラーマップ.

覚え書き

カラー画像を読み込んでテクスチャを作成する.

[gg.h](#) の 5381 行目に定義があります。

9.4.2 構築子と解体子

9.4.2.1 GgColorTexture() [1/3]

```
gg::GgColorTexture::GgColorTexture () [inline]
コンストラクタ.
gg.h の 5391 行目に定義があります。
```

9.4.2.2 GgColorTexture() [2/3]

```
gg::GgColorTexture::GgColorTexture (
    const GLvoid * image,
    GLsizei width,
    GLsizei height,
    GLenum format = GL_RGB,
    GLenum type = GL_UNSIGNED_BYTE,
    GLenum internal = GL_RGB,
    GLenum wrap = GL_CLAMP_TO_EDGE,
    bool swizzle = true) [inline]
```

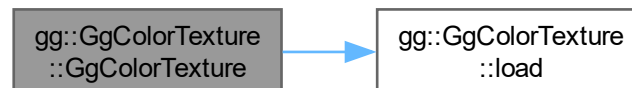
メモリ上のデータからカラーのテクスチャを作成するコンストラクタ。

引数

<i>image</i>	テクスチャとして用いる画像データ, nullptr ならデータを読み込まない.
<i>width</i>	読み込む画像の横の画素数.
<i>height</i>	読み込む画像の縦の画素数.
<i>format</i>	読み込む画像のフォーマット.
<i>type</i>	読み込む画像のデータ型.
<i>internal</i>	テクスチャの内部フォーマット.
<i>wrap</i>	テクスチャのラッピングモード.
<i>swizzle</i>	true ならテクスチャの赤と青を入れ替える, デフォルトは true.

gg.h の 5407 行目に定義があります。

呼び出し関係図:



9.4.2.3 GgColorTexture() [3/3]

```
gg::GgColorTexture::GgColorTexture (
    const std::string & name,
    GLenum internal = 0,
    GLenum wrap = GL_CLAMP_TO_EDGE) [inline]
```

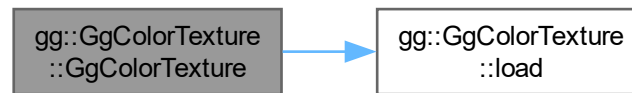
TGA フォーマットの画像ファイルを読み込んでカラーのテクスチャを作成するコンストラクタ。

引数

<i>name</i>	読み込むファイル名.
<i>internal</i>	glTexImage2D() に指定するテクスチャの内部フォーマット, 0 なら外部フォーマットに合わせる.

<i>wrap</i>	テクスチャのラッピングモード, <code>GL_TEXTURE_WRAP_S</code> および <code>GL_TEXTURE_WRAP_T</code> に設定する値.
-------------	---

[gg.h](#) の 5428 行目に定義があります。
呼び出し関係図:



9.4.2.4 ~GgColorTexture()

```
virtual gg::GgColorTexture::~~GgColorTexture () [inline], [virtual]
```

デストラクタ.
[gg.h](#) の 5440 行目に定義があります。

9.4.3 関数詳解

9.4.3.1 bind()

```
void gg::GgColorTexture::bind () const [inline]
```

テクスチャの使用開始 (このテクスチャを使用する際に呼び出す).
[gg.h](#) の 5567 行目に定義があります。

9.4.3.2 get()

```
const GgTexture * gg::GgColorTexture::get () const [inline]
```

テクスチャオブジェクトを取り出す。
戻り値

[GgTexture](#) オブジェクトのポインタ.
[gg.h](#) の 5509 行目に定義があります。

9.4.3.3 getHeight()

```
GLsizei gg::GgColorTexture::getHeight () const [inline]
```

使用しているテクスチャの縦の画素数を取り出す。
戻り値

テクスチャの縦の画素数.
[gg.h](#) の 5539 行目に定義があります。

9.4.3.4 getSize() [1/2]

```
const GLsizei * gg::GgColorTexture::getSize () const [inline]
```

使用しているテクスチャのサイズを取り出す。
戻り値

テクスチャのサイズを格納した配列へのポインタ.
[gg.h](#) の 5559 行目に定義があります。

9.4.3.5 getSize() [2/2]

```
void gg::GgColorTexture::getSize (
    GLsizei * size) const [inline]
```

使用しているテクスチャのサイズを取り出す。

引数

<i>size</i>	テクスチャのサイズを格納する GLsizei 型の 2 要素の配列変数.
-------------	--------------------------------------

gg.h の 5549 行目に定義があります。

9.4.3.6 getTexture()

```
GLuint gg::GgColorTexture::getTexture () const [inline]
```

使用しているテクスチャのテクスチャ名を得る。

戻り値

テクスチャ名.

gg.h の 5519 行目に定義があります。

9.4.3.7 getWidth()

```
GLsizei gg::GgColorTexture::getWidth () const [inline]
```

使用しているテクスチャの横の画素数を取り出す。

戻り値

テクスチャの横の画素数.

gg.h の 5529 行目に定義があります。

9.4.3.8 load() [1/2]

```
void gg::GgColorTexture::load (
    const GLvoid * image,
    GLsizei width,
    GLsizei height,
    GLenum format = GL_RGB,
    GLenum type = GL_UNSIGNED_BYTE,
    GLenum internal = GL_RGB,
    GLenum wrap = GL_CLAMP_TO_EDGE,
    bool swizzle = true) [inline]
```

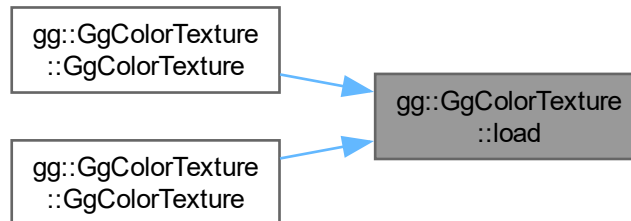
メモリ上のデータを読み込んでテクスチャを作成する。

引数

<i>image</i>	テクスチャとして用いる画像データ, nullptr ならデータを読み込まない.
<i>width</i>	テクスチャの横の画素数.
<i>height</i>	テクスチャの縦の画素数.
<i>format</i>	読み込む画像のフォーマット.
<i>type</i>	読み込む画像のデータ型.
<i>internal</i>	glTexImage2D() に指定するテクスチャの内部フォーマット.
<i>wrap</i>	テクスチャのラッピングモード (GL_CLAMP_TO_EDGE, GL_CLAMP_TO_BORDER, GL_REPEAT, GL_MIRRORED_REPEAT).

swizzle	true ならテクスチャの赤と青を入れ替える, デフォルトは true.
----------------	--------------------------------------

gg.h の 5456 行目に定義があります。
被呼び出し関係図:



9.4.3.9 load() [2/2]

```

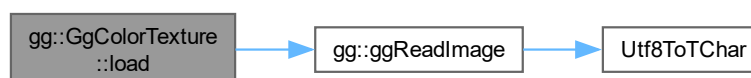
void gg::GgColorTexture::load (
    const std::string & name,
    GLenum internal = 0,
    GLenum wrap = GL_CLAMP_TO_EDGE)
  
```

TGA フォーマットの画像ファイルを読み込んでカラーのテクスチャを作成する.

引数

<i>name</i>	読み込むファイル名.
<i>internal</i>	glTexImage2D() に指定するテクスチャの内部フォーマット, 0 ならファイルの画像フォーマットに合わせる.
<i>wrap</i>	テクスチャのラッピングモード (GL_CLAMP_TO_EDGE, GL_CLAMP_TO_BORDER, GL_REPEAT, GL_MIRRORED_REPEAT).

gg.cpp の 4044 行目に定義があります。
呼び出し関係図:



9.4.3.10 operator bool()

```

gg::GgColorTexture::operator bool () const [inline], [explicit], [noexcept]
  
```

オブジェクトが有効かどうか調べる.

戻り値

オブジェクトが有効なら `true`.

[gg.h](#) の 5489 行目に定義があります。

9.4.3.11 operator!()

```
bool gg::GgColorTexture::operator! () const [inline], [noexcept]
```

オブジェクトが有効かどうかの結果を反転する.

戻り値

オブジェクトが有効なら `false`, 無効なら `true`.

[gg.h](#) の 5499 行目に定義があります。

9.4.3.12 unbind()

```
void gg::GgColorTexture::unbind () const [inline]
```

テクスチャの使用終了 (このテクスチャを使用しなくなったら呼び出す).

[gg.h](#) の 5575 行目に定義があります。

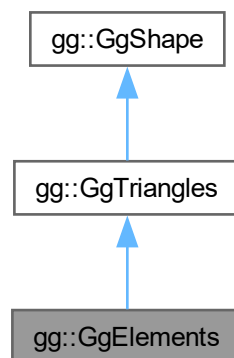
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

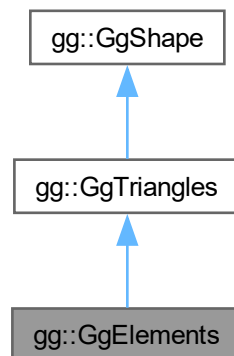
9.5 gg::GgElements クラス

```
#include <gg.h>
```

gg::GgElements の継承関係図



gg::GgElements 連携図



公開メンバ関数

- **GgElements** (GLenum mode=GL_TRIANGLES)
- **GgElements** (const **GgVertex** *vert, GLsizei countv, const GLuint *face, GLsizei countf, GLenum mode=GL_TRIANGLES, GLenum usage=GL_STATIC_DRAW)
- virtual **~GgElements** ()
- const GLsizei & **getIndexCount** () const
- const GLuint & **getIndexBuffer** () const
- void **send** (const **GgVertex** *vert, GLuint firstv, GLsizei countv, const GLuint *face=NULLPTR, GLuint firstf=0, GLsizei countf=0) const
- void **load** (const **GgVertex** *vert, GLsizei countv, const GLuint *face, GLsizei countf, GLenum usage=GL_STATIC_DRAW)
- virtual void **draw** (GLuint first=0, GLsizei count=0) const

基底クラス **gg::GgTriangles** に属する継承公開メンバ関数

- **GgTriangles** (GLenum mode=GL_TRIANGLES)
- **GgTriangles** (const **GgVertex** *vert, GLsizei count, GLenum mode=GL_TRIANGLES, GLenum usage=GL_STATIC_DRAW)
- virtual **~GgTriangles** ()
- const GLsizei & **getCount** () const
- const GLuint & **getBuffer** () const
- void **send** (const **GgVertex** *vert, GLuint first=0, GLsizei count=0) const
- void **load** (const **GgVertex** *vert, GLsizei count, GLenum usage=GL_STATIC_DRAW)

基底クラス **gg::GgShape** に属する継承公開メンバ関数

- **GgShape** (GLenum mode=0)
- virtual **~GgShape** ()
- virtual **operator bool** () const noexcept
- virtual **bool operator!** () const noexcept
- const GLuint & **get** () const
- void **setMode** (GLenum mode)
- const GLenum & **getMode** () const

9.5.1 詳解

三角形で表した形状データ (Elements 形式).
[gg.h](#) の 6821 行目に定義があります。

9.5.2 構築子と解体子

9.5.2.1 GgElements() [1/2]

```
gg::GgElements::GgElements (
    GLenum mode = GL_TRIANGLES) [inline]
コンストラクタ.
```

引数

<i>mode</i>	描画する基本図形の種類.
-------------	--------------

[gg.h](#) の 6834 行目に定義があります。
 呼び出し関係図:



被呼び出し関係図:



9.5.2.2 GgElements() [2/2]

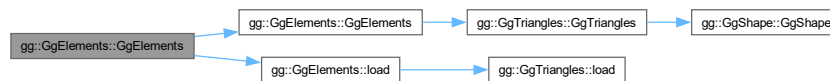
```
gg::GgElements::GgElements (
    const GgVertex * vert,
    GLsizei countv,
    const GLuint * face,
    GLsizei countf,
    GLenum mode = GL_TRIANGLES,
    GLenum usage = GL_STATIC_DRAW) [inline]
コンストラクタ.
```

引数

<i>vert</i>	この図形の頂点属性の配列 (nullptr ならデータを転送しない).
<i>countv</i>	頂点数.
<i>face</i>	三角形の頂点インデックス.
<i>countf</i>	三角形の頂点数.

<i>mode</i>	描画する基本図形の種類.
<i>usage</i>	バッファオブジェクトの使い方.

gg.h の 6849 行目に定義があります。
呼び出し関係図:



9.5.2.3 ~GgElements()

virtual gg::GgElements::~~GgElements () [inline], [virtual]
デストラクタ.

gg.h の 6865 行目に定義があります。

9.5.3 関数詳解

9.5.3.1 draw()

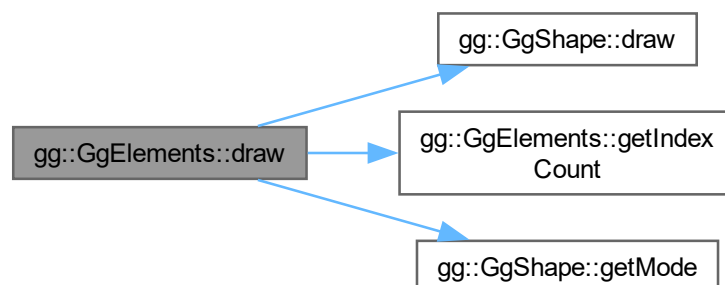
```
void gg::GgElements::draw (
    GLint first = 0,
    GLsizei count = 0) const [virtual]
```

インデックスを使った三角形の描画.

引数

<i>first</i>	描画を開始する最初の三角形番号.
<i>count</i>	描画する三角形数, 0 なら全部の三角形を描く.

gg::GgTriangles を再実装しています。
gg.cpp の 5188 行目に定義があります。
呼び出し関係図:



9.5.3.2 getIndexBuffer()

`const GLuint & gg::GgElements::getIndexBuffer () const [inline]`
 三角形の頂点インデックスデータを格納した頂点バッファオブジェクト名を取り出す。

戻り値

この図形の三角形の頂点インデックスデータを格納した頂点バッファオブジェクト名。

gg.h の 6883 行目に定義があります。

9.5.3.3 getIndexCount()

`const GLsizei & gg::GgElements::getIndexCount () const [inline]`
 データの数を取り出す。

戻り値

この図形の三角形数。

gg.h の 6873 行目に定義があります。

被呼び出し関係図:



9.5.3.4 load()

`void gg::GgElements::load (`
 `const GgVertex * vert,`
 `GLsizei countv,`
 `const GLuint * face,`
 `GLsizei countf,`
 `GLenum usage = GL_STATIC_DRAW) [inline]`
 バッファオブジェクトを確保して頂点属性と三角形の頂点インデックスデータを格納する。

引数

<i>vert</i>	頂点属性が格納されている領域の先頭のポインタ。
<i>countv</i>	頂点のデータの数 (頂点数)。
<i>face</i>	三角形の頂点インデックスデータ。
<i>countf</i>	三角形の頂点数。
<i>usage</i>	バッファオブジェクトの使い方。

gg.h の 6920 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.5.3.5 send()

```

void gg::GgElements::send (
    const GgVertex * vert,
    GLuint firstv,
    GLsizei countv,
    const GLuint * face = nullptr,
    GLuint firstf = 0,
    GLsizei countf = 0) const [inline]
  
```

既存のバッファオブジェクトに頂点属性と三角形の頂点インデックスデータを転送する。

引数

<i>vert</i>	頂点属性が格納されている領域の先頭のポインタ。
<i>firstv</i>	頂点属性の転送先のバッファオブジェクトの先頭の要素番号。
<i>countv</i>	頂点のデータの数 (頂点数)。
<i>face</i>	三角形の頂点インデックスデータ。
<i>firstf</i>	インデックスの転送先のバッファオブジェクトの先頭の要素番号。
<i>countf</i>	三角形の頂点数。

[gg.h](#) の 6898 行目に定義があります。

呼び出し関係図:



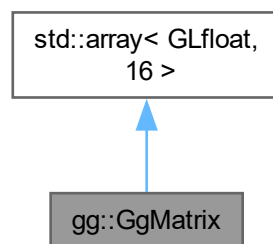
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

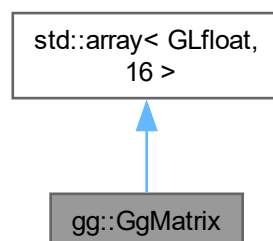
9.6 gg::GgMatrix クラス

```
#include <gg.h>
```

gg::GgMatrix の継承関係図



gg::GgMatrix 連携図



公開メンバ関数

- [GgMatrix](#) ()=default

- constexpr [GgMatrix](#) (GLfloat m00, GLfloat m01, GLfloat m02, GLfloat m03, GLfloat m10, GLfloat m11, GLfloat m12, GLfloat m13, GLfloat m20, GLfloat m21, GLfloat m22, GLfloat m23, GLfloat m30, GLfloat m31, GLfloat m32, GLfloat m33)
- constexpr [GgMatrix](#) (const GLfloat *a)
- constexpr [GgMatrix](#) (GLfloat c)
- [GgMatrix](#) (const [GgMatrix](#) &m)=default
- [GgMatrix](#) ([GgMatrix](#) &&m)=default
- [~GgMatrix](#) ()=default
- [GgMatrix](#) & [operator=](#) (const [GgMatrix](#) &m)=default
- [GgMatrix](#) & [operator=](#) ([GgMatrix](#) &&m)=default
- [GgMatrix](#) & [operator=](#) (const GLfloat *a)
- [GgMatrix](#) [operator+](#) (const GLfloat *a) const
- [GgMatrix](#) [operator+](#) (const [GgMatrix](#) &m) const
- [GgMatrix](#) & [operator+=](#) (const GLfloat *a)
- [GgMatrix](#) & [operator+=](#) (const [GgMatrix](#) &m)
- [GgMatrix](#) [operator-](#) (const GLfloat *a) const
- [GgMatrix](#) [operator-](#) (const [GgMatrix](#) &m) const
- [GgMatrix](#) & [operator-=](#) (const GLfloat *a)
- [GgMatrix](#) & [operator-=](#) (const [GgMatrix](#) &m)
- [GgMatrix](#) [operator*](#) (const GLfloat *a) const
- [GgMatrix](#) [operator*](#) (const [GgMatrix](#) &m) const
- [GgMatrix](#) & [operator*+=](#) (const GLfloat *a)
- [GgMatrix](#) & [operator*+=](#) (const [GgMatrix](#) &m)
- [GgMatrix](#) [operator/](#) (const GLfloat *a) const
- [GgMatrix](#) [operator/](#) (const [GgMatrix](#) &m) const
- [GgMatrix](#) & [operator/=](#) (const GLfloat *a)
- [GgMatrix](#) & [operator/=](#) (const [GgMatrix](#) &m)
- [GgMatrix](#) & [loadIdentity](#) ()
- [GgMatrix](#) & [loadTranslate](#) (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f)
- [GgMatrix](#) & [loadTranslate](#) (const GLfloat *t)
- [GgMatrix](#) & [loadTranslate](#) (const [GgVector](#) &t)
- [GgMatrix](#) & [loadScale](#) (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f)
- [GgMatrix](#) & [loadScale](#) (const GLfloat *s)
- [GgMatrix](#) & [loadScale](#) (const [GgVector](#) &s)
- [GgMatrix](#) & [loadRotateX](#) (GLfloat a)
- [GgMatrix](#) & [loadRotateY](#) (GLfloat a)
- [GgMatrix](#) & [loadRotateZ](#) (GLfloat a)
- [GgMatrix](#) & [loadRotate](#) (GLfloat x, GLfloat y, GLfloat z, GLfloat a)
- [GgMatrix](#) & [loadRotate](#) (const GLfloat *r, GLfloat a)
- [GgMatrix](#) & [loadRotate](#) (const [GgVector](#) &r, GLfloat a)
- [GgMatrix](#) & [loadRotate](#) (const GLfloat *r)
- [GgMatrix](#) & [loadRotate](#) (const [GgVector](#) &r)
- [GgMatrix](#) & [loadLookat](#) (GLfloat ex, GLfloat ey, GLfloat ez, GLfloat tx, GLfloat ty, GLfloat tz, GLfloat ux, GLfloat uy, GLfloat uz)
- [GgMatrix](#) & [loadLookat](#) (const GLfloat *e, const GLfloat *t, const GLfloat *u)
- [GgMatrix](#) & [loadLookat](#) (const [GgVector](#) &e, const [GgVector](#) &t, const [GgVector](#) &u)
- [GgMatrix](#) & [loadOrthogonal](#) (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar)
- [GgMatrix](#) & [loadFrustum](#) (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar)
- [GgMatrix](#) & [loadPerspective](#) (GLfloat fovy, GLfloat aspect, GLfloat zNear, GLfloat zFar)
- [GgMatrix](#) & [loadTranspose](#) (const GLfloat *a)
- [GgMatrix](#) & [loadTranspose](#) (const [GgMatrix](#) &m)
- [GgMatrix](#) & [loadInvert](#) (const GLfloat *a)
- [GgMatrix](#) & [loadInvert](#) (const [GgMatrix](#) &m)
- [GgMatrix](#) & [loadNormal](#) (const GLfloat *a)

- `GgMatrix & loadNormal` (const `GgMatrix` &m)
- `GgMatrix translate` (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f) const
- `GgMatrix translate` (const GLfloat *t) const
- `GgMatrix translate` (const `GgVector` &t) const
- `GgMatrix scale` (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f) const
- `GgMatrix scale` (const GLfloat *s) const
- `GgMatrix scale` (const `GgVector` &s) const
- `GgMatrix rotateX` (GLfloat a) const
- `GgMatrix rotateY` (GLfloat a) const
- `GgMatrix rotateZ` (GLfloat a) const
- `GgMatrix rotate` (GLfloat x, GLfloat y, GLfloat z, GLfloat a) const
- `GgMatrix rotate` (const GLfloat *r, GLfloat a) const
- `GgMatrix rotate` (const `GgVector` &r, GLfloat a) const
- `GgMatrix rotate` (const GLfloat *r) const
- `GgMatrix rotate` (const `GgVector` &r) const
- `GgMatrix lookout` (GLfloat ex, GLfloat ey, GLfloat ez, GLfloat tx, GLfloat ty, GLfloat tz, GLfloat ux, GLfloat uy, GLfloat uz) const
- `GgMatrix lookout` (const GLfloat *e, const GLfloat *t, const GLfloat *u) const
- `GgMatrix lookout` (const `GgVector` &e, const `GgVector` &t, const `GgVector` &u) const
- `GgMatrix orthogonal` (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar) const
- `GgMatrix frustum` (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar) const
- `GgMatrix perspective` (GLfloat fovy, GLfloat aspect, GLfloat zNear, GLfloat zFar) const
- `GgMatrix transpose` () const
- `GgMatrix invert` () const
- `GgMatrix normal` () const
- void `projection` (GLfloat *c, const GLfloat *v) const
- void `projection` (GLfloat *c, const `GgVector` &v) const
- void `projection` (`GgVector` &c, const GLfloat *v) const
- void `projection` (`GgVector` &c, const `GgVector` &v) const
- `GgVector operator*` (const `GgVector` &v) const
- const GLfloat * `get` () const
- void `get` (GLfloat *a) const

9.6.1 詳解

変換行列.

`gg.h` の 2157 行目に定義があります。

9.6.2 構築子と解体子

9.6.2.1 GgMatrix() [1/6]

```
gg::GgMatrix::GgMatrix () [default]
コンストラクタ.
```

9.6.2.2 GgMatrix() [2/6]

```
gg::GgMatrix::GgMatrix (
    GLfloat m00,
    GLfloat m01,
    GLfloat m02,
    GLfloat m03,
    GLfloat m10,
    GLfloat m11,
    GLfloat m12,
    GLfloat m13,
```

```

GLfloat m20,
GLfloat m21,
GLfloat m22,
GLfloat m23,
GLfloat m30,
GLfloat m31,
GLfloat m32,
GLfloat m33) [inline], [constexpr]

```

コンストラクタ.

引数

<i>m00</i>	GLfloat 型の値.
<i>m01</i>	GLfloat 型の値.
<i>m02</i>	GLfloat 型の値.
<i>m03</i>	GLfloat 型の値.
<i>m10</i>	GLfloat 型の値.
<i>m11</i>	GLfloat 型の値.
<i>m12</i>	GLfloat 型の値.
<i>m13</i>	GLfloat 型の値.
<i>m20</i>	GLfloat 型の値.
<i>m21</i>	GLfloat 型の値.
<i>m22</i>	GLfloat 型の値.
<i>m23</i>	GLfloat 型の値.
<i>m30</i>	GLfloat 型の値.
<i>m31</i>	GLfloat 型の値.
<i>m32</i>	GLfloat 型の値.
<i>m33</i>	GLfloat 型の値.

[gg.h](#) の 2192 行目に定義があります。

9.6.2.3 GgMatrix() [3/6]

```

gg::GgMatrix::GgMatrix (
    const GLfloat * a) [inline], [constexpr]

```

コンストラクタ.

引数

<i>a</i>	GLfloat 型の 16 要素の配列変数.
----------	------------------------

[gg.h](#) の 2207 行目に定義があります。

呼び出し関係図:



9.6.2.4 GgMatrix() [4/6]

```
gg::GgMatrix::GgMatrix (  
    GLfloat c) [inline], [constexpr]  
コンストラクタ.
```

引数

c	GLfloat 型の値.
---	--------------

[gg.h](#) の 2217 行目に定義があります。
呼び出し関係図:



9.6.2.5 GgMatrix() [5/6]

```
gg::GgMatrix::GgMatrix (  
    const GgMatrix & m) [default]  
コピーコンストラクタ.
```

引数

m	コピー元の変換行列.
---	------------

呼び出し関係図:



9.6.2.6 GgMatrix() [6/6]

```
gg::GgMatrix::GgMatrix (
    GgMatrix && m) [default]
```

ムーブコンストラクタ.

引数

<i>m</i>	ムーブ元の変換行列.
----------	------------

呼び出し関係図:



9.6.2.7 ~GgMatrix()

```
gg::GgMatrix::~~GgMatrix () [default]
```

デストラクタ.

9.6.3 関数詳解

9.6.3.1 frustum()

```
GgMatrix gg::GgMatrix::frustum (
    GLfloat left,
    GLfloat right,
    GLfloat bottom,
    GLfloat top,
    GLfloat zNear,
    GLfloat zFar) const [inline]
```

透視投影変換を乗じた結果を返す.

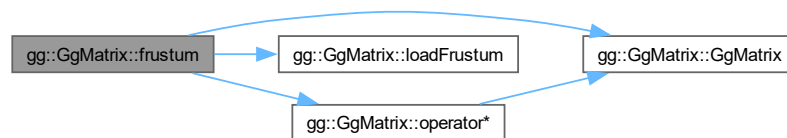
引数

<i>left</i>	ウィンドウの左端の位置.
<i>right</i>	ウィンドウの右端の位置.
<i>bottom</i>	ウィンドウの下端の位置.
<i>top</i>	ウィンドウの上端の位置.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

透視投影変換行列を乗じた変換行列.

gg.h の 3000 行目に定義があります。
呼び出し関係図:



9.6.3.2 get() [1/2]

```
const GLfloat * gg::GgMatrix::get () const [inline]
```

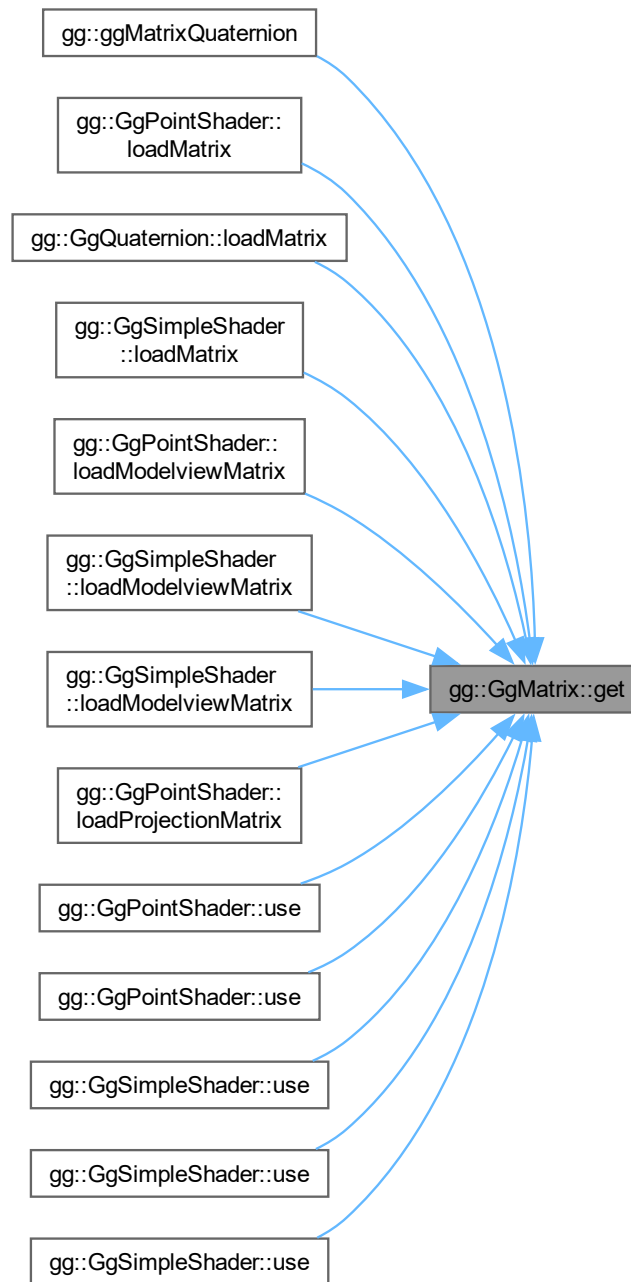
変換行列を取り出す.

戻り値

変換行列を格納した GLfloat 型の 16 要素の配列変数.

gg.h の 3123 行目に定義があります。

被呼び出し関係図:



9.6.3.3 get() [2/2]

```
void gg::GgMatrix::get (
    GLfloat * a) const [inline]
```

変換行列を取り出す。

引数

a	変換行列を格納する GLfloat 型の 16 要素の配列変数.
----------	----------------------------------

gg.h の 3133 行目に定義があります。

9.6.3.4 invert()

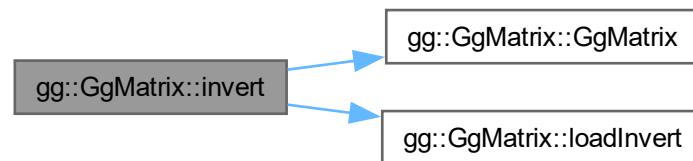
GgMatrix gg::GgMatrix::invert () const [inline]
逆行列を返す.

戻り値

逆行列.

gg.h の 3044 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.6.3.5 loadFrustum()

```

gg::GgMatrix & gg::GgMatrix::loadFrustum (
    GLfloat left,
    GLfloat right,
    GLfloat bottom,
    GLfloat top,
    GLfloat zNear,
    GLfloat zFar)
  
```

透視投影変換行列を格納する.

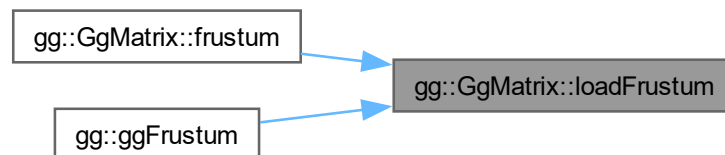
引数

<i>left</i>	ウィンドウの左端の位置.
<i>right</i>	ウィンドウの右端の位置.
<i>bottom</i>	ウィンドウの下端の位置.
<i>top</i>	ウィンドウの上端の位置.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

設定した透視投影変換行列.

`gg.cpp` の 3123 行目に定義があります。
被呼び出し関係図:



9.6.3.6 loadIdentity()

`gg::GgMatrix & gg::GgMatrix::loadIdentity ()`
単位行列を格納する.

`gg.cpp` の 2734 行目に定義があります。
被呼び出し関係図:



9.6.3.7 loadInvert() [1/2]

`GgMatrix & gg::GgMatrix::loadInvert (`
 `const GgMatrix & m) [inline]`

逆行列を格納する.

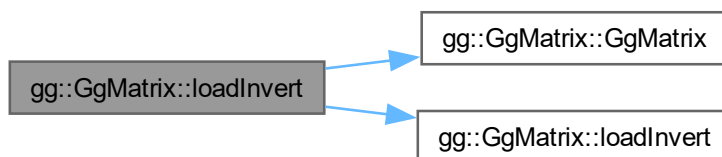
引数

m	GgMatrix 型の変換行列.
-----	------------------

戻り値

設定した m の逆行列.

gg.h の 2723 行目に定義があります。
呼び出し関係図:



9.6.3.8 loadInvert() [2/2]

```
gg::GgMatrix & gg::GgMatrix::loadInvert (
    const GLfloat * a)
```

逆行列を格納する.

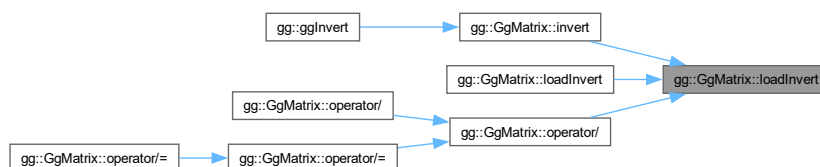
引数

a	GLfloat 型の 16 要素の変換行列.
-----	------------------------

戻り値

設定した a の逆行列.

gg.cpp の 2909 行目に定義があります。
被呼び出し関係図:



9.6.3.9 loadLookat() [1/3]

```
GgMatrix & gg::GgMatrix::loadLookat (
    const GgVector & e,
```

```
const GgVector & t,  
const GgVector & u) [inline]
```

ビュー変換行列を格納する.

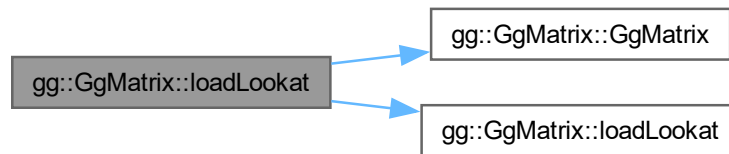
引数

<i>e</i>	視点の位置の GgVector 型の変数.
<i>t</i>	目標点の位置の GgVector 型の変数.
<i>u</i>	上方向のベクトルの GgVector 型の変数.

戻り値

設定したビュー変換行列.

[gg.h](#) の 2643 行目に定義があります。
呼び出し関係図:



9.6.3.10 loadLookat() [2/3]

```
GgMatrix & gg::GgMatrix::loadLookat (
    const GLfloat * e,
    const GLfloat * t,
    const GLfloat * u) [inline]
```

ビュー変換行列を格納する.

引数

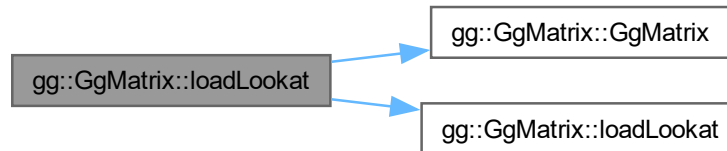
<i>e</i>	視点の位置の配列変数.
<i>t</i>	目標点の位置の配列変数.
<i>u</i>	上方向のベクトルの配列変数.

戻り値

設定したビュー変換行列.

[gg.h](#) の 2630 行目に定義があります。

呼び出し関係図:



9.6.3.11 loadLookat() [3/3]

```

gg::GgMatrix & gg::GgMatrix::loadLookat (
    GLfloat ex,
    GLfloat ey,
    GLfloat ez,
    GLfloat tx,
    GLfloat ty,
    GLfloat tz,
    GLfloat ux,
    GLfloat uy,
    GLfloat uz)
  
```

ビュー変換行列を格納する.

引数

<i>ex</i>	視点の位置の x 座標値.
<i>ey</i>	視点の位置の y 座標値.
<i>ez</i>	視点の位置の z 座標値.
<i>tx</i>	目標点の位置の x 座標値.
<i>ty</i>	目標点の位置の y 座標値.
<i>tz</i>	目標点の位置の z 座標値.
<i>ux</i>	上方向のベクトルの x 成分.
<i>uy</i>	上方向のベクトルの y 成分.
<i>uz</i>	上方向のベクトルの z 成分.

9.6.3.13 loadNormal() [2/2]

```
gg::GgMatrix & gg::GgMatrix::loadNormal (
    const GLfloat * a)
```

法線変換行列を格納する。

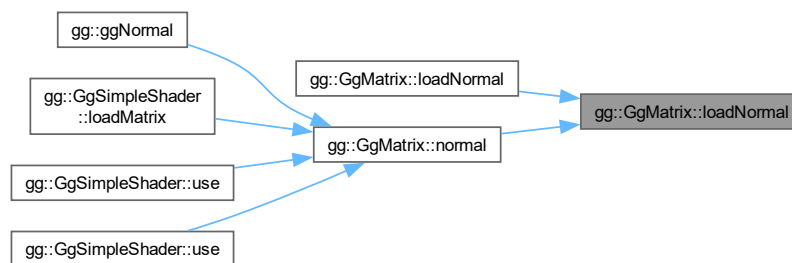
引数

<i>a</i>	GLfloat 型の 16 要素の変換行列.
----------	------------------------

戻り値

設定した *a* の法線変換行列.

[gg.cpp](#) の 2992 行目に定義があります。
被呼び出し関係図:



9.6.3.14 loadOrthogonal()

```
gg::GgMatrix & gg::GgMatrix::loadOrthogonal (
    GLfloat left,
    GLfloat right,
    GLfloat bottom,
    GLfloat top,
    GLfloat zNear,
    GLfloat zFar)
```

直交投影変換行列を格納する。

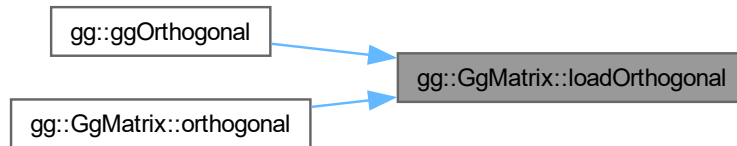
引数

<i>left</i>	ウィンドウの左端の位置.
<i>right</i>	ウィンドウの右端の位置.
<i>bottom</i>	ウィンドウの下端の位置.
<i>top</i>	ウィンドウの上端の位置.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

設定した直交投影変換行列.

gg.cpp の 3082 行目に定義があります。
被呼び出し関係図:



9.6.3.15 loadPerspective()

```

gg::GgMatrix & gg::GgMatrix::loadPerspective (
    GLfloat fovy,
    GLfloat aspect,
    GLfloat zNear,
    GLfloat zFar)
  
```

画角を指定して透視投影変換行列を格納する.

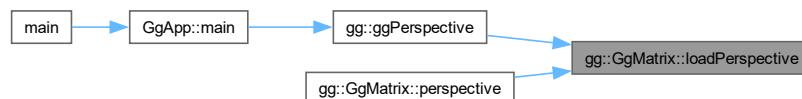
引数

<i>fovy</i>	y 方向の画角.
<i>aspect</i>	縦横比.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

設定した透視投影変換行列.

gg.cpp の 3164 行目に定義があります。
被呼び出し関係図:



9.6.3.16 loadRotate() [1/5]

```

GgMatrix & gg::GgMatrix::loadRotate (
    const GgVector & r) [inline]
  
```

r 方向のベクトルを軸とする回転の変換行列を格納する.

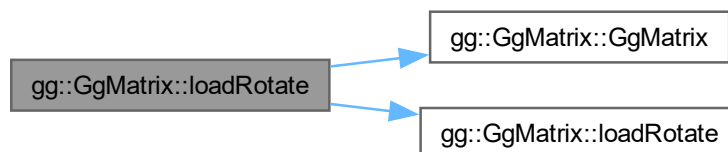
引数

<i>r</i>	回転軸の方向ベクトルと回転角を格納した GgVector 型のベクトル, 第 4 要素を回転角に用いる.
----------	---

戻り値

設定した変換行列.

gg.h の 2599 行目に定義があります。
呼び出し関係図:



9.6.3.17 loadRotate() [2/5]

```
GgMatrix & gg::GgMatrix::loadRotate (
    const GgVector & r,
    GLfloat a) [inline]
```

r 方向のベクトルを軸とする回転の変換行列を格納する.

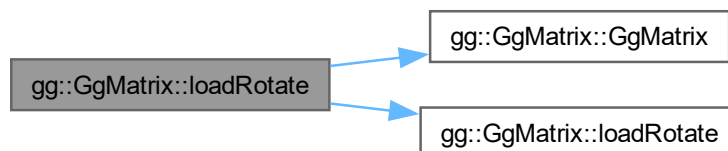
引数

<i>r</i>	回転軸の方向ベクトルを格納した GgVector 型のベクトル, 第 4 要素は無視する.
<i>a</i>	回転角.

戻り値

設定した変換行列.

gg.h の 2577 行目に定義があります。
呼び出し関係図:



9.6.3.18 loadRotate() [3/5]

```
GgMatrix & gg::GgMatrix::loadRotate (
    const GLfloat * r) [inline]
```

`r` 方向のベクトルを軸とする回転の変換行列を格納する.

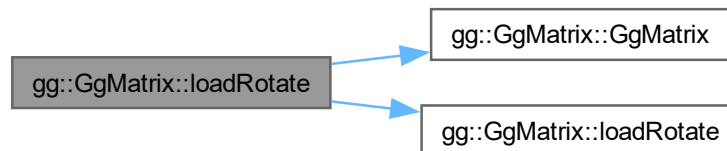
引数

<code>r</code>	回転軸の方向ベクトルと回転角を格納した GLfloat 型の 4 要素の配列変数, 第 4 要素を回転角に用いる.
----------------	---

戻り値

設定した変換行列.

[gg.h](#) の 2588 行目に定義があります。
呼び出し関係図:



9.6.3.19 loadRotate() [4/5]

```
GgMatrix & gg::GgMatrix::loadRotate (
    const GLfloat * r,
    GLfloat a) [inline]
```

`r` 方向のベクトルを軸とする回転の変換行列を格納する.

引数

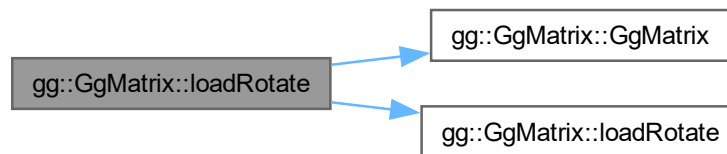
<code>r</code>	回転軸の方向ベクトルを格納した GLfloat 型の 3 要素の配列変数 (x, y, z).
<code>a</code>	回転角.

戻り値

設定した変換行列.

gg.h の 2565 行目に定義があります。

呼び出し関係図:



9.6.3.20 loadRotate() [5/5]

```

gg::GgMatrix & gg::GgMatrix::loadRotate (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat a)
  
```

(x, y, z) 方向のベクトルを軸とする回転の変換行列を格納する.

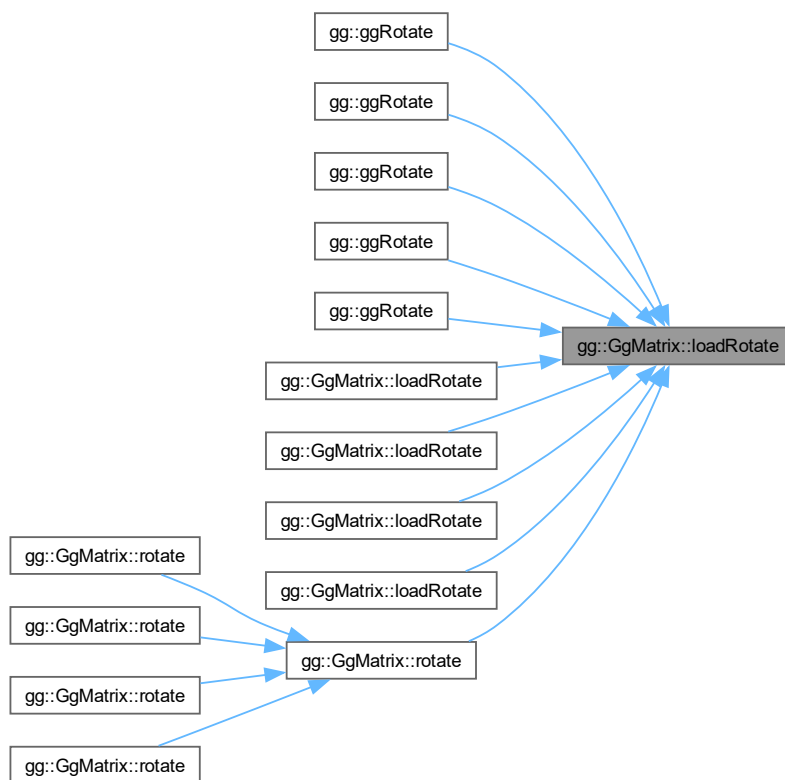
引数

<i>x</i>	回転軸の x 成分.
<i>y</i>	回転軸の y 成分.
<i>z</i>	回転軸の z 成分.
<i>a</i>	回転角.

戻り値

設定した変換行列.

`gg.cpp` の 2839 行目に定義があります。
被呼び出し関係図:



9.6.3.21 loadRotateX()

```
gg::GgMatrix & gg::GgMatrix::loadRotateX (
    GLfloat a)
```

x 軸中心の回転の変換行列を格納する.

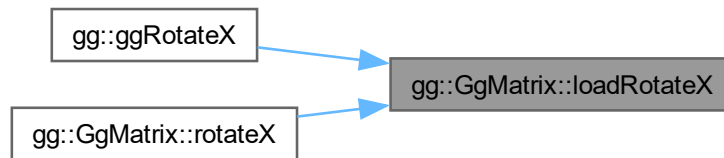
引数

<code>a</code>	回転角.
----------------	------

戻り値

設定した変換行列.

gg.cpp の 2782 行目に定義があります。
被呼び出し関係図:



9.6.3.22 loadRotateY()

```
gg::GgMatrix & gg::GgMatrix::loadRotateY (  
    GLfloat a)
```

y 軸中心の回転の変換行列を格納する.

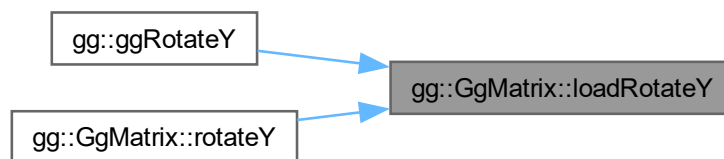
引数

<i>a</i>	回転角.
----------	------

戻り値

設定した変換行列.

gg.cpp の 2801 行目に定義があります。
被呼び出し関係図:



9.6.3.23 loadRotateZ()

```
gg::GgMatrix & gg::GgMatrix::loadRotateZ (  
    GLfloat a)
```

z 軸中心の回転の変換行列を格納する.

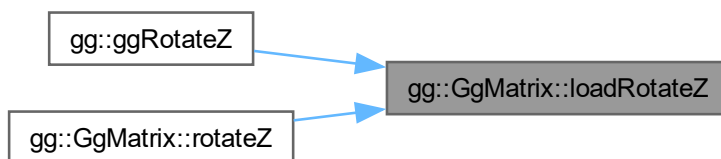
引数

a	回転角.
----------	------

戻り値

設定した変換行列.

[gg.cpp](#) の 2820 行目に定義があります。
被呼び出し関係図:



9.6.3.24 loadScale() [1/3]

```
GgMatrix & gg::GgMatrix::loadScale (
    const GgVector & s) [inline]
```

拡大縮小の変換行列を格納する.

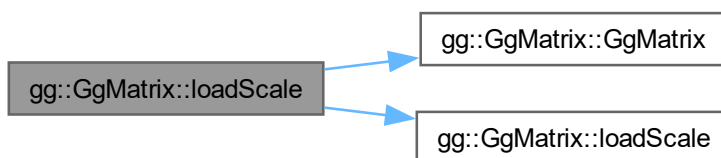
引数

s	拡大率の GgVector 型の変数.
----------	-------------------------------------

戻り値

設定した変換行列.

[gg.h](#) の 2518 行目に定義があります。
呼び出し関係図:



9.6.3.25 loadScale() [2/3]

```
GgMatrix & gg::GgMatrix::loadScale (
    const GLfloat * s) [inline]
```

拡大縮小の変換行列を格納する.

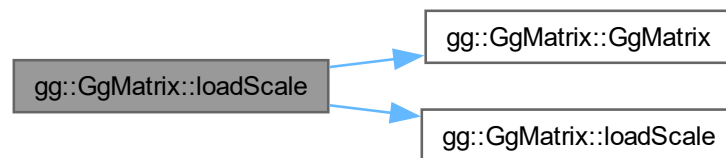
引数

s	拡大率の GLfloat 型の配列 (x, y, z).
----------	------------------------------

戻り値

設定した変換行列.

gg.h の 2507 行目に定義があります。
呼び出し関係図:



9.6.3.26 loadScale() [3/3]

```
gg::GgMatrix & gg::GgMatrix::loadScale (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w = 1.0f)
```

拡大縮小の変換行列を格納する.

引数

x	x 方向の拡大率.
y	y 方向の拡大率.
z	z 方向の拡大率.
w	w 拡大率のスケールファクタ (= 1.0f).

9.6.3.28 loadTranslate() [2/3]

```
GgMatrix & gg::GgMatrix::loadTranslate (
    const GLfloat * t) [inline]
```

平行移動の変換行列を格納する.

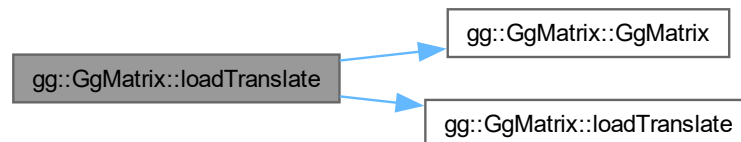
引数

<i>t</i>	移動量の GLfloat 型の配列 (x, y, z).
----------	------------------------------

戻り値

設定した変換行列.

gg.h の 2474 行目に定義があります。
呼び出し関係図:



9.6.3.29 loadTranslate() [3/3]

```
gg::GgMatrix & gg::GgMatrix::loadTranslate (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w = 1.0f)
```

平行移動の変換行列を格納する.

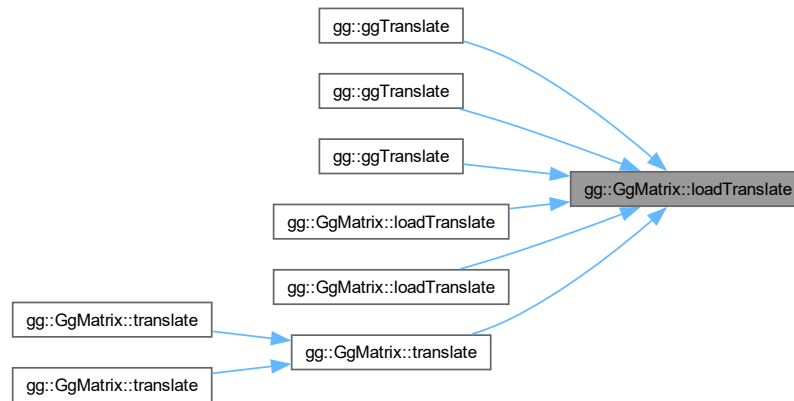
引数

<i>x</i>	<i>x</i> 方向の移動量.
<i>y</i>	<i>y</i> 方向の移動量.
<i>z</i>	<i>z</i> 方向の移動量.
<i>w</i>	<i>w</i> 移動量のスケールファクタ (= 1.0f).

戻り値

設定した変換行列.

[gg.cpp](#) の 2750 行目に定義があります。
被呼び出し関係図:



9.6.3.30 loadTranspose() [1/2]

```
GgMatrix & gg::GgMatrix::loadTranspose (
    const GgMatrix & m) [inline]
```

転置行列を格納する.

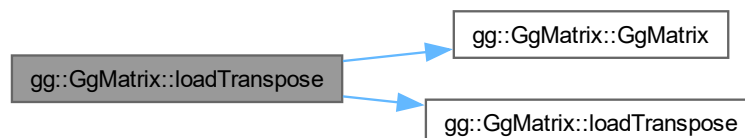
引数

<i>m</i>	GgMatrix 型の変換行列.
----------	----------------------------------

戻り値

設定した *m* の転置行列.

[gg.h](#) の 2704 行目に定義があります。
呼び出し関係図:



9.6.3.31 loadTranspose() [2/2]

```
gg::GgMatrix & gg::GgMatrix::loadTranspose (
    const GLfloat * a)
```

転置行列を格納する.

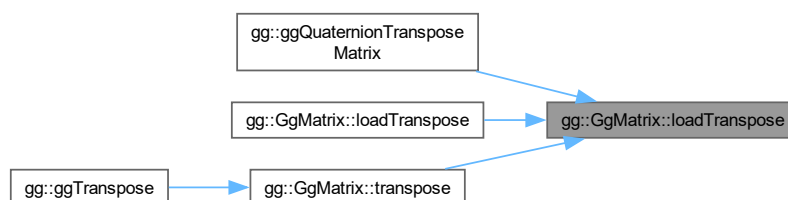
引数

<i>a</i>	GLfloat 型の 16 要素の変換行列.
----------	------------------------

戻り値

設定した *a* の転置行列.

gg.cpp の 2881 行目に定義があります。
被呼び出し関係図:



9.6.3.32 lookat() [1/3]

```
GgMatrix gg::GgMatrix::lookat (
    const GgVector & e,
    const GgVector & t,
    const GgVector & u) const [inline]
```

ビュー変換を乗じた結果を返す.

引数

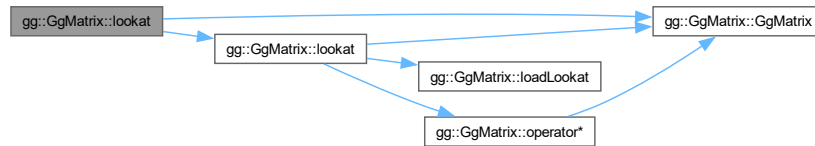
<i>e</i>	視点の位置を格納した GgVector 型の変数.
<i>t</i>	目標点の位置を格納した GgVector 型の変数.
<i>u</i>	上方向のベクトルを格納した GgVector 型の変数.

戻り値

ビュー変換行列を乗じた変換行列.

gg.h の 2963 行目に定義があります。

呼び出し関係図:



9.6.3.33 lookat() [2/3]

```
GgMatrix gg::GgMatrix::lookat (
    const GLfloat * e,
    const GLfloat * t,
    const GLfloat * u) const [inline]
```

ビュー変換を乗じた結果を返す.

引数

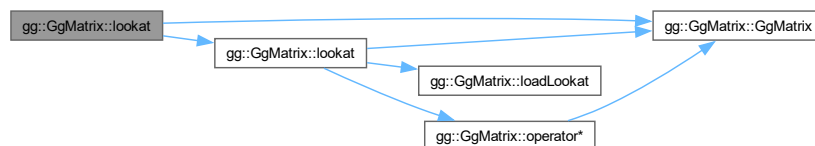
<i>e</i>	視点の位置を格納した GLfloat 型の 3 要素の配列変数.
<i>t</i>	目標点の位置を格納した GLfloat 型の 3 要素の配列変数.
<i>u</i>	上方向のベクトルを格納した GLfloat 型の 3 要素の配列変数.

戻り値

ビュー変換行列を乗じた変換行列.

gg.h の 2950 行目に定義があります。

呼び出し関係図:



9.6.3.34 lookat() [3/3]

```
GgMatrix gg::GgMatrix::lookat (
    GLfloat ex,
    GLfloat ey,
    GLfloat ez,
    GLfloat tx,
    GLfloat ty,
```

```

GLfloat tz,
GLfloat ux,
GLfloat uy,
GLfloat uz) const [inline]

```

ビュー変換を乗じた結果を返す。

引数

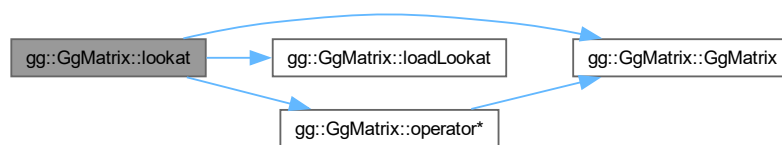
<i>ex</i>	視点の位置の x 座標値.
<i>ey</i>	視点の位置の y 座標値.
<i>ez</i>	視点の位置の z 座標値.
<i>tx</i>	目標点の位置の x 座標値.
<i>ty</i>	目標点の位置の y 座標値.
<i>tz</i>	目標点の位置の z 座標値.
<i>ux</i>	上方向のベクトルの x 成分.
<i>uy</i>	上方向のベクトルの y 成分.
<i>uz</i>	上方向のベクトルの z 成分.

戻り値

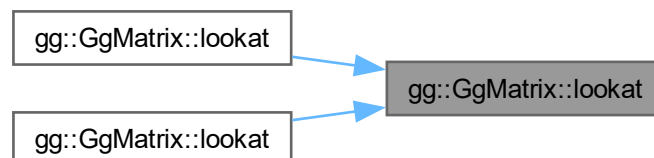
ビュー変換行列を乗じた変換行列。

[gg.h](#) の 2932 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.6.3.35 normal()

```
GgMatrix gg::GgMatrix::normal () const [inline]
```

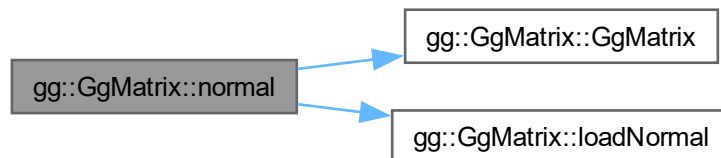
法線変換行列を返す.

戻り値

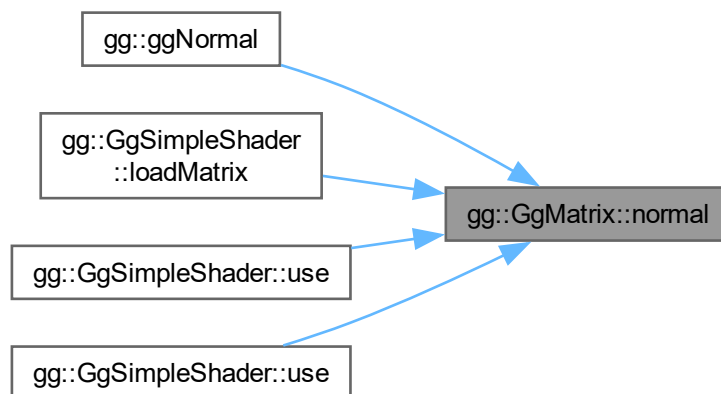
法線変換行列.

gg.h の 3055 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.6.3.36 operator*() [1/3]

```
GgMatrix gg::GgMatrix::operator* (
    const GgMatrix & m) const [inline]
```

変換行列に別の変換行列を乗算した値を返す.

引数

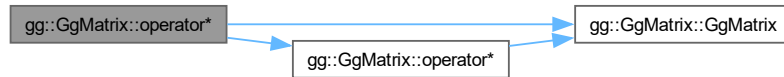
<i>m</i>	<code>GgMatrix</code> 型の変換行列.
----------	-------------------------------

戻り値

変換行列に m を乗じた **GgMatrix** 型の変換行列.

gg.h の 2376 行目に定義があります。

呼び出し関係図:



9.6.3.37 operator*() [2/3]

```
GgVector gg::GgMatrix::operator* (
    const GgVector & v) const [inline]
```

ベクトルに対して投影変換を行う。

引数

v	元のベクトルの GgVector 型の変数.
-----	-------------------------------

戻り値

v 変換結果の **GgVector** 型のベクトル.

gg.h の 3111 行目に定義があります。

9.6.3.38 operator*() [3/3]

```
GgMatrix gg::GgMatrix::operator* (
    const GLfloat * a) const [inline]
```

変換行列に配列に格納した変換行列を乗算した値を返す.

引数

a	変換行列を格納した GLfloat 型の 16 要素の配列変数.
-----	---

戻り値

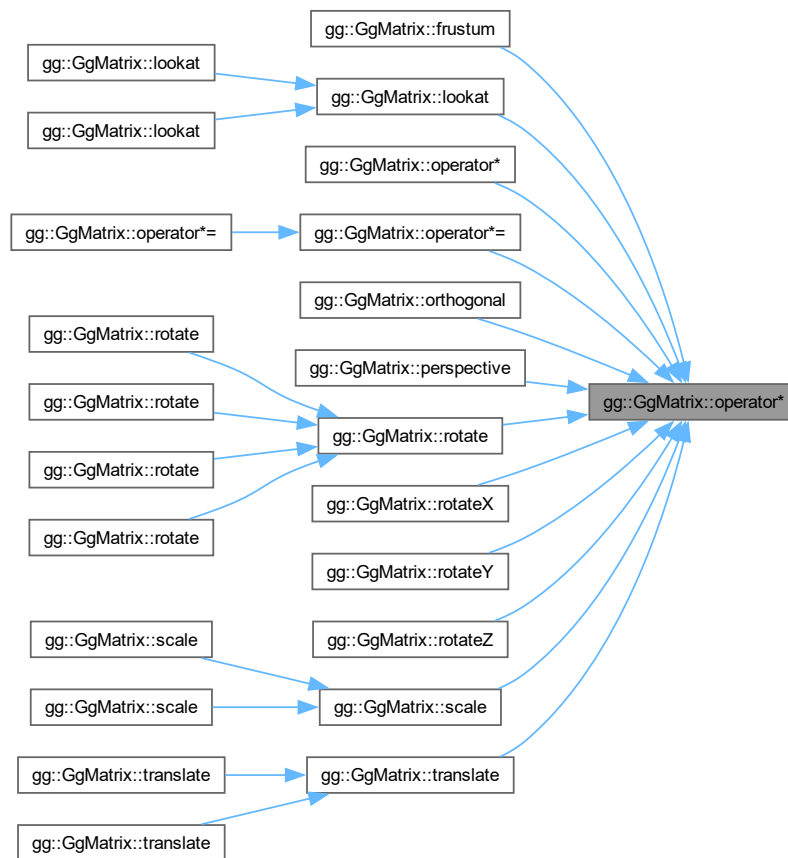
変換行列に a を乗じた **GgMatrix** 型の変換行列.

gg.h の 2363 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.6.3.39 operator*=() [1/2]

```
GgMatrix & gg::GgMatrix::operator*= (
    const GgMatrix & m) [inline]
```

変換行列に別の変換行列を乗算した結果を格納する。

引数

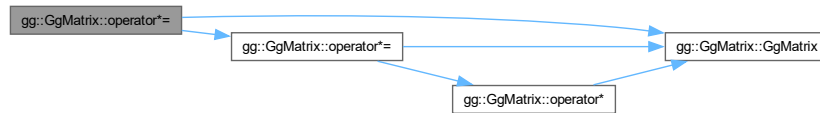
<i>m</i>	GgMatrix 型の変換行列.
----------	------------------

戻り値

m を掛けた変換行列の参照.

gg.h の 2399 行目に定義があります。

呼び出し関係図:



9.6.3.40 operator*=() [2/2]

```
GgMatrix & gg::GgMatrix::operator*= (
    const GLfloat * a) [inline]
```

変換行列に配列に格納した変換行列を乗算した結果を格納する.

引数

a	変換行列を格納した GLfloat 型の 16 要素の配列変数.
---	----------------------------------

戻り値

a を掛けた変換行列の参照.

gg.h の 2387 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.6.3.41 operator+() [1/2]

```
GgMatrix gg::GgMatrix::operator+ (
    const GgMatrix & m) const [inline]
```

変換行列に別の変換行列を加算した値を返す.

引数

<i>m</i>	GgMatrix 型の変換行列.
----------	------------------

戻り値

変換行列に *m* を加えた GgMatrix 型の変換行列.

gg.h の 2282 行目に定義があります。

呼び出し関係図:



9.6.3.42 operator+() [2/2]

```
GgMatrix gg::GgMatrix::operator+ (
    const GLfloat * a) const [inline]
```

変換行列に配列に格納した変換行列を加算した値を返す.

引数

<i>a</i>	変換行列を格納した GLfloat 型の 16 要素の配列変数.
----------	----------------------------------

戻り値

変換行列に *a* を加えた GgMatrix 型の変換行列.

gg.h の 2269 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.6.3.43 operator+=() [1/2]

```
GgMatrix & gg::GgMatrix::operator+= (
    const GgMatrix & m) [inline]
```

変換行列に別の変換行列を加算した結果を格納する.

引数

<i>m</i>	GgMatrix 型の変換行列.
----------	------------------

戻り値

m を加えた変換行列の参照.

gg.h の 2305 行目に定義があります。
呼び出し関係図:



9.6.3.44 operator+=() [2/2]

```
GgMatrix & gg::GgMatrix::operator+= (
    const GLfloat * a) [inline]
```

変換行列に配列に格納した変換行列を加算した結果を格納する.

引数

<i>a</i>	変換行列を格納した GLfloat 型の 16 要素の配列変数.
----------	----------------------------------

戻り値

a を加えた変換行列の参照.

gg.h の 2293 行目に定義があります。
呼び出し関係図:



被呼び出し関係図:



9.6.3.45 operator-() [1/2]

```
GgMatrix gg::GgMatrix::operator- (
    const GgMatrix & m) const [inline]
```

変換行列から別の変換行列を減算した値を返す.

引数

<i>m</i>	GgMatrix 型の変換行列.
----------	------------------

戻り値

変換行列から *m* を引いた GgMatrix 型の変換行列.

gg.h の 2329 行目に定義があります.

呼び出し関係図:



9.6.3.46 operator-() [2/2]

```
GgMatrix gg::GgMatrix::operator- (
    const GLfloat * a) const [inline]
```

変換行列から配列に格納した変換行列を減算した結果を格納する.

引数

<i>a</i>	変換行列を格納した GLfloat 型の 16 要素の配列変数.
----------	----------------------------------

戻り値

変換行列から *a* を引いた GgMatrix 型の変換行列.

gg.h の 2316 行目に定義があります.

呼び出し関係図:



被呼び出し関係図:



9.6.3.47 operator-=() [1/2]

```
GgMatrix & gg::GgMatrix::operator-= (
    const GgMatrix & m) [inline]
```

変換行列に別の変換行列を減算した結果を格納する.

引数

<i>m</i>	GgMatrix 型の変換行列.
----------	------------------

戻り値

m を引いた変換行列の参照.

[gg.h](#) の 2352 行目に定義があります。
呼び出し関係図:



9.6.3.48 operator-=() [2/2]

```
GgMatrix & gg::GgMatrix::operator-= (
    const GLfloat * a) [inline]
```

変換行列に配列に格納した変換行列を減算した結果を格納する.

引数

<i>a</i>	変換行列を格納した GLfloat 型の 16 要素の配列変数.
----------	----------------------------------

戻り値

a を引いた変換行列の参照.

[gg.h](#) の 2340 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.6.3.49 operator/() [1/2]

```
GgMatrix gg::GgMatrix::operator/ (
    const GgMatrix & m) const [inline]
```

変換行列を変換行列で除算した値を返す.

引数

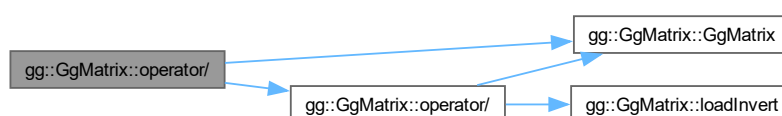
<i>m</i>	GgMatrix 型の変換行列.
----------	----------------------------------

戻り値

変換行列を *m* で割った [GgMatrix](#) 型の変換行列.

[gg.h](#) の 2424 行目に定義があります。

呼び出し関係図:



9.6.3.50 operator/() [2/2]

```
GgMatrix gg::GgMatrix::operator/ (
    const GLfloat * a) const [inline]
```

変換行列を配列に格納した変換行列で除算した値を返す.

引数

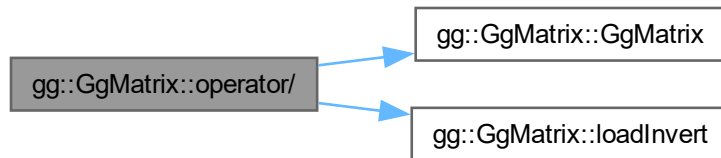
a	変換行列を格納した GLfloat 型の 16 要素の配列変数.
----------	----------------------------------

戻り値

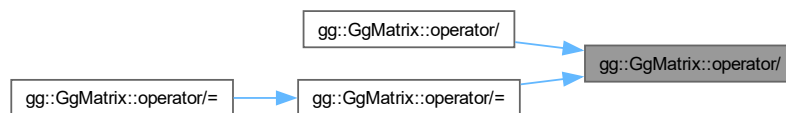
変換行列を **a** で割った **GgMatrix** 型の変換行列.

gg.h の 2410 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.6.3.51 operator/=() [1/2]

```
GgMatrix & gg::GgMatrix::operator/= (
    const GgMatrix & m) [inline]
```

変換行列を別の変換行列で除算した結果を格納する.

引数

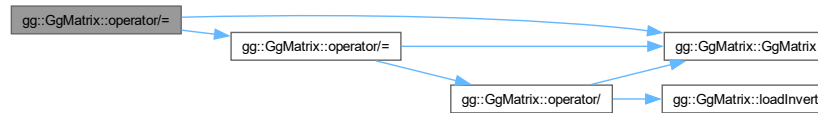
m	GgMatrix 型の変換行列.
----------	-------------------------

戻り値

m で割った変換行列の参照.

gg.h の 2447 行目に定義があります。

呼び出し関係図:



9.6.3.52 operator/=() [2/2]

```
GgMatrix & gg::GgMatrix::operator/= (
    const GLfloat * a) [inline]
```

変換行列を配列に格納した変換行列で除算した結果を格納する.

引数

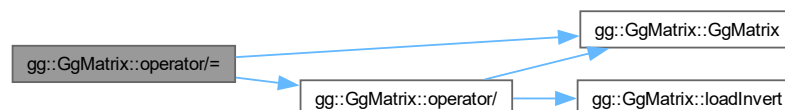
a	変換行列を格納した GLfloat 型の 16 要素の配列変数.
---	----------------------------------

戻り値

a で割った変換行列の参照.

gg.h の 2435 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.6.3.53 operator=() [1/3]

```
GgMatrix & gg::GgMatrix::operator= (
    const GgMatrix & m) [default]
```

代入演算子. 呼び出し関係図:



9.6.3.54 operator=() [2/3]

```
GgMatrix & gg::GgMatrix::operator= (
    const GLfloat * a) [inline]
```

配列変数の値を格納する.

引数

<i>a</i>	GLfloat 型の 16 要素の配列変数.
----------	------------------------

戻り値

a を代入したこのオブジェクトの参照.

[gg.h](#) の 2257 行目に定義があります.

呼び出し関係図:



9.6.3.55 operator=() [3/3]

```
GgMatrix & gg::GgMatrix::operator= (
    GgMatrix && m) [default]
```

ムーブ代入演算子. 呼び出し関係図:



9.6.3.56 orthogonal()

```
GgMatrix gg::GgMatrix::orthogonal (
    GLfloat left,
    GLfloat right,
    GLfloat bottom,
    GLfloat top,
    GLfloat zNear,
    GLfloat zFar) const [inline]
```

直交投影変換を乗じた結果を返す.

引数

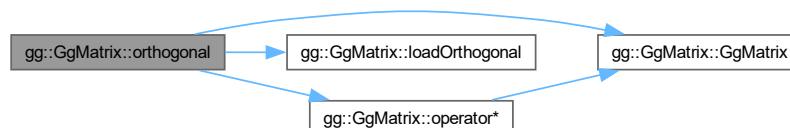
<i>left</i>	ウィンドウの左端の位置.
<i>right</i>	ウィンドウの右端の位置.
<i>bottom</i>	ウィンドウの下端の位置.
<i>top</i>	ウィンドウの上端の位置.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

直交投影変換行列を乗じた変換行列.

[gg.h](#) の 2979 行目に定義があります.

呼び出し関係図:



9.6.3.57 perspective()

```
GgMatrix gg::GgMatrix::perspective (
    GLfloat fovy,
```

```
GLfloat aspect,
GLfloat zNear,
GLfloat zFar) const [inline]
```

画角を指定して透視投影変換を乗じた結果を返す。

引数

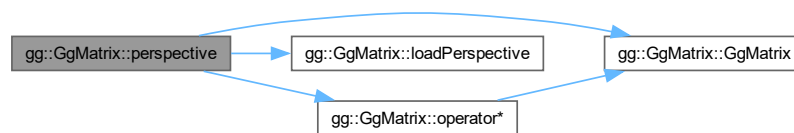
<i>fovy</i>	y 方向の画角.
<i>aspect</i>	縦横比.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

透視投影変換行列を乗じた変換行列.

gg.h の 3019 行目に定義があります。

呼び出し関係図:



9.6.3.58 projection() [1/4]

```
void gg::GgMatrix::projection (
    GgVector & c,
    const GgVector & v) const [inline]
```

ベクトルに対して投影変換を行う。

引数

<i>c</i>	変換結果を格納する GgVector 型の変数.
<i>v</i>	元のベクトルの GgVector 型の変数.

gg.h の 3100 行目に定義があります。

9.6.3.59 projection() [2/4]

```
void gg::GgMatrix::projection (
    GgVector & c,
    const GLfloat * v) const [inline]
```

ベクトルに対して投影変換を行う。

引数

<i>c</i>	変換結果を格納する GgVector 型の変数.
----------	---------------------------------

<code>v</code>	元のベクトルの <code>GLfloat</code> 型の 4 要素の配列変数.
----------------	--

`gg.h` の 3089 行目に定義があります。

9.6.3.60 projection() [3/4]

```
void gg::GgMatrix::projection (
    GLfloat * c,
    const GgVector & v) const [inline]
```

ベクトルに対して投影変換を行う。

引数

<code>c</code>	変換結果を格納する <code>GLfloat</code> 型の 4 要素の配列変数.
<code>v</code>	元のベクトルの <code>GgVector</code> 型の変数.

`gg.h` の 3078 行目に定義があります。

9.6.3.61 projection() [4/4]

```
void gg::GgMatrix::projection (
    GLfloat * c,
    const GLfloat * v) const [inline]
```

ベクトルに対して投影変換を行う。

引数

<code>c</code>	変換結果を格納する <code>GLfloat</code> 型の 4 要素の配列変数.
<code>v</code>	元のベクトルの <code>GLfloat</code> 型の 4 要素の配列変数.

`gg.h` の 3067 行目に定義があります。

9.6.3.62 rotate() [1/5]

```
GgMatrix gg::GgMatrix::rotate (
    const GgVector & r) const [inline]
```

`r` 方向のベクトルを軸とする回転の変換行列を乗じた結果を返す。

引数

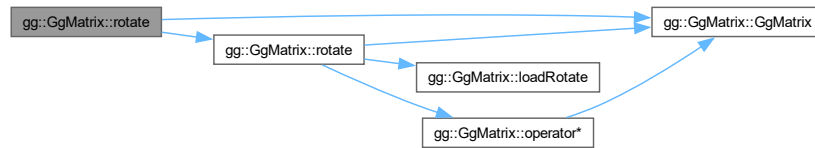
<code>r</code>	回転軸の方向ベクトルと回転角を格納した <code>GgVector</code> 型の変数).
----------------	--

戻り値

($r[0]$, $r[1]$, $r[2]$) を軸にさらに $r[3]$ 回転した変換行列.

gg.h の 2913 行目に定義があります.

呼び出し関係図:



9.6.3.63 rotate() [2/5]

```
GgMatrix gg::GgMatrix::rotate (
    const GgVector & r,
    GLfloat a) const [inline]
```

r 方向のベクトルを軸とする回転変換を乗じた結果を返す.

引数

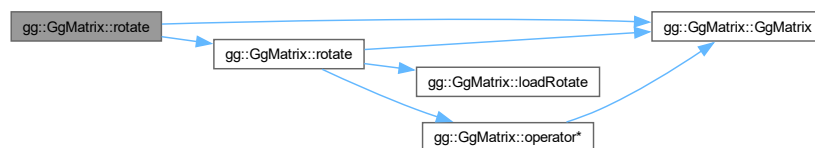
r	回転軸の方向ベクトルを格納した GgVector 型の変数.
a	回転角.

戻り値

($r[0]$, $r[1]$, $r[2]$) を軸にさらに a 回転した変換行列.

gg.h の 2891 行目に定義があります.

呼び出し関係図:



9.6.3.64 rotate() [3/5]

```
GgMatrix gg::GgMatrix::rotate (
    const GLfloat * r) const [inline]
```

r 方向のベクトルを軸とする回転の変換行列を乗じた結果を返す.

引数

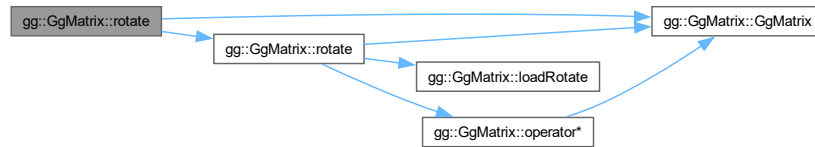
r	回転軸の方向ベクトルと回転角を格納した GLfloat 型の 4 要素の配列変数 (x , y , z , a).
-----	--

戻り値

($r[0]$, $r[1]$, $r[2]$) を軸にさらに $r[3]$ 回転した変換行列.

gg.h の 2902 行目に定義があります。

呼び出し関係図:



9.6.3.65 rotate() [4/5]

```
GgMatrix gg::GgMatrix::rotate (
    const GLfloat * r,
    GLfloat a) const [inline]
```

r 方向のベクトルを軸とする回転変換を乗じた結果を返す.

引数

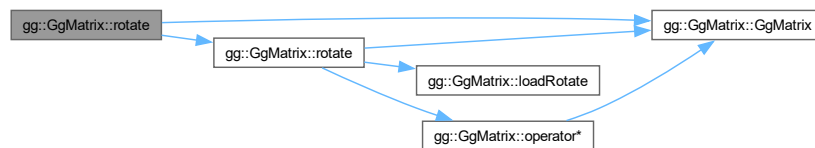
r	回転軸の方向ベクトルを格納した GLfloat 型の 3 要素の配列変数 (x, y, z).
a	回転角.

戻り値

($r[0]$, $r[1]$, $r[2]$) を軸にさらに a 回転した変換行列.

gg.h の 2879 行目に定義があります。

呼び出し関係図:



9.6.3.66 rotate() [5/5]

```
GgMatrix gg::GgMatrix::rotate (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat a) const [inline]
```

(x, y, z) 方向のベクトルを軸とする回転変換を乗じた結果を返す.

引数

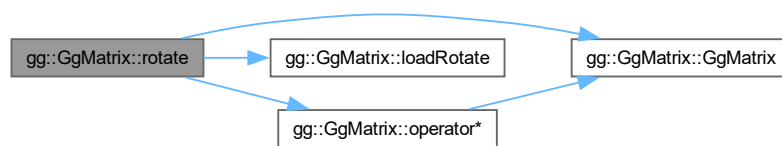
x	回転軸の x 成分.
y	回転軸の y 成分.
z	回転軸の z 成分.
a	回転角.

戻り値

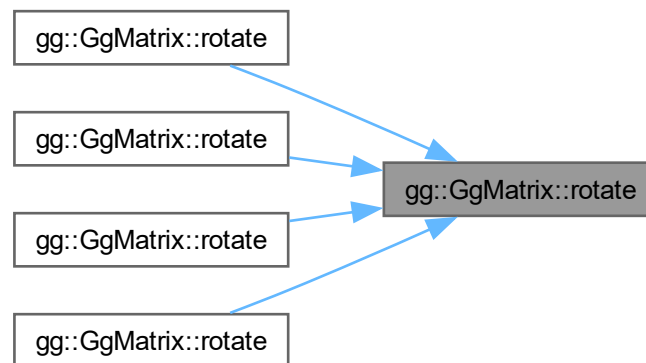
(x, y, z) を軸にさらに a 回転した変換行列.

gg.h の 2866 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.6.3.67 rotateX()

```
GgMatrix gg::GgMatrix::rotateX (
    GLfloat a) const [inline]
```

x 軸中心の回転変換を乗じた結果を返す.

引数

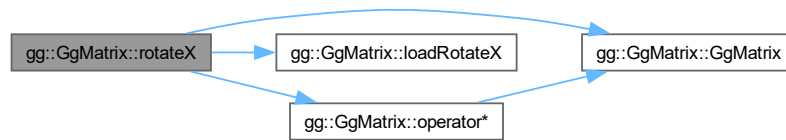
a	回転角.
-----	------

戻り値

x 軸中心にさらに a 回転した変換行列.

gg.h の 2827 行目に定義があります.

呼び出し関係図:



9.6.3.68 rotateY()

```
GgMatrix gg::GgMatrix::rotateY (
    GLfloat a) const [inline]
```

y 軸中心の回転変換を乗じた結果を返す.

引数

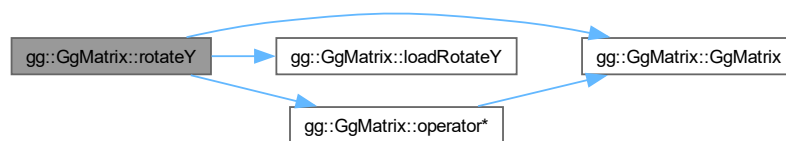
a	回転角.
---	------

戻り値

y 軸中心にさらに a 回転した変換行列.

gg.h の 2839 行目に定義があります.

呼び出し関係図:



9.6.3.69 rotateZ()

```
GgMatrix gg::GgMatrix::rotateZ (
    GLfloat a) const [inline]
```

z 軸中心の回転変換を乗じた結果を返す.

引数

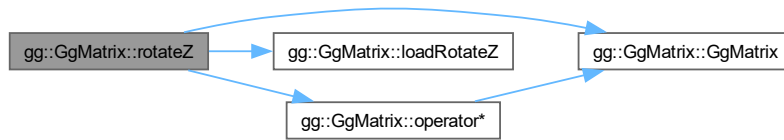
a	回転角.
---	------

戻り値

z 軸中心にさらに a 回転した変換行列.

gg.h の 2851 行目に定義があります。

呼び出し関係図:



9.6.3.70 scale() [1/3]

```
GgMatrix gg::GgMatrix::scale (
    const GgVector & s) const [inline]
```

拡大縮小変換を乗じた結果を返す.

引数

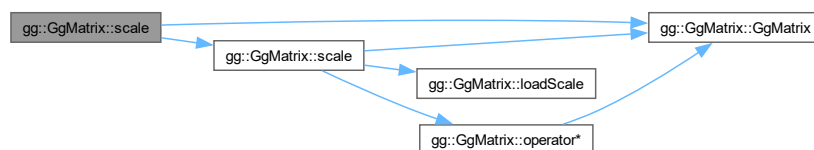
s	拡大率の GgVector 型の変数.
---	---------------------

戻り値

拡大縮小した結果の変換行列.

gg.h の 2816 行目に定義があります。

呼び出し関係図:



9.6.3.71 scale() [2/3]

```
GgMatrix gg::GgMatrix::scale (
    const GLfloat * s) const [inline]
```

拡大縮小変換を乗じた結果を返す.

引数

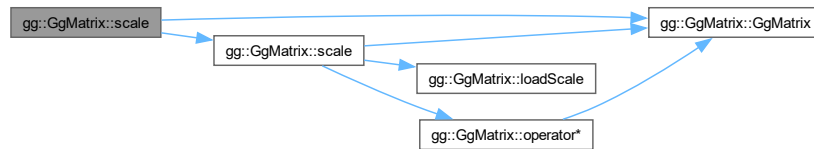
s	拡大率の GLfloat 型の 3 要素の配列変数 (x, y, z).
---	--------------------------------------

戻り値

拡大縮小した結果の変換行列.

[gg.h](#) の 2805 行目に定義があります。

呼び出し関係図:



9.6.3.72 scale() [3/3]

```

GgMatrix gg::GgMatrix::scale (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w = 1.0f) const [inline]
  
```

拡大縮小変換を乗じた結果を返す.

引数

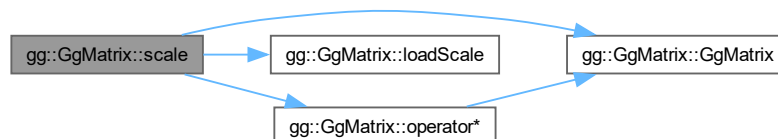
<i>x</i>	<i>x</i> 方向の拡大率.
<i>y</i>	<i>y</i> 方向の拡大率.
<i>z</i>	<i>z</i> 方向の拡大率.
<i>w</i>	<i>w</i> 移動量のスケールファクタ (= 1.0f).

戻り値

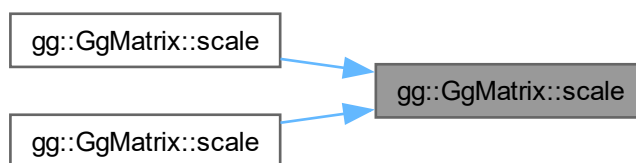
拡大縮小した結果の変換行列.

[gg.h](#) の 2793 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.6.3.73 translate() [1/3]

```
GgMatrix gg::GgMatrix::translate (
    const GgVector & t) const [inline]
```

平行移動変換を乗じた結果を返す.

引数

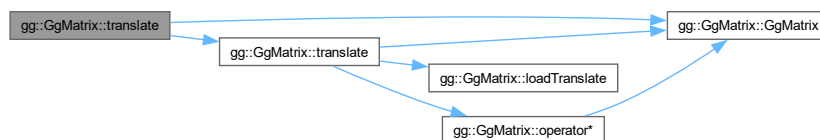
<i>t</i>	移動量の GgVector 型の変数.
----------	-------------------------------------

戻り値

平行移動した結果の変換行列.

[gg.h](#) の 2779 行目に定義があります。

呼び出し関係図:



9.6.3.74 translate() [2/3]

```
GgMatrix gg::GgMatrix::translate (
    const GLfloat * t) const [inline]
```

平行移動変換を乗じた結果を返す.

引数

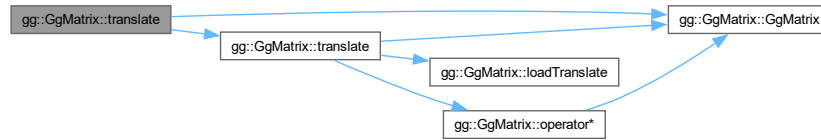
<i>t</i>	移動量の GLfloat 型の 3 要素の配列変数 (x, y, z).
----------	--------------------------------------

戻り値

平行移動した結果の変換行列.

[gg.h](#) の 2768 行目に定義があります。

呼び出し関係図:



9.6.3.75 translate() [3/3]

```

GgMatrix gg::GgMatrix::translate (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w = 1.0f) const [inline]
  
```

平行移動変換を乗じた結果を返す.

引数

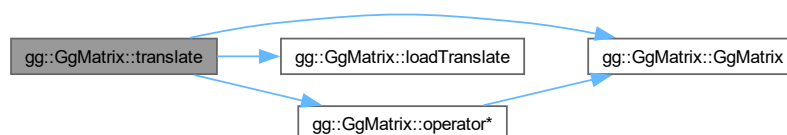
<i>x</i>	<i>x</i> 方向の移動量.
<i>y</i>	<i>y</i> 方向の移動量.
<i>z</i>	<i>z</i> 方向の移動量.
<i>w</i>	<i>w</i> 移動量のスケールファクタ (= 1.0f).

戻り値

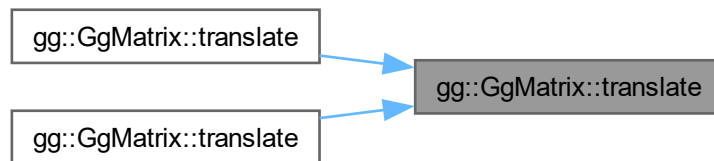
平行移動した結果の変換行列.

[gg.h](#) の 2756 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



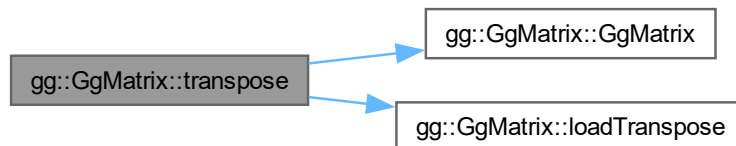
9.6.3.76 transpose()

`GgMatrix` `gg::GgMatrix::transpose () const [inline]`
転置行列を返す.

戻り値

転置行列.

`gg.h` の 3033 行目に定義があります。
呼び出し関係図:



被呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- `gg.h`
- `gg.cpp`

9.7 gg::GgNormalTexture クラス

```
#include <gg.h>
```

公開メンバ関数

- `GgNormalTexture ()`
- `GgNormalTexture (const GLubyte *image, GLsizei width, GLsizei height, GLenum format=GL_RED, GLfloat nz=1.0f, GLenum internal=GL_RGBA)`
- `GgNormalTexture (const std::string &name, GLfloat nz=1.0f, GLenum internal=GL_RGBA)`
- `virtual ~GgNormalTexture ()`
- `void load (const GLubyte *hmap, GLsizei width, GLsizei height, GLenum format=GL_RED, GLfloat nz=1.0f, GLenum internal=GL_RGBA)`
- `void load (const std::string &name, GLfloat nz=1.0f, GLenum internal=GL_RGBA)`
- `operator bool () const noexcept`
- `bool operator! () const noexcept`
- `const GgTexture * get () const`
- `GLuint getTexture () const`
- `GLsizei getWidth () const`
- `GLsizei getHeight () const`
- `void getSize (GLsizei *size) const`
- `const GLsizei * getSize () const`
- `void bind () const`
- `void unbind () const`

9.7.1 詳解

法線マップ.

覚え書き

高さマップ（グレイスケール画像）を読み込んで法線マップのテクスチャを作成する.

`gg.h` の 5587 行目に定義があります。

9.7.2 構築子と解体子

9.7.2.1 GgNormalTexture() [1/3]

```
gg::GgNormalTexture::GgNormalTexture () [inline]
コンストラクタ.
```

`gg.h` の 5597 行目に定義があります。

9.7.2.2 GgNormalTexture() [2/3]

```
gg::GgNormalTexture::GgNormalTexture (
    const GLubyte * image,
    GLsizei width,
    GLsizei height,
    GLenum format = GL_RED,
    GLfloat nz = 1.0f,
    GLenum internal = GL_RGBA) [inline]
```

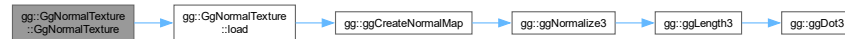
メモリ上のデータから法線マップのテクスチャを作成するコンストラクタ.

引数

<code>image</code>	テクスチャとして用いる画像データ, nullptr ならデータを読み込まない.
--------------------	---

<i>width</i>	テクスチャとして用いる画像データの横幅.
<i>height</i>	テクスチャとして用いる画像データの高さ.
<i>format</i>	テクスチャとして用いる画像データのフォーマット (GL_RED, GL_RG, GL_RGB, GL_RGBA).
<i>nz</i>	法線マップの z 成分の値.
<i>internal</i>	テクスチャの内部フォーマット.

gg.h の 5611 行目に定義があります。
呼び出し関係図:



9.7.2.3 GgNormalTexture() [3/3]

```

gg::GgNormalTexture::GgNormalTexture (
    const std::string & name,
    GLfloat nz = 1.0f,
    GLenum internal = GL_RGBA) [inline]

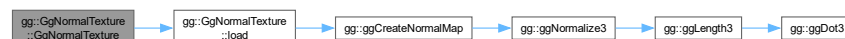
```

ファイルからデータを読み込んで法線マップのテクスチャを作成するコンストラクタ.

引数

<i>name</i>	画像ファイル名.
<i>nz</i>	法線マップの z 成分の値.
<i>internal</i>	テクスチャの内部フォーマット.

gg.h の 5631 行目に定義があります。
呼び出し関係図:



9.7.2.4 ~GgNormalTexture()

```

virtual gg::GgNormalTexture::~~GgNormalTexture () [inline], [virtual]

```

デストラクタ.

gg.h の 5644 行目に定義があります。

9.7.3 関数詳解

9.7.3.1 bind()

```

void gg::GgNormalTexture::bind () const [inline]

```

テクスチャの使用開始 (このテクスチャを使用する際に呼び出す).
gg.h の 5773 行目に定義があります。

9.7.3.2 get()

`const GgTexture * gg::GgNormalTexture::get () const [inline]`
 テクスチャオブジェクトを取り出す。

戻り値

`GgTexture` オブジェクトのポインタ。

`gg.h` の 5715 行目に定義があります。

9.7.3.3 getHeight()

`GLsizei gg::GgNormalTexture::getHeight () const [inline]`
 使用しているテクスチャの縦の画素数を取り出す。

戻り値

テクスチャの縦の画素数。

`gg.h` の 5745 行目に定義があります。

9.7.3.4 getSize() [1/2]

`const GLsizei * gg::GgNormalTexture::getSize () const [inline]`
 使用しているテクスチャのサイズを取り出す。

戻り値

テクスチャのサイズを格納した配列へのポインタ。

`gg.h` の 5765 行目に定義があります。

9.7.3.5 getSize() [2/2]

`void gg::GgNormalTexture::getSize (`
 `GLsizei * size) const [inline]`
 使用しているテクスチャのサイズを取り出す。

引数

<code>size</code>	テクスチャのサイズを格納する <code>GLsizei</code> 型の 2 要素の配列変数。
-------------------	---

`gg.h` の 5755 行目に定義があります。

9.7.3.6 getTexture()

`GLuint gg::GgNormalTexture::getTexture () const [inline]`
 使用しているテクスチャのテクスチャ名を得る。

戻り値

テクスチャ名。

`gg.h` の 5725 行目に定義があります。

9.7.3.7 getWidth()

`GLsizei gg::GgNormalTexture::getWidth () const [inline]`
 使用しているテクスチャの横の画素数を取り出す。

戻り値

テクスチャの横の画素数。

`gg.h` の 5735 行目に定義があります。

9.7.3.8 load() [1/2]

```
void gg::GgNormalTexture::load (
    const GLubyte * hmap,
    GLsizei width,
    GLsizei height,
    GLenum format = GL_RED,
    GLfloat nz = 1.0f,
    GLenum internal = GL_RGBA) [inline]
```

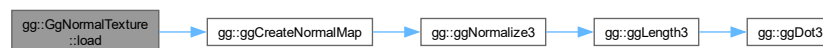
メモリ上のデータから法線マップのテクスチャを作成する.

引数

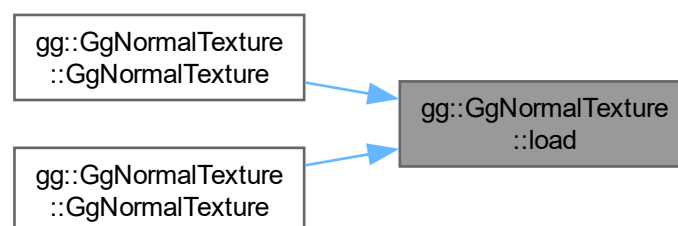
<i>hmap</i>	テクスチャとして用いる画像データ, nullptr ならデータを読み込まない.
<i>width</i>	テクスチャとして用いる画像データの横幅.
<i>height</i>	テクスチャとして用いる画像データの高さ.
<i>format</i>	テクスチャとして用いる画像データのフォーマット (GL_RED, GL_RG, GL_RGB, GL_RGBA).
<i>nz</i>	法線マップの z 成分の値.
<i>internal</i>	テクスチャの内部フォーマット.

gg.h の 5658 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.7.3.9 load() [2/2]

```
void gg::GgNormalTexture::load (
    const std::string & name,
    GLfloat nz = 1.0f,
    GLenum internal = GL_RGBA)
```

TGA フォーマットの画像ファイルから高さマップ読み込んで法線マップのテクスチャを作成する.

引数

<i>name</i>	画像ファイル名 (1 チャンネルの TGA 画像).
<i>nz</i>	法線マップの z 成分の値.
<i>internal</i>	テクスチャの内部フォーマット.

[gg.cpp](#) の 4080 行目に定義があります。
呼び出し関係図:



9.7.3.10 operator bool()

```
gg::GgNormalTexture::operator bool () const [inline], [explicit], [noexcept]
```

オブジェクトが有効かどうか調べる.

戻り値

オブジェクトが有効なら true.

[gg.h](#) の 5695 行目に定義があります。

9.7.3.11 operator"!()"

```
bool gg::GgNormalTexture::operator! () const [inline], [noexcept]
```

オブジェクトが有効かどうかの結果を反転する.

戻り値

オブジェクトが有効なら false, 無効なら true.

[gg.h](#) の 5705 行目に定義があります。

9.7.3.12 unbind()

```
void gg::GgNormalTexture::unbind () const [inline]
```

テクスチャの使用終了 (このテクスチャを使用しなくなったら呼び出す).

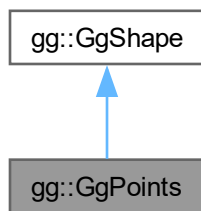
[gg.h](#) の 5781 行目に定義があります。
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

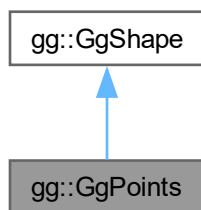
9.8 gg::GgPoints クラス

```
#include <gg.h>
```

gg::GgPoints の継承関係図



gg::GgPoints 連携図



公開メンバ関数

- `GgPoints` (GLenum mode=GL_POINTS)
- `GgPoints` (const `GgVector` *pos, GLsizei countv, GLenum mode=GL_POINTS, GLenum usage=GL_STATIC_DRAW)
- virtual `~GgPoints` ()
- `operator bool` () const noexcept
- `bool operator!` () const noexcept
- const GLsizei & `getCount` () const
- const GLuint & `getBuffer` () const
- void `send` (const `GgVector` *pos, GLint first=0, GLsizei count=0) const
- void `load` (const `GgVector` *pos, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- virtual void `draw` (GLint first=0, GLsizei count=0) const

基底クラス `gg::GgShape` に属する継承公開メンバ関数

- `GgShape` (GLenum mode=0)
- virtual `~GgShape` ()
- const GLuint & `get` () const
- void `setMode` (GLenum mode)
- const GLenum & `getMode` () const

9.8.1 詳解

点.

[gg.h](#) の 6548 行目に定義があります。

9.8.2 構築子と解体子

9.8.2.1 GgPoints() [1/2]

```
gg::GgPoints::GgPoints (
    GLenum mode = GL_POINTS) [inline]
```

コンストラクタ.

[gg.h](#) の 6559 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.8.2.2 GgPoints() [2/2]

```
gg::GgPoints::GgPoints (
    const GgVector * pos,
    GLsizei countv,
    GLenum mode = GL_POINTS,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

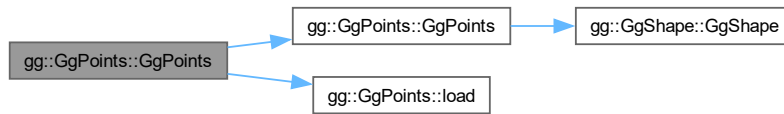
コンストラクタ.

引数

<i>pos</i>	この図形の頂点の位置のデータの配列 (nullptr ならデータを転送しない).
<i>countv</i>	頂点数.
<i>mode</i>	描画する基本図形の種類.
<i>usage</i>	バッファオブジェクトの使い方.

[gg.h](#) の 6572 行目に定義があります。

呼び出し関係図:



9.8.2.3 ~GgPoints()

`virtual gg::GgPoints::~GgPoints () [inline], [virtual]`
デストラクタ.

[gg.h](#) の [6586](#) 行目に定義があります。

9.8.3 関数詳解

9.8.3.1 draw()

```
void gg::GgPoints::draw (
    GLint first = 0,
    GLsizei count = 0) const [virtual]
```

点の描画.

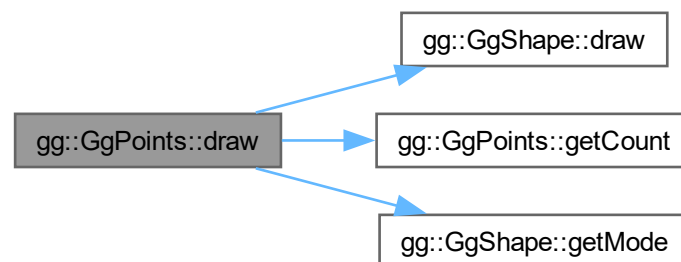
引数

<i>first</i>	描画を開始する最初の点の番号.
<i>count</i>	描画する点の数, 0 なら全部の点を描く.

[gg::GgShape](#) を再実装しています。

[gg.cpp](#) の [5145](#) 行目に定義があります。

呼び出し関係図:



9.8.3.2 getBuffer()

`const GLuint & gg::GgPoints::getBuffer () const [inline]`

頂点の位置データを格納した頂点バッファオブジェクト名を取り出す.

戻り値

この図形の頂点の位置データを格納した頂点バッファオブジェクト名.

gg.h の 6625 行目に定義があります。

9.8.3.3 getCount()

```
const GLsizei & gg::GgPoints::getCount () const [inline]
```

データの数を取り出す.

戻り値

この図形の頂点の位置データの数 (頂点数).

gg.h の 6615 行目に定義があります。

被呼び出し関係図:



9.8.3.4 load()

```
void gg::GgPoints::load (
    const GgVector * pos,
    GLsizei count,
    GLenum usage = GL_STATIC_DRAW)
```

バッファオブジェクトを確保して頂点の位置データを格納する.

引数

<i>pos</i>	頂点の位置データが格納されている領域の先頭のポインタ.
<i>count</i>	頂点のデータの数 (頂点数).
<i>usage</i>	バッファオブジェクトの使い方.

gg.cpp の 5132 行目に定義があります。

被呼び出し関係図:



9.8.3.5 operator bool()

`gg::GgPoints::operator bool () const [inline], [explicit], [virtual], [noexcept]`
バッファが有効かどうか調べる.

戻り値

バッファが有効なら `true`

[gg::GgShape](#) を再実装しています。
[gg.h](#) の 6595 行目に定義があります。

9.8.3.6 operator"!()"

`bool gg::GgPoints::operator! () const [inline], [virtual], [noexcept]`
バッファが有効かどうかの結果を反転する.

戻り値

バッファが有効なら `false`, 無効なら `true`.

[gg::GgShape](#) を再実装しています。
[gg.h](#) の 6605 行目に定義があります。

9.8.3.7 send()

```
void gg::GgPoints::send (
    const GgVector * pos,
    GLint first = 0,
    GLsizei count = 0) const [inline]
```

既存のバッファオブジェクトに頂点の位置データを転送する.

引数

<i>pos</i>	転送元の頂点の位置データが格納されている領域の先頭のポインタ.
<i>first</i>	転送先のバッファオブジェクトの先頭の要素番号.
<i>count</i>	転送する頂点の位置データの数 (0 ならバッファオブジェクト全体).

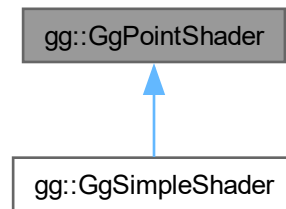
[gg.h](#) の 6637 行目に定義があります。
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

9.9 gg::GgPointShader クラス

```
#include <gg.h>
```

gg::GgPointShader の継承関係図



公開メンバ関数

- [GgPointShader\(\)](#)
- [GgPointShader](#) (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- [GgPointShader](#) (const std::array< std::string, 3 > &files, int nvarying=0, const char *const *varyings=nullptr)
- virtual [~GgPointShader\(\)](#)
- bool [load](#) (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- bool [load](#) (const std::array< std::string, 3 > &files, GLint nvarying=0, const char *const *varyings=nullptr)
- virtual void [loadProjectionMatrix](#) (const GLfloat *mp) const
- virtual void [loadProjectionMatrix](#) (const [GgMatrix](#) &mp) const
- virtual void [loadModelviewMatrix](#) (const GLfloat *mv) const
- virtual void [loadModelviewMatrix](#) (const [GgMatrix](#) &mv) const
- virtual void [loadMatrix](#) (const GLfloat *mp, const GLfloat *mv) const
- virtual void [loadMatrix](#) (const [GgMatrix](#) &mp, const [GgMatrix](#) &mv) const
- virtual void [use](#) () const
- void [use](#) (const GLfloat *mp) const
- void [use](#) (const [GgMatrix](#) &mp) const
- void [use](#) (const GLfloat *mp, const GLfloat *mv) const
- void [use](#) (const [GgMatrix](#) &mp, const [GgMatrix](#) &mv) const
- void [unuse](#) () const
- GLuint [get](#) () const

9.9.1 詳解

点のシェーダ.

[gg.h](#) の 7184 行目に定義があります。

9.9.2 構築子と解体子

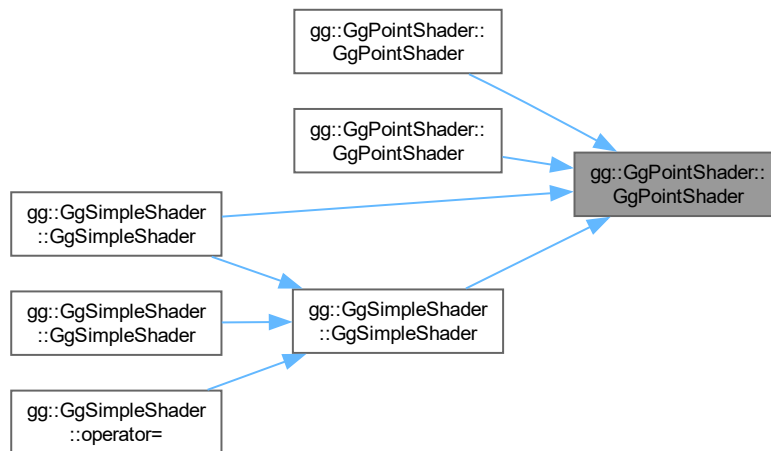
9.9.2.1 GgPointShader() [1/3]

`gg::GgPointShader::GgPointShader () [inline]`

コンストラクタ.

[gg.h](#) の 7200 行目に定義があります。

被呼び出し関係図:



9.9.2.2 GgPointShader() [2/3]

```

gg::GgPointShader::GgPointShader (
    const std::string & vert,
    const std::string & frag = "",
    const std::string & geom = "",
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [inline]

```

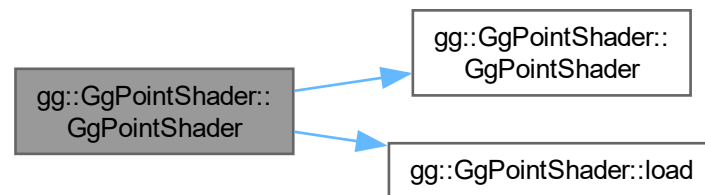
コンストラクタ.

引数

<i>vert</i>	バーテックスシェーダのソースファイル名.
<i>frag</i>	フラグメントシェーダのソースファイル名 (空文字列なら不使用).
<i>geom</i>	ジオメトリシェーダのソースファイル名 (空文字列なら不使用).
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト.

[gg.h](#) の 7215 行目に定義があります。

呼び出し関係図:



9.9.2.3 GgPointShader() [3/3]

```
gg::GgPointShader::GgPointShader (
    const std::array< std::string, 3 > & files,
    int nvarying = 0,
    const char *const * varyings = nullptr) [inline]
```

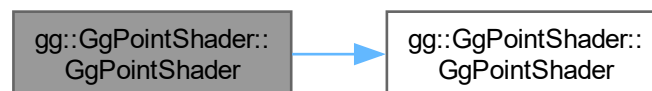
コンストラクタ.

引数

<i>files</i>	バーテックスシェーダのソースファイル名の配列.
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (nullptr なら不使用).

[gg.h](#) の 7234 行目に定義があります。

呼び出し関係図:



9.9.2.4 ~GgPointShader()

```
virtual gg::GgPointShader::~GgPointShader () [inline], [virtual]
```

デストラクタ.

[gg.h](#) の 7246 行目に定義があります。

9.9.3 関数詳解

9.9.3.1 get()

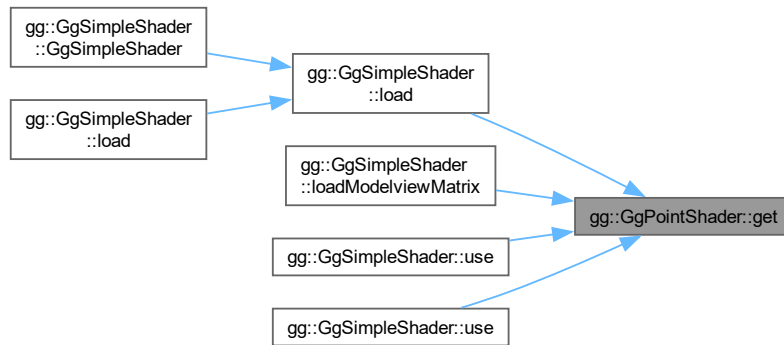
```
GLuint gg::GgPointShader::get () const [inline]
```

シェーダのプログラム名を得る.

戻り値

シェーダのプログラム名.

gg.h の 7430 行目に定義があります。
被呼び出し関係図:



9.9.3.2 load() [1/2]

```

bool gg::GgPointShader::load (
    const std::array< std::string, 3 > & files,
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [inline]
  
```

シェーダのソースファイルを読み込んでプログラムオブジェクトを作成する.

引数

<i>files</i>	バーテックスシェーダのソースファイル名の配列.
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (nullptr なら不使用).

戻り値

プログラムオブジェクトの作成に成功したら true.

gg.h の 7293 行目に定義があります。
呼び出し関係図:



9.9.3.3 load() [2/2]

```
bool gg::GgPointShader::load (
    const std::string & vert,
    const std::string & frag = "",
    const std::string & geom = "",
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [inline]
```

シェーダのソースファイルを読み込む。

引数

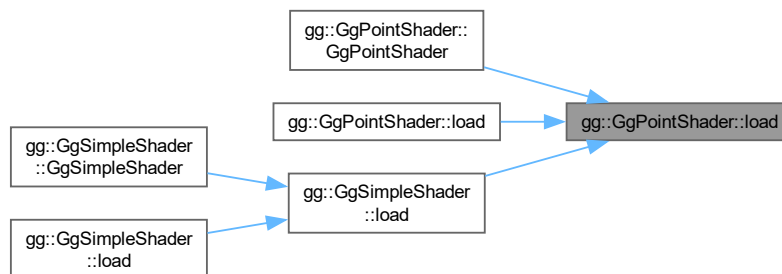
<i>vert</i>	バーテックスシェーダのソースファイル名.
<i>frag</i>	フラグメントシェーダのソースファイル名 (空文字列なら不使用).
<i>geom</i>	ジオメトリシェーダのソースファイル名 (空文字列なら不使用).
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト.

戻り値

プログラムオブジェクトが作成できれば `true`.

[gg.h](#) の 7260 行目に定義があります。

被呼び出し関係図:



9.9.3.4 loadMatrix() [1/2]

```
virtual void gg::GgPointShader::loadMatrix (
    const GgMatrix & mp,
    const GgMatrix & mv) const [inline], [virtual]
```

投影変換行列とモデルビュー変換行列を設定する。

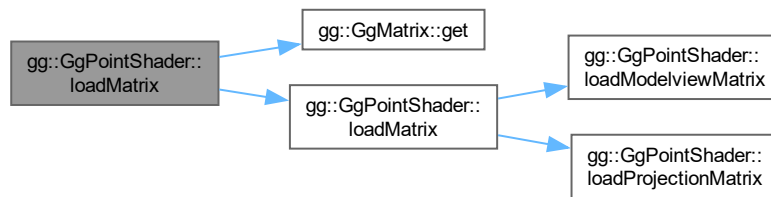
引数

<i>mp</i>	<code>GgMatrix</code> 型の投影変換行列.
<i>mv</i>	<code>GgMatrix</code> 型のモデルビュー変換行列.

`gg::GgSimpleShader` で再実装されています。

[gg.h](#) の 7360 行目に定義があります。

呼び出し関係図:



9.9.3.5 loadMatrix() [2/2]

```
virtual void gg::GgPointShader::loadMatrix (
    const GLfloat * mp,
    const GLfloat * mv) const [inline], [virtual]
```

投影変換行列とモデルビュー変換行列を設定する.

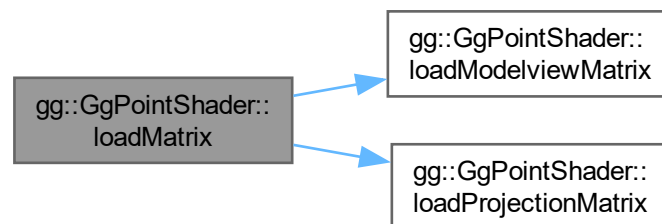
引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.

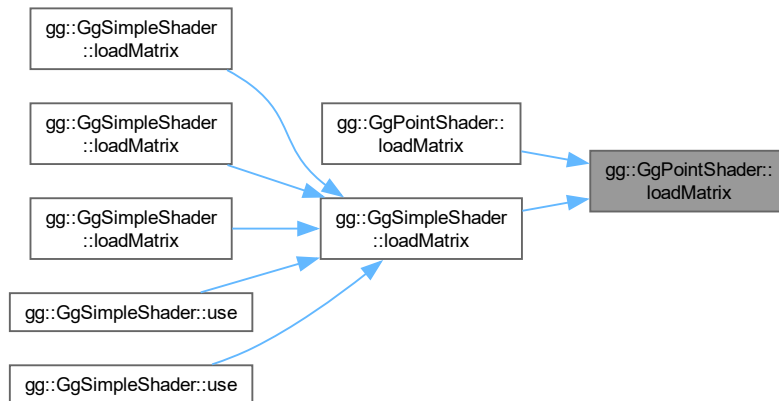
[gg::GgSimpleShader](#)で再実装されています。

[gg.h](#) の 7348 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.9.3.6 loadModelviewMatrix() [1/2]

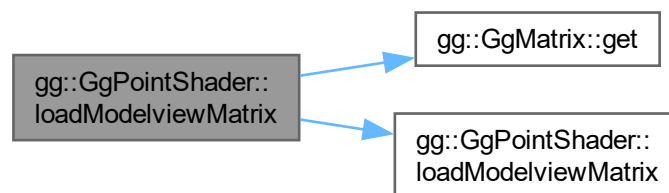
```
virtual void gg::GgPointShader::loadModelviewMatrix (
    const GgMatrix & mv) const [inline], [virtual]
```

モデルビュー変換行列を設定する。

引数

<i>mv</i>	GgMatrix 型のモデルビュー変換行列.
-----------	-------------------------------

[gg::GgSimpleShader](#)で再実装されています。
[gg.h](#) の 7337 行目に定義があります。
 呼び出し関係図:



9.9.3.7 loadModelviewMatrix() [2/2]

```
virtual void gg::GgPointShader::loadModelviewMatrix (
    const GLfloat * mv) const [inline], [virtual]
```

モデルビュー変換行列を設定する。

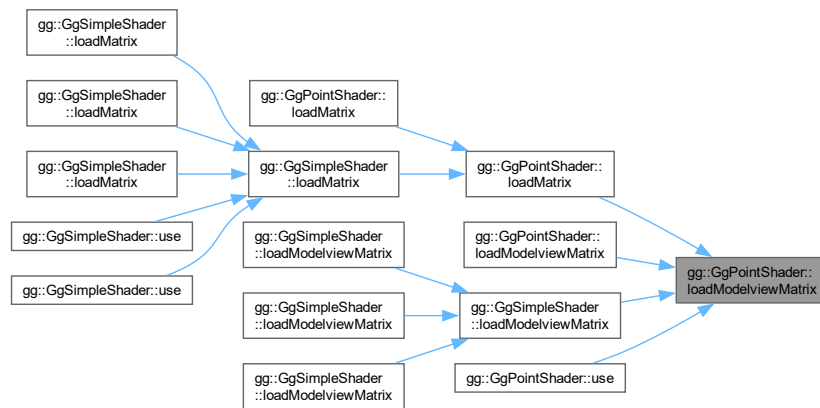
引数

<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.
-----------	--

gg::GgSimpleShader で再実装されています。

gg.h の 7327 行目に定義があります。

被呼び出し関係図:



9.9.3.8 loadProjectionMatrix() [1/2]

```
virtual void gg::GgPointShader::loadProjectionMatrix (
    const GgMatrix & mp) const [inline], [virtual]
```

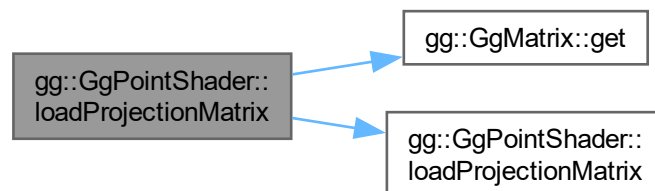
投影変換行列を設定する。

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
-----------	--------------------

gg.h の 7317 行目に定義があります。

呼び出し関係図:



9.9.3.9 loadProjectionMatrix() [2/2]

```
virtual void gg::GgPointShader::loadProjectionMatrix (
    const GLfloat * mp) const [inline], [virtual]
```

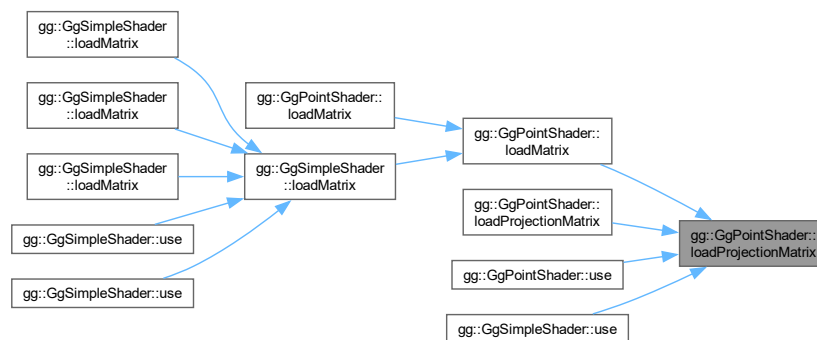
投影変換行列を設定する。

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列。
-----------	------------------------------------

[gg.h](#) の 7307 行目に定義があります。

被呼び出し関係図:



9.9.3.10 unuse()

```
void gg::GgPointShader::unuse () const [inline]
```

シェーダプログラムの使用を終了する。

[gg.h](#) の 7420 行目に定義があります。

9.9.3.11 use() [1/5]

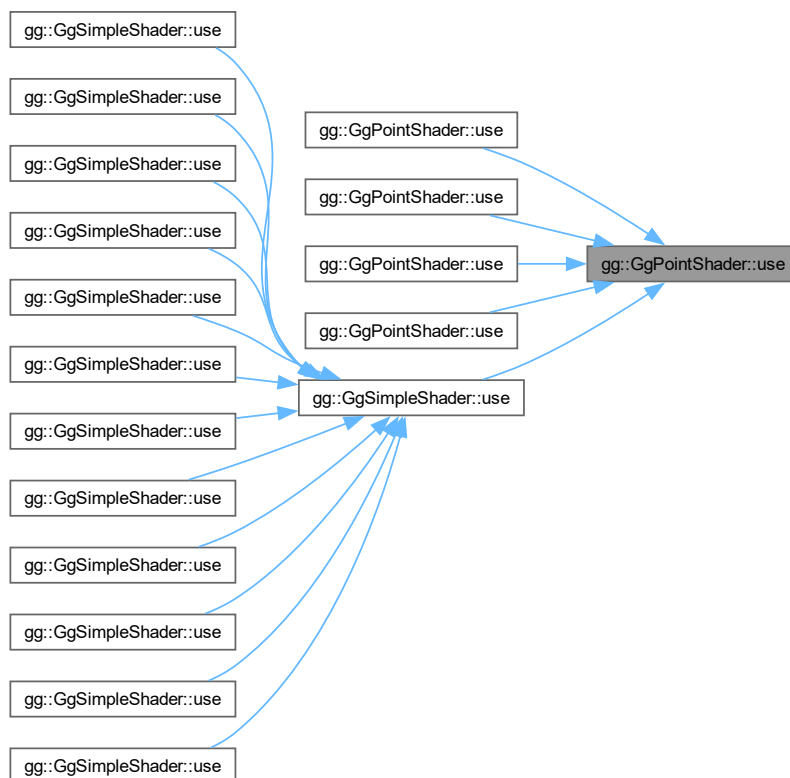
```
virtual void gg::GgPointShader::use () const [inline], [virtual]
```

シェーダプログラムの使用を開始する。

[gg::GgSimpleShader](#) で再実装されています。

[gg.h](#) の 7368 行目に定義があります。

被呼び出し関係図:



9.9.3.12 use() [2/5]

```
void gg::GgPointShader::use (
    const GgMatrix & mp) const [inline]
```

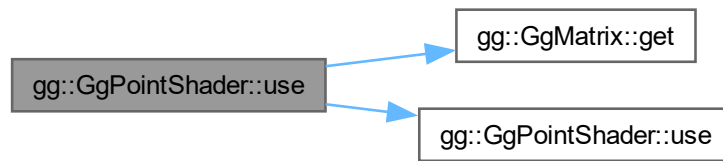
投影変換行列を設定してシェーダプログラムの使用を開始する。

引数

<i>mp</i>	<code>GgMatrix</code> 型の投影変換行列.
-----------	---------------------------------

`gg.h` の 7389 行目に定義があります。

呼び出し関係図:



9.9.3.13 use() [3/5]

```
void gg::GgPointShader::use (
    const GgMatrix & mp,
    const GgMatrix & mv) const [inline]
```

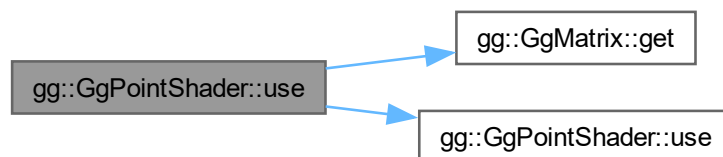
投影変換行列とモデルビューを設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.

gg.h の 7412 行目に定義があります。

呼び出し関係図:



9.9.3.14 use() [4/5]

```
void gg::GgPointShader::use (
    const GLfloat * mp) const [inline]
```

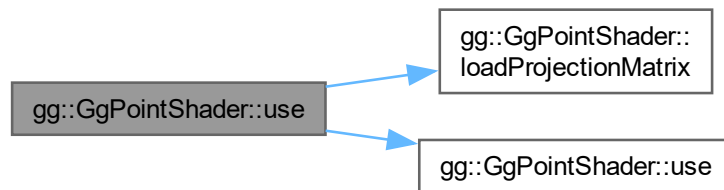
投影変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
-----------	------------------------------------

gg.h の 7378 行目に定義があります。

呼び出し関係図:



9.9.3.15 use() [5/5]

```
void gg::GgPointShader::use (
    const GLfloat * mp,
    const GLfloat * mv) const [inline]
```

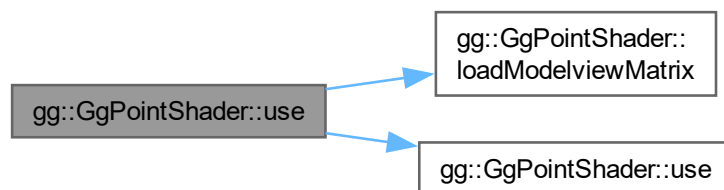
投影変換行列とモデルビューを設定してシェーダプログラムの使用を開始する。

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.

[gg.h](#) の 7400 行目に定義があります。

呼び出し関係図:



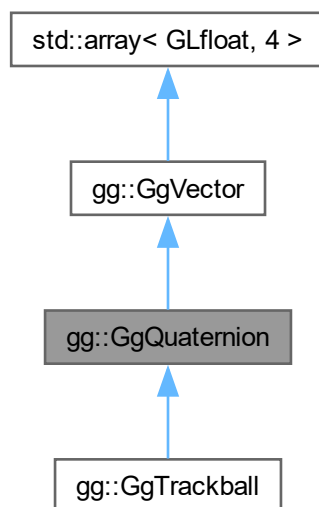
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)

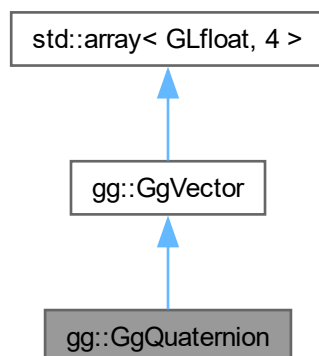
9.10 gg::GgQuaternion クラス

```
#include <gg.h>
```

gg::GgQuaternion の継承関係図



gg::GgQuaternion 連携図



公開メンバ関数

- `GgQuaternion()`=default
- `constexpr GgQuaternion (GLfloat x, GLfloat y, GLfloat z, GLfloat w)`
- `constexpr GgQuaternion (GLfloat c)`
- `GgQuaternion (const GLfloat *a)`
- `GgQuaternion (const GgVector &v)`
- `GgQuaternion (const GgQuaternion &q)`=default
- `GgQuaternion (GgQuaternion &&q)`=default
- `~GgQuaternion()`=default

- GgQuaternion & operator= (const GgQuaternion &q)=default
- GgQuaternion & operator= (GgQuaternion &&q)=default
- GLfloat norm () const
- GgQuaternion & loadAdd (GLfloat x, GLfloat y, GLfloat z, GLfloat w)
- GgQuaternion & loadAdd (const GLfloat *a)
- GgQuaternion & loadAdd (const GgVector &v)
- GgQuaternion & loadAdd (const GgQuaternion &q)
- GgQuaternion & loadSubtract (GLfloat x, GLfloat y, GLfloat z, GLfloat w)
- GgQuaternion & loadSubtract (const GLfloat *a)
- GgQuaternion & loadSubtract (const GgVector &v)
- GgQuaternion & loadSubtract (const GgQuaternion &q)
- GgQuaternion & loadMultiply (GLfloat x, GLfloat y, GLfloat z, GLfloat w)
- GgQuaternion & loadMultiply (const GLfloat *a)
- GgQuaternion & loadMultiply (const GgVector &v)
- GgQuaternion & loadMultiply (const GgQuaternion &q)
- GgQuaternion & loadDivide (GLfloat x, GLfloat y, GLfloat z, GLfloat w)
- GgQuaternion & loadDivide (const GLfloat *a)
- GgQuaternion & loadDivide (const GgVector &v)
- GgQuaternion & loadDivide (const GgQuaternion &q)
- GgQuaternion add (GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
- GgQuaternion add (const GLfloat *a) const
- GgQuaternion add (const GgVector &v) const
- GgQuaternion add (const GgQuaternion &q) const
- GgQuaternion subtract (GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
- GgQuaternion subtract (const GLfloat *a) const
- GgQuaternion subtract (const GgVector &v) const
- GgQuaternion subtract (const GgQuaternion &q) const
- GgQuaternion multiply (GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
- GgQuaternion multiply (const GLfloat *a) const
- GgQuaternion multiply (const GgVector &v) const
- GgQuaternion multiply (const GgQuaternion &q) const
- GgQuaternion divide (GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
- GgQuaternion divide (const GLfloat *a) const
- GgQuaternion divide (const GgVector &v) const
- GgQuaternion divide (const GgQuaternion &q) const
- GgQuaternion & operator= (const GLfloat *a)
- GgQuaternion & operator= (const GgVector &v)
- GgQuaternion & operator+= (const GLfloat *a)
- GgQuaternion & operator+= (const GgVector &v)
- GgQuaternion & operator+= (const GgQuaternion &q)
- GgQuaternion & operator-= (const GLfloat *a)
- GgQuaternion & operator-= (const GgVector &v)
- GgQuaternion & operator-= (const GgQuaternion &q)
- GgQuaternion & operator*= (const GLfloat *a)
- GgQuaternion & operator*= (const GgVector &v)
- GgQuaternion & operator*= (const GgQuaternion &q)
- GgQuaternion & operator/= (const GLfloat *a)
- GgQuaternion & operator/= (const GgVector &v)
- GgQuaternion & operator/= (const GgQuaternion &q)
- GgQuaternion operator+ (const GLfloat *a) const
- GgQuaternion operator+ (const GgVector &v) const
- GgQuaternion operator+ (const GgQuaternion &q) const
- GgQuaternion operator- (const GLfloat *a) const
- GgQuaternion operator- (const GgVector &v) const
- GgQuaternion operator- (const GgQuaternion &q) const

- `GgQuaternion operator*` (const GLfloat *a) const
- `GgQuaternion operator*` (const `GgVector` &v) const
- `GgQuaternion operator*` (const `GgQuaternion` &q) const
- `GgQuaternion operator/` (const GLfloat *a) const
- `GgQuaternion operator/` (const `GgVector` &v) const
- `GgQuaternion operator/` (const `GgQuaternion` &q) const
- `GgQuaternion & loadMatrix` (const GLfloat *a)
- `GgQuaternion & loadMatrix` (const `GgMatrix` &m)
- `GgQuaternion & loadIdentity` ()
- `GgQuaternion & loadRotate` (GLfloat x, GLfloat y, GLfloat z, GLfloat a)
- `GgQuaternion & loadRotate` (const GLfloat *v, GLfloat a)
- `GgQuaternion & loadRotate` (const GLfloat *v)
- `GgQuaternion & loadRotateX` (GLfloat a)
- `GgQuaternion & loadRotateY` (GLfloat a)
- `GgQuaternion & loadRotateZ` (GLfloat a)
- `GgQuaternion rotate` (GLfloat x, GLfloat y, GLfloat z, GLfloat a) const
- `GgQuaternion rotate` (const GLfloat *v, GLfloat a) const
- `GgQuaternion rotate` (const GLfloat *v) const
- `GgQuaternion rotateX` (GLfloat a) const
- `GgQuaternion rotateY` (GLfloat a) const
- `GgQuaternion rotateZ` (GLfloat a) const
- `GgQuaternion & loadEuler` (GLfloat heading, GLfloat pitch, GLfloat roll)
- `GgQuaternion & loadEuler` (const GLfloat *e)
- `GgQuaternion euler` (GLfloat heading, GLfloat pitch, GLfloat roll) const
- `GgQuaternion euler` (const GLfloat *e) const
- `GgQuaternion euler` (const `GgVector` &e) const
- `GgQuaternion & loadSlerp` (const GLfloat *a, const GLfloat *b, GLfloat t)
- `GgQuaternion & loadSlerp` (const `GgQuaternion` &q, const `GgQuaternion` &r, GLfloat t)
- `GgQuaternion & loadSlerp` (const `GgQuaternion` &q, const GLfloat *a, GLfloat t)
- `GgQuaternion & loadSlerp` (const GLfloat *a, const `GgQuaternion` &q, GLfloat t)
- `GgQuaternion & loadNormalize` (const GLfloat *a)
- `GgQuaternion & loadNormalize` (const `GgQuaternion` &q)
- `GgQuaternion & loadConjugate` (const GLfloat *a)
- `GgQuaternion & loadConjugate` (const `GgQuaternion` &q)
- `GgQuaternion & loadInvert` (const GLfloat *a)
- `GgQuaternion & loadInvert` (const `GgQuaternion` &q)
- `GgQuaternion slerp` (GLfloat *a, GLfloat t) const
- `GgQuaternion slerp` (const `GgQuaternion` &q, GLfloat t) const
- `GgQuaternion normalize` () const
- `GgQuaternion conjugate` () const
- `GgQuaternion invert` () const
- void `get` (GLfloat *a) const
- void `getMatrix` (GLfloat *a) const
- void `getMatrix` (`GgMatrix` &m) const
- `GgMatrix getMatrix` () const
- void `getConjugateMatrix` (GLfloat *a) const
- void `getConjugateMatrix` (`GgMatrix` &m) const
- `GgMatrix getConjugateMatrix` () const

基底クラス **gg::GgVector** に属する継承公開メンバ関数

- **GgVector** ()=default
- constexpr **GgVector** (GLfloat v0, GLfloat v1, GLfloat v2, GLfloat v3)
- constexpr **GgVector** (const GLfloat *a)
- constexpr **GgVector** (GLfloat c)
- **GgVector** (const GgVector &v)=default
- **GgVector** (GgVector &&v)=default
- ~**GgVector** ()=default
- **GgVector** & operator= (const **GgVector** &v)=default
- **GgVector** & operator= (**GgVector** &&v)=default
- constexpr **GgVector** operator+ (const **GgVector** &v) const noexcept
- constexpr **GgVector** operator+ (GLfloat c) const noexcept
- constexpr **GgVector** & operator+= (const **GgVector** &v) noexcept
- constexpr **GgVector** & operator+= (GLfloat c) noexcept
- constexpr **GgVector** operator- (const **GgVector** &v) const noexcept
- constexpr **GgVector** operator- (GLfloat c) const noexcept
- constexpr **GgVector** & operator-= (const **GgVector** &v) noexcept
- constexpr **GgVector** & operator-= (GLfloat c) noexcept
- constexpr **GgVector** operator* (const **GgVector** &v) const noexcept
- constexpr **GgVector** operator* (GLfloat c) const noexcept
- constexpr **GgVector** & operator*= (const **GgVector** &v) noexcept
- constexpr **GgVector** & operator*= (GLfloat c) noexcept
- constexpr **GgVector** operator/ (const **GgVector** &v) const noexcept
- constexpr **GgVector** operator/ (GLfloat c) const noexcept
- constexpr **GgVector** & operator/= (const **GgVector** &v) noexcept
- constexpr **GgVector** & operator/= (GLfloat c) noexcept
- GLfloat dot3 (const **GgVector** &v) const
- GLfloat length3 () const
- GLfloat distance3 (const **GgVector** &v) const
- **GgVector** normalize3 () const
- GLfloat dot4 (const **GgVector** &v) const
- GLfloat length4 () const
- GLfloat distance4 (const **GgVector** &v) const
- **GgVector** normalize4 () const

9.10.1 詳解

四元数.

[gg.h](#) の 3483 行目に定義があります。

9.10.2 構築子と解体子

9.10.2.1 GgQuaternion() [1/7]

```
gg::GgQuaternion::GgQuaternion () [default]
コンストラクタ.
```

9.10.2.2 GgQuaternion() [2/7]

```
gg::GgQuaternion::GgQuaternion (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) [inline], [constexpr]
コンストラクタ.
```

引数

<i>x</i>	四元数の <i>x</i> 要素.
<i>y</i>	四元数の <i>y</i> 要素.
<i>z</i>	四元数の <i>z</i> 要素.
<i>w</i>	四元数の <i>w</i> 要素.

[gg.h](#) の 3512 行目に定義があります。
呼び出し関係図:



9.10.2.3 GgQuaternion() [3/7]

```
gg::GgQuaternion::GgQuaternion (
    GLfloat c) [inline], [constexpr]
```

コンストラクタ.

引数

<i>c</i>	GLfloat 型の値.
----------	--------------

[gg.h](#) の 3522 行目に定義があります。
呼び出し関係図:



9.10.2.4 GgQuaternion() [4/7]

```
gg::GgQuaternion::GgQuaternion (
    const GLfloat * a) [inline]
```

コンストラクタ.

引数

<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
----------	--------------------------------

[gg.h](#) の 3532 行目に定義があります。

呼び出し関係図:



9.10.2.5 GgQuaternion() [5/7]

```
gg::GgQuaternion::GgQuaternion (
    const GgVector & v) [inline]
```

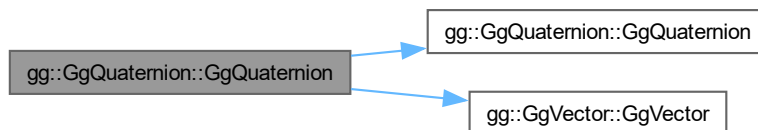
コンストラクタ.

引数

<code>v</code>	四元数を格納した <code>GgVector</code> 型の変数.
----------------	--------------------------------------

`gg.h` の 3542 行目に定義があります。

呼び出し関係図:



9.10.2.6 GgQuaternion() [6/7]

```
gg::GgQuaternion::GgQuaternion (
    const GgQuaternion & q) [default]
```

コピーコンストラクタ.

引数

<code>q</code>	コピー元の四元数.
----------------	-----------

呼び出し関係図:



9.10.2.7 GgQuaternion() [7/7]

```
gg::GgQuaternion::GgQuaternion (
    GgQuaternion && q) [default]
ムーブコンストラクタ.
```

引数

<i>q</i>	ムーブ元の四元数.
----------	-----------

呼び出し関係図:



9.10.2.8 ~GgQuaternion()

```
gg::GgQuaternion::~~GgQuaternion () [default]
デストラクタ.
```

9.10.3 関数詳解

9.10.3.1 add() [1/4]

```
GgQuaternion gg::GgQuaternion::add (
    const GgQuaternion & q) const [inline]
四元数に別の四元数を加算した結果を返す.
```

引数

<i>q</i>	<code>GgQuaternion</code> 型の四元数.
----------	----------------------------------

戻り値

q を加えた四元数.

`gg.h` の 3835 行目に定義があります。

呼び出し関係図:



9.10.3.2 add() [2/4]

```
GgQuaternion gg::GgQuaternion::add (
    const GgVector & v) const [inline]
```

四元数に別の四元数を加算した結果を返す.

引数

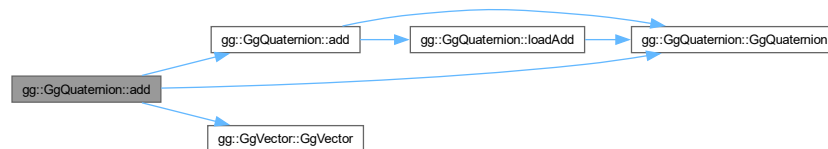
v	四元数を格納した GgVector 型の変数.
----------	--------------------------------

戻り値

v を加えた四元数.

gg.h の 3824 行目に定義があります.

呼び出し関係図:



9.10.3.3 add() [3/4]

```
GgQuaternion gg::GgQuaternion::add (
    const GLfloat * a) const [inline]
```

四元数に別の四元数を加算した結果を返す.

引数

a	四元数を格納した GLfloat 型の 4 要素の配列変数.
----------	---------------------------------------

戻り値

a を加えた四元数.

gg.h の 3813 行目に定義があります.

呼び出し関係図:



9.10.3.4 add() [4/4]

```
GgQuaternion gg::GgQuaternion::add (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) const [inline]
```

四元数に別の四元数を加算した結果を返す.

引数

x	加える四元数の x 要素.
y	加える四元数の y 要素.
z	加える四元数の z 要素.
w	加える四元数の w 要素.

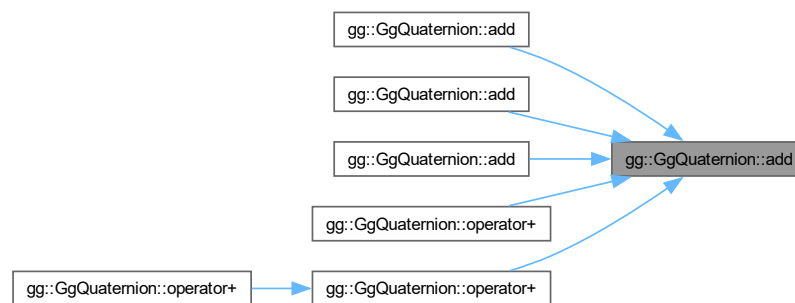
戻り値

(x , y , z , w) を加えた四元数.

gg.h の 3801 行目に定義があります。
呼び出し関係図:



被呼び出し関係図:



9.10.3.5 conjugate()

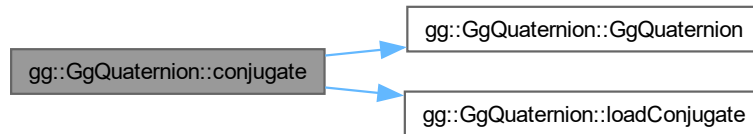
`GgQuaternion gg::GgQuaternion::conjugate () const [inline]`
共役四元数に変換する.

戻り値

共役四元数.

[gg.h](#) の 4472 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.10.3.6 divide() [1/4]

```
GgQuaternion gg::GgQuaternion::divide (
    const GgQuaternion & q) const [inline]
```

四元数を別の四元数で除算した結果を返す.

引数

<i>q</i>	GgQuaternion 型の四元数.
----------	-------------------------------------

戻り値

q で割った四元数.

[gg.h](#) の 3984 行目に定義があります。

呼び出し関係図:



9.10.3.7 divide() [2/4]

```
GgQuaternion gg::GgQuaternion::divide (
    const GgVector & v) const [inline]
```

四元数を別の四元数で除算した結果を返す.

引数

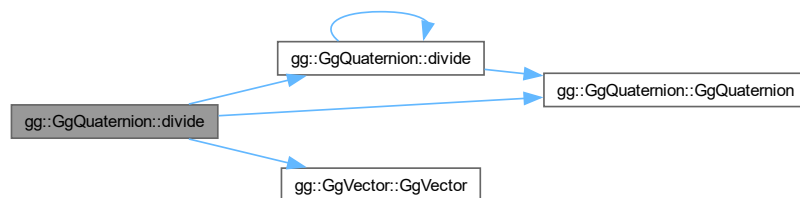
v	四元数を格納した GgVector 型の変数.
---	--------------------------------

戻り値

v で割った四元数.

gg.h の 3973 行目に定義があります。

呼び出し関係図:



9.10.3.8 divide() [3/4]

```
GgQuaternion gg::GgQuaternion::divide (
    const GLfloat * a) const [inline]
```

四元数を別の四元数で除算した結果を返す.

引数

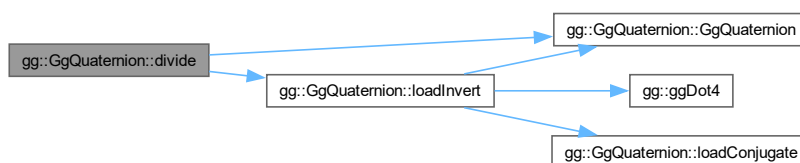
a	四元数を格納した GLfloat 型の 4 要素の配列変数.
---	--------------------------------

戻り値

a で割った四元数.

gg.h の 3959 行目に定義があります。

呼び出し関係図:



9.10.3.9 divide() [4/4]

```
GgQuaternion gg::GgQuaternion::divide (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) const [inline]
```

四元数を別の四元数で除算した結果を返す.

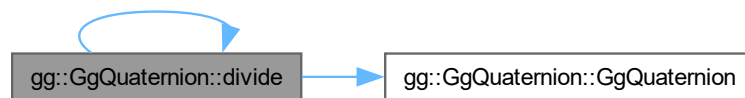
引数

x	割る四元数の x 要素.
y	割る四元数の y 要素.
z	割る四元数の z 要素.
w	割る四元数の w 要素.

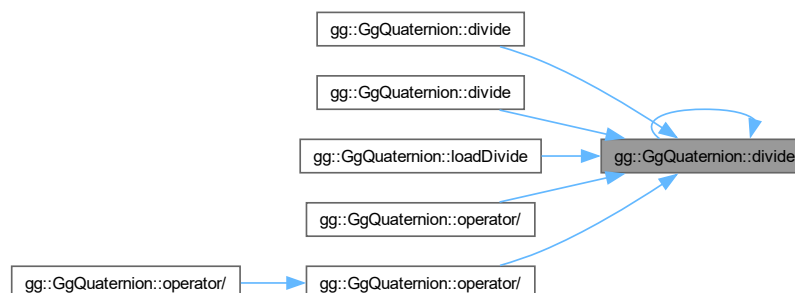
戻り値

(x, y, z, w) を割った四元数.

[gg.h](#) の 3947 行目に定義があります。
呼び出し関係図:



被呼び出し関係図:



9.10.3.10 euler() [1/3]

```
GgQuaternion gg::GgQuaternion::euler (
    const GgVector & e) const [inline]
```

四元数をオイラー角 ($e[0], e[1], e[2]$) で回転した四元数を返す.

引数

e	オイラー角を表す GgVector 型の変数 (heading, pitch, roll) の参照.
---	--

戻り値

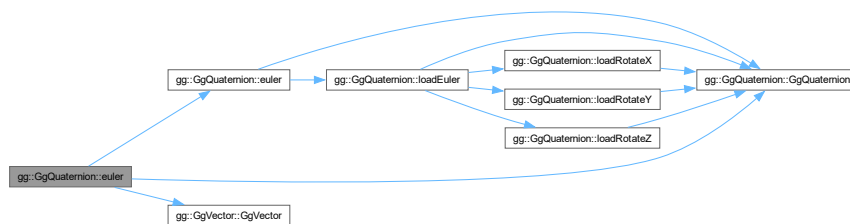
回転した四元数.

覚え書き

第 4 要素は無視する.

[gg.h](#) の 4312 行目に定義があります.

呼び出し関係図:



9.10.3.11 euler() [2/3]

```
GgQuaternion gg::GgQuaternion::euler (
    const GLfloat * e) const [inline]
```

四元数をオイラー角 (e[0], e[1], e[2]) で回転した四元数を返す.

引数

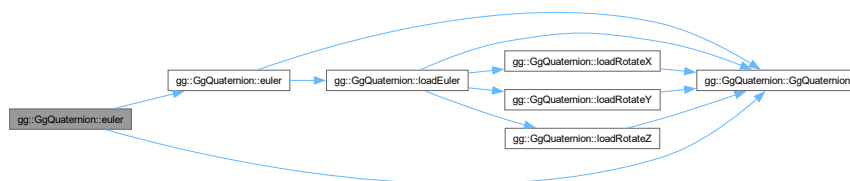
e	オイラー角を表す GLfloat 型の 3 要素の配列変数 (heading, pitch, roll).
---	---

戻り値

回転した四元数.

[gg.h](#) の 4298 行目に定義があります.

呼び出し関係図:



9.10.3.12 euler() [3/3]

```
GgQuaternion gg::GgQuaternion::euler (
    GLfloat heading,
    GLfloat pitch,
    GLfloat roll) const [inline]
```

四元数をオイラー角 (heading, pitch, roll) で回転した四元数を返す.

引数

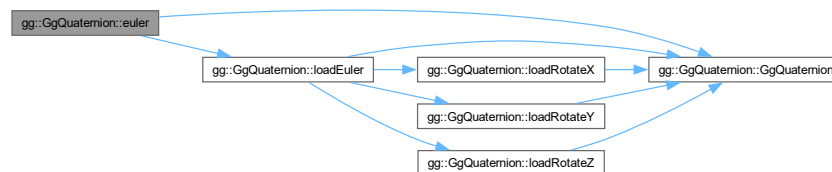
<i>heading</i>	y 軸中心の回転角.
<i>pitch</i>	x 軸中心の回転角.
<i>roll</i>	z 軸中心の回転角.

戻り値

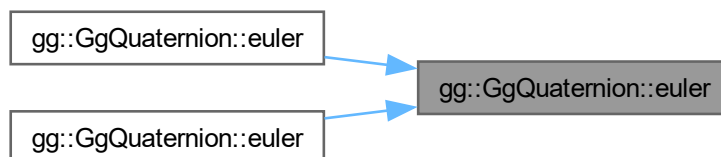
回転した四元数.

[gg.h](#) の 4286 行目に定義があります.

呼び出し関係図:



被呼び出し関係図:



9.10.3.13 get()

```
void gg::GgQuaternion::get (
    GLfloat * a) const [inline]
```

四元数を取り出す.

引数

<i>a</i>	四元数を格納する GLfloat 型の 4 要素の配列変数.
----------	--------------------------------

gg.h の 4496 行目に定義があります。

9.10.3.14 getConjugateMatrix() [1/3]

```
GgMatrix gg::GgQuaternion::getConjugateMatrix () const [inline]
```

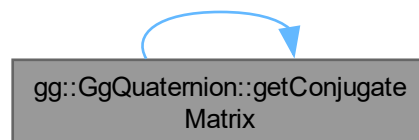
四元数の共役が表す回転の変換行列を取り出す.

戻り値

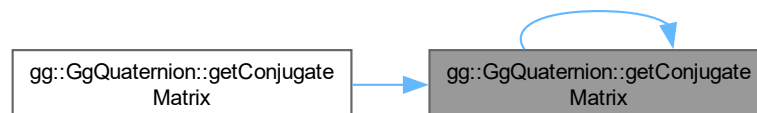
回転の変換を表す GgMatrix 型の変換行列.

gg.h の 4563 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.10.3.15 getConjugateMatrix() [2/3]

```
void gg::GgQuaternion::getConjugateMatrix (
    GgMatrix & m) const [inline]
```

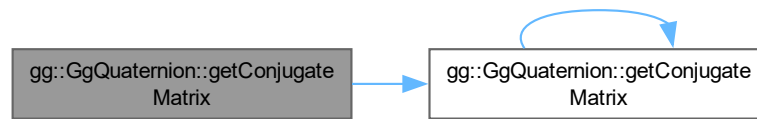
四元数の共役が表す回転の変換行列を m に求める.

引数

<i>m</i>	回転の変換行列を格納する GgMatrix 型の変数.
----------	-----------------------------

gg.h の 4553 行目に定義があります。

呼び出し関係図:



9.10.3.16 getConjugateMatrix() [3/3]

```
void gg::GgQuaternion::getConjugateMatrix (
    GLfloat * a) const [inline]
```

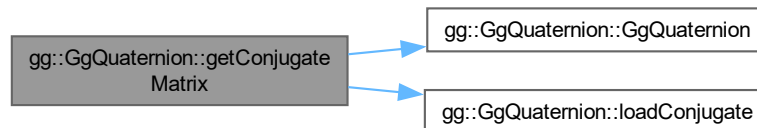
四元数の共役が表す回転の変換行列を **a** に求める.

引数

a	回転の変換行列を格納する GLfloat 型の 16 要素の配列変数.
----------	-------------------------------------

[gg.h](#) の [4541](#) 行目に定義があります。

呼び出し関係図:



9.10.3.17 getMatrix() [1/3]

```
GgMatrix gg::GgQuaternion::getMatrix () const [inline]
```

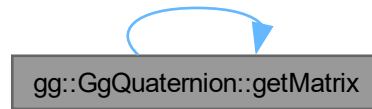
四元数が表す回転の変換行列を取り出す.

戻り値

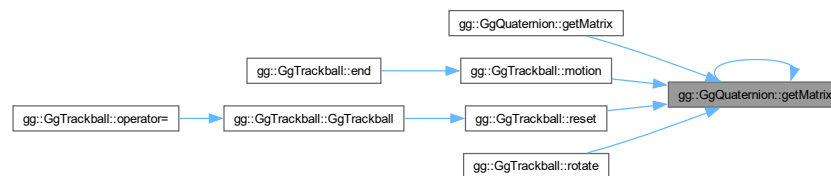
回転の変換を表す **GgMatrix** 型の変換行列.

gg.h の 4529 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.10.3.18 getMatrix() [2/3]

```
void gg::GgQuaternion::getMatrix (
    GgMatrix & m) const [inline]
```

四元数が表す回転の変換行列を m に求める。

引数

<i>m</i>	回転の変換行列を格納する GgMatrix 型の変数.
----------	------------------------------------

gg.h の 4519 行目に定義があります。

呼び出し関係図:



9.10.3.19 getMatrix() [3/3]

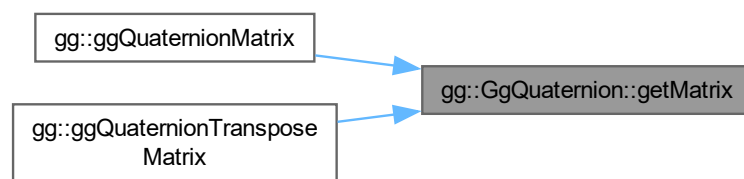
```
void gg::GgQuaternion::getMatrix (
    GLfloat * a) const [inline]
```

四元数が表す回転の変換行列を **a** に求める。

引数

a	回転の変換行列を格納する GLfloat 型の 16 要素の配列変数.
----------	-------------------------------------

[gg.h](#) の 4509 行目に定義があります。
被呼び出し関係図:



9.10.3.20 invert()

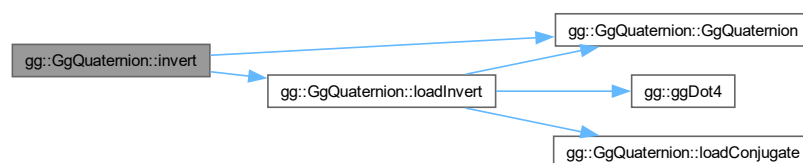
```
GgQuaternion gg::GgQuaternion::invert () const [inline]
```

逆元に変換する。

戻り値

四元数の逆元。

[gg.h](#) の 4484 行目に定義があります。
呼び出し関係図:



被呼び出し関係図:



9.10.3.21 loadAdd() [1/4]

```
GgQuaternion & gg::GgQuaternion::loadAdd (
    const GgQuaternion & q) [inline]
```

四元数に別の四元数を加算した結果を格納する.

引数

<i>q</i>	GgQuaternion 型の四元数.
----------	---------------------

戻り値

q を加えた四元数.

gg.h の 3639 行目に定義があります。
呼び出し関係図:



9.10.3.22 loadAdd() [2/4]

```
GgQuaternion & gg::GgQuaternion::loadAdd (
    const GgVector & v) [inline]
```

四元数に別の四元数を加算した結果を格納する.

引数

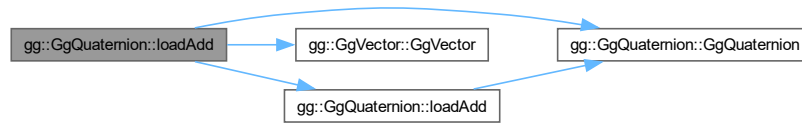
<i>v</i>	四元数を格納した GgVector 型の変数.
----------	-------------------------

戻り値

v を加えた四元数.

gg.h の 3628 行目に定義があります。

呼び出し関係図:



9.10.3.23 loadAdd() [3/4]

```
GgQuaternion & gg::GgQuaternion::loadAdd (
    const GLfloat * a) [inline]
```

四元数に別の四元数を加算した結果を格納する.

引数

<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
----------	--------------------------------

戻り値

a を加えた四元数.

[gg.h](#) の 3617 行目に定義があります。

呼び出し関係図:



9.10.3.24 loadAdd() [4/4]

```
GgQuaternion & gg::GgQuaternion::loadAdd (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) [inline]
```

四元数に別の四元数を加算した結果を格納する.

引数

<i>x</i>	加える四元数の <i>x</i> 要素.
<i>y</i>	加える四元数の <i>y</i> 要素.
<i>z</i>	加える四元数の <i>z</i> 要素.
<i>w</i>	加える四元数の <i>w</i> 要素.

戻り値

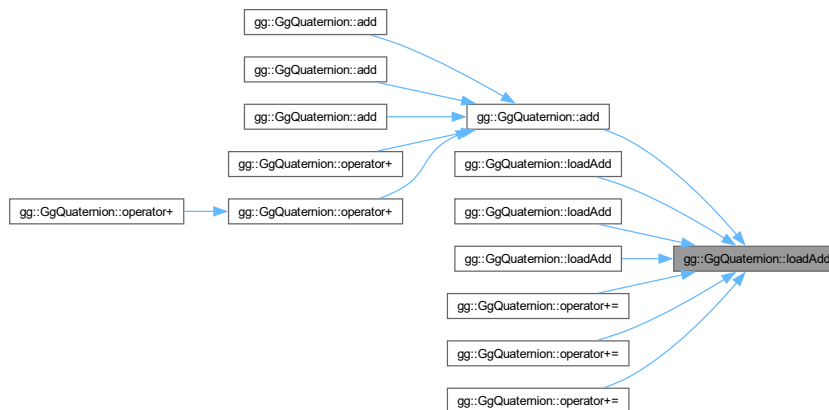
(x, y, z, w) を加えた四元数.

gg.h の 3601 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.10.3.25 loadConjugate() [1/2]

```
GgQuaternion & gg::GgQuaternion::loadConjugate (
    const GgQuaternion & q) [inline]
```

引数に指定した四元数の共役四元数を格納する。

引数

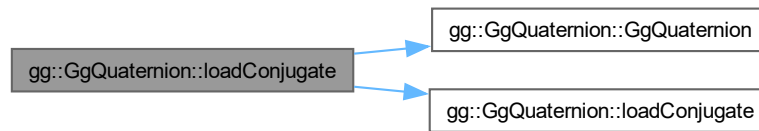
<i>q</i>	GgQuaternion 型の四元数.
----------	---------------------

戻り値

共役四元数.

[gg.h](#) の 4403 行目に定義があります。

呼び出し関係図:



9.10.3.26 loadConjugate() [2/2]

```
gg::GgQuaternion & gg::GgQuaternion::loadConjugate (
    const GLfloat * a)
```

引数に指定した四元数の共役四元数を格納する.

引数

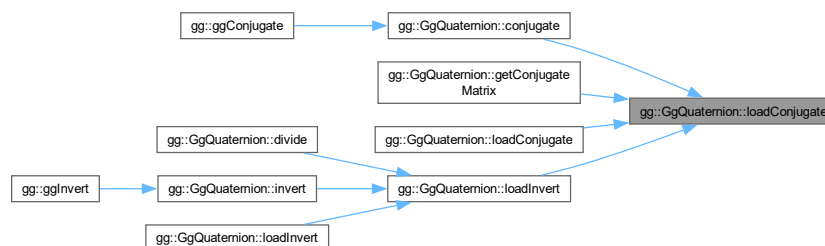
a	四元数を格納した GLfloat 型の 4 要素の配列変数.
----------	--------------------------------

戻り値

共役四元数.

[gg.cpp](#) の 3393 行目に定義があります。

被呼び出し関係図:



9.10.3.27 loadDivide() [1/4]

```
GgQuaternion & gg::GgQuaternion::loadDivide (
    const GgQuaternion & q) [inline]
```

四元を別の四元数で除算した結果を格納する.

引数

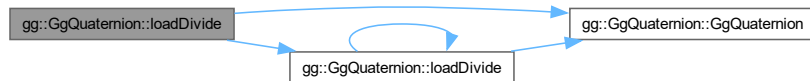
q	GgQuaternion 型の四元数.
-----	---------------------

戻り値

q で割った四元数.

gg.h の 3787 行目に定義があります。

呼び出し関係図:



9.10.3.28 loadDivide() [2/4]

```
GgQuaternion & gg::GgQuaternion::loadDivide (
    const GgVector & v) [inline]
```

四元を別の四元数で除算した結果を格納する.

引数

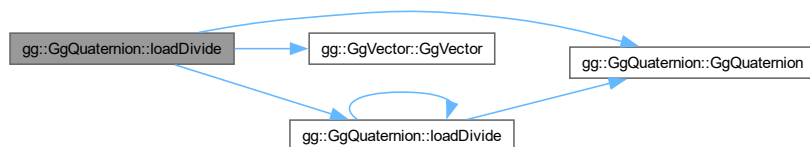
v	四元数を格納した GgVector 型の変数.
-----	-------------------------

戻り値

v で割った四元数.

gg.h の 3776 行目に定義があります。

呼び出し関係図:



9.10.3.29 loadDivide() [3/4]

```
GgQuaternion & gg::GgQuaternion::loadDivide (
    const GLfloat * a) [inline]
```

四元を別の四元数で除算した結果を格納する.

引数

a	四元数を格納した GLfloat 型の 4 要素の配列変数.
-----	--------------------------------

戻り値

a で割った四元数.

gg.h の 3765 行目に定義があります。

呼び出し関係図:



9.10.3.30 loadDivide() [4/4]

```

GgQuaternion & gg::GgQuaternion::loadDivide (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) [inline]
  
```

四元を別の四元数で除算した結果を格納する.

引数

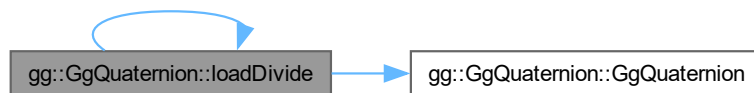
<i>x</i>	割る四元数の x 要素.
<i>y</i>	割る四元数の y 要素.
<i>z</i>	割る四元数の z 要素.
<i>w</i>	割る四元数の w 要素.

戻り値

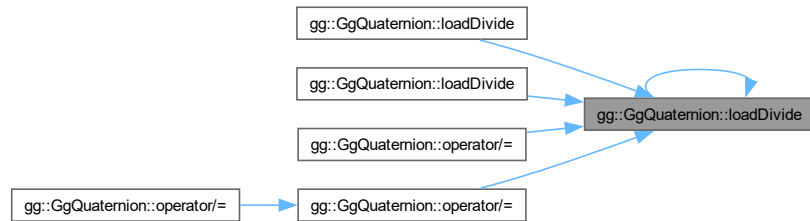
(x, y, z, w) を割った四元数.

gg.h の 3753 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.10.3.31 loadEuler() [1/2]

```
GgQuaternion & gg::GgQuaternion::loadEuler (
    const GLfloat * e) [inline]
```

オイラー角 (e[0], e[1], e[2]) で与えられた回転を表す四元数を格納する.

引数

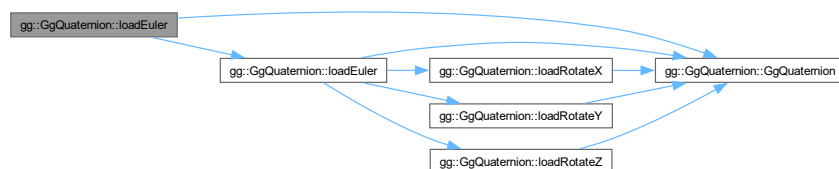
<i>e</i>	オイラー角を表す GLfloat 型の 3 要素の配列変数 (heading, pitch, roll).
----------	---

戻り値

格納した回転を表す四元数.

gg.h の 4273 行目に定義があります.

呼び出し関係図:



9.10.3.32 loadEuler() [2/2]

```
gg::GgQuaternion & gg::GgQuaternion::loadEuler (
    GLfloat heading,
    GLfloat pitch,
    GLfloat roll)
```

オイラー角 (heading, pitch, roll) で与えられた回転を表す四元数を格納する.

引数

<i>heading</i>	y 軸中心の回転角.
<i>pitch</i>	x 軸中心の回転角.

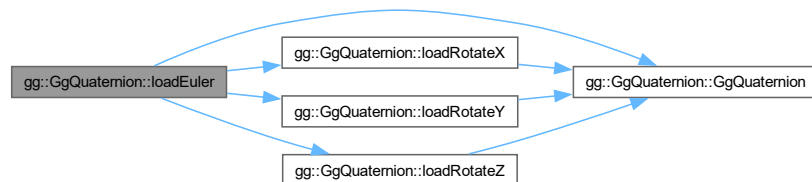
<i>roll</i>	z 軸中心の回転角.
-------------	------------

戻り値

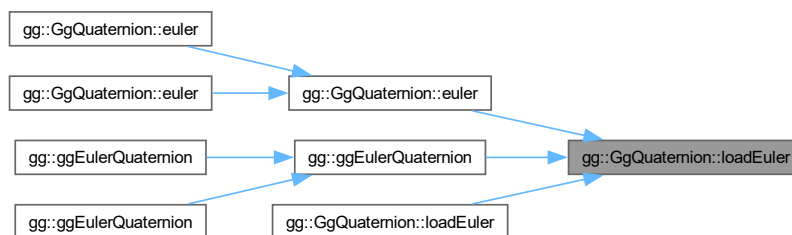
格納した回転を表す四元数.

[gg.cpp](#) の 3365 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.10.3.33 loadIdentity()

[GgQuaternion](#) & [gg::GgQuaternion::loadIdentity](#) () [inline]

単位元を格納する.

戻り値

格納された単位元.

[gg.h](#) の 4123 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.10.3.34 loadInvert() [1/2]

```
GgQuaternion & gg::GgQuaternion::loadInvert (
    const GgQuaternion & q) [inline]
```

引数に指定した四元数の逆元を格納する.

引数

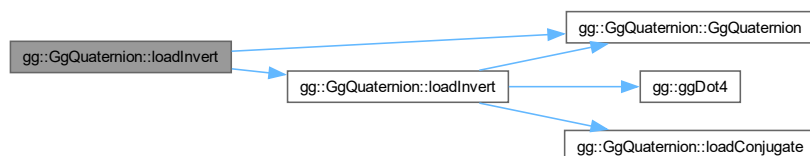
<i>q</i>	GgQuaternion 型の四元数.
----------	---------------------

戻り値

四元数の逆元.

gg.h の 4422 行目に定義があります。

呼び出し関係図:



9.10.3.35 loadInvert() [2/2]

```
gg::GgQuaternion & gg::GgQuaternion::loadInvert (
    const GLfloat * a)
```

引数に指定した四元数の逆元を格納する.

引数

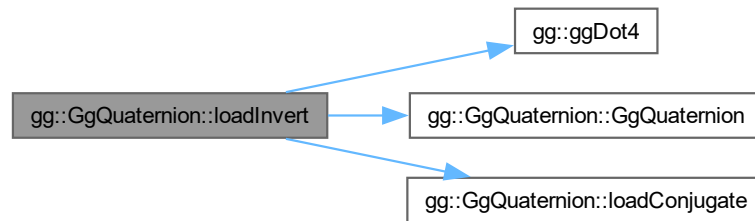
<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
----------	--------------------------------

戻り値

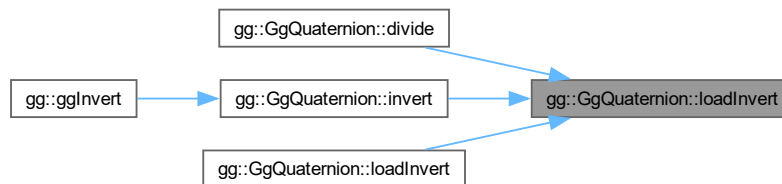
四元数の逆元.

`gg.cpp` の 3407 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.10.3.36 loadMatrix() [1/2]

```
GgQuaternion & gg::GgQuaternion::loadMatrix (
    const GgMatrix & m) [inline]
```

回転の変換行列 `m` を表す四元数を格納する.

引数

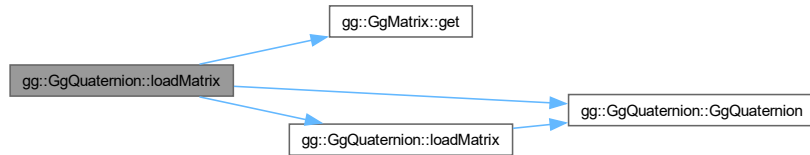
<code>m</code>	<code>GgMatrix</code> 型の変換行列.
----------------	-------------------------------

戻り値

m による回転の変換に相当する四元数.

gg.h の 4113 行目に定義があります。

呼び出し関係図:



9.10.3.37 loadMatrix() [2/2]

```
GgQuaternion & gg::GgQuaternion::loadMatrix (
    const GLfloat * a) [inline]
```

回転の変換行列を表す四元数を格納する.

引数

a	GLfloat 型の 16 要素の変換行列.
---	------------------------

戻り値

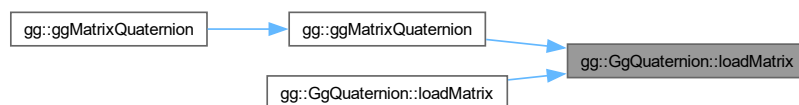
a による回転の変換に相当する四元数.

gg.h の 4101 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.10.3.38 loadMultiply() [1/4]

```
GgQuaternion & gg::GgQuaternion::loadMultiply (
    const GgQuaternion & q) [inline]
```

四元数に別の四元数を乗算した結果を格納する.

引数

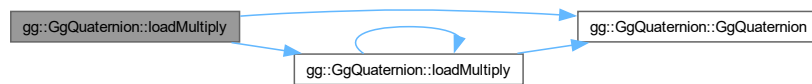
<i>q</i>	GgQuaternion 型の四元数.
----------	---------------------

戻り値

q を乗じた四元数.

gg.h の 3739 行目に定義があります。

呼び出し関係図:



9.10.3.39 loadMultiply() [2/4]

```
GgQuaternion & gg::GgQuaternion::loadMultiply (
    const GgVector & v) [inline]
```

四元数に別の四元数を乗算した結果を格納する.

引数

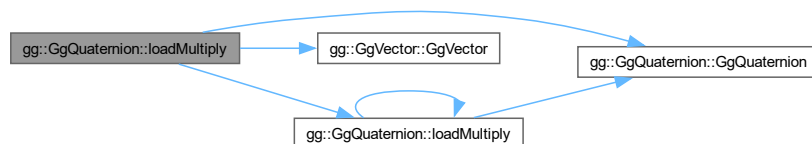
<i>v</i>	四元数を格納した GgVector 型の変数.
----------	-------------------------

戻り値

v を乗じた四元数.

gg.h の 3728 行目に定義があります。

呼び出し関係図:



9.10.3.40 loadMultiply() [3/4]

```
GgQuaternion & gg::GgQuaternion::loadMultiply (
    const GLfloat * a) [inline]
```

四元数に別の四元数を乗算した結果を格納する.

引数

<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
----------	--------------------------------

戻り値

a を乗じた四元数.

gg.h の 3717 行目に定義があります。

呼び出し関係図:



9.10.3.41 loadMultiply() [4/4]

```

GgQuaternion & gg::GgQuaternion::loadMultiply (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) [inline]
  
```

四元数に別の四元数を乗算した結果を格納する.

引数

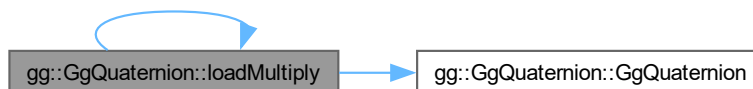
<i>x</i>	掛ける四元数の <i>x</i> 要素.
<i>y</i>	掛ける四元数の <i>y</i> 要素.
<i>z</i>	掛ける四元数の <i>z</i> 要素.
<i>w</i>	掛ける四元数の <i>w</i> 要素.

戻り値

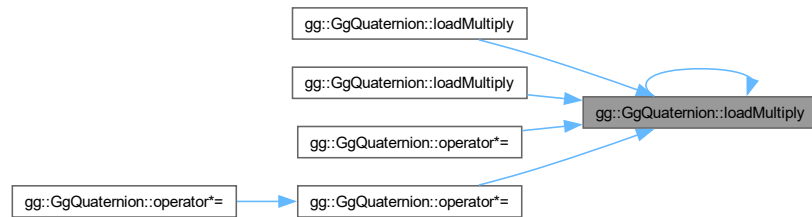
(*x*, *y*, *z*, *w*) を掛けた四元数.

gg.h の 3705 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.10.3.42 loadNormalize() [1/2]

```
GgQuaternion & gg::GgQuaternion::loadNormalize (
    const GgQuaternion & q) [inline]
```

引数に指定した四元数を正規化して格納する.

引数

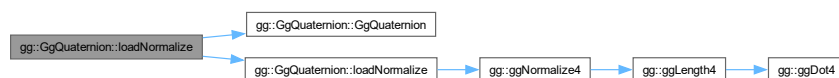
<i>q</i>	GgQuaternion 型の四元数.
----------	---------------------

戻り値

正規化された四元数.

gg.h の 4384 行目に定義があります.

呼び出し関係図:



9.10.3.43 loadNormalize() [2/2]

```
gg::GgQuaternion & gg::GgQuaternion::loadNormalize (
    const GLfloat * a)
```

引数に指定した四元数を正規化して格納する.

引数

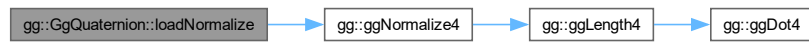
<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
----------	--------------------------------

戻り値

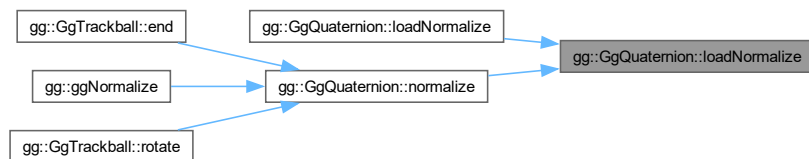
正規化された四元数.

[gg.cpp](#) の 3381 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.10.3.44 loadRotate() [1/3]

```
GgQuaternion & gg::GgQuaternion::loadRotate (
    const GLfloat * v) [inline]
```

(v[0], v[1], v[2]) を軸として角度 v[3] 回転する四元数を格納する.

引数

v	軸ベクトルと回転角を格納した GLfloat 型の 4 要素の配列変数.
---	--------------------------------------

戻り値

格納された回転を表す四元数.

[gg.h](#) の 4157 行目に定義があります。

呼び出し関係図:



9.10.3.45 loadRotate() [2/3]

```
GgQuaternion & gg::GgQuaternion::loadRotate (
    const GLfloat * v,
    GLfloat a) [inline]
```

(v[0], v[1], v[2]) を軸として角度 a 回転する四元数を格納する.

引数

<i>v</i>	軸ベクトルを表す GLfloat 型の 3 要素の配列変数.
<i>a</i>	回転角.

戻り値

格納された回転を表す四元数.

gg.h の 4146 行目に定義があります。
呼び出し関係図:



9.10.3.46 loadRotate() [3/3]

```

gg::GgQuaternion & gg::GgQuaternion::loadRotate (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat a)

```

(x, y, z) を軸として角度 a 回転する四元数を格納する.

引数

<i>x</i>	軸ベクトルの x 成分.
<i>y</i>	軸ベクトルの y 成分.
<i>z</i>	軸ベクトルの z 成分.
<i>a</i>	回転角.

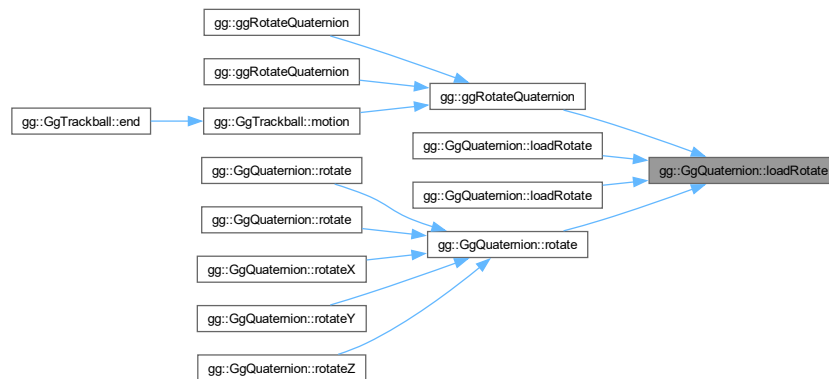
戻り値

格納された回転を表す四元数.

gg.cpp の 3307 行目に定義があります。
呼び出し関係図:



被呼び出し関係図:



9.10.3.47 loadRotateX()

`gg::GgQuaternion & gg::GgQuaternion::loadRotateX (GLfloat a)`

x 軸中心に角度 *a* 回転する四元数を格納する.

引数

<i>a</i>	回転角.
----------	------

戻り値

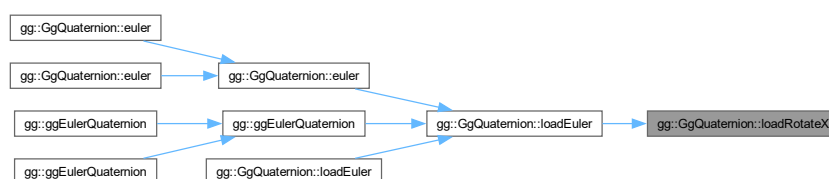
格納された回転を表す四元数.

`gg.cpp` の 3329 行目に定義があります.

呼び出し関係図:



被呼び出し関係図:



9.10.3.48 loadRotateY()

```
gg::GgQuaternion & gg::GgQuaternion::loadRotateY (
    GLfloat a)
```

y 軸中心に角度 **a** 回転する四元数を格納する.

引数

a	回転角.
----------	------

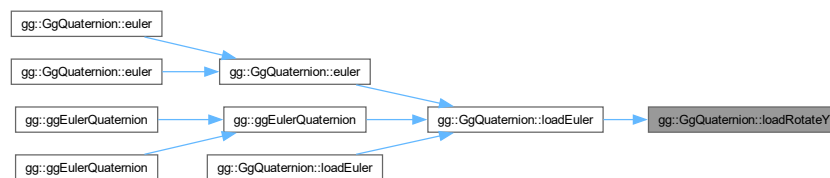
戻り値

格納された回転を表す四元数.

[gg.cpp](#) の 3341 行目に定義があります。
呼び出し関係図:



被呼び出し関係図:



9.10.3.49 loadRotateZ()

```
gg::GgQuaternion & gg::GgQuaternion::loadRotateZ (
    GLfloat a)
```

z 軸中心に角度 **a** 回転する四元数を格納する.

引数

a	回転角.
----------	------

戻り値

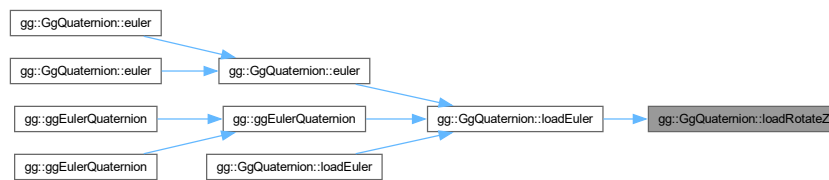
格納された回転を表す四元数.

gg.cpp の 3353 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.10.3.50 loadSlerp() [1/4]

```

GgQuaternion & gg::GgQuaternion::loadSlerp (
    const GgQuaternion & q,
    const GgQuaternion & r,
    GLfloat t) [inline]
  
```

球面線形補間の結果を格納する.

引数

<i>q</i>	GgQuaternion 型の四元数.
<i>r</i>	GgQuaternion 型の四元数.
<i>t</i>	補間パラメータ.

戻り値

格納した *q*, *r* を *t* で内分した四元数.

gg.h の 4339 行目に定義があります。

呼び出し関係図:



9.10.3.51 loadSlerp() [2/4]

```
GgQuaternion & gg::GgQuaternion::loadSlerp (
    const GgQuaternion & q,
    const GLfloat * a,
    GLfloat t) [inline]
```

球面線形補間の結果を格納する.

引数

<i>q</i>	GgQuaternion 型の四元数.
<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
<i>t</i>	補間パラメータ.

戻り値

格納した *q*, *a* を *t* で内分した四元数.

[gg.h](#) の 4352 行目に定義があります.

呼び出し関係図:



9.10.3.52 loadSlerp() [3/4]

```
GgQuaternion & gg::GgQuaternion::loadSlerp (
    const GLfloat * a,
    const GgQuaternion & q,
    GLfloat t) [inline]
```

球面線形補間の結果を格納する.

引数

<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
<i>q</i>	GgQuaternion 型の四元数.
<i>t</i>	補間パラメータ.

戻り値

格納した a, q を t で内分した四元数.

gg.h の 4365 行目に定義があります。

呼び出し関係図:



9.10.3.53 loadSlerp() [4/4]

```
GgQuaternion & gg::GgQuaternion::loadSlerp (
    const GLfloat * a,
    const GLfloat * b,
    GLfloat t) [inline]
```

球面線形補間の結果を格納する.

引数

a	四元数を格納した GLfloat 型の 4 要素の配列変数.
b	四元数を格納した GLfloat 型の 4 要素の配列変数.
t	補間パラメータ.

戻り値

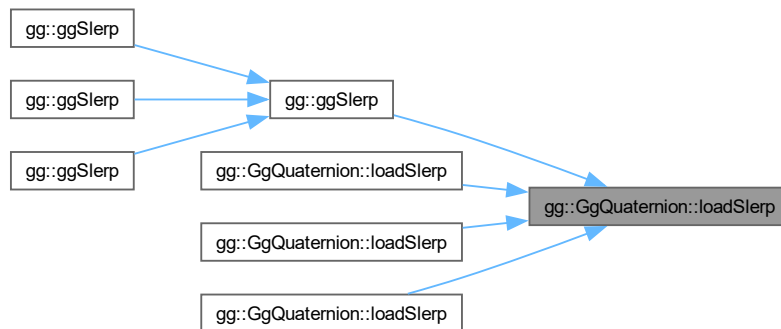
格納した a, b を t で内分した四元数.

gg.h の 4325 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.10.3.54 loadSubtract() [1/4]

```
GgQuaternion & gg::GgQuaternion::loadSubtract (
    const GgQuaternion & q) [inline]
```

四元数から別の四元数を減算した結果を格納する。

引数

<i>q</i>	<code>GgQuaternion</code> 型の四元数.
----------	----------------------------------

戻り値

q を引いた四元数.

`gg.h` の 3691 行目に定義があります。

呼び出し関係図:



9.10.3.55 loadSubtract() [2/4]

```
GgQuaternion & gg::GgQuaternion::loadSubtract (
    const GgVector & v) [inline]
```

四元数から別の四元数を減算した結果を格納する。

引数

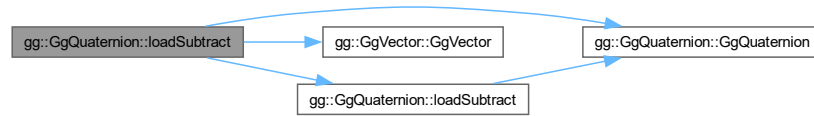
<i>v</i>	四元数を格納した <code>GgVector</code> 型の変数.
----------	--------------------------------------

戻り値

v を引いた四元数.

gg.h の 3680 行目に定義があります.

呼び出し関係図:



9.10.3.56 loadSubtract() [3/4]

```
GgQuaternion & gg::GgQuaternion::loadSubtract (
    const GLfloat * a) [inline]
```

四元数から別の四元数を減算した結果を格納する.

引数

a	四元数を格納した GLfloat 型の 4 要素の配列変数.
-----	--------------------------------

戻り値

a を引いた四元数.

gg.h の 3669 行目に定義があります.

呼び出し関係図:



9.10.3.57 loadSubtract() [4/4]

```
GgQuaternion & gg::GgQuaternion::loadSubtract (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) [inline]
```

四元数から別の四元数を減算した結果を格納する.

引数

x	引く四元数の x 要素.
y	引く四元数の y 要素.
z	引く四元数の z 要素.

戻り値

q を掛けた四元数.

gg.h の 3933 行目に定義があります。

呼び出し関係図:



9.10.3.59 multiply() [2/4]

```
GgQuaternion gg::GgQuaternion::multiply (
    const GgVector & v) const [inline]
```

四元数に別の四元数を乗算した結果を返す.

引数

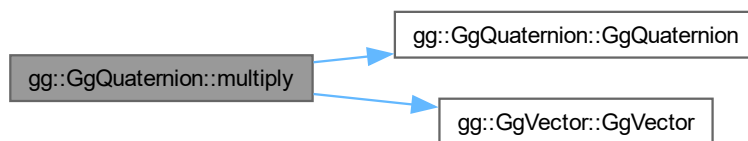
v	四元数を格納した GgVector 型の変数.
---	---

戻り値

v を掛けた四元数.

gg.h の 3922 行目に定義があります。

呼び出し関係図:



9.10.3.60 multiply() [3/4]

```
GgQuaternion gg::GgQuaternion::multiply (
    const GLfloat * a) const [inline]
```

四元数に別の四元数を乗算した結果を返す.

引数

a	四元数を格納した GLfloat 型の 4 要素の配列変数.
---	--------------------------------

戻り値

a を掛けた四元数.

gg.h の 3909 行目に定義があります。

呼び出し関係図:



9.10.3.61 multiply() [4/4]

```
GgQuaternion gg::GgQuaternion::multiply (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) const [inline]
```

四元数に別の四元数を乗算した結果を返す.

引数

x	掛ける四元数の x 要素.
y	掛ける四元数の y 要素.
z	掛ける四元数の z 要素.
w	掛ける四元数の w 要素.

戻り値

(x, y, z, w) を掛けた四元数.

gg.h の 3897 行目に定義があります。

呼び出し関係図:



9.10.3.62 norm()

```
GLfloat gg::GgQuaternion::norm () const [inline]
```

四元数のノルムを求める.

戻り値

四元数のノルム.

gg.h の 3587 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.10.3.63 normalize()

```
GgQuaternion gg::GgQuaternion::normalize () const [inline]
```

正規化する.

戻り値

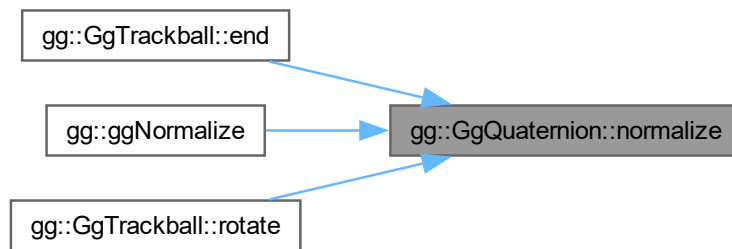
正規化された四元数.

gg.h の 4460 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.10.3.64 `operator*()` [1/3]

```
GgQuaternion gg::GgQuaternion::operator* (
    const GgQuaternion & q) const [inline]
```

`gg.h` の 4078 行目に定義があります。

呼び出し関係図:

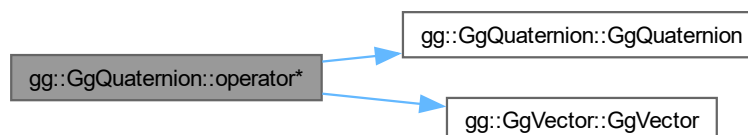


9.10.3.65 `operator*()` [2/3]

```
GgQuaternion gg::GgQuaternion::operator* (
    const GgVector & v) const [inline]
```

`gg.h` の 4074 行目に定義があります。

呼び出し関係図:



9.10.3.66 `operator*()` [3/3]

```
GgQuaternion gg::GgQuaternion::operator* (
    const GLfloat * a) const [inline]
```

gg.h の 4070 行目に定義があります。
呼び出し関係図:



被呼び出し関係図:



9.10.3.67 operator*=() [1/3]

```
GgQuaternion & gg::GgQuaternion::operator*= (
    const GgQuaternion & q) [inline]
```

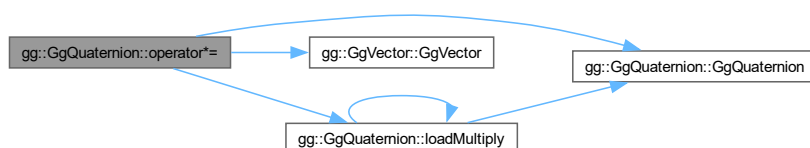
gg.h の 4030 行目に定義があります。
呼び出し関係図:



9.10.3.68 operator*=() [2/3]

```
GgQuaternion & gg::GgQuaternion::operator*= (
    const GgVector & v) [inline]
```

gg.h の 4026 行目に定義があります。
呼び出し関係図:



9.10.3.69 operator*=() [3/3]

```
GgQuaternion & gg::GgQuaternion::operator*= (
    const GLfloat * a) [inline]
```

gg.h の 4022 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.10.3.70 operator+() [1/3]

```
GgQuaternion gg::GgQuaternion::operator+ (
    const GgQuaternion & q) const [inline]
```

gg.h の 4054 行目に定義があります。

呼び出し関係図:

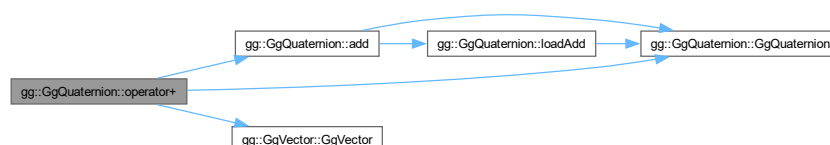


9.10.3.71 operator+() [2/3]

```
GgQuaternion gg::GgQuaternion::operator+ (
    const GgVector & v) const [inline]
```

gg.h の 4050 行目に定義があります。

呼び出し関係図:



9.10.3.72 operator+() [3/3]

```
GgQuaternion gg::GgQuaternion::operator+ (
    const GLfloat * a) const [inline]
```

gg.h の 4046 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:

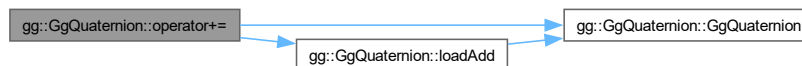


9.10.3.73 operator+=() [1/3]

```
GgQuaternion & gg::GgQuaternion::operator+= (
    const GgQuaternion & q) [inline]
```

gg.h の 4006 行目に定義があります。

呼び出し関係図:

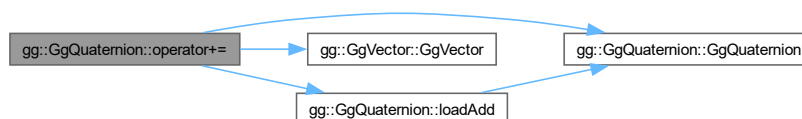


9.10.3.74 operator+=() [2/3]

```
GgQuaternion & gg::GgQuaternion::operator+= (
    const GgVector & v) [inline]
```

gg.h の 4002 行目に定義があります。

呼び出し関係図:

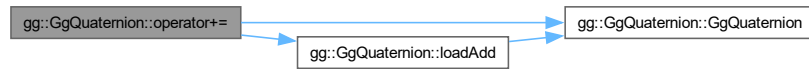


9.10.3.75 operator+=() [3/3]

```
GgQuaternion & gg::GgQuaternion::operator+= (
    const GLfloat * a) [inline]
```

gg.h の 3998 行目に定義があります。

呼び出し関係図:

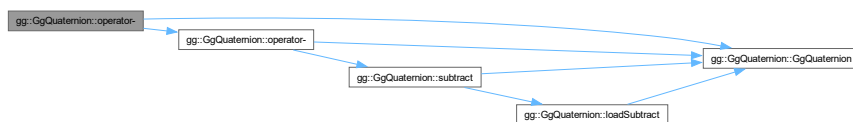


9.10.3.76 operator-() [1/3]

```
GgQuaternion gg::GgQuaternion::operator- (
    const GgQuaternion & q) const [inline]
```

gg.h の 4066 行目に定義があります。

呼び出し関係図:



9.10.3.77 operator-() [2/3]

```
GgQuaternion gg::GgQuaternion::operator- (
    const GgVector & v) const [inline]
```

gg.h の 4062 行目に定義があります。

呼び出し関係図:



9.10.3.78 operator-() [3/3]

```
GgQuaternion gg::GgQuaternion::operator- (
    const GLfloat * a) const [inline]
```

gg.h の 4058 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.10.3.79 operator-=() [1/3]

```
GgQuaternion & gg::GgQuaternion::operator-= (
    const GgQuaternion & q) [inline]
```

gg.h の 4018 行目に定義があります。

呼び出し関係図:

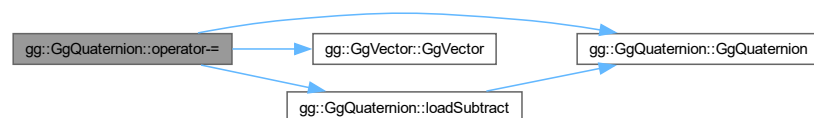


9.10.3.80 operator-=() [2/3]

```
GgQuaternion & gg::GgQuaternion::operator-= (
    const GgVector & v) [inline]
```

gg.h の 4014 行目に定義があります。

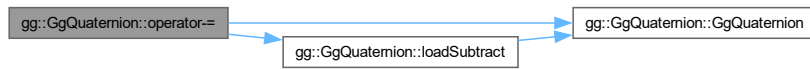
呼び出し関係図:



9.10.3.81 operator-=() [3/3]

```
GgQuaternion & gg::GgQuaternion::operator-= (
    const GLfloat * a) [inline]
```

gg.h の 4010 行目に定義があります。
呼び出し関係図:



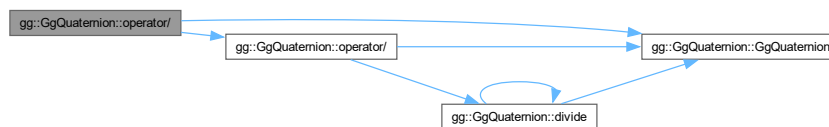
被呼び出し関係図:



9.10.3.82 operator/() [1/3]

```
GgQuaternion gg::GgQuaternion::operator/ (
    const GgQuaternion & q) const [inline]
```

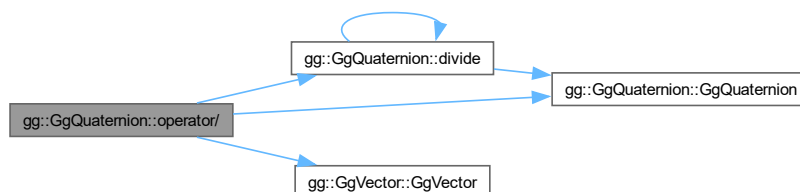
gg.h の 4090 行目に定義があります。
呼び出し関係図:



9.10.3.83 operator/() [2/3]

```
GgQuaternion gg::GgQuaternion::operator/ (
    const GgVector & v) const [inline]
```

gg.h の 4086 行目に定義があります。
呼び出し関係図:

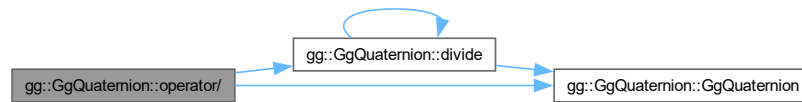


9.10.3.84 operator/() [3/3]

```
GgQuaternion gg::GgQuaternion::operator/ (
    const GLfloat * a) const [inline]
```

gg.h の 4082 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.10.3.85 operator/=() [1/3]

```
GgQuaternion & gg::GgQuaternion::operator/= (
    const GgQuaternion & q) [inline]
```

gg.h の 4042 行目に定義があります。

呼び出し関係図:

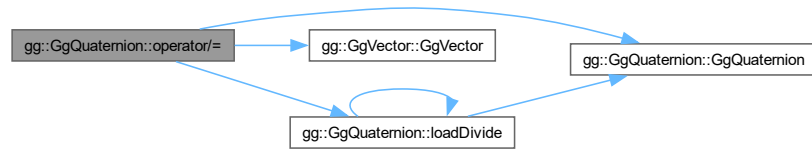


9.10.3.86 operator/=() [2/3]

```
GgQuaternion & gg::GgQuaternion::operator/= (
    const GgVector & v) [inline]
```

gg.h の 4038 行目に定義があります。

呼び出し関係図:

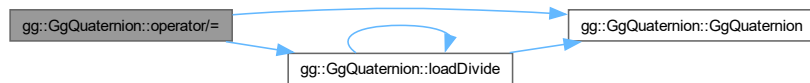


9.10.3.87 operator/=() [3/3]

```
GgQuaternion & gg::GgQuaternion::operator/= (
    const GLfloat * a) [inline]
```

gg.h の 4034 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.10.3.88 operator=() [1/4]

```
GgQuaternion & gg::GgQuaternion::operator= (
    const GgQuaternion & q) [default]
```

代入演算子.

引数

<i>q</i>	代入元の四元数.
----------	----------

戻り値

代入後のこの四元数の参照.

呼び出し関係図:



被呼び出し関係図:

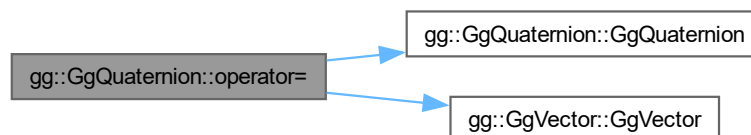


9.10.3.89 operator=() [2/4]

```
GgQuaternion & gg::GgQuaternion::operator= (  
    const GgVector & v) [inline]
```

gg.h の 3994 行目に定義があります。

呼び出し関係図:

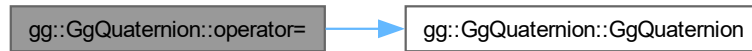


9.10.3.90 operator=() [3/4]

```
GgQuaternion & gg::GgQuaternion::operator= (  
    const GLfloat * a) [inline]
```

gg.h の 3990 行目に定義があります。

呼び出し関係図:



9.10.3.91 operator=() [4/4]

```
GgQuaternion & gg::GgQuaternion::operator= (
    GgQuaternion && q) [default]
```

ムーブ代入演算子.

引数

<i>q</i>	ムーブ代入元の四元数.
----------	-------------

戻り値

ムーブ代入後のこの四元数の参照.

呼び出し関係図:



9.10.3.92 rotate() [1/3]

```
GgQuaternion gg::GgQuaternion::rotate (
    const GLfloat * v) const [inline]
```

四元数を (v[0], v[1], v[2]) を軸として角度 v[3] 回転した四元数を返す.

引数

<i>v</i>	軸ベクトルを表す GLfloat 型の 4 要素の配列変数.
----------	--------------------------------

戻り値

回転した四元数.

gg.h の 4219 行目に定義があります。

呼び出し関係図:



9.10.3.93 rotate() [2/3]

```
GgQuaternion gg::GgQuaternion::rotate (
    const GLfloat * v,
    GLfloat a) const [inline]
```

四元数を (v[0], v[1], v[2]) を軸として角度 a 回転した四元数を返す.

引数

<i>v</i>	軸ベクトルを表す GLfloat 型の 3 要素の配列変数.
<i>a</i>	回転角.

戻り値

回転した四元数.

gg.h の 4208 行目に定義があります。

呼び出し関係図:



9.10.3.94 rotate() [3/3]

```
GgQuaternion gg::GgQuaternion::rotate (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat a) const [inline]
```

四元数を (x, y, z) を軸として角度 a 回転した四元数を返す.

引数

<i>x</i>	軸ベクトルの x 成分.
----------	--------------

<i>y</i>	軸ベクトルの <i>y</i> 成分.
<i>z</i>	軸ベクトルの <i>z</i> 成分.
<i>a</i>	回転角.

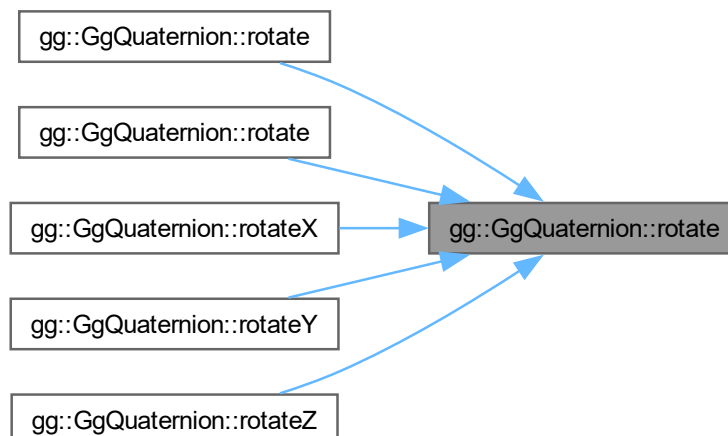
戻り値

回転した四元数.

[gg.h](#) の 4195 行目に定義があります。
呼び出し関係図:



被呼び出し関係図:



9.10.3.95 rotateX()

```
GgQuaternion gg::GgQuaternion::rotateX (
    GLfloat a) const [inline]
```

四元数を *x* 軸中心に角度 *a* 回転した四元数を返す.

引数

<i>a</i>	回転角.
----------	------

戻り値

回転した四元数.

gg.h の 4230 行目に定義があります。

呼び出し関係図:



9.10.3.96 rotateY()

```
GgQuaternion gg::GgQuaternion::rotateY (
    GLfloat a) const [inline]
```

四元数を y 軸中心に角度 a 回転した四元数を返す.

引数

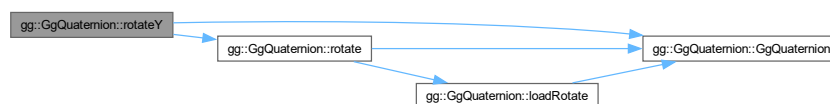
a	回転角.
---	------

戻り値

回転した四元数.

gg.h の 4241 行目に定義があります。

呼び出し関係図:



9.10.3.97 rotateZ()

```
GgQuaternion gg::GgQuaternion::rotateZ (
    GLfloat a) const [inline]
```

四元数を z 軸中心に角度 a 回転した四元数を返す.

引数

a	回転角.
---	------

戻り値

回転した四元数.

[gg.h](#) の 4252 行目に定義があります。

呼び出し関係図:



9.10.3.98 slerp() [1/2]

```
GgQuaternion gg::GgQuaternion::slerp (
    const GgQuaternion & q,
    GLfloat t) const [inline]
```

球面線形補間の結果を返す.

引数

<i>q</i>	GgQuaternion 型の四元数.
<i>t</i>	補間パラメータ.

戻り値

四元数を *q* に対して *t* で内分した結果.

[gg.h](#) の 4448 行目に定義があります。

呼び出し関係図:



9.10.3.99 slerp() [2/2]

```
GgQuaternion gg::GgQuaternion::slerp (
    GLfloat * a,
    GLfloat t) const [inline]
```

球面線形補間の結果を返す.

引数

<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
<i>t</i>	補間パラメータ.

戻り値

四元数を **a** に対して **t** で内分した結果.

gg.h の 4434 行目に定義があります。

呼び出し関係図:



9.10.3.100 subtract() [1/4]

```
GgQuaternion gg::GgQuaternion::subtract (
    const GgQuaternion & q) const [inline]
```

四元数から別の四元数を減算した結果を返す.

引数

<i>q</i>	GgQuaternion 型の四元数.
----------	---------------------

戻り値

q を引いた四元数.

gg.h の 3883 行目に定義があります。

呼び出し関係図:



9.10.3.101 subtract() [2/4]

```
GgQuaternion gg::GgQuaternion::subtract (
    const GgVector & v) const [inline]
```

四元数から別の四元数を減算した結果を返す.

引数

<i>v</i>	四元数を格納した GgVector 型の変数.
----------	-------------------------

戻り値

v を引いた四元数.

[gg.h](#) の 3872 行目に定義があります。

呼び出し関係図:



9.10.3.102 subtract() [3/4]

```
GgQuaternion gg::GgQuaternion::subtract (
    const GLfloat * a) const [inline]
```

四元数から別の四元数を減算した結果を返す.

引数

a	四元数を格納した GLfloat 型の 4 要素の配列変数.
-----	--------------------------------

戻り値

a を引いた四元数.

[gg.h](#) の 3861 行目に定義があります。

呼び出し関係図:



9.10.3.103 subtract() [4/4]

```
GgQuaternion gg::GgQuaternion::subtract (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) const [inline]
```

四元数から別の四元数を減算した結果を返す.

引数

x	引く四元数の x 要素.
y	引く四元数の y 要素.
z	引く四元数の z 要素.

<code>w</code>	引く四元数の <code>w</code> 要素.
----------------	---------------------------

戻り値

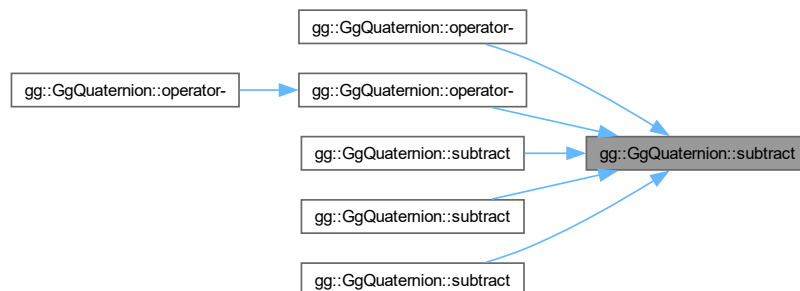
(`x`, `y`, `z`, `w`) を引いた四元数.

`gg.h` の 3849 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- `gg.h`
- `gg.cpp`

9.11 gg::GgShader クラス

```
#include <gg.h>
```

公開メンバ関数

- `GgShader` (`const std::string &vert`, `const std::string &frag=""`, `const std::string &geom=""`, `int nvarying=0`, `const char *const *varyings=nullptr`)
- `GgShader` (`const std::array< std::string, 3 > &files`, `int nvarying=0`, `const char *const *varyings=nullptr`)
- `GgShader` (`const GgShader &shader`)=`delete`
- `GgShader` (`GgShader &&shader`)=`default`
- `virtual ~GgShader ()`
- `GgShader & operator=` (`const GgShader &shader`)=`delete`
- `void use () const`
- `void unuse () const`
- `GLuint get () const`

9.11.1 詳解

シェーダの基底クラス.

覚え書き

シェーダのクラスはこのクラスを派生して作る.

[gg.h](#) の 7079 行目に定義があります。

9.11.2 構築子と解体子

9.11.2.1 GgShader() [1/4]

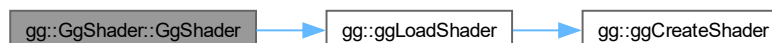
```
gg::GgShader::GgShader (
    const std::string & vert,
    const std::string & frag = "",
    const std::string & geom = "",
    int nvarying = 0,
    const char *const * varyings = nullptr) [inline]
コンストラクタ.
```

引数

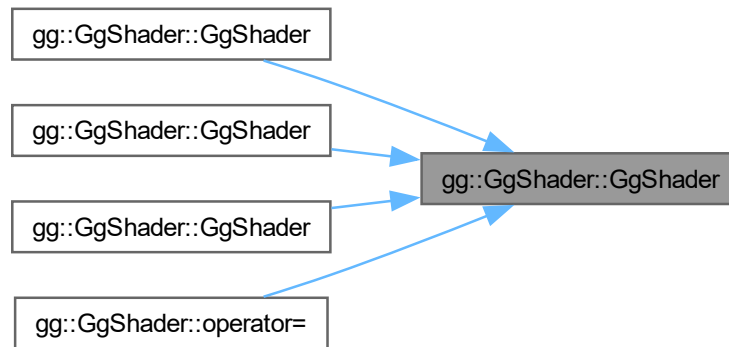
<i>vert</i>	バーテックスシェーダのソースファイル名.
<i>frag</i>	フラグメントシェーダのソースファイル名 (空文字列なら不使用).
<i>geom</i>	ジオメトリシェーダのソースファイル名 (空文字列なら不使用).
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト.

[gg.h](#) の 7095 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.11.2.2 GgShader() [2/4]

```

gg::GgShader::GgShader (
    const std::array< std::string, 3 > & files,
    int nvarying = 0,
    const char *const * varyings = nullptr) [inline]

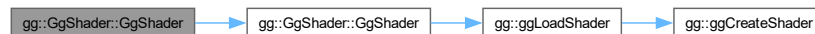
```

コンストラクタ.

引数

<i>files</i>	バーテックスシェーダのソースファイル名の配列.
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (nullptr なら不使用).

gg.h の 7113 行目に定義があります。
呼び出し関係図:



9.11.2.3 GgShader() [3/4]

```

gg::GgShader::GgShader (
    const GgShader & shader) [delete]

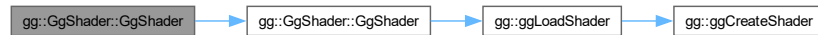
```

コピーコンストラクタは使用しない.

引数

<i>shader</i>	コピー元のシェーダ.
---------------	------------

呼び出し関係図:



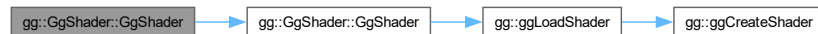
9.11.2.4 GgShader() [4/4]

```
gg::GgShader::GgShader (
    GgShader && shader) [default]
ムーブコンストラクタ.
```

引数

<i>shader</i>	ムーブ元のシェーダ.
---------------	------------

呼び出し関係図:



9.11.2.5 ~GgShader()

```
virtual gg::GgShader::~GgShader () [inline], [virtual]
デストラクタ.
gg.h の 7139 行目に定義があります。
```

9.11.3 関数詳解

9.11.3.1 get()

```
GLuint gg::GgShader::get () const [inline]
シェーダのプログラム名を得る.
```

戻り値

シェーダのプログラム名.

gg.h の 7175 行目に定義があります。

9.11.3.2 operator=()

```
GgShader & gg::GgShader::operator= (
    const GgShader & shader) [delete]
代入演算子は使用しない.
```

引数

<i>shader</i>	代入元のシェーダ.
---------------	-----------

戻り値

代入後のこのシェーダの参照.

呼び出し関係図:



9.11.3.3 unuse()

```
void gg::GgShader::unuse () const [inline]
```

シェーダプログラムの使用を終了する.

[gg.h](#) の 7165 行目に定義があります。

9.11.3.4 use()

```
void gg::GgShader::use () const [inline]
```

シェーダプログラムの使用を開始する.

[gg.h](#) の 7157 行目に定義があります。

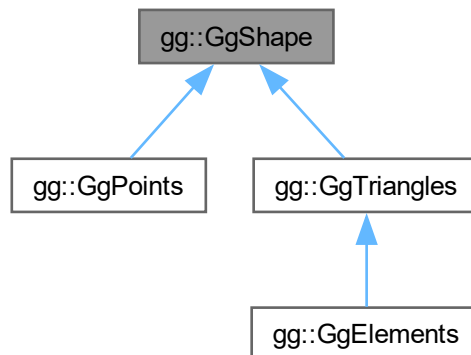
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)

9.12 gg::GgShape クラス

```
#include <gg.h>
```

gg::GgShape の継承関係図



公開メンバ関数

- [GgShape](#) (GLenum mode=0)
- virtual [~GgShape](#) ()
- virtual [operator bool](#) () const noexcept
- virtual bool [operator!](#) () const noexcept

- const GLuint & [get](#) () const
- void [setMode](#) (GLenum mode)
- const GLenum & [getMode](#) () const
- virtual void [draw](#) (GLuint first=0, GLsizei count=0) const

9.12.1 詳解

形状データの基底クラス.

覚え書き

形状データのクラスはこのクラスを派生して作る. 基本図形の種類と頂点配列オブジェクトを保持する.

[gg.h](#) の 6455 行目に定義があります.

9.12.2 構築子と解体子

9.12.2.1 GgShape()

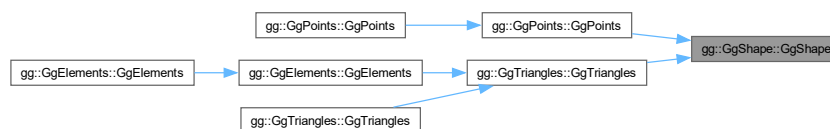
```
gg::GgShape::GgShape (
    GLenum mode = 0) [inline]
```

コンストラクタ.

引数

<i>mode</i>	基本図形の種類.
-------------	----------

[gg.h](#) の 6470 行目に定義があります.
被呼び出し関係図:



9.12.2.2 ~GgShape()

```
virtual gg::GgShape::~~GgShape () [inline], [virtual]
```

デストラクタ.

[gg.h](#) の 6479 行目に定義があります.

9.12.3 関数詳解

9.12.3.1 draw()

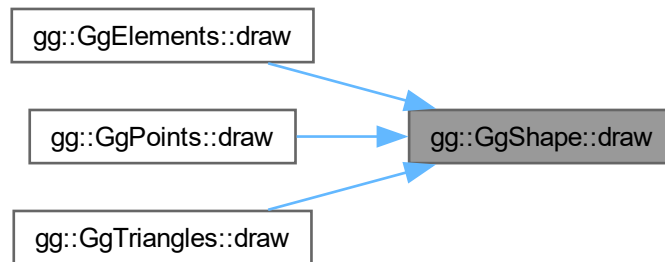
```
virtual void gg::GgShape::draw (
    GLuint first = 0,
    GLsizei count = 0) const [inline], [virtual]
```

図形の描画, 派生クラスでこの手続きをオーバーライドする.

引数

<i>first</i>	描画する最初のアイテム.
<i>count</i>	描画するアイテムの数, 0 なら全部のアイテムを描画する.

gg::GgElements, gg::GgPoints, gg::GgTriangles で再実装されています。
 gg.h の 6539 行目に定義があります。
 被呼び出し関係図:



9.12.3.2 get()

```
const GLuint & gg::GgShape::get () const [inline]
```

頂点配列オブジェクト名を取り出す.

戻り値

頂点配列オブジェクト名.

gg.h の 6508 行目に定義があります。
 被呼び出し関係図:



9.12.3.3 getMode()

```
const GLenum & gg::GgShape::getMode () const [inline]
```

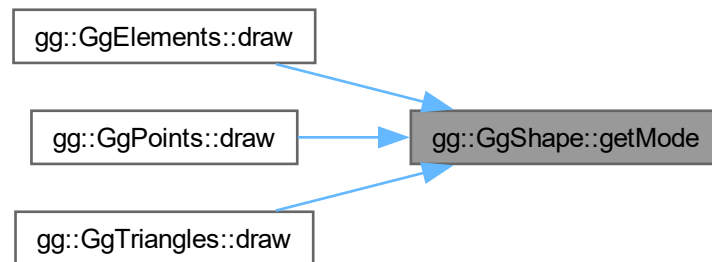
基本図形の検査.

戻り値

この頂点配列オブジェクトの基本図形の種類.

gg.h の 6528 行目に定義があります。

被呼び出し関係図:



9.12.3.4 operator bool()

```
virtual gg::GgShape::operator bool () const [inline], [explicit], [virtual], [noexcept]
```

頂点配列オブジェクトが有効かどうか調べる.

戻り値

頂点配列オブジェクトが有効なら **true**

[gg::GgPoints](#) で再実装されています。
[gg.h](#) の 6488 行目に定義があります。

9.12.3.5 operator"!()

```
virtual bool gg::GgShape::operator! () const [inline], [virtual], [noexcept]
```

頂点配列オブジェクトが有効かどうかの結果を反転する.

戻り値

頂点配列オブジェクトが有効なら **false**, 無効なら **true**.

[gg::GgPoints](#) で再実装されています。
[gg.h](#) の 6498 行目に定義があります。

9.12.3.6 setMode()

```
void gg::GgShape::setMode (
    GLenum mode) [inline]
```

基本図形の設定.

引数

<i>mode</i>	基本図形の種類.
-------------	----------

[gg.h](#) の 6518 行目に定義があります。
 このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)

9.13 gg::GgSimpleObj クラス

```
#include <gg.h>
```

公開メンバ関数

- `GgSimpleObj` (const std::string &name, bool normalize=false)
- virtual `~GgSimpleObj` ()
デストラクタ.
- `operator bool` () const noexcept
- bool `operator!` () const noexcept
- const `GgTriangles` * `get` () const
- virtual void `draw` (GLint first=0, GLsizei count=0) const

9.13.1 詳解

Wavefront OBJ 形式のファイル (Elements 形式).

`gg.h` の 8451 行目に定義があります。

9.13.2 構築子と解体子

9.13.2.1 GgSimpleObj()

```
gg::GgSimpleObj::GgSimpleObj (
    const std::string & name,
    bool normalize = false)
```

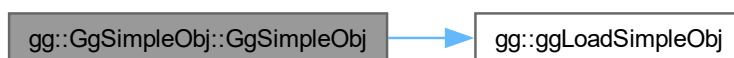
コンストラクタ.

引数

<i>name</i>	三角形分割された Alias OBJ 形式のファイルのファイル名.
<i>normalize</i>	true なら図形のサイズを [-1, 1] に正規化する.

`gg.cpp` の 6011 行目に定義があります。

呼び出し関係図:



9.13.2.2 ~GgSimpleObj()

```
virtual gg::GgSimpleObj::~~GgSimpleObj () [inline], [virtual]
```

デストラクタ.

`gg.h` の 8474 行目に定義があります。

9.13.3 関数詳解

9.13.3.1 draw()

```
void gg::GgSimpleObj::draw (
    GLint first = 0,
    GLsizei count = 0) const [virtual]
```

Wavefront OBJ 形式のデータを描画する手続き。

引数

<i>first</i>	描画する最初のパーツ番号.
<i>count</i>	描画するパーツの数, 0 なら全部のパーツを描く.

[gg.cpp](#) の 6039 行目に定義があります。

9.13.3.2 get()

```
const GgTriangles * gg::GgSimpleObj::get () const [inline]
```

形状データの取り出し。

戻り値

[GgTriangles](#) 型の形状データのポインタ。

[gg.h](#) の 8503 行目に定義があります。

呼び出し関係図:



9.13.3.3 operator bool()

```
gg::GgSimpleObj::operator bool () const [inline], [explicit], [noexcept]
```

オブジェクトが有効かどうか調べる。

戻り値

オブジェクトが有効なら `true`

[gg.h](#) の 8483 行目に定義があります。

9.13.3.4 operator"!()

```
bool gg::GgSimpleObj::operator! () const [inline], [noexcept]
```

オブジェクトが有効かどうかの結果を反転する。

戻り値

オブジェクトが有効なら `false`, 無効なら `true`.

[gg.h](#) の 8493 行目に定義があります。

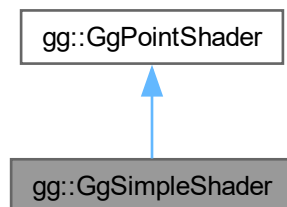
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

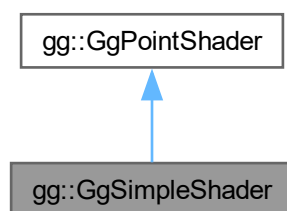
9.14 gg::GgSimpleShader クラス

```
#include <gg.h>
```

gg::GgSimpleShader の継承関係図



gg::GgSimpleShader 連携図



クラス

- struct [Light](#)
- class [LightBuffer](#)
- struct [Material](#)
- class [MaterialBuffer](#)

公開メンバ関数

- [GgSimpleShader](#) ()
- [GgSimpleShader](#) (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- [GgSimpleShader](#) (const std::array< std::string, 3 > &files, GLint nvarying=0, const char *const *varyings=nullptr)
- [GgSimpleShader](#) (const GgSimpleShader &o)
- virtual [~GgSimpleShader](#) ()
- [GgSimpleShader &operator=](#) (const [GgSimpleShader](#) &o)
- bool [load](#) (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- bool [load](#) (const std::array< std::string, 3 > &files, GLint nvarying=0, const char *const *varyings=nullptr)
- virtual void [loadModelviewMatrix](#) (const GLfloat *mv, const GLfloat *mn) const

- virtual void `loadModelviewMatrix` (const `GgMatrix` &mv, const `GgMatrix` &mn) const
- virtual void `loadModelviewMatrix` (const GLfloat *mv) const
- virtual void `loadModelviewMatrix` (const `GgMatrix` &mv) const
- virtual void `loadMatrix` (const GLfloat *mp, const GLfloat *mv, const GLfloat *mn) const
- virtual void `loadMatrix` (const `GgMatrix` &mp, const `GgMatrix` &mv, const `GgMatrix` &mn) const
- virtual void `loadMatrix` (const GLfloat *mp, const GLfloat *mv) const
- virtual void `loadMatrix` (const `GgMatrix` &mp, const `GgMatrix` &mv) const
- void `use` () const
- void `use` (const GLfloat *mp, const GLfloat *mv, const GLfloat *mn) const
- void `use` (const `GgMatrix` &mp, const `GgMatrix` &mv, const `GgMatrix` &mn) const
- void `use` (const GLfloat *mp, const GLfloat *mv) const
- void `use` (const `GgMatrix` &mp, const `GgMatrix` &mv) const
- void `use` (const `LightBuffer` *light, GLint i=0) const
- void `use` (const `LightBuffer` &light, GLint i=0) const
- void `use` (const GLfloat *mp, const GLfloat *mv, const GLfloat *mn, const `LightBuffer` *light, GLint i=0) const
- void `use` (const `GgMatrix` &mp, const `GgMatrix` &mv, const `GgMatrix` &mn, const `LightBuffer` &light, GLint i=0) const
- void `use` (const GLfloat *mp, const GLfloat *mv, const `LightBuffer` *light, GLint i=0) const
- void `use` (const `GgMatrix` &mp, const `GgMatrix` &mv, const `LightBuffer` &light, GLint i=0) const
- void `use` (const GLfloat *mp, const `LightBuffer` *light, GLint i=0) const
- void `use` (const `GgMatrix` &mp, const `LightBuffer` &light, GLint i=0) const

基底クラス `gg::GgPointShader` に属する継承公開メンバ関数

- `GgPointShader` ()
- `GgPointShader` (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- `GgPointShader` (const std::array< std::string, 3 > &files, int nvarying=0, const char *const *varyings=nullptr)
- virtual `~GgPointShader` ()
- bool `load` (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- bool `load` (const std::array< std::string, 3 > &files, GLint nvarying=0, const char *const *varyings=nullptr)
- virtual void `loadProjectionMatrix` (const GLfloat *mp) const
- virtual void `loadProjectionMatrix` (const `GgMatrix` &mp) const
- void `use` (const GLfloat *mp) const
- void `use` (const `GgMatrix` &mp) const
- void `use` (const GLfloat *mp, const GLfloat *mv) const
- void `use` (const `GgMatrix` &mp, const `GgMatrix` &mv) const
- void `unuse` () const
- GLuint `get` () const

9.14.1 詳解

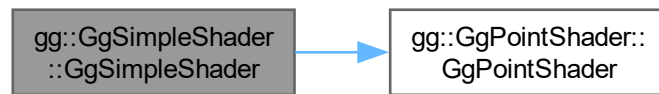
三角形に単純な陰影付けを行うシェーダ。
[gg.h](#) の 7439 行目に定義があります。

9.14.2 構築子と解体子

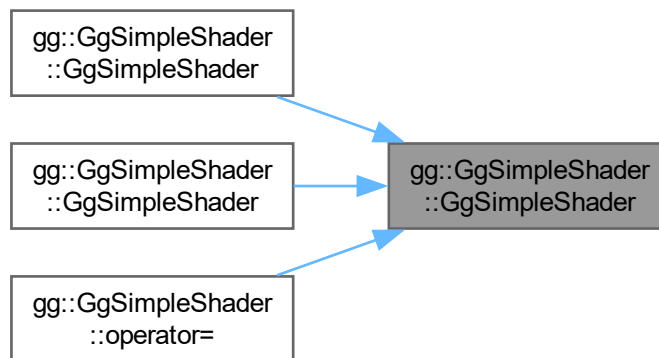
9.14.2.1 GgSimpleShader() [1/4]

`gg::GgSimpleShader::GgSimpleShader () [inline]`
 コンストラクタ。
[gg.h](#) の 7450 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.14.2.2 GgSimpleShader() [2/4]

```

gg::GgSimpleShader::GgSimpleShader (
    const std::string & vert,
    const std::string & frag = "",
    const std::string & geom = "",
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [inline]
  
```

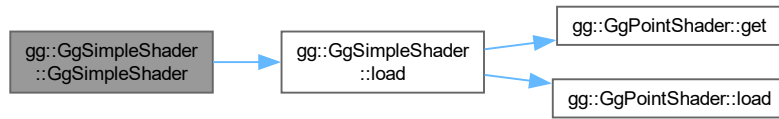
コンストラクタ.

引数

<i>vert</i>	バーテックスシェーダのソースファイル名.
<i>frag</i>	フラグメントシェーダのソースファイル名 (空文字列なら不使用).
<i>geom</i>	ジオメトリシェーダのソースファイル名 (空文字列なら不使用).
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト.

gg.h の 7465 行目に定義があります。

呼び出し関係図:



9.14.2.3 GgSimpleShader() [3/4]

```

gg::GgSimpleShader::GgSimpleShader (
    const std::array< std::string, 3 > & files,
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [inline]
  
```

コンストラクタ.

引数

<i>files</i>	バーテックスシェーダのソースファイル名の配列.
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (nullptr なら不使用).

[gg.h](#) の 7483 行目に定義があります。

呼び出し関係図:



9.14.2.4 GgSimpleShader() [4/4]

```

gg::GgSimpleShader::GgSimpleShader (
    const GgSimpleShader & o) [inline]
  
```

コピーコンストラクタ.

[gg.h](#) の 7495 行目に定義があります。

呼び出し関係図:



9.14.2.5 ~GgSimpleShader()

virtual gg::GgSimpleShader::~~GgSimpleShader () [inline], [virtual]
デストラクタ.

gg.h の 7504 行目に定義があります。

9.14.3 関数詳解

9.14.3.1 load() [1/2]

```
bool gg::GgSimpleShader::load (
    const std::array< std::string, 3 > & files,
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [inline]
```

シェーダのソースファイルを読み込んでプログラムオブジェクトを作成する。

引数

<i>files</i>	バーテックスシェーダのソースファイル名の配列.
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (nullptr なら不使用).

戻り値

プログラムオブジェクトの作成に成功したら true.

gg.h の 7551 行目に定義があります。

呼び出し関係図:



9.14.3.2 load() [2/2]

```
bool gg::GgSimpleShader::load (
    const std::string & vert,
    const std::string & frag = "",
    const std::string & geom = "",
    GLint nvarying = 0,
    const char *const * varyings = nullptr)
```

シェーダのソースプログラムの文字列からプログラムオブジェクトを作成する。

引数

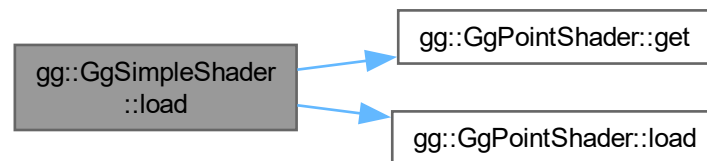
<i>vert</i>	バーテックスシェーダのソースプログラムの文字列.
<i>frag</i>	フラグメントシェーダのソースプログラムの文字列 (空文字列なら不使用).
<i>geom</i>	ジオメトリシェーダのソースプログラムの文字列 (空文字列なら不使用).

<code>nvarying</code>	フィードバックする <code>varying</code> 変数の数 (0 なら不使用).
<code>varyings</code>	フィードバックする <code>varying</code> 変数のリスト (<code>nullptr</code> なら不使用).

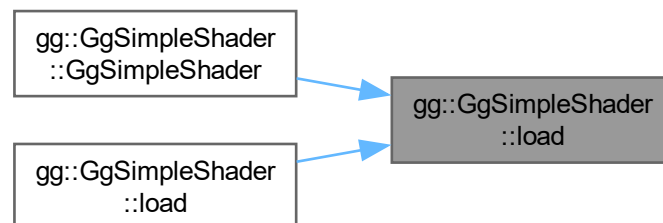
戻り値

プログラムオブジェクトの作成に成功したら `true`.

`gg.cpp` の 5994 行目に定義があります。
呼び出し関係図:



被呼び出し関係図:



9.14.3.3 loadMatrix() [1/4]

```
virtual void gg::GgSimpleShader::loadMatrix (
    const GgMatrix & mp,
    const GgMatrix & mv) const [inline], [virtual]
```

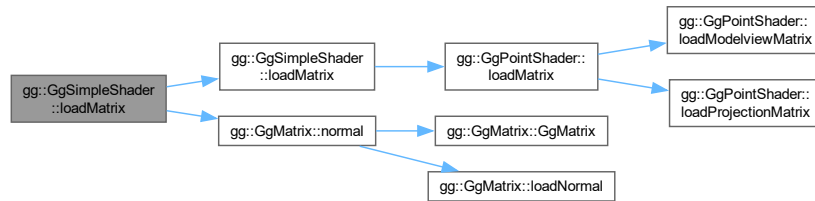
投影変換行列とモデルビュー変換行列を設定しモデルビュー変換行列から求めた法線変換行列を設定する.

引数

<code>mp</code>	<code>GgMatrix</code> 型の投影変換行列.
<code>mv</code>	<code>GgMatrix</code> 型のモデルビュー変換行列.

`gg::GgPointShader` を再実装しています。

gg.h の 7645 行目に定義があります。
呼び出し関係図:



9.14.3.4 loadMatrix() [2/4]

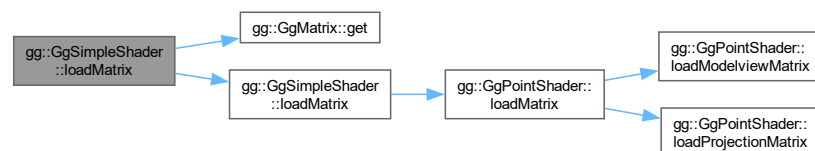
```
virtual void gg::GgSimpleShader::loadMatrix (
    const GgMatrix & mp,
    const GgMatrix & mv,
    const GgMatrix & mn) const [inline], [virtual]
```

投影変換行列とモデルビュー変換行列と法線変換行列を設定する。

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.
<i>mn</i>	GgMatrix 型のモデルビュー変換行列の法線変換行列.

gg.h の 7623 行目に定義があります。
呼び出し関係図:



9.14.3.5 loadMatrix() [3/4]

```
virtual void gg::GgSimpleShader::loadMatrix (
    const GLfloat * mp,
    const GLfloat * mv) const [inline], [virtual]
```

投影変換行列とモデルビュー変換行列を設定しモデルビュー変換行列から求めた法線変換行列を設定する。

引数

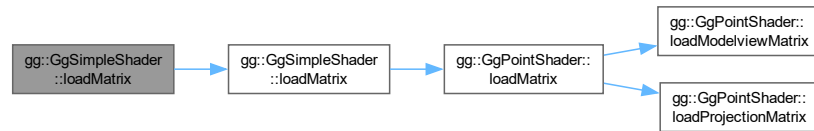
<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
-----------	------------------------------------

<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.
-----------	--

[gg::GgPointShader](#)を再実装しています。

[gg.h](#) の 7634 行目に定義があります。

呼び出し関係図:



9.14.3.6 loadMatrix() [4/4]

```
virtual void gg::GgSimpleShader::loadMatrix (
    const GLfloat * mp,
    const GLfloat * mv,
    const GLfloat * mn) const [inline], [virtual]
```

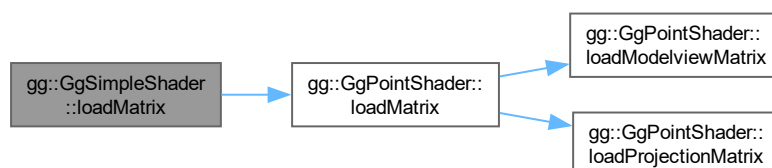
投影変換行列とモデルビュー変換行列と法線変換行列を設定する。

引数

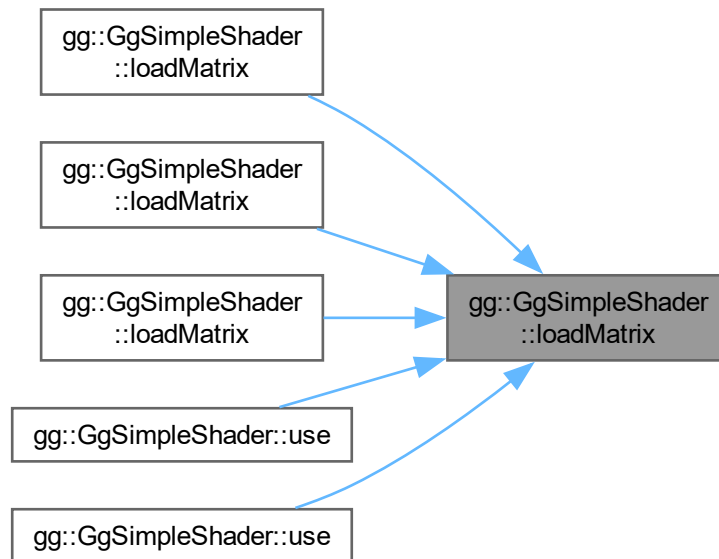
<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.
<i>mn</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列の法線変換行列.

[gg.h](#) の 7610 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.14.3.7 loadModelviewMatrix() [1/4]

```
virtual void gg::GgSimpleShader::loadModelviewMatrix (
    const GgMatrix & mv) const [inline], [virtual]
```

モデルビュー変換行列とそれから求めた法線変換行列を設定する.

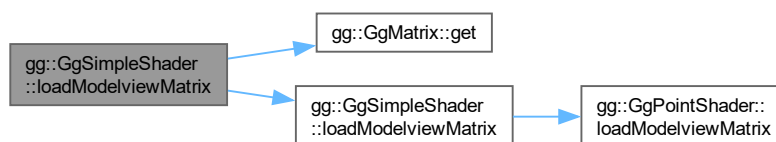
引数

<i>mv</i>	GgMatrix 型のモデルビュー変換行列.
-----------	------------------------

gg::GgPointShaderを再実装しています。

gg.h の 7598 行目に定義があります。

呼び出し関係図:



9.14.3.8 loadModelviewMatrix() [2/4]

```
virtual void gg::GgSimpleShader::loadModelviewMatrix (
```

```
const GgMatrix & mv,
const GgMatrix & mn) const [inline], [virtual]
```

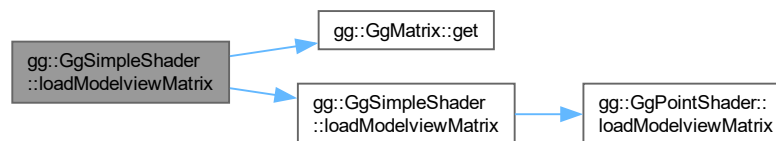
モデルビュー変換行列と法線変換行列を設定する。

引数

<i>mv</i>	GgMatrix 型のモデルビュー変換行列.
<i>mn</i>	GgMatrix 型のモデルビュー変換行列の法線変換行列.

gg.h の 7578 行目に定義があります。

呼び出し関係図:



9.14.3.9 loadModelviewMatrix() [3/4]

```
virtual void gg::GgSimpleShader::loadModelviewMatrix (
const GLfloat * mv) const [inline], [virtual]
```

モデルビュー変換行列とそれから求めた法線変換行列を設定する。

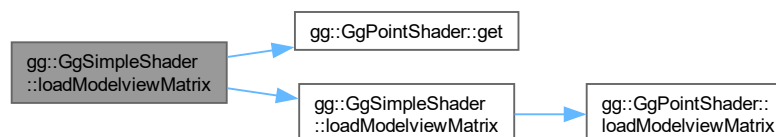
引数

<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.
-----------	--

gg::GgPointShader を再実装しています。

gg.h の 7588 行目に定義があります。

呼び出し関係図:



9.14.3.10 loadModelviewMatrix() [4/4]

```
virtual void gg::GgSimpleShader::loadModelviewMatrix (
const GLfloat * mv,
const GLfloat * mn) const [inline], [virtual]
```

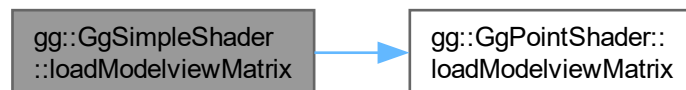
モデルビュー変換行列と法線変換行列を設定する。

引数

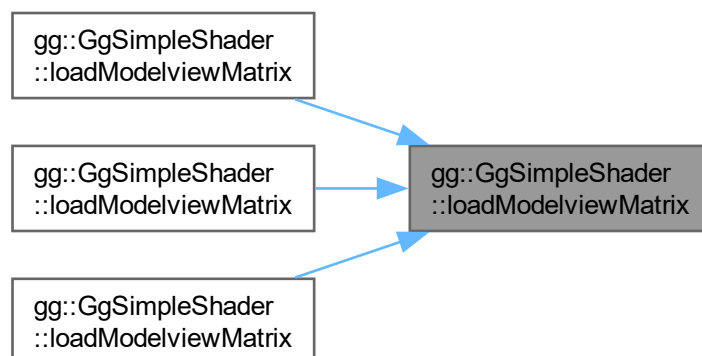
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.
<i>mn</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列の法線変換行列.

gg.h の 7566 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.14.3.11 operator=()

```
GgSimpleShader & gg::GgSimpleShader::operator= (
    const GgSimpleShader & o) [inline]
```

代入演算子.

引数

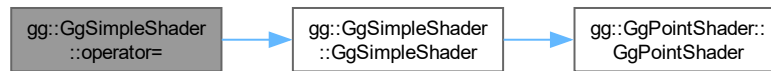
<i>o</i>	代入する GgSimpleShader 型のオブジェクト.
----------	-------------------------------

戻り値

代入後の [GgSimpleShader](#) 型のオブジェクトの参照.

[gg.h](#) の 7514 行目に定義があります。

呼び出し関係図:



9.14.3.12 use() [1/13]

```
void gg::GgSimpleShader::use () const [inline], [virtual]
```

シェーダプログラムの使用を開始する.

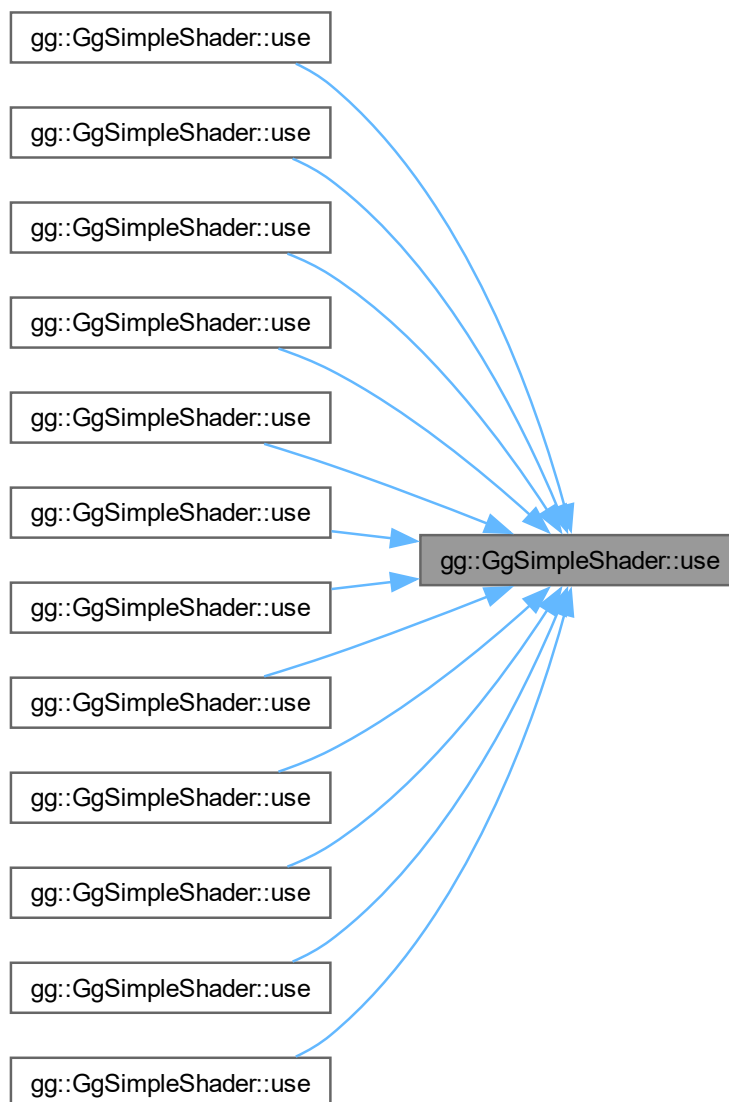
[gg::GgPointShader](#)を再実装しています。

[gg.h](#) の 8218 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.14.3.13 use() [2/13]

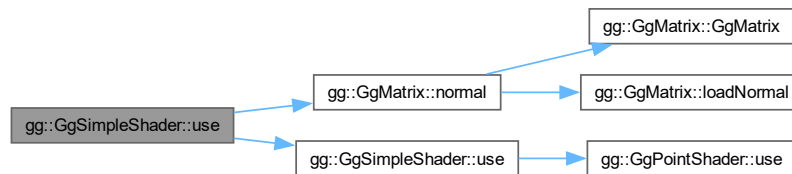
```
void gg::GgSimpleShader::use (
    const GgMatrix & mp,
    const GgMatrix & mv) const [inline]
```

投影変換行列とモデルビュー変換行列を設定しモデルビュー変換行列から求めた法線変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.

gg.h の 8269 行目に定義があります。
呼び出し関係図:



9.14.3.14 use() [3/13]

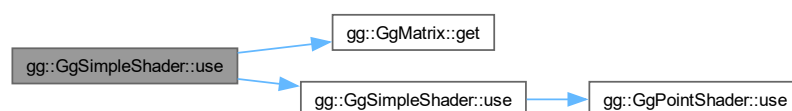
```
void gg::GgSimpleShader::use (
    const GgMatrix & mp,
    const GgMatrix & mv,
    const GgMatrix & mn) const [inline]
```

投影変換行列とモデルビュー変換行列と法線変換行列を指定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.
<i>mn</i>	GgMatrix 型のモデルビュー変換行列の法線変換行列.

gg.h の 8247 行目に定義があります。
呼び出し関係図:



9.14.3.15 use() [4/13]

```
void gg::GgSimpleShader::use (
    const GgMatrix & mp,
    const GgMatrix & mv,
    const GgMatrix & mn,
    const LightBuffer & light,
    GLint i = 0) const [inline]
```

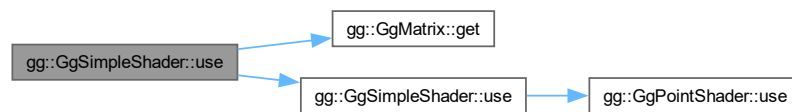
光源を指定し投影変換行列とモデルビュー変換行列と法線変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.
<i>mn</i>	GgMatrix 型のモデルビュー変換行列の法線変換行列.
<i>light</i>	光源の特性の gg::LightBuffer 構造体.
<i>i</i>	光源データの uniform block のインデックス.

gg.h の 8333 行目に定義があります。

呼び出し関係図:



9.14.3.16 use() [5/13]

```

void gg::GgSimpleShader::use (
    const GgMatrix & mp,
    const GgMatrix & mv,
    const LightBuffer & light,
    GLint i = 0) const [inline]
  
```

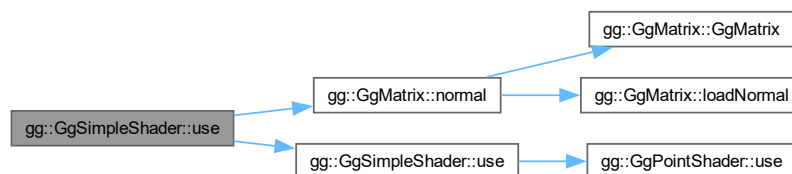
光源を指定し投影変換行列とモデルビュー変換行列を設定しモデルビュー変換行列から求めた法線変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.
<i>light</i>	光源の特性の gg::LightBuffer 構造体.
<i>i</i>	光源データの uniform block のインデックス.

gg.h の 8370 行目に定義があります。

呼び出し関係図:



9.14.3.17 use() [6/13]

```
void gg::GgSimpleShader::use (
    const GgMatrix & mp,
    const LightBuffer & light,
    GLint i = 0) const [inline]
```

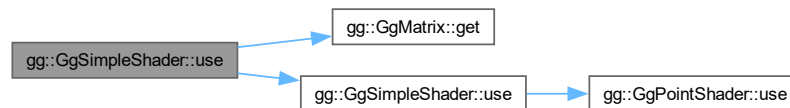
光源を指定し投影変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>light</i>	光源の特性の gg::LightBuffer 構造体.
<i>i</i>	光源データの uniform block のインデックス.

gg.h の 8403 行目に定義があります。

呼び出し関係図:



9.14.3.18 use() [7/13]

```
void gg::GgSimpleShader::use (
    const GLfloat * mp,
    const GLfloat * mv) const [inline]
```

投影変換行列とモデルビュー変換行列を設定しモデルビュー変換行列から求めた法線変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.

gg.h の 8258 行目に定義があります。

呼び出し関係図:



9.14.3.19 use() [8/13]

```
void gg::GgSimpleShader::use (
    const GLfloat * mp,
    const GLfloat * mv,
    const GLfloat * mn) const [inline]
```

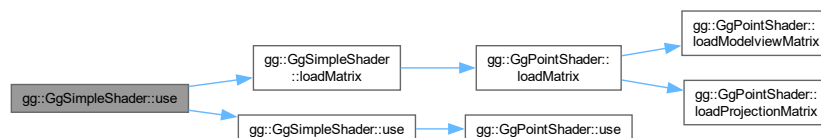
投影変換行列とモデルビュー変換行列と法線変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列。
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列。
<i>mn</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列の法線変換行列。

gg.h の 8231 行目に定義があります。

呼び出し関係図:



9.14.3.20 use() [9/13]

```
void gg::GgSimpleShader::use (
    const GLfloat * mp,
    const GLfloat * mv,
    const GLfloat * mn,
    const LightBuffer * light,
    GLint i = 0) const [inline]
```

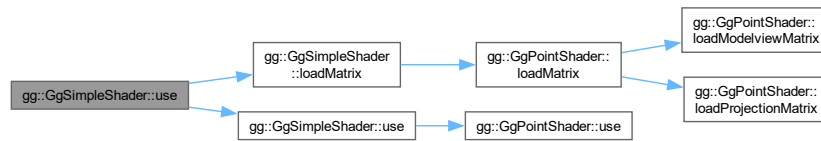
光源を指定し投影変換行列とモデルビュー変換行列と法線変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列。
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列。
<i>mn</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列の法線変換行列。
<i>light</i>	光源の特性の gg::LightBuffer 構造体のポインタ。
<i>i</i>	光源データの uniform block のインデックス。

gg.h の 8309 行目に定義があります。

呼び出し関係図:



9.14.3.21 use() [10/13]

```
void gg::GgSimpleShader::use (
    const GLfloat * mp,
    const GLfloat * mv,
    const LightBuffer * light,
    GLint i = 0) const [inline]
```

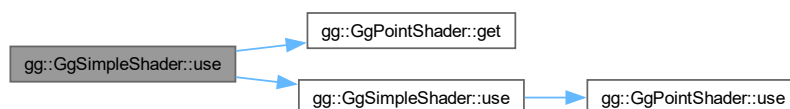
光源を指定し投影変換行列とモデルビュー変換行列を設定しモデルビュー変換行列から求めた法線変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.
<i>light</i>	光源の特性の gg::LightBuffer 構造体のポインタ.
<i>i</i>	光源データの uniform block のインデックス.

gg.h の 8352 行目に定義があります。

呼び出し関係図:



9.14.3.22 use() [11/13]

```
void gg::GgSimpleShader::use (
    const GLfloat * mp,
    const LightBuffer * light,
    GLint i = 0) const [inline]
```

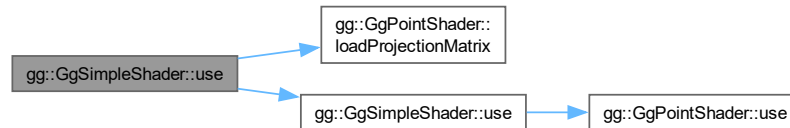
光源を指定し投影変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>light</i>	光源の特性の gg::LightBuffer 構造体のポインタ.

<i>i</i>	光源データの uniform block のインデックス.
----------	-------------------------------

gg.h の 8387 行目に定義があります。
呼び出し関係図:



9.14.3.23 use() [12/13]

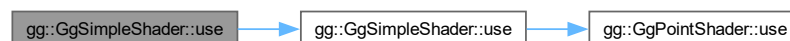
```
void gg::GgSimpleShader::use (
    const LightBuffer & light,
    GLint i = 0) const [inline]
```

光源を指定してシェーダプログラムの使用を開始する.

引数

<i>light</i>	光源の特性の gg::LightBuffer 構造体.
<i>i</i>	光源データの uniform block のインデックス.

gg.h の 8295 行目に定義があります。
呼び出し関係図:



9.14.3.24 use() [13/13]

```
void gg::GgSimpleShader::use (
    const LightBuffer * light,
    GLint i = 0) const [inline]
```

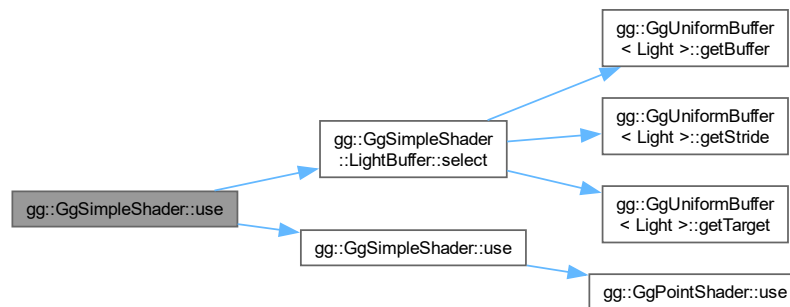
光源を指定してシェーダプログラムの使用を開始する.

引数

<i>light</i>	光源の特性の gg::LightBuffer 構造体のポインタ.
<i>i</i>	光源データの uniform block のインデックス.

gg.h の 8280 行目に定義があります。

呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

9.15 gg::GgTexture クラス

```
#include <gg.h>
```

公開メンバ関数

- [GgTexture](#) (const GLvoid *image, GLsizei width, GLsizei height, GLenum format=GL_RGB, GLenum type=GL_UNSIGNED_BYTE, GLenum internal=GL_RGBA, GLenum wrap=GL_CLAMP_TO_EDGE, bool swizzle=false)
- [GgTexture](#) (const GgTexture &texture)=delete
- [GgTexture](#) (GgTexture &&texture)=default
- virtual ~[GgTexture](#) ()
- [GgTexture](#) & operator= (const [GgTexture](#) &texture)=delete
- [GgTexture](#) & operator= ([GgTexture](#) &&texture)=default
- void [bind](#) () const
- void [unbind](#) () const
- void [swapRandB](#) (bool swizzle) const
- const GLsizei & [getWidth](#) () const
- const GLsizei & [getHeight](#) () const
- void [getSize](#) (GLsizei *size) const
- const GLsizei * [getSize](#) () const
- const GLuint & [getTexture](#) () const

9.15.1 詳解

テクスチャ.

覚え書き

画像データを読み込んでテクスチャマップを作成する.

[gg.h](#) の 5224 行目に定義があります。

9.15.2 構築子と解体子

9.15.2.1 GgTexture() [1/3]

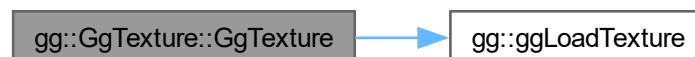
```
gg::GgTexture::GgTexture (
    const GLvoid * image,
    GLsizei width,
    GLsizei height,
    GLenum format = GL_RGB,
    GLenum type = GL_UNSIGNED_BYTE,
    GLenum internal = GL_RGBA,
    GLenum wrap = GL_CLAMP_TO_EDGE,
    bool swizzle = false) [inline]
```

メモリ上のデータからテクスチャを作成するコンストラクタ。

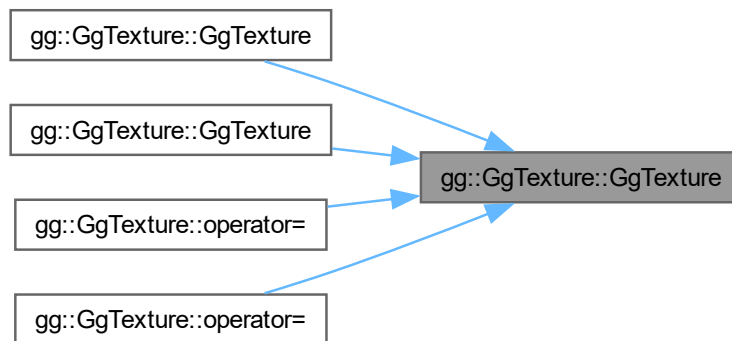
引数

<i>image</i>	テクスチャとして用いる画像データ, nullptr ならデータを読み込まない.
<i>width</i>	テクスチャの横の画素数.
<i>height</i>	テクスチャの縦の画素数.
<i>format</i>	読み込む画像のフォーマット.
<i>type</i>	画像のデータ型.
<i>internal</i>	テクスチャの内部フォーマット.
<i>wrap</i>	テクスチャのラッピングモード, デフォルトは GL_CLAMP_TO_EDGE.
<i>swizzle</i>	true ならテクスチャの赤と青を入れ替える, デフォルトは false.

gg.h の 5246 行目に定義があります。
呼び出し関係図:



被呼び出し関係図:



9.15.2.2 GgTexture() [2/3]

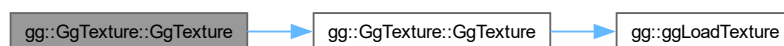
```
gg::GgTexture::GgTexture (
    const GgTexture & texture) [delete]
```

コピーコンストラクタは使用しない.

引数

<i>texture</i>	コピー元のテクスチャ.
----------------	-------------

呼び出し関係図:



9.15.2.3 GgTexture() [3/3]

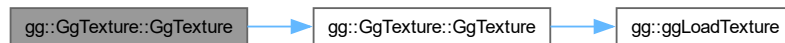
```
gg::GgTexture::GgTexture (
    GgTexture && texture) [default]
```

ムーブコンストラクタ.

引数

<i>texture</i>	ムーブ元のテクスチャ.
----------------	-------------

呼び出し関係図:



9.15.2.4 ~GgTexture()

```
virtual gg::GgTexture::~~GgTexture () [inline], [virtual]
```

デストラクタ.

gg.h の 5278 行目に定義があります。

9.15.3 関数詳解

9.15.3.1 bind()

```
void gg::GgTexture::bind () const [inline]
```

テクスチャの使用開始 (このテクスチャを使用する際に呼び出す).

gg.h の 5303 行目に定義があります。

9.15.3.2 getHeight()

```
const GLsizei & gg::GgTexture::getHeight () const [inline]
```

使用しているテクスチャの縦の画素数を取り出す.

戻り値

テクスチャの縦の画素数.

gg.h の 5338 行目に定義があります。

被呼び出し関係図:



9.15.3.3 getSize() [1/2]

```
const GLsizei * gg::GgTexture::getSize () const [inline]
```

使用しているテクスチャのサイズを取り出す.

戻り値

テクスチャのサイズを格納した配列へのポインタ.

gg.h の 5359 行目に定義があります。

9.15.3.4 getSize() [2/2]

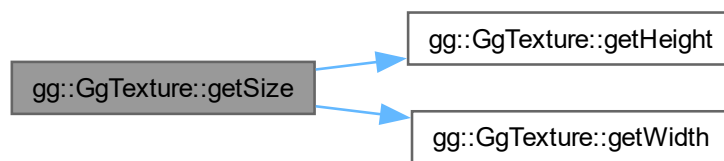
```
void gg::GgTexture::getSize (
    GLsizei * size) const [inline]
```

使用しているテクスチャのサイズを取り出す.

引数

<i>size</i>	テクスチャのサイズを格納する GLsizei 型の 2 要素の配列変数.
-------------	--------------------------------------

[gg.h](#) の 5348 行目に定義があります。
呼び出し関係図:



9.15.3.5 getTexture()

```
const GLuint & gg::GgTexture::getTexture () const [inline]
```

使用しているテクスチャのテクスチャ名を得る.

戻り値

テクスチャ名.

[gg.h](#) の 5369 行目に定義があります。

9.15.3.6 getWidth()

```
const GLsizei & gg::GgTexture::getWidth () const [inline]
```

使用しているテクスチャの横の画素数を取り出す.

戻り値

テクスチャの横の画素数.

[gg.h](#) の 5328 行目に定義があります。
被呼び出し関係図:



9.15.3.7 operator=() [1/2]

```
GgTexture & gg::GgTexture::operator= (
    const GgTexture & texture) [delete]
```

代入演算子は使用しない.

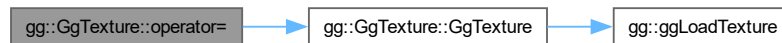
引数

<i>texture</i>	代入元のテクスチャ.
----------------	------------

戻り値

代入後のこのテクスチャの参照.

呼び出し関係図:



9.15.3.8 operator=() [2/2]

```
GgTexture & gg::GgTexture::operator= (
    GgTexture && texture) [default]
```

ムーブ代入演算子.

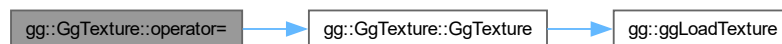
引数

<i>texture</i>	ムーブ代入元のテクスチャ.
----------------	---------------

戻り値

ムーブ代入後のこのテクスチャの参照.

呼び出し関係図:



9.15.3.9 swapRandB()

```
void gg::GgTexture::swapRandB (
    bool swizzle) const
```

テクスチャの赤と青を交換する

引数

<code>swizzle</code>	赤と青を交換するなら <code>true</code>
----------------------	------------------------------

`gg.cpp` の 4022 行目に定義があります。

9.15.3.10 unbind()

```
void gg::GgTexture::unbind () const [inline]
```

テクスチャの使用終了 (このテクスチャを使用しなくなったら呼び出す).

`gg.h` の 5311 行目に定義があります。

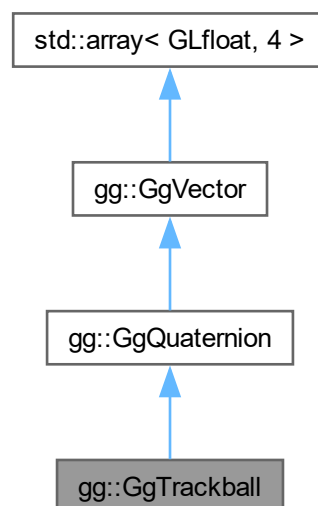
このクラス詳解は次のファイルから抽出されました:

- `gg.h`
- `gg.cpp`

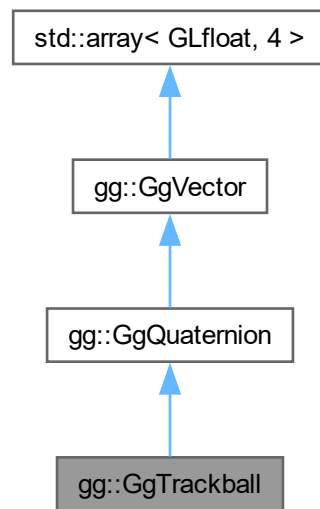
9.16 gg::GgTrackball クラス

```
#include <gg.h>
```

`gg::GgTrackball` の継承関係図



gg::GgTrackball 連携図



公開メンバ関数

- `GgTrackball` (const `GgQuaternion` &q=`ggIdentityQuaternion`())
- `~GgTrackball` ()=default
- `GgTrackball` & `operator=` (const `GgQuaternion` &q)
- void `region` (GLfloat w, GLfloat h)
- void `region` (int w, int h)
- void `begin` (GLfloat x, GLfloat y)
- void `motion` (GLfloat x, GLfloat y)
- void `rotate` (const `GgQuaternion` &q)
- void `end` (GLfloat x, GLfloat y)
- void `reset` (const `GgQuaternion` &q=`ggIdentityQuaternion`())
- const GLfloat * `getStart` () const
- const GLfloat & `getStart` (int direction) const
- void `getStart` (GLfloat *position) const
- const GLfloat * `getScale` () const
- const GLfloat `getScale` (int direction) const
- void `getScale` (GLfloat *factor) const
- const `GgQuaternion` & `getQuaternion` () const
- const `GgMatrix` & `getMatrix` () const
- const GLfloat * `get` () const

基底クラス `gg::GgQuaternion` に属する継承公開メンバ関数

- `GgQuaternion` ()=default
- constexpr `GgQuaternion` (GLfloat x, GLfloat y, GLfloat z, GLfloat w)
- constexpr `GgQuaternion` (GLfloat c)
- `GgQuaternion` (const GLfloat *a)
- `GgQuaternion` (const `GgVector` &v)
- `GgQuaternion` (const `GgQuaternion` &q)=default

- `GgQuaternion` (`GgQuaternion &&q`)=default
- `~GgQuaternion` ()=default
- `GgQuaternion & operator=` (const `GgQuaternion` &q)=default
- `GgQuaternion & operator=` (`GgQuaternion` &&q)=default
- `GLfloat norm` () const
- `GgQuaternion & loadAdd` (GLfloat x, GLfloat y, GLfloat z, GLfloat w)
- `GgQuaternion & loadAdd` (const GLfloat *a)
- `GgQuaternion & loadAdd` (const `GgVector` &v)
- `GgQuaternion & loadAdd` (const `GgQuaternion` &q)
- `GgQuaternion & loadSubtract` (GLfloat x, GLfloat y, GLfloat z, GLfloat w)
- `GgQuaternion & loadSubtract` (const GLfloat *a)
- `GgQuaternion & loadSubtract` (const `GgVector` &v)
- `GgQuaternion & loadSubtract` (const `GgQuaternion` &q)
- `GgQuaternion & loadMultiply` (GLfloat x, GLfloat y, GLfloat z, GLfloat w)
- `GgQuaternion & loadMultiply` (const GLfloat *a)
- `GgQuaternion & loadMultiply` (const `GgVector` &v)
- `GgQuaternion & loadMultiply` (const `GgQuaternion` &q)
- `GgQuaternion & loadDivide` (GLfloat x, GLfloat y, GLfloat z, GLfloat w)
- `GgQuaternion & loadDivide` (const GLfloat *a)
- `GgQuaternion & loadDivide` (const `GgVector` &v)
- `GgQuaternion & loadDivide` (const `GgQuaternion` &q)
- `GgQuaternion add` (GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
- `GgQuaternion add` (const GLfloat *a) const
- `GgQuaternion add` (const `GgVector` &v) const
- `GgQuaternion add` (const `GgQuaternion` &q) const
- `GgQuaternion subtract` (GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
- `GgQuaternion subtract` (const GLfloat *a) const
- `GgQuaternion subtract` (const `GgVector` &v) const
- `GgQuaternion subtract` (const `GgQuaternion` &q) const
- `GgQuaternion multiply` (GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
- `GgQuaternion multiply` (const GLfloat *a) const
- `GgQuaternion multiply` (const `GgVector` &v) const
- `GgQuaternion multiply` (const `GgQuaternion` &q) const
- `GgQuaternion divide` (GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
- `GgQuaternion divide` (const GLfloat *a) const
- `GgQuaternion divide` (const `GgVector` &v) const
- `GgQuaternion divide` (const `GgQuaternion` &q) const
- `GgQuaternion & operator=` (const GLfloat *a)
- `GgQuaternion & operator=` (const `GgVector` &v)
- `GgQuaternion & operator+=` (const GLfloat *a)
- `GgQuaternion & operator+=` (const `GgVector` &v)
- `GgQuaternion & operator+=` (const `GgQuaternion` &q)
- `GgQuaternion & operator-=` (const GLfloat *a)
- `GgQuaternion & operator-=` (const `GgVector` &v)
- `GgQuaternion & operator-=` (const `GgQuaternion` &q)
- `GgQuaternion & operator*=` (const GLfloat *a)
- `GgQuaternion & operator*=` (const `GgVector` &v)
- `GgQuaternion & operator*=` (const `GgQuaternion` &q)
- `GgQuaternion & operator/=` (const GLfloat *a)
- `GgQuaternion & operator/=` (const `GgVector` &v)
- `GgQuaternion & operator/=` (const `GgQuaternion` &q)
- `GgQuaternion operator+` (const GLfloat *a) const
- `GgQuaternion operator+` (const `GgVector` &v) const
- `GgQuaternion operator+` (const `GgQuaternion` &q) const
- `GgQuaternion operator-` (const GLfloat *a) const

- GgQuaternion operator- (const GgVector &v) const
- GgQuaternion operator- (const GgQuaternion &q) const
- GgQuaternion operator* (const GLfloat *a) const
- GgQuaternion operator* (const GgVector &v) const
- GgQuaternion operator* (const GgQuaternion &q) const
- GgQuaternion operator/ (const GLfloat *a) const
- GgQuaternion operator/ (const GgVector &v) const
- GgQuaternion operator/ (const GgQuaternion &q) const
- GgQuaternion & loadMatrix (const GLfloat *a)
- GgQuaternion & loadMatrix (const GgMatrix &m)
- GgQuaternion & loadIdentity ()
- GgQuaternion & loadRotate (GLfloat x, GLfloat y, GLfloat z, GLfloat a)
- GgQuaternion & loadRotate (const GLfloat *v, GLfloat a)
- GgQuaternion & loadRotate (const GLfloat *v)
- GgQuaternion & loadRotateX (GLfloat a)
- GgQuaternion & loadRotateY (GLfloat a)
- GgQuaternion & loadRotateZ (GLfloat a)
- GgQuaternion rotate (GLfloat x, GLfloat y, GLfloat z, GLfloat a) const
- GgQuaternion rotate (const GLfloat *v, GLfloat a) const
- GgQuaternion rotate (const GLfloat *v) const
- GgQuaternion rotateX (GLfloat a) const
- GgQuaternion rotateY (GLfloat a) const
- GgQuaternion rotateZ (GLfloat a) const
- GgQuaternion & loadEuler (GLfloat heading, GLfloat pitch, GLfloat roll)
- GgQuaternion & loadEuler (const GLfloat *e)
- GgQuaternion euler (GLfloat heading, GLfloat pitch, GLfloat roll) const
- GgQuaternion euler (const GLfloat *e) const
- GgQuaternion euler (const GgVector &e) const
- GgQuaternion & loadSlerp (const GLfloat *a, const GLfloat *b, GLfloat t)
- GgQuaternion & loadSlerp (const GgQuaternion &q, const GgQuaternion &r, GLfloat t)
- GgQuaternion & loadSlerp (const GgQuaternion &q, const GLfloat *a, GLfloat t)
- GgQuaternion & loadSlerp (const GLfloat *a, const GgQuaternion &q, GLfloat t)
- GgQuaternion & loadNormalize (const GLfloat *a)
- GgQuaternion & loadNormalize (const GgQuaternion &q)
- GgQuaternion & loadConjugate (const GLfloat *a)
- GgQuaternion & loadConjugate (const GgQuaternion &q)
- GgQuaternion & loadInvert (const GLfloat *a)
- GgQuaternion & loadInvert (const GgQuaternion &q)
- GgQuaternion slerp (GLfloat *a, GLfloat t) const
- GgQuaternion slerp (const GgQuaternion &q, GLfloat t) const
- GgQuaternion normalize () const
- GgQuaternion conjugate () const
- GgQuaternion invert () const
- void get (GLfloat *a) const
- void getMatrix (GLfloat *a) const
- void getMatrix (GgMatrix &m) const
- GgMatrix getMatrix () const
- void getConjugateMatrix (GLfloat *a) const
- void getConjugateMatrix (GgMatrix &m) const
- GgMatrix getConjugateMatrix () const

基底クラス **gg::GgVector** に属する継承公開メンバ関数

- **GgVector** ()=default
- constexpr **GgVector** (GLfloat v0, GLfloat v1, GLfloat v2, GLfloat v3)
- constexpr **GgVector** (const GLfloat *a)
- constexpr **GgVector** (GLfloat c)
- **GgVector** (const GgVector &v)=default
- **GgVector** (GgVector &&v)=default
- ~**GgVector** ()=default
- **GgVector** & operator= (const **GgVector** &v)=default
- **GgVector** & operator= (**GgVector** &&v)=default
- constexpr **GgVector** operator+ (const **GgVector** &v) const noexcept
- constexpr **GgVector** operator+ (GLfloat c) const noexcept
- constexpr **GgVector** & operator+= (const **GgVector** &v) noexcept
- constexpr **GgVector** & operator+= (GLfloat c) noexcept
- constexpr **GgVector** operator- (const **GgVector** &v) const noexcept
- constexpr **GgVector** operator- (GLfloat c) const noexcept
- constexpr **GgVector** & operator-= (const **GgVector** &v) noexcept
- constexpr **GgVector** & operator-= (GLfloat c) noexcept
- constexpr **GgVector** operator* (const **GgVector** &v) const noexcept
- constexpr **GgVector** operator* (GLfloat c) const noexcept
- constexpr **GgVector** & operator*= (const **GgVector** &v) noexcept
- constexpr **GgVector** & operator*= (GLfloat c) noexcept
- constexpr **GgVector** operator/ (const **GgVector** &v) const noexcept
- constexpr **GgVector** operator/ (GLfloat c) const noexcept
- constexpr **GgVector** & operator/= (const **GgVector** &v) noexcept
- constexpr **GgVector** & operator/= (GLfloat c) noexcept
- GLfloat dot3 (const **GgVector** &v) const
- GLfloat length3 () const
- GLfloat distance3 (const **GgVector** &v) const
- **GgVector** normalize3 () const
- GLfloat dot4 (const **GgVector** &v) const
- GLfloat length4 () const
- GLfloat distance4 (const **GgVector** &v) const
- **GgVector** normalize4 () const

9.16.1 詳解

簡易トラックボール処理.
[gg.h](#) の 4806 行目に定義があります。

9.16.2 構築子と解体子

9.16.2.1 GgTrackball()

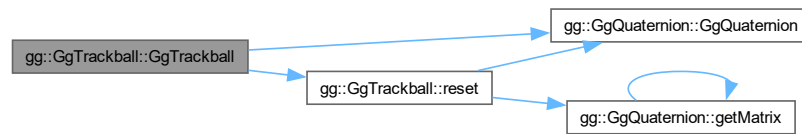
```
gg::GgTrackball::GgTrackball (
    const GgQuaternion & q = ggIdentityQuaternion()) [inline]
コンストラクタ.
```

引数

<i>q</i>	トラックボールの回転の初期値の四元数.
----------	---------------------

[gg.h](#) の 4821 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.16.2.2 ~GgTrackball()

`gg::GgTrackball::~~GgTrackball () [default]`
デストラクタ.

9.16.3 関数詳解

9.16.3.1 begin()

```
void gg::GgTrackball::begin (
    GLfloat x,
    GLfloat y)
```

トラックボール処理を開始する.

引数

<i>x</i>	現在のマウスの <i>x</i> 座標.
<i>y</i>	現在のマウスの <i>y</i> 座標.

覚え書き

マウスのドラッグ開始時 (マウスボタンを押したとき) に呼び出す.

[gg.cpp](#) の 3462 行目に定義があります.

9.16.3.2 end()

```
void gg::GgTrackball::end (
    GLfloat x,
    GLfloat y)
```

トラックボール処理を停止する.

引数

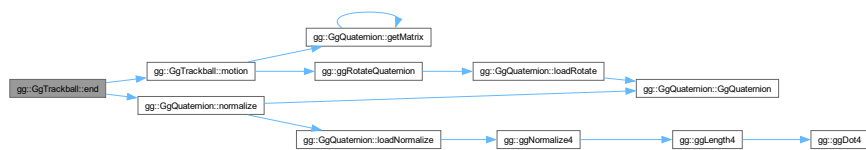
x	現在のマウスの x 座標.
y	現在のマウスの y 座標.

覚え書き

マウスのドラッグ終了時 (マウスボタンを離したとき) に呼び出す.

[gg.cpp](#) の 3528 行目に定義があります.

呼び出し関係図:



9.16.3.3 get()

```
const GLfloat * gg::GgTrackball::get () const [inline]
```

現在の回転の変換行列を取り出す.

戻り値

回転の変換を表す GLfloat 型の 16 要素の配列.

[gg.h](#) の 4999 行目に定義があります.

9.16.3.4 getMatrix()

```
const GgMatrix & gg::GgTrackball::getMatrix () const [inline]
```

現在の回転の変換行列を取り出す.

戻り値

回転の変換を表す GgMatrix 型の変換行列.

[gg.h](#) の 4989 行目に定義があります.

9.16.3.5 getQuaternion()

```
const GgQuaternion & gg::GgTrackball::getQuaternion () const [inline]
```

現在の回転の四元数を取り出す.

戻り値

回転の変換を表す Quaternion 型の四元数.

[gg.h](#) の 4979 行目に定義があります.

呼び出し関係図:



9.16.3.6 `getScale()` [1/3]

```
const GLfloat * gg::GgTrackball::getScale () const [inline]
```

トラックボール処理の換算係数を取り出す。

戻り値

トラックボールの換算係数のポインタ。

[gg.h](#) の 4948 行目に定義があります。

9.16.3.7 `getScale()` [2/3]

```
void gg::GgTrackball::getScale (
    GLfloat * factor) const [inline]
```

トラックボール処理の換算係数を取り出す。

引数

<i>factor</i>	トラックボールの換算係数を格納する 2 要素の配列。
---------------	----------------------------

[gg.h](#) の 4968 行目に定義があります。

9.16.3.8 `getScale()` [3/3]

```
const GLfloat gg::GgTrackball::getScale (
    int direction) const [inline]
```

トラックボール処理の換算係数を取り出す。

引数

<i>direction</i>	0 なら x 方向, 1 なら y 方向。
------------------	-----------------------

[gg.h](#) の 4958 行目に定義があります。

9.16.3.9 `getStart()` [1/3]

```
const GLfloat * gg::GgTrackball::getStart () const [inline]
```

トラックボール処理の開始位置を取り出す。

戻り値

トラックボールの開始位置のポインタ。

[gg.h](#) の 4919 行目に定義があります。

9.16.3.10 `getStart()` [2/3]

```
void gg::GgTrackball::getStart (
    GLfloat * position) const [inline]
```

トラックボール処理の開始位置を取り出す。

引数

<i>position</i>	トラックボールの開始位置を格納する 2 要素の配列。
-----------------	----------------------------

[gg.h](#) の 4937 行目に定義があります。

9.16.3.11 `getStart()` [3/3]

```
const GLfloat & gg::GgTrackball::getStart (
    int direction) const [inline]
```

トラックボール処理の開始位置を取り出す。

引数

<i>direction</i>	0 なら x 方向, 1 なら y 方向.
------------------	-----------------------

[gg.h](#) の 4927 行目に定義があります。

9.16.3.12 `motion()`

```
void gg::GgTrackball::motion (
    GLfloat x,
    GLfloat y)
```

回転の変換行列を計算する。

引数

<i>x</i>	現在のマウスの x 座標.
<i>y</i>	現在のマウスの y 座標.

覚え書き

マウスのドラッグ中に呼び出す。

[gg.cpp](#) の 3478 行目に定義があります。
呼び出し関係図:



被呼び出し関係図:



9.16.3.13 `operator=()`

```
GgTrackball & gg::GgTrackball::operator= (
    const GgQuaternion & q) [inline]
```

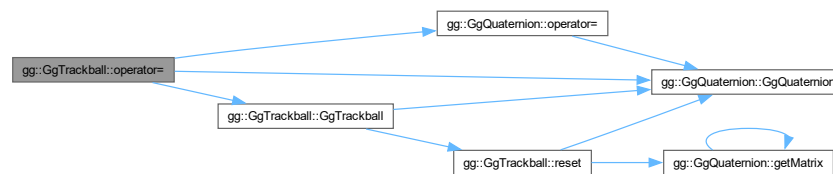

代入.

引数

q	トラックボールの回転の初期値の四元数.
-----	---------------------

gg.h の 4836 行目に定義があります。

呼び出し関係図:



9.16.3.14 region() [1/2]

```
void gg::GgTrackball::region (
    GLfloat w,
    GLfloat h)
```

トラックボール処理するマウスの移動範囲を指定する.

引数

w	領域の横幅.
h	領域の高さ.

覚え書き

ウィンドウのリサイズ時に呼び出す.

gg.cpp の 3449 行目に定義があります。

被呼び出し関係図:



9.16.3.15 region() [2/2]

```
void gg::GgTrackball::region (
    int w,
    int h) [inline]
```

トラックボール処理するマウスの移動範囲を指定する.

引数

<i>w</i>	領域の横幅.
<i>h</i>	領域の高さ.

覚え書き

ウィンドウのリサイズ時に呼び出す.

[gg.h](#) の 4862 行目に定義があります.

呼び出し関係図:



9.16.3.16 reset()

```
void gg::GgTrackball::reset (
    const GgQuaternion & q = ggIdentityQuaternion())
```

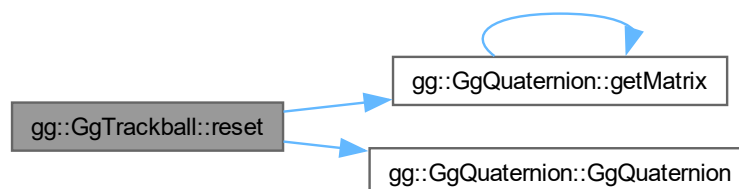
トラックボールをリセットする.

引数

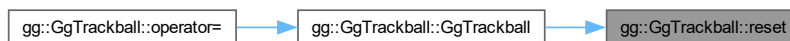
<i>q</i>	トラックボールの回転の初期値の四元数.
----------	---------------------

[gg.cpp](#) の 3431 行目に定義があります.

呼び出し関係図:



被呼び出し関係図:



9.16.3.17 rotate()

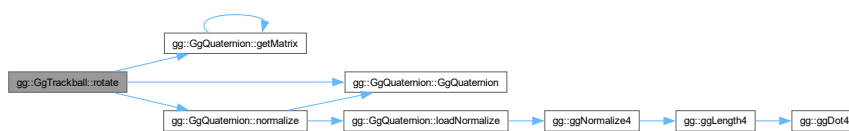
```
void gg::GgTrackball::rotate (
    const GgQuaternion & q)
    
```

トラックボールの回転角を修正する.

引数

<i>q</i>	修正分の回転角の四元数.
----------	--------------

[gg.cpp](#) の 3507 行目に定義があります。
呼び出し関係図:



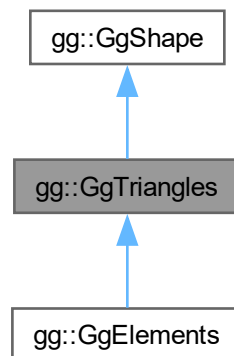
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

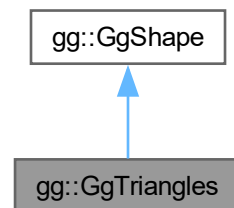
9.17 gg::GgTriangles クラス

```
#include <gg.h>
```

gg::GgTriangles の継承関係図



gg::GgTriangles 連携図



公開メンバ関数

- [GgTriangles](#) (GLenum mode=GL_TRIANGLES)
- [GgTriangles](#) (const [GgVertex](#) *vert, GLsizei count, GLenum mode=GL_TRIANGLES, GLenum usage=GL_STATIC_DRAW)
- virtual [~GgTriangles](#) ()
- const GLsizei & [getCount](#) () const
- const GLuint & [getBuffer](#) () const
- void [send](#) (const [GgVertex](#) *vert, GLint first=0, GLsizei count=0) const
- void [load](#) (const [GgVertex](#) *vert, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- virtual void [draw](#) (GLint first=0, GLsizei count=0) const

基底クラス [gg::GgShape](#) に属する継承公開メンバ関数

- [GgShape](#) (GLenum mode=0)
- virtual [~GgShape](#) ()
- virtual [operator bool](#) () const noexcept
- virtual bool [operator!](#) () const noexcept
- const GLuint & [get](#) () const

- void [setMode](#) (GLenum mode)
- const GLenum & [getMode](#) () const

9.17.1 詳解

三角形で表した形状データ (Arrays 形式).
[gg.h](#) の 6724 行目に定義があります。

9.17.2 構築子と解体子

9.17.2.1 GgTriangles() [1/2]

```
gg::GgTriangles::GgTriangles (
    GLenum mode = GL_TRIANGLES) [inline]
コンストラクタ.
```

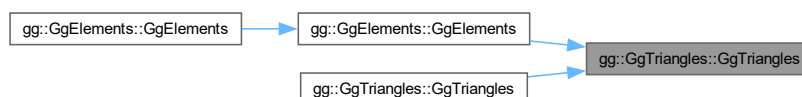
引数

<i>mode</i>	描画する基本図形の種類.
-------------	--------------

[gg.h](#) の 6737 行目に定義があります。
 呼び出し関係図:



被呼び出し関係図:



9.17.2.2 GgTriangles() [2/2]

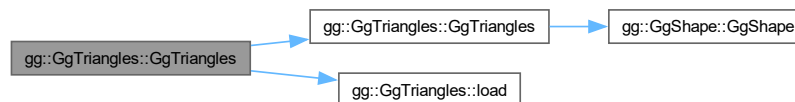
```
gg::GgTriangles::GgTriangles (
    const GgVertex * vert,
    GLsizei count,
    GLenum mode = GL_TRIANGLES,
    GLenum usage = GL_STATIC_DRAW) [inline]
コンストラクタ.
```

引数

<i>vert</i>	この図形の頂点属性の配列 (nullptr ならデータを転送しない).
-------------	-------------------------------------

<i>count</i>	頂点数.
<i>mode</i>	描画する基本図形の種類.
<i>usage</i>	バッファオブジェクトの使い方.

[gg.h](#) の 6750 行目に定義があります。
呼び出し関係図:



9.17.2.3 ~GgTriangles()

```
virtual gg::GgTriangles::~~GgTriangles () [inline], [virtual]
```

デストラクタ.

[gg.h](#) の 6764 行目に定義があります。

9.17.3 関数詳解

9.17.3.1 draw()

```
void gg::GgTriangles::draw (
    GLint first = 0,
    GLsizei count = 0) const [virtual]
```

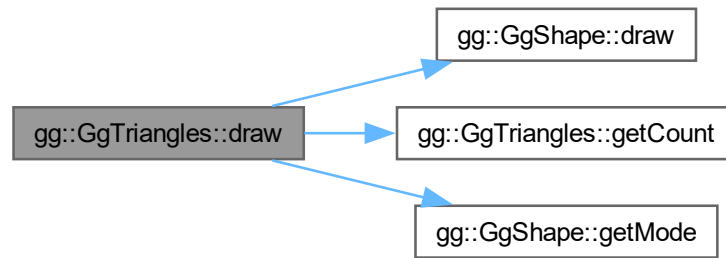
三角形の描画.

引数

<i>first</i>	描画を開始する最初の三角形番号.
<i>count</i>	描画する三角形数, 0 なら全部の三角形を描く.

[gg::GgShape](#) を再実装しています。
[gg::GgElements](#) で再実装されています。
[gg.cpp](#) の 5176 行目に定義があります。

呼び出し関係図:



9.17.3.2 getBuffer()

```
const GLuint & gg::GgTriangles::getBuffer () const [inline]
```

頂点属性を格納した頂点バッファオブジェクト名を取り出す.

戻り値

この図形の頂点属性を格納した頂点バッファオブジェクト名.

[gg.h](#) の 6783 行目に定義があります。

9.17.3.3 getCount()

```
const GLsizei & gg::GgTriangles::getCount () const [inline]
```

データの数を取り出す.

戻り値

この図形の頂点属性の数 (頂点数).

[gg.h](#) の 6773 行目に定義があります。

被呼び出し関係図:



9.17.3.4 load()

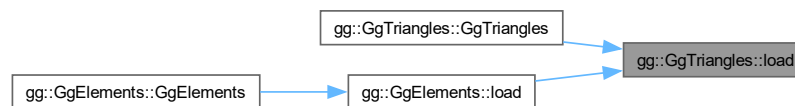
```
void gg::GgTriangles::load (
    const GgVertex * vert,
    GLsizei count,
    GLenum usage = GL_STATIC_DRAW)
```

バッファオブジェクトを確保して頂点属性を格納する.

引数

<i>vert</i>	頂点属性が格納されている領域の先頭のポインタ.
<i>count</i>	頂点のデータの数 (頂点数).
<i>usage</i>	バッファオブジェクトの使い方.

[gg.cpp](#) の 5157 行目に定義があります。
被呼び出し関係図:



9.17.3.5 send()

```
void gg::GgTriangles::send (
    const GgVertex * vert,
    GLint first = 0,
    GLsizei count = 0) const [inline]
```

既存のバッファオブジェクトに頂点属性を転送する.

引数

<i>vert</i>	転送元の頂点属性が格納されている領域の先頭のポインタ.
<i>first</i>	転送先のバッファオブジェクトの先頭の要素番号.
<i>count</i>	転送する頂点の位置データの数 (0 ならバッファオブジェクト全体).

[gg.h](#) の 6795 行目に定義があります。
被呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

9.18 gg::GgUniformBuffer< T > クラステンプレート

```
#include <gg.h>
```


公開メンバ関数

- [GgUniformBuffer](#) ()
- [GgUniformBuffer](#) (const T *data, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- [GgUniformBuffer](#) (const T &data, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- virtual [~GgUniformBuffer](#) ()
- const GLuint & [getTarget](#) () const
- GLsizeiptr [getStride](#) () const
- const GLsizei & [getCount](#) () const
- const GLuint & [getBuffer](#) () const
- void [bind](#) () const
- void [unbind](#) () const
- void * [map](#) () const
- void * [map](#) (GLuint first, GLsizei count) const
- void [unmap](#) () const
- void [load](#) (const T *data, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- void [load](#) (const T &data, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- void [send](#) (const GLvoid *data, GLint offset=0, GLsizei size=sizeof(T), GLint first=0, GLsizei count=0) const
- void [fill](#) (const GLvoid *data, GLint offset=0, GLsizei size=sizeof(T), GLint first=0, GLsizei count=0) const
- void [read](#) (GLvoid *data, GLint offset=0, GLsizei size=sizeof(T), GLint first=0, GLsizei count=0) const
- void [copy](#) (GLuint src_buffer, GLint src_first=0, GLint dst_first=0, GLsizei count=0) const

9.18.1 詳解

```
template<typename T>
class gg::GgUniformBuffer< T >
```

ユニフォームバッファオブジェクト。

覚え書き

ユニフォーム変数を格納するバッファオブジェクトの基底クラス。

[gg.h](#) の 6050 行目に定義があります。

9.18.2 構築子と解体子

9.18.2.1 GgUniformBuffer() [1/3]

```
template<typename T>
gg::GgUniformBuffer< T >::GgUniformBuffer () [inline]
コンストラクタ。
```

[gg.h](#) の 6060 行目に定義があります。

9.18.2.2 GgUniformBuffer() [2/3]

```
template<typename T>
gg::GgUniformBuffer< T >::GgUniformBuffer (
    const T * data,
    GLsizei count,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

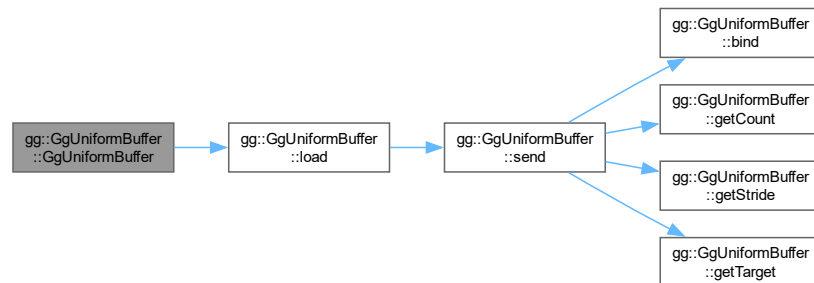
ユニフォームバッファオブジェクトのブロックごとにデータを転送するコンストラクタ。

引数

<i>data</i>	データが格納されている領域の先頭のポインタ (nullptr ならデータを転送しない)。
<i>count</i>	データの数。

<i>usage</i>	バッファオブジェクトの使い方.
--------------	-----------------

[gg.h](#) の 6071 行目に定義があります。
呼び出し関係図:



9.18.2.3 GgUniformBuffer() [3/3]

```

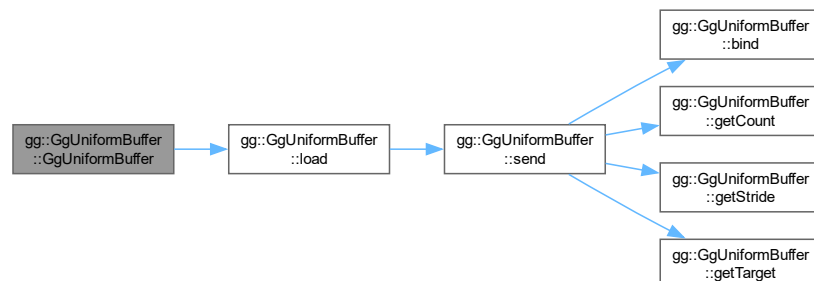
template<typename T>
gg::GgUniformBuffer< T >::GgUniformBuffer (
    const T & data,
    GLsizei count,
    GLenum usage = GL_STATIC_DRAW) [inline]
  
```

ユニフォームバッファオブジェクトの全ブロックに同じデータを格納するコンストラクタ.

引数

<i>data</i>	格納するデータ.
<i>count</i>	格納する数.
<i>usage</i>	バッファオブジェクトの使い方.

[gg.h](#) の 6083 行目に定義があります。
呼び出し関係図:



9.18.2.4 ~GgUniformBuffer()

```

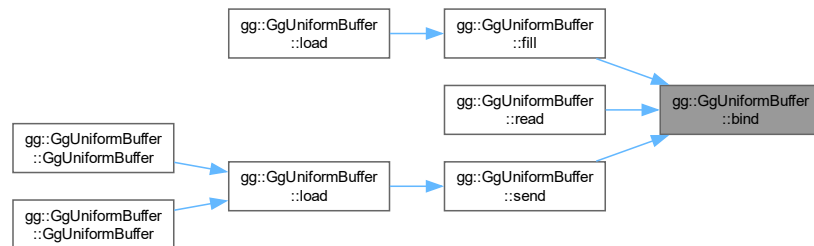
template<typename T>
  
```

virtual gg::GgUniformBuffer< T >::~~GgUniformBuffer () [inline], [virtual]
 デストラクタ.
 gg.h の 6091 行目に定義があります。

9.18.3 関数詳解

9.18.3.1 bind()

template<typename T>
 void gg::GgUniformBuffer< T >::bind () const [inline]
 ユニフォームバッファオブジェクトを結合する.
 gg.h の 6138 行目に定義があります。
 被呼び出し関係図:



9.18.3.2 copy()

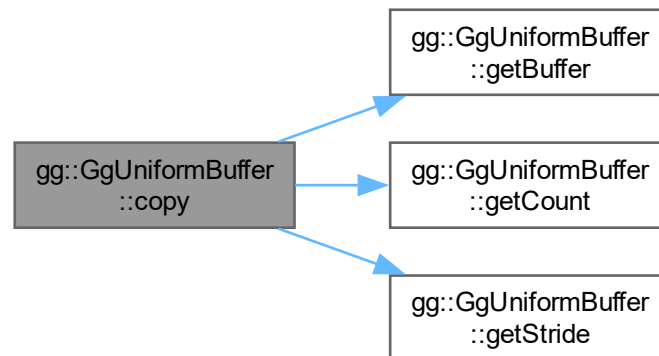
template<typename T>
 void gg::GgUniformBuffer< T >::copy (
 GLuint src_buffer,
 GLint src_first = 0,
 GLint dst_first = 0,
 GLsizei count = 0) const [inline]
 別のバッファオブジェクトからデータを複写する.

引数

<i>src_buffer</i>	複写元のバッファオブジェクト名.
<i>src_first</i>	複写元 (buffer) の先頭のデータの位置.
<i>dst_first</i>	複写先 (getBuffer()) の先頭のデータの位置.
<i>count</i>	複写するデータの数 (0 ならバッファオブジェクト全体).

gg.h の 6349 行目に定義があります。

呼び出し関係図:



9.18.3.3 fill()

```

template<typename T>
void gg::GgUniformBuffer< T >::fill (
    const GLvoid * data,
    GLint offset = 0,
    GLsizei size = sizeof(T),
    GLint first = 0,
    GLsizei count = 0) const [inline]
  
```

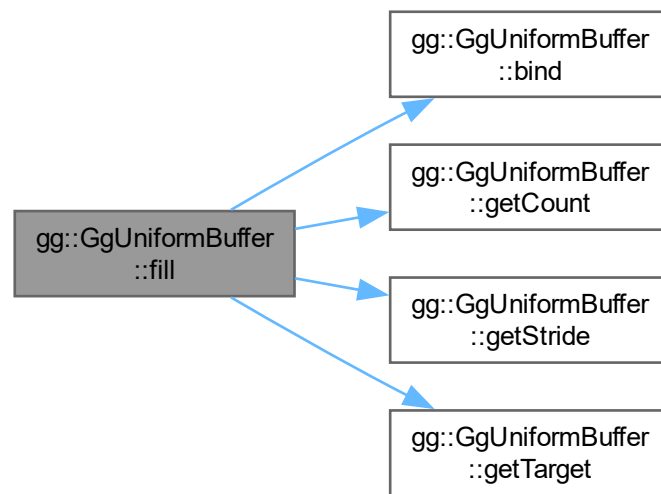
ユニフォームバッファオブジェクトの全ブロックのメンバーを同じデータを格納する.

引数

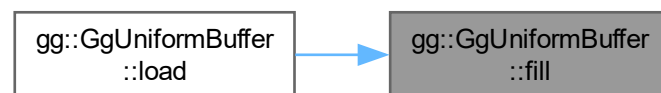
<i>data</i>	格納するデータ.
<i>offset</i>	格納先のメンバのブロックの先頭からのバイトオフセット.
<i>size</i>	格納するデータの一個あたりのバイト数.
<i>first</i>	格納先のバッファオブジェクトのブロックの先頭の番号.
<i>count</i>	格納するデータの数.

[gg.h](#) の 6266 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.18.3.4 getBuffer()

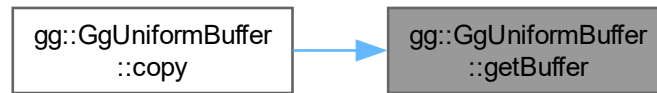
```
template<typename T>
const GLuint & gg::GgUniformBuffer< T >::getBuffer () const [inline]
ユニフォームバッファオブジェクト名を取り出す.
```

戻り値

このユニフォームバッファオブジェクト名.

gg.h の 6130 行目に定義があります。

被呼び出し関係図:



9.18.3.5 getCount()

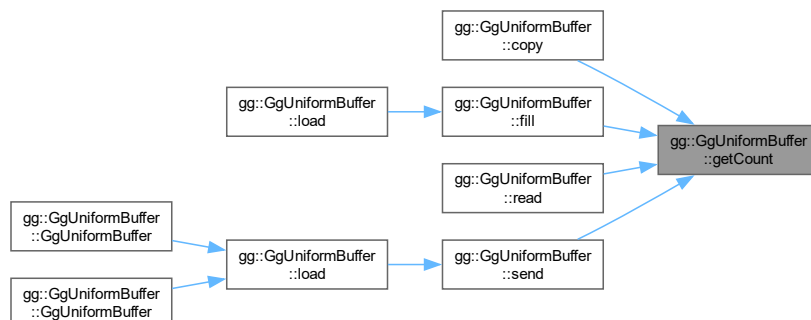
```
template<typename T>
const GLsizei & gg::GgUniformBuffer< T >::getCount () const [inline]
データの数を取り出す.
```

戻り値

このユニフォームバッファオブジェクトのデータの数.

gg.h の 6120 行目に定義があります。

被呼び出し関係図:



9.18.3.6 getStride()

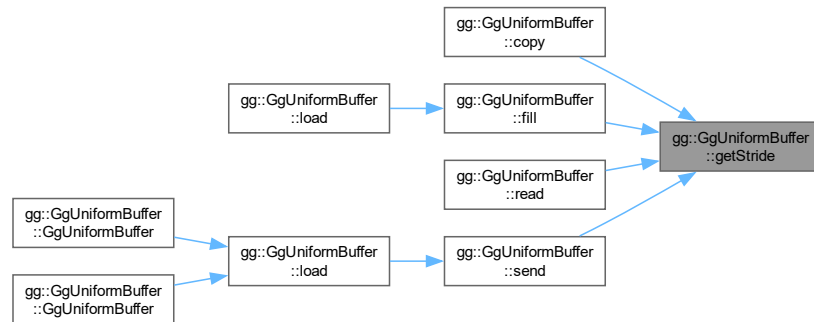
```
template<typename T>
GLsizeiptr gg::GgUniformBuffer< T >::getStride () const [inline]
ユニフォームバッファオブジェクトのアライメントを考慮したデータの間隔を取り出す.
```

戻り値

このユニフォームバッファオブジェクトのデータの間隔.

gg.h の 6110 行目に定義があります。

被呼び出し関係図:



9.18.3.7 getTarget()

```
template<typename T>
```

```
const GLuint & gg::GgUniformBuffer< T >::getTarget () const [inline]
```

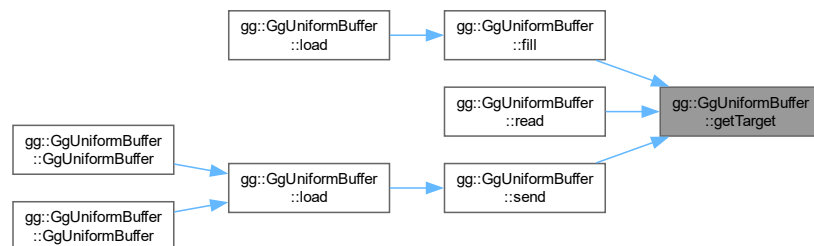
ユニフォームバッファオブジェクトのターゲットを取り出す。

戻り値

このユニフォームバッファオブジェクトのターゲット.

gg.h の 6100 行目に定義があります。

被呼び出し関係図:



9.18.3.8 load() [1/2]

```
template<typename T>
```

```
void gg::GgUniformBuffer< T >::load (
```

```
    const T & data,
```

```
    GLsizei count,
```

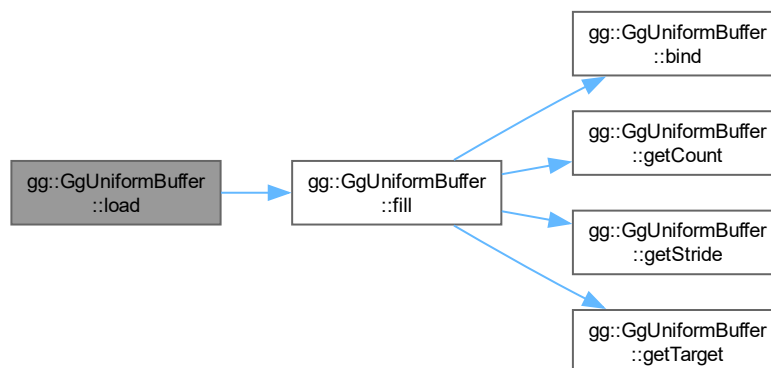
```
    GLenum usage = GL_STATIC_DRAW) [inline]
```

ユニフォームバッファオブジェクトを確保して全てのブロックに同じデータを格納する。

引数

<i>data</i>	格納するデータ.
<i>count</i>	格納する数.
<i>usage</i>	バッファオブジェクトの使い方.

gg.h の 6207 行目に定義があります。
呼び出し関係図:



9.18.3.9 load() [2/2]

```

template<typename T>
void gg::GgUniformBuffer< T >::load (
    const T * data,
    GLsizei count,
    GLenum usage = GL_STATIC_DRAW) [inline]

```

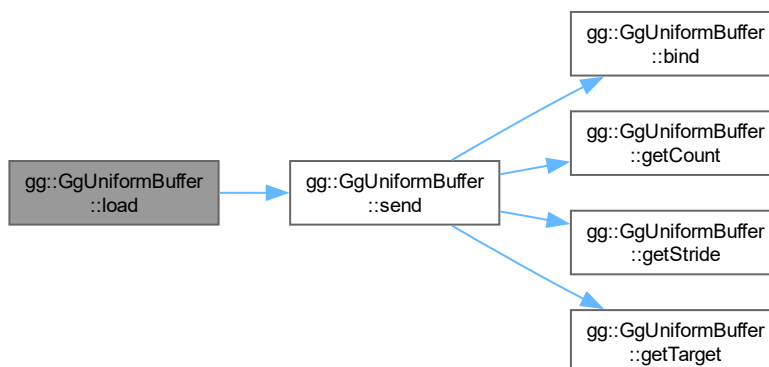
ユニフォームバッファオブジェクトを確保してブロックごとにデータを転送する.

引数

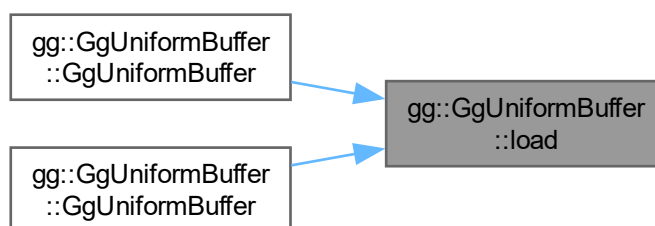
<i>data</i>	データが格納されている領域の先頭のポインタ (nullptr ならデータを転送しない).
<i>count</i>	データの数.
<i>usage</i>	バッファオブジェクトの使い方.

gg.h の 6188 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.18.3.10 map() [1/2]

```
template<typename T>
void * gg::GgUniformBuffer< T >::map () const [inline]
ユニフォームバッファオブジェクトをマップする.
```

戻り値

マップしたメモリの先頭のポインタ.

[gg.h](#) の 6156 行目に定義があります。

9.18.3.11 map() [2/2]

```
template<typename T>
void * gg::GgUniformBuffer< T >::map (
    GLint first,
    GLsizei count) const [inline]
ユニフォームバッファオブジェクトの指定した範囲をマップする.
```

引数

<i>first</i>	マップする範囲のバッファオブジェクトの先頭からの位置.
<i>count</i>	マップするデータの数 (0 ならバッファオブジェクト全体).

戻り値

マップしたメモリの先頭のポインタ.

[gg.h](#) の 6168 行目に定義があります。

9.18.3.12 read()

```
template<typename T>
void gg::GgUniformBuffer< T >::read (
    GLvoid * data,
    GLint offset = 0,
    GLsizei size = sizeof(T),
    GLint first = 0,
    GLsizei count = 0) const [inline]
```

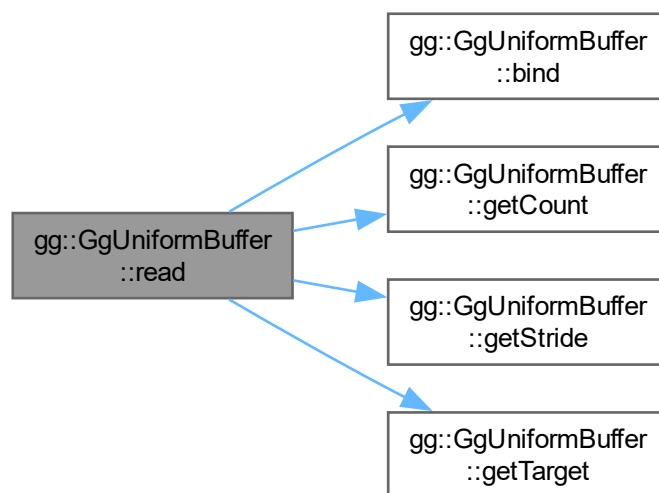
ユニフォームバッファオブジェクトからデータを抽出する.

引数

<i>data</i>	抽出先の領域の先頭のポインタ.
<i>offset</i>	抽出元のユニフォームバッファオブジェクトのメンバのブロックの先頭からのバイトオフセット.
<i>size</i>	抽出するデータの一個あたりのバイト数.
<i>first</i>	抽出元のユニフォームバッファオブジェクトのブロックの先頭の番号.
<i>count</i>	抽出するデータの数 (0 ならユニフォームバッファオブジェクト全体).

[gg.h](#) の 6301 行目に定義があります。

呼び出し関係図:



9.18.3.13 send()

```

template<typename T>
void gg::GgUniformBuffer< T >::send (
    const GLvoid * data,
    GLint offset = 0,
    GLsizei size = sizeof(T),
    GLint first = 0,
    GLsizei count = 0) const [inline]
  
```

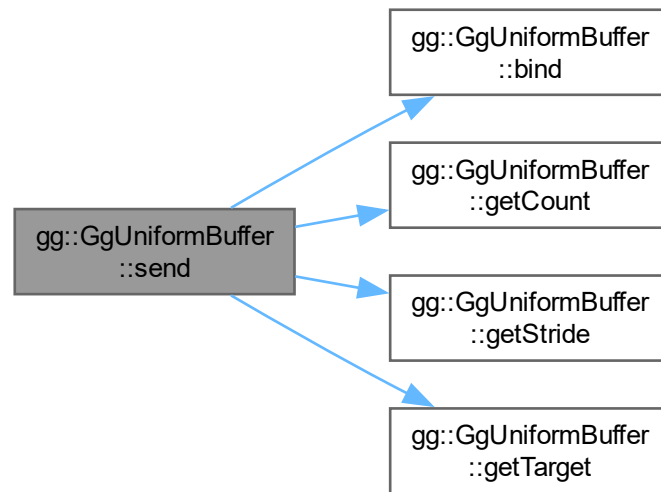
ユニフォームバッファオブジェクトを確保してユニフォームバッファオブジェクトのブロックごとのメンバを同じデータで埋める.

引数

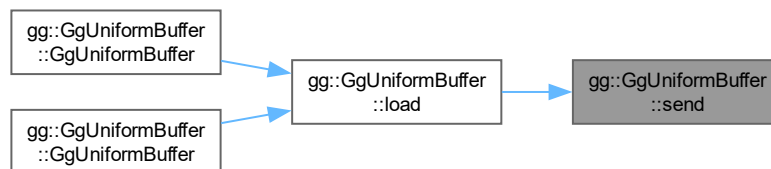
<i>data</i>	データが格納されている領域の先頭のポインタ.
<i>offset</i>	格納先のメンバのブロックの先頭からのバイトオフセット.
<i>size</i>	格納するデータの一個あたりのバイト数.
<i>first</i>	格納先のバッファオブジェクトのブロックの先頭の番号.
<i>count</i>	格納するデータの数.

[gg.h](#) の 6228 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.18.3.14 unbind()

```
template<typename T>
void gg::GgUniformBuffer< T >::unbind () const [inline]
```

ユニフォームバッファオブジェクトを解放する。
[gg.h](#) の 6146 行目に定義があります。

9.18.3.15 unmap()

```
template<typename T>
void gg::GgUniformBuffer< T >::unmap () const [inline]
```

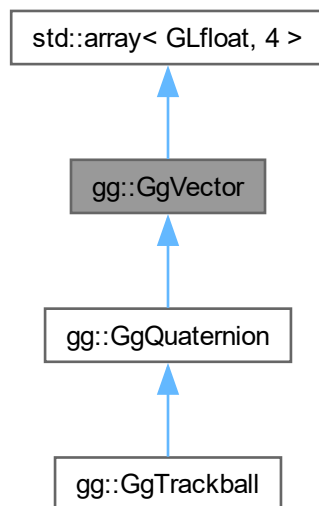
バッファオブジェクトをアンマップする。
[gg.h](#) の 6176 行目に定義があります。
 このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)

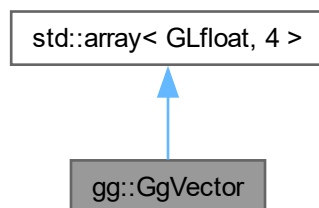
9.19 gg::GgVector クラス

```
#include <gg.h>
```

gg::GgVector の継承関係図



gg::GgVector 連携図



公開メンバ関数

- `GgVector` ()=default
- `constexpr GgVector` (GLfloat v0, GLfloat v1, GLfloat v2, GLfloat v3)
- `constexpr GgVector` (const GLfloat *a)
- `constexpr GgVector` (GLfloat c)
- `GgVector` (const GgVector &v)=default
- `GgVector` (GgVector &&v)=default
- `~GgVector` ()=default
- `GgVector & operator=` (const `GgVector` &v)=default
- `GgVector & operator=` (`GgVector` &&v)=default

- constexpr `GgVector operator+` (const `GgVector` &*v*) const noexcept
- constexpr `GgVector operator+` (GLfloat *c*) const noexcept
- constexpr `GgVector & operator+=` (const `GgVector` &*v*) noexcept
- constexpr `GgVector & operator+=` (GLfloat *c*) noexcept
- constexpr `GgVector operator-` (const `GgVector` &*v*) const noexcept
- constexpr `GgVector operator-` (GLfloat *c*) const noexcept
- constexpr `GgVector & operator-=` (const `GgVector` &*v*) noexcept
- constexpr `GgVector & operator-=` (GLfloat *c*) noexcept
- constexpr `GgVector operator*` (const `GgVector` &*v*) const noexcept
- constexpr `GgVector operator*` (GLfloat *c*) const noexcept
- constexpr `GgVector & operator*=` (const `GgVector` &*v*) noexcept
- constexpr `GgVector & operator*=` (GLfloat *c*) noexcept
- constexpr `GgVector operator/` (const `GgVector` &*v*) const noexcept
- constexpr `GgVector operator/` (GLfloat *c*) const noexcept
- constexpr `GgVector & operator/=` (const `GgVector` &*v*) noexcept
- constexpr `GgVector & operator/=` (GLfloat *c*) noexcept
- GLfloat `dot3` (const `GgVector` &*v*) const
- GLfloat `length3` () const
- GLfloat `distance3` (const `GgVector` &*v*) const
- `GgVector normalize3` () const
- GLfloat `dot4` (const `GgVector` &*v*) const
- GLfloat `length4` () const
- GLfloat `distance4` (const `GgVector` &*v*) const
- `GgVector normalize4` () const

9.19.1 詳解

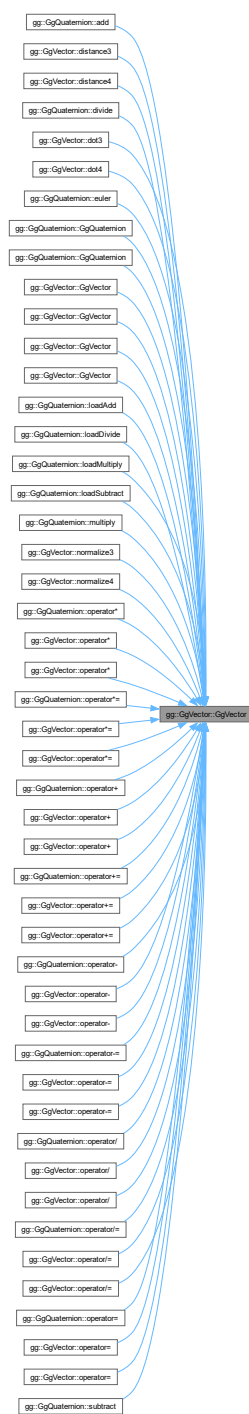
単精度実数の4要素のベクトル。
[gg.h](#) の 1577 行目に定義があります。

9.19.2 構築子と解体子

9.19.2.1 `GgVector()` [1/6]

```
gg::GgVector::GgVector () [default]
```

コンストラクタ. 被呼び出し関係図:



9.19.2.2 GgVector() [2/6]

```
gg::GgVector::GgVector (
    GLfloat v0,
    GLfloat v1,
    GLfloat v2,
    GLfloat v3) [inline], [constexpr]
```

コンストラクタ.

引数

<i>v0</i>	GLfloat 型の値.
<i>v1</i>	GLfloat 型の値.
<i>v2</i>	GLfloat 型の値.
<i>v3</i>	GLfloat 型の値.

[gg.h](#) の 1594 行目に定義があります。

9.19.2.3 GgVector() [3/6]

```
gg::GgVector::GgVector (
    const GLfloat * a) [inline], [constexpr]
コンストラクタ.
```

引数

<i>a</i>	GLfloat 型の 4 要素の配列変数.
----------	-----------------------

[gg.h](#) の 1604 行目に定義があります。
呼び出し関係図:



9.19.2.4 GgVector() [4/6]

```
gg::GgVector::GgVector (
    GLfloat c) [inline], [constexpr]
コンストラクタ.
```

引数

<i>c</i>	GLfloat 型の値.
----------	--------------

[gg.h](#) の 1614 行目に定義があります。
呼び出し関係図:



9.19.2.5 GgVector() [5/6]

```
gg::GgVector::GgVector (
    const GgVector & v) [default]
コピーコンストラクタ.
```

引数

<i>v</i>	コピー元のベクトル.
----------	------------

呼び出し関係図:

**9.19.2.6 GgVector()** [6/6]

```
gg::GgVector::GgVector (
    GgVector && v) [default]
ムーブコンストラクタ.
```

引数

<i>v</i>	ムーブ元のベクトル.
----------	------------

呼び出し関係図:

**9.19.2.7 ~GgVector()**

```
gg::GgVector::~~GgVector () [default]
デストラクタ.
```

9.19.3 関数詳解**9.19.3.1 distance3()**

```
GLfloat gg::GgVector::distance3 (
    const GgVector & v) const [inline]
```

GgVector 型の 3 要素の距離.

引数

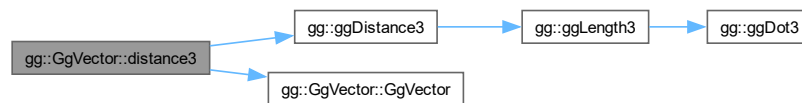
v	GgVector 型のベクトル.
----------	-------------------------

戻り値

オブジェクトと **v** の 3 要素の距離.

gg.h の 1883 行目に定義があります.

呼び出し関係図:



9.19.3.2 distance4()

```
GLfloat gg::GgVector::distance4 (
    const GgVector & v) const [inline]
```

GgVector 型の 4 要素の距離.

引数

v	GgVector 型のベクトル.
----------	-------------------------

戻り値

オブジェクトと **v** の 4 要素の距離.

gg.h の 1927 行目に定義があります.

呼び出し関係図:



9.19.3.3 dot3()

```
GLfloat gg::GgVector::dot3 (
    const GgVector & v) const [inline]
```

GgVector 型の 3 要素の内積.

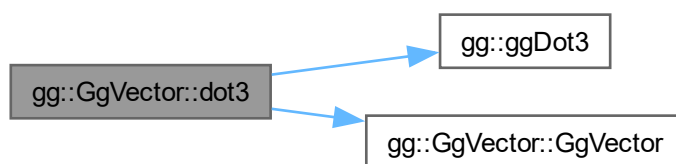
引数

v	GgVector 型のベクトル.
----------	-------------------------

戻り値

オブジェクトと **v** のそれぞれの 3 要素の内積.

gg.h の 1862 行目に定義があります。
呼び出し関係図:



9.19.3.4 dot4()

```
GLfloat gg::GgVector::dot4 (
    const GgVector & v) const [inline]
```

GgVector 型の 4 要素の内積.

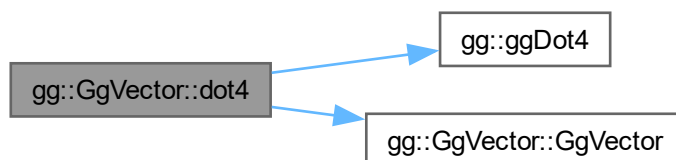
引数

v	GgVector 型のベクトル.
----------	-------------------------

戻り値

オブジェクトと **v** のそれぞれの 4 要素の内積.

gg.h の 1906 行目に定義があります。
呼び出し関係図:



9.19.3.5 length3()

GLfloat gg::GgVector::length3 () const [inline]

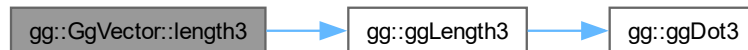
GgVector 型の 3 要素の長さ.

戻り値

オブジェクトの 3 要素の長さ.

gg.h の 1872 行目に定義があります。

呼び出し関係図:



9.19.3.6 length4()

GLfloat gg::GgVector::length4 () const [inline]

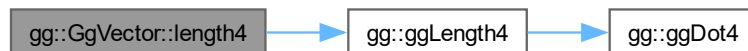
GgVector 型の 4 要素の長さ.

戻り値

オブジェクトの 4 要素の長さ.

gg.h の 1916 行目に定義があります。

呼び出し関係図:



9.19.3.7 normalize3()

GgVector gg::GgVector::normalize3 () const [inline]

GgVector 型の 3 要素の正規化.

戻り値

GLfloat 型の 4 要素の配列変数.

gg.h の 1893 行目に定義があります。

呼び出し関係図:



9.19.3.8 normalize4()

`GgVector` `gg::GgVector::normalize4 () const [inline]`

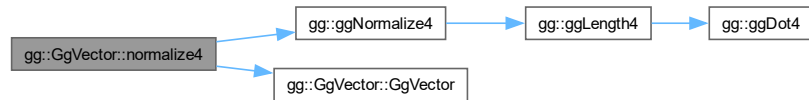
`GgVector` 型の 4 要素の正規化.

戻り値

GLfloat 型の 4 要素の配列変数.

`gg.h` の 1937 行目に定義があります.

呼び出し関係図:



9.19.3.9 operator*() [1/2]

`GgVector` `gg::GgVector::operator* (const GgVector & v) const [inline], [constexpr], [noexcept]`

`GgVector` 型の積を返す.

引数

<code>v</code>	<code>GgVector</code> 型の変数.
----------------	-----------------------------

戻り値

オブジェクトの各要素と `v` の各要素の要素ごとの積のオブジェクト.

`gg.h` の 1758 行目に定義があります.

呼び出し関係図:



9.19.3.10 operator*() [2/2]

`GgVector` `gg::GgVector::operator* (GLfloat c) const [inline], [constexpr], [noexcept]`

`GgVector` 型の各要素にスカラーを乗じた積を返す.

引数

<code>c</code>	GLfloat 型の変数.
----------------	---------------

戻り値

オブジェクトの各要素に `c` を乗じたオブジェクト.

`gg.h` の 1769 行目に定義があります.

呼び出し関係図:



9.19.3.11 operator*=() [1/2]

```
GgVector & gg::GgVector::operator*= (
    const GgVector & v) [inline], [constexpr], [noexcept]
```

`GgVector` 型を乗算する.

引数

<code>v</code>	<code>GgVector</code> 型の変数.
----------------	-----------------------------

戻り値

オブジェクトの各要素に `v` の各要素をそれぞれ乗算したオブジェクトの参照.

`gg.h` の 1780 行目に定義があります.

呼び出し関係図:



9.19.3.12 operator*=() [2/2]

```
GgVector & gg::GgVector::operator*= (
    GLfloat c) [inline], [constexpr], [noexcept]
```

`GgVector` 型の各要素にスカラーを乗じる.

引数

c	GLfloat 型の値.
---	--------------

戻り値

オブジェクトの各要素に **c** を乗じたオブジェクトの参照.

gg.h の 1795 行目に定義があります。
呼び出し関係図:



9.19.3.13 operator+() [1/2]

```
GgVector gg::GgVector::operator+ (
    const GgVector & v) const [inline], [constexpr], [noexcept]
```

GgVector 型の和を返す.

引数

v	GgVector 型のベクトル.
---	-------------------------

戻り値

オブジェクトの各要素と **v** の各要素の要素ごとの和のオブジェクト.

gg.h の 1654 行目に定義があります。
呼び出し関係図:



9.19.3.14 operator+() [2/2]

```
GgVector gg::GgVector::operator+ (
    GLfloat c) const [inline], [constexpr], [noexcept]
```

GgVector 型の各要素にスカラーを足した和を返す.

引数

c	GLfloat 型の値.
---	--------------

戻り値

a の各要素に b を足した和のオブジェクト.

gg.h の 1665 行目に定義があります.

呼び出し関係図:



9.19.3.15 operator+=() [1/2]

```
GgVector & gg::GgVector::operator+= (
    const GgVector & v) [inline], [constexpr], [noexcept]
```

GgVector 型を加算する.

引数

v	GgVector 型の変数.
---	----------------

戻り値

オブジェクトの各要素に v の各要素をそれぞれ加算したオブジェクトの参照.

gg.h の 1676 行目に定義があります.

呼び出し関係図:



9.19.3.16 operator+=() [2/2]

```
GgVector & gg::GgVector::operator+= (
    GLfloat c) [inline], [constexpr], [noexcept]
```

GgVector 型の各要素にスカラーを加算する.

引数

<code>c</code>	GLfloat 型の値.
----------------	--------------

戻り値

オブジェクトの各要素に `c` を足したオブジェクトの参照.

[gg.h](#) の 1691 行目に定義があります。

呼び出し関係図:



9.19.3.17 operator-() [1/2]

```
GgVector gg::GgVector::operator- (
    const GgVector & v) const [inline], [constexpr], [noexcept]
```

`GgVector` 型の差を返す.

引数

<code>v</code>	<code>GgVector</code> 型の変数.
----------------	-----------------------------

戻り値

オブジェクトの各要素と `v` の各要素の要素ごとの差のオブジェクト.

[gg.h](#) の 1706 行目に定義があります。

呼び出し関係図:



9.19.3.18 operator-() [2/2]

```
GgVector gg::GgVector::operator- (
    GLfloat c) const [inline], [constexpr], [noexcept]
```

`GgVector` 型の各要素からスカラーを引いた差を返す.

引数

<code>c</code>	GLfloat 型の変数.
----------------	---------------

戻り値

オブジェクトの各要素から `c` を引いたオブジェクト.

`gg.h` の 1717 行目に定義があります.

呼び出し関係図:



9.19.3.19 operator-=() [1/2]

```
GgVector & gg::GgVector::operator-= (
    const GgVector & v) [inline], [constexpr], [noexcept]
```

`GgVector` 型を減算する.

引数

<code>v</code>	<code>GgVector</code> 型の変数.
----------------	-----------------------------

戻り値

オブジェクトの各要素に `v` の各要素をそれぞれ減算したオブジェクトの参照.

`gg.h` の 1728 行目に定義があります.

呼び出し関係図:



9.19.3.20 operator-=() [2/2]

```
GgVector & gg::GgVector::operator-= (
    GLfloat c) [inline], [constexpr], [noexcept]
```

`GgVector` 型の各要素からスカラーを引く.

引数

c	GLfloat 型の変数.
---	---------------

戻り値

オブジェクトの各要素から **c** を引いたオブジェクトの参照.

gg.h の 1743 行目に定義があります。

呼び出し関係図:



9.19.3.21 operator/() [1/2]

```
GgVector gg::GgVector::operator/ (
    const GgVector & v) const [inline], [constexpr], [noexcept]
```

GgVector 型の商を返す.

引数

v	GgVector 型の変数.
---	-----------------------

戻り値

オブジェクトの各要素を **v** の各要素で要素ごとに割った結果のオブジェクト.

gg.h の 1810 行目に定義があります。

呼び出し関係図:



9.19.3.22 operator/() [2/2]

```
GgVector gg::GgVector::operator/ (
    GLfloat c) const [inline], [constexpr], [noexcept]
```

GgVector 型の各要素をスカラーで割った商を返す.

引数

c	GLfloat 型の変数.
---	---------------

戻り値

オブジェクトの各要素を `c` で割ったオブジェクト.

`gg.h` の 1821 行目に定義があります.

呼び出し関係図:



9.19.3.23 operator/=() [1/2]

```
GgVector & gg::GgVector::operator/= (
    const GgVector & v) [inline], [constexpr], [noexcept]
```

`GgVector` 型を除算する.

引数

v	<code>GgVector</code> 型の変数.
---	-----------------------------

戻り値

オブジェクトの各要素を `v` の各要素でそれぞれ除算したオブジェクトの参照.

`gg.h` の 1832 行目に定義があります.

呼び出し関係図:



9.19.3.24 operator/=() [2/2]

```
GgVector & gg::GgVector::operator/= (
    GLfloat c) [inline], [constexpr], [noexcept]
```

`GgVector` 型の各要素をスカラーで割る.

引数

c	GLfloat 型の変数.
---	---------------

戻り値

オブジェクトの各要素から c で割ったオブジェクトの参照.

gg.h の 1847 行目に定義があります。

呼び出し関係図:



9.19.3.25 operator=() [1/2]

```
GgVector & gg::GgVector::operator= (
    const GgVector & v) [default]
```

代入演算子. 呼び出し関係図:



9.19.3.26 operator=() [2/2]

```
GgVector & gg::GgVector::operator= (
    GgVector && v) [default]
```

ムーブ代入演算子. 呼び出し関係図:

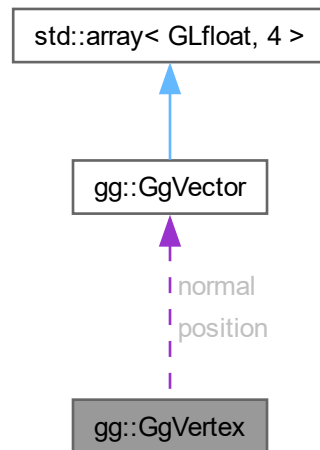


このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)

9.20 gg::GgVertex 構造体

```
#include <gg.h>
gg::GgVertex 連携図
```



公開メンバ関数

- [GgVertex \(\)](#)
- [GgVertex \(const \[GgVector\]\(#\) &pos, const \[GgVector\]\(#\) &norm\)](#)
- [GgVertex \(GLfloat px, GLfloat py, GLfloat pz, GLfloat nx, GLfloat ny, GLfloat nz\)](#)
- [GgVertex \(const GLfloat *pos, const GLfloat *norm\)](#)

公開変数類

- [GgVector position](#)
位置.
- [GgVector normal](#)
法線.

9.20.1 詳解

三角形の頂点データ.
[gg.h](#) の 6663 行目に定義があります。

9.20.2 構築子と解体子

9.20.2.1 GgVertex() [1/4]

```
gg::GgVertex::GgVertex () [inline]
```

コンストラクタ.
[gg.h](#) の 6674 行目に定義があります。

被呼び出し関係図:



9.20.2.2 GgVertex() [2/4]

```
gg::GgVertex::GgVertex (
    const GgVector & pos,
    const GgVector & norm) [inline]
```

コンストラクタ.

引数

<i>pos</i>	GgVector 型の位置データ.
<i>norm</i>	GgVector 型の法線データ.

[gg.h](#) の 6684 行目に定義があります。

9.20.2.3 GgVertex() [3/4]

```
gg::GgVertex::GgVertex (
    GLfloat px,
    GLfloat py,
    GLfloat pz,
    GLfloat nx,
    GLfloat ny,
    GLfloat nz) [inline]
```

コンストラクタ.

引数

<i>px</i>	GgVector 型の位置データの x 成分.
<i>py</i>	GgVector 型の位置データの y 成分.
<i>pz</i>	GgVector 型の位置データの z 成分.
<i>nx</i>	GgVector 型の法線データの x 成分.
<i>ny</i>	GgVector 型の法線データの y 成分.
<i>nz</i>	GgVector 型の法線データの z 成分.

[gg.h](#) の 6700 行目に定義があります。

9.20.2.4 GgVertex() [4/4]

```
gg::GgVertex::GgVertex (
    const GLfloat * pos,
    const GLfloat * norm) [inline]
```

コンストラクタ.

引数

<i>pos</i>	3 要素の GLfloat 型の位置データのポインタ.
<i>norm</i>	3 要素の GLfloat 型の法線データのポインタ.

gg.h の 6715 行目に定義があります。

呼び出し関係図:



9.20.3 メンバ詳解

9.20.3.1 normal

GgVector gg::GgVertex::normal

法線.

gg.h の 6669 行目に定義があります。

9.20.3.2 position

GgVector gg::GgVertex::position

位置.

gg.h の 6666 行目に定義があります。

この構造体詳解は次のファイルから抽出されました:

- gg.h

9.21 gg::GgVertexArray クラス

```
#include <gg.h>
```

公開メンバ関数

- GgVertexArray (GLenum mode=0)
- GgVertexArray (const GgVertexArray &array)=delete
- GgVertexArray (GgVertexArray &&array)=default
- virtual ~GgVertexArray ()
- GgVertexArray & operator= (const GgVertexArray &array)=delete
- const GLuint & get () const
- void bind () const

9.21.1 詳解

頂点配列 クラス.

gg.h の 6380 行目に定義があります。

9.21.2 構築子と解体子

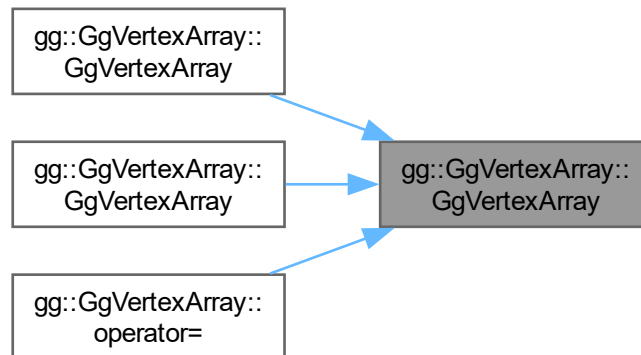
9.21.2.1 GgVertexArray() [1/3]

```
gg::GgVertexArray::GgVertexArray (
    GLenum mode = 0) [inline]
コンストラクタ.
```

引数

<i>mode</i>	基本図形の種類.
-------------	----------

[gg.h](#) の 6392 行目に定義があります。
被呼び出し関係図:



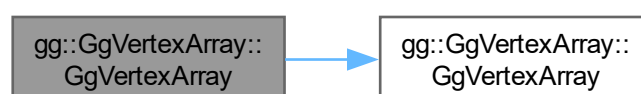
9.21.2.2 GgVertexArray() [2/3]

```
gg::GgVertexArray::GgVertexArray (
    const GgVertexArray & array) [delete]
コピーコンストラクタは使用しない.
```

引数

<i>array</i>	コピー元の頂点配列オブジェクト.
--------------	------------------

呼び出し関係図:



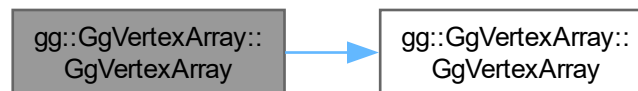
9.21.2.3 GgVertexArray() [3/3]

```
gg::GgVertexArray::GgVertexArray (
    GgVertexArray && array) [default]
ムーブコンストラクタ.
```

引数

<i>array</i>	ムーブ元の頂点配列オブジェクト.
--------------	------------------

呼び出し関係図:



9.21.2.4 ~GgVertexArray()

```
virtual gg::GgVertexArray::~~GgVertexArray () [inline], [virtual]
デストラクタ.
```

[gg.h](#) の 6415 行目に定義があります。

9.21.3 関数詳解

9.21.3.1 bind()

```
void gg::GgVertexArray::bind () const [inline]
```

頂点配列オブジェクトを結合する.

[gg.h](#) の 6442 行目に定義があります。

9.21.3.2 get()

```
const GLuint & gg::GgVertexArray::get () const [inline]
```

頂点配列オブジェクト名を取り出す.

戻り値

頂点配列オブジェクト名.

[gg.h](#) の 6434 行目に定義があります。

9.21.3.3 operator=()

```
GgVertexArray & gg::GgVertexArray::operator= (
    const GgVertexArray & array) [delete]
```

代入演算子は使用しない.

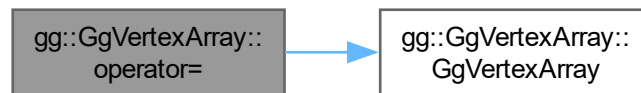
引数

<i>array</i>	代入元の頂点配列オブジェクト.
--------------	-----------------

戻り値

代入後のこの頂点配列オブジェクトの参照.

呼び出し関係図:



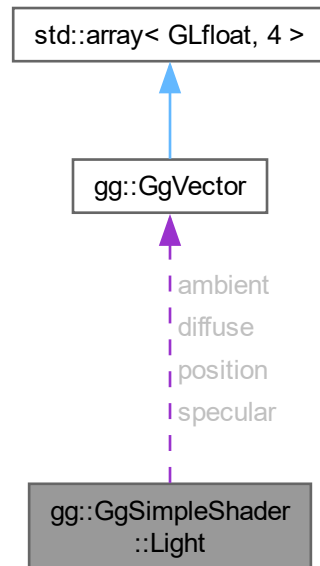
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)

9.22 gg::GgSimpleShader::Light 構造体

```
#include <gg.h>
```

```
gg::GgSimpleShader::Light 連携図
```



公開変数類

- [GgVector ambient](#)
光源強度の環境光成分.
- [GgVector diffuse](#)
光源強度の拡散反射光成分.

- [GgVector specular](#)
光源強度の鏡面反射光成分.
- [GgVector position](#)
光源の位置.

9.22.1 詳解

三角形に単純な陰影付けを行うシェーダが参照する光源データ.
[gg.h](#) の 7653 行目に定義があります。

9.22.2 メンバ詳解

9.22.2.1 ambient

[GgVector](#) `gg::GgSimpleShader::Light::ambient`
光源強度の環境光成分.
[gg.h](#) の 7655 行目に定義があります。

9.22.2.2 diffuse

[GgVector](#) `gg::GgSimpleShader::Light::diffuse`
光源強度の拡散反射光成分.
[gg.h](#) の 7656 行目に定義があります。

9.22.2.3 position

[GgVector](#) `gg::GgSimpleShader::Light::position`
光源の位置.
[gg.h](#) の 7658 行目に定義があります。

9.22.2.4 specular

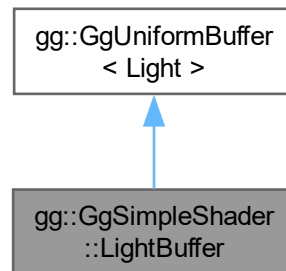
[GgVector](#) `gg::GgSimpleShader::Light::specular`
光源強度の鏡面反射光成分.
[gg.h](#) の 7657 行目に定義があります。
この構造体詳解は次のファイルから抽出されました:

- [gg.h](#)

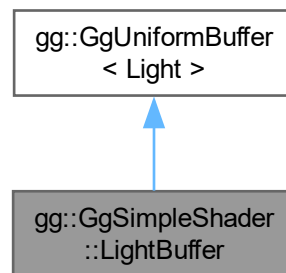
9.23 `gg::GgSimpleShader::LightBuffer` クラス

```
#include <gg.h>
```

gg::GgSimpleShader::LightBuffer の継承関係図



gg::GgSimpleShader::LightBuffer 連携図



公開メンバ関数

- `LightBuffer` (const `Light` *light=nullptr, GLsizei count=1, GLenum usage=GL_STATIC_DRAW)
- `LightBuffer` (const `Light` &light, GLsizei count=1, GLenum usage=GL_STATIC_DRAW)
- `LightBuffer` (`GgVector` ambient, `GgVector` diffuse, `GgVector` specular, `GgVector` position, GLsizei count=1, GLenum usage=GL_STATIC_DRAW)
- virtual `~LightBuffer` ()
- void `loadAmbient` (GLfloat r, GLfloat g, GLfloat b, GLfloat a=1.0f, GLint first=0, GLsizei count=1) const
- void `loadAmbient` (const `GgVector` &ambient, GLint first=0, GLsizei count=1) const
- void `loadAmbient` (const GLfloat *ambient, GLint first=0, GLsizei count=1) const
- void `loadDiffuse` (GLfloat r, GLfloat g, GLfloat b, GLfloat a=1.0f, GLint first=0, GLsizei count=1) const
- void `loadDiffuse` (const `GgVector` &diffuse, GLint first=0, GLsizei count=1) const
- void `loadDiffuse` (const GLfloat *diffuse, GLint first=0, GLsizei count=1) const
- void `loadSpecular` (GLfloat r, GLfloat g, GLfloat b, GLfloat a=1.0f, GLint first=0, GLsizei count=1) const
- void `loadSpecular` (const `GgVector` &specular, GLint first=0, GLsizei count=1) const
- void `loadSpecular` (const GLfloat *specular, GLint first=0, GLsizei count=1) const
- void `loadColor` (const `Light` &color, GLint first=0, GLsizei count=1) const
- void `loadPosition` (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f, GLint first=0, GLsizei count=1) const
- void `loadPosition` (const `GgVector` &position, GLint first=0, GLsizei count=1) const

- void `loadPosition` (const GLfloat *position, GLint first=0, GLsizei count=1) const
- void `loadPosition` (const GgVector *position, GLint first=0, GLsizei count=1) const
- void `load` (const Light *light, GLint first=0, GLsizei count=1) const
- void `load` (const Light &light, GLint first=0, GLsizei count=1) const
- void `select` (GLint i=0) const

基底クラス `gg::GgUniformBuffer< Light >` に属する継承公開メンバ関数

- `GgUniformBuffer` ()
- virtual `~GgUniformBuffer` ()
- const GLuint & `getTarget` () const
- GLsizeiptr `getStride` () const
- const GLsizei & `getCount` () const
- const GLuint & `getBuffer` () const
- void `bind` () const
- void `unbind` () const
- void * `map` () const
- void `unmap` () const
- void `load` (const Light *data, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- void `send` (const GLvoid *data, GLint offset=0, GLsizei size=sizeof(Light), GLint first=0, GLsizei count=0) const
- void `fill` (const GLvoid *data, GLint offset=0, GLsizei size=sizeof(Light), GLint first=0, GLsizei count=0) const
- void `read` (GLvoid *data, GLint offset=0, GLsizei size=sizeof(Light), GLint first=0, GLsizei count=0) const
- void `copy` (GLuint src_buffer, GLint src_first=0, GLint dst_first=0, GLsizei count=0) const

9.23.1 詳解

三角形に単純な陰影付けを行うシェーダが参照する光源データのユニフォームバッファオブジェクト。`gg.h` の 7664 行目に定義があります。

9.23.2 構築子と解体子

9.23.2.1 LightBuffer() [1/3]

```
gg::GgSimpleShader::LightBuffer::LightBuffer (
    const Light * light = nullptr,
    GLsizei count = 1,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

デフォルトコンストラクタ。

引数

<i>light</i>	GgSimpleShader::Light 型の光源データのポインタ。
<i>count</i>	バッファ中の GgSimpleShader::Light 型の光源データの数。
<i>usage</i>	バッファの使い方のパターン, glBufferData() の第 4 引数の usage に渡される。

`gg.h` の 7676 行目に定義があります。

呼び出し関係図:



9.23.2.2 LightBuffer() [2/3]

```

gg::GgSimpleShader::LightBuffer::LightBuffer (
    const Light & light,
    GLsizei count = 1,
    GLenum usage = GL_STATIC_DRAW) [inline]
  
```

同じデータで埋めるコンストラクタ.

引数

<i>light</i>	GgSimpleShader::Light 型の光源データ.
<i>count</i>	バッファ中の GgSimpleShader::Light 型の光源データの数.
<i>usage</i>	バッファの使い方のパターン, <code>glBufferData()</code> の第 4 引数の <code>usage</code> に渡される.

[gg.h](#) の 7692 行目に定義があります。

呼び出し関係図:



9.23.2.3 LightBuffer() [3/3]

```

gg::GgSimpleShader::LightBuffer::LightBuffer (
    GgVector ambient,
    GgVector diffuse,
    GgVector specular,
    GgVector position,
    GLsizei count = 1,
    GLenum usage = GL_STATIC_DRAW) [inline]
  
```

同じ引数で埋めるコンストラクタ.

引数

<i>ambient</i>	光源強度の環境光成分.
<i>diffuse</i>	光源強度の拡散反射光成分.
<i>specular</i>	光源強度の鏡面反射光成分.
<i>position</i>	光源の位置.
<i>count</i>	バッファ中の GgSimpleShader::Light 型の光源データの数.
<i>usage</i>	バッファの使い方のパターン, <code>glBufferData()</code> の第 4 引数の <code>usage</code> に渡される.

[gg.h](#) の 7711 行目に定義があります。
呼び出し関係図:



9.23.2.4 ~LightBuffer()

`virtual gg::GgSimpleShader::LightBuffer::~~LightBuffer () [inline], [virtual]`
デストラクタ.

[gg.h](#) の 7726 行目に定義があります。

9.23.3 関数詳解

9.23.3.1 load() [1/2]

```

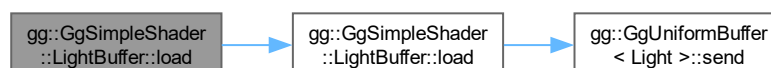
void gg::GgSimpleShader::LightBuffer::load (
    const Light & light,
    GLint first = 0,
    GLsizei count = 1) const [inline]
  
```

光源の色と位置を設定する。

引数

<i>light</i>	光源の特性の GgSimpleShader::Light 構造体.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.h](#) の 7918 行目に定義があります。
呼び出し関係図:



9.23.3.2 load() [2/2]

```
void gg::GgSimpleShader::LightBuffer::load (
    const Light * light,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

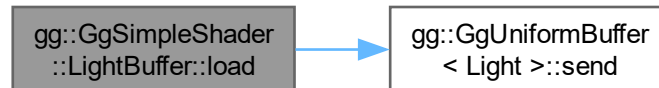
光源の色と位置を設定する。

引数

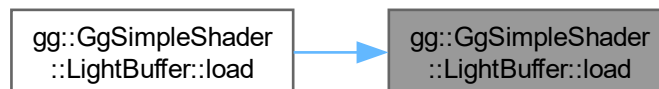
<i>light</i>	光源の特性の GgSimpleShader::Light 構造体のポインタ。
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.h](#) の 7906 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.23.3.3 loadAmbient() [1/3]

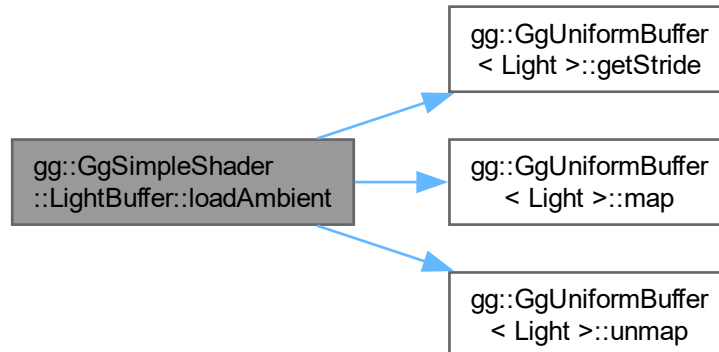
```
void gg::GgSimpleShader::LightBuffer::loadAmbient (
    const GgVector & ambient,
    GLint first = 0,
    GLsizei count = 1) const
```

光源の強度の環境光成分を設定する。

引数

<i>ambient</i>	光源の強度の環境光成分を格納した GgVector 型の変数。
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.cpp](#) の 5504 行目に定義があります。
呼び出し関係図:



9.23.3.4 loadAmbient() [2/3]

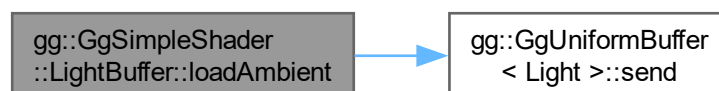
```
void gg::GgSimpleShader::LightBuffer::loadAmbient (
    const GLfloat * ambient,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

光源の強度の環境光成分を設定する。

引数

<i>ambient</i>	光源の強度の環境光成分を格納した GLfloat 型の 4 要素の配列変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.h](#) の 7761 行目に定義があります。
呼び出し関係図:



9.23.3.5 loadAmbient() [3/3]

```
void gg::GgSimpleShader::LightBuffer::loadAmbient (
    GLfloat r,
```

```

GLfloat g,
GLfloat b,
GLfloat a = 1.0f,
GLint first = 0,
GLsizei count = 1) const

```

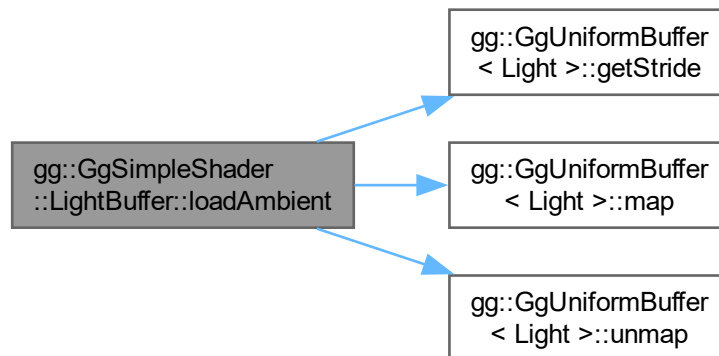
光源の強度の環境光成分を設定する。

引数

<i>r</i>	光源の強度の環境光成分の赤成分.
<i>g</i>	光源の強度の環境光成分の緑成分.
<i>b</i>	光源の強度の環境光成分の青成分.
<i>a</i>	光源の強度の拡散反射光成分の不透明度, デフォルトは 1.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

gg.cpp の 5478 行目に定義があります。

呼び出し関係図:



9.23.3.6 loadColor()

```

void gg::GgSimpleShader::LightBuffer::loadColor (
    const Light & color,
    GLint first = 0,
    GLsizei count = 1) const

```

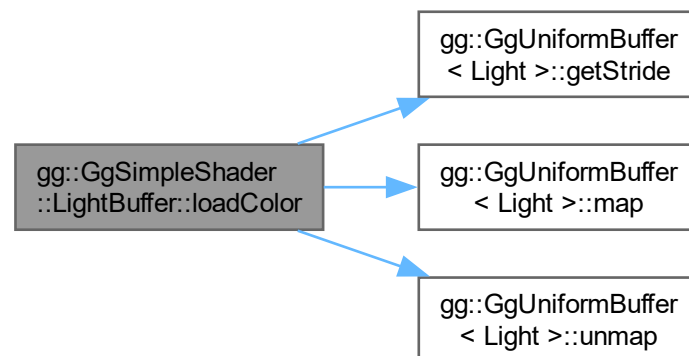
光源の色を設定するか位置は変更しない。

引数

<i>color</i>	光源の特性の <code>GgSimpleShader::Light</code> 構造体.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

gg.cpp の 5631 行目に定義があります。

呼び出し関係図:



9.23.3.7 loadDiffuse() [1/3]

```
void gg::GgSimpleShader::LightBuffer::loadDiffuse (  
    const GgVector & diffuse,  
    GLint first = 0,  
    GLsizei count = 1) const
```

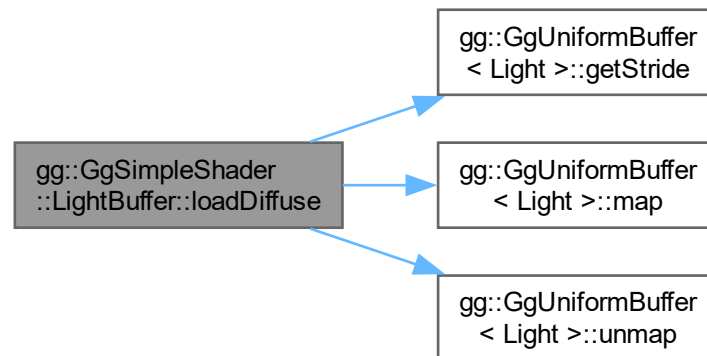
光源の強度の拡散反射光成分を設定する.

引数

<i>diffuse</i>	光源の強度の拡散反射光成分を格納した <code>GgVector</code> 型の変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

`gg.cpp` の 5556 行目に定義があります。

呼び出し関係図:



9.23.3.8 loadDiffuse() [2/3]

```
void gg::GgSimpleShader::LightBuffer::loadDiffuse (
    const GLfloat * diffuse,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

光源の強度の拡散反射光成分を設定する。

引数

<i>diffuse</i>	光源の強度の拡散反射光成分を格納した GLfloat 型の 4 要素の配列変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

gg.h の 7798 行目に定義があります。

呼び出し関係図:



9.23.3.9 loadDiffuse() [3/3]

```
void gg::GgSimpleShader::LightBuffer::loadDiffuse (
    GLfloat r,
    GLfloat g,
```

```

GLfloat b,
GLfloat a = 1.0f,
GLint first = 0,
GLsizei count = 1) const

```

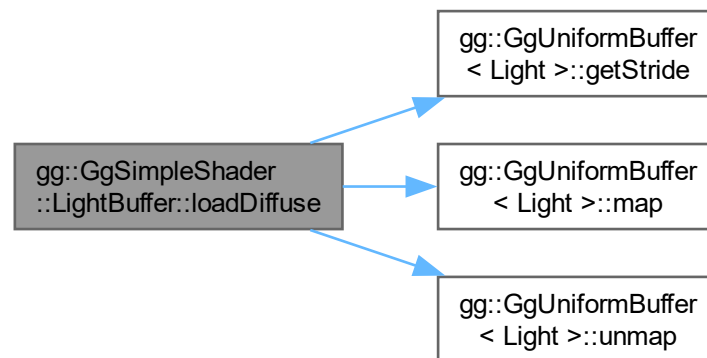
光源の強度の拡散反射光成分を設定する。

引数

<i>r</i>	光源の強度の拡散反射光成分の赤成分.
<i>g</i>	光源の強度の拡散反射光成分の緑成分.
<i>b</i>	光源の強度の拡散反射光成分の青成分.
<i>a</i>	光源の強度の拡散反射光成分の不透明度, デフォルトは 1.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.cpp](#) の 5530 行目に定義があります。

呼び出し関係図:



9.23.3.10 loadPosition() [1/4]

```

void gg::GgSimpleShader::LightBuffer::loadPosition (
    const GgVector & position,
    GLint first = 0,
    GLsizei count = 1) const

```

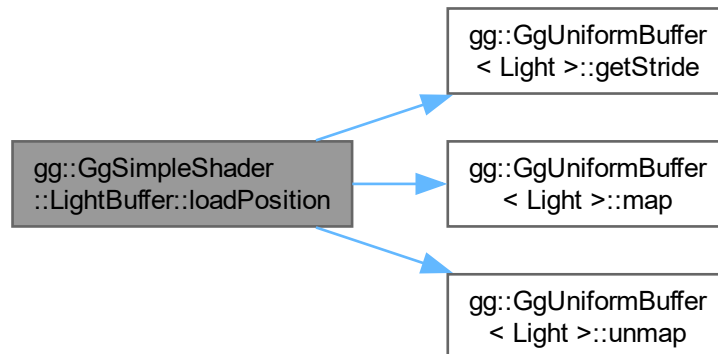
光源の位置を設定する。

引数

<i>position</i>	光源の位置の同次座標を格納した GgVector 型の変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.cpp](#) の 5685 行目に定義があります。

呼び出し関係図:



9.23.3.11 loadPosition() [2/4]

```
void gg::GgSimpleShader::LightBuffer::loadPosition (
    const GgVector * position,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

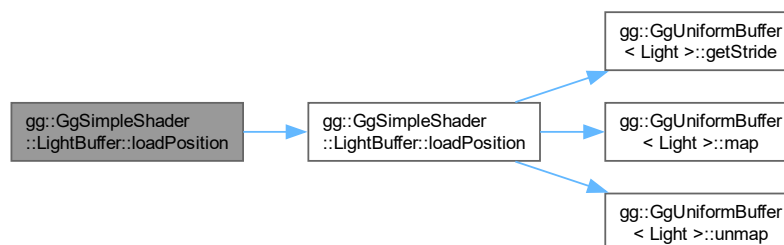
光源の位置を設定する.

引数

<i>position</i>	光源の位置の同次座標を格納した GgVector 型の配列.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.h](#) の 7894 行目に定義があります。

呼び出し関係図:



9.23.3.12 loadPosition() [3/4]

```
void gg::GgSimpleShader::LightBuffer::loadPosition (
```

```
const GLfloat * position,
GLint first = 0,
GLsizei count = 1) const [inline]
```

光源の位置を設定する。

引数

<i>position</i>	光源の位置の同次座標を格納した GLfloat 型の 4 要素の配列変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

gg.h の 7881 行目に定義があります。

呼び出し関係図:



9.23.3.13 loadPosition() [4/4]

```
void gg::GgSimpleShader::LightBuffer::loadPosition (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w = 1.0f,
    GLint first = 0,
    GLsizei count = 1) const
```

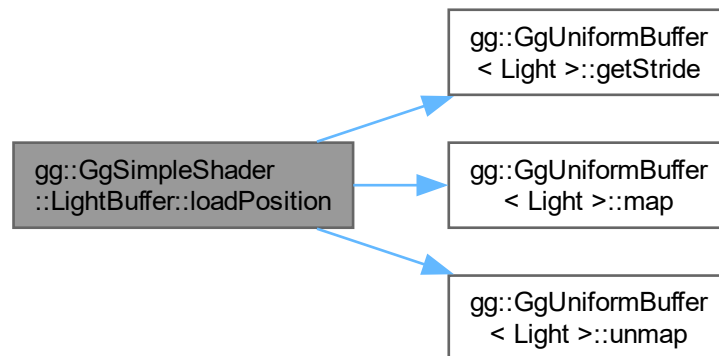
光源の位置を設定する。

引数

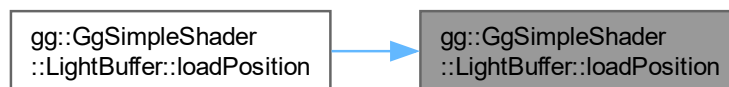
<i>x</i>	光源の位置の x 座標.
<i>y</i>	光源の位置の y 座標.
<i>z</i>	光源の位置の z 座標.
<i>w</i>	光源の位置の w 座標, デフォルトは 1.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

gg.cpp の 5659 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.23.3.14 loadSpecular() [1/3]

```
void gg::GgSimpleShader::LightBuffer::loadSpecular (
    const GgVector & specular,
    GLint first = 0,
    GLsizei count = 1) const
```

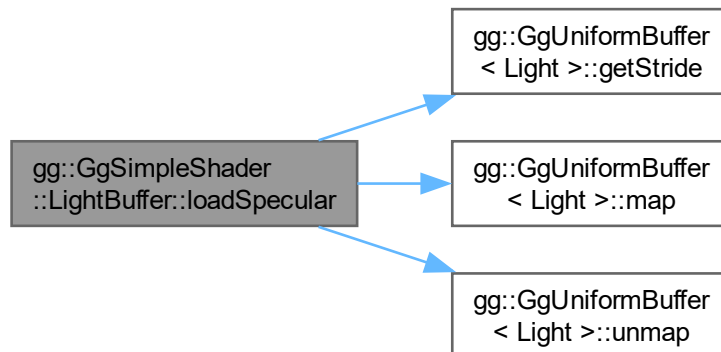
光源の強度の鏡面反射光成分を設定する.

引数

<i>specular</i>	光源の強度の鏡面反射光成分を格納した <code>GgVector</code> 型の変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

`gg.cpp` の 5608 行目に定義があります。

呼び出し関係図:



9.23.3.15 loadSpecular() [2/3]

```
void gg::GgSimpleShader::LightBuffer::loadSpecular (
    const GLfloat * specular,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

光源の強度の鏡面反射光成分を設定する.

引数

<i>specular</i>	光源の強度の鏡面反射光成分を格納した GLfloat 型の 4 要素の配列変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.h](#) の 7835 行目に定義があります.

呼び出し関係図:



9.23.3.16 loadSpecular() [3/3]

```
void gg::GgSimpleShader::LightBuffer::loadSpecular (
    GLfloat r,
    GLfloat g,
```

```

GLfloat b,
GLfloat a = 1.0f,
GLint first = 0,
GLsizei count = 1) const

```

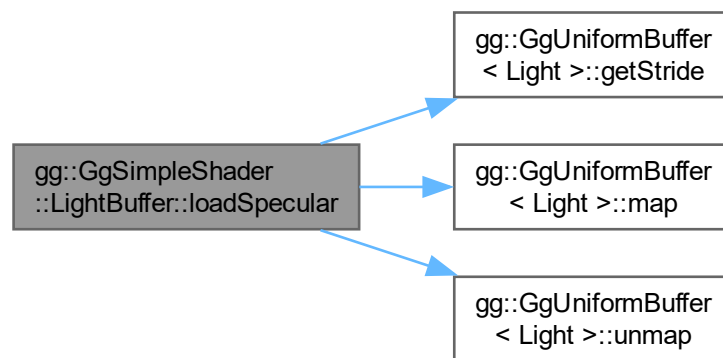
光源の強度の鏡面反射光成分を設定する。

引数

<i>r</i>	光源の強度の鏡面反射光成分の赤成分.
<i>g</i>	光源の強度の鏡面反射光成分の緑成分.
<i>b</i>	光源の強度の鏡面反射光成分の青成分.
<i>a</i>	光源の強度の鏡面反射光成分の不透明度, デフォルトは 1.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

gg.cpp の 5582 行目に定義があります。

呼び出し関係図:



9.23.3.17 select()

```

void gg::GgSimpleShader::LightBuffer::select (
    GLint i = 0) const [inline]

```

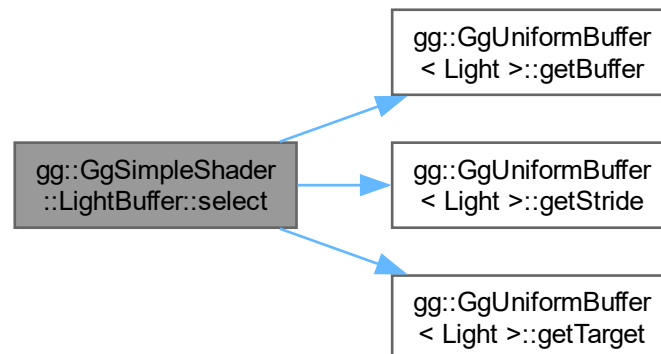
光源を選択する。

引数

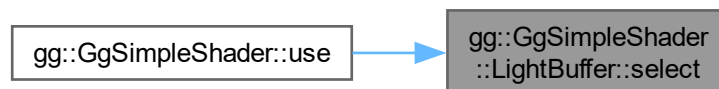
<i>i</i>	光源データの uniform block のインデックス.
----------	-------------------------------

gg.h の 7928 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



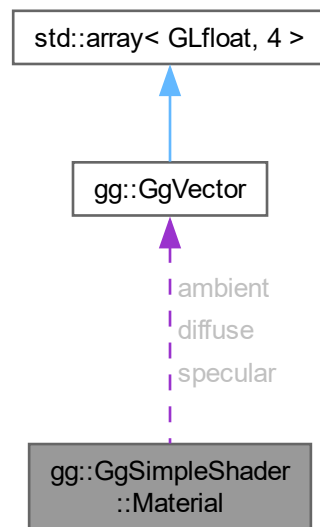
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

9.24 gg::GgSimpleShader::Material 構造体

```
#include <gg.h>
```

gg::GgSimpleShader::Material 連携図



公開変数類

- `GgVector ambient`
環境光に対する反射係数.
- `GgVector diffuse`
拡散反射係数.
- `GgVector specular`
鏡面反射係数.
- `GLfloat shininess`
輝き係数.

9.24.1 詳解

三角形に単純な陰影付けを行うシェーダが参照する材質データ.
`gg.h` の 7939 行目に定義があります。

9.24.2 メンバ詳解

9.24.2.1 ambient

`GgVector gg::GgSimpleShader::Material::ambient`
環境光に対する反射係数.
`gg.h` の 7941 行目に定義があります。

9.24.2.2 diffuse

`GgVector gg::GgSimpleShader::Material::diffuse`
拡散反射係数.
`gg.h` の 7942 行目に定義があります。

9.24.2.3 shininess

GLfloat gg::GgSimpleShader::Material::shininess
 輝き係数.

gg.h の 7944 行目に定義があります。

9.24.2.4 specular

GgVector gg::GgSimpleShader::Material::specular
 鏡面反射係数.

gg.h の 7943 行目に定義があります。

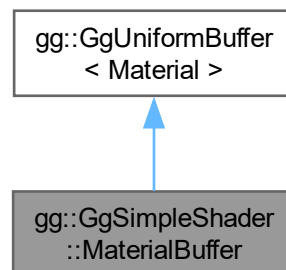
この構造体詳解は次のファイルから抽出されました:

- gg.h

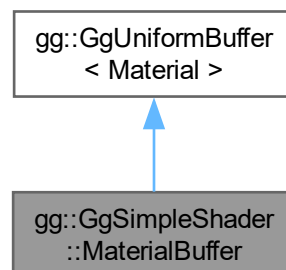
9.25 gg::GgSimpleShader::MaterialBuffer クラス

```
#include <gg.h>
```

gg::GgSimpleShader::MaterialBuffer の継承関係図



gg::GgSimpleShader::MaterialBuffer 連携図



公開メンバ関数

- **MaterialBuffer** (const **Material** *material=nullptr, GLsizei count=1, GLenum usage=GL_STATIC_DRAW)

- `MaterialBuffer` (const `Material` &material, GLsizei count=1, GLenum usage=GL_STATIC_DRAW)
- `MaterialBuffer` (`GgVector` ambient, `GgVector` diffuse, `GgVector` specular, GLfloat shininess, GLsizei count=1, GLenum usage=GL_STATIC_DRAW)
- virtual `~MaterialBuffer` ()
- void `loadAmbient` (GLfloat r, GLfloat g, GLfloat b, GLfloat a=1.0f, GLint first=0, GLsizei count=1) const
- void `loadAmbient` (const `GgVector` &ambient, GLint first=0, GLsizei count=1) const
- void `loadAmbient` (const GLfloat *ambient, GLint first=0, GLsizei count=1) const
- void `loadDiffuse` (GLfloat r, GLfloat g, GLfloat b, GLfloat a=1.0f, GLint first=0, GLsizei count=1) const
- void `loadDiffuse` (const `GgVector` &diffuse, GLint first=0, GLsizei count=1) const
- void `loadDiffuse` (const GLfloat *diffuse, GLint first=0, GLsizei count=1) const
- void `loadAmbientAndDiffuse` (GLfloat r, GLfloat g, GLfloat b, GLfloat a=1.0f, GLint first=0, GLsizei count=1) const
- void `loadAmbientAndDiffuse` (const `GgVector` &color, GLint first=0, GLsizei count=1) const
- void `loadAmbientAndDiffuse` (const GLfloat *color, GLint first=0, GLsizei count=1) const
- void `loadSpecular` (GLfloat r, GLfloat g, GLfloat b, GLfloat a=1.0f, GLint first=0, GLsizei count=1) const
- void `loadSpecular` (const `GgVector` &specular, GLint first=0, GLsizei count=1) const
- void `loadSpecular` (const GLfloat *specular, GLint first=0, GLsizei count=1) const
- void `loadShininess` (GLfloat shininess, GLint first=0, GLsizei count=1) const
- void `loadShininess` (const GLfloat *shininess, GLint first=0, GLsizei count=1) const
- void `load` (const `Material` *material, GLint first=0, GLsizei count=1) const
- void `load` (const `Material` &material, GLint first=0, GLsizei count=1) const
- void `select` (GLint i=0) const

基底クラス `gg::GgUniformBuffer< Material >` に属する継承公開メンバ関数

- `GgUniformBuffer` ()
- virtual `~GgUniformBuffer` ()
- const GLuint & `getTarget` () const
- GLsizeiptr `getStride` () const
- const GLsizei & `getCount` () const
- const GLuint & `getBuffer` () const
- void `bind` () const
- void `unbind` () const
- void * `map` () const
- void `unmap` () const
- void `load` (const `Material` *data, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- void `send` (const GLvoid *data, GLint offset=0, GLsizei size=sizeof(Material), GLint first=0, GLsizei count=0) const
- void `fill` (const GLvoid *data, GLint offset=0, GLsizei size=sizeof(Material), GLint first=0, GLsizei count=0) const
- void `read` (GLvoid *data, GLint offset=0, GLsizei size=sizeof(Material), GLint first=0, GLsizei count=0) const
- void `copy` (GLuint src_buffer, GLint src_first=0, GLint dst_first=0, GLsizei count=0) const

9.25.1 詳解

三角形に単純な陰影付けを行うシェーダが参照する材質データのユニフォームバッファオブジェクト. `gg.h` の 7950 行目に定義があります。

9.25.2 構築子と解体子

9.25.2.1 MaterialBuffer() [1/3]

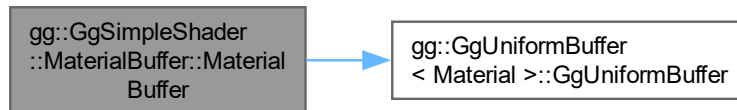
```
gg::GgSimpleShader::MaterialBuffer::MaterialBuffer (  
    const Material * material = nullptr,  
    GLsizei count = 1,  
    GLenum usage = GL_STATIC_DRAW) [inline]
```

デフォルトコンストラクタ.

引数

<i>material</i>	GgSimpleShader::Material 型の材質データのポインタ.
<i>count</i>	バッファ中の GgSimpleShader::Material 型の材質データの数.
<i>usage</i>	バッファの使い方のパターン, glBufferData() の第 4 引数の usage に渡される.

gg.h の 7962 行目に定義があります。
呼び出し関係図:



9.25.2.2 MaterialBuffer() [2/3]

```

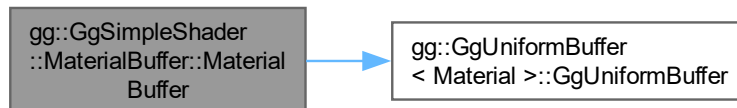
gg::GgSimpleShader::MaterialBuffer::MaterialBuffer (
    const Material & material,
    GLsizei count = 1,
    GLenum usage = GL_STATIC_DRAW) [inline]
  
```

同じデータで埋めるコンストラクタ.

引数

<i>material</i>	GgSimpleShader::Material 型の材質データ.
<i>count</i>	バッファ中の GgSimpleShader::Material 型の材質データの数.
<i>usage</i>	バッファの使い方のパターン, glBufferData() の第 4 引数の usage に渡される.

gg.h の 7978 行目に定義があります。
呼び出し関係図:



9.25.2.3 MaterialBuffer() [3/3]

```

gg::GgSimpleShader::MaterialBuffer::MaterialBuffer (
    GgVector ambient,
    GgVector diffuse,
    GgVector specular,
    GLfloat shininess,
  
```

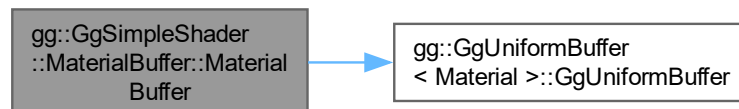
```
GLsizei count = 1,
GLenum usage = GL_STATIC_DRAW) [inline]
```

同じ引数で埋めるコンストラクタ.

引数

<i>ambient</i>	環境光に対する反射係数.
<i>diffuse</i>	拡散反射係数.
<i>specular</i>	鏡面反射係数.
<i>shininess</i>	輝き係数.
<i>count</i>	バッファ中の GgSimpleShader::Material 型の材質データの数.
<i>usage</i>	バッファの使い方のパターン, <code>glBufferData()</code> の第 4 引数の <code>usage</code> に渡される.

[gg.h](#) の 7997 行目に定義があります。
呼び出し関係図:



9.25.2.4 ~MaterialBuffer()

```
virtual gg::GgSimpleShader::MaterialBuffer::~MaterialBuffer () [inline], [virtual]
```

デストラクタ.

[gg.h](#) の 8012 行目に定義があります。

9.25.3 関数詳解

9.25.3.1 load() [1/2]

```
void gg::GgSimpleShader::MaterialBuffer::load (
    const Material & material,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

材質を設定する.

引数

<i>material</i>	材質の特性の GgSimpleShader::Material 構造体.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

[gg.h](#) の 8197 行目に定義があります。

呼び出し関係図:



9.25.3.2 load() [2/2]

```
void gg::GgSimpleShader::MaterialBuffer::load (
    const Material * material,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

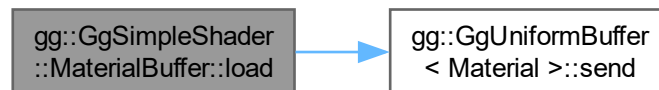
材質を設定する.

引数

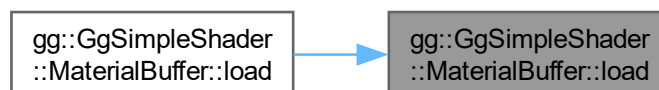
<i>material</i>	材質の特性の GgSimpleShader::Material 構造体のポインタ.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

[gg.h](#) の 8185 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.25.3.3 loadAmbient() [1/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadAmbient (
```

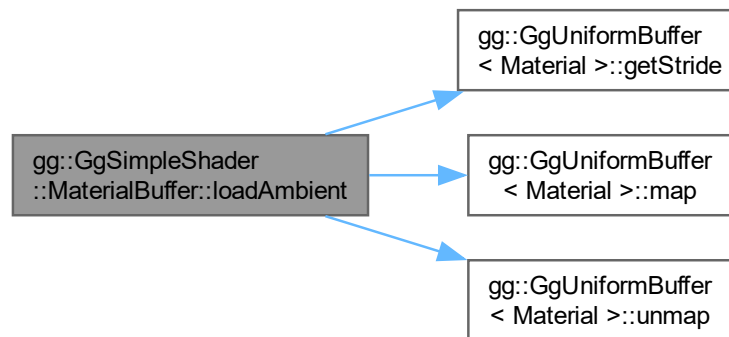
```
const GgVector & ambient,
GLint first = 0,
GLsizei count = 1) const
```

三角形に単純な陰影付けを行うシェーダが参照する材質データ：環境光に対する反射係数を設定する。

引数

<i>ambient</i>	環境光に対する反射係数を格納した GgVector 型の変数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

gg.cpp の 5737 行目に定義があります。
呼び出し関係図:



9.25.3.4 loadAmbient() [2/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadAmbient (
    const GLfloat * ambient,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

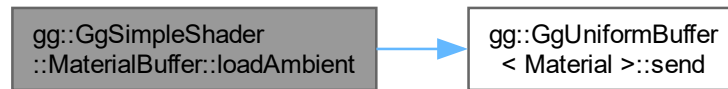
環境光に対する反射係数を設定する。

引数

<i>ambient</i>	環境光に対する反射係数を格納した GLfloat 型の 4 要素の配列変数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

gg.h の 8047 行目に定義があります。

呼び出し関係図:



9.25.3.5 loadAmbient() [3/3]

```

void gg::GgSimpleShader::MaterialBuffer::loadAmbient (
    GLfloat r,
    GLfloat g,
    GLfloat b,
    GLfloat a = 1.0f,
    GLint first = 0,
    GLsizei count = 1) const
  
```

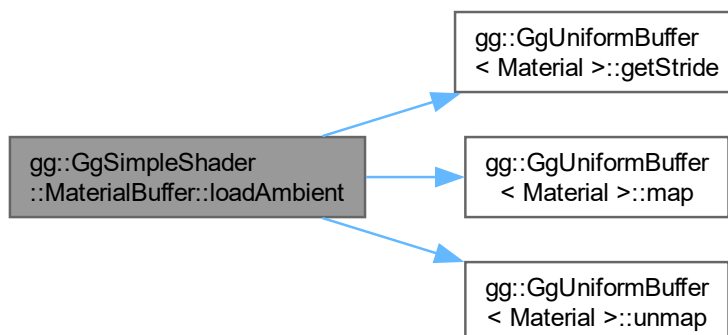
環境光に対する反射係数を設定する。

引数

<i>r</i>	環境光に対する反射係数の赤成分.
<i>g</i>	環境光に対する反射係数の緑成分.
<i>b</i>	環境光に対する反射係数の青成分.
<i>a</i>	環境光に対する反射係数の不透明度, デフォルトは 1.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

gg.cpp の 5711 行目に定義があります。

呼び出し関係図:



9.25.3.6 loadAmbientAndDiffuse() [1/3]

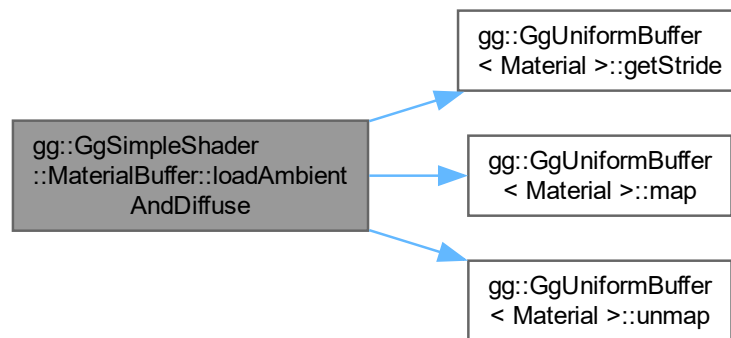
```
void gg::GgSimpleShader::MaterialBuffer::loadAmbientAndDiffuse (
    const GgVector & color,
    GLint first = 0,
    GLsizei count = 1) const
```

三角形に単純な陰影付けを行うシェーダが参照する材質データ：環境光に対する反射係数と拡散反射係数を設定する。

引数

<i>color</i>	環境光に対する反射係数と拡散反射係数を格納した GgVector 型の変数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

[gg.cpp](#) の 5841 行目に定義があります。
呼び出し関係図:



9.25.3.7 loadAmbientAndDiffuse() [2/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadAmbientAndDiffuse (
    const GLfloat * color,
    GLint first = 0,
    GLsizei count = 1) const
```

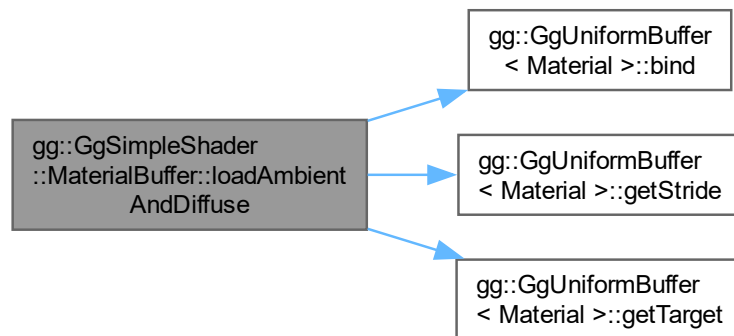
環境光に対する反射係数と拡散反射係数を設定する。

引数

<i>color</i>	環境光に対する反射係数と拡散反射係数を格納した GLfloat 型の 4 要素の配列変数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

[gg.cpp](#) の 5864 行目に定義があります。

呼び出し関係図:



9.25.3.8 loadAmbientAndDiffuse() [3/3]

```

void gg::GgSimpleShader::MaterialBuffer::loadAmbientAndDiffuse (
    GLfloat r,
    GLfloat g,
    GLfloat b,
    GLfloat a = 1.0f,
    GLint first = 0,
    GLsizei count = 1) const
  
```

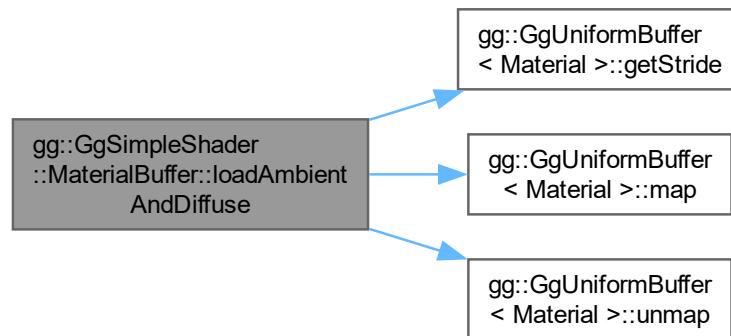
環境光に対する反射係数と拡散反射係数を設定する。

引数

<i>r</i>	環境光に対する反射係数と拡散反射係数の赤成分.
<i>g</i>	環境光に対する反射係数と拡散反射係数の緑成分.
<i>b</i>	環境光に対する反射係数と拡散反射係数の青成分.
<i>a</i>	環境光に対する反射係数と拡散反射係数の不透明度, デフォルトは 1.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

gg.cpp の 5815 行目に定義があります。

呼び出し関係図:



9.25.3.9 loadDiffuse() [1/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadDiffuse (
    const GgVector & diffuse,
    GLint first = 0,
    GLsizei count = 1) const
```

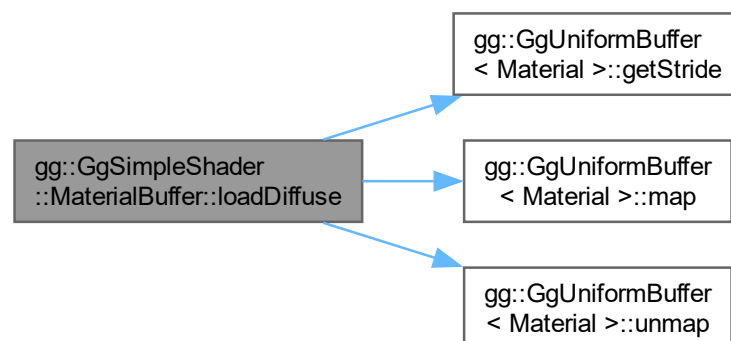
三角形に単純な陰影付けを行うシェーダが参照する材質データ：拡散反射係数を設定する。

引数

<i>diffuse</i>	拡散反射係数を格納した GgVector 型の変数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

[gg.cpp](#) の 5789 行目に定義があります。

呼び出し関係図:



9.25.3.10 loadDiffuse() [2/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadDiffuse (
    const GLfloat * diffuse,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

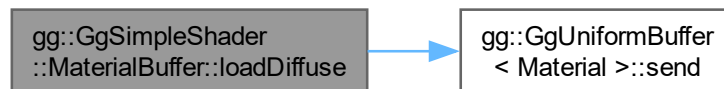
拡散反射係数を設定する。

引数

<i>diffuse</i>	拡散反射係数を格納した GLfloat 型の 4 要素の配列変数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

gg.h の 8084 行目に定義があります。

呼び出し関係図:



9.25.3.11 loadDiffuse() [3/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadDiffuse (
    GLfloat r,
    GLfloat g,
    GLfloat b,
    GLfloat a = 1.0f,
    GLint first = 0,
    GLsizei count = 1) const
```

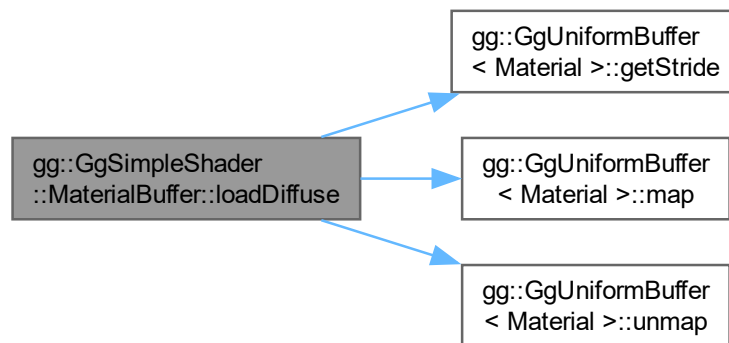
拡散反射係数を設定する。

引数

<i>r</i>	拡散反射係数の赤成分.
<i>g</i>	拡散反射係数の緑成分.
<i>b</i>	拡散反射係数の青成分.
<i>a</i>	拡散反射係数の不透明度, デフォルトは 1.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

gg.cpp の 5763 行目に定義があります。

呼び出し関係図:



9.25.3.12 loadShininess() [1/2]

```

void gg::GgSimpleShader::MaterialBuffer::loadShininess (
    const GLfloat * shininess,
    GLint first = 0,
    GLsizei count = 1) const
  
```

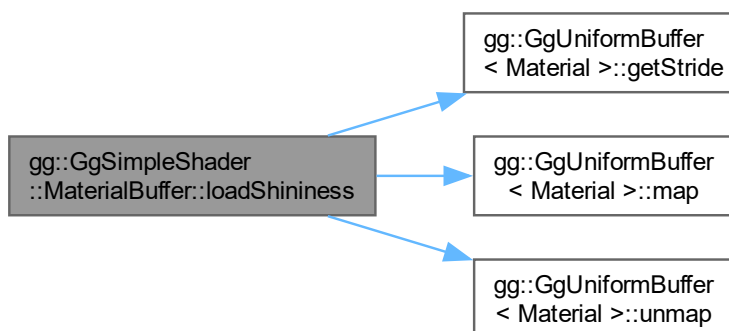
輝き係数を設定する.

引数

<i>shininess</i>	輝き 係数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

[gg.cpp](#) の 5977 行目に定義があります。

呼び出し関係図:



9.25.3.13 loadShininess() [2/2]

```
void gg::GgSimpleShader::MaterialBuffer::loadShininess (
    GLfloat shininess,
    GLint first = 0,
    GLsizei count = 1) const
```

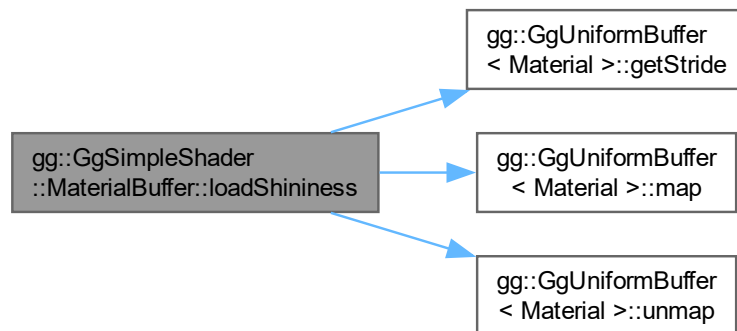
輝き係数を設定する。

引数

<i>shininess</i>	輝き係数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

gg.cpp の 5956 行目に定義があります。

呼び出し関係図:



9.25.3.14 loadSpecular() [1/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadSpecular (
    const GgVector & specular,
    GLint first = 0,
    GLsizei count = 1) const
```

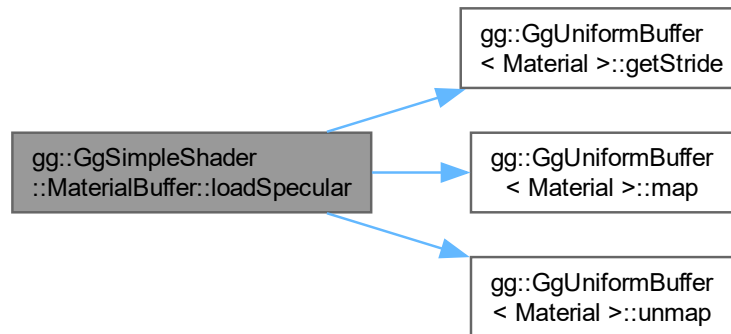
三角形に単純な陰影付けを行うシェーダが参照する材質データ：鏡面反射係数を設定する。

引数

<i>specular</i>	鏡面反射係数を格納した <code>GgVector</code> 型の変数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

gg.cpp の 5933 行目に定義があります。

呼び出し関係図:



9.25.3.15 loadSpecular() [2/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadSpecular (
    const GLfloat * specular,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

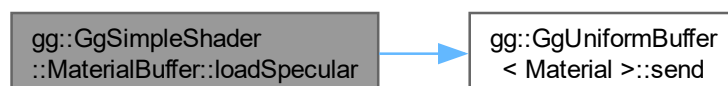
鏡面反射係数を設定する.

引数

<i>specular</i>	鏡面反射係数を格納した GLfloat 型の 4 要素の配列変数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

[gg.h](#) の 8154 行目に定義があります。

呼び出し関係図:



9.25.3.16 loadSpecular() [3/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadSpecular (
    GLfloat r,
    GLfloat g,
    GLfloat b,
    GLfloat a = 1.0f,
```

```
GLint first = 0,
GLsizei count = 1) const
```

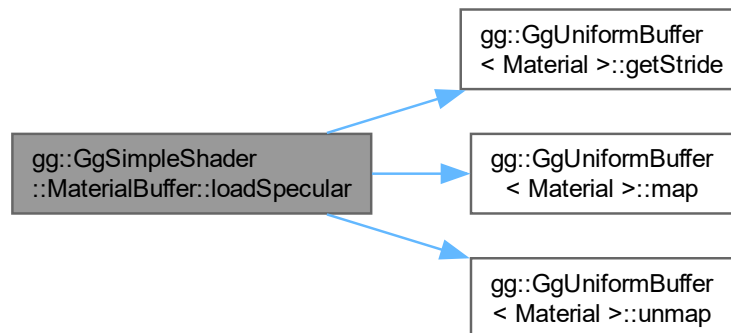
鏡面反射係数を設定する。

引数

<i>r</i>	鏡面反射係数の赤成分.
<i>g</i>	鏡面反射係数の緑成分.
<i>b</i>	鏡面反射係数の青成分.
<i>a</i>	鏡面反射係数の不透明度, デフォルトは 1.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

gg.cpp の 5907 行目に定義があります。

呼び出し関係図:



9.25.3.17 select()

```
void gg::GgSimpleShader::MaterialBuffer::select (
    GLint i = 0) const [inline]
```

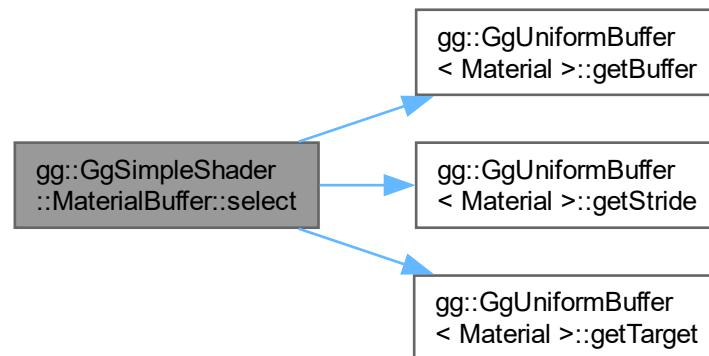
材質を選択する。

引数

<i>i</i>	材質データの uniform block のインデックス.
----------	--------------------------------------

gg.h の 8207 行目に定義があります。

呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

9.26 Menu クラス

```
#include <Menu.h>
```

公開メンバ関数

- [Menu](#) (const [Config](#) &config)
- [Menu](#) (const [Menu](#) &menu)=delete
- [Menu](#) ([Menu](#) &&menu)=default
- virtual [~Menu](#) ()
- [Menu](#) & [operator=](#) (const [Menu](#) &menu)=delete
- const auto & [getLight](#) () const
- const auto & [getShader](#) () const
- const auto & [getModel](#) () const
- void [draw](#) ()

フレンド

- class [Scene](#)

9.26.1 詳解

メニューの描画
[Menu.h](#) の 23 行目に定義があります。

9.26.2 構築子と解体子

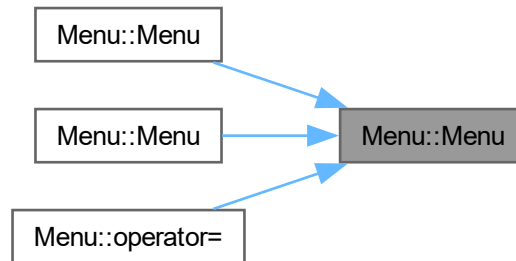
9.26.2.1 Menu() [1/3]

```
Menu::Menu (
    const Config & config)
```

コンストラクタ

`Menu.cpp` の 16 行目に定義があります。

被呼び出し関係図:



9.26.2.2 Menu() [2/3]

```
Menu::Menu (
    const Menu & menu) [delete]
```

コピーコンストラクタは使用しない 呼び出し関係図:



9.26.2.3 Menu() [3/3]

```
Menu::Menu (
    Menu && menu) [default]
```

ムーブコンストラクタはデフォルトのものを使用する 呼び出し関係図:



9.26.2.4 ~Menu()

```
Menu::~Menu () [virtual]
```

デストラクタ.

[Menu.cpp](#) の 52 行目に定義があります。

9.26.3 関数詳解

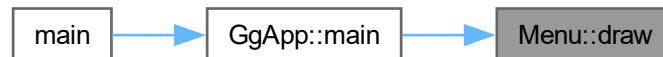
9.26.3.1 draw()

```
void Menu::draw ()
```

描画する

[Menu.cpp](#) の 82 行目に定義があります。

被呼び出し関係図:



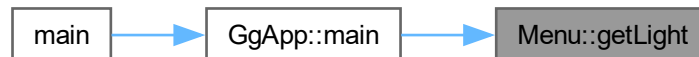
9.26.3.2 getLight()

```
const auto & Menu::getLight () const [inline]
```

光源データを取り出す

[Menu.h](#) の 76 行目に定義があります。

被呼び出し関係図:



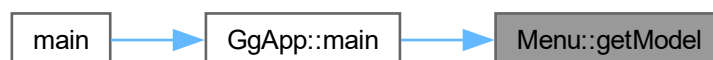
9.26.3.3 getModel()

```
const auto & Menu::getModel () const [inline]
```

モデルデータを取り出す

[Menu.h](#) の 92 行目に定義があります。

被呼び出し関係図:



9.26.3.4 getShader()

```
const auto & Menu::getShader () const [inline]
```

シェードを取り出す

[Menu.h](#) の 84 行目に定義があります。

被呼び出し関係図:



9.26.3.5 operator=()

```
Menu & Menu::operator= (
    const Menu & menu) [delete]
```

代入演算子は使用しない 呼び出し関係図:



9.26.4 フレンドと関連関数の詳解

9.26.4.1 Scene

```
friend class Scene [friend]
```

[Menu.h](#) の 26 行目に定義があります。

このクラス詳解は次のファイルから抽出されました:

- [Menu.h](#)
- [Menu.cpp](#)

9.27 GgApp::Window クラス

```
#include <GgApp.h>
```

公開メンバ関数

- [Window](#) (const std::string &title="GLFW Window", int width=640, int height=480, int fullscreen=0, GLFWwindow *share=nullptr)
- [Window](#) (const Window &w)=delete
- [Window](#) (Window &&w) noexcept
- virtual ~[Window](#) ()
- [Window](#) & operator= (const [Window](#) &w)=delete

- `Window & operator= (Window &&w) noexcept`
- `auto * get () const`
- `void setClose (int flag=GLFW_TRUE) const`
- `bool shouldClose () const`
- `operator bool ()`
- `void swapBuffers () const`
- `void restoreViewport () const`
- `void updateViewport ()`
- `auto getWidth () const`
- `auto getHeight () const`
- `auto getFboWidth () const`
- `auto getFboHeight () const`
- `const auto & getSize () const`
- `void getSize (GLsizei *size) const`
- `const auto & getFboSize () const`
- `void getFboSize (GLsizei *fboSize) const`
- `auto getAspect () const`
- `bool getKey (int key) const`
- `void selectInterface (int no)`
- `void setVelocity (GLfloat vx, GLfloat vy, GLfloat vz=0.1f)`
- `int getLastKey ()`
- `auto getArrow (int direction=0, int mods=0) const`
- `auto getArrowX (int mods=0) const`
- `auto getArrowY (int mods=0) const`
- `void getArrow (GLfloat *arrow, int mods=0) const`
- `auto getShiftArrowX () const`
- `auto getShiftArrowY () const`
- `void getShiftArrow (GLfloat *shift_arrow) const`
- `auto getControlArrowX () const`
- `auto getControlArrowY () const`
- `void getControlArrow (GLfloat *control_arrow) const`
- `auto getAltArrowX () const`
- `auto getAltArrowY () const`
- `void getAltArrow (GLfloat *alt_arrow) const`
- `const auto * getMouse () const`
- `void getMouse (GLfloat *position) const`
- `auto getMouse (int direction) const`
- `auto getMouseX () const`
- `auto getMouseY () const`
- `const auto * getWheel () const`
- `void getWheel (GLfloat *rotation) const`
- `auto getWheel (int direction) const`
- `auto getWheelX () const`
- `auto getWheelY () const`
- `const auto & getTranslation (int button=GLFW_MOUSE_BUTTON_1) const`
- `auto getTranslationMatrix (int button=GLFW_MOUSE_BUTTON_1) const`
- `auto getScrollMatrix (int button=GLFW_MOUSE_BUTTON_1) const`
- `auto getRotation (int button=GLFW_MOUSE_BUTTON_1) const`
- `auto getRotationMatrix (int button=GLFW_MOUSE_BUTTON_1) const`
- `void resetRotation ()`
- `void resetTranslation ()`
- `void reset ()`
- `void * getUserPointer () const`
- `void setUserPointer (void *pointer)`
- `void setResizeFunc (void(*func)(const Window *window, int width, int height))`
- `void setKeyboardFunc (void(*func)(const Window *window, int key, int scancode, int action, int mods))`
- `void setMouseFunc (void(*func)(const Window *window, int button, int action, int mods))`
- `void setWheelFunc (void(*func)(const Window *window, double x, double y))`

9.27.1 詳解

ウィンドウ関連の処理.

覚え書き

GLFW を使って OpenGL のウィンドウを操作するラッパークラス.

[GgApp.h](#) の 155 行目に定義があります。

9.27.2 構築子と解体子

9.27.2.1 Window() [1/3]

```
GgApp::Window::Window (  
    const std::string & title = "GLFW Window",  
    int width = 640,  
    int height = 480,  
    int fullscreen = 0,  
    GLFWwindow * share = nullptr)
```

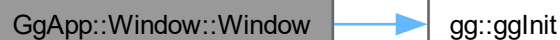
コンストラクタ.

引数

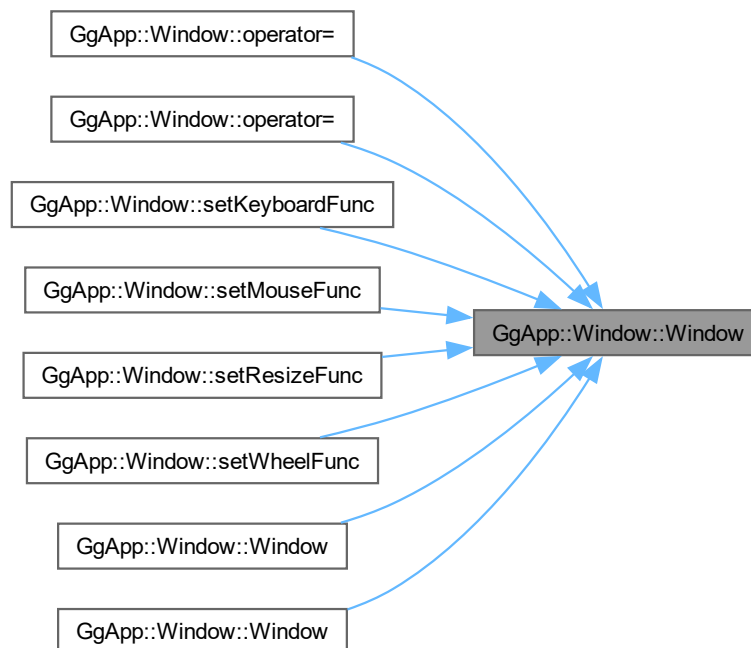
<i>title</i>	ウィンドウタイトルの文字列.
<i>width</i>	開くウィンドウの幅, フルスクリーン時は無視され実際のディスプレイの幅が使われる.
<i>height</i>	開くウィンドウの高さ, フルスクリーン時は無視され実際のディスプレイの高さが使われる.
<i>fullscreen</i>	フルスクリーン表示を行うディスプレイ番号, 0 ならフルスクリーン表示を行わない.
<i>share</i>	共有するコンテキスト, nullptr ならコンテキストを共有しない.

[GgApp.cpp](#) の 344 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.27.2.2 Window() [2/3]

```
GgApp::Window::Window (
    const Window & w) [delete]
```

コピーコンストラクタは使用しない.

引数

<i>w</i>	コピー元のウィンドウ.
----------	-------------

呼び出し関係図:



9.27.2.3 Window() [3/3]

```
GgApp::Window::Window (
    Window && w) [noexcept]
```

ムーブコンストラクタ.

引数

<i>w</i>	ムーブ代入元のウィンドウ.
----------	---------------

GgApp.cpp の 424 行目に定義があります。
呼び出し関係図:



9.27.2.4 ~Window()

```
virtual GgApp::Window::~~Window () [inline], [virtual]
```

デストラクタ.

GgApp.h の 284 行目に定義があります。

9.27.3 関数詳解

9.27.3.1 get()

```
auto * GgApp::Window::get () const [inline]
```

ウィンドウの識別子のポインタを取得する.

戻り値

GLFWwindow 型のウィンドウ識別子のポインタ.

GgApp.h の 314 行目に定義があります。

9.27.3.2 getAltArrow()

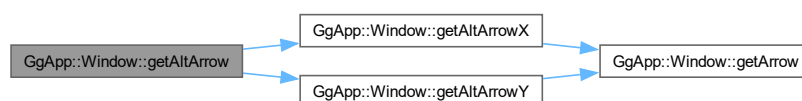
```
void GgApp::Window::getAltArrow (
    GLfloat * alt.arrow) const [inline]
```

ALT キーを押しながら矢印キーを押したときの現在の値を得る.

引数

<i>alt.arrow</i>	ALT キーを押しながら矢印キーを押したときの値を格納する GLfloat 型の 2 要素の配列.
------------------	---

GgApp.h の 658 行目に定義があります。
呼び出し関係図:



9.27.3.3 getAltArrowX()

```
auto GgApp::Window::getAltArrowX () const [inline]
```

ALT キーを押しながら矢印キーを押したときの現在の X 値を得る.

戻り値

ALT キーを押しながら矢印キーを押したときの現在の X 値.

[GgApp.h](#) の 638 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.27.3.4 getAltArrowY()

```
auto GgApp::Window::getAltArrowY () const [inline]
```

ALT キーを押しながら矢印キーを押したときの現在の Y 値を得る.

戻り値

ALT キーを押しながら矢印キーを押したときの現在の Y 値.

[GgApp.h](#) の 648 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.27.3.5 getArrow() [1/2]

```
void GgApp::Window::getArrow (
    GLfloat * arrow,
    int mods = 0) const [inline]
```

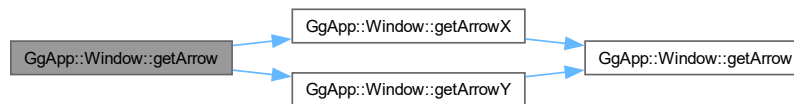
矢印キーの現在の値を得る.

引数

<i>arrow</i>	矢印キーの値を格納する GLfloat[2] の配列.
<i>mods</i>	修飾キーの状態 (0:なし, 1: SHIFT, 2: CTRL, 3: ALT).

[GgApp.h](#) の 565 行目に定義があります。

呼び出し関係図:



9.27.3.6 getArrow() [2/2]

```
auto GgApp::Window::getArrow (
    int direction = 0,
    int mods = 0) const [inline]
```

矢印キーの現在の値を得る.

引数

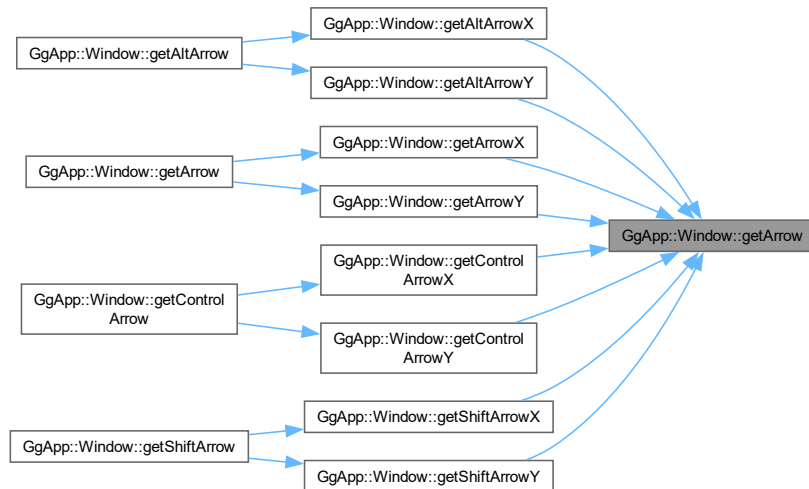
<i>direction</i>	方向 (0: X, 1:Y).
<i>mods</i>	修飾キーの状態 (0:なし, 1: SHIFT, 2: CTRL, 3: ALT).

戻り値

矢印キーの値.

[GgApp.h](#) の 531 行目に定義があります。

被呼び出し関係図:



9.27.3.7 getArrowX()

```
auto GgApp::Window::getArrowX (
    int mods = 0) const [inline]
```

矢印キーの現在の X 値を得る.

引数

<i>mods</i>	修飾キーの状態 (0:なし, 1: SHIFT, 2: CTRL, 3: ALT).
-------------	--

戻り値

矢印キーの X 値.

[GgApp.h](#) の 543 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.27.3.8 getArrowY()

```
auto GgApp::Window::getArrowY (
    int mods = 0) const [inline]
```

矢印キーの現在の Y 値を得る.

引数

<i>mods</i>	修飾キーの状態 (0:なし, 1, SHIFT, 2: CTRL, 3: ALT).
-------------	--

戻り値

矢印キーの Y 値.

[GgApp.h](#) の 554 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.27.3.9 getAspect()

```
auto GgApp::Window::getAspect () const [inline]
```

ウィンドウの縦横比を得る.

戻り値

ウィンドウの縦横比.

[GgApp.h](#) の 468 行目に定義があります。

被呼び出し関係図:



9.27.3.10 getControlArrow()

```
void GgApp::Window::getControlArrow (
    GLfloat * control_arrow) const [inline]
```

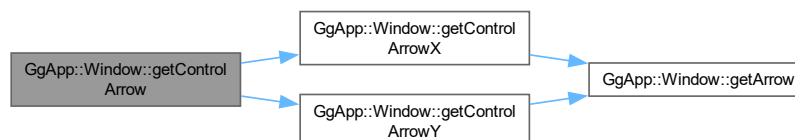
CTRL キーを押しながら矢印キーを押したときの現在の値を得る.

引数

<code>control_arrow</code>	CTRL キーを押しながら矢印キーを押したときの値を格納する GLfloat 型の 2 要素の配列.
----------------------------	--

[GgApp.h](#) の 627 行目に定義があります。

呼び出し関係図:



9.27.3.11 getControlArrowX()

```
auto GgApp::Window::getControlArrowX () const [inline]
```

CTRL キーを押しながら矢印キーを押したときの現在の X 値を得る.

戻り値

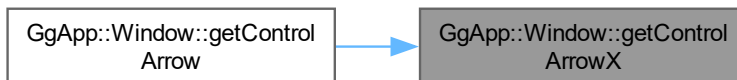
CTRL キーを押しながら矢印キーを押したときの現在の X 値.

GgApp.h の 607 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.27.3.12 getControlArrowY()

```
auto GgApp::Window::getControlArrowY () const [inline]
```

CTRL キーを押しながら矢印キーを押したときの現在の Y 値を得る.

戻り値

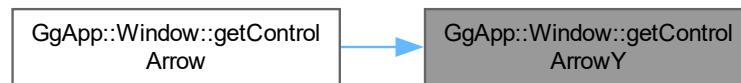
CTRL キーを押しながら矢印キーを押したときの現在の Y 値.

GgApp.h の 617 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.27.3.13 getFboHeight()

```
auto GgApp::Window::getFboHeight () const [inline]
```

FBO の高さを得る.

戻り値

FBO の高さ.

[GgApp.h](#) の 416 行目に定義があります。

被呼び出し関係図:



9.27.3.14 getFboSize() [1/2]

```
const auto & GgApp::Window::getFboSize () const [inline]
```

FBO のサイズを得る.

戻り値

FBO の幅と高さを格納した GLsizei 型の 2 要素の配列.

[GgApp.h](#) の 447 行目に定義があります。

9.27.3.15 getFboSize() [2/2]

```
void GgApp::Window::getFboSize (
    GLsizei * fboSize) const [inline]
```

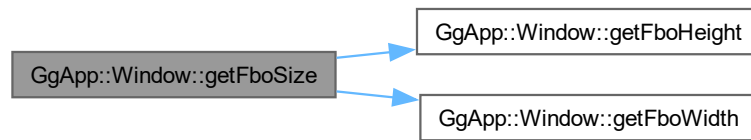
FBO のサイズを得る.

引数

<i>fboSize</i>	FBO の幅と高さを格納する GLsizei 型の 2 要素の配列.
----------------	------------------------------------

[GgApp.h](#) の 457 行目に定義があります。

呼び出し関係図:



9.27.3.16 getFboWidth()

```
auto GgApp::Window::getFboWidth () const [inline]
```

FBO の横幅を得る.

戻り値

FBO の横幅.

[GgApp.h](#) の 406 行目に定義があります。

被呼び出し関係図:



9.27.3.17 getHeight()

```
auto GgApp::Window::getHeight () const [inline]
```

ウィンドウの高さを得る.

戻り値

ウィンドウの高さ.

[GgApp.h](#) の 396 行目に定義があります。

被呼び出し関係図:



9.27.3.18 getKey()

```
bool GgApp::Window::getKey (
    int key) const [inline]
```

キーが押されているかどうかを判定する.

戻り値

キーが押されていれば true.

[GgApp.h](#) の 478 行目に定義があります。

9.27.3.19 getLastKey()

```
int GgApp::Window::getLastKey () [inline]
```

最後にタイプしたキーを得る.

戻り値

最後にタイプしたキーの文字.

[GgApp.h](#) の 516 行目に定義があります。

9.27.3.20 getMouse() [1/3]

```
const auto * GgApp::Window::getMouse () const [inline]
```

マウスカーソルの現在位置を得る.

戻り値

マウスカーソルの現在位置を格納した GLfloat 型の 2 要素の配列.

[GgApp.h](#) の 669 行目に定義があります。

9.27.3.21 getMouse() [2/3]

```
void GgApp::Window::getMouse (
    GLfloat * position) const [inline]
```

マウスカーソルの現在位置を得る.

引数

<i>position</i>	マウスカーソルの現在位置を格納した GLfloat 型の 2 要素の配列.
-----------------	---------------------------------------

[GgApp.h](#) の 680 行目に定義があります。

9.27.3.22 getMouse() [3/3]

```
auto GgApp::Window::getMouse (
    int direction) const [inline]
```

マウスカーソルの現在位置を得る.

引数

<i>direction</i>	方向 (0:X, 1:Y).
------------------	----------------

戻り値

direction 方向のマウスカーソルの現在位置.

[GgApp.h](#) の 693 行目に定義があります。

9.27.3.23 getMouseX()

```
auto GgApp::Window::getMouseX () const [inline]
```

マウスカーソルの現在位置の X 座標を得る.

戻り値

`direction` 方向のマウスカーソルの X 方向の現在位置.

[GgApp.h](#) の 704 行目に定義があります。

9.27.3.24 getMouseY()

```
auto GgApp::Window::getMouseY () const [inline]
```

マウスカーソルの現在位置の Y 座標を得る.

戻り値

`direction` 方向のマウスカーソルの Y 方向の現在位置.

[GgApp.h](#) の 715 行目に定義があります。

9.27.3.25 getRotation()

```
auto GgApp::Window::getRotation (
    int button = GLFW_MOUSE_BUTTON_1) const [inline]
```

トラックボールの回転変換行列を得る.

引数

<i>button</i>	回転変換行列を取得するマウスボタン (GLFW_MOUSE_BUTTON_[1,2]).
---------------	--

戻り値

回転を行う GgQuaternion 型の四元数.

[GgApp.h](#) の 843 行目に定義があります。

9.27.3.26 getRotationMatrix()

```
auto GgApp::Window::getRotationMatrix (
    int button = GLFW_MOUSE_BUTTON_1) const [inline]
```

トラックボールの回転変換行列を得る.

引数

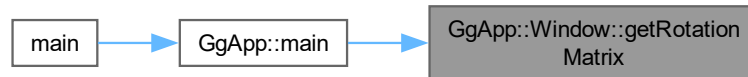
<i>button</i>	回転変換行列を取得するマウスボタン (GLFW_MOUSE_BUTTON_[1,2]).
---------------	--

戻り値

回転を行う GgMatrix 型の変換行列。

GgApp.h の 856 行目に定義があります。

被呼び出し関係図:



9.27.3.27 getScrollMatrix()

```
auto GgApp::Window::getScrollMatrix (
    int button = GLFW_MOUSE_BUTTON_1) const [inline]
```

マウスによってオブジェクトの平行移動の変換行列を得る。

引数

<i>button</i>	平行移動量を取得するマウスボタン (GLFW_MOUSE_BUTTON_[1,2]).
---------------	---

戻り値

クリッピング座標系で平行移動を行う GgMatrix 型の変換行列。

GgApp.h の 820 行目に定義があります。

9.27.3.28 getShiftArrow()

```
void GgApp::Window::getShiftArrow (
    GLfloat * shift_arrow) const [inline]
```

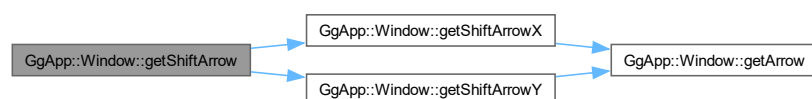
SHIFT キーを押しながら矢印キーを押したときの現在の値を得る。

引数

<i>shift_arrow</i>	SHIFT キーを押しながら矢印キーを押したときの値を格納する GLfloat 型の 2 要素の配列。
--------------------	---

GgApp.h の 596 行目に定義があります。

呼び出し関係図:



9.27.3.29 getShiftArrowX()

```
auto GgApp::Window::getShiftArrowX () const [inline]
```

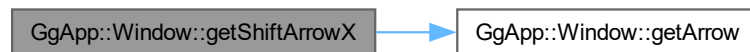
SHIFT キーを押しながら矢印キーを押したときの現在の X 値を得る.

戻り値

SHIFT キーを押しながら矢印キーを押したときの現在の X 値.

[GgApp.h](#) の 576 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.27.3.30 getShiftArrowY()

```
auto GgApp::Window::getShiftArrowY () const [inline]
```

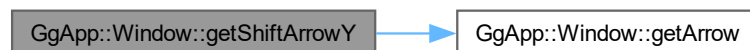
SHIFT キーを押しながら矢印キーを押したときの現在の Y 値を得る.

戻り値

SHIFT キーを押しながら矢印キーを押したときの現在の Y 値.

[GgApp.h](#) の 586 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



9.27.3.31 getSize() [1/2]

```
const auto & GgApp::Window::getSize () const [inline]
```

ウィンドウのサイズを得る.

戻り値

ウィンドウの幅と高さを格納した GLsizei 型の 2 要素の配列の参照.

[GgApp.h](#) の 426 行目に定義があります。

9.27.3.32 getSize() [2/2]

```
void GgApp::Window::getSize (
    GLsizei * size) const [inline]
```

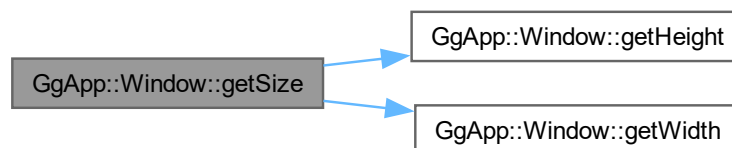
ウィンドウのサイズを得る.

引数

size	ウィンドウの幅と高さを格納した GLsizei 型の 2 要素の配列.
------	-------------------------------------

[GgApp.h](#) の 436 行目に定義があります。

呼び出し関係図:



9.27.3.33 getTranslation()

```
const auto & GgApp::Window::getTranslation (
    int button = GLFW_MOUSE_BUTTON_1) const [inline]
```

トラックボール処理を考慮したマウスによるスクロールの変換行列を得る.

引数

<i>button</i>	平行移動量を取得するマウスボタン (GLFW_MOUSE_BUTTON_[1,2]).
---------------	---

戻り値

平行移動量を格納した GLfloat[3] の配列のポインタ.

GgApp.h の 784 行目に定義があります。

9.27.3.34 getTranslationMatrix()

```
auto GgApp::Window::getTranslationMatrix (
    int button = GLFW_MOUSE_BUTTON_1) const [inline]
```

マウスによって視点の平行移動の変換行列を得る.

引数

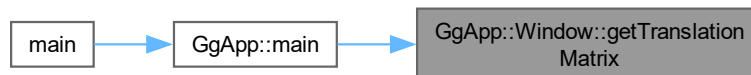
<i>button</i>	平行移動量を取得するマウスボタン (GLFW_MOUSE_BUTTON_[1,2]).
---------------	---

戻り値

視点座標系で平行移動を行う GgMatrix 型の変換行列.

GgApp.h の 797 行目に定義があります。

被呼び出し関係図:



9.27.3.35 getUserPointer()

```
void * GgApp::Window::getUserPointer () const [inline]
```

ユーザーポインタを取り出す.

戻り値

保存されているユーザポインタ.

GgApp.h の 898 行目に定義があります。

9.27.3.36 getWheel() [1/3]

```
const auto * GgApp::Window::getWheel () const [inline]
```

マウスホイールの回転量を得る.

戻り値

マウスホイールの回転量を格納した GLfloat 型の 2 要素の配列.

GgApp.h の 726 行目に定義があります。

9.27.3.37 getWheel() [2/3]

```
void GgApp::Window::getWheel (
    GLfloat * rotation) const [inline]
```

マウスホイールの回転量を得る.

引数

<i>rotation</i>	マウスホイールの回転量を格納した GLfloat 型の 2 要素の配列.
-----------------	--------------------------------------

[GgApp.h](#) の 737 行目に定義があります。

9.27.3.38 getWheel() [3/3]

```
auto GgApp::Window::getWheel (
    int direction) const [inline]
```

マウスホイールの回転量を得る.

引数

<i>direction</i>	方向 (0:X, 1:Y).
------------------	----------------

戻り値

direction 方向のマウスホイールの回転量.

[GgApp.h](#) の 750 行目に定義があります。

9.27.3.39 getWheelX()

```
auto GgApp::Window::getWheelX () const [inline]
```

マウスホイールの X 方向の回転量を得る.

戻り値

マウスホイールの X 方向の回転量.

[GgApp.h](#) の 761 行目に定義があります。

9.27.3.40 getWheelY()

```
auto GgApp::Window::getWheelY () const [inline]
```

マウスホイールの Y 方向の回転量を得る.

戻り値

マウスホイールの Y 方向の回転量.

[GgApp.h](#) の 772 行目に定義があります。

9.27.3.41 getWidth()

```
auto GgApp::Window::getWidth () const [inline]
```

ウィンドウの横幅を得る.

戻り値

ウィンドウの横幅.

[GgApp.h](#) の 386 行目に定義があります。

被呼び出し関係図:



9.27.3.42 operator bool()

`GgApp::Window::operator bool () [explicit]`

イベントを取得してループを継続すべきかどうか調べる.

戻り値

ループを継続すべきなら true.

[GgApp.cpp](#) の 477 行目に定義があります。

呼び出し関係図:



9.27.3.43 operator=() [1/2]

`Window & GgApp::Window::operator= (const Window & w) [delete]`

代入演算子は使用しない.

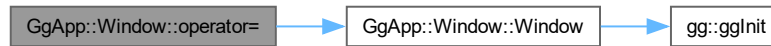
引数

<i>w</i>	代入元のウィンドウ.
----------	------------

戻り値

代入後のこのオブジェクトの参照.

呼び出し関係図:



9.27.3.44 operator=() [2/2]

```
GgApp::Window & GgApp::Window::operator= (
    Window && w) [noexcept]
```

ムーブ代入演算子.

引数

<i>w</i>	ムーブ代入元のウィンドウ.
----------	---------------

戻り値

ムーブ代入後のこのオブジェクトの参照.

[GgApp.cpp](#) の 446 行目に定義があります。

呼び出し関係図:



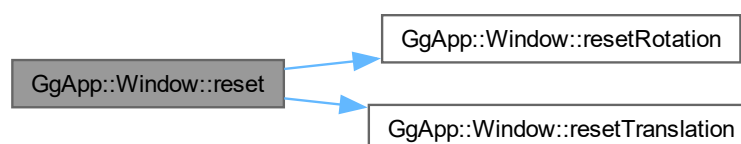
9.27.3.45 reset()

```
void GgApp::Window::reset () [inline]
```

トラックボール・マウスホイール・矢印キーの値を初期化する.

[GgApp.h](#) の 884 行目に定義があります。

呼び出し関係図:



9.27.3.46 resetRotation()

```
void GgApp::Window::resetRotation () [inline]
```

トラックボール処理を初期化する.

[GgApp.h](#) の 866 行目に定義があります.

被呼び出し関係図:



9.27.3.47 resetTranslation()

```
void GgApp::Window::resetTranslation () [inline]
```

平行移動量を初期化する.

[GgApp.h](#) の 875 行目に定義があります.

被呼び出し関係図:



9.27.3.48 restoreViewport()

```
void GgApp::Window::restoreViewport () const [inline]
```

ビューポートを元のサイズに復帰する.

[GgApp.h](#) の 355 行目に定義があります.

被呼び出し関係図:



9.27.3.49 selectInterface()

```
void GgApp::Window::selectInterface (  
    int no) [inline]
```

インタフェースを選択する.

引数

<i>no</i>	インターフェース番号.
-----------	-------------

[GgApp.h](#) の 493 行目に定義があります。

9.27.3.50 setClose()

```
void GgApp::Window::setClose (
    int flag = GLFW_TRUE) const [inline]
```

ウィンドウのクローズフラグを設定する.

引数

<i>flag</i>	クローズフラグ, 0 (GLFW_FALSE) 以外ならウィンドウを閉じる.
-------------	--

[GgApp.h](#) の 324 行目に定義があります。

9.27.3.51 setKeyboardFunc()

```
void GgApp::Window::setKeyboardFunc (
    void(* func) (const Window *window, int key, int scancode, int action, int mods))
[inline]
```

ユーザ定義の keyboard 関数を設定する.

引数

<i>func</i>	ユーザ定義の keyboard 関数, キーボードの操作時に呼び出される.
-------------	---------------------------------------

[GgApp.h](#) の 928 行目に定義があります。
呼び出し関係図:



9.27.3.52 setMouseFunc()

```
void GgApp::Window::setMouseFunc (
    void(* func) (const Window *window, int button, int action, int mods)) [inline]
```

ユーザ定義の mouse 関数を設定する.

引数

<i>func</i>	ユーザ定義の mouse 関数, マウスボタンの操作時に呼び出される.
-------------	-------------------------------------

[GgApp.h](#) の 938 行目に定義があります。

呼び出し関係図:



9.27.3.53 setResizeFunc()

```
void GgApp::Window::setResizeFunc (
    void(* func )(const Window *window, int width, int height)) [inline]
ユーザ定義の resize 関数を設定する.
```

引数

<i>func</i>	ユーザ定義の <code>resize</code> 関数, ウィンドウのサイズ変更時に呼び出される.
-------------	---

[GgApp.h](#) の 918 行目に定義があります。
呼び出し関係図:



9.27.3.54 setUserPointer()

```
void GgApp::Window::setUserPointer (
    void * pointer) [inline]
任意のユーザポインタを保存する.
```

引数

<i>pointer</i>	保存するユーザポインタ.
----------------	--------------

[GgApp.h](#) の 908 行目に定義があります。

9.27.3.55 setVelocity()

```
void GgApp::Window::setVelocity (
    GLfloat vx,
    GLfloat vy,
    GLfloat vz = 0.1f) [inline]
マウスの移動速度を設定する.
```

引数

<i>vx</i>	x 方向の移動速度.
-----------	------------

<code>vy</code>	y 方向の移動速度.
<code>vz</code>	z 方向の移動速度.

[GgApp.h](#) の 506 行目に定義があります。

9.27.3.56 setWheelFunc()

```
void GgApp::Window::setWheelFunc (
    void(* func )(const Window *window, double x, double y)) [inline]
```

ユーザ定義の wheel 関数を設定する.

引数

<code>func</code>	ユーザ定義の wheel 関数, マウスホイールの操作時に呼び出される.
-------------------	--------------------------------------

[GgApp.h](#) の 948 行目に定義があります。
呼び出し関係図:



9.27.3.57 shouldClose()

```
bool GgApp::Window::shouldClose () const [inline]
```

ウィンドウを閉じるべきかどうか調べる.

戻り値

ウィンドウを閉じるべきなら true.

[GgApp.h](#) の 334 行目に定義があります。
被呼び出し関係図:



9.27.3.58 swapBuffers()

```
void GgApp::Window::swapBuffers () const
```

カラーバッファを入れ替える.
[GgApp.cpp](#) の 528 行目に定義があります。

被呼び出し関係図:



9.27.3.59 updateViewport()

`void GgApp::Window::updateViewport ()`
ビューポートのサイズを更新する。
[GgApp.cpp](#) の 547 行目に定義があります。
呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [GgApp.h](#)
- [GgApp.cpp](#)

Chapter 10

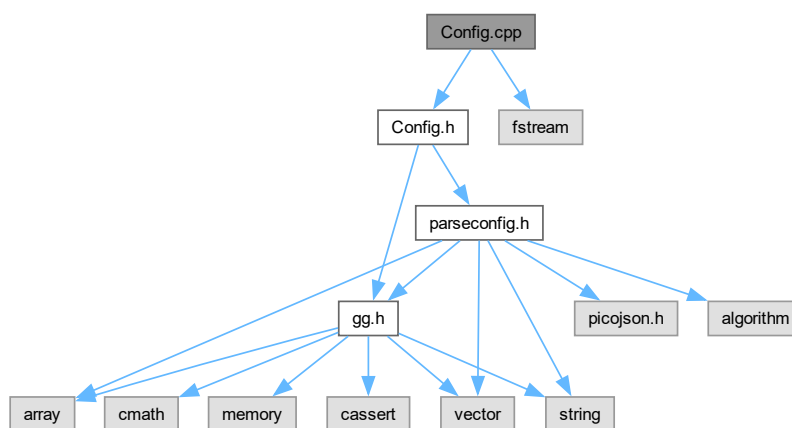
ファイル詳解

10.1 Config.cpp ファイル

```
#include "Config.h"
```

```
#include <fstream>
```

Config.cpp の依存先関係図:



変数

- constexpr [GgSimpleShader::Light defaultLight](#)

10.1.1 詳解

設定構造体の実装

著者

Kohe Tokoi

日付

November 15, 2022

[Config.cpp](#) に定義があります。

10.1.2 変数詳解

10.1.2.1 defaultLight

GgSimpleShader::Light defaultLight [constexpr]

初期値:

```
{
    { 0.2f, 0.2f, 0.2f, 1.0f },
    { 1.0f, 1.0f, 1.0f, 0.0f },
    { 1.0f, 1.0f, 1.0f, 0.0f },
    { 0.5f, 0.5f, 1.0f, 1.0f }
}
```

Config.cpp の 17 行目に定義があります。

10.2 Config.cpp

[詳解]

```
00001
00008 #include "Config.h"
00009
00010 // 補助プログラム
00011 using namespace gg;
00012
00013 // 標準ライブラリ
00014 #include <fstream>
00015
00016 // デフォルトの光源データ
00017 constexpr GgSimpleShader::Light defaultLight
00018 {
00019     { 0.2f, 0.2f, 0.2f, 1.0f }, // 環境光成分
00020     { 1.0f, 1.0f, 1.0f, 0.0f }, // 拡散反射光成分
00021     { 1.0f, 1.0f, 1.0f, 0.0f }, // 鏡面反射光成分
00022     { 0.5f, 0.5f, 1.0f, 1.0f } // 光源位置
00023 };
00024
00025 //
00026 // コンストラクタ
00027 //
00028 Config::Config() :
00029     winSize{ 960, 540 },
00030     menuFont{ "Mplus1-Regular.ttf" },
00031     menuFontSize{ 20.0f },
00032     light{ defaultLight },
00033     model{ "logo.obj" },
00034     #if defined(GL_GLES_PROTOTYPES)
00035     shader{ "simple-es3.vert", "simple-es3.frag" }
00036     #else
00037     shader{ "simple.vert", "simple.frag" }
00038     #endif
00039 {
00040 }
00041
00042 //
00043 // ファイルから構成データを読み込むコンストラクタ
00044 //
00045 Config::Config(const std::string& filename) :
00046     Config{}
00047 {
00048     // 構成ファイルが読み込めなかったらデフォルト値の構成ファイルを作る
00049     if (!load(filename)) save(filename);
00050 }
00051
00052 //
00053 // デストラクタ
00054 //
00055 Config::~Config()
00056 {
00057 }
00058
00059 //
00060 // 設定ファイルを読み込む
00061 //
00062 bool Config::load(const std::string& filename)
00063 {
00064     // 構成ファイルを開く
00065     std::ifstream file{ Utf8ToTChar(filename) };
00066
00067     // 開けなかったらエラー
00068     if (!file) return false;
00069
00070     // JSON の読み込み
```

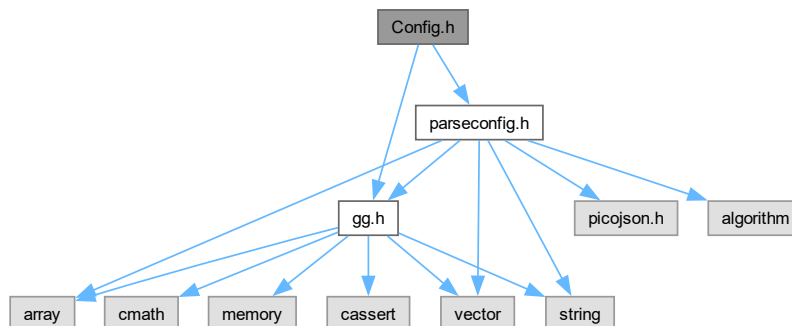
```

00071     picojson::value value;
00072     file >> value;
00073     file.close();
00074
00075     // JSON のパースに失敗しているかオブジェクトでなかったら戻る
00076     if (!value.is<picojson::object>()) return false;
00077
00078     // 構成データの取り出し
00079     const auto& object{ value.get<picojson::object>() };
00080
00081     //
00082     // 構成データの読み込み
00083     //
00084
00085     // ウィンドウサイズ
00086     getValue(object, "window_size", winSize);
00087
00088     // メニューフォント
00089     getString(object, "menu_font", menuFont);
00090
00091     // メニューフォントサイズ
00092     getValue(object, "menu_font_size", menuFontSize);
00093
00094     // 光源
00095     getVector(object, "ambient", light.ambient);
00096     getVector(object, "diffuse", light.diffuse);
00097     getVector(object, "specular", light.specular);
00098     getVector(object, "position", light.position);
00099
00100     // モデル
00101     getString(object, "model", model);
00102
00103     // シェーダ
00104     getString(object, "shader", shader);
00105
00106     // オブジェクトが空だったらエラー
00107     if (object.empty()) return false;
00108
00109     return true;
00110 }
00111
00112 //
00113 // 設定ファイルを書き出す
00114 //
00115 bool Config::save(const std::string& filename) const
00116 {
00117     // 構成ファイルを開く
00118     std::ofstream file{ Utf8ToTChar(filename) };
00119
00120     // 開けなかったらエラー
00121     if (!file) return false;
00122
00123     // 構成データの書き出しに使うオブジェクト
00124     picojson::object object;
00125
00126     //
00127     // 構成データの書き出し
00128     //
00129
00130     // ウィンドウサイズ
00131     setValue(object, "window_size", winSize);
00132
00133     // メニューフォント
00134     setString(object, "menu_font", menuFont);
00135
00136     // メニューフォントサイズ
00137     setValue(object, "menu_font_size", menuFontSize);
00138
00139     // 光源
00140     setVector(object, "ambient", light.ambient);
00141     setVector(object, "diffuse", light.diffuse);
00142     setVector(object, "specular", light.specular);
00143     setVector(object, "position", light.position);
00144
00145     // モデル
00146     setString(object, "model", model);
00147
00148     // シェーダ
00149     setString(object, "shader", shader);
00150
00151     // 構成データをシリアライズして JSON で保存
00152     picojson::value v{ object };
00153     file << v.serialize(true);
00154
00155     return true;
00156 }

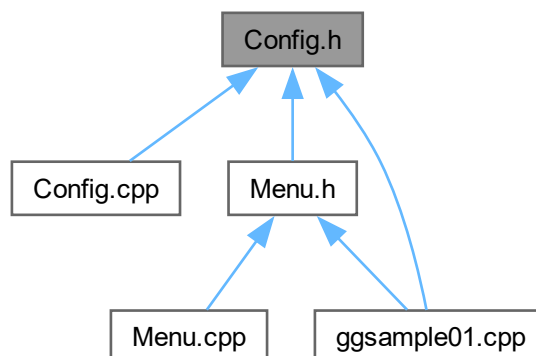
```

10.3 Config.h ファイル

```
#include "gg.h"
#include "parseconfig.h"
Config.h の依存先関係図:
```



被依存関係図:



クラス

- class [Config](#)

10.3.1 詳解

構成データクラスの定義

著者

Kohe Tokoi

日付

November 15, 2022

[Config.h](#) に定義があります。

10.4 Config.h

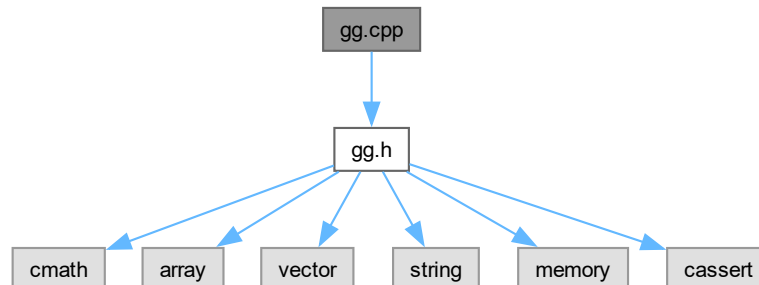
[詳解]

```
00001 #pragma once
00002
00010
00011 // 補助プログラム
00012 #include "gg.h"
00013
00014 // 構成ファイルの読み取り補助
00015 #include "parseconfig.h"
00016
00020 class Config
00021 {
00022     // メニュークラスから参照する
00023     friend class Menu;
00024
00025     // ウィンドウサイズ
00026     std::array<GLsizei, 2> winSize;
00027
00028     // メニューフォント名
00029     std::string menuFont;
00030
00031     // メニューフォントサイズ
00032     float menuFontSize;
00033
00034     // 光源
00035     gg::GgSimpleShader::Light light;
00036
00037     // 形状ファイル名
00038     std::string model;
00039
00040     // シェーダのソースファイル名
00041     std::array<std::string, 3> shader;
00042
00043 public:
00044
00048     Config();
00049
00055     Config(const std::string& filename);
00056
00060     virtual ~Config();
00061
00067     auto getWidth() const
00068     {
00069         return winSize[0];
00070     }
00071
00077     auto getHeight() const
00078     {
00079         return winSize[1];
00080     }
00081
00087     bool load(const std::string& filename);
00088
00094     bool save(const std::string& filename) const;
00095 };
```

10.5 gg.cpp ファイル

```
#include "gg.h"
```

gg.cpp の依存先関係図:



10.5.1 詳解

ゲームグラフィックス特論の宿題用補助プログラム GLFW3 版の定義.

著者

Kohe Tokoi

日付

July 17, 2025

[gg.cpp](#) に定義があります。

10.6 gg.cpp

[詳解]

```
00001 /*
00002
00003 ゲームグラフィックス特論用補助プログラム GLFW3 版
00004
00005 Copyright (c) 2011-2025 Kohe Tokoi. All Rights Reserved.
00006
00007 Permission is hereby granted, free of charge, to any person obtaining a copy
00008 of this software and associated documentation files (the "Software"), to deal
00009 in the Software without restriction, including without limitation the rights
00010 to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00011 copies or substantial portions of the Software.
00012
00013 The above copyright notice and this permission notice shall be included in
00014 all copies or substantial portions of the Software.
00015
00016 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00017 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
00018 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL
00019 KOHE TOKOI BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN
00020 AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN
00021 CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
00022
00023 */
00024
00032 #include "gg.h"
00033
00035
00036 // 標準ライブラリ
00037 #include <float>
00038 #include <stdlib>
```

```

00039 #include <iostream>
00040 #include <fstream>
00041 #include <sstream>
00042 #include <limits>
00043 #include <map>
00044
00046 #define READ_TEXTURE_COORDINATE_FROM_OBJ 0
00047
00048 // Windows (Visual Studio) のとき
00049 #if defined(_MSC_VER)
00050 // リリースビルドではコンソールを出さない
00051 # if !defined(DEBUG)
00052 #   pragma comment(linker, "/subsystem:\"windows\" /entry:\"mainCRTStartup\"")
00053 # endif
00054 // リンクするライブラリ
00055 # pragma comment(lib, "lib\\\" GLFW3-PLATFORM \"\\\" GLFW3-CONFIGURATION \"\\\"glfw3.lib")
00056
00057 //
00058 // For VC++ MFC Convert UTF-8 to TCHAR, or Convert TCHAR to UTF-8. VC++ MFC用 UTF-8⇔TCHARの変換処理
00059 //
00060 //   Original author: mt-u
00061 //   Copyright (c) 2013 mt-u
00062 //   https://gist.github.com/mt-u/6878251
00063 //
00064 //   Modified by: Kohe Tokoi
00065 //
00066
00067 //
00068 // UTF-8 文字列を CString に変換する
00069 //
00070 pathString Utf8ToTChar(const std::string& string)
00071 {
00072     // UTF-8 文字列を UTF-16 に変換した後の文字列の長さを求める
00073     const INT length{ MultiByteToWideChar(CP_UTF8, 0, string.c_str(), -1, NULL, 0) };
00074
00075     // 変換結果の格納に必要な長さのメモリを確保する
00076     std::vector<WCHAR> utf16(length);
00077
00078     // UTF-8 文字列を UTF-16 に変換する
00079     MultiByteToWideChar(CP_UTF8, 0, string.c_str(), -1, utf16.data(), length);
00080
00081     // 変換した文字列を CString にして返す
00082     return CString{ utf16.data(), static_cast<int>(wcslen(utf16.data())) };
00083 }
00084
00085 //
00086 // CString を UTF-8 文字列に変換する
00087 //
00088 std::string TCharToUtf8(const pathString& cstring)
00089 {
00090     // 与えられた文字列を一旦 UTF-16 に変換する
00091     CStringW cstringw{ cstring };
00092
00093     // UTF-16 を UTF-8 に変換した後の文字列の長さを求める
00094     const INT length{ WideCharToMultiByte(CP_UTF8, 0, cstringw, -1, NULL, 0, NULL, NULL) };
00095
00096     // 変換結果の格納に必要な長さのメモリを確保する
00097     std::vector<CHAR> utf8(length);
00098
00099     // UTF-16 を UTF-8 に変換する
00100     WideCharToMultiByte(CP_UTF8, 0, cstringw, -1, utf8.data(), length, NULL, NULL);
00101
00102     // 変換した文字列を std::string にして返す
00103     return std::string{ utf8.data(), strlen(utf8.data()) };
00104 }
00105 #endif
00106
00107 // OpenGL 3.2 の API のエントリポイント
00108 #if !defined(GL3-PROTOTYPES) && !defined(GL_GLES-PROTOTYPES)
00109 PFNGLACTIVEPROGRAMEXTPROC glActiveProgramEXT;
00110 PFNGLACTIVESHADERPROGRAMPROC glActiveShaderProgram;
00111 PFNGLACTIVETEXTUREPROC glActiveTexture;
00112 PFNGLAPPLYFRAMEBUFFERATTACHMENTCMAAINTELPROC glApplyFramebufferAttachmentCMAAINTEL;
00113 PFNGLATTACHSHADERPROC glAttachShader;
00114 PFNGLBEGINCONDITIONALRENDERNVPROC glBeginConditionalRenderNV;
00115 PFNGLBEGINCONDITIONALRENDERPROC glBeginConditionalRender;
00116 PFNGLBEGINPERFMONITORAMDPROC glBeginPerfMonitorAMD;
00117 PFNGLBEGINPERFQUERYINTELPROC glBeginPerfQueryINTEL;
00118 PFNGLBEGINQUERYINDEXEDPROC glBeginQueryIndexed;
00119 PFNGLBEGINQUERYPROC glBeginQuery;
00120 PFNGLBEGINTRANSFORMFEEDBACKPROC glBeginTransformFeedback;
00121 PFNGLBINDATTRIBLOCATIONPROC glBindAttribLocation;
00122 PFNGLBINDBUFFERBASEPROC glBindBufferBase;
00123 PFNGLBINDBUFFERPROC glBindBuffer;
00124 PFNGLBINDBUFFERRANGEPROC glBindBufferRange;
00125 PFNGLBINDBUFFERSBASEPROC glBindBuffersBase;
00126 PFNGLBINDBUFFERSRANGEPROC glBindBuffersRange;

```

```
00127 PFNGLBINDFRAGDATALOCATIONINDEXEDPROC glBindFragDataLocationIndexed;
00128 PFNGLBINDFRAGDATALOCATIONPROC glBindFragDataLocation;
00129 PFNGLBINDFRAMEBUFFERPROC glBindFramebuffer;
00130 PFNGLBINDIMAGETEXTUREPROC glBindImageTexture;
00131 PFNGLBINDIMAGETEXTURESPROC glBindImageTextures;
00132 PFNGLBINDMULTITEXTUREEXTPROC glBindMultiTextureEXT;
00133 PFNGLBINDPROGRAMPIPELINEPROC glBindProgramPipeline;
00134 PFNGLBINDRENDERBUFFERPROC glBindRenderbuffer;
00135 PFNGLBINDSAMPLERPROC glBindSampler;
00136 PFNGLBINDSAMPLERSPROC glBindSamplers;
00137 PFNGLBINDTEXTUREPROC glBindTexture;
00138 PFNGLBINDTEXTURESPROC glBindTextures;
00139 PFNGLBINDTEXTUREUNITPROC glBindTextureUnit;
00140 PFNGLBINDTRANSFORMFEEDBACKPROC glBindTransformFeedback;
00141 PFNGLBINDVERTEXARRAYPROC glBindVertexArray;
00142 PFNGLBINDVERTEXBUFFERPROC glBindVertexBuffer;
00143 PFNGLBINDVERTEXBUFFERSPROC glBindVertexBuffers;
00144 PFNGLBLENDARRIERKHRPROC glBlendBarrierKHR;
00145 PFNGLBLENDARRIERNVPROC glBlendBarrierNV;
00146 PFNGLBLENDCOLORPROC glBlendColor;
00147 PFNGLLENDEQUATIONIARBPROC glBlendEquationiARB;
00148 PFNGLLENDEQUATIONIPROC glBlendEquationi;
00149 PFNGLLENDEQUATIONPROC glBlendEquation;
00150 PFNGLLENDEQUATIONSEPARATEIARBPROC glBlendEquationSeparateiARB;
00151 PFNGLLENDEQUATIONSEPARATEIPROC glBlendEquationSeparatei;
00152 PFNGLLENDEQUATIONSEPARATEPROC glBlendEquationSeparate;
00153 PFNGLLENDFUNCIARBPROC glBlendFunciARB;
00154 PFNGLLENDFUNCIPROC glBlendFunci;
00155 PFNGLLENDFUNCPROC glBlendFunc;
00156 PFNGLLENDFUNCSEPARATEIARBPROC glBlendFuncSeparateiARB;
00157 PFNGLLENDFUNCSEPARATEIPROC glBlendFuncSeparatei;
00158 PFNGLLENDFUNCSEPARATEPROC glBlendFuncSeparate;
00159 PFNGLLENDPARAMETERINVPROC glBlendParameteriNV;
00160 PFNGLBLITFRAMEBUFFERPROC glBlitFramebuffer;
00161 PFNGLBLITNAMEDFRAMEBUFFERPROC glBlitNamedFramebuffer;
00162 PFNGLBUFFERADDRESSRANGENVPROC glBufferAddressRangeNV;
00163 PFNGLBUFFERDATAPROC glBufferData;
00164 PFNGLBUFFERPAGECOMMITMENTARBPROC glBufferPageCommitmentARB;
00165 PFNGLBUFFERSTORAGEPROC glBufferStorage;
00166 PFNGLBUFFERSUBDATAPROC glBufferSubData;
00167 PFNGLCALLCOMMANDLISTNVPROC glCallCommandListNV;
00168 PFNGLCHECKFRAMEBUFFERSTATUSPROC glCheckFramebufferStatus;
00169 PFNGLCHECKNAMEDFRAMEBUFFERSTATUSEXTPROC glCheckNamedFramebufferStatusEXT;
00170 PFNGLCHECKNAMEDFRAMEBUFFERSTATUSPROC glCheckNamedFramebufferStatus;
00171 PFNGLCLAMPCOLORPROC glClampColor;
00172 PFNGLCLEARBUFFERDATAPROC glClearBufferData;
00173 PFNGLCLEARBUFFERFIPROC glClearBufferfi;
00174 PFNGLCLEARBUFFERFVPROC glClearBufferfv;
00175 PFNGLCLEARBUFFERIVPROC glClearBufferiv;
00176 PFNGLCLEARBUFFERSUBDATAPROC glClearBufferSubData;
00177 PFNGLCLEARBUFFERUIVPROC glClearBufferuiv;
00178 PFNGLCLEARCOLORPROC glClearColor;
00179 PFNGLCLEARDEPTHFPROC glClearDepthf;
00180 PFNGLCLEARDEPTHPROC glClearDepth;
00181 PFNGLCLEARNAMEDBUFFERDATAEXTPROC glClearNamedBufferDataEXT;
00182 PFNGLCLEARNAMEDBUFFERDATAPROC glClearNamedBufferData;
00183 PFNGLCLEARNAMEDBUFFERSUBDATAEXTPROC glClearNamedBufferSubDataEXT;
00184 PFNGLCLEARNAMEDBUFFERSUBDATAPROC glClearNamedBufferSubData;
00185 PFNGLCLEARNAMEDFRAMEBUFFERFIPROC glClearNamedFramebufferfi;
00186 PFNGLCLEARNAMEDFRAMEBUFFERFVPROC glClearNamedFramebufferfv;
00187 PFNGLCLEARNAMEDFRAMEBUFFERIVPROC glClearNamedFramebufferiv;
00188 PFNGLCLEARNAMEDFRAMEBUFFERUIVPROC glClearNamedFramebufferuiv;
00189 PFNGLCLEARPROC glClear;
00190 PFNGLCLEARSTENCILPROC glClearStencil;
00191 PFNGLCLEARTEXIMAGEPROC glClearTexImage;
00192 PFNGLCLEARTEXSUBIMAGEPROC glClearTexSubImage;
00193 PFNGLCLIENTATTRIBDEFAULTTEXTPROC glClientAttribDefaultEXT;
00194 PFNGLCLIENTWAITSYNCPROC glClientWaitSync;
00195 PFNGLCLIPCONTROLPROC glClipControl;
00196 PFNGLCOLORFORMATNVPROC glColorFormatNV;
00197 PFNGLCOLORMASKIPROC glColorMaski;
00198 PFNGLCOLORMASKPROC glColorMask;
00199 PFNGLCOMMANDLISTSEGMENTSNVPROC glCommandListSegmentsNV;
00200 PFNGLCOMPILECOMMANDLISTNVPROC glCompileCommandListNV;
00201 PFNGLCOMPILESHADERINCLUDEARBPROC glCompileShaderIncludeARB;
00202 PFNGLCOMPILESHADERPROC glCompileShader;
00203 PFNGLCOMPRESSEDMULTITEXIMAGE1DXTPROC glCompressedMultiTexImage1DXT;
00204 PFNGLCOMPRESSEDMULTITEXIMAGE2DXTPROC glCompressedMultiTexImage2DXT;
00205 PFNGLCOMPRESSEDMULTITEXIMAGE3DXTPROC glCompressedMultiTexImage3DXT;
00206 PFNGLCOMPRESSEDMULTITEXSUBIMAGE1DXTPROC glCompressedMultiTexSubImage1DXT;
00207 PFNGLCOMPRESSEDMULTITEXSUBIMAGE2DXTPROC glCompressedMultiTexSubImage2DXT;
00208 PFNGLCOMPRESSEDMULTITEXSUBIMAGE3DXTPROC glCompressedMultiTexSubImage3DXT;
00209 PFNGLCOMPRESSEDTEXIMAGE1DPROC glCompressedTexImage1D;
00210 PFNGLCOMPRESSEDTEXIMAGE2DPROC glCompressedTexImage2D;
00211 PFNGLCOMPRESSEDTEXIMAGE3DPROC glCompressedTexImage3D;
00212 PFNGLCOMPRESSEDTEXSUBIMAGE1DPROC glCompressedTexSubImage1D;
00213 PFNGLCOMPRESSEDTEXSUBIMAGE2DPROC glCompressedTexSubImage2D;
```

```
00214 PFNGLCOMPRESSEDTEXSUBIMAGE3DPROC glCompressedTexSubImage3D;
00215 PFNGLCOMPRESSEDTEXTUREIMAGE1DEXTPROC glCompressedTextureImage1DEXT;
00216 PFNGLCOMPRESSEDTEXTUREIMAGE2DEXTPROC glCompressedTextureImage2DEXT;
00217 PFNGLCOMPRESSEDTEXTUREIMAGE3DEXTPROC glCompressedTextureImage3DEXT;
00218 PFNGLCOMPRESSEDTEXTURESUBIMAGE1DEXTPROC glCompressedTextureSubImage1DEXT;
00219 PFNGLCOMPRESSEDTEXTURESUBIMAGE1DPROC glCompressedTextureSubImage1D;
00220 PFNGLCOMPRESSEDTEXTURESUBIMAGE2DEXTPROC glCompressedTextureSubImage2DEXT;
00221 PFNGLCOMPRESSEDTEXTURESUBIMAGE2DPROC glCompressedTextureSubImage2D;
00222 PFNGLCOMPRESSEDTEXTURESUBIMAGE3DEXTPROC glCompressedTextureSubImage3DEXT;
00223 PFNGLCOMPRESSEDTEXTURESUBIMAGE3DPROC glCompressedTextureSubImage3D;
00224 PFNGLCONSERVATIVERASTERPARAMETERFNVPROC glConservativeRasterParameterfNV;
00225 PFNGLCONSERVATIVERASTERPARAMETERINVPROC glConservativeRasterParameteriNV;
00226 PFNGLCOPYBUFFERSUBDATAPROC glCopyBufferSubData;
00227 PFNGLCOPYIMAGESUBDATAPROC glCopyImageSubData;
00228 PFNGLCOPYMULTITEXTUREIMAGE1DEXTPROC glCopyMultiTexImage1DEXT;
00229 PFNGLCOPYMULTITEXTUREIMAGE2DEXTPROC glCopyMultiTexImage2DEXT;
00230 PFNGLCOPYMULTITEXTURESUBIMAGE1DEXTPROC glCopyMultiTexSubImage1DEXT;
00231 PFNGLCOPYMULTITEXTURESUBIMAGE2DEXTPROC glCopyMultiTexSubImage2DEXT;
00232 PFNGLCOPYMULTITEXTURESUBIMAGE3DEXTPROC glCopyMultiTexSubImage3DEXT;
00233 PFNGLCOPYNAMEDBUFFERSUBDATAPROC glCopyNamedBufferSubData;
00234 PFNGLCOPYPATHNVPROC glCopyPathNV;
00235 PFNGLCOPYTEXIMAGE1DPROC glCopyTexImage1D;
00236 PFNGLCOPYTEXIMAGE2DPROC glCopyTexImage2D;
00237 PFNGLCOPYTEXSUBIMAGE1DPROC glCopyTexSubImage1D;
00238 PFNGLCOPYTEXSUBIMAGE2DPROC glCopyTexSubImage2D;
00239 PFNGLCOPYTEXSUBIMAGE3DPROC glCopyTexSubImage3D;
00240 PFNGLCOPYTEXTUREIMAGE1DEXTPROC glCopyTextureImage1DEXT;
00241 PFNGLCOPYTEXTUREIMAGE2DEXTPROC glCopyTextureImage2DEXT;
00242 PFNGLCOPYTEXTURESUBIMAGE1DEXTPROC glCopyTextureSubImage1DEXT;
00243 PFNGLCOPYTEXTURESUBIMAGE1DPROC glCopyTextureSubImage1D;
00244 PFNGLCOPYTEXTURESUBIMAGE2DEXTPROC glCopyTextureSubImage2DEXT;
00245 PFNGLCOPYTEXTURESUBIMAGE2DPROC glCopyTextureSubImage2D;
00246 PFNGLCOPYTEXTURESUBIMAGE3DEXTPROC glCopyTextureSubImage3DEXT;
00247 PFNGLCOPYTEXTURESUBIMAGE3DPROC glCopyTextureSubImage3D;
00248 PFNGLCOVERAGEMODULATIONNVPROC glCoverageModulationNV;
00249 PFNGLCOVERAGEMODULATIONTABLENVPROC glCoverageModulationTableNV;
00250 PFNGLCOVERFILLPATHINSTANCEDNVPROC glCoverFillPathInstancedNV;
00251 PFNGLCOVERFILLPATHNVPROC glCoverFillPathNV;
00252 PFNGLCOVERSTROKEPATHINSTANCEDNVPROC glCoverStrokePathInstancedNV;
00253 PFNGLCOVERSTROKEPATHNVPROC glCoverStrokePathNV;
00254 PFNGLCREATEBUFFERSPROC glCreateBuffers;
00255 PFNGLCREATECOMMANDLISTSNVPROC glCreateCommandListsNV;
00256 PFNGLCREATEFRAMEBUFFERSPROC glCreateFramebuffers;
00257 PFNGLCREATEPERFQUERYINTELPROC glCreatePerfQueryINTEL;
00258 PFNGLCREATEPROGRAMPIPELINESPROC glCreateProgramPipelines;
00259 PFNGLCREATEPROGRAMPROC glCreateProgram;
00260 PFNGLCREATEQUERIESPROC glCreateQueries;
00261 PFNGLCREATERENDERBUFFERSPROC glCreateRenderbuffers;
00262 PFNGLCREATESAMPLERSPROC glCreateSamplers;
00263 PFNGLCREATESHADERPROC glCreateShader;
00264 PFNGLCREATESHADERPROGRAMEXTPROC glCreateShaderProgramEXT;
00265 PFNGLCREATESHADERPROGRAMVPROC glCreateShaderProgramv;
00266 PFNGLCREATESTATESNVPROC glCreateStatesNV;
00267 PFNGLCREATESYNCFROMCLEVENTARBPROC glCreateSyncFromCLeventARB;
00268 PFNGLCREATETEXTURESPROC glCreateTextures;
00269 PFNGLCREATETRANSFORMFEEDBACKSPROC glCreateTransformFeedbacks;
00270 PFNGLCREATEVERTEXARRAYSPROC glCreateVertexArrays;
00271 PFNGLCULLFACEPROC glCullFace;
00272 PFNGLDEBUGMESSAGECALLBACKARBPROC glDebugMessageCallbackARB;
00273 PFNGLDEBUGMESSAGECALLBACKPROC glDebugMessageCallback;
00274 PFNGLDEBUGMESSAGECONTROLARBPROC glDebugMessageControlARB;
00275 PFNGLDEBUGMESSAGECONTROLPROC glDebugMessageControl;
00276 PFNGLDEBUGMESSAGEINSERTARBPROC glDebugMessageInsertARB;
00277 PFNGLDEBUGMESSAGEINSERTPROC glDebugMessageInsert;
00278 PFNGLDELETEBUFFERSPROC glDeleteBuffers;
00279 PFNGLDELETECOMMANDLISTSNVPROC glDeleteCommandListsNV;
00280 PFNGLDELETEFRAMEBUFFERSPROC glDeleteFramebuffers;
00281 PFNGLDELETENAMEDSTRINGARBPROC glDeleteNamedStringARB;
00282 PFNGLDELETEPATHSNVPROC glDeletePathsNV;
00283 PFNGLDELETEPERFMONITORSAMDPROC glDeletePerfMonitorsAMD;
00284 PFNGLDELETEPERFQUERYINTELPROC glDeletePerfQueryINTEL;
00285 PFNGLDELETEPROGRAMPIPELINESPROC glDeleteProgramPipelines;
00286 PFNGLDELETEPROGRAMPROC glDeleteProgram;
00287 PFNGLDELETEQUERIESPROC glDeleteQueries;
00288 PFNGLDELETERENDERBUFFERSPROC glDeleteRenderbuffers;
00289 PFNGLDELETESAMPLERSPROC glDeleteSamplers;
00290 PFNGLDELETESHADERPROC glDeleteShader;
00291 PFNGLDELETESSTATESNVPROC glDeleteStatesNV;
00292 PFNGLDELETESYNCPROC glDeleteSync;
00293 PFNGLDELETETEXTURESPROC glDeleteTextures;
00294 PFNGLDELETETRANSFORMFEEDBACKSPROC glDeleteTransformFeedbacks;
00295 PFNGLDELETEVERTEXARRAYSPROC glDeleteVertexArrays;
00296 PFNGLDEPTHFUNCPROC glDepthFunc;
00297 PFNGLDEPTHMASKPROC glDepthMask;
00298 PFNGLDEPTHRANGEARRAYVPROC glDepthRangeArrayv;
00299 PFNGLDEPTHRANGEFPROC glDepthRangef;
00300 PFNGLDEPTHRANGEINDEXEDPROC glDepthRangeIndexed;
```

```
00301 PFNGLDEPTHRangePROC glDepthRange;
00302 PFNGLDETACHSHADERPROC glDetachShader;
00303 PFNGLDISABLECLIENTSTATEIEXTPROC glDisableClientStateiEXT;
00304 PFNGLDISABLECLIENTSTATEINDEXEDEXTPROC glDisableClientStateIndexedEXT;
00305 PFNGLDISABLEINDEXEDEXTPROC glDisableIndexedEXT;
00306 PFNGLDISABLEIPROC glDisablei;
00307 PFNGLDISABLEPROC glDisable;
00308 PFNGLDISABLEVERTEXARRAYATTRIBEXTPROC glDisableVertexArrayAttribEXT;
00309 PFNGLDISABLEVERTEXARRAYATTRIBPROC glDisableVertexArrayAttrib;
00310 PFNGLDISABLEVERTEXARRAYEXTPROC glDisableVertexArrayEXT;
00311 PFNGLDISABLEVERTEXATTRIBARRAYPROC glDisableVertexAttribArray;
00312 PFNGLDISPATCHCOMPUTEGROUPSIZEARBPROC glDispatchComputeGroupSizeARB;
00313 PFNGLDISPATCHCOMPUTEINDIRECTPROC glDispatchComputeIndirect;
00314 PFNGLDISPATCHCOMPUTEPROC glDispatchCompute;
00315 PFNGLDRAWARRAYSINDIRECTPROC glDrawArraysIndirect;
00316 PFNGLDRAWARRAYSINSTANCEDARBPROC glDrawArraysInstancedARB;
00317 PFNGLDRAWARRAYSINSTANCEDBASEINSTANCEPROC glDrawArraysInstancedBaseInstance;
00318 PFNGLDRAWARRAYSINSTANCEDEEXTPROC glDrawArraysInstancedEXT;
00319 PFNGLDRAWARRAYSINSTANCEDPROC glDrawArraysInstanced;
00320 PFNGLDRAWARRAYSPROC glDrawArrays;
00321 PFNGLDRAWBUFFERPROC glDrawBuffer;
00322 PFNGLDRAWBUFFERSPROC glDrawBuffers;
00323 PFNGLDRAWCOMMANDSADDRESSNVPROC glDrawCommandsAddressNV;
00324 PFNGLDRAWCOMMANDSNVPROC glDrawCommandsNV;
00325 PFNGLDRAWCOMMANDSSTATESADDRESSNVPROC glDrawCommandsStatesAddressNV;
00326 PFNGLDRAWCOMMANDSSTATENVPROC glDrawCommandsStatesNV;
00327 PFNGLDRAWELEMENTSBASEVERTEXPROC glDrawElementsBaseVertex;
00328 PFNGLDRAWELEMENTSINDIRECTPROC glDrawElementsIndirect;
00329 PFNGLDRAWELEMENTSINSTANCEDARBPROC glDrawElementsInstancedARB;
00330 PFNGLDRAWELEMENTSINSTANCEDBASEINSTANCEPROC glDrawElementsInstancedBaseInstance;
00331 PFNGLDRAWELEMENTSINSTANCEDBASEVERTEXBASEINSTANCEPROC glDrawElementsInstancedBaseVertexBaseInstance;
00332 PFNGLDRAWELEMENTSINSTANCEDBASEVERTEXPROC glDrawElementsInstancedBaseVertex;
00333 PFNGLDRAWELEMENTSINSTANCEDEEXTPROC glDrawElementsInstancedEXT;
00334 PFNGLDRAWELEMENTSINSTANCEDPROC glDrawElementsInstanced;
00335 PFNGLDRAWELEMENTSPROC glDrawElements;
00336 PFNGLDRAWRANGEELEMENTSBASEVERTEXPROC glDrawRangeElementsBaseVertex;
00337 PFNGLDRAWRANGEELEMENTSPROC glDrawRangeElements;
00338 PFNGLDRAWTRANSFORMFEEDBACKINSTANCEDPROC glDrawTransformFeedbackInstanced;
00339 PFNGLDRAWTRANSFORMFEEDBACKPROC glDrawTransformFeedback;
00340 PFNGLDRAWTRANSFORMFEEDBACKSTREAMINSTANCEDPROC glDrawTransformFeedbackStreamInstanced;
00341 PFNGLDRAWTRANSFORMFEEDBACKSTREAMPROC glDrawTransformFeedbackStream;
00342 PFNGLDRAWVKIMAGENVPROC glDrawVkImageNV;
00343 PFNGLEDGEFLAGFORMATNVPROC glEdgeFlagFormatNV;
00344 PFNGLENABLECLIENTSTATEIEXTPROC glEnableClientStateiEXT;
00345 PFNGLENABLECLIENTSTATEINDEXEDEXTPROC glEnableClientStateIndexedEXT;
00346 PFNGLENABLEINDEXEDEXTPROC glEnableIndexedEXT;
00347 PFNGLENABLEIPROC glEnablei;
00348 PFNGLENABLEPROC glEnable;
00349 PFNGLENABLEVERTEXARRAYATTRIBEXTPROC glEnableVertexArrayAttribEXT;
00350 PFNGLENABLEVERTEXARRAYATTRIBPROC glEnableVertexArrayAttrib;
00351 PFNGLENABLEVERTEXARRAYEXTPROC glEnableVertexArrayEXT;
00352 PFNGLENABLEVERTEXATTRIBARRAYPROC glEnableVertexAttribArray;
00353 PFNGLENDCONDITIONALRENDERNVPROC glEndConditionalRenderNV;
00354 PFNGLENDCONDITIONALRENDERPROC glEndConditionalRender;
00355 PFNGLENDPERFMONITORAMDPROC glEndPerfMonitorAMD;
00356 PFNGLENDPERFQUERYINTELPROC glEndPerfQueryINTEL;
00357 PFNGLENDQUERYINDEXEDPROC glEndQueryIndexed;
00358 PFNGLENDQUERYPROC glEndQuery;
00359 PFNGLENDTRANSFORMFEEDBACKPROC glEndTransformFeedback;
00360 PFNGLEVALUATEDEPTHVALUESARBPROC glEvaluateDepthValuesARB;
00361 PFNGLFENCESYNCPROC glFenceSync;
00362 PFNGLFINISHPROC glFinish;
00363 PFNGLFLUSHMAPPEDBUFFERRANGEPROC glFlushMappedBufferRange;
00364 PFNGLFLUSHMAPPEDNAMEDBUFFERRANGEEXTPROC glFlushMappedNamedBufferRangeEXT;
00365 PFNGLFLUSHMAPPEDNAMEDBUFFERRANGEPROC glFlushMappedNamedBufferRange;
00366 PFNGLFLUSHPROC glFlush;
00367 PFNGLFOGCOORDFORMATNVPROC glFogCoordFormatNV;
00368 PFNGLFRAGMENTCOVERAGECOLORNVPROC glFragmentCoverageColorNV;
00369 PFNGLFRAMEBUFFERDRAWBUFFEREXTPROC glFramebufferDrawBufferEXT;
00370 PFNGLFRAMEBUFFERDRAWBUFFERSEXTPROC glFramebufferDrawBuffersEXT;
00371 PFNGLFRAMEBUFFERPARAMETERIPROC glFramebufferParameteri;
00372 PFNGLFRAMEBUFFERREADBUFFEREXTPROC glFramebufferReadBufferEXT;
00373 PFNGLFRAMEBUFFERRENDERBUFFERPROC glFramebufferRenderbuffer;
00374 PFNGLFRAMEBUFFERSAMPLELOCATIONSFVARBPROC glFramebufferSampleLocationsfvARB;
00375 PFNGLFRAMEBUFFERSAMPLELOCATIONSFVNVPROC glFramebufferSampleLocationsfvNV;
00376 PFNGLFRAMEBUFFERTEXTURE1DPROC glFramebufferTexture1D;
00377 PFNGLFRAMEBUFFERTEXTURE2DPROC glFramebufferTexture2D;
00378 PFNGLFRAMEBUFFERTEXTURE3DPROC glFramebufferTexture3D;
00379 PFNGLFRAMEBUFFERTEXTUREARBPROC glFramebufferTextureARB;
00380 PFNGLFRAMEBUFFERTEXTUREFACEARBPROC glFramebufferTextureFaceARB;
00381 PFNGLFRAMEBUFFERTEXTURELAYERARBPROC glFramebufferTextureLayerARB;
00382 PFNGLFRAMEBUFFERTEXTURELAYERPROC glFramebufferTextureLayer;
00383 PFNGLFRAMEBUFFERTEXTUREMULTIVIEWOVRPROC glFramebufferTextureMultiviewOVR;
00384 PFNGLFRAMEBUFFERTEXTUREPROC glFramebufferTexture;
00385 PFNGLFRONTFACEPROC glFrontFace;
00386 PFNGLGENBUFFERSPROC glGenBuffers;
00387 PFNGLGENERATEMIPMAPPROC glGenerateMipmap;
```

```
00388 PFNGLGENERATEMULTITEXMIPMAPEXTPROC glGenerateMultiTexMipmapEXT;
00389 PFNGLGENERATETEXTUREMIPMAPEXTPROC glGenerateTextureMipmapEXT;
00390 PFNGLGENERATETEXTUREMIPMAPPROC glGenerateTextureMipmap;
00391 PFNGLGENFRAMEBUFFERSPROC glGenFramebuffers;
00392 PFNGLGENPATHSNVPROC glGenPathsNV;
00393 PFNGLGENPERFMONITORSAMDPROC glGenPerfMonitorsAMD;
00394 PFNGLGENPROGRAMPIPELINESPROC glGenProgramPipelines;
00395 PFNGLGENQUERIESPROC glGenQueries;
00396 PFNGLGENRENDERBUFFERSPROC glGenRenderbuffers;
00397 PFNGLGENSAMPLERSPROC glGenSamplers;
00398 PFNGLGENTEXTURESPROC glGenTextures;
00399 PFNGLGENTRANSFORMFEEDBACKSPROC glGenTransformFeedbacks;
00400 PFNGLGENVERTEXARRAYSPROC glGenVertexArrays;
00401 PFNGLGETACTIVEATOMICCOUNTERBUFFERIVPROC glGetActiveAtomicCounterBufferiv;
00402 PFNGLGETACTIVEATTRIBPROC glGetActiveAttrib;
00403 PFNGLGETACTIVESUBROUTINENAMEPROC glGetActiveSubroutineName;
00404 PFNGLGETACTIVESUBROUTINEUNIFORMIVPROC glGetActiveSubroutineUniformiv;
00405 PFNGLGETACTIVESUBROUTINEUNIFORMNAMEPROC glGetActiveSubroutineUniformName;
00406 PFNGLGETACTIVEUNIFORMBLOCKIVPROC glGetActiveUniformBlockiv;
00407 PFNGLGETACTIVEUNIFORMBLOCKNAMEPROC glGetActiveUniformBlockName;
00408 PFNGLGETACTIVEUNIFORMNAMEPROC glGetActiveUniformName;
00409 PFNGLGETACTIVEUNIFORMPROC glGetActiveUniform;
00410 PFNGLGETACTIVEUNIFORMSIVPROC glGetActiveUniformsiv;
00411 PFNGLGETATTACHEDSHADERSPROC glGetAttachedShaders;
00412 PFNGLGETATTRIBLOCATIONPROC glGetAttribLocation;
00413 PFNGLGETBOOLEANINDEXEDVEXTPROC glGetBooleanIndexedvEXT;
00414 PFNGLGETBOOLEANI_VPROC glGetBooleani_v;
00415 PFNGLGETBOOLEANVPROC glGetBooleanv;
00416 PFNGLGETBUFFERPARAMETERI64VPROC glGetBufferParameteri64v;
00417 PFNGLGETBUFFERPARAMETERIVPROC glGetBufferParameteriv;
00418 PFNGLGETBUFFERPARAMETERUI64VNVPROC glGetBufferParameterui64vNV;
00419 PFNGLGETBUFFERPOINTERVPROC glGetBufferPointerv;
00420 PFNGLGETBUFFERSUBDATAPROC glGetBufferSubData;
00421 PFNGLGETCOMMANDHEADERNVPROC glGetCommandHeaderNV;
00422 PFNGLGETCOMPRESSEDMULTITEXIMAGEEXTPROC glGetCompressedMultiTexImageEXT;
00423 PFNGLGETCOMPRESSEDTEXIMAGEPROC glGetCompressedTexImage;
00424 PFNGLGETCOMPRESSEDTEXTUREIMAGEEXTPROC glGetCompressedTextureImageEXT;
00425 PFNGLGETCOMPRESSEDTEXTUREIMAGEPROC glGetCompressedTextureImage;
00426 PFNGLGETCOMPRESSEDTEXTURESUBIMAGEPROC glGetCompressedTextureSubImage;
00427 PFNGLGETCOVERAGEMODULATIONTABLENVPROC glGetCoverageModulationTableNV;
00428 PFNGLGETDEBUGMESSAGELOGARBPROC glGetDebugMessageLogARB;
00429 PFNGLGETDEBUGMESSAGELOGPROC glGetDebugMessageLog;
00430 PFNGLGETDOUBLEINDEXEDVEXTPROC glGetDoubleIndexedvEXT;
00431 PFNGLGETDOUBLEI_VEXTPROC glGetDoublei_vEXT;
00432 PFNGLGETDOUBLEI_VPROC glGetDoublei_v;
00433 PFNGLGETDOUBLEVPROC glGetDoublev;
00434 PFNGLGETERRORPROC glGetError;
00435 PFNGLGETFIRSTPERFQUERYIDINTELPROC glGetFirstPerfQueryIdINTEL;
00436 PFNGLGETFLOATINDEXEDVEXTPROC glGetFloatIndexedvEXT;
00437 PFNGLGETFLOATI_VEXTPROC glGetFloati_vEXT;
00438 PFNGLGETFLOATI_VPROC glGetFloati_v;
00439 PFNGLGETFLOATVPROC glGetFloatv;
00440 PFNGLGETFRAGDATAINDEXPROC glGetFragDataIndex;
00441 PFNGLGETFRAGDATALOCATIONPROC glGetFragDataLocation;
00442 PFNGLGETFRAMEBUFFERATTACHMENTPARAMETERIVPROC glGetFramebufferAttachmentParameteriv;
00443 PFNGLGETFRAMEBUFFERPARAMETERIVEXTPROC glGetFramebufferParameterivEXT;
00444 PFNGLGETFRAMEBUFFERPARAMETERIVPROC glGetFramebufferParameteriv;
00445 PFNGLGETGRAPHICSRESETSTATUSARBPROC glGetGraphicsResetStatusARB;
00446 PFNGLGETGRAPHICSRESETSTATUSPROC glGetGraphicsResetStatus;
00447 PFNGLGETIMAGEHANDLEARBPROC glGetImageHandleARB;
00448 PFNGLGETIMAGEHANDLENVPROC glGetImageHandleNV;
00449 PFNGLGETINTEGER64I_VPROC glGetInteger64i_v;
00450 PFNGLGETINTEGER64VPROC glGetInteger64v;
00451 PFNGLGETINTEGERINDEXEDVEXTPROC glGetIntegerIndexedvEXT;
00452 PFNGLGETINTEGERI_VPROC glGetIntegeri_v;
00453 PFNGLGETINTEGERUI64I_VNVPROC glGetIntegerui64i_vNV;
00454 PFNGLGETINTEGERUI64VNVPROC glGetIntegerui64vNV;
00455 PFNGLGETINTEGERVPROC glGetIntegerv;
00456 PFNGLGETINTERNALFORMATI64VPROC glGetInternalformati64v;
00457 PFNGLGETINTERNALFORMATIVPROC glGetInternalformativ;
00458 PFNGLGETINTERNALFORMATSAMPLEIIVNVPROC glGetInternalformatSampleiivNV;
00459 PFNGLGETMULTISAMPLEFVPROC glGetMultisamplefv;
00460 PFNGLGETMULTITEXENVFVEXTPROC glGetMultiTexEnvfvEXT;
00461 PFNGLGETMULTITEXENVIVEXTPROC glGetMultiTexEnvivEXT;
00462 PFNGLGETMULTITEXGENDVEXTPROC glGetMultiTexGendvEXT;
00463 PFNGLGETMULTITEXGENFVEXTPROC glGetMultiTexGenfvEXT;
00464 PFNGLGETMULTITEXGENIVEXTPROC glGetMultiTexGenivEXT;
00465 PFNGLGETMULTITEXIMAGEEXTPROC glGetMultiTexImageEXT;
00466 PFNGLGETMULTITEXLEVELPARAMETERFVEXTPROC glGetMultiTexLevelParameterfvEXT;
00467 PFNGLGETMULTITEXLEVELPARAMETERIVEXTPROC glGetMultiTexLevelParameterivEXT;
00468 PFNGLGETMULTITEXPARAMETERFVEXTPROC glGetMultiTexParameterfvEXT;
00469 PFNGLGETMULTITEXPARAMETERIVEXTPROC glGetMultiTexParameterivEXT;
00470 PFNGLGETMULTITEXPARAMETERIUIVEXTPROC glGetMultiTexParameteriuiEXT;
00471 PFNGLGETMULTITEXPARAMETERIVEXTPROC glGetMultiTexParameterivEXT;
00472 PFNGLGETNAMEDBUFFERPARAMETERI64VPROC glGetNamedBufferParameteri64v;
00473 PFNGLGETNAMEDBUFFERPARAMETERIVEXTPROC glGetNamedBufferParameterivEXT;
00474 PFNGLGETNAMEDBUFFERPARAMETERIVPROC glGetNamedBufferParameteriv;
```

```
00475 PFNGLGETNAMEDBUFFERPARAMETERUI64VNVPROC glGetNamedBufferParameterui64vNV;
00476 PFNGLGETNAMEDBUFFERPOINTERVEXTPROC glGetNamedBufferPointervEXT;
00477 PFNGLGETNAMEDBUFFERPOINTERVPROC glGetNamedBufferPointerv;
00478 PFNGLGETNAMEDBUFFERSUBDATAEXTPROC glGetNamedBufferSubDataEXT;
00479 PFNGLGETNAMEDBUFFERSUBDATAPROC glGetNamedBufferSubData;
00480 PFNGLGETNAMEDFRAMEBUFFERATTACHMENTPARAMETERIVEXTPROC glGetNamedFramebufferAttachmentParameterivEXT;
00481 PFNGLGETNAMEDFRAMEBUFFERATTACHMENTPARAMETERIVPROC glGetNamedFramebufferAttachmentParameteriv;
00482 PFNGLGETNAMEDFRAMEBUFFERPARAMETERIVEXTPROC glGetNamedFramebufferParameterivEXT;
00483 PFNGLGETNAMEDFRAMEBUFFERPARAMETERIVPROC glGetNamedFramebufferParameteriv;
00484 PFNGLGETNAMEDPROGRAMIVEXTPROC glGetNamedProgramivEXT;
00485 PFNGLGETNAMEDPROGRAMLOCALPARAMETERDVEXTPROC glGetNamedProgramLocalParameterdvEXT;
00486 PFNGLGETNAMEDPROGRAMLOCALPARAMETERFVEXTPROC glGetNamedProgramLocalParameterfvEXT;
00487 PFNGLGETNAMEDPROGRAMLOCALPARAMETERIIVEXTPROC glGetNamedProgramLocalParameterIivEXT;
00488 PFNGLGETNAMEDPROGRAMLOCALPARAMETERIUIVEXTPROC glGetNamedProgramLocalParameterIuivEXT;
00489 PFNGLGETNAMEDPROGRAMSTRINGEXTPROC glGetNamedProgramStringEXT;
00490 PFNGLGETNAMEDRENDERBUFFERPARAMETERIVEXTPROC glGetNamedRenderbufferParameterivEXT;
00491 PFNGLGETNAMEDRENDERBUFFERPARAMETERIVPROC glGetNamedRenderbufferParameteriv;
00492 PFNGLGETNAMEDSTRINGARBPROC glGetNamedStringARB;
00493 PFNGLGETNAMEDSTRINGIVARBPROC glGetNamedStringivARB;
00494 PFNGLGETNCOMPRESSEDTEXIMAGEARBPROC glGetnCompressedTexImageARB;
00495 PFNGLGETNCOMPRESSEDTEXIMAGEPROC glGetnCompressedTexImage;
00496 PFNGLGETNEXTPERFQUERYIDINTELPROC glGetNextPerfQueryIdINTEL;
00497 PFNGLGETNTEXIMAGEARBPROC glGetnTexImageARB;
00498 PFNGLGETNTEXIMAGEPROC glGetnTexImage;
00499 PFNGLGETNUNIFORMDVARBPROC glGetnUniformdvARB;
00500 PFNGLGETNUNIFORMDVPROC glGetnUniformdv;
00501 PFNGLGETNUNIFORMFVARBPROC glGetnUniformfvARB;
00502 PFNGLGETNUNIFORMFVPROC glGetnUniformfv;
00503 PFNGLGETNUNIFORMI64VARBPROC glGetnUniformi64vARB;
00504 PFNGLGETNUNIFORMIVARBPROC glGetnUniformivARB;
00505 PFNGLGETNUNIFORMIVPROC glGetnUniformiv;
00506 PFNGLGETNUNIFORMUI64VARBPROC glGetnUniformui64vARB;
00507 PFNGLGETNUNIFORMUIVARBPROC glGetnUniformuivARB;
00508 PFNGLGETNUNIFORMUIVPROC glGetnUniformuiv;
00509 PFNGLGETOBJECTLABELEXTPROC glGetObjectLabelEXT;
00510 PFNGLGETOBJECTLABELPROC glGetObjectLabel;
00511 PFNGLGETOBJECTPTRLABELPROC glGetObjectPtrLabel;
00512 PFNGLGETPATHCOMMANDSNVPROC glGetPathCommandsNV;
00513 PFNGLGETPATHCOORDSNVPROC glGetPathCoordsNV;
00514 PFNGLGETPATHDASHARRAYNVPROC glGetPathDashArrayNV;
00515 PFNGLGETPATHLENGTHNVPROC glGetPathLengthNV;
00516 PFNGLGETPATHMETRICRANGENVPROC glGetPathMetricRangeNV;
00517 PFNGLGETPATHMETRICSNVPROC glGetPathMetricsNV;
00518 PFNGLGETPATHPARAMETERFVNVPROC glGetPathParameterfvNV;
00519 PFNGLGETPATHPARAMETERIVNVPROC glGetPathParameterivNV;
00520 PFNGLGETPATHSPACINGNVPROC glGetPathSpacingNV;
00521 PFNGLGETPERFCOUNTERINFOINTELPROC glGetPerfCounterInfoINTEL;
00522 PFNGLGETPERFMONITORCOUNTERDATAAMDPROC glGetPerfMonitorCounterDataAMD;
00523 PFNGLGETPERFMONITORCOUNTERINFOAMDPROC glGetPerfMonitorCounterInfoAMD;
00524 PFNGLGETPERFMONITORCOUNTERSAMDPROC glGetPerfMonitorCountersAMD;
00525 PFNGLGETPERFMONITORCOUNTERSTRINGAMDPROC glGetPerfMonitorCounterStringAMD;
00526 PFNGLGETPERFMONITORGROUPSAMDPROC glGetPerfMonitorGroupsAMD;
00527 PFNGLGETPERFMONITORGROUPSTRINGAMDPROC glGetPerfMonitorGroupStringAMD;
00528 PFNGLGETPERFQUERYDATAINTELPROC glGetPerfQueryDataINTEL;
00529 PFNGLGETPERFQUERYIDBYNAMEINTELPROC glGetPerfQueryIdByNameINTEL;
00530 PFNGLGETPERFQUERYINFOINTELPROC glGetPerfQueryInfoINTEL;
00531 PFNGLGETPOINTERINDEXEDVEXTPROC glGetPointerIndexedvEXT;
00532 PFNGLGETPOINTERIVEXTPROC glGetPointerivEXT;
00533 PFNGLGETPOINTERVPROC glGetPointerv;
00534 PFNGLGETPROGRAMBINARYPROC glGetProgramBinary;
00535 PFNGLGETPROGRAMINFOLOGPROC glGetProgramInfoLog;
00536 PFNGLGETPROGRAMINTERFACEIVPROC glGetProgramInterfaceiv;
00537 PFNGLGETPROGRAMIVPROC glGetProgramiv;
00538 PFNGLGETPROGRAMPIPELINEINFOLOGPROC glGetProgramPipelineInfoLog;
00539 PFNGLGETPROGRAMPIPELINEIVPROC glGetProgramPipelineiv;
00540 PFNGLGETPROGRAMRESOURCEFVNVPROC glGetProgramResourcefvNV;
00541 PFNGLGETPROGRAMRESOURCEINDEXPROC glGetProgramResourceIndex;
00542 PFNGLGETPROGRAMRESOURCEIVPROC glGetProgramResourceiv;
00543 PFNGLGETPROGRAMRESOURCELOCATIONINDEXPROC glGetProgramResourceLocationIndex;
00544 PFNGLGETPROGRAMRESOURCELOCATIONPROC glGetProgramResourceLocation;
00545 PFNGLGETPROGRAMRESOURCENAMEPROC glGetProgramResourceName;
00546 PFNGLGETPROGRAMSTAGEIVPROC glGetProgramStageiv;
00547 PFNGLGETQUERYBUFFEROBJECTI64VPROC glGetQueryBufferObjecti64v;
00548 PFNGLGETQUERYBUFFEROBJECTIVPROC glGetQueryBufferObjectiv;
00549 PFNGLGETQUERYBUFFEROBJECTUI64VPROC glGetQueryBufferObjectui64v;
00550 PFNGLGETQUERYBUFFEROBJECTUIVPROC glGetQueryBufferObjectuiv;
00551 PFNGLGETQUERYINDEXEDIVPROC glGetQueryIndexediv;
00552 PFNGLGETQUERYIVPROC glGetQueryiv;
00553 PFNGLGETQUERYOBJECTI64VPROC glGetQueryObjecti64v;
00554 PFNGLGETQUERYOBJECTIVPROC glGetQueryObjectiv;
00555 PFNGLGETQUERYOBJECTUI64VPROC glGetQueryObjectui64v;
00556 PFNGLGETQUERYOBJECTUIVPROC glGetQueryObjectuiv;
00557 PFNGLGETRENDERBUFFERPARAMETERIVPROC glGetRenderbufferParameteriv;
00558 PFNGLGETSAMPLERPARAMETERFVPROC glGetSamplerParameterfv;
00559 PFNGLGETSAMPLERPARAMETERIIVPROC glGetSamplerParameterIiv;
00560 PFNGLGETSAMPLERPARAMETERIUIVPROC glGetSamplerParameterIuiv;
00561 PFNGLGETSAMPLERPARAMETERIVPROC glGetSamplerParameteriv;
```



```
00562 PFNGLGETSHADERINFOLOGPROC glGetShaderInfoLog;
00563 PFNGLGETSHADERIVPROC glGetShaderiv;
00564 PFNGLGETSHADERPRECISIONFORMATPROC glGetShaderPrecisionFormat;
00565 PFNGLGETSHADERSOURCEPROC glGetShaderSource;
00566 PFNGLGETSTAGEINDEXNVPROC glGetStageIndexNV;
00567 PFNGLGETSTRINGIPROC glGetStringi;
00568 PFNGLGETSTRINGPROC glGetString;
00569 PFNGLGETSUBROUTINEINDEXPROC glGetSubroutineIndex;
00570 PFNGLGETSUBROUTINEUNIFORMLOCATIONPROC glGetSubroutineUniformLocation;
00571 PFNGLGETSYNCPROC glGetSynciv;
00572 PFNGLGETTEXIMAGEPROC glGetTexImage;
00573 PFNGLGETTEXLEVELPARAMETERFVPROC glGetTexLevelParameterfv;
00574 PFNGLGETTEXLEVELPARAMETERIVPROC glGetTexLevelParameteriv;
00575 PFNGLGETTEXPARAMETERFVPROC glGetTexParameterfv;
00576 PFNGLGETTEXPARAMETERIIVPROC glGetTexParameterIiv;
00577 PFNGLGETTEXPARAMETERIUIVPROC glGetTexParameterIuiv;
00578 PFNGLGETTEXPARAMETERIVPROC glGetTexParameteriv;
00579 PFNGLGETTEXTUREHANDLEARBPROC glGetTextureHandleARB;
00580 PFNGLGETTEXTUREHANDLENVPROC glGetTextureHandleNV;
00581 PFNGLGETTEXTUREIMAGEEXTPROC glGetTextureImageEXT;
00582 PFNGLGETTEXTUREIMAGEPROC glGetTextureImage;
00583 PFNGLGETTEXTURELEVELPARAMETERFVEXTPROC glGetTextureLevelParameterfvEXT;
00584 PFNGLGETTEXTURELEVELPARAMETERFVPROC glGetTextureLevelParameterfv;
00585 PFNGLGETTEXTURELEVELPARAMETERIIVEXTPROC glGetTextureLevelParameterivEXT;
00586 PFNGLGETTEXTURELEVELPARAMETERIVPROC glGetTextureLevelParameteriv;
00587 PFNGLGETTEXTUREPARAMETERFVEXTPROC glGetTextureParameterfvEXT;
00588 PFNGLGETTEXTUREPARAMETERFVPROC glGetTextureParameterfv;
00589 PFNGLGETTEXTUREPARAMETERIIVEXTPROC glGetTextureParameterIivEXT;
00590 PFNGLGETTEXTUREPARAMETERIIVPROC glGetTextureParameterIiv;
00591 PFNGLGETTEXTUREPARAMETERIUIVEXTPROC glGetTextureParameterIuivEXT;
00592 PFNGLGETTEXTUREPARAMETERIUIVPROC glGetTextureParameterIuiv;
00593 PFNGLGETTEXTUREPARAMETERIIVEXTPROC glGetTextureParameterivEXT;
00594 PFNGLGETTEXTUREPARAMETERIVPROC glGetTextureParameteriv;
00595 PFNGLGETTEXTURESAMPLERHANDLEARBPROC glGetTextureSamplerHandleARB;
00596 PFNGLGETTEXTURESAMPLERHANDLENVPROC glGetTextureSamplerHandleNV;
00597 PFNGLGETTEXTURESUBIMAGEPROC glGetTextureSubImage;
00598 PFNGLGETTRANSFORMFEEDBACKI64VPROC glGetTransformFeedbacki64_v;
00599 PFNGLGETTRANSFORMFEEDBACKIVPROC glGetTransformFeedbackiv;
00600 PFNGLGETTRANSFORMFEEDBACKI_VPROC glGetTransformFeedbacki_v;
00601 PFNGLGETTRANSFORMFEEDBACKVARYINGPROC glGetTransformFeedbackVarying;
00602 PFNGLGETUNIFORMBLOCKINDEXPROC glGetUniformBlockIndex;
00603 PFNGLGETUNIFORMDVPROC glGetUniformdv;
00604 PFNGLGETUNIFORMFVPROC glGetUniformfv;
00605 PFNGLGETUNIFORMI64VARBPROC glGetUniformi64vARB;
00606 PFNGLGETUNIFORMI64VNVPROC glGetUniformi64vNV;
00607 PFNGLGETUNIFORMINDICESPROC glGetUniformIndices;
00608 PFNGLGETUNIFORMIVPROC glGetUniformiv;
00609 PFNGLGETUNIFORMLOCATIONPROC glGetUniformLocation;
00610 PFNGLGETUNIFORMSUBROUTINEUIVPROC glGetUniformSubroutineuiv;
00611 PFNGLGETUNIFORMUI64VARBPROC glGetUniformui64vARB;
00612 PFNGLGETUNIFORMUI64VNVPROC glGetUniformui64vNV;
00613 PFNGLGETUNIFORMUIVPROC glGetUniformuiv;
00614 PFNGLGETVERTEXARRAYINDEXED64IVPROC glGetVertexArrayIndexed64iv;
00615 PFNGLGETVERTEXARRAYINDEXEDIVPROC glGetVertexArrayIndexediv;
00616 PFNGLGETVERTEXARRAYINTEGERI_VEXTPROC glGetVertexArrayIntegeri_vEXT;
00617 PFNGLGETVERTEXARRAYINTEGERVEXTPROC glGetVertexArrayIntegerivEXT;
00618 PFNGLGETVERTEXARRAYIVPROC glGetVertexArrayiv;
00619 PFNGLGETVERTEXARRAYPOINTERI_VEXTPROC glGetVertexArrayPointeri_vEXT;
00620 PFNGLGETVERTEXARRAYPOINTERVEXTPROC glGetVertexArrayPointerivEXT;
00621 PFNGLGETVERTEXATTRIBDVPROC glGetVertexAttribdv;
00622 PFNGLGETVERTEXATTRIBFVPROC glGetVertexAttribfv;
00623 PFNGLGETVERTEXATTRIBIIVPROC glGetVertexAttribIiv;
00624 PFNGLGETVERTEXATTRIBIUIVPROC glGetVertexAttribIuiv;
00625 PFNGLGETVERTEXATTRIBIVPROC glGetVertexAttribiv;
00626 PFNGLGETVERTEXATTRIBLDVPROC glGetVertexAttribLdv;
00627 PFNGLGETVERTEXATTRIBLI64VNVPROC glGetVertexAttribLi64vNV;
00628 PFNGLGETVERTEXATTRIBLUI64VARBPROC glGetVertexAttribLui64vARB;
00629 PFNGLGETVERTEXATTRIBLUI64VNVPROC glGetVertexAttribLui64vNV;
00630 PFNGLGETVERTEXATTRIBPOINTERVPROC glGetVertexAttribPointerv;
00631 PFNGLGETVKPROCADDRNVPROC glGetVkProcAddrNV;
00632 PFNGLHINTPROC glHint;
00633 PFNGLINDEXFORMATNVPROC glIndexFormatNV;
00634 PFNGLINSERTEVENTMARKEREXTPROC glInsertEventMarkerEXT;
00635 PFNGLINTERPOLATEPATHSNVPROC glInterpolatePathsNV;
00636 PFNGLINVALIDATEBUFFERDATAPROC glInvalidateBufferData;
00637 PFNGLINVALIDATEBUFFERSUBDATAPROC glInvalidateBufferSubData;
00638 PFNGLINVALIDATEFRAMEBUFFERPROC glInvalidateFramebuffer;
00639 PFNGLINVALIDATENAMEDFRAMEBUFFERDATAPROC glInvalidateNamedFramebufferData;
00640 PFNGLINVALIDATENAMEDFRAMEBUFFERSUBDATAPROC glInvalidateNamedFramebufferSubData;
00641 PFNGLINVALIDATESUBFRAMEBUFFERPROC glInvalidateSubFramebuffer;
00642 PFNGLINVALIDATETEXIMAGEPROC glInvalidateTexImage;
00643 PFNGLINVALIDATETEXSUBIMAGEPROC glInvalidateTexSubImage;
00644 PFNGLISBUFFERPROC glIsBuffer;
00645 PFNGLISBUFFERRESIDENTNVPROC glIsBufferResidentNV;
00646 PFNGLISCOMMANDLISTNVPROC glIsCommandListNV;
00647 PFNGLISENABLEDINDEXEDEXTPROC glIsEnabledIndexedEXT;
00648 PFNGLISENABLEDIPROC glIsEnabledi;
```

```
00649 PFNGLISENABLEDPROC glIsEnabled;
00650 PFNGLISFRAMEBUFFERPROC glIsFramebuffer;
00651 PFNGLISIMAGEHANDLERESIDENTARBPROC glIsImageHandleResidentARB;
00652 PFNGLISIMAGEHANDLERESIDENTNVPROC glIsImageHandleResidentNV;
00653 PFNGLISNAMEDBUFFERRESIDENTNVPROC glIsNamedBufferResidentNV;
00654 PFNGLISNAMEDSTRINGARBPROC glIsNamedStringARB;
00655 PFNGLISPATHNVPROC glIsPathNV;
00656 PFNGLISPOINTINFILLPATHNVPROC glIsPointInFillPathNV;
00657 PFNGLISPOINTINSTROKEPATHNVPROC glIsPointInStrokePathNV;
00658 PFNGLISPROGRAMPIPELINEPROC glIsProgramPipeline;
00659 PFNGLISPROGRAMPROC glIsProgram;
00660 PFNGLISQUERYPROC glIsQuery;
00661 PFNGLISRENDERBUFFERPROC glIsRenderbuffer;
00662 PFNGLISSAMPLERPROC glIsSampler;
00663 PFNGLISSHADERPROC glIsShader;
00664 PFNGLISSTATENVPROC glIsStateNV;
00665 PFNGLISSYNCPROC glIsSync;
00666 PFNGLISTEXTUREHANDLERESIDENTARBPROC glIsTextureHandleResidentARB;
00667 PFNGLISTEXTUREHANDLERESIDENTNVPROC glIsTextureHandleResidentNV;
00668 PFNGLISTEXTUREPROC glIsTexture;
00669 PFNGLISTRANSFORMFEEDBACKPROC glIsTransformFeedback;
00670 PFNGLISVERTEXARRAYPROC glIsVertexArray;
00671 PFNGLLABELOBJECTEXTPROC glLabelObjectEXT;
00672 PFNGLLINEWIDTHPROC glLineWidth;
00673 PFNGLLINKPROGRAMPROC glLinkProgram;
00674 PFNGLLISTDRAWCOMMANDSSTATESCLIENTNVPROC glListDrawCommandsStatesClientNV;
00675 PFNGLLOGICOPPROC glLogicOp;
00676 PFNGLMAKEBUFFERNONRESIDENTNVPROC glMakeBufferNonResidentNV;
00677 PFNGLMAKEBUFFERRESIDENTNVPROC glMakeBufferResidentNV;
00678 PFNGLMAKEIMAGEHANDLENONRESIDENTARBPROC glMakeImageHandleNonResidentARB;
00679 PFNGLMAKEIMAGEHANDLENONRESIDENTNVPROC glMakeImageHandleNonResidentNV;
00680 PFNGLMAKEIMAGEHANDLERESIDENTARBPROC glMakeImageHandleResidentARB;
00681 PFNGLMAKEIMAGEHANDLERESIDENTNVPROC glMakeImageHandleResidentNV;
00682 PFNGLMAKENAMEDBUFFERNONRESIDENTNVPROC glMakeNamedBufferNonResidentNV;
00683 PFNGLMAKENAMEDBUFFERRESIDENTNVPROC glMakeNamedBufferResidentNV;
00684 PFNGLMAKETEXTUREHANDLENONRESIDENTARBPROC glMakeTextureHandleNonResidentARB;
00685 PFNGLMAKETEXTUREHANDLENONRESIDENTNVPROC glMakeTextureHandleNonResidentNV;
00686 PFNGLMAKETEXTUREHANDLERESIDENTARBPROC glMakeTextureHandleResidentARB;
00687 PFNGLMAKETEXTUREHANDLERESIDENTNVPROC glMakeTextureHandleResidentNV;
00688 PFNGLMAPBUFFERPROC glMapBuffer;
00689 PFNGLMAPBUFFERRANGEPROC glMapBufferRange;
00690 PFNGLMAPNAMEDBUFFEREXTPROC glMapNamedBufferEXT;
00691 PFNGLMAPNAMEDBUFFERPROC glMapNamedBuffer;
00692 PFNGLMAPNAMEDBUFFERRANGEEXTPROC glMapNamedBufferRangeEXT;
00693 PFNGLMAPNAMEDBUFFERRANGEPROC glMapNamedBufferRange;
00694 PFNGLMATRIXFRUSTUMEXTPROC glMatrixFrustumEXT;
00695 PFNGLMATRIXLOAD3X2FNVPROC glMatrixLoad3x2fNV;
00696 PFNGLMATRIXLOAD3X3FNVPROC glMatrixLoad3x3fNV;
00697 PFNGLMATRIXLOADDEXTPROC glMatrixLoaddEXT;
00698 PFNGLMATRIXLOADFEXTPROC glMatrixLoadfEXT;
00699 PFNGLMATRIXLOADIDENTITYEXTPROC glMatrixLoadIdentityEXT;
00700 PFNGLMATRIXLOADTRANSPOSE3X3FNVPROC glMatrixLoadTranspose3x3fNV;
00701 PFNGLMATRIXLOADTRANSPPOSEEXTPROC glMatrixLoadTransposedEXT;
00702 PFNGLMATRIXLOADTRANSPOSEFEXTPROC glMatrixLoadTransposefEXT;
00703 PFNGLMATRIXMULT3X2FNVPROC glMatrixMult3x2fNV;
00704 PFNGLMATRIXMULT3X3FNVPROC glMatrixMult3x3fNV;
00705 PFNGLMATRIXMULTDEXTPROC glMatrixMultdEXT;
00706 PFNGLMATRIXMULTFEXTPROC glMatrixMultfEXT;
00707 PFNGLMATRIXMULTTRANSPOSE3X3FNVPROC glMatrixMultTranspose3x3fNV;
00708 PFNGLMATRIXMULTTRANSPPOSEEXTPROC glMatrixMultTransposedEXT;
00709 PFNGLMATRIXMULTTRANSPPOSEFEXTPROC glMatrixMultTransposefEXT;
00710 PFNGLMATRIXORTHOEXTPROC glMatrixOrthoEXT;
00711 PFNGLMATRIXPOPEXTPROC glMatrixPopEXT;
00712 PFNGLMATRIXPUSHEXTPROC glMatrixPushEXT;
00713 PFNGLMATRIXROTATEDEXTPROC glMatrixRotatedEXT;
00714 PFNGLMATRIXROTATEFEXTPROC glMatrixRotatefEXT;
00715 PFNGLMATRIXSCALEDEXTPROC glMatrixScaledEXT;
00716 PFNGLMATRIXSCALEFEXTPROC glMatrixScalefEXT;
00717 PFNGLMATRIXTRANSLATEDEXTPROC glMatrixTranslatedEXT;
00718 PFNGLMATRIXTRANSLATEFEXTPROC glMatrixTranslatefEXT;
00719 PFNGLMAXSHADERCOMPILERTHREADSARBPROC glMaxShaderCompilerThreadsARB;
00720 PFNGLMEMORYBARRIERBYREGIONPROC glMemoryBarrierByRegion;
00721 PFNGLMEMORYBARRIERPROC glMemoryBarrier;
00722 PFNGLMINSAMPLESHADINGARBPROC glMinSampleShadingARB;
00723 PFNGLMINSAMPLESHADINGPROC glMinSampleShading;
00724 PFNGLMULTIDRAWARRAYSINDIRECTBINDLESSCOUNTNVPROC glMultiDrawArraysIndirectBindlessCountNV;
00725 PFNGLMULTIDRAWARRAYSINDIRECTBINDLESSNVPROC glMultiDrawArraysIndirectBindlessNV;
00726 PFNGLMULTIDRAWARRAYSINDIRECTCOUNTARBPROC glMultiDrawArraysIndirectCountARB;
00727 PFNGLMULTIDRAWARRAYSINDIRECTPROC glMultiDrawArraysIndirect;
00728 PFNGLMULTIDRAWARRAYSPROC glMultiDrawArrays;
00729 PFNGLMULTIDRAWELEMENTSBASEVERTEXPROC glMultiDrawElementsBaseVertex;
00730 PFNGLMULTIDRAWELEMENTSINDIRECTBINDLESSCOUNTNVPROC glMultiDrawElementsIndirectBindlessCountNV;
00731 PFNGLMULTIDRAWELEMENTSINDIRECTBINDLESSNVPROC glMultiDrawElementsIndirectBindlessNV;
00732 PFNGLMULTIDRAWELEMENTSINDIRECTCOUNTARBPROC glMultiDrawElementsIndirectCountARB;
00733 PFNGLMULTIDRAWELEMENTSINDIRECTPROC glMultiDrawElementsIndirect;
00734 PFNGLMULTIDRAWELEMENTSPROC glMultiDrawElements;
00735 PFNGLMULTITEXBUFFEREXTPROC glMultiTexBufferEXT;
```

```

00736 PFNGLMULTITEXCOORDPOINTEREXTPROC glMultiTexCoordPointerEXT;
00737 PFNGLMULTITEXENVFEXTPROC glMultiTexEnvfEXT;
00738 PFNGLMULTITEXENVFVEXTPROC glMultiTexEnvfvEXT;
00739 PFNGLMULTITEXENVIEXTPROC glMultiTexEnviEXT;
00740 PFNGLMULTITEXENVIVEXTPROC glMultiTexEnvivEXT;
00741 PFNGLMULTITEXGENDEXTPROC glMultiTexGendEXT;
00742 PFNGLMULTITEXGENDVEXTPROC glMultiTexGendvEXT;
00743 PFNGLMULTITEXGENFEXTPROC glMultiTexGenfEXT;
00744 PFNGLMULTITEXGENFVEXTPROC glMultiTexGenfvEXT;
00745 PFNGLMULTITEXGENIEXTPROC glMultiTexGeniEXT;
00746 PFNGLMULTITEXGENIVEXTPROC glMultiTexGenivEXT;
00747 PFNGLMULTITEXIMAGE1DEXTPROC glMultiTexImage1DEXT;
00748 PFNGLMULTITEXIMAGE2DEXTPROC glMultiTexImage2DEXT;
00749 PFNGLMULTITEXIMAGE3DEXTPROC glMultiTexImage3DEXT;
00750 PFNGLMULTITEXPARAMETERFEXTPROC glMultiTexParameterfEXT;
00751 PFNGLMULTITEXPARAMETERFVEXTPROC glMultiTexParameterfvEXT;
00752 PFNGLMULTITEXPARAMETERIEXTPROC glMultiTexParameteriEXT;
00753 PFNGLMULTITEXPARAMETERIIVEXTPROC glMultiTexParameterIivEXT;
00754 PFNGLMULTITEXPARAMETERIUIVEXTPROC glMultiTexParameterIuivEXT;
00755 PFNGLMULTITEXPARAMETERIVEXTPROC glMultiTexParameterivEXT;
00756 PFNGLMULTITEXRENDERBUFFEREXTPROC glMultiTexRenderbufferEXT;
00757 PFNGLMULTITEXSUBIMAGE1DEXTPROC glMultiTexSubImage1DEXT;
00758 PFNGLMULTITEXSUBIMAGE2DEXTPROC glMultiTexSubImage2DEXT;
00759 PFNGLMULTITEXSUBIMAGE3DEXTPROC glMultiTexSubImage3DEXT;
00760 PFNGLNAMEDBUFFERDATAEXTPROC glNamedBufferDataEXT;
00761 PFNGLNAMEDBUFFERDATAAPROC glNamedBufferData;
00762 PFNGLNAMEDBUFFERPAGECOMMITMENTARBPROC glNamedBufferPageCommitmentARB;
00763 PFNGLNAMEDBUFFERPAGECOMMITMENTEXTPROC glNamedBufferPageCommitmentEXT;
00764 PFNGLNAMEDBUFFERSTORAGEEXTPROC glNamedBufferStorageEXT;
00765 PFNGLNAMEDBUFFERSTORAGEPROC glNamedBufferStorage;
00766 PFNGLNAMEDBUFFERSUBDATAEXTPROC glNamedBufferSubDataEXT;
00767 PFNGLNAMEDBUFFERSUBDATAAPROC glNamedBufferSubData;
00768 PFNGLNAMEDCOPYBUFFERSUBDATAEXTPROC glNamedCopyBufferSubDataEXT;
00769 PFNGLNAMEDFRAMEBUFFERDRAWBUFFERPROC glNamedFramebufferDrawBuffer;
00770 PFNGLNAMEDFRAMEBUFFERDRAWBUFFERSPROC glNamedFramebufferDrawBuffers;
00771 PFNGLNAMEDFRAMEBUFFERPARAMETERIEXTPROC glNamedFramebufferParameteriEXT;
00772 PFNGLNAMEDFRAMEBUFFERPARAMETERIPROC glNamedFramebufferParameteri;
00773 PFNGLNAMEDFRAMEBUFFERREADBUFFERPROC glNamedFramebufferReadBuffer;
00774 PFNGLNAMEDFRAMEBUFFERRENDERBUFFEREXTPROC glNamedFramebufferRenderbufferEXT;
00775 PFNGLNAMEDFRAMEBUFFERRENDERBUFFERPROC glNamedFramebufferRenderbuffer;
00776 PFNGLNAMEDFRAMEBUFFERSAMPLELOCATIONSFVARBPROC glNamedFramebufferSampleLocationsfvARB;
00777 PFNGLNAMEDFRAMEBUFFERSAMPLELOCATIONSFVNVPROC glNamedFramebufferSampleLocationsfvNV;
00778 PFNGLNAMEDFRAMEBUFFERTEXTURE1DEXTPROC glNamedFramebufferTexture1DEXT;
00779 PFNGLNAMEDFRAMEBUFFERTEXTURE2DEXTPROC glNamedFramebufferTexture2DEXT;
00780 PFNGLNAMEDFRAMEBUFFERTEXTURE3DEXTPROC glNamedFramebufferTexture3DEXT;
00781 PFNGLNAMEDFRAMEBUFFERTEXTUREEXTPROC glNamedFramebufferTextureEXT;
00782 PFNGLNAMEDFRAMEBUFFERTEXTUREFACEEXTPROC glNamedFramebufferTextureFaceEXT;
00783 PFNGLNAMEDFRAMEBUFFERTEXTURELAYEREXTPROC glNamedFramebufferTextureLayerEXT;
00784 PFNGLNAMEDFRAMEBUFFERTEXTURELAYERPROC glNamedFramebufferTextureLayer;
00785 PFNGLNAMEDFRAMEBUFFERTEXTUREPROC glNamedFramebufferTexture;
00786 PFNGLNAMEDPROGRAMLOCALPARAMETER4DEXTPROC glNamedProgramLocalParameter4dEXT;
00787 PFNGLNAMEDPROGRAMLOCALPARAMETER4DVEXTPROC glNamedProgramLocalParameter4dvEXT;
00788 PFNGLNAMEDPROGRAMLOCALPARAMETER4FEXTPROC glNamedProgramLocalParameter4fEXT;
00789 PFNGLNAMEDPROGRAMLOCALPARAMETER4FVEXTPROC glNamedProgramLocalParameter4fvEXT;
00790 PFNGLNAMEDPROGRAMLOCALPARAMETERI4IEXTPROC glNamedProgramLocalParameterI4iEXT;
00791 PFNGLNAMEDPROGRAMLOCALPARAMETERI4IVEXTPROC glNamedProgramLocalParameterI4ivEXT;
00792 PFNGLNAMEDPROGRAMLOCALPARAMETERI4UIEXTPROC glNamedProgramLocalParameterI4uiEXT;
00793 PFNGLNAMEDPROGRAMLOCALPARAMETERI4UIVEXTPROC glNamedProgramLocalParameterI4uivEXT;
00794 PFNGLNAMEDPROGRAMLOCALPARAMETERS4FVEXTPROC glNamedProgramLocalParameters4fvEXT;
00795 PFNGLNAMEDPROGRAMLOCALPARAMETERSI4IVEXTPROC glNamedProgramLocalParametersI4ivEXT;
00796 PFNGLNAMEDPROGRAMLOCALPARAMETERSI4UIVEXTPROC glNamedProgramLocalParametersI4uivEXT;
00797 PFNGLNAMEDPROGRAMSTRINGEXTPROC glNamedProgramStringEXT;
00798 PFNGLNAMEDRENDERBUFFERSTORAGEEXTPROC glNamedRenderbufferStorageEXT;
00799 PFNGLNAMEDRENDERBUFFERSTORAGEMULTISAMPLECOVERAGEEXTPROC
    glNamedRenderbufferStorageMultisampleCoverageEXT;
00800 PFNGLNAMEDRENDERBUFFERSTORAGEMULTISAMPLEEXTPROC glNamedRenderbufferStorageMultisampleEXT;
00801 PFNGLNAMEDRENDERBUFFERSTORAGEMULTISAMPLEPROC glNamedRenderbufferStorageMultisample;
00802 PFNGLNAMEDRENDERBUFFERSTORAGEPROC glNamedRenderbufferStorage;
00803 PFNGLNAMEDSTRINGARBPROC glNamedStringARB;
00804 PFNGLNORMALFORMATNVPROC glNormalFormatNV;
00805 PFNGLOBJCTLABELPROC glObjectLabel;
00806 PFNGLOBJCTPTRLABELPROC glObjectPtrLabel;
00807 PFNGLPATCHPARAMETERFVPROC glPatchParameterfv;
00808 PFNGLPATCHPARAMETERIPROC glPatchParameteri;
00809 PFNGLPATHCOMMANDSNVPROC glPathCommandsNV;
00810 PFNGLPATHCOORDSNVPROC glPathCoordsNV;
00811 PFNGLPATHCOVERDEPTHFUNCNVPROC glPathCoverDepthFuncNV;
00812 PFNGLPATHDASHARRAYNVPROC glPathDashArrayNV;
00813 PFNGLPATHGLYPHINDEXARRAYNVPROC glPathGlyphIndexArrayNV;
00814 PFNGLPATHGLYPHINDEXRANGENVPROC glPathGlyphIndexRangeNV;
00815 PFNGLPATHGLYPHRANGENVPROC glPathGlyphRangeNV;
00816 PFNGLPATHGLYPHSNVPROC glPathGlyphsNV;
00817 PFNGLPATHMEMORYGLYPHINDEXARRAYNVPROC glPathMemoryGlyphIndexArrayNV;
00818 PFNGLPATHPARAMETERFNVPROC glPathParameterfNV;
00819 PFNGLPATHPARAMETERFVNVPROC glPathParameterfvNV;
00820 PFNGLPATHPARAMETERINVPROC glPathParameteriNV;
00821 PFNGLPATHPARAMETERIVNVPROC glPathParameterivNV;

```

```
00822 PFNGLPATHSTENCILDEPTHOFFSETNVPROC glPathStencilDepthOffsetNV;
00823 PFNGLPATHSTENCILFUNCNVPROC glPathStencilFuncNV;
00824 PFNGLPATHSTRINGNVPROC glPathStringNV;
00825 PFNGLPATHSUBCOMMANDSNVPROC glPathSubCommandsNV;
00826 PFNGLPATHSUBCOORDSNVPROC glPathSubCoordsNV;
00827 PFNGLPAUSETRANSFORMFEEDBACKPROC glPauseTransformFeedback;
00828 PFNGLPIXELSTOREFPROC glPixelStoref;
00829 PFNGLPIXELSTOREIPROC glPixelStorei;
00830 PFNGLPOINTALONGPATHNVPROC glPointAlongPathNV;
00831 PFNGLPOINTPARAMETERFPROC glPointParameterf;
00832 PFNGLPOINTPARAMETERFVPROC glPointParameterfv;
00833 PFNGLPOINTPARAMETERIPROC glPointParameteri;
00834 PFNGLPOINTPARAMETERIVPROC glPointParameteriv;
00835 PFNGLPOINTSIZEPROC glPointSize;
00836 PFNGLPOLYGONMODEPROC glPolygonMode;
00837 PFNGLPOLYGONOFFSETCLAMPEXTPROC glPolygonOffsetClampEXT;
00838 PFNGLPOLYGONOFFSETPROC glPolygonOffset;
00839 PFNGLPOPDEBUGGROUPPROC glPopDebugGroup;
00840 PFNGLPOPGROUPMARKEREXTPROC glPopGroupMarkerEXT;
00841 PFNGLPRIMITIVEBOUNDINGBOXARBPROC glPrimitiveBoundingBoxARB;
00842 PFNGLPRIMITIVERESTARTINDEXPROC glPrimitiveRestartIndex;
00843 PFNGLPROGRAMBINARYPROC glProgramBinary;
00844 PFNGLPROGRAMPARAMETERIARBPROC glProgramParameteriARB;
00845 PFNGLPROGRAMPARAMETERIPROC glProgramParameteri;
00846 PFNGLPROGRAMPATHFRAGMENTINPUTGENNVPROC glProgramPathFragmentInputGenNV;
00847 PFNGLPROGRAMUNIFORM1DEXTPROC glProgramUniform1dEXT;
00848 PFNGLPROGRAMUNIFORM1DPROC glProgramUniform1d;
00849 PFNGLPROGRAMUNIFORM1DVECTPROC glProgramUniform1dvEXT;
00850 PFNGLPROGRAMUNIFORM1DVPROC glProgramUniform1dv;
00851 PFNGLPROGRAMUNIFORM1FEXTPROC glProgramUniform1fEXT;
00852 PFNGLPROGRAMUNIFORM1FPROC glProgramUniform1f;
00853 PFNGLPROGRAMUNIFORM1FVECTPROC glProgramUniform1fvEXT;
00854 PFNGLPROGRAMUNIFORM1FVPROC glProgramUniform1fv;
00855 PFNGLPROGRAMUNIFORM1I64ARBPROC glProgramUniform1i64ARB;
00856 PFNGLPROGRAMUNIFORM1I64NVPROC glProgramUniform1i64NV;
00857 PFNGLPROGRAMUNIFORM1I64VARBPROC glProgramUniform1i64vARB;
00858 PFNGLPROGRAMUNIFORM1I64VNVPROC glProgramUniform1i64vNV;
00859 PFNGLPROGRAMUNIFORM1IEXTPROC glProgramUniform1iEXT;
00860 PFNGLPROGRAMUNIFORM1IPROC glProgramUniform1i;
00861 PFNGLPROGRAMUNIFORM1IEXTPROC glProgramUniform1ivEXT;
00862 PFNGLPROGRAMUNIFORM1IVPROC glProgramUniform1iv;
00863 PFNGLPROGRAMUNIFORM1UI64ARBPROC glProgramUniform1ui64ARB;
00864 PFNGLPROGRAMUNIFORM1UI64NVPROC glProgramUniform1ui64NV;
00865 PFNGLPROGRAMUNIFORM1UI64VARBPROC glProgramUniform1ui64vARB;
00866 PFNGLPROGRAMUNIFORM1UI64VNVPROC glProgramUniform1ui64vNV;
00867 PFNGLPROGRAMUNIFORM1UIEXTPROC glProgramUniform1uiEXT;
00868 PFNGLPROGRAMUNIFORM1UIPROC glProgramUniform1ui;
00869 PFNGLPROGRAMUNIFORM1UIVEXTPROC glProgramUniform1uivEXT;
00870 PFNGLPROGRAMUNIFORM1UIVPROC glProgramUniform1uiv;
00871 PFNGLPROGRAMUNIFORM2DEXTPROC glProgramUniform2dEXT;
00872 PFNGLPROGRAMUNIFORM2DPROC glProgramUniform2d;
00873 PFNGLPROGRAMUNIFORM2DVECTPROC glProgramUniform2dvEXT;
00874 PFNGLPROGRAMUNIFORM2DVPROC glProgramUniform2dv;
00875 PFNGLPROGRAMUNIFORM2FEXTPROC glProgramUniform2fEXT;
00876 PFNGLPROGRAMUNIFORM2FPROC glProgramUniform2f;
00877 PFNGLPROGRAMUNIFORM2FVECTPROC glProgramUniform2fvEXT;
00878 PFNGLPROGRAMUNIFORM2FVPROC glProgramUniform2fv;
00879 PFNGLPROGRAMUNIFORM2I64ARBPROC glProgramUniform2i64ARB;
00880 PFNGLPROGRAMUNIFORM2I64NVPROC glProgramUniform2i64NV;
00881 PFNGLPROGRAMUNIFORM2I64VARBPROC glProgramUniform2i64vARB;
00882 PFNGLPROGRAMUNIFORM2I64VNVPROC glProgramUniform2i64vNV;
00883 PFNGLPROGRAMUNIFORM2IEXTPROC glProgramUniform2iEXT;
00884 PFNGLPROGRAMUNIFORM2IPROC glProgramUniform2i;
00885 PFNGLPROGRAMUNIFORM2IEXTPROC glProgramUniform2ivEXT;
00886 PFNGLPROGRAMUNIFORM2IVPROC glProgramUniform2iv;
00887 PFNGLPROGRAMUNIFORM2UI64ARBPROC glProgramUniform2ui64ARB;
00888 PFNGLPROGRAMUNIFORM2UI64NVPROC glProgramUniform2ui64NV;
00889 PFNGLPROGRAMUNIFORM2UI64VARBPROC glProgramUniform2ui64vARB;
00890 PFNGLPROGRAMUNIFORM2UI64VNVPROC glProgramUniform2ui64vNV;
00891 PFNGLPROGRAMUNIFORM2UIEXTPROC glProgramUniform2uiEXT;
00892 PFNGLPROGRAMUNIFORM2UIPROC glProgramUniform2ui;
00893 PFNGLPROGRAMUNIFORM2UIVEXTPROC glProgramUniform2uivEXT;
00894 PFNGLPROGRAMUNIFORM2UIVPROC glProgramUniform2uiv;
00895 PFNGLPROGRAMUNIFORM3DEXTPROC glProgramUniform3dEXT;
00896 PFNGLPROGRAMUNIFORM3DPROC glProgramUniform3d;
00897 PFNGLPROGRAMUNIFORM3DVECTPROC glProgramUniform3dvEXT;
00898 PFNGLPROGRAMUNIFORM3DVPROC glProgramUniform3dv;
00899 PFNGLPROGRAMUNIFORM3FEXTPROC glProgramUniform3fEXT;
00900 PFNGLPROGRAMUNIFORM3FPROC glProgramUniform3f;
00901 PFNGLPROGRAMUNIFORM3FVECTPROC glProgramUniform3fvEXT;
00902 PFNGLPROGRAMUNIFORM3FVPROC glProgramUniform3fv;
00903 PFNGLPROGRAMUNIFORM3I64ARBPROC glProgramUniform3i64ARB;
00904 PFNGLPROGRAMUNIFORM3I64NVPROC glProgramUniform3i64NV;
00905 PFNGLPROGRAMUNIFORM3I64VARBPROC glProgramUniform3i64vARB;
00906 PFNGLPROGRAMUNIFORM3I64VNVPROC glProgramUniform3i64vNV;
00907 PFNGLPROGRAMUNIFORM3IEXTPROC glProgramUniform3iEXT;
00908 PFNGLPROGRAMUNIFORM3IPROC glProgramUniform3i;
```

```
00909 PFNGLPROGRAMUNIFORM3IVEXTPROC glProgramUniform3ivEXT;
00910 PFNGLPROGRAMUNIFORM3IVPROC glProgramUniform3iv;
00911 PFNGLPROGRAMUNIFORM3UI64ARBPROC glProgramUniform3ui64ARB;
00912 PFNGLPROGRAMUNIFORM3UI64NVPROC glProgramUniform3ui64NV;
00913 PFNGLPROGRAMUNIFORM3UI64VARBPROC glProgramUniform3ui64vARB;
00914 PFNGLPROGRAMUNIFORM3UI64VNVPROC glProgramUniform3ui64vNV;
00915 PFNGLPROGRAMUNIFORM3UIEXTPROC glProgramUniform3uiEXT;
00916 PFNGLPROGRAMUNIFORM3UIPROC glProgramUniform3ui;
00917 PFNGLPROGRAMUNIFORM3UIVEXTPROC glProgramUniform3uivEXT;
00918 PFNGLPROGRAMUNIFORM3UIVPROC glProgramUniform3uiv;
00919 PFNGLPROGRAMUNIFORM4DEXTPROC glProgramUniform4dEXT;
00920 PFNGLPROGRAMUNIFORM4DPROC glProgramUniform4d;
00921 PFNGLPROGRAMUNIFORM4DVECTPROC glProgramUniform4dvEXT;
00922 PFNGLPROGRAMUNIFORM4DVPROC glProgramUniform4dv;
00923 PFNGLPROGRAMUNIFORM4FEXTPROC glProgramUniform4fEXT;
00924 PFNGLPROGRAMUNIFORM4FPROC glProgramUniform4f;
00925 PFNGLPROGRAMUNIFORM4FVECTPROC glProgramUniform4fvEXT;
00926 PFNGLPROGRAMUNIFORM4FVPROC glProgramUniform4fv;
00927 PFNGLPROGRAMUNIFORM4I64ARBPROC glProgramUniform4i64ARB;
00928 PFNGLPROGRAMUNIFORM4I64NVPROC glProgramUniform4i64NV;
00929 PFNGLPROGRAMUNIFORM4I64VARBPROC glProgramUniform4i64vARB;
00930 PFNGLPROGRAMUNIFORM4I64VNVPROC glProgramUniform4i64vNV;
00931 PFNGLPROGRAMUNIFORM4IEXTPROC glProgramUniform4iEXT;
00932 PFNGLPROGRAMUNIFORM4IPROC glProgramUniform4i;
00933 PFNGLPROGRAMUNIFORM4IVEXTPROC glProgramUniform4ivEXT;
00934 PFNGLPROGRAMUNIFORM4IVPROC glProgramUniform4iv;
00935 PFNGLPROGRAMUNIFORM4UI64ARBPROC glProgramUniform4ui64ARB;
00936 PFNGLPROGRAMUNIFORM4UI64NVPROC glProgramUniform4ui64NV;
00937 PFNGLPROGRAMUNIFORM4UI64VARBPROC glProgramUniform4ui64vARB;
00938 PFNGLPROGRAMUNIFORM4UI64VNVPROC glProgramUniform4ui64vNV;
00939 PFNGLPROGRAMUNIFORM4UIEXTPROC glProgramUniform4uiEXT;
00940 PFNGLPROGRAMUNIFORM4UIPROC glProgramUniform4ui;
00941 PFNGLPROGRAMUNIFORM4UIVEXTPROC glProgramUniform4uivEXT;
00942 PFNGLPROGRAMUNIFORM4UIVPROC glProgramUniform4uiv;
00943 PFNGLPROGRAMUNIFORMHANDLEUI64ARBPROC glProgramUniformHandleui64ARB;
00944 PFNGLPROGRAMUNIFORMHANDLEUI64NVPROC glProgramUniformHandleui64NV;
00945 PFNGLPROGRAMUNIFORMHANDLEUI64VARBPROC glProgramUniformHandleui64vARB;
00946 PFNGLPROGRAMUNIFORMHANDLEUI64VNVPROC glProgramUniformHandleui64vNV;
00947 PFNGLPROGRAMUNIFORMMATRIX2DVEXTPROC glProgramUniformMatrix2dvEXT;
00948 PFNGLPROGRAMUNIFORMMATRIX2DVPROC glProgramUniformMatrix2dv;
00949 PFNGLPROGRAMUNIFORMMATRIX2FVECTPROC glProgramUniformMatrix2fvEXT;
00950 PFNGLPROGRAMUNIFORMMATRIX2FVPROC glProgramUniformMatrix2fv;
00951 PFNGLPROGRAMUNIFORMMATRIX2X3DVEXTPROC glProgramUniformMatrix2x3dvEXT;
00952 PFNGLPROGRAMUNIFORMMATRIX2X3DVPROC glProgramUniformMatrix2x3dv;
00953 PFNGLPROGRAMUNIFORMMATRIX2X3FVECTPROC glProgramUniformMatrix2x3fvEXT;
00954 PFNGLPROGRAMUNIFORMMATRIX2X3FVPROC glProgramUniformMatrix2x3fv;
00955 PFNGLPROGRAMUNIFORMMATRIX2X4DVEXTPROC glProgramUniformMatrix2x4dvEXT;
00956 PFNGLPROGRAMUNIFORMMATRIX2X4DVPROC glProgramUniformMatrix2x4dv;
00957 PFNGLPROGRAMUNIFORMMATRIX2X4FVECTPROC glProgramUniformMatrix2x4fvEXT;
00958 PFNGLPROGRAMUNIFORMMATRIX2X4FVPROC glProgramUniformMatrix2x4fv;
00959 PFNGLPROGRAMUNIFORMMATRIX3DVEXTPROC glProgramUniformMatrix3dvEXT;
00960 PFNGLPROGRAMUNIFORMMATRIX3DVPROC glProgramUniformMatrix3dv;
00961 PFNGLPROGRAMUNIFORMMATRIX3FVECTPROC glProgramUniformMatrix3fvEXT;
00962 PFNGLPROGRAMUNIFORMMATRIX3FVPROC glProgramUniformMatrix3fv;
00963 PFNGLPROGRAMUNIFORMMATRIX3X2DVEXTPROC glProgramUniformMatrix3x2dvEXT;
00964 PFNGLPROGRAMUNIFORMMATRIX3X2DVPROC glProgramUniformMatrix3x2dv;
00965 PFNGLPROGRAMUNIFORMMATRIX3X2FVECTPROC glProgramUniformMatrix3x2fvEXT;
00966 PFNGLPROGRAMUNIFORMMATRIX3X2FVPROC glProgramUniformMatrix3x2fv;
00967 PFNGLPROGRAMUNIFORMMATRIX3X4DVEXTPROC glProgramUniformMatrix3x4dvEXT;
00968 PFNGLPROGRAMUNIFORMMATRIX3X4DVPROC glProgramUniformMatrix3x4dv;
00969 PFNGLPROGRAMUNIFORMMATRIX3X4FVECTPROC glProgramUniformMatrix3x4fvEXT;
00970 PFNGLPROGRAMUNIFORMMATRIX3X4FVPROC glProgramUniformMatrix3x4fv;
00971 PFNGLPROGRAMUNIFORMMATRIX4DVEXTPROC glProgramUniformMatrix4dvEXT;
00972 PFNGLPROGRAMUNIFORMMATRIX4DVPROC glProgramUniformMatrix4dv;
00973 PFNGLPROGRAMUNIFORMMATRIX4FVECTPROC glProgramUniformMatrix4fvEXT;
00974 PFNGLPROGRAMUNIFORMMATRIX4FVPROC glProgramUniformMatrix4fv;
00975 PFNGLPROGRAMUNIFORMMATRIX4X2DVEXTPROC glProgramUniformMatrix4x2dvEXT;
00976 PFNGLPROGRAMUNIFORMMATRIX4X2DVPROC glProgramUniformMatrix4x2dv;
00977 PFNGLPROGRAMUNIFORMMATRIX4X2FVECTPROC glProgramUniformMatrix4x2fvEXT;
00978 PFNGLPROGRAMUNIFORMMATRIX4X2FVPROC glProgramUniformMatrix4x2fv;
00979 PFNGLPROGRAMUNIFORMMATRIX4X3DVEXTPROC glProgramUniformMatrix4x3dvEXT;
00980 PFNGLPROGRAMUNIFORMMATRIX4X3DVPROC glProgramUniformMatrix4x3dv;
00981 PFNGLPROGRAMUNIFORMMATRIX4X3FVECTPROC glProgramUniformMatrix4x3fvEXT;
00982 PFNGLPROGRAMUNIFORMMATRIX4X3FVPROC glProgramUniformMatrix4x3fv;
00983 PFNGLPROGRAMUNIFORMUI64NVPROC glProgramUniformui64NV;
00984 PFNGLPROGRAMUNIFORMUI64VNVPROC glProgramUniformui64vNV;
00985 PFNGLPROVOKINGVERTEXPROC glProvokingVertex;
00986 PFNGLPUSHCLIENTATTRIBDEFAULTTEXTPROC glPushClientAttribDefaultEXT;
00987 PFNGLPUSHDEBUGGROUPPROC glPushDebugGroup;
00988 PFNGLPUSHGROUPMARKEREXTPROC glPushGroupMarkerEXT;
00989 PFNGLQUERYCOUNTERPROC glQueryCounter;
00990 PFNGLRASTERSAMPLESEXTPROC glRasterSamplesEXT;
00991 PFNGLREADBUFFERPROC glReadBuffer;
00992 PFNGLREADNPIXELSARBPROC glReadnPixelsARB;
00993 PFNGLREADNPIXELSPROC glReadnPixels;
00994 PFNGLREADPIXELSPROC glReadPixels;
00995 PFNGLRELEASESHADERCOMPILERPROC glReleaseShaderCompiler;
```

```
00996 PFNGLRENDERBUFFERSTORAGEMULTISAMPLECOVERAGENVPROC glRenderbufferStorageMultisampleCoverageNV;
00997 PFNGLRENDERBUFFERSTORAGEMULTISAMPLEPROC glRenderbufferStorageMultisample;
00998 PFNGLRENDERBUFFERSTORAGEPROC glRenderbufferStorage;
00999 PFNGLRESOLVEDEPTHVALUESNVPROC glResolveDepthValuesNV;
01000 PFNGLRESUMETRANSFORMFEEDBACKPROC glResumeTransformFeedback;
01001 PFNGLSAMPLECOVERAGEPROC glSampleCoverage;
01002 PFNGLSAMPLEMASKIPROC glSampleMaski;
01003 PFNGLSAMPLERPARAMETERFPROC glSamplerParameterf;
01004 PFNGLSAMPLERPARAMETERFVPROC glSamplerParameterfv;
01005 PFNGLSAMPLERPARAMETERIIVPROC glSamplerParameterIiv;
01006 PFNGLSAMPLERPARAMETERIPROC glSamplerParameteri;
01007 PFNGLSAMPLERPARAMETERIUIVPROC glSamplerParameterIuiv;
01008 PFNGLSAMPLERPARAMETERIVPROC glSamplerParameteriv;
01009 PFNGLSCISSORARRAYVPROC glScissorArrayv;
01010 PFNGLSCISSORINDEXEDPROC glScissorIndexed;
01011 PFNGLSCISSORINDEXEDVPROC glScissorIndexedv;
01012 PFNGLSCISSORPROC glScissor;
01013 PFNGLSECONDARYCOLORFORMATNVPROC glSecondaryColorFormatNV;
01014 PFNGLSELECTPERFMONITORCOUNTERSAMDPROC glSelectPerfMonitorCountersAMD;
01015 PFNGLSHADERBINARYPROC glShaderBinary;
01016 PFNGLSHADERSOURCEPROC glShaderSource;
01017 PFNGLSHADERSTORAGEBLOCKBINDINGPROC glShaderStorageBlockBinding;
01018 PFNGLSIGNALVKFENCENVPROC glSignalVkFenceNV;
01019 PFNGLSIGNALVKSEMAPHORENVPROC glSignalVkSemaphoreNV;
01020 PFNGLSPECIALIZESHADERARBPROC glSpecializeShaderARB;
01021 PFNGLSTATECAPTURENVPROC glStateCaptureNV;
01022 PFNGLSTENCILFILLPATHINSTANCEDNVPROC glStencilFillPathInstancedNV;
01023 PFNGLSTENCILFILLPATHNVPROC glStencilFillPathNV;
01024 PFNGLSTENCILFUNCPROC glStencilFunc;
01025 PFNGLSTENCILFUNCSEPARATEPROC glStencilFuncSeparate;
01026 PFNGLSTENCILMASKPROC glStencilMask;
01027 PFNGLSTENCILMASKSEPARATEPROC glStencilMaskSeparate;
01028 PFNGLSTENCILOPPROC glStencilOp;
01029 PFNGLSTENCILOPSEPARATEPROC glStencilOpSeparate;
01030 PFNGLSTENCILSTROKEPATHINSTANCEDNVPROC glStencilStrokePathInstancedNV;
01031 PFNGLSTENCILSTROKEPATHNVPROC glStencilStrokePathNV;
01032 PFNGLSTENCILTHENCOVERFILLPATHINSTANCEDNVPROC glStencilThenCoverFillPathInstancedNV;
01033 PFNGLSTENCILTHENCOVERFILLPATHNVPROC glStencilThenCoverFillPathNV;
01034 PFNGLSTENCILTHENCOVERSTROKEPATHINSTANCEDNVPROC glStencilThenCoverStrokePathInstancedNV;
01035 PFNGLSTENCILTHENCOVERSTROKEPATHNVPROC glStencilThenCoverStrokePathNV;
01036 PFNGLSUBPIXELPRECISIONBIASNVPROC glSubpixelPrecisionBiasNV;
01037 PFNGLTEXBUFFERARBPROC glTexBufferARB;
01038 PFNGLTEXBUFFERPROC glTexBuffer;
01039 PFNGLTEXBUFFERRANGEPROC glTexBufferRange;
01040 PFNGLTEXCOORDFORMATNVPROC glTexCoordFormatNV;
01041 PFNGLTEXIMAGE1DPROC glTexImage1D;
01042 PFNGLTEXIMAGE2DMULTISAMPLEPROC glTexImage2DMultisample;
01043 PFNGLTEXIMAGE2DPROC glTexImage2D;
01044 PFNGLTEXIMAGE3DMULTISAMPLEPROC glTexImage3DMultisample;
01045 PFNGLTEXIMAGE3DPROC glTexImage3D;
01046 PFNGLTEXPAGECOMMITMENTARBPROC glTexPageCommitmentARB;
01047 PFNGLTEXPARAMETERFPROC glTexParameterf;
01048 PFNGLTEXPARAMETERFVPROC glTexParameterfv;
01049 PFNGLTEXPARAMETERIIVPROC glTexParameterIiv;
01050 PFNGLTEXPARAMETERIPROC glTexParameterI;
01051 PFNGLTEXPARAMETERIUIVPROC glTexParameterIuiv;
01052 PFNGLTEXPARAMETERIVPROC glTexParameteriv;
01053 PFNGLTEXSTORAGE1DPROC glTexStorage1D;
01054 PFNGLTEXSTORAGE2DMULTISAMPLEPROC glTexStorage2DMultisample;
01055 PFNGLTEXSTORAGE2DPROC glTexStorage2D;
01056 PFNGLTEXSTORAGE3DMULTISAMPLEPROC glTexStorage3DMultisample;
01057 PFNGLTEXSTORAGE3DPROC glTexStorage3D;
01058 PFNGLTEXSUBIMAGE1DPROC glTexSubImage1D;
01059 PFNGLTEXSUBIMAGE2DPROC glTexSubImage2D;
01060 PFNGLTEXSUBIMAGE3DPROC glTexSubImage3D;
01061 PFNGLTEXTUREBARRIERNVPROC glTextureBarrierNV;
01062 PFNGLTEXTUREBARRIERPROC glTextureBarrier;
01063 PFNGLTEXTUREBUFFEREXTPROC glTextureBufferEXT;
01064 PFNGLTEXTUREBUFFERPROC glTextureBuffer;
01065 PFNGLTEXTUREBUFFERRANGEEXTPROC glTextureBufferRangeEXT;
01066 PFNGLTEXTUREBUFFERRANGEPROC glTextureBufferRange;
01067 PFNGLTEXTUREIMAGE1DEXTPROC glTextureImage1DEXT;
01068 PFNGLTEXTUREIMAGE2DEXTPROC glTextureImage2DEXT;
01069 PFNGLTEXTUREIMAGE3DEXTPROC glTextureImage3DEXT;
01070 PFNGLTEXTUREPAGECOMMITMENTEXTPROC glTexturePageCommitmentEXT;
01071 PFNGLTEXTUREPARAMETERFEXTPROC glTextureParameterfEXT;
01072 PFNGLTEXTUREPARAMETERFVEXTPROC glTextureParameterfvEXT;
01073 PFNGLTEXTUREPARAMETERFVPROC glTextureParameterfv;
01074 PFNGLTEXTUREPARAMETERIEXTPROC glTextureParameteriEXT;
01075 PFNGLTEXTUREPARAMETERIIVEXTPROC glTextureParameterIivEXT;
01076 PFNGLTEXTUREPARAMETERIUIVEXTPROC glTextureParameterIuivEXT;
01077 PFNGLTEXTUREPARAMETERIIVPROC glTextureParameterIiv;
01078 PFNGLTEXTUREPARAMETERIPROC glTextureParameteri;
01079 PFNGLTEXTUREPARAMETERIUIVEXTPROC glTextureParameterIuivEXT;
01080 PFNGLTEXTUREPARAMETERIUIVPROC glTextureParameterIuiv;
01081 PFNGLTEXTUREPARAMETERIVEXTPROC glTextureParameterivEXT;
01082 PFNGLTEXTUREPARAMETERIVPROC glTextureParameteriv;
```

```
01083 PFNGLTEXTURERENDERBUFFEREXTPROC glTextureRenderbufferEXT;
01084 PFNGLTEXTURESTORAGE1DEXTPROC glTextureStorage1DEXT;
01085 PFNGLTEXTURESTORAGE1DPROC glTextureStorage1D;
01086 PFNGLTEXTURESTORAGE2DEXTPROC glTextureStorage2DEXT;
01087 PFNGLTEXTURESTORAGE2DMULTISAMPLEEXTPROC glTextureStorage2DMultisampleEXT;
01088 PFNGLTEXTURESTORAGE2DMULTISAMPLEPROC glTextureStorage2DMultisample;
01089 PFNGLTEXTURESTORAGE2DPROC glTextureStorage2D;
01090 PFNGLTEXTURESTORAGE3DEXTPROC glTextureStorage3DEXT;
01091 PFNGLTEXTURESTORAGE3DMULTISAMPLEEXTPROC glTextureStorage3DMultisampleEXT;
01092 PFNGLTEXTURESTORAGE3DMULTISAMPLEPROC glTextureStorage3DMultisample;
01093 PFNGLTEXTURESTORAGE3DPROC glTextureStorage3D;
01094 PFNGLTEXTURESUBIMAGE1DEXTPROC glTextureSubImage1DEXT;
01095 PFNGLTEXTURESUBIMAGE1DPROC glTextureSubImage1D;
01096 PFNGLTEXTURESUBIMAGE2DEXTPROC glTextureSubImage2DEXT;
01097 PFNGLTEXTURESUBIMAGE2DPROC glTextureSubImage2D;
01098 PFNGLTEXTURESUBIMAGE3DEXTPROC glTextureSubImage3DEXT;
01099 PFNGLTEXTURESUBIMAGE3DPROC glTextureSubImage3D;
01100 PFNGLTEXTUREVIEWPROC glTextureView;
01101 PFNGLTRANSFORMFEEDBACKBUFFERBASEPROC glTransformFeedbackBufferBase;
01102 PFNGLTRANSFORMFEEDBACKBUFFERRANGEPROC glTransformFeedbackBufferRange;
01103 PFNGLTRANSFORMFEEDBACKVARYINGSPROC glTransformFeedbackVaryings;
01104 PFNGLTRANSFORMPATHNVPROC glTransformPathNV;
01105 PFNGLUNIFORM1DPROC glUniform1d;
01106 PFNGLUNIFORM1DVPROC glUniform1dv;
01107 PFNGLUNIFORM1FPROC glUniform1f;
01108 PFNGLUNIFORM1FVPROC glUniform1fv;
01109 PFNGLUNIFORM1I64ARBPROC glUniform1i64ARB;
01110 PFNGLUNIFORM1I64NVPROC glUniform1i64NV;
01111 PFNGLUNIFORM1I64VARBPROC glUniform1i64vARB;
01112 PFNGLUNIFORM1I64VNVPROC glUniform1i64vNV;
01113 PFNGLUNIFORM1IPROC glUniform1i;
01114 PFNGLUNIFORM1IVPROC glUniform1iv;
01115 PFNGLUNIFORM1UI64ARBPROC glUniform1ui64ARB;
01116 PFNGLUNIFORM1UI64NVPROC glUniform1ui64NV;
01117 PFNGLUNIFORM1UI64VARBPROC glUniform1ui64vARB;
01118 PFNGLUNIFORM1UI64VNVPROC glUniform1ui64vNV;
01119 PFNGLUNIFORM1UIPROC glUniform1ui;
01120 PFNGLUNIFORM1UIVPROC glUniform1uiv;
01121 PFNGLUNIFORM2DPROC glUniform2d;
01122 PFNGLUNIFORM2DVPROC glUniform2dv;
01123 PFNGLUNIFORM2FPROC glUniform2f;
01124 PFNGLUNIFORM2FVPROC glUniform2fv;
01125 PFNGLUNIFORM2I64ARBPROC glUniform2i64ARB;
01126 PFNGLUNIFORM2I64NVPROC glUniform2i64NV;
01127 PFNGLUNIFORM2I64VARBPROC glUniform2i64vARB;
01128 PFNGLUNIFORM2I64VNVPROC glUniform2i64vNV;
01129 PFNGLUNIFORM2IPROC glUniform2i;
01130 PFNGLUNIFORM2IVPROC glUniform2iv;
01131 PFNGLUNIFORM2UI64ARBPROC glUniform2ui64ARB;
01132 PFNGLUNIFORM2UI64NVPROC glUniform2ui64NV;
01133 PFNGLUNIFORM2UI64VARBPROC glUniform2ui64vARB;
01134 PFNGLUNIFORM2UI64VNVPROC glUniform2ui64vNV;
01135 PFNGLUNIFORM2UIPROC glUniform2ui;
01136 PFNGLUNIFORM2UIVPROC glUniform2uiv;
01137 PFNGLUNIFORM3DPROC glUniform3d;
01138 PFNGLUNIFORM3DVPROC glUniform3dv;
01139 PFNGLUNIFORM3FPROC glUniform3f;
01140 PFNGLUNIFORM3FVPROC glUniform3fv;
01141 PFNGLUNIFORM3I64ARBPROC glUniform3i64ARB;
01142 PFNGLUNIFORM3I64NVPROC glUniform3i64NV;
01143 PFNGLUNIFORM3I64VARBPROC glUniform3i64vARB;
01144 PFNGLUNIFORM3I64VNVPROC glUniform3i64vNV;
01145 PFNGLUNIFORM3IPROC glUniform3i;
01146 PFNGLUNIFORM3IVPROC glUniform3iv;
01147 PFNGLUNIFORM3UI64ARBPROC glUniform3ui64ARB;
01148 PFNGLUNIFORM3UI64NVPROC glUniform3ui64NV;
01149 PFNGLUNIFORM3UI64VARBPROC glUniform3ui64vARB;
01150 PFNGLUNIFORM3UI64VNVPROC glUniform3ui64vNV;
01151 PFNGLUNIFORM3UIPROC glUniform3ui;
01152 PFNGLUNIFORM3UIVPROC glUniform3uiv;
01153 PFNGLUNIFORM4DPROC glUniform4d;
01154 PFNGLUNIFORM4DVPROC glUniform4dv;
01155 PFNGLUNIFORM4FPROC glUniform4f;
01156 PFNGLUNIFORM4FVPROC glUniform4fv;
01157 PFNGLUNIFORM4I64ARBPROC glUniform4i64ARB;
01158 PFNGLUNIFORM4I64NVPROC glUniform4i64NV;
01159 PFNGLUNIFORM4I64VARBPROC glUniform4i64vARB;
01160 PFNGLUNIFORM4I64VNVPROC glUniform4i64vNV;
01161 PFNGLUNIFORM4IPROC glUniform4i;
01162 PFNGLUNIFORM4IVPROC glUniform4iv;
01163 PFNGLUNIFORM4UI64ARBPROC glUniform4ui64ARB;
01164 PFNGLUNIFORM4UI64NVPROC glUniform4ui64NV;
01165 PFNGLUNIFORM4UI64VARBPROC glUniform4ui64vARB;
01166 PFNGLUNIFORM4UI64VNVPROC glUniform4ui64vNV;
01167 PFNGLUNIFORM4UIPROC glUniform4ui;
01168 PFNGLUNIFORM4UIVPROC glUniform4uiv;
01169 PFNGLUNIFORMBLOCKBINDINGPROC glUniformBlockBinding;
```

```
01170 PFNGLUNIFORMHANDLEUI64ARBPROC glUniformHandleui64ARB;
01171 PFNGLUNIFORMHANDLEUI64NVPROC glUniformHandleui64NV;
01172 PFNGLUNIFORMHANDLEUI64VARBPROC glUniformHandleui64vARB;
01173 PFNGLUNIFORMHANDLEUI64VNVPROC glUniformHandleui64vNV;
01174 PFNGLUNIFORMMATRIX2DVPROC glUniformMatrix2dv;
01175 PFNGLUNIFORMMATRIX2FVPROC glUniformMatrix2fv;
01176 PFNGLUNIFORMMATRIX2X3DVPROC glUniformMatrix2x3dv;
01177 PFNGLUNIFORMMATRIX2X3FVPROC glUniformMatrix2x3fv;
01178 PFNGLUNIFORMMATRIX2X4DVPROC glUniformMatrix2x4dv;
01179 PFNGLUNIFORMMATRIX2X4FVPROC glUniformMatrix2x4fv;
01180 PFNGLUNIFORMMATRIX3DVPROC glUniformMatrix3dv;
01181 PFNGLUNIFORMMATRIX3FVPROC glUniformMatrix3fv;
01182 PFNGLUNIFORMMATRIX3X2DVPROC glUniformMatrix3x2dv;
01183 PFNGLUNIFORMMATRIX3X2FVPROC glUniformMatrix3x2fv;
01184 PFNGLUNIFORMMATRIX3X4DVPROC glUniformMatrix3x4dv;
01185 PFNGLUNIFORMMATRIX3X4FVPROC glUniformMatrix3x4fv;
01186 PFNGLUNIFORMMATRIX4DVPROC glUniformMatrix4dv;
01187 PFNGLUNIFORMMATRIX4FVPROC glUniformMatrix4fv;
01188 PFNGLUNIFORMMATRIX4X2DVPROC glUniformMatrix4x2dv;
01189 PFNGLUNIFORMMATRIX4X2FVPROC glUniformMatrix4x2fv;
01190 PFNGLUNIFORMMATRIX4X3DVPROC glUniformMatrix4x3dv;
01191 PFNGLUNIFORMMATRIX4X3FVPROC glUniformMatrix4x3fv;
01192 PFNGLUNIFORMSUBROUTINESUIVPROC glUniformSubroutinesuiv;
01193 PFNGLUNIFORMUI64NVPROC glUniformui64NV;
01194 PFNGLUNIFORMUI64VNVPROC glUniformui64vNV;
01195 PFNGLUNMAPBUFFERPROC glUnmapBuffer;
01196 PFNGLUNMAPNAMEDBUFFEREXTPROC glUnmapNamedBufferEXT;
01197 PFNGLUNMAPNAMEDBUFFERPROC glUnmapNamedBuffer;
01198 PFNGLUSEPROGRAMPROC glUseProgram;
01199 PFNGLUSEPROGRAMSTAGESPROC glUseProgramStages;
01200 PFNGLUSESHADERPROGRAMEXTPROC glUseProgramEXT;
01201 PFNGLVALIDATEPROGRAMPIPELINEPROC glValidateProgramPipeline;
01202 PFNGLVALIDATEPROGRAMPROC glValidateProgram;
01203 PFNGLVERTEXARRAYATTRIBBINDINGPROC glVertexArrayAttribBinding;
01204 PFNGLVERTEXARRAYATTRIBFORMATPROC glVertexArrayAttribFormat;
01205 PFNGLVERTEXARRAYATTRIBIFORMATPROC glVertexArrayAttribIFormat;
01206 PFNGLVERTEXARRAYATTRIBLFORMATPROC glVertexArrayAttribLFormat;
01207 PFNGLVERTEXARRAYBINDINGDIVISORPROC glVertexArrayBindingDivisor;
01208 PFNGLVERTEXARRAYBINDVERTEXBUFFEREXTPROC glVertexArrayBindVertexBufferEXT;
01209 PFNGLVERTEXARRAYCOLOROFFSETEXTPROC glVertexArrayColorOffsetEXT;
01210 PFNGLVERTEXARRAYEDGEFLAGOFFSETEXTPROC glVertexArrayEdgeFlagOffsetEXT;
01211 PFNGLVERTEXARRAYELEMENTBUFFERPROC glVertexArrayElementBuffer;
01212 PFNGLVERTEXARRAYFOGCOORDOFFSETEXTPROC glVertexArrayFogCoordOffsetEXT;
01213 PFNGLVERTEXARRAYINDEXOFFSETEXTPROC glVertexArrayIndexOffsetEXT;
01214 PFNGLVERTEXARRAYMULTITEXCOORDOFFSETEXTPROC glVertexArrayMultiTexCoordOffsetEXT;
01215 PFNGLVERTEXARRAYNORMALOFFSETEXTPROC glVertexArrayNormalOffsetEXT;
01216 PFNGLVERTEXARRAYSECONDARYCOLOROFFSETEXTPROC glVertexArraySecondaryColorOffsetEXT;
01217 PFNGLVERTEXARRAYTEXCOORDOFFSETEXTPROC glVertexArrayTexCoordOffsetEXT;
01218 PFNGLVERTEXARRAYVERTEXATTRIBBINDINGEXTPROC glVertexArrayVertexAttribBindingEXT;
01219 PFNGLVERTEXARRAYVERTEXATTRIBDIVISOREXTPROC glVertexArrayVertexAttribDivisorEXT;
01220 PFNGLVERTEXARRAYVERTEXATTRIBFORMATEXTPROC glVertexArrayVertexAttribFormatEXT;
01221 PFNGLVERTEXARRAYVERTEXATTRIBIFORMATEXTPROC glVertexArrayVertexAttribIFormatEXT;
01222 PFNGLVERTEXARRAYVERTEXATTRIBIOFFSETEXTPROC glVertexArrayVertexAttribIOffsetEXT;
01223 PFNGLVERTEXARRAYVERTEXATTRIBLOFFSETEXTPROC glVertexArrayVertexAttribLFormatEXT;
01224 PFNGLVERTEXARRAYVERTEXATTRIBLOFFSETEXTPROC glVertexArrayVertexAttribLOffsetEXT;
01225 PFNGLVERTEXARRAYVERTEXATTRIBOFFSETEXTPROC glVertexArrayVertexAttribOffsetEXT;
01226 PFNGLVERTEXARRAYVERTEXBINDINGDIVISOREXTPROC glVertexArrayVertexBindingDivisorEXT;
01227 PFNGLVERTEXARRAYVERTEXBUFFERPROC glVertexArrayVertexBuffer;
01228 PFNGLVERTEXARRAYVERTEXBUFFERSPROC glVertexArrayVertexBuffers;
01229 PFNGLVERTEXARRAYVERTEXOFFSETEXTPROC glVertexArrayVertexOffsetEXT;
01230 PFNGLVERTEXATTRIB1DPROC glVertexAttrib1d;
01231 PFNGLVERTEXATTRIB1DVPROC glVertexAttrib1dv;
01232 PFNGLVERTEXATTRIB1FPROC glVertexAttrib1f;
01233 PFNGLVERTEXATTRIB1FVPROC glVertexAttrib1fv;
01234 PFNGLVERTEXATTRIB1SPROC glVertexAttrib1s;
01235 PFNGLVERTEXATTRIB1SVPROC glVertexAttrib1sv;
01236 PFNGLVERTEXATTRIB2DPROC glVertexAttrib2d;
01237 PFNGLVERTEXATTRIB2DVPROC glVertexAttrib2dv;
01238 PFNGLVERTEXATTRIB2FPROC glVertexAttrib2f;
01239 PFNGLVERTEXATTRIB2FVPROC glVertexAttrib2fv;
01240 PFNGLVERTEXATTRIB2SPROC glVertexAttrib2s;
01241 PFNGLVERTEXATTRIB2SVPROC glVertexAttrib2sv;
01242 PFNGLVERTEXATTRIB3DPROC glVertexAttrib3d;
01243 PFNGLVERTEXATTRIB3DVPROC glVertexAttrib3dv;
01244 PFNGLVERTEXATTRIB3FPROC glVertexAttrib3f;
01245 PFNGLVERTEXATTRIB3FVPROC glVertexAttrib3fv;
01246 PFNGLVERTEXATTRIB3SPROC glVertexAttrib3s;
01247 PFNGLVERTEXATTRIB3SVPROC glVertexAttrib3sv;
01248 PFNGLVERTEXATTRIB4BVPROC glVertexAttrib4bv;
01249 PFNGLVERTEXATTRIB4DPROC glVertexAttrib4d;
01250 PFNGLVERTEXATTRIB4DVPROC glVertexAttrib4dv;
01251 PFNGLVERTEXATTRIB4FPROC glVertexAttrib4f;
01252 PFNGLVERTEXATTRIB4FVPROC glVertexAttrib4fv;
01253 PFNGLVERTEXATTRIB4IVPROC glVertexAttrib4iv;
01254 PFNGLVERTEXATTRIB4NBVPROC glVertexAttrib4Nbv;
01255 PFNGLVERTEXATTRIB4NIVPROC glVertexAttrib4Niv;
01256 PFNGLVERTEXATTRIB4NSVPROC glVertexAttrib4Nsv;
```



```

01257 PFNGLVERTEXATTRIB4NUBPROC glVertexAttrib4Nub;
01258 PFNGLVERTEXATTRIB4NUBPROC glVertexAttrib4Nubv;
01259 PFNGLVERTEXATTRIB4NUIVPROC glVertexAttrib4Nuiv;
01260 PFNGLVERTEXATTRIB4NUSVPROC glVertexAttrib4Nusv;
01261 PFNGLVERTEXATTRIB4SPROC glVertexAttrib4s;
01262 PFNGLVERTEXATTRIB4SVPROC glVertexAttrib4sv;
01263 PFNGLVERTEXATTRIB4UBVPROC glVertexAttrib4ubv;
01264 PFNGLVERTEXATTRIB4UIVPROC glVertexAttrib4uiv;
01265 PFNGLVERTEXATTRIB4USVPROC glVertexAttrib4usv;
01266 PFNGLVERTEXATTRIBBINDINGPROC glVertexAttribBinding;
01267 PFNGLVERTEXATTRIBDIVISORARBPROC glVertexAttribDivisorARB;
01268 PFNGLVERTEXATTRIBDIVISORPROC glVertexAttribDivisor;
01269 PFNGLVERTEXATTRIBFORMATNVPROC glVertexAttribFormatNV;
01270 PFNGLVERTEXATTRIBFORMATPROC glVertexAttribFormat;
01271 PFNGLVERTEXATTRIBI1IPROC glVertexAttribI1i;
01272 PFNGLVERTEXATTRIBI1IVPROC glVertexAttribI1iv;
01273 PFNGLVERTEXATTRIBI1UIPROC glVertexAttribI1ui;
01274 PFNGLVERTEXATTRIBI1UIVPROC glVertexAttribI1uiv;
01275 PFNGLVERTEXATTRIBI2IPROC glVertexAttribI2i;
01276 PFNGLVERTEXATTRIBI2IVPROC glVertexAttribI2iv;
01277 PFNGLVERTEXATTRIBI2UIPROC glVertexAttribI2ui;
01278 PFNGLVERTEXATTRIBI2UIVPROC glVertexAttribI2uiv;
01279 PFNGLVERTEXATTRIBI3IPROC glVertexAttribI3i;
01280 PFNGLVERTEXATTRIBI3IVPROC glVertexAttribI3iv;
01281 PFNGLVERTEXATTRIBI3UIPROC glVertexAttribI3ui;
01282 PFNGLVERTEXATTRIBI3UIVPROC glVertexAttribI3uiv;
01283 PFNGLVERTEXATTRIBI4BVPROC glVertexAttribI4bv;
01284 PFNGLVERTEXATTRIBI4IPROC glVertexAttribI4i;
01285 PFNGLVERTEXATTRIBI4IVPROC glVertexAttribI4iv;
01286 PFNGLVERTEXATTRIBI4SVPROC glVertexAttribI4sv;
01287 PFNGLVERTEXATTRIBI4UBVPROC glVertexAttribI4ubv;
01288 PFNGLVERTEXATTRIBI4UIPROC glVertexAttribI4ui;
01289 PFNGLVERTEXATTRIBI4UIVPROC glVertexAttribI4uiv;
01290 PFNGLVERTEXATTRIBI4USVPROC glVertexAttribI4usv;
01291 PFNGLVERTEXATTRIBIFORMATNVPROC glVertexAttribIFormatNV;
01292 PFNGLVERTEXATTRIBIFORMATPROC glVertexAttribIFormat;
01293 PFNGLVERTEXATTRIBIPOINTERPROC glVertexAttribIPointer;
01294 PFNGLVERTEXATTRIBL1DPROC glVertexAttribL1d;
01295 PFNGLVERTEXATTRIBL1DVPROC glVertexAttribL1dv;
01296 PFNGLVERTEXATTRIBL1I64NVPROC glVertexAttribL1i64NV;
01297 PFNGLVERTEXATTRIBL1I64VNVPROC glVertexAttribL1i64vNV;
01298 PFNGLVERTEXATTRIBL1UI64ARBPROC glVertexAttribL1ui64ARB;
01299 PFNGLVERTEXATTRIBL1UI64NVPROC glVertexAttribL1ui64NV;
01300 PFNGLVERTEXATTRIBL1UI64VARBPROC glVertexAttribL1ui64vARB;
01301 PFNGLVERTEXATTRIBL1UI64VNVPROC glVertexAttribL1ui64vNV;
01302 PFNGLVERTEXATTRIBL2DPROC glVertexAttribL2d;
01303 PFNGLVERTEXATTRIBL2DVPROC glVertexAttribL2dv;
01304 PFNGLVERTEXATTRIBL2I64NVPROC glVertexAttribL2i64NV;
01305 PFNGLVERTEXATTRIBL2I64VNVPROC glVertexAttribL2i64vNV;
01306 PFNGLVERTEXATTRIBL2UI64NVPROC glVertexAttribL2ui64NV;
01307 PFNGLVERTEXATTRIBL2UI64VNVPROC glVertexAttribL2ui64vNV;
01308 PFNGLVERTEXATTRIBL3DPROC glVertexAttribL3d;
01309 PFNGLVERTEXATTRIBL3DVPROC glVertexAttribL3dv;
01310 PFNGLVERTEXATTRIBL3I64NVPROC glVertexAttribL3i64NV;
01311 PFNGLVERTEXATTRIBL3I64VNVPROC glVertexAttribL3i64vNV;
01312 PFNGLVERTEXATTRIBL3UI64NVPROC glVertexAttribL3ui64NV;
01313 PFNGLVERTEXATTRIBL3UI64VNVPROC glVertexAttribL3ui64vNV;
01314 PFNGLVERTEXATTRIBL4DPROC glVertexAttribL4d;
01315 PFNGLVERTEXATTRIBL4DVPROC glVertexAttribL4dv;
01316 PFNGLVERTEXATTRIBL4I64NVPROC glVertexAttribL4i64NV;
01317 PFNGLVERTEXATTRIBL4I64VNVPROC glVertexAttribL4i64vNV;
01318 PFNGLVERTEXATTRIBL4UI64NVPROC glVertexAttribL4ui64NV;
01319 PFNGLVERTEXATTRIBL4UI64VNVPROC glVertexAttribL4ui64vNV;
01320 PFNGLVERTEXATTRIBLFORMATNVPROC glVertexAttribLFormatNV;
01321 PFNGLVERTEXATTRIBLFORMATPROC glVertexAttribLFormat;
01322 PFNGLVERTEXATTRIBLPOINTERPROC glVertexAttribLPointer;
01323 PFNGLVERTEXATTRIBP1UIPROC glVertexAttribP1ui;
01324 PFNGLVERTEXATTRIBP1UIVPROC glVertexAttribP1uiv;
01325 PFNGLVERTEXATTRIBP2UIPROC glVertexAttribP2ui;
01326 PFNGLVERTEXATTRIBP2UIVPROC glVertexAttribP2uiv;
01327 PFNGLVERTEXATTRIBP3UIPROC glVertexAttribP3ui;
01328 PFNGLVERTEXATTRIBP3UIVPROC glVertexAttribP3uiv;
01329 PFNGLVERTEXATTRIBP4UIPROC glVertexAttribP4ui;
01330 PFNGLVERTEXATTRIBP4UIVPROC glVertexAttribP4uiv;
01331 PFNGLVERTEXATTRIBPOINTERPROC glVertexAttribPointer;
01332 PFNGLVERTEXBINDINGDIVISORPROC glVertexBindingDivisor;
01333 PFNGLVERTEXFORMATNVPROC glVertexFormatNV;
01334 PFNGLVIEWPORTARRAYVPROC glVertexArrayv;
01335 PFNGLVIEWPORTINDEXEDFPROC glVertexIndexedf;
01336 PFNGLVIEWPORTINDEXEDFVPROC glVertexIndexedfv;
01337 PFNGLVIEWPORTPOSITIONWSALENVPROC glVertexPositionWScaleNV;
01338 PFNGLVIEWPORTPROC glVertexport;
01339 PFNGLVIEWPORTSWIZZLENVPROC glVertexportSwizzleNV;
01340 PFNGLWAITSYNCPROC glWaitSync;
01341 PFNGLWAITVKSEMAPHORENVPROC glWaitVkSemaphoreNV;
01342 PFNGLWEIGHTPATHSNVPROC glWeightPathsNV;
01343 PFNGLWINDOWRECTANGLESEXTPROC glWindowRectanglesEXT;

```

```

01344 #endif
01345
01347
01349 GLint gg::ggBufferAlignment(0);
01350
01351 //
01352 // ゲームグラフィックス特論の都合にもとづく初期化
01353 //
01354 void gg::ggInit()
01355 {
01356     // すでにこの関数が実行されていたら以降の処理を行わない
01357     if (ggBufferAlignment) return;
01358
01359     // macOS 以外で OpenGL 3.2 以降の API を取得する
01360     #if !defined(GL3_PROTOTYPES) && !defined(GL_GLES_PROTOTYPES)
01361         glActiveProgramEXT = PFNGLACTIVEPROGRAMEXTPROC(glwGetProcAddress("glActiveProgramEXT"));
01362         glActiveShaderProgram = PFNGLACTIVESHADERPROGRAMPROC(glwGetProcAddress("glActiveShaderProgram"));
01363         glActiveTexture = PFNGLACTIVETEXTUREPROC(glwGetProcAddress("glActiveTexture"));
01364         glApplyFramebufferAttachmentCMAAINTEL =
01365             PFNGLAPPLYFRAMEBUFFERATTACHMENTCMAAINTELPROC(glwGetProcAddress("glApplyFramebufferAttachmentCMAAINTEL"));
01366         glAttachShader = PFNGLATTACHSHADERPROC(glwGetProcAddress("glAttachShader"));
01367         glBeginConditionalRender =
01368             PFNGLBEGINCONDITIONALRENDERPROC(glwGetProcAddress("glBeginConditionalRender"));
01369         glBeginConditionalRenderNV =
01370             PFNGLBEGINCONDITIONALRENDERNVPROC(glwGetProcAddress("glBeginConditionalRenderNV"));
01371         glBeginPerfMonitorAMD = PFNGLBEGINPERFMONITORAMDPROC(glwGetProcAddress("glBeginPerfMonitorAMD"));
01372         glBeginPerfQueryINTEL = PFNGLBEGINPERFQUERYINTELPROC(glwGetProcAddress("glBeginPerfQueryINTEL"));
01373         glBeginQuery = PFNGLBEGINQUERYPROC(glwGetProcAddress("glBeginQuery"));
01374         glBeginQueryIndexed = PFNGLBEGINQUERYINDEXEDPROC(glwGetProcAddress("glBeginQueryIndexed"));
01375         glBeginTransformFeedback =
01376             PFNGLBEGINTRANSFORMFEEDBACKPROC(glwGetProcAddress("glBeginTransformFeedback"));
01377         glBindAttribLocation = PFNGLBINDATTRIBLOCATIONPROC(glwGetProcAddress("glBindAttribLocation"));
01378         glBindBuffer = PFNGLBINDBUFFERPROC(glwGetProcAddress("glBindBuffer"));
01379         glBindBufferBase = PFNGLBINDBUFFERBASEPROC(glwGetProcAddress("glBindBufferBase"));
01380         glBindBufferRange = PFNGLBINDBUFFERRANGEPROC(glwGetProcAddress("glBindBufferRange"));
01381         glBindBuffersBase = PFNGLBINDBUFFERSBASEPROC(glwGetProcAddress("glBindBuffersBase"));
01382         glBindBuffersRange = PFNGLBINDBUFFERSRANGEPROC(glwGetProcAddress("glBindBuffersRange"));
01383         glBindFragDataLocation =
01384             PFNGLBINDFRAGDATALOCATIONPROC(glwGetProcAddress("glBindFragDataLocation"));
01385         glBindFragDataLocationIndexed =
01386             PFNGLBINDFRAGDATALOCATIONINDEXEDPROC(glwGetProcAddress("glBindFragDataLocationIndexed"));
01387         glBindFramebuffer = PFNGLBINDFRAMEBUFFERPROC(glwGetProcAddress("glBindFramebuffer"));
01388         glBindImageTexture = PFNGLBINDIMAGETEXTUREPROC(glwGetProcAddress("glBindImageTexture"));
01389         glBindImageTextures = PFNGLBINDIMAGETEXTURESPROC(glwGetProcAddress("glBindImageTextures"));
01390         glBindMultiTextureEXT = PFNGLBINDMULTITEXTUREEXTPROC(glwGetProcAddress("glBindMultiTextureEXT"));
01391         glBindProgramPipeline = PFNGLBINDPROGRAMPIPELINEPROC(glwGetProcAddress("glBindProgramPipeline"));
01392         glBindRenderbuffer = PFNGLBINDRENDERBUFFERPROC(glwGetProcAddress("glBindRenderbuffer"));
01393         glBindSampler = PFNGLBINDSAMPLERPROC(glwGetProcAddress("glBindSampler"));
01394         glBindSamplers = PFNGLBINDSAMPLERSPROC(glwGetProcAddress("glBindSamplers"));
01395         glBindTexture = PFNGLBINDTEXTUREPROC(glwGetProcAddress("glBindTexture"));
01396         glBindTextureUnit = PFNGLBINDTEXTUREUNITPROC(glwGetProcAddress("glBindTextureUnit"));
01397         glBindTextures = PFNGLBINDTEXTURESPROC(glwGetProcAddress("glBindTextures"));
01398         glBindTransformFeedback =
01399             PFNGLBINDTRANSFORMFEEDBACKPROC(glwGetProcAddress("glBindTransformFeedback"));
01400         glBindVertexArray = PFNGLBINDVERTEXARRAYPROC(glwGetProcAddress("glBindVertexArray"));
01401         glBindVertexBuffer = PFNGLBINDVERTEXBUFFERPROC(glwGetProcAddress("glBindVertexBuffer"));
01402         glBindVertexBuffers = PFNGLBINDVERTEXBUFFERSPROC(glwGetProcAddress("glBindVertexBuffers"));
01403         glBlendBarrierKHR = PFNGLBLENDBARRIERKHRPROC(glwGetProcAddress("glBlendBarrierKHR"));
01404         glBlendBarrierNV = PFNGLBLENDBARRIERNVPROC(glwGetProcAddress("glBlendBarrierNV"));
01405         glBlendColor = PFNGLBLENDCOLORPROC(glwGetProcAddress("glBlendColor"));
01406         glBlendEquation = PFNGLBLEND EQUATIONPROC(glwGetProcAddress("glBlendEquation"));
01407         glBlendEquationSeparate =
01408             PFNGLBLEND EQUATIONSEPARATEPROC(glwGetProcAddress("glBlendEquationSeparate"));
01409         glBlendEquationSeparatei =
01410             PFNGLBLEND EQUATIONSEPARATEIPROC(glwGetProcAddress("glBlendEquationSeparatei"));
01411         glBlendEquationSeparateiARB =
01412             PFNGLBLEND EQUATIONSEPARATEIARBPROC(glwGetProcAddress("glBlendEquationSeparateiARB"));
01413         glBlendEquationi = PFNGLBLEND EQUATIONIPROC(glwGetProcAddress("glBlendEquationi"));
01414         glBlendEquationiARB = PFNGLBLEND EQUATIONIARBPROC(glwGetProcAddress("glBlendEquationiARB"));
01415         glBlendFunc = PFNGLBLEND FUNCPROC(glwGetProcAddress("glBlendFunc"));
01416         glBlendFuncSeparate = PFNGLBLEND FUNCSEPARATEPROC(glwGetProcAddress("glBlendFuncSeparate"));
01417         glBlendFuncSeparatei = PFNGLBLEND FUNCSEPARATEIPROC(glwGetProcAddress("glBlendFuncSeparatei"));
01418         glBlendFuncSeparateiARB =
01419             PFNGLBLEND FUNCSEPARATEIARBPROC(glwGetProcAddress("glBlendFuncSeparateiARB"));
01420         glBlendFunci = PFNGLBLEND FUNCIPROC(glwGetProcAddress("glBlendFunci"));
01421         glBlendFunciARB = PFNGLBLEND FUNCIARBPROC(glwGetProcAddress("glBlendFunciARB"));
01422         glBlendParameteriNV = PFNGLBLEND PARAMETERINVPROC(glwGetProcAddress("glBlendParameteriNV"));
01423         glBlitFramebuffer = PFNGLBLITFRAMEBUFFERPROC(glwGetProcAddress("glBlitFramebuffer"));
01424         glBlitNamedFramebuffer =
01425             PFNGLBLITNAMEDFRAMEBUFFERPROC(glwGetProcAddress("glBlitNamedFramebuffer"));
01426         glBufferAddressRangeNV =
01427             PFNGLBUFFERADDRESSRANGENVPROC(glwGetProcAddress("glBufferAddressRangeNV"));
01428         glBufferData = PFNGLBUFFERDATAPROC(glwGetProcAddress("glBufferData"));
01429         glBufferPageCommitmentARB =
01430             PFNGLBUFFERPAGECOMMITMENTARBPROC(glwGetProcAddress("glBufferPageCommitmentARB"));
01431         glBufferStorage = PFNGLBUFFERSTORAGEPROC(glwGetProcAddress("glBufferStorage"));
01432         glBufferSubData = PFNGLBUFFERSUBDATAPROC(glwGetProcAddress("glBufferSubData"));

```

```

01419     glCallCommandListNV = PFNGLCALLCOMMANDLISTNVPROC (glfwGetProcAddress("glCallCommandListNV"));
01420     glCheckFramebufferStatus =
PFNGLCHECKFRAMEBUFFERSTATUSPROC (glfwGetProcAddress("glCheckFramebufferStatus"));
01421     glCheckNamedFramebufferStatus =
PFNGLCHECKNAMEDFRAMEBUFFERSTATUSPROC (glfwGetProcAddress("glCheckNamedFramebufferStatus"));
01422     glCheckNamedFramebufferStatusEXT =
PFNGLCHECKNAMEDFRAMEBUFFERSTATUSEXTPROC (glfwGetProcAddress("glCheckNamedFramebufferStatusEXT"));
01423     glClampColor = PFNGLCLAMPCOLORPROC (glfwGetProcAddress("glClampColor"));
01424     glClear = PFNGLCLEARPROC (glfwGetProcAddress("glClear"));
01425     glClearBufferData = PFNGLCLEARBUFFERDATAPROC (glfwGetProcAddress("glClearBufferData"));
01426     glClearBufferSubData = PFNGLCLEARBUFFERSUBDATAPROC (glfwGetProcAddress("glClearBufferSubData"));
01427     glClearBufferfi = PFNGLCLEARBUFFERFIPROC (glfwGetProcAddress("glClearBufferfi"));
01428     glClearBufferfv = PFNGLCLEARBUFFERFVPROC (glfwGetProcAddress("glClearBufferfv"));
01429     glClearBufferiv = PFNGLCLEARBUFFERIVPROC (glfwGetProcAddress("glClearBufferiv"));
01430     glClearBufferuiv = PFNGLCLEARBUFFERUIVPROC (glfwGetProcAddress("glClearBufferuiv"));
01431     glClearColor = PFNGLCLEARCOLORPROC (glfwGetProcAddress("glClearColor"));
01432     glClearDepth = PFNGLCLEARDEPTHPROC (glfwGetProcAddress("glClearDepth"));
01433     glClearDepthf = PFNGLCLEARDEPTHFPROC (glfwGetProcAddress("glClearDepthf"));
01434     glClearNamedBufferData =
PFNGLCLEARNAMEDBUFFERDATAPROC (glfwGetProcAddress("glClearNamedBufferData"));
01435     glClearNamedBufferDataEXT =
PFNGLCLEARNAMEDBUFFERDATAEXTPROC (glfwGetProcAddress("glClearNamedBufferDataEXT"));
01436     glClearNamedBufferSubData =
PFNGLCLEARNAMEDBUFFERSUBDATAPROC (glfwGetProcAddress("glClearNamedBufferSubData"));
01437     glClearNamedBufferSubDataEXT =
PFNGLCLEARNAMEDBUFFERSUBDATAEXTPROC (glfwGetProcAddress("glClearNamedBufferSubDataEXT"));
01438     glClearNamedFramebufferfi =
PFNGLCLEARNAMEDFRAMEBUFFERFIPROC (glfwGetProcAddress("glClearNamedFramebufferfi"));
01439     glClearNamedFramebufferfv =
PFNGLCLEARNAMEDFRAMEBUFFERFVPROC (glfwGetProcAddress("glClearNamedFramebufferfv"));
01440     glClearNamedFramebufferiv =
PFNGLCLEARNAMEDFRAMEBUFFERIVPROC (glfwGetProcAddress("glClearNamedFramebufferiv"));
01441     glClearNamedBufferuiv =
PFNGLCLEARNAMEDFRAMEBUFFERUIVPROC (glfwGetProcAddress("glClearNamedBufferuiv"));
01442     glClearStencil = PFNGLCLEARSTENCILPROC (glfwGetProcAddress("glClearStencil"));
01443     glClearTexImage = PFNGLCLEARTEXIMAGEPROC (glfwGetProcAddress("glClearTexImage"));
01444     glClearTexSubImage = PFNGLCLEARTEXSUBIMAGEPROC (glfwGetProcAddress("glClearTexSubImage"));
01445     glClientAttribDefaultEXT =
PFNGLCLIENTATTRIBDEFAULTEXTPROC (glfwGetProcAddress("glClientAttribDefaultEXT"));
01446     glClientWaitSync = PFNGLCLIENTWAITSYNCPROC (glfwGetProcAddress("glClientWaitSync"));
01447     glClipControl = PFNGLCLIPCONTROLPROC (glfwGetProcAddress("glClipControl"));
01448     glColorFormatNV = PFNGLCOLORFORMATNVPROC (glfwGetProcAddress("glColorFormatNV"));
01449     glColorMask = PFNGLCOLORMASKPROC (glfwGetProcAddress("glColorMask"));
01450     glColorMaski = PFNGLCOLORMASKIPROC (glfwGetProcAddress("glColorMaski"));
01451     glCommandListSegmentsNV =
PFNGLCOMMANDLISTSEGMENTSNNVPROC (glfwGetProcAddress("glCommandListSegmentsNV"));
01452     glCompileCommandListNV =
PFNGLCOMPILECOMMANDLISTNVPROC (glfwGetProcAddress("glCompileCommandListNV"));
01453     glCompileShader = PFNGLCOMPILESHADERPROC (glfwGetProcAddress("glCompileShader"));
01454     glCompileShaderIncludeARB =
PFNGLCOMPILESHADERINCLUDEARBPROC (glfwGetProcAddress("glCompileShaderIncludeARB"));
01455     glCompressedMultiTexImage1DEXT =
PFNGLCOMPRESSEDMULTITEXIMAGE1DEXTPROC (glfwGetProcAddress("glCompressedMultiTexImage1DEXT"));
01456     glCompressedMultiTexImage2DEXT =
PFNGLCOMPRESSEDMULTITEXIMAGE2DEXTPROC (glfwGetProcAddress("glCompressedMultiTexImage2DEXT"));
01457     glCompressedMultiTexImage3DEXT =
PFNGLCOMPRESSEDMULTITEXIMAGE3DEXTPROC (glfwGetProcAddress("glCompressedMultiTexImage3DEXT"));
01458     glCompressedMultiTexSubImage1DEXT =
PFNGLCOMPRESSEDMULTITEXSUBIMAGE1DEXTPROC (glfwGetProcAddress("glCompressedMultiTexSubImage1DEXT"));
01459     glCompressedMultiTexSubImage2DEXT =
PFNGLCOMPRESSEDMULTITEXSUBIMAGE2DEXTPROC (glfwGetProcAddress("glCompressedMultiTexSubImage2DEXT"));
01460     glCompressedMultiTexSubImage3DEXT =
PFNGLCOMPRESSEDMULTITEXSUBIMAGE3DEXTPROC (glfwGetProcAddress("glCompressedMultiTexSubImage3DEXT"));
01461     glCompressedTexImage1D =
PFNGLCOMPRESSEDTEXIMAGE1DPROC (glfwGetProcAddress("glCompressedTexImage1D"));
01462     glCompressedTexImage2D =
PFNGLCOMPRESSEDTEXIMAGE2DPROC (glfwGetProcAddress("glCompressedTexImage2D"));
01463     glCompressedTexImage3D =
PFNGLCOMPRESSEDTEXIMAGE3DPROC (glfwGetProcAddress("glCompressedTexImage3D"));
01464     glCompressedTexSubImage1D =
PFNGLCOMPRESSEDTEXSUBIMAGE1DPROC (glfwGetProcAddress("glCompressedTexSubImage1D"));
01465     glCompressedTexSubImage2D =
PFNGLCOMPRESSEDTEXSUBIMAGE2DPROC (glfwGetProcAddress("glCompressedTexSubImage2D"));
01466     glCompressedTexSubImage3D =
PFNGLCOMPRESSEDTEXSUBIMAGE3DPROC (glfwGetProcAddress("glCompressedTexSubImage3D"));
01467     glCompressedTextureImage1DEXT =
PFNGLCOMPRESSEDTEXTUREIMAGE1DEXTPROC (glfwGetProcAddress("glCompressedTextureImage1DEXT"));
01468     glCompressedTextureImage2DEXT =
PFNGLCOMPRESSEDTEXTUREIMAGE2DEXTPROC (glfwGetProcAddress("glCompressedTextureImage2DEXT"));
01469     glCompressedTextureImage3DEXT =
PFNGLCOMPRESSEDTEXTUREIMAGE3DEXTPROC (glfwGetProcAddress("glCompressedTextureImage3DEXT"));
01470     glCompressedTextureSubImage1D =
PFNGLCOMPRESSEDTEXTURESUBIMAGE1DPROC (glfwGetProcAddress("glCompressedTextureSubImage1D"));
01471     glCompressedTextureSubImage1DEXT =
PFNGLCOMPRESSEDTEXTURESUBIMAGE1DEXTPROC (glfwGetProcAddress("glCompressedTextureSubImage1DEXT"));
01472     glCompressedTextureSubImage2D =
PFNGLCOMPRESSEDTEXTURESUBIMAGE2DPROC (glfwGetProcAddress("glCompressedTextureSubImage2D"));

```

```

01473     glCompressedTextureSubImage2DEXT =
PFNGLCOMPRESSEDTEXTURESUBIMAGE2DEXTPROC (glfwGetProcAddress ("glCompressedTextureSubImage2DEXT"));
01474     glCompressedTextureSubImage3D =
PFNGLCOMPRESSEDTEXTURESUBIMAGE3DPROC (glfwGetProcAddress ("glCompressedTextureSubImage3D"));
01475     glCompressedTextureSubImage3DEXT =
PFNGLCOMPRESSEDTEXTURESUBIMAGE3DEXTPROC (glfwGetProcAddress ("glCompressedTextureSubImage3DEXT"));
01476     glConservativeRasterParameterfNV =
PFNGLCONSERVATIVERASTERPARAMETERFNVPROC (glfwGetProcAddress ("glConservativeRasterParameterfNV"));
01477     glConservativeRasterParameteriNV =
PFNGLCONSERVATIVERASTERPARAMETERINVPROC (glfwGetProcAddress ("glConservativeRasterParameteriNV"));
01478     glCopyBufferSubData = PFNGLCOPYBUFFERSUBDATAPROC (glfwGetProcAddress ("glCopyBufferSubData"));
01479     glCopyImageSubData = PFNGLCOPYIMAGESUBDATAPROC (glfwGetProcAddress ("glCopyImageSubData"));
01480     glCopyMultiTexImage1DEXT =
PFNGLCOPYMULTITEXIMAGE1DEXTPROC (glfwGetProcAddress ("glCopyMultiTexImage1DEXT"));
01481     glCopyMultiTexImage2DEXT =
PFNGLCOPYMULTITEXIMAGE2DEXTPROC (glfwGetProcAddress ("glCopyMultiTexImage2DEXT"));
01482     glCopyMultiTexSubImage1DEXT =
PFNGLCOPYMULTITEXSUBIMAGE1DEXTPROC (glfwGetProcAddress ("glCopyMultiTexSubImage1DEXT"));
01483     glCopyMultiTexSubImage2DEXT =
PFNGLCOPYMULTITEXSUBIMAGE2DEXTPROC (glfwGetProcAddress ("glCopyMultiTexSubImage2DEXT"));
01484     glCopyMultiTexSubImage3DEXT =
PFNGLCOPYMULTITEXSUBIMAGE3DEXTPROC (glfwGetProcAddress ("glCopyMultiTexSubImage3DEXT"));
01485     glCopyNamedBufferSubData =
PFNGLCOPYNAMEDBUFFERSUBDATAPROC (glfwGetProcAddress ("glCopyNamedBufferSubData"));
01486     glCopyPathNV = PFNGLCOPYPATHNVPROC (glfwGetProcAddress ("glCopyPathNV"));
01487     glCopyTexImage1D = PFNGLCOPYTEXIMAGE1DPROC (glfwGetProcAddress ("glCopyTexImage1D"));
01488     glCopyTexImage2D = PFNGLCOPYTEXIMAGE2DPROC (glfwGetProcAddress ("glCopyTexImage2D"));
01489     glCopyTexSubImage1D = PFNGLCOPYTEXSUBIMAGE1DPROC (glfwGetProcAddress ("glCopyTexSubImage1D"));
01490     glCopyTexSubImage2D = PFNGLCOPYTEXSUBIMAGE2DPROC (glfwGetProcAddress ("glCopyTexSubImage2D"));
01491     glCopyTexSubImage3D = PFNGLCOPYTEXSUBIMAGE3DPROC (glfwGetProcAddress ("glCopyTexSubImage3D"));
01492     glCopyTextureImage1DEXT =
PFNGLCOPYTEXTUREIMAGE1DEXTPROC (glfwGetProcAddress ("glCopyTextureImage1DEXT"));
01493     glCopyTextureImage2DEXT =
PFNGLCOPYTEXTUREIMAGE2DEXTPROC (glfwGetProcAddress ("glCopyTextureImage2DEXT"));
01494     glCopyTextureSubImage1D =
PFNGLCOPYTEXTURESUBIMAGE1DPROC (glfwGetProcAddress ("glCopyTextureSubImage1D"));
01495     glCopyTextureSubImage1DEXT =
PFNGLCOPYTEXTURESUBIMAGE1DEXTPROC (glfwGetProcAddress ("glCopyTextureSubImage1DEXT"));
01496     glCopyTextureSubImage2D =
PFNGLCOPYTEXTURESUBIMAGE2DPROC (glfwGetProcAddress ("glCopyTextureSubImage2D"));
01497     glCopyTextureSubImage2DEXT =
PFNGLCOPYTEXTURESUBIMAGE2DEXTPROC (glfwGetProcAddress ("glCopyTextureSubImage2DEXT"));
01498     glCopyTextureSubImage3D =
PFNGLCOPYTEXTURESUBIMAGE3DPROC (glfwGetProcAddress ("glCopyTextureSubImage3D"));
01499     glCopyTextureSubImage3DEXT =
PFNGLCOPYTEXTURESUBIMAGE3DEXTPROC (glfwGetProcAddress ("glCopyTextureSubImage3DEXT"));
01500     glCoverFillPathInstancedNV =
PFNGLCOVERFILLPATHINSTANCEDNVPROC (glfwGetProcAddress ("glCoverFillPathInstancedNV"));
01501     glCoverFillPathNV = PFNGLCOVERFILLPATHNVPROC (glfwGetProcAddress ("glCoverFillPathNV"));
01502     glCoverStrokePathInstancedNV =
PFNGLCOVERSTROKEPATHINSTANCEDNVPROC (glfwGetProcAddress ("glCoverStrokePathInstancedNV"));
01503     glCoverStrokePathNV = PFNGLCOVERSTROKEPATHNVPROC (glfwGetProcAddress ("glCoverStrokePathNV"));
01504     glCoverageModulationNV =
PFNGLCOVERAGEMODULATIONNVPROC (glfwGetProcAddress ("glCoverageModulationNV"));
01505     glCoverageModulationTableNV =
PFNGLCOVERAGEMODULATIONTABLENVPROC (glfwGetProcAddress ("glCoverageModulationTableNV"));
01506     glCreateBuffers = PFNGLCREATEBUFFERSPROC (glfwGetProcAddress ("glCreateBuffers"));
01507     glCreateCommandListsNV =
PFNGLCREATECOMMANDLISTSNVPROC (glfwGetProcAddress ("glCreateCommandListsNV"));
01508     glCreateFramebuffers = PFNGLCREATEFRAMEBUFFERSPROC (glfwGetProcAddress ("glCreateFramebuffers"));
01509     glCreatePerfQueryINTEL =
PFNGLCREATEPERFQUERYINTELPROC (glfwGetProcAddress ("glCreatePerfQueryINTEL"));
01510     glCreateProgram = PFNGLCREATEPROGRAMPROC (glfwGetProcAddress ("glCreateProgram"));
01511     glCreateProgramPipelines =
PFNGLCREATEPROGRAMPIPELINESPROC (glfwGetProcAddress ("glCreateProgramPipelines"));
01512     glCreateQueries = PFNGLCREATEQUERIESPROC (glfwGetProcAddress ("glCreateQueries"));
01513     glCreateRenderbuffers = PFNGLCREATERENDERBUFFERSPROC (glfwGetProcAddress ("glCreateRenderbuffers"));
01514     glCreateSamplers = PFNGLCREATESAMPLERSPROC (glfwGetProcAddress ("glCreateSamplers"));
01515     glCreateShader = PFNGLCREATESHADERPROC (glfwGetProcAddress ("glCreateShader"));
01516     glCreateShaderProgramEXT =
PFNGLCREATESHADERPROGRAMEXTPROC (glfwGetProcAddress ("glCreateShaderProgramEXT"));
01517     glCreateShaderProgramv =
PFNGLCREATESHADERPROGRAMVPROC (glfwGetProcAddress ("glCreateShaderProgramv"));
01518     glCreateStatesNV = PFNGLCREATESTATESNVPROC (glfwGetProcAddress ("glCreateStatesNV"));
01519     glCreateSyncFromCLeventARB =
PFNGLCREATESYNCFROMCLEVENTARBPROC (glfwGetProcAddress ("glCreateSyncFromCLeventARB"));
01520     glCreateTextures = PFNGLCREATETEXTURESPROC (glfwGetProcAddress ("glCreateTextures"));
01521     glCreateTransformFeedbacks =
PFNGLCREATETRANSFORMFEEDBACKSPROC (glfwGetProcAddress ("glCreateTransformFeedbacks"));
01522     glCreateVertexArrays = PFNGLCREATEVERTEXARRAYSPROC (glfwGetProcAddress ("glCreateVertexArrays"));
01523     glCullFace = PFNGLCULLFACEPROC (glfwGetProcAddress ("glCullFace"));
01524     glDebugMessageCallback =
PFNGLDEBUGMESSAGECALLBACKPROC (glfwGetProcAddress ("glDebugMessageCallback"));
01525     glDebugMessageCallbackARB =
PFNGLDEBUGMESSAGECALLBACKARBPROC (glfwGetProcAddress ("glDebugMessageCallbackARB"));
01526     glDebugMessageControl = PFNGLDEBUGMESSAGECONTROLPROC (glfwGetProcAddress ("glDebugMessageControl"));
01527     glDebugMessageControlARB =

```



```

        PFNGLDEBUGMESSAGECONTROLARBPROC (glfwGetProcAddress ("glDebugMessageControlARB"));
01528     glDebugMessageInsert = PFNGLDEBUGMESSAGEINSERTPROC (glfwGetProcAddress ("glDebugMessageInsert"));
01529     glDebugMessageInsertARB =
        PFNGLDEBUGMESSAGEINSERTARBPROC (glfwGetProcAddress ("glDebugMessageInsertARB"));
01530     glDeleteBuffers = PFNGLDELETEBUFFERSPROC (glfwGetProcAddress ("glDeleteBuffers"));
01531     glDeleteCommandListsNV =
        PFNGLDELETECOMMANDLISTSNVPROC (glfwGetProcAddress ("glDeleteCommandListsNV"));
01532     glDeleteFramebuffers = PFNGLDELETEFRAMEBUFFERSPROC (glfwGetProcAddress ("glDeleteFramebuffers"));
01533     glDeleteNamedStringARB =
        PFNGLDELETENAMEDSTRINGARBPROC (glfwGetProcAddress ("glDeleteNamedStringARB"));
01534     glDeletePathsNV = PFNGLDELETEPATHSNVPROC (glfwGetProcAddress ("glDeletePathsNV"));
01535     glDeletePerfMonitorsAMD =
        PFNGLDELETEPERFMONITORSAMDPROC (glfwGetProcAddress ("glDeletePerfMonitorsAMD"));
01536     glDeletePerfQueryINTEL =
        PFNGLDELETEPERFQUERYINTELPROC (glfwGetProcAddress ("glDeletePerfQueryINTEL"));
01537     glDeleteProgram = PFNGLDELETEPROGRAMPROC (glfwGetProcAddress ("glDeleteProgram"));
01538     glDeleteProgramPipelines =
        PFNGLDELETEPROGRAMPIPELINESPROC (glfwGetProcAddress ("glDeleteProgramPipelines"));
01539     glDeleteQueries = PFNGLDELETEQUERIESPROC (glfwGetProcAddress ("glDeleteQueries"));
01540     glDeleteRenderbuffers = PFNGLDELETERENDERBUFFERSPROC (glfwGetProcAddress ("glDeleteRenderbuffers"));
01541     glDeleteSamplers = PFNGLDELETESAMPLERSPROC (glfwGetProcAddress ("glDeleteSamplers"));
01542     glDeleteShader = PFNGLDELETESHADERPROC (glfwGetProcAddress ("glDeleteShader"));
01543     glDeleteStatesNV = PFNGLDELETESSTATESNVPROC (glfwGetProcAddress ("glDeleteStatesNV"));
01544     glDeleteSync = PFNGLDELETESYNCPROC (glfwGetProcAddress ("glDeleteSync"));
01545     glDeleteTextures = PFNGLDELETETEXTURESPROC (glfwGetProcAddress ("glDeleteTextures"));
01546     glDeleteTransformFeedbacks =
        PFNGLDELETETRANSFORMFEEDBACKSPROC (glfwGetProcAddress ("glDeleteTransformFeedbacks"));
01547     glDeleteVertexArrays = PFNGLDELETEVERTEXARRAYSPROC (glfwGetProcAddress ("glDeleteVertexArrays"));
01548     glDepthFunc = PFNGLDEPTHFUNCPROC (glfwGetProcAddress ("glDepthFunc"));
01549     glDepthMask = PFNGLDEPTHMASKPROC (glfwGetProcAddress ("glDepthMask"));
01550     glDepthRange = PFNGLDEPTHRANGEPROC (glfwGetProcAddress ("glDepthRange"));
01551     glDepthRangeArrayv = PFNGLDEPTHRANGEARRAYVPROC (glfwGetProcAddress ("glDepthRangeArrayv"));
01552     glDepthRangeIndexed = PFNGLDEPTHRANGEINDEXEDPROC (glfwGetProcAddress ("glDepthRangeIndexed"));
01553     glDepthRangef = PFNGLDEPTHRANGEFPROC (glfwGetProcAddress ("glDepthRangef"));
01554     glDetachShader = PFNGLDETACHSHADERPROC (glfwGetProcAddress ("glDetachShader"));
01555     glDisable = PFNGLDISABLEPROC (glfwGetProcAddress ("glDisable"));
01556     glDisableClientStateIndexedEXT =
        PFNGLDISABLECLIENTSTATEINDEXEDEXTPROC (glfwGetProcAddress ("glDisableClientStateIndexedEXT"));
01557     glDisableClientStateiEXT =
        PFNGLDISABLECLIENTSTATEIEXTPROC (glfwGetProcAddress ("glDisableClientStateiEXT"));
01558     glDisableIndexedEXT = PFNGLDISABLEINDEXEDEXTPROC (glfwGetProcAddress ("glDisableIndexedEXT"));
01559     glDisableVertexArrayAttrib =
        PFNGLDISABLEVERTEXARRAYATTRIBPROC (glfwGetProcAddress ("glDisableVertexArrayAttrib"));
01560     glDisableVertexArrayAttribEXT =
        PFNGLDISABLEVERTEXARRAYATTRIBEXTPROC (glfwGetProcAddress ("glDisableVertexArrayAttribEXT"));
01561     glDisableVertexArrayEXT =
        PFNGLDISABLEVERTEXARRAYEXTPROC (glfwGetProcAddress ("glDisableVertexArrayEXT"));
01562     glDisableVertexAttribArray =
        PFNGLDISABLEVERTEXATTRIBARRAYPROC (glfwGetProcAddress ("glDisableVertexAttribArray"));
01563     glDisablei = PFNGLDISABLEIPROC (glfwGetProcAddress ("glDisablei"));
01564     glDispatchCompute = PFNGLDISPATCHCOMPUTEPROC (glfwGetProcAddress ("glDispatchCompute"));
01565     glDispatchComputeGroupSizeARB =
        PFNGLDISPATCHCOMPUTEGROUPSIZEARBPROC (glfwGetProcAddress ("glDispatchComputeGroupSizeARB"));
01566     glDispatchComputeIndirect =
        PFNGLDISPATCHCOMPUTEINDIRECTPROC (glfwGetProcAddress ("glDispatchComputeIndirect"));
01567     glDrawArrays = PFNGLDRAWARRAYSPROC (glfwGetProcAddress ("glDrawArrays"));
01568     glDrawArraysIndirect = PFNGLDRAWARRAYSINDIRECTPROC (glfwGetProcAddress ("glDrawArraysIndirect"));
01569     glDrawArraysInstanced = PFNGLDRAWARRAYSINSTANCEDPROC (glfwGetProcAddress ("glDrawArraysInstanced"));
01570     glDrawArraysInstancedARB =
        PFNGLDRAWARRAYSINSTANCEDARBPROC (glfwGetProcAddress ("glDrawArraysInstancedARB"));
01571     glDrawArraysInstancedBaseInstance =
        PFNGLDRAWARRAYSINSTANCEDBASEINSTANCEPROC (glfwGetProcAddress ("glDrawArraysInstancedBaseInstance"));
01572     glDrawArraysInstancedEXT =
        PFNGLDRAWARRAYSINSTANCEDEXTPROC (glfwGetProcAddress ("glDrawArraysInstancedEXT"));
01573     glDrawBuffer = PFNGLDRAWBUFFERPROC (glfwGetProcAddress ("glDrawBuffer"));
01574     glDrawBuffers = PFNGLDRAWBUFFERSPROC (glfwGetProcAddress ("glDrawBuffers"));
01575     glDrawCommandsAddressNV =
        PFNGLDRAWCOMMANDSADDRESSNVPROC (glfwGetProcAddress ("glDrawCommandsAddressNV"));
01576     glDrawCommandsNV = PFNGLDRAWCOMMANDSNVPROC (glfwGetProcAddress ("glDrawCommandsNV"));
01577     glDrawCommandsStatesAddressNV =
        PFNGLDRAWCOMMANDSSTATESADDRESSNVPROC (glfwGetProcAddress ("glDrawCommandsStatesAddressNV"));
01578     glDrawCommandsStatesNV =
        PFNGLDRAWCOMMANDSSTATESNVPROC (glfwGetProcAddress ("glDrawCommandsStatesNV"));
01579     glDrawElements = PFNGLDRAWELEMENTSPROC (glfwGetProcAddress ("glDrawElements"));
01580     glDrawElementsBaseVertex =
        PFNGLDRAWELEMENTSBASEVERTEXPROC (glfwGetProcAddress ("glDrawElementsBaseVertex"));
01581     glDrawElementsIndirect =
        PFNGLDRAWELEMENTSINDIRECTPROC (glfwGetProcAddress ("glDrawElementsIndirect"));
01582     glDrawElementsInstanced =
        PFNGLDRAWELEMENTSINSTANCEDPROC (glfwGetProcAddress ("glDrawElementsInstanced"));
01583     glDrawElementsInstancedARB =
        PFNGLDRAWELEMENTSINSTANCEDARBPROC (glfwGetProcAddress ("glDrawElementsInstancedARB"));
01584     glDrawElementsInstancedBaseInstance =
        PFNGLDRAWELEMENTSINSTANCEDBASEINSTANCEPROC (glfwGetProcAddress ("glDrawElementsInstancedBaseInstance"));
01585     glDrawElementsInstancedBaseVertex =
        PFNGLDRAWELEMENTSINSTANCEDBASEVERTEXPROC (glfwGetProcAddress ("glDrawElementsInstancedBaseVertex"));
01586     glDrawElementsInstancedBaseVertexBaseInstance =

```

```

PFNGLDRAWELEMENTSINSTANCEDBASEVERTEXBASEINSTANCEPROC (glfwGetProcAddress("glDrawElementsInstancedBaseVertexBaseInstance"));
01587 glDrawElementsInstancedEXT =
PFNGLDRAWELEMENTSINSTANCEDEXTPROC (glfwGetProcAddress("glDrawElementsInstancedEXT"));
01588 glDrawRangeElements = PFNGLDRAWRANGEELEMENTSPROC (glfwGetProcAddress("glDrawRangeElements"));
01589 glDrawRangeElementsBaseVertex =
PFNGLDRAWRANGEELEMENTSBASEVERTEXPROC (glfwGetProcAddress("glDrawRangeElementsBaseVertex"));
01590 glDrawTransformFeedback =
PFNGLDRAWTRANSFORMFEEDBACKPROC (glfwGetProcAddress("glDrawTransformFeedback"));
01591 glDrawTransformFeedbackInstanced =
PFNGLDRAWTRANSFORMFEEDBACKINSTANCEDPROC (glfwGetProcAddress("glDrawTransformFeedbackInstanced"));
01592 glDrawTransformFeedbackStream =
PFNGLDRAWTRANSFORMFEEDBACKSTREAMPROC (glfwGetProcAddress("glDrawTransformFeedbackStream"));
01593 glDrawTransformFeedbackStreamInstanced =
PFNGLDRAWTRANSFORMFEEDBACKSTREAMINSTANCEDPROC (glfwGetProcAddress("glDrawTransformFeedbackStreamInstanced"));
01594 glDrawVkImageNV = PFNGLDRAWVKIMAGENVPROC (glfwGetProcAddress("glDrawVkImageNV"));
01595 glEdgeFlagFormatNV = PFNGLEDGEFLAGFORMATNVPROC (glfwGetProcAddress("glEdgeFlagFormatNV"));
01596 glEnable = PFNGLENABLEPROC (glfwGetProcAddress("glEnable"));
01597 glEnableClientStateIndexedEXT =
PFNGLENABLECLIENTSTATEINDEXEDEXTPROC (glfwGetProcAddress("glEnableClientStateIndexedEXT"));
01598 glEnableClientStateiEXT =
PFNGLENABLECLIENTSTATEIEXTPROC (glfwGetProcAddress("glEnableClientStateiEXT"));
01599 glEnableIndexedEXT = PFNGLENABLEINDEXEDEXTPROC (glfwGetProcAddress("glEnableIndexedEXT"));
01600 glEnableVertexArrayAttrib =
PFNGLENABLEVERTEXARRAYATTRIBPROC (glfwGetProcAddress("glEnableVertexArrayAttrib"));
01601 glEnableVertexArrayAttribEXT =
PFNGLENABLEVERTEXARRAYATTRIBEXTPROC (glfwGetProcAddress("glEnableVertexArrayAttribEXT"));
01602 glEnableVertexArrayEXT =
PFNGLENABLEVERTEXARRAYEXTPROC (glfwGetProcAddress("glEnableVertexArrayEXT"));
01603 glEnableVertexAttribArray =
PFNGLENABLEVERTEXATTRIBARRAYPROC (glfwGetProcAddress("glEnableVertexAttribArray"));
01604 glEnablei = PFNGLENABLEIPROC (glfwGetProcAddress("glEnablei"));
01605 glEndConditionalRender =
PFNGLENDCONDITIONALRENDERPROC (glfwGetProcAddress("glEndConditionalRender"));
01606 glEndConditionalRenderNV =
PFNGLENDCONDITIONALRENDERNVPROC (glfwGetProcAddress("glEndConditionalRenderNV"));
01607 glEndPerfMonitorAMD = PFNGLENDPERFMONITORAMDPROC (glfwGetProcAddress("glEndPerfMonitorAMD"));
01608 glEndPerfQueryINTEL = PFNGLENDPERFQUERYINTELPROC (glfwGetProcAddress("glEndPerfQueryINTEL"));
01609 glEndQuery = PFNGLENDQUERYPROC (glfwGetProcAddress("glEndQuery"));
01610 glEndQueryIndexed = PFNGLENDQUERYINDEXEDPROC (glfwGetProcAddress("glEndQueryIndexed"));
01611 glEndTransformFeedback =
PFNGLENDTRANSFORMFEEDBACKPROC (glfwGetProcAddress("glEndTransformFeedback"));
01612 glEvaluateDepthValuesARB =
PFNGLEVALUATEDEPTHVALUESARBPROC (glfwGetProcAddress("glEvaluateDepthValuesARB"));
01613 glFenceSync = PFNGLFENCESYNCPROC (glfwGetProcAddress("glFenceSync"));
01614 glFinish = PFNGLFINISHPROC (glfwGetProcAddress("glFinish"));
01615 glFlush = PFNGLFLUSHPROC (glfwGetProcAddress("glFlush"));
01616 glFlushMappedBufferRange =
PFNGLFLUSHMAPPEDBUFFERRANGEPROC (glfwGetProcAddress("glFlushMappedBufferRange"));
01617 glFlushMappedNamedBufferRange =
PFNGLFLUSHMAPPEDNAMEDBUFFERRANGEPROC (glfwGetProcAddress("glFlushMappedNamedBufferRange"));
01618 glFlushMappedNamedBufferRangeEXT =
PFNGLFLUSHMAPPEDNAMEDBUFFERRANGEEXTPROC (glfwGetProcAddress("glFlushMappedNamedBufferRangeEXT"));
01619 glFogCoordFormatNV = PFNGLFOGCOORDFORMATNVPROC (glfwGetProcAddress("glFogCoordFormatNV"));
01620 glFragmentCoverageColorNV =
PFNGLFRAGMENTCOVERAGECOLORNVPROC (glfwGetProcAddress("glFragmentCoverageColorNV"));
01621 glFramebufferDrawBufferEXT =
PFNGLFRAMEBUFFERDRAWBUFFEREXTPROC (glfwGetProcAddress("glFramebufferDrawBufferEXT"));
01622 glFramebufferDrawBuffersEXT =
PFNGLFRAMEBUFFERDRAWBUFFERSEXTPROC (glfwGetProcAddress("glFramebufferDrawBuffersEXT"));
01623 glFramebufferParameteri =
PFNGLFRAMEBUFFERPARAMETERIPROC (glfwGetProcAddress("glFramebufferParameteri"));
01624 glFramebufferReadBufferEXT =
PFNGLFRAMEBUFFERREADBUFFEREXTPROC (glfwGetProcAddress("glFramebufferReadBufferEXT"));
01625 glFramebufferRenderbuffer =
PFNGLFRAMEBUFFERRENDERBUFFERPROC (glfwGetProcAddress("glFramebufferRenderbuffer"));
01626 glFramebufferSampleLocationsfvARB =
PFNGLFRAMEBUFFERSAMPLELOCATIONSFVARBPROC (glfwGetProcAddress("glFramebufferSampleLocationsfvARB"));
01627 glFramebufferSampleLocationsfvNV =
PFNGLFRAMEBUFFERSAMPLELOCATIONSFVNVPROC (glfwGetProcAddress("glFramebufferSampleLocationsfvNV"));
01628 glFramebufferTexture = PFNGLFRAMEBUFFERTEXTUREPROC (glfwGetProcAddress("glFramebufferTexture"));
01629 glFramebufferTexture1D =
PFNGLFRAMEBUFFERTEXTURE1DPROC (glfwGetProcAddress("glFramebufferTexture1D"));
01630 glFramebufferTexture2D =
PFNGLFRAMEBUFFERTEXTURE2DPROC (glfwGetProcAddress("glFramebufferTexture2D"));
01631 glFramebufferTexture3D =
PFNGLFRAMEBUFFERTEXTURE3DPROC (glfwGetProcAddress("glFramebufferTexture3D"));
01632 glFramebufferTextureARB =
PFNGLFRAMEBUFFERTEXTUREARBPROC (glfwGetProcAddress("glFramebufferTextureARB"));
01633 glFramebufferTextureFaceARB =
PFNGLFRAMEBUFFERTEXTUREFACEARBPROC (glfwGetProcAddress("glFramebufferTextureFaceARB"));
01634 glFramebufferTextureLayer =
PFNGLFRAMEBUFFERTEXTURELAYERPROC (glfwGetProcAddress("glFramebufferTextureLayer"));
01635 glFramebufferTextureLayerARB =
PFNGLFRAMEBUFFERTEXTURELAYERARBPROC (glfwGetProcAddress("glFramebufferTextureLayerARB"));
01636 glFramebufferTextureMultiviewOVR =
PFNGLFRAMEBUFFERTEXTUREMULTIVIEWOVRPROC (glfwGetProcAddress("glFramebufferTextureMultiviewOVR"));
01637 glFrontFace = PFNGLFRONTFACEPROC (glfwGetProcAddress("glFrontFace"));

```

```

01638     glGenBuffers = PFNGLGENBUFFERSPROC (glfwGetProcAddress("glGenBuffers"));
01639     glGenFramebuffers = PFNGLGENFRAMEBUFFERSPROC (glfwGetProcAddress("glGenFramebuffers"));
01640     glGenPathsNV = PFNGLGENPATHSNVPROC (glfwGetProcAddress("glGenPathsNV"));
01641     glGenPerfMonitorsAMD = PFNGLGENPERFMONITORSAMDPROC (glfwGetProcAddress("glGenPerfMonitorsAMD"));
01642     glGenProgramPipelines = PFNGLGENPROGRAMPIPELINESPROC (glfwGetProcAddress("glGenProgramPipelines"));
01643     glGenQueries = PFNGLGENQUERIESPROC (glfwGetProcAddress("glGenQueries"));
01644     glGenRenderbuffers = PFNGLGENRENDERBUFFERSPROC (glfwGetProcAddress("glGenRenderbuffers"));
01645     glGenSamplers = PFNGLGENSAMPLERSPROC (glfwGetProcAddress("glGenSamplers"));
01646     glGenTextures = PFNGLGENTEXTURESPROC (glfwGetProcAddress("glGenTextures"));
01647     glGenTransformFeedbacks =
PFNGLGENTRANSFORMFEEDBACKSPROC (glfwGetProcAddress("glGenTransformFeedbacks"));
01648     glGenVertexArrays = PFNGLGENVERTEXARRAYSPROC (glfwGetProcAddress("glGenVertexArrays"));
01649     glGenerateMipmap = PFNGLGENERATEMIPMAPPROC (glfwGetProcAddress("glGenerateMipmap"));
01650     glGenerateMultiTexMipmapEXT =
PFNGLGENERATEMULTITEXMIPMAPEXTPROC (glfwGetProcAddress("glGenerateMultiTexMipmapEXT"));
01651     glGenerateTextureMipmap =
PFNGLGENERATETEXTUREMIPMAPPROC (glfwGetProcAddress("glGenerateTextureMipmap"));
01652     glGenerateTextureMipmapEXT =
PFNGLGENERATETEXTUREMIPMAPEXTPROC (glfwGetProcAddress("glGenerateTextureMipmapEXT"));
01653     glGetActiveAtomicCounterBufferiv =
PFNGLGETACTIVEATOMICCOUNTERBUFFERIVPROC (glfwGetProcAddress("glGetActiveAtomicCounterBufferiv"));
01654     glGetActiveAttrib = PFNGLGETACTIVEATTRIBPROC (glfwGetProcAddress("glGetActiveAttrib"));
01655     glGetActiveSubroutineName =
PFNGLGETACTIVESUBROUTINENAMEPROC (glfwGetProcAddress("glGetActiveSubroutineName"));
01656     glGetActiveSubroutineUniformName =
PFNGLGETACTIVESUBROUTINEUNIFORMNAMEPROC (glfwGetProcAddress("glGetActiveSubroutineUniformName"));
01657     glGetActiveSubroutineUniformiv =
PFNGLGETACTIVESUBROUTINEUNIFORMIVPROC (glfwGetProcAddress("glGetActiveSubroutineUniformiv"));
01658     glGetActiveUniform = PFNGLGETACTIVEUNIFORMPROC (glfwGetProcAddress("glGetActiveUniform"));
01659     glGetActiveUniformBlockName =
PFNGLGETACTIVEUNIFORMBLOCKNAMEPROC (glfwGetProcAddress("glGetActiveUniformBlockName"));
01660     glGetActiveUniformBlockiv =
PFNGLGETACTIVEUNIFORMBLOCKIVPROC (glfwGetProcAddress("glGetActiveUniformBlockiv"));
01661     glGetActiveUniformName =
PFNGLGETACTIVEUNIFORMNAMEPROC (glfwGetProcAddress("glGetActiveUniformName"));
01662     glGetActiveUniformsiv = PFNGLGETACTIVEUNIFORMSIVPROC (glfwGetProcAddress("glGetActiveUniformsiv"));
01663     glGetAttachedShaders = PFNGLGETATTACHEDSHADERSPROC (glfwGetProcAddress("glGetAttachedShaders"));
01664     glGetAttribLocation = PFNGLGETATTRIBLOCATIONPROC (glfwGetProcAddress("glGetAttribLocation"));
01665     glGetBooleanIndexedvEXT =
PFNGLGETBOOLEANINDEXEDVEXTPROC (glfwGetProcAddress("glGetBooleanIndexedvEXT"));
01666     glGetBooleani_v = PFNGLGETBOOLEANI_VPROC (glfwGetProcAddress("glGetBooleani_v"));
01667     glGetBooleani_v = PFNGLGETBOOLEANI_VPROC (glfwGetProcAddress("glGetBooleani_v"));
01668     glGetBufferParameteri64v =
PFNGLGETBUFFERPARAMETERI64VPROC (glfwGetProcAddress("glGetBufferParameteri64v"));
01669     glGetBufferParameteriv =
PFNGLGETBUFFERPARAMETERIVPROC (glfwGetProcAddress("glGetBufferParameteriv"));
01670     glGetBufferParameterui64vNV =
PFNGLGETBUFFERPARAMETERUI64VNVPROC (glfwGetProcAddress("glGetBufferParameterui64vNV"));
01671     glGetBufferPointerv = PFNGLGETBUFFERPOINTERVPROC (glfwGetProcAddress("glGetBufferPointerv"));
01672     glGetBufferSubData = PFNGLGETBUFFERSUBDATAPROC (glfwGetProcAddress("glGetBufferSubData"));
01673     glGetCommandHeaderNV = PFNGLGETCOMMANDHEADERNVPROC (glfwGetProcAddress("glGetCommandHeaderNV"));
01674     glGetCompressedMultiTexImageEXT =
PFNGLGETCOMPRESSEDMULTITEXIMAGEEXTPROC (glfwGetProcAddress("glGetCompressedMultiTexImageEXT"));
01675     glGetCompressedTexImage =
PFNGLGETCOMPRESSEDTEXIMAGEPROC (glfwGetProcAddress("glGetCompressedTexImage"));
01676     glGetCompressedTextureImage =
PFNGLGETCOMPRESSEDTEXTUREIMAGEPROC (glfwGetProcAddress("glGetCompressedTextureImage"));
01677     glGetCompressedTextureImageEXT =
PFNGLGETCOMPRESSEDTEXTUREIMAGEEXTPROC (glfwGetProcAddress("glGetCompressedTextureImageEXT"));
01678     glGetCompressedTextureSubImage =
PFNGLGETCOMPRESSEDTEXTURESUBIMAGEPROC (glfwGetProcAddress("glGetCompressedTextureSubImage"));
01679     glGetCoverageModulationTableNV =
PFNGLGETCOVERAGEMODULATIONTABLENVPROC (glfwGetProcAddress("glGetCoverageModulationTableNV"));
01680     glGetDebugMessageLog = PFNGLGETDEBUGMESSAGELOGPROC (glfwGetProcAddress("glGetDebugMessageLog"));
01681     glGetDebugMessageLogARB =
PFNGLGETDEBUGMESSAGELOGARBPROC (glfwGetProcAddress("glGetDebugMessageLogARB"));
01682     glGetDoubleIndexedvEXT =
PFNGLGETDOUBLEINDEXEDVEXTPROC (glfwGetProcAddress("glGetDoubleIndexedvEXT"));
01683     glGetDoublei_v = PFNGLGETDOUBLEI_VPROC (glfwGetProcAddress("glGetDoublei_v"));
01684     glGetDoublei_vEXT = PFNGLGETDOUBLEI_VEXTPROC (glfwGetProcAddress("glGetDoublei_vEXT"));
01685     glGetDoublev = PFNGLGETDOUBLEVPROC (glfwGetProcAddress("glGetDoublev"));
01686     glGetError = PFNGLGETERRORPROC (glfwGetProcAddress("glGetError"));
01687     glGetFirstPerfQueryIdINTEL =
PFNGLGETFIRSTPERFQUERYIDINTELPROC (glfwGetProcAddress("glGetFirstPerfQueryIdINTEL"));
01688     glGetFloatIndexedvEXT = PFNGLGETFLOATINDEXEDVEXTPROC (glfwGetProcAddress("glGetFloatIndexedvEXT"));
01689     glGetFloati_v = PFNGLGETFLOATI_VPROC (glfwGetProcAddress("glGetFloati_v"));
01690     glGetFloati_vEXT = PFNGLGETFLOATI_VEXTPROC (glfwGetProcAddress("glGetFloati_vEXT"));
01691     glGetFloatv = PFNGLGETFLOATVPROC (glfwGetProcAddress("glGetFloatv"));
01692     glGetFragDataIndex = PFNGLGETFRAGDATAINDEXPROC (glfwGetProcAddress("glGetFragDataIndex"));
01693     glGetFragDataLocation = PFNGLGETFRAGDATALOCATIONPROC (glfwGetProcAddress("glGetFragDataLocation"));
01694     glGetFramebufferAttachmentParameteriv =
PFNGLGETFRAMEBUFFERATTACHMENTPARAMETERIVPROC (glfwGetProcAddress("glGetFramebufferAttachmentParameteriv"));
01695     glGetFramebufferParameteriv =
PFNGLGETFRAMEBUFFERPARAMETERIVPROC (glfwGetProcAddress("glGetFramebufferParameteriv"));
01696     glGetFramebufferParameterivEXT =
PFNGLGETFRAMEBUFFERPARAMETERIVEXTPROC (glfwGetProcAddress("glGetFramebufferParameterivEXT"));
01697     glGetGraphicsResetStatus =

```

```
PFNGLGETGRAPHICSRESETSTATUSPROC (glfwGetProcAddress ("glGetGraphicsResetStatus"));
01698 glGetGraphicsResetStatusARB =
PFNGLGETGRAPHICSRESETSTATUSARBPROC (glfwGetProcAddress ("glGetGraphicsResetStatusARB"));
01699 glGetImageHandleARB = PFNGLGETIMAGEHANDLEARBPROC (glfwGetProcAddress ("glGetImageHandleARB"));
01700 glGetImageHandleNV = PFNGLGETIMAGEHANDLENVPROC (glfwGetProcAddress ("glGetImageHandleNV"));
01701 glGetInteger64i = PFNGLGETINTEGER64I_VPROC (glfwGetProcAddress ("glGetInteger64i_v"));
01702 glGetInteger64v = PFNGLGETINTEGER64VPROC (glfwGetProcAddress ("glGetInteger64v"));
01703 glGetIntegerIndexedvEXT =
PFNGLGETINTEGERINDEXEDVEXTPROC (glfwGetProcAddress ("glGetIntegerIndexedvEXT"));
01704 glGetIntegeri_v = PFNGLGETINTEGERI_VPROC (glfwGetProcAddress ("glGetIntegeri_v"));
01705 glGetIntegerui64i_vNV = PFNGLGETINTEGERUI64I_VNVPROC (glfwGetProcAddress ("glGetIntegerui64i_vNV"));
01706 glGetIntegerui64vNV = PFNGLGETINTEGERUI64VNVPROC (glfwGetProcAddress ("glGetIntegerui64vNV"));
01707 glGetIntegerv = PFNGLGETINTEGERVPROC (glfwGetProcAddress ("glGetIntegerv"));
01708 glGetInternalformatSampleivNV =
PFNGLGETINTERNALFORMATSAMPLEI_VNVPROC (glfwGetProcAddress ("glGetInternalformatSampleivNV"));
01709 glGetInternalformati64v =
PFNGLGETINTERNALFORMATI64VPROC (glfwGetProcAddress ("glGetInternalformati64v"));
01710 glGetInternalformativ = PFNGLGETINTERNALFORMATI_VPROC (glfwGetProcAddress ("glGetInternalformativ"));
01711 glGetMultiTexEnvfvEXT = PFNGLGETMULTITEXENVFVEXTPROC (glfwGetProcAddress ("glGetMultiTexEnvfvEXT"));
01712 glGetMultiTexEnvivEXT = PFNGLGETMULTITEXENVIVEXTPROC (glfwGetProcAddress ("glGetMultiTexEnvivEXT"));
01713 glGetMultiTexGendvEXT = PFNGLGETMULTITEXGENDVEXTPROC (glfwGetProcAddress ("glGetMultiTexGendvEXT"));
01714 glGetMultiTexGenfvEXT = PFNGLGETMULTITEXGENFVEXTPROC (glfwGetProcAddress ("glGetMultiTexGenfvEXT"));
01715 glGetMultiTexGenivEXT = PFNGLGETMULTITEXGENIVEXTPROC (glfwGetProcAddress ("glGetMultiTexGenivEXT"));
01716 glGetMultiTexImageEXT = PFNGLGETMULTITEXIMAGEEXTPROC (glfwGetProcAddress ("glGetMultiTexImageEXT"));
01717 glGetMultiTexLevelParameterfvEXT =
PFNGLGETMULTITEXLEVELPARAMETERFVEXTPROC (glfwGetProcAddress ("glGetMultiTexLevelParameterfvEXT"));
01718 glGetMultiTexLevelParameterivEXT =
PFNGLGETMULTITEXLEVELPARAMETERIVEXTPROC (glfwGetProcAddress ("glGetMultiTexLevelParameterivEXT"));
01719 glGetMultiTexParameterIivEXT =
PFNGLGETMULTITEXPARAMETERIIVEXTPROC (glfwGetProcAddress ("glGetMultiTexParameterIivEXT"));
01720 glGetMultiTexParameterIuivEXT =
PFNGLGETMULTITEXPARAMETERIUIVEXTPROC (glfwGetProcAddress ("glGetMultiTexParameterIuivEXT"));
01721 glGetMultiTexParameterfvEXT =
PFNGLGETMULTITEXPARAMETERFVEXTPROC (glfwGetProcAddress ("glGetMultiTexParameterfvEXT"));
01722 glGetMultiTexParameterivEXT =
PFNGLGETMULTITEXPARAMETERIVEXTPROC (glfwGetProcAddress ("glGetMultiTexParameterivEXT"));
01723 glGetMultisamplefv = PFNGLGETMULTISAMPLEFVPROC (glfwGetProcAddress ("glGetMultisamplefv"));
01724 glGetNamedBufferParameteri64v =
PFNGLGETNAMEDBUFFERPARAMETERI64VPROC (glfwGetProcAddress ("glGetNamedBufferParameteri64v"));
01725 glGetNamedBufferParameteriv =
PFNGLGETNAMEDBUFFERPARAMETERIVPROC (glfwGetProcAddress ("glGetNamedBufferParameteriv"));
01726 glGetNamedBufferParameterivEXT =
PFNGLGETNAMEDBUFFERPARAMETERIVEXTPROC (glfwGetProcAddress ("glGetNamedBufferParameterivEXT"));
01727 glGetNamedBufferParameterui64vNV =
PFNGLGETNAMEDBUFFERPARAMETERUI64VNVPROC (glfwGetProcAddress ("glGetNamedBufferParameterui64vNV"));
01728 glGetNamedBufferPointerv =
PFNGLGETNAMEDBUFFERPOINTERVPROC (glfwGetProcAddress ("glGetNamedBufferPointerv"));
01729 glGetNamedBufferPointervEXT =
PFNGLGETNAMEDBUFFERPOINTERVEXTPROC (glfwGetProcAddress ("glGetNamedBufferPointervEXT"));
01730 glGetNamedBufferSubData =
PFNGLGETNAMEDBUFFERSUBDATAPROC (glfwGetProcAddress ("glGetNamedBufferSubData"));
01731 glGetNamedBufferSubDataEXT =
PFNGLGETNAMEDBUFFERSUBDATAEXTPROC (glfwGetProcAddress ("glGetNamedBufferSubDataEXT"));
01732 glGetNamedFramebufferAttachmentParameteriv =
PFNGLGETNAMEDFRAMEBUFFERATTACHMENTPARAMETERIVPROC (glfwGetProcAddress ("glGetNamedFramebufferAttachmentParameteriv"));
01733 glGetNamedFramebufferAttachmentParameterivEXT =
PFNGLGETNAMEDFRAMEBUFFERATTACHMENTPARAMETERIVEXTPROC (glfwGetProcAddress ("glGetNamedFramebufferAttachmentParameterivEXT"));
01734 glGetNamedFramebufferParameteriv =
PFNGLGETNAMEDFRAMEBUFFERPARAMETERIVPROC (glfwGetProcAddress ("glGetNamedFramebufferParameteriv"));
01735 glGetNamedFramebufferParameterivEXT =
PFNGLGETNAMEDFRAMEBUFFERPARAMETERIVEXTPROC (glfwGetProcAddress ("glGetNamedFramebufferParameterivEXT"));
01736 glGetNamedProgramLocalParameterIivEXT =
PFNGLGETNAMEDPROGRAMLOCALPARAMETERIIVEXTPROC (glfwGetProcAddress ("glGetNamedProgramLocalParameterIivEXT"));
01737 glGetNamedProgramLocalParameterIuivEXT =
PFNGLGETNAMEDPROGRAMLOCALPARAMETERIUIVEXTPROC (glfwGetProcAddress ("glGetNamedProgramLocalParameterIuivEXT"));
01738 glGetNamedProgramLocalParameterdvEXT =
PFNGLGETNAMEDPROGRAMLOCALPARAMETERDVEXTPROC (glfwGetProcAddress ("glGetNamedProgramLocalParameterdvEXT"));
01739 glGetNamedProgramLocalParameterfvEXT =
PFNGLGETNAMEDPROGRAMLOCALPARAMETERFVEXTPROC (glfwGetProcAddress ("glGetNamedProgramLocalParameterfvEXT"));
01740 glGetNamedProgramStringEXT =
PFNGLGETNAMEDPROGRAMSTRINGEXTPROC (glfwGetProcAddress ("glGetNamedProgramStringEXT"));
01741 glGetNamedProgramivEXT =
PFNGLGETNAMEDPROGRAMIVEXTPROC (glfwGetProcAddress ("glGetNamedProgramivEXT"));
01742 glGetNamedRenderbufferParameteriv =
PFNGLGETNAMEDRENDERBUFFERPARAMETERIVPROC (glfwGetProcAddress ("glGetNamedRenderbufferParameteriv"));
01743 glGetNamedRenderbufferParameterivEXT =
PFNGLGETNAMEDRENDERBUFFERPARAMETERIVEXTPROC (glfwGetProcAddress ("glGetNamedRenderbufferParameterivEXT"));
01744 glGetNamedStringARB = PFNGLGETNAMEDSTRINGARBPROC (glfwGetProcAddress ("glGetNamedStringARB"));
01745 glGetNamedStringivARB = PFNGLGETNAMEDSTRINGIVARBPROC (glfwGetProcAddress ("glGetNamedStringivARB"));
01746 glGetNextPerfQueryIdINTEL =
PFNGLGETNEXTPERFQUERYIDINTELPROC (glfwGetProcAddress ("glGetNextPerfQueryIdINTEL"));
01747 glGetObjectLabel = PFNGLGETOBJECTLABELPROC (glfwGetProcAddress ("glGetObjectLabel"));
01748 glGetObjectLabelEXT = PFNGLGETOBJECTLABELEXTPROC (glfwGetProcAddress ("glGetObjectLabelEXT"));
01749 glGetObjectPtrLabel = PFNGLGETOBJECTPTRLABELPROC (glfwGetProcAddress ("glGetObjectPtrLabel"));
01750 glGetPathCommandsNV = PFNGLGETPATHCOMMANDSNVPROC (glfwGetProcAddress ("glGetPathCommandsNV"));
01751 glGetPathCoordsNV = PFNGLGETPATHCOORDSNVPROC (glfwGetProcAddress ("glGetPathCoordsNV"));
01752 glGetPathDashArrayNV = PFNGLGETPATHDASHARRAYNVPROC (glfwGetProcAddress ("glGetPathDashArrayNV"));
```



```

01753     glGetPathLengthNV = PFNGLGETPATHLENGTHNVPROC (glfwGetProcAddress("glGetPathLengthNV"));
01754     glGetPathMetricRangeNV =
PFNGLGETPATHMETRICRANGENVPROC (glfwGetProcAddress("glGetPathMetricRangeNV"));
01755     glGetPathMetricsNV = PFNGLGETPATHMETRICSNVPROC (glfwGetProcAddress("glGetPathMetricsNV"));
01756     glGetPathParameterfvNV =
PFNGLGETPATHPARAMETERFVNVPROC (glfwGetProcAddress("glGetPathParameterfvNV"));
01757     glGetPathParameterivNV =
PFNGLGETPATHPARAMETERIVNVPROC (glfwGetProcAddress("glGetPathParameterivNV"));
01758     glGetPathSpacingNV = PFNGLGETPATHSPACINGNVPROC (glfwGetProcAddress("glGetPathSpacingNV"));
01759     glGetPerfCounterInfoINTEL =
PFNGLGETPERFCOUNTERINFOINTELPROC (glfwGetProcAddress("glGetPerfCounterInfoINTEL"));
01760     glGetPerfMonitorCounterDataAMD =
PFNGLGETPERFMONITORCOUNTERDATAAMDPROC (glfwGetProcAddress("glGetPerfMonitorCounterDataAMD"));
01761     glGetPerfMonitorCounterInfoAMD =
PFNGLGETPERFMONITORCOUNTERINFOAMDPROC (glfwGetProcAddress("glGetPerfMonitorCounterInfoAMD"));
01762     glGetPerfMonitorCounterStringAMD =
PFNGLGETPERFMONITORCOUNTERSTRINGAMDPROC (glfwGetProcAddress("glGetPerfMonitorCounterStringAMD"));
01763     glGetPerfMonitorCountersAMD =
PFNGLGETPERFMONITORCOUNTERSAMDPROC (glfwGetProcAddress("glGetPerfMonitorCountersAMD"));
01764     glGetPerfMonitorGroupStringAMD =
PFNGLGETPERFMONITORGROUPSTRINGAMDPROC (glfwGetProcAddress("glGetPerfMonitorGroupStringAMD"));
01765     glGetPerfMonitorGroupsAMD =
PFNGLGETPERFMONITORGROUPSAMDPROC (glfwGetProcAddress("glGetPerfMonitorGroupsAMD"));
01766     glGetPerfQueryDataINTEL =
PFNGLGETPERFQUERYDATAINTELPROC (glfwGetProcAddress("glGetPerfQueryDataINTEL"));
01767     glGetPerfQueryIdByNameINTEL =
PFNGLGETPERFQUERYIDBYNAMEINTELPROC (glfwGetProcAddress("glGetPerfQueryIdByNameINTEL"));
01768     glGetPerfQueryInfoINTEL =
PFNGLGETPERFQUERYINFOINTELPROC (glfwGetProcAddress("glGetPerfQueryInfoINTEL"));
01769     glGetPointerIndexedvEXT =
PFNGLGETPOINTERINDEXEDVEXTPROC (glfwGetProcAddress("glGetPointerIndexedvEXT"));
01770     glGetPointerivEXT = PFNGLGETPOINTERIVEXTPROC (glfwGetProcAddress("glGetPointerivEXT"));
01771     glGetPointerv = PFNGLGETPOINTERVPROC (glfwGetProcAddress("glGetPointerv"));
01772     glGetProgramBinary = PFNGLGETPROGRAMBINARYPROC (glfwGetProcAddress("glGetProgramBinary"));
01773     glGetProgramInfoLog = PFNGLGETPROGRAMINFOLOGPROC (glfwGetProcAddress("glGetProgramInfoLog"));
01774     glGetProgramInterfaceiv =
PFNGLGETPROGRAMINTERFACEIVPROC (glfwGetProcAddress("glGetProgramInterfaceiv"));
01775     glGetProgramPipelineInfoLog =
PFNGLGETPROGRAMPIPELINEINFOLOGPROC (glfwGetProcAddress("glGetProgramPipelineInfoLog"));
01776     glGetProgramPipelineiv =
PFNGLGETPROGRAMPIPELINEIVPROC (glfwGetProcAddress("glGetProgramPipelineiv"));
01777     glGetProgramResourceIndex =
PFNGLGETPROGRAMRESOURCEINDEXPROC (glfwGetProcAddress("glGetProgramResourceIndex"));
01778     glGetProgramResourceLocation =
PFNGLGETPROGRAMRESOURCELOCATIONPROC (glfwGetProcAddress("glGetProgramResourceLocation"));
01779     glGetProgramResourceLocationIndex =
PFNGLGETPROGRAMRESOURCELOCATIONINDEXPROC (glfwGetProcAddress("glGetProgramResourceLocationIndex"));
01780     glGetProgramResourceName =
PFNGLGETPROGRAMRESOURCENAMEPROC (glfwGetProcAddress("glGetProgramResourceName"));
01781     glGetProgramResourcefNV =
PFNGLGETPROGRAMRESOURCEFVNVPROC (glfwGetProcAddress("glGetProgramResourcefNV"));
01782     glGetProgramResourceiv =
PFNGLGETPROGRAMRESOURCEIVPROC (glfwGetProcAddress("glGetProgramResourceiv"));
01783     glGetProgramStageiv = PFNGLGETPROGRAMSTAGEIVPROC (glfwGetProcAddress("glGetProgramStageiv"));
01784     glGetProgramiv = PFNGLGETPROGRAMIVPROC (glfwGetProcAddress("glGetProgramiv"));
01785     glGetQueryBufferObjecti64v =
PFNGLGETQUERYBUFFEROBJECTI64VPROC (glfwGetProcAddress("glGetQueryBufferObjecti64v"));
01786     glGetQueryBufferObjectiv =
PFNGLGETQUERYBUFFEROBJECTIVPROC (glfwGetProcAddress("glGetQueryBufferObjectiv"));
01787     glGetQueryBufferObjectui64v =
PFNGLGETQUERYBUFFEROBJECTUI64VPROC (glfwGetProcAddress("glGetQueryBufferObjectui64v"));
01788     glGetQueryBufferObjectuiv =
PFNGLGETQUERYBUFFEROBJECTUIVPROC (glfwGetProcAddress("glGetQueryBufferObjectuiv"));
01789     glGetQueryIndexediv = PFNGLGETQUERYINDEXEDIVPROC (glfwGetProcAddress("glGetQueryIndexediv"));
01790     glGetQueryObjecti64v = PFNGLGETQUERYOBJECTI64VPROC (glfwGetProcAddress("glGetQueryObjecti64v"));
01791     glGetQueryObjectiv = PFNGLGETQUERYOBJECTIVPROC (glfwGetProcAddress("glGetQueryObjectiv"));
01792     glGetQueryObjectui64v = PFNGLGETQUERYOBJECTUI64VPROC (glfwGetProcAddress("glGetQueryObjectui64v"));
01793     glGetQueryObjectuiv = PFNGLGETQUERYOBJECTUIVPROC (glfwGetProcAddress("glGetQueryObjectuiv"));
01794     glGetQueryiv = PFNGLGETQUERYIVPROC (glfwGetProcAddress("glGetQueryiv"));
01795     glGetRenderbufferParameteriv =
PFNGLGETRENDERBUFFERPARAMETERIVPROC (glfwGetProcAddress("glGetRenderbufferParameteriv"));
01796     glGetSamplerParameterIiv =
PFNGLGETSAMPLERPARAMETERIIVPROC (glfwGetProcAddress("glGetSamplerParameterIiv"));
01797     glGetSamplerParameterIuiv =
PFNGLGETSAMPLERPARAMETERUIVPROC (glfwGetProcAddress("glGetSamplerParameterIuiv"));
01798     glGetSamplerParameterfv =
PFNGLGETSAMPLERPARAMETERFVPROC (glfwGetProcAddress("glGetSamplerParameterfv"));
01799     glGetSamplerParameteriv =
PFNGLGETSAMPLERPARAMETERIVPROC (glfwGetProcAddress("glGetSamplerParameteriv"));
01800     glGetShaderInfoLog = PFNGLGETSHADERINFOLOGPROC (glfwGetProcAddress("glGetShaderInfoLog"));
01801     glGetShaderPrecisionFormat =
PFNGLGETSHADERPRECISIONFORMATPROC (glfwGetProcAddress("glGetShaderPrecisionFormat"));
01802     glGetShaderSource = PFNGLGETSHADERSOURCEPROC (glfwGetProcAddress("glGetShaderSource"));
01803     glGetShaderiv = PFNGLGETSHADERIVPROC (glfwGetProcAddress("glGetShaderiv"));
01804     glGetStageIndexNV = PFNGLGETSTAGEINDEXNVPROC (glfwGetProcAddress("glGetStageIndexNV"));
01805     glGetString = PFNGLGETSTRINGPROC (glfwGetProcAddress("glGetString"));
01806     glGetStringi = PFNGLGETSTRINGIPROC (glfwGetProcAddress("glGetStringi"));

```

```

01807     glGetSubroutineIndex = PFNGLGETSUBROUTINEINDEXPROC (glfwGetProcAddress ("glGetSubroutineIndex"));
01808     glGetSubroutineUniformLocation =
PFNGLGETSUBROUTINEUNIFORMLOCATIONPROC (glfwGetProcAddress ("glGetSubroutineUniformLocation"));
01809     glGetSynciv = PFNGLGETSYNCIVPROC (glfwGetProcAddress ("glGetSynciv"));
01810     glGetTexImage = PFNGLGETTEXTUREIMAGEPROC (glfwGetProcAddress ("glGetTexImage"));
01811     glGetTexLevelParameterfiv =
PFNGLGETTEXTURELEVELPARAMETERFVPROC (glfwGetProcAddress ("glGetTexLevelParameterfiv"));
01812     glGetTexLevelParameteriv =
PFNGLGETTEXTURELEVELPARAMETERIVPROC (glfwGetProcAddress ("glGetTexLevelParameteriv"));
01813     glGetTexParameterIiv = PFNGLGETTEXPARAMETERIIVPROC (glfwGetProcAddress ("glGetTexParameterIiv"));
01814     glGetTexParameterIuiv = PFNGLGETTEXPARAMETERIUIVPROC (glfwGetProcAddress ("glGetTexParameterIuiv"));
01815     glGetTexParameterfiv = PFNGLGETTEXPARAMETERFVPROC (glfwGetProcAddress ("glGetTexParameterfiv"));
01816     glGetTexParameteriv = PFNGLGETTEXPARAMETERIVPROC (glfwGetProcAddress ("glGetTexParameteriv"));
01817     glGetTextureHandleARB = PFNGLGETTEXTUREHANDLEARBPROC (glfwGetProcAddress ("glGetTextureHandleARB"));
01818     glGetTextureHandleNV = PFNGLGETTEXTUREHANDLENVPROC (glfwGetProcAddress ("glGetTextureHandleNV"));
01819     glGetTextureImage = PFNGLGETTEXTUREIMAGEPROC (glfwGetProcAddress ("glGetTextureImage"));
01820     glGetTextureImageEXT = PFNGLGETTEXTUREIMAGEEXTPROC (glfwGetProcAddress ("glGetTextureImageEXT"));
01821     glGetTextureLevelParameterfiv =
PFNGLGETTEXTURELEVELPARAMETERFVPROC (glfwGetProcAddress ("glGetTextureLevelParameterfiv"));
01822     glGetTextureLevelParameterfivEXT =
PFNGLGETTEXTURELEVELPARAMETERFVEXTPROC (glfwGetProcAddress ("glGetTextureLevelParameterfivEXT"));
01823     glGetTextureLevelParameteriv =
PFNGLGETTEXTURELEVELPARAMETERIVPROC (glfwGetProcAddress ("glGetTextureLevelParameteriv"));
01824     glGetTextureLevelParameterivEXT =
PFNGLGETTEXTURELEVELPARAMETERIVEXTPROC (glfwGetProcAddress ("glGetTextureLevelParameterivEXT"));
01825     glGetTextureParameterIiv =
PFNGLGETTEXTUREPARAMETERIIVPROC (glfwGetProcAddress ("glGetTextureParameterIiv"));
01826     glGetTextureParameterIivEXT =
PFNGLGETTEXTUREPARAMETERIIVEXTPROC (glfwGetProcAddress ("glGetTextureParameterIivEXT"));
01827     glGetTextureParameterIuiv =
PFNGLGETTEXTUREPARAMETERIUIVPROC (glfwGetProcAddress ("glGetTextureParameterIuiv"));
01828     glGetTextureParameterIuivEXT =
PFNGLGETTEXTUREPARAMETERIUIVEXTPROC (glfwGetProcAddress ("glGetTextureParameterIuivEXT"));
01829     glGetTextureParameterfiv =
PFNGLGETTEXTUREPARAMETERFVPROC (glfwGetProcAddress ("glGetTextureParameterfiv"));
01830     glGetTextureParameterfivEXT =
PFNGLGETTEXTUREPARAMETERFVEXTPROC (glfwGetProcAddress ("glGetTextureParameterfivEXT"));
01831     glGetTextureParameteriv =
PFNGLGETTEXTUREPARAMETERIVPROC (glfwGetProcAddress ("glGetTextureParameteriv"));
01832     glGetTextureParameterivEXT =
PFNGLGETTEXTUREPARAMETERIVEXTPROC (glfwGetProcAddress ("glGetTextureParameterivEXT"));
01833     glGetTextureSamplerHandleARB =
PFNGLGETTEXTURESAMPLERHANDLEARBPROC (glfwGetProcAddress ("glGetTextureSamplerHandleARB"));
01834     glGetTextureSamplerHandleNV =
PFNGLGETTEXTURESAMPLERHANDLENVPROC (glfwGetProcAddress ("glGetTextureSamplerHandleNV"));
01835     glGetTextureSubImage = PFNGLGETTEXTURESUBIMAGEPROC (glfwGetProcAddress ("glGetTextureSubImage"));
01836     glGetTransformFeedbackVarying =
PFNGLGETTRANSFORMFEEDBACKVARYINGPROC (glfwGetProcAddress ("glGetTransformFeedbackVarying"));
01837     glGetTransformFeedbacki64_v =
PFNGLGETTRANSFORMFEEDBACKI64_VPROC (glfwGetProcAddress ("glGetTransformFeedbacki64_v"));
01838     glGetTransformFeedbacki_v =
PFNGLGETTRANSFORMFEEDBACKI_VPROC (glfwGetProcAddress ("glGetTransformFeedbacki_v"));
01839     glGetTransformFeedbackiv =
PFNGLGETTRANSFORMFEEDBACKIVPROC (glfwGetProcAddress ("glGetTransformFeedbackiv"));
01840     glGetUniformBlockIndex =
PFNGLGETUNIFORMBLOCKINDEXPROC (glfwGetProcAddress ("glGetUniformBlockIndex"));
01841     glGetUniformIndices = PFNGLGETUNIFORMINDICESPROC (glfwGetProcAddress ("glGetUniformIndices"));
01842     glGetUniformLocation = PFNGLGETUNIFORMLOCATIONPROC (glfwGetProcAddress ("glGetUniformLocation"));
01843     glGetUniformSubroutineuiv =
PFNGLGETUNIFORMSUBROUTINEUIVPROC (glfwGetProcAddress ("glGetUniformSubroutineuiv"));
01844     glGetUniformdv = PFNGLGETUNIFORMDVPROC (glfwGetProcAddress ("glGetUniformdv"));
01845     glGetUniformfv = PFNGLGETUNIFORMFVPROC (glfwGetProcAddress ("glGetUniformfv"));
01846     glGetUniformi64vARB = PFNGLGETUNIFORMI64VARBPROC (glfwGetProcAddress ("glGetUniformi64vARB"));
01847     glGetUniformi64vNV = PFNGLGETUNIFORMI64VNVPROC (glfwGetProcAddress ("glGetUniformi64vNV"));
01848     glGetUniformiv = PFNGLGETUNIFORMIVPROC (glfwGetProcAddress ("glGetUniformiv"));
01849     glGetUniformui64vARB = PFNGLGETUNIFORMUI64VARBPROC (glfwGetProcAddress ("glGetUniformui64vARB"));
01850     glGetUniformui64vNV = PFNGLGETUNIFORMUI64VNVPROC (glfwGetProcAddress ("glGetUniformui64vNV"));
01851     glGetUniformuiv = PFNGLGETUNIFORMUIVPROC (glfwGetProcAddress ("glGetUniformuiv"));
01852     glGetVertexArrayIndexed64iv =
PFNGLGETVERTEXARRAYINDEXED64IVPROC (glfwGetProcAddress ("glGetVertexArrayIndexed64iv"));
01853     glGetVertexArrayIndexediv =
PFNGLGETVERTEXARRAYINDEXEDIVPROC (glfwGetProcAddress ("glGetVertexArrayIndexediv"));
01854     glGetVertexArrayIntegeri_vEXT =
PFNGLGETVERTEXARRAYINTEGERI_VEXTPROC (glfwGetProcAddress ("glGetVertexArrayIntegeri_vEXT"));
01855     glGetVertexArrayIntegerivEXT =
PFNGLGETVERTEXARRAYINTEGERVEXTPROC (glfwGetProcAddress ("glGetVertexArrayIntegerivEXT"));
01856     glGetVertexArrayPointeri_vEXT =
PFNGLGETVERTEXARRAYPOINTERI_VEXTPROC (glfwGetProcAddress ("glGetVertexArrayPointeri_vEXT"));
01857     glGetVertexArrayPointervEXT =
PFNGLGETVERTEXARRAYPOINTERVEXTPROC (glfwGetProcAddress ("glGetVertexArrayPointervEXT"));
01858     glGetVertexArrayiv = PFNGLGETVERTEXARRAYIVPROC (glfwGetProcAddress ("glGetVertexArrayiv"));
01859     glGetVertexAttribIiv = PFNGLGETVERTEXATTRIBIIVPROC (glfwGetProcAddress ("glGetVertexAttribIiv"));
01860     glGetVertexAttribIuiv = PFNGLGETVERTEXATTRIBUIVPROC (glfwGetProcAddress ("glGetVertexAttribIuiv"));
01861     glGetVertexAttribLdv = PFNGLGETVERTEXATTRIBLDVPROC (glfwGetProcAddress ("glGetVertexAttribLdv"));
01862     glGetVertexAttribLi64vNV =
PFNGLGETVERTEXATTRIBLI64VNVPROC (glfwGetProcAddress ("glGetVertexAttribLi64vNV"));
01863     glGetVertexAttribLui64vARB =

```

```

    PFNGLGETVERTEXATTRIBLUI64VARBPROC (glfwGetProcAddress("glGetVertexAttribLui64vARB"));
01864   glGetVertexAttribLui64vNV =
    PFNGLGETVERTEXATTRIBLUI64VNVPROC (glfwGetProcAddress("glGetVertexAttribLui64vNV"));
01865   glGetVertexAttribPointerv =
    PFNGLGETVERTEXATTRIBPOINTERVPROC (glfwGetProcAddress("glGetVertexAttribPointerv"));
01866   glGetVertexAttribdv = PFNGLGETVERTEXATTRIBDVPROC (glfwGetProcAddress("glGetVertexAttribdv"));
01867   glGetVertexAttribfv = PFNGLGETVERTEXATTRIBFVPROC (glfwGetProcAddress("glGetVertexAttribfv"));
01868   glGetVertexAttribiv = PFNGLGETVERTEXATTRIBIVPROC (glfwGetProcAddress("glGetVertexAttribiv"));
01869   glGetVkProcAddrNV = PFNGLGETVKPROCADDRNVPROC (glfwGetProcAddress("glGetVkProcAddrNV"));
01870   glGetnCompressedTexImage =
    PFNGLGETNCOMPRESSEDTEXIMAGEPROC (glfwGetProcAddress("glGetnCompressedTexImage"));
01871   glGetnCompressedTexImageARB =
    PFNGLGETNCOMPRESSEDTEXIMAGEARBPROC (glfwGetProcAddress("glGetnCompressedTexImageARB"));
01872   glGetnTexImage = PFNGLGETNTEXIMAGEPROC (glfwGetProcAddress("glGetnTexImage"));
01873   glGetnTexImageARB = PFNGLGETNTEXIMAGEARBPROC (glfwGetProcAddress("glGetnTexImageARB"));
01874   glGetnUniformdv = PFNGLGETNUNIFORMDVPROC (glfwGetProcAddress("glGetnUniformdv"));
01875   glGetnUniformdvARB = PFNGLGETNUNIFORMDVARBPROC (glfwGetProcAddress("glGetnUniformdvARB"));
01876   glGetnUniformfv = PFNGLGETNUNIFORMFVPROC (glfwGetProcAddress("glGetnUniformfv"));
01877   glGetnUniformfvARB = PFNGLGETNUNIFORMFVARBPROC (glfwGetProcAddress("glGetnUniformfvARB"));
01878   glGetnUniformi64vARB = PFNGLGETNUNIFORMI64VARBPROC (glfwGetProcAddress("glGetnUniformi64vARB"));
01879   glGetnUniformiv = PFNGLGETNUNIFORMIVPROC (glfwGetProcAddress("glGetnUniformiv"));
01880   glGetnUniformivARB = PFNGLGETNUNIFORMIVARBPROC (glfwGetProcAddress("glGetnUniformivARB"));
01881   glGetnUniformui64vARB = PFNGLGETNUNIFORMUI64VARBPROC (glfwGetProcAddress("glGetnUniformui64vARB"));
01882   glGetnUniformuiv = PFNGLGETNUNIFORMUIVPROC (glfwGetProcAddress("glGetnUniformuiv"));
01883   glGetnUniformuivARB = PFNGLGETNUNIFORMUIVARBPROC (glfwGetProcAddress("glGetnUniformuivARB"));
01884   glHint = PFNGLHINTPROC (glfwGetProcAddress("glHint"));
01885   glIndexFormatNV = PFNGLINDEXFORMATNVPROC (glfwGetProcAddress("glIndexFormatNV"));
01886   glInsertEventMarkerEXT =
    PFNGLINSERTEVENTMARKEREXTPROC (glfwGetProcAddress("glInsertEventMarkerEXT"));
01887   glInterpolatePathsNV = PFNGLINTERPOLATEPATHSNVPROC (glfwGetProcAddress("glInterpolatePathsNV"));
01888   glInvalidateBufferData =
    PFNGLINVALIDATEBUFFERDATAPROC (glfwGetProcAddress("glInvalidateBufferData"));
01889   glInvalidateBufferSubData =
    PFNGLINVALIDATEBUFFERSUBDATAPROC (glfwGetProcAddress("glInvalidateBufferSubData"));
01890   glInvalidateFramebuffer =
    PFNGLINVALIDATEFRAMEBUFFERPROC (glfwGetProcAddress("glInvalidateFramebuffer"));
01891   glInvalidateNamedFramebufferData =
    PFNGLINVALIDATENAMEDFRAMEBUFFERDATAPROC (glfwGetProcAddress("glInvalidateNamedFramebufferData"));
01892   glInvalidateNamedFramebufferSubData =
    PFNGLINVALIDATENAMEDFRAMEBUFFERSUBDATAPROC (glfwGetProcAddress("glInvalidateNamedFramebufferSubData"));
01893   glInvalidateSubFramebuffer =
    PFNGLINVALIDATESUBFRAMEBUFFERPROC (glfwGetProcAddress("glInvalidateSubFramebuffer"));
01894   glInvalidateTexImage = PFNGLINVALIDATETEXIMAGEPROC (glfwGetProcAddress("glInvalidateTexImage"));
01895   glInvalidateTexSubImage =
    PFNGLINVALIDATETEXSUBIMAGEPROC (glfwGetProcAddress("glInvalidateTexSubImage"));
01896   glIsBuffer = PFNGLISBUFFERPROC (glfwGetProcAddress("glIsBuffer"));
01897   glIsBufferResidentNV = PFNGLISBUFFERRESIDENTNVPROC (glfwGetProcAddress("glIsBufferResidentNV"));
01898   glIsCommandListNV = PFNGLISCOMMANDLISTNVPROC (glfwGetProcAddress("glIsCommandListNV"));
01899   glIsEnabled = PFNGLISENABLEDPROC (glfwGetProcAddress("glIsEnabled"));
01900   glIsEnabledIndexedEXT = PFNGLISENABLEDINDEXEDEXTPROC (glfwGetProcAddress("glIsEnabledIndexedEXT"));
01901   glIsEnabledi = PFNGLISENABLEDIPROC (glfwGetProcAddress("glIsEnabledi"));
01902   glIsFramebuffer = PFNGLISFRAMEBUFFERPROC (glfwGetProcAddress("glIsFramebuffer"));
01903   glIsImageHandleResidentARB =
    PFNGLISIMAGEHANDLERESIDENTARBPROC (glfwGetProcAddress("glIsImageHandleResidentARB"));
01904   glIsImageHandleResidentNV =
    PFNGLISIMAGEHANDLERESIDENTNVPROC (glfwGetProcAddress("glIsImageHandleResidentNV"));
01905   glIsNamedBufferResidentNV =
    PFNGLISNAMEDBUFFERRESIDENTNVPROC (glfwGetProcAddress("glIsNamedBufferResidentNV"));
01906   glIsNamedStringARB = PFNGLISNAMEDSTRINGARBPROC (glfwGetProcAddress("glIsNamedStringARB"));
01907   glIsPathNV = PFNGLISPATHNVPROC (glfwGetProcAddress("glIsPathNV"));
01908   glIsPointInFillPathNV = PFNGLISPOINTINFILLPATHNVPROC (glfwGetProcAddress("glIsPointInFillPathNV"));
01909   glIsPointInStrokePathNV =
    PFNGLISPOINTINSTROKEPATHNVPROC (glfwGetProcAddress("glIsPointInStrokePathNV"));
01910   glIsProgram = PFNGLISPROGRAMPROC (glfwGetProcAddress("glIsProgram"));
01911   glIsProgramPipeline = PFNGLISPROGRAMPIPELINEPROC (glfwGetProcAddress("glIsProgramPipeline"));
01912   glIsQuery = PFNGLISQUERYPROC (glfwGetProcAddress("glIsQuery"));
01913   glIsRenderbuffer = PFNGLISRENDERBUFFERPROC (glfwGetProcAddress("glIsRenderbuffer"));
01914   glIsSampler = PFNGLISSAMPLERPROC (glfwGetProcAddress("glIsSampler"));
01915   glIsShader = PFNGLISSHADERPROC (glfwGetProcAddress("glIsShader"));
01916   glIsStateNV = PFNGLISSTATENVPROC (glfwGetProcAddress("glIsStateNV"));
01917   glIsSync = PFNGLISSYNCPROC (glfwGetProcAddress("glIsSync"));
01918   glIsTexture = PFNGLISTEXTUREPROC (glfwGetProcAddress("glIsTexture"));
01919   glIsTextureHandleResidentARB =
    PFNGLISTEXTUREHANDLERESIDENTARBPROC (glfwGetProcAddress("glIsTextureHandleResidentARB"));
01920   glIsTextureHandleResidentNV =
    PFNGLISTEXTUREHANDLERESIDENTNVPROC (glfwGetProcAddress("glIsTextureHandleResidentNV"));
01921   glIsTransformFeedback = PFNGLISTRANSFORMFEEDBACKPROC (glfwGetProcAddress("glIsTransformFeedback"));
01922   glIsVertexArray = PFNGLISVERTEXARRAYPROC (glfwGetProcAddress("glIsVertexArray"));
01923   glLabelObjectEXT = PFNGLLABELOBJECTEXTPROC (glfwGetProcAddress("glLabelObjectEXT"));
01924   glLineWidth = PFNGLLINEWIDTHPROC (glfwGetProcAddress("glLineWidth"));
01925   glLinkProgram = PFNGLLINKPROGRAMPROC (glfwGetProcAddress("glLinkProgram"));
01926   glListDrawCommandsStatesClientNV =
    PFNGLLISTDRAWCOMMANDSSTATESCLIENTNVPROC (glfwGetProcAddress("glListDrawCommandsStatesClientNV"));
01927   glLogicOp = PFNGLLOGICOPPROC (glfwGetProcAddress("glLogicOp"));
01928   glMakeBufferNonResidentNV =
    PFNGLMAKEBUFFERNONRESIDENTNVPROC (glfwGetProcAddress("glMakeBufferNonResidentNV"));
01929   glMakeBufferResidentNV =

```

```
PFNGLMAKEBUFFERRESIDENTNVPROC (glfwGetProcAddress ("glMakeBufferResidentNV"));
01930 glMakeImageHandleNonResidentARB =
PFNGLMAKEIMAGEHANDLENONRESIDENTARBPROC (glfwGetProcAddress ("glMakeImageHandleNonResidentARB"));
01931 glMakeImageHandleNonResidentNV =
PFNGLMAKEIMAGEHANDLENONRESIDENTNVPROC (glfwGetProcAddress ("glMakeImageHandleNonResidentNV"));
01932 glMakeImageHandleResidentARB =
PFNGLMAKEIMAGEHANDLERESIDENTARBPROC (glfwGetProcAddress ("glMakeImageHandleResidentARB"));
01933 glMakeImageHandleResidentNV =
PFNGLMAKEIMAGEHANDLERESIDENTNVPROC (glfwGetProcAddress ("glMakeImageHandleResidentNV"));
01934 glMakeNamedBufferNonResidentNV =
PFNGLMAKENAMEDBUFFERNONRESIDENTNVPROC (glfwGetProcAddress ("glMakeNamedBufferNonResidentNV"));
01935 glMakeNamedBufferResidentNV =
PFNGLMAKENAMEDBUFFERRESIDENTNVPROC (glfwGetProcAddress ("glMakeNamedBufferResidentNV"));
01936 glMakeTextureHandleNonResidentARB =
PFNGLMAKETEXTUREHANDLENONRESIDENTARBPROC (glfwGetProcAddress ("glMakeTextureHandleNonResidentARB"));
01937 glMakeTextureHandleNonResidentNV =
PFNGLMAKETEXTUREHANDLENONRESIDENTNVPROC (glfwGetProcAddress ("glMakeTextureHandleNonResidentNV"));
01938 glMakeTextureHandleResidentARB =
PFNGLMAKETEXTUREHANDLERESIDENTARBPROC (glfwGetProcAddress ("glMakeTextureHandleResidentARB"));
01939 glMakeTextureHandleResidentNV =
PFNGLMAKETEXTUREHANDLERESIDENTNVPROC (glfwGetProcAddress ("glMakeTextureHandleResidentNV"));
01940 glMapBuffer = PFNGLMAPBUFFERPROC (glfwGetProcAddress ("glMapBuffer"));
01941 glMapBufferRange = PFNGLMAPBUFFERRANGEPROC (glfwGetProcAddress ("glMapBufferRange"));
01942 glMapNamedBuffer = PFNGLMAPNAMEDBUFFERPROC (glfwGetProcAddress ("glMapNamedBuffer"));
01943 glMapNamedBufferEXT = PFNGLMAPNAMEDBUFFEREXTPROC (glfwGetProcAddress ("glMapNamedBufferEXT"));
01944 glMapNamedBufferRange = PFNGLMAPNAMEDBUFFERRANGEPROC (glfwGetProcAddress ("glMapNamedBufferRange"));
01945 glMapNamedBufferRangeEXT =
PFNGLMAPNAMEDBUFFERRANGEEXTPROC (glfwGetProcAddress ("glMapNamedBufferRangeEXT"));
01946 glMatrixFrustumEXT = PFNGLMATRIXFRUSTUMEXTPROC (glfwGetProcAddress ("glMatrixFrustumEXT"));
01947 glMatrixLoad3x2fNV = PFNGLMATRIXLOAD3X2FNVPROC (glfwGetProcAddress ("glMatrixLoad3x2fNV"));
01948 glMatrixLoad3x3fNV = PFNGLMATRIXLOAD3X3FNVPROC (glfwGetProcAddress ("glMatrixLoad3x3fNV"));
01949 glMatrixLoadIdentityEXT =
PFNGLMATRIXLOADIDENTITYEXTPROC (glfwGetProcAddress ("glMatrixLoadIdentityEXT"));
01950 glMatrixLoadTranspose3x3fNV =
PFNGLMATRIXLOADTRANSPOSE3X3FNVPROC (glfwGetProcAddress ("glMatrixLoadTranspose3x3fNV"));
01951 glMatrixLoadTransposedEXT =
PFNGLMATRIXLOADTRANPOSEDEXTPROC (glfwGetProcAddress ("glMatrixLoadTransposedEXT"));
01952 glMatrixLoadTransposefEXT =
PFNGLMATRIXLOADTRANPOSEFEEXTPROC (glfwGetProcAddress ("glMatrixLoadTransposefEXT"));
01953 glMatrixLoaddeEXT = PFNGLMATRIXLOADDEXTPROC (glfwGetProcAddress ("glMatrixLoaddeEXT"));
01954 glMatrixLoadfEXT = PFNGLMATRIXLOADFEEXTPROC (glfwGetProcAddress ("glMatrixLoadfEXT"));
01955 glMatrixMult3x2fNV = PFNGLMATRIXMULT3X2FNVPROC (glfwGetProcAddress ("glMatrixMult3x2fNV"));
01956 glMatrixMult3x3fNV = PFNGLMATRIXMULT3X3FNVPROC (glfwGetProcAddress ("glMatrixMult3x3fNV"));
01957 glMatrixMultTranspose3x3fNV =
PFNGLMATRIXMULTTRANPOSE3X3FNVPROC (glfwGetProcAddress ("glMatrixMultTranspose3x3fNV"));
01958 glMatrixMultTransposedEXT =
PFNGLMATRIXMULTTRANPOSEDEXTPROC (glfwGetProcAddress ("glMatrixMultTransposedEXT"));
01959 glMatrixMultTransposefEXT =
PFNGLMATRIXMULTTRANPOSEFEEXTPROC (glfwGetProcAddress ("glMatrixMultTransposefEXT"));
01960 glMatrixMultdeEXT = PFNGLMATRIXMULTDEXTPROC (glfwGetProcAddress ("glMatrixMultdeEXT"));
01961 glMatrixMultfEXT = PFNGLMATRIXMULTFEEXTPROC (glfwGetProcAddress ("glMatrixMultfEXT"));
01962 glMatrixOrthoEXT = PFNGLMATRIXORTHOEXTPROC (glfwGetProcAddress ("glMatrixOrthoEXT"));
01963 glMatrixPopEXT = PFNGLMATRIXPOPEXTPROC (glfwGetProcAddress ("glMatrixPopEXT"));
01964 glMatrixPushEXT = PFNGLMATRIXPUSHEXTPROC (glfwGetProcAddress ("glMatrixPushEXT"));
01965 glMatrixRotatedEXT = PFNGLMATRIXROTATEDEXTPROC (glfwGetProcAddress ("glMatrixRotatedEXT"));
01966 glMatrixRotatefEXT = PFNGLMATRIXROTATEFEEXTPROC (glfwGetProcAddress ("glMatrixRotatefEXT"));
01967 glMatrixScaledEXT = PFNGLMATRIXSCALEDEXTPROC (glfwGetProcAddress ("glMatrixScaledEXT"));
01968 glMatrixScalefEXT = PFNGLMATRIXSCALEFEEXTPROC (glfwGetProcAddress ("glMatrixScalefEXT"));
01969 glMatrixTranslatedEXT = PFNGLMATRIXTRANSLATEDEXTPROC (glfwGetProcAddress ("glMatrixTranslatedEXT"));
01970 glMatrixTranslatefEXT = PFNGLMATRIXTRANSLATEFEEXTPROC (glfwGetProcAddress ("glMatrixTranslatefEXT"));
01971 glMaxShaderCompilerThreadsARB =
PFNGLMAXSHADERCOMPILERTHREADSARBPROC (glfwGetProcAddress ("glMaxShaderCompilerThreadsARB"));
01972 glMemoryBarrier = PFNGLMEMORYBARRIERPROC (glfwGetProcAddress ("glMemoryBarrier"));
01973 glMemoryBarrierByRegion =
PFNGLMEMORYBARRIERBYREGIONPROC (glfwGetProcAddress ("glMemoryBarrierByRegion"));
01974 glMinSampleShading = PFNGLMINSAMPLESHADINGPROC (glfwGetProcAddress ("glMinSampleShading"));
01975 glMinSampleShadingARB = PFNGLMINSAMPLESHADINGARBPROC (glfwGetProcAddress ("glMinSampleShadingARB"));
01976 glMultiDrawArrays = PFNGLMULTIDRAWARRAYSPROC (glfwGetProcAddress ("glMultiDrawArrays"));
01977 glMultiDrawArraysIndirect =
PFNGLMULTIDRAWARRAYSINDIRECTPROC (glfwGetProcAddress ("glMultiDrawArraysIndirect"));
01978 glMultiDrawArraysIndirectBindlessCountNV =
PFNGLMULTIDRAWARRAYSINDIRECTBINDLESSCOUNTNVPROC (glfwGetProcAddress ("glMultiDrawArraysIndirectBindlessCountNV"));
01979 glMultiDrawArraysIndirectBindlessNV =
PFNGLMULTIDRAWARRAYSINDIRECTBINDLESSNVPROC (glfwGetProcAddress ("glMultiDrawArraysIndirectBindlessNV"));
01980 glMultiDrawArraysIndirectCountARB =
PFNGLMULTIDRAWARRAYSINDIRECTCOUNTARBPROC (glfwGetProcAddress ("glMultiDrawArraysIndirectCountARB"));
01981 glMultiDrawElements = PFNGLMULTIDRAWELEMENTSPROC (glfwGetProcAddress ("glMultiDrawElements"));
01982 glMultiDrawElementsBaseVertex =
PFNGLMULTIDRAWELEMENTSBASEVERTEXPROC (glfwGetProcAddress ("glMultiDrawElementsBaseVertex"));
01983 glMultiDrawElementsIndirect =
PFNGLMULTIDRAWELEMENTSINDIRECTPROC (glfwGetProcAddress ("glMultiDrawElementsIndirect"));
01984 glMultiDrawElementsIndirectBindlessCountNV =
PFNGLMULTIDRAWELEMENTSINDIRECTBINDLESSCOUNTNVPROC (glfwGetProcAddress ("glMultiDrawElementsIndirectBindlessCountNV"));
01985 glMultiDrawElementsIndirectBindlessNV =
PFNGLMULTIDRAWELEMENTSINDIRECTBINDLESSNVPROC (glfwGetProcAddress ("glMultiDrawElementsIndirectBindlessNV"));
01986 glMultiDrawElementsIndirectCountARB =
PFNGLMULTIDRAWELEMENTSINDIRECTCOUNTARBPROC (glfwGetProcAddress ("glMultiDrawElementsIndirectCountARB"));
```



```

01987     glMultiTexBufferEXT = PFNGLMULTITEXBUFFEREXTPROC (glfwGetProcAddress("glMultiTexBufferEXT"));
01988     glMultiTexCoordPointerEXT =
PFNGLMULTITEXCOORDPOINTEREXTPROC (glfwGetProcAddress("glMultiTexCoordPointerEXT"));
01989     glMultiTexEnvfEXT = PFNGLMULTITEXENVFEXTPROC (glfwGetProcAddress("glMultiTexEnvfEXT"));
01990     glMultiTexEnvfvEXT = PFNGLMULTITEXENVFVEXTPROC (glfwGetProcAddress("glMultiTexEnvfvEXT"));
01991     glMultiTexEnviEXT = PFNGLMULTITEXENVIEXTPROC (glfwGetProcAddress("glMultiTexEnviEXT"));
01992     glMultiTexEnvivEXT = PFNGLMULTITEXENVIVEXTPROC (glfwGetProcAddress("glMultiTexEnvivEXT"));
01993     glMultiTexGendEXT = PFNGLMULTITEXGENDEXTPROC (glfwGetProcAddress("glMultiTexGendEXT"));
01994     glMultiTexGendvEXT = PFNGLMULTITEXGENDVEXTPROC (glfwGetProcAddress("glMultiTexGendvEXT"));
01995     glMultiTexGenfEXT = PFNGLMULTITEXGENFEXTPROC (glfwGetProcAddress("glMultiTexGenfEXT"));
01996     glMultiTexGenfvEXT = PFNGLMULTITEXGENFVEXTPROC (glfwGetProcAddress("glMultiTexGenfvEXT"));
01997     glMultiTexGeniEXT = PFNGLMULTITEXGENIEXTPROC (glfwGetProcAddress("glMultiTexGeniEXT"));
01998     glMultiTexGenivEXT = PFNGLMULTITEXGENIVEXTPROC (glfwGetProcAddress("glMultiTexGenivEXT"));
01999     glMultiTexImage1DEXT = PFNGLMULTITEXIMAGE1DEXTPROC (glfwGetProcAddress("glMultiTexImage1DEXT"));
02000     glMultiTexImage2DEXT = PFNGLMULTITEXIMAGE2DEXTPROC (glfwGetProcAddress("glMultiTexImage2DEXT"));
02001     glMultiTexImage3DEXT = PFNGLMULTITEXIMAGE3DEXTPROC (glfwGetProcAddress("glMultiTexImage3DEXT"));
02002     glMultiTexParameterIivEXT =
PFNGLMULTITEXPARAMETERIIVEXTPROC (glfwGetProcAddress("glMultiTexParameterIivEXT"));
02003     glMultiTexParameterIuivEXT =
PFNGLMULTITEXPARAMETERIUIVEXTPROC (glfwGetProcAddress("glMultiTexParameterIuivEXT"));
02004     glMultiTexParameterfEXT =
PFNGLMULTITEXPARAMETERFEXTPROC (glfwGetProcAddress("glMultiTexParameterfEXT"));
02005     glMultiTexParameterfvEXT =
PFNGLMULTITEXPARAMETERFVEXTPROC (glfwGetProcAddress("glMultiTexParameterfvEXT"));
02006     glMultiTexParameteriEXT =
PFNGLMULTITEXPARAMETERIEXTPROC (glfwGetProcAddress("glMultiTexParameteriEXT"));
02007     glMultiTexParameterivEXT =
PFNGLMULTITEXPARAMETERIVEXTPROC (glfwGetProcAddress("glMultiTexParameterivEXT"));
02008     glMultiTexRenderbufferEXT =
PFNGLMULTITEXRENDERBUFFEREXTPROC (glfwGetProcAddress("glMultiTexRenderbufferEXT"));
02009     glMultiTexSubImage1DEXT =
PFNGLMULTITEXSUBIMAGE1DEXTPROC (glfwGetProcAddress("glMultiTexSubImage1DEXT"));
02010     glMultiTexSubImage2DEXT =
PFNGLMULTITEXSUBIMAGE2DEXTPROC (glfwGetProcAddress("glMultiTexSubImage2DEXT"));
02011     glMultiTexSubImage3DEXT =
PFNGLMULTITEXSUBIMAGE3DEXTPROC (glfwGetProcAddress("glMultiTexSubImage3DEXT"));
02012     glNamedBufferData = PFNGLNAMEDBUFFERDATAPROC (glfwGetProcAddress("glNamedBufferData"));
02013     glNamedBufferDataEXT = PFNGLNAMEDBUFFERDATAEXTPROC (glfwGetProcAddress("glNamedBufferDataEXT"));
02014     glNamedBufferPageCommitmentARB =
PFNGLNAMEDBUFFERPAGECOMMITMENTARBPROC (glfwGetProcAddress("glNamedBufferPageCommitmentARB"));
02015     glNamedBufferPageCommitmentEXT =
PFNGLNAMEDBUFFERPAGECOMMITMENTEXTPROC (glfwGetProcAddress("glNamedBufferPageCommitmentEXT"));
02016     glNamedBufferStorage = PFNGLNAMEDBUFFERSTORAGEPROC (glfwGetProcAddress("glNamedBufferStorage"));
02017     glNamedBufferStorageEXT =
PFNGLNAMEDBUFFERSTORAGEEXTPROC (glfwGetProcAddress("glNamedBufferStorageEXT"));
02018     glNamedBufferSubData = PFNGLNAMEDBUFFERSUBDATAPROC (glfwGetProcAddress("glNamedBufferSubData"));
02019     glNamedBufferSubDataEXT =
PFNGLNAMEDBUFFERSUBDATAEXTPROC (glfwGetProcAddress("glNamedBufferSubDataEXT"));
02020     glNamedCopyBufferSubDataEXT =
PFNGLNAMEDCOPYBUFFERSUBDATAEXTPROC (glfwGetProcAddress("glNamedCopyBufferSubDataEXT"));
02021     glNamedFramebufferDrawBuffer =
PFNGLNAMEDFRAMEBUFFERDRAWBUFFERPROC (glfwGetProcAddress("glNamedFramebufferDrawBuffer"));
02022     glNamedFramebufferDrawBuffers =
PFNGLNAMEDFRAMEBUFFERDRAWBUFFERSPROC (glfwGetProcAddress("glNamedFramebufferDrawBuffers"));
02023     glNamedFramebufferParameteri =
PFNGLNAMEDFRAMEBUFFERPARAMETERIPROC (glfwGetProcAddress("glNamedFramebufferParameteri"));
02024     glNamedFramebufferParameteriEXT =
PFNGLNAMEDFRAMEBUFFERPARAMETERIEXTPROC (glfwGetProcAddress("glNamedFramebufferParameteriEXT"));
02025     glNamedFramebufferReadBuffer =
PFNGLNAMEDFRAMEBUFFERREADBUFFERPROC (glfwGetProcAddress("glNamedFramebufferReadBuffer"));
02026     glNamedFramebufferRenderbuffer =
PFNGLNAMEDFRAMEBUFFERRENDERBUFFERPROC (glfwGetProcAddress("glNamedFramebufferRenderbuffer"));
02027     glNamedFramebufferRenderbufferEXT =
PFNGLNAMEDFRAMEBUFFERRENDERBUFFEREXTPROC (glfwGetProcAddress("glNamedFramebufferRenderbufferEXT"));
02028     glNamedFramebufferSampleLocationsfvARB =
PFNGLNAMEDFRAMEBUFFERSAMPLELOCATIONSFVARBPROC (glfwGetProcAddress("glNamedFramebufferSampleLocationsfvARB"));
02029     glNamedFramebufferSampleLocationsfvNV =
PFNGLNAMEDFRAMEBUFFERSAMPLELOCATIONSFVNVPROC (glfwGetProcAddress("glNamedFramebufferSampleLocationsfvNV"));
02030     glNamedFramebufferTexture =
PFNGLNAMEDFRAMEBUFFERTEXTUREPROC (glfwGetProcAddress("glNamedFramebufferTexture"));
02031     glNamedFramebufferTexture1DEXT =
PFNGLNAMEDFRAMEBUFFERTEXTURE1DEXTPROC (glfwGetProcAddress("glNamedFramebufferTexture1DEXT"));
02032     glNamedFramebufferTexture2DEXT =
PFNGLNAMEDFRAMEBUFFERTEXTURE2DEXTPROC (glfwGetProcAddress("glNamedFramebufferTexture2DEXT"));
02033     glNamedFramebufferTexture3DEXT =
PFNGLNAMEDFRAMEBUFFERTEXTURE3DEXTPROC (glfwGetProcAddress("glNamedFramebufferTexture3DEXT"));
02034     glNamedFramebufferTextureEXT =
PFNGLNAMEDFRAMEBUFFERTEXTUREEXTPROC (glfwGetProcAddress("glNamedFramebufferTextureEXT"));
02035     glNamedFramebufferTextureFaceEXT =
PFNGLNAMEDFRAMEBUFFERTEXTUREFACEEXTPROC (glfwGetProcAddress("glNamedFramebufferTextureFaceEXT"));
02036     glNamedFramebufferTextureLayer =
PFNGLNAMEDFRAMEBUFFERTEXTURELAYERPROC (glfwGetProcAddress("glNamedFramebufferTextureLayer"));
02037     glNamedFramebufferTextureLayerEXT =
PFNGLNAMEDFRAMEBUFFERTEXTURELAYEREXTPROC (glfwGetProcAddress("glNamedFramebufferTextureLayerEXT"));
02038     glNamedProgramLocalParameter4dEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETER4DEXTPROC (glfwGetProcAddress("glNamedProgramLocalParameter4dEXT"));
02039     glNamedProgramLocalParameter4dvEXT =

```

```

PFNGLNAMEDPROGRAMLOCALPARAMETER4DVEXTPROC (glfwGetProcAddress("glNamedProgramLocalParameter4dvEXT"));
02040 glNamedProgramLocalParameter4fEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETER4FEXTPROC (glfwGetProcAddress("glNamedProgramLocalParameter4fEXT"));
02041 glNamedProgramLocalParameter4fvEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETER4FVEXTPROC (glfwGetProcAddress("glNamedProgramLocalParameter4fvEXT"));
02042 glNamedProgramLocalParameterI4iEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETERI4IEXTPROC (glfwGetProcAddress("glNamedProgramLocalParameterI4iEXT"));
02043 glNamedProgramLocalParameterI4ivEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETERI4IVEXTPROC (glfwGetProcAddress("glNamedProgramLocalParameterI4ivEXT"));
02044 glNamedProgramLocalParameterI4uiEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETERI4UIEXTPROC (glfwGetProcAddress("glNamedProgramLocalParameterI4uiEXT"));
02045 glNamedProgramLocalParameterI4uivEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETERI4UIVEXTPROC (glfwGetProcAddress("glNamedProgramLocalParameterI4uivEXT"));
02046 glNamedProgramLocalParameters4fvEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETERS4FVEXTPROC (glfwGetProcAddress("glNamedProgramLocalParameters4fvEXT"));
02047 glNamedProgramLocalParametersI4ivEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETERSI4IVEXTPROC (glfwGetProcAddress("glNamedProgramLocalParametersI4ivEXT"));
02048 glNamedProgramLocalParametersI4uivEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETERSI4UIVEXTPROC (glfwGetProcAddress("glNamedProgramLocalParametersI4uivEXT"));
02049 glNamedProgramStringEXT =
PFNGLNAMEDPROGRAMSTRINGEXTPROC (glfwGetProcAddress("glNamedProgramStringEXT"));
02050 glNamedRenderbufferStorage =
PFNGLNAMEDRENDERBUFFERSTORAGEPROC (glfwGetProcAddress("glNamedRenderbufferStorage"));
02051 glNamedRenderbufferStorageEXT =
PFNGLNAMEDRENDERBUFFERSTORAGEEXTPROC (glfwGetProcAddress("glNamedRenderbufferStorageEXT"));
02052 glNamedRenderbufferStorageMultisample =
PFNGLNAMEDRENDERBUFFERSTORAGEMULTISAMPLEPROC (glfwGetProcAddress("glNamedRenderbufferStorageMultisample"));
02053 glNamedRenderbufferStorageMultisampleCoverageEXT =
PFNGLNAMEDRENDERBUFFERSTORAGEMULTISAMPLECOVERAGEEXTPROC (glfwGetProcAddress("glNamedRenderbufferStorageMultisampleCoverageEXT"));
02054 glNamedRenderbufferStorageMultisampleEXT =
PFNGLNAMEDRENDERBUFFERSTORAGEMULTISAMPLEEXTPROC (glfwGetProcAddress("glNamedRenderbufferStorageMultisampleEXT"));
02055 glNamedStringARB = PFNGLNAMEDSTRINGARBPROC (glfwGetProcAddress("glNamedStringARB"));
02056 glNormalFormatNV = PFNGLNORMALFORMATNVPROC (glfwGetProcAddress("glNormalFormatNV"));
02057 glObjectLabel = PFNGLOBJECTLABELPROC (glfwGetProcAddress("glObjectLabel"));
02058 glObjectPtrLabel = PFNGLOBJECTPTRLABELPROC (glfwGetProcAddress("glObjectPtrLabel"));
02059 glPatchParameterfv = PFNGLPATCHPARAMETERFVPROC (glfwGetProcAddress("glPatchParameterfv"));
02060 glPatchParameteri = PFNGLPATCHPARAMETERIPROC (glfwGetProcAddress("glPatchParameteri"));
02061 glPathCommandsNV = PFNGLPATHCOMMANDSNVPROC (glfwGetProcAddress("glPathCommandsNV"));
02062 glPathCoordsNV = PFNGLPATHCOORDSNVPROC (glfwGetProcAddress("glPathCoordsNV"));
02063 glPathCoverDepthFuncNV =
PFNGLPATHCOVERDEPTHFUNCNVPROC (glfwGetProcAddress("glPathCoverDepthFuncNV"));
02064 glPathDashArrayNV = PFNGLPATHDASHARRAYNVPROC (glfwGetProcAddress("glPathDashArrayNV"));
02065 glPathGlyphIndexArrayNV =
PFNGLPATHGLYPHINDEXARRAYNVPROC (glfwGetProcAddress("glPathGlyphIndexArrayNV"));
02066 glPathGlyphIndexRangeNV =
PFNGLPATHGLYPHINDEXRANGENVPROC (glfwGetProcAddress("glPathGlyphIndexRangeNV"));
02067 glPathGlyphRangeNV = PFNGLPATHGLYPHRANGENVPROC (glfwGetProcAddress("glPathGlyphRangeNV"));
02068 glPathGlyphsNV = PFNGLPATHGLYPHSNVPROC (glfwGetProcAddress("glPathGlyphsNV"));
02069 glPathMemoryGlyphIndexArrayNV =
PFNGLPATHMEMORYGLYPHINDEXARRAYNVPROC (glfwGetProcAddress("glPathMemoryGlyphIndexArrayNV"));
02070 glPathParameterfNV = PFNGLPATHPARAMETERFNVPROC (glfwGetProcAddress("glPathParameterfNV"));
02071 glPathParameterfvNV = PFNGLPATHPARAMETERFVNVPROC (glfwGetProcAddress("glPathParameterfvNV"));
02072 glPathParameteriNV = PFNGLPATHPARAMETERINVPROC (glfwGetProcAddress("glPathParameteriNV"));
02073 glPathParameterivNV = PFNGLPATHPARAMETERIVNVPROC (glfwGetProcAddress("glPathParameterivNV"));
02074 glPathStencilDepthOffsetNV =
PFNGLPATHSTENCILDEPTHOFFSETNVPROC (glfwGetProcAddress("glPathStencilDepthOffsetNV"));
02075 glPathStencilFuncNV = PFNGLPATHSTENCILFUNCNVPROC (glfwGetProcAddress("glPathStencilFuncNV"));
02076 glPathStringNV = PFNGLPATHSTRINGNVPROC (glfwGetProcAddress("glPathStringNV"));
02077 glPathSubCommandsNV = PFNGLPATHSUBCOMMANDSNVPROC (glfwGetProcAddress("glPathSubCommandsNV"));
02078 glPathSubCoordsNV = PFNGLPATHSUBCOORDSNVPROC (glfwGetProcAddress("glPathSubCoordsNV"));
02079 glPauseTransformFeedback =
PFNGLPAUSETRANSFORMFEEDBACKPROC (glfwGetProcAddress("glPauseTransformFeedback"));
02080 glPixelStoref = PFNGLPIXELSTOREFPROC (glfwGetProcAddress("glPixelStoref"));
02081 glPixelStorei = PFNGLPIXELSTOREIPROC (glfwGetProcAddress("glPixelStorei"));
02082 glPointAlongPathNV = PFNGLPOINTALONGPATHNVPROC (glfwGetProcAddress("glPointAlongPathNV"));
02083 glPointParameterf = PFNGLPOINTPARAMETERFNVPROC (glfwGetProcAddress("glPointParameterf"));
02084 glPointParameterfv = PFNGLPOINTPARAMETERFVPROC (glfwGetProcAddress("glPointParameterfv"));
02085 glPointParameteri = PFNGLPOINTPARAMETERIPROC (glfwGetProcAddress("glPointParameteri"));
02086 glPointParameteriv = PFNGLPOINTPARAMETERIVPROC (glfwGetProcAddress("glPointParameteriv"));
02087 glPointSize = PFNGLPOINTSIZEPROC (glfwGetProcAddress("glPointSize"));
02088 glPolygonMode = PFNGLPOLYGONMODEPROC (glfwGetProcAddress("glPolygonMode"));
02089 glPolygonOffset = PFNGLPOLYGONOFFSETPROC (glfwGetProcAddress("glPolygonOffset"));
02090 glPolygonOffsetClampEXT =
PFNGLPOLYGONOFFSETCLAMPEXTPROC (glfwGetProcAddress("glPolygonOffsetClampEXT"));
02091 glPopDebugGroup = PFNGLPOPDEBUGGROUPPROC (glfwGetProcAddress("glPopDebugGroup"));
02092 glPopGroupMarkerEXT = PFNGLPOPGROUPMARKEREXTPROC (glfwGetProcAddress("glPopGroupMarkerEXT"));
02093 glPrimitiveBoundingBoxARB =
PFNGLPRIMITIVEBOUNDINGBOXARBPROC (glfwGetProcAddress("glPrimitiveBoundingBoxARB"));
02094 glPrimitiveRestartIndex =
PFNGLPRIMITIVERESTARTINDEXPROC (glfwGetProcAddress("glPrimitiveRestartIndex"));
02095 glProgramBinary = PFNGLPROGRAMBINARYPROC (glfwGetProcAddress("glProgramBinary"));
02096 glProgramParameteri = PFNGLPROGRAMPARAMETERIPROC (glfwGetProcAddress("glProgramParameteri"));
02097 glProgramParameteriARB =
PFNGLPROGRAMPARAMETERIARBPROC (glfwGetProcAddress("glProgramParameteriARB"));
02098 glProgramPathFragmentInputGenNV =
PFNGLPROGRAMPATHFRAGMENTINPUTGENNVPROC (glfwGetProcAddress("glProgramPathFragmentInputGenNV"));
02099 glProgramUniformId = PFNGLPROGRAMUNIFORMIDPROC (glfwGetProcAddress("glProgramUniformId"));

```

```

02100     glProgramUniformldEXT = PFNGLPROGRAMUNIFORM1DTEXTPROC (glfwGetProcAddress ("glProgramUniformldEXT"));
02101     glProgramUniformldv = PFNGLPROGRAMUNIFORM1DVPROC (glfwGetProcAddress ("glProgramUniformldv"));
02102     glProgramUniformldvEXT =
PFNGLPROGRAMUNIFORM1DVEXTPROC (glfwGetProcAddress ("glProgramUniformldvEXT"));
02103     glProgramUniform1f = PFNGLPROGRAMUNIFORM1FPROC (glfwGetProcAddress ("glProgramUniform1f"));
02104     glProgramUniform1fEXT = PFNGLPROGRAMUNIFORM1FEXTPROC (glfwGetProcAddress ("glProgramUniform1fEXT"));
02105     glProgramUniform1fv = PFNGLPROGRAMUNIFORM1FVPROC (glfwGetProcAddress ("glProgramUniform1fv"));
02106     glProgramUniform1fvEXT =
PFNGLPROGRAMUNIFORM1FVEXTPROC (glfwGetProcAddress ("glProgramUniform1fvEXT"));
02107     glProgramUniform1i = PFNGLPROGRAMUNIFORM1IPROC (glfwGetProcAddress ("glProgramUniform1i"));
02108     glProgramUniform1i64ARB =
PFNGLPROGRAMUNIFORM1I64ARBPROC (glfwGetProcAddress ("glProgramUniform1i64ARB"));
02109     glProgramUniform1i64NV =
PFNGLPROGRAMUNIFORM1I64NVPROC (glfwGetProcAddress ("glProgramUniform1i64NV"));
02110     glProgramUniform1i64vARB =
PFNGLPROGRAMUNIFORM1I64VARBPROC (glfwGetProcAddress ("glProgramUniform1i64vARB"));
02111     glProgramUniform1i64vNV =
PFNGLPROGRAMUNIFORM1I64VNVPROC (glfwGetProcAddress ("glProgramUniform1i64vNV"));
02112     glProgramUniform1iEXT = PFNGLPROGRAMUNIFORM1IEXTPROC (glfwGetProcAddress ("glProgramUniform1iEXT"));
02113     glProgramUniform1iv = PFNGLPROGRAMUNIFORM1IVPROC (glfwGetProcAddress ("glProgramUniform1iv"));
02114     glProgramUniform1ivEXT =
PFNGLPROGRAMUNIFORM1IVEXTPROC (glfwGetProcAddress ("glProgramUniform1ivEXT"));
02115     glProgramUniform1ui = PFNGLPROGRAMUNIFORM1UIPROC (glfwGetProcAddress ("glProgramUniform1ui"));
02116     glProgramUniform1ui64ARB =
PFNGLPROGRAMUNIFORM1UI64ARBPROC (glfwGetProcAddress ("glProgramUniform1ui64ARB"));
02117     glProgramUniform1ui64NV =
PFNGLPROGRAMUNIFORM1UI64NVPROC (glfwGetProcAddress ("glProgramUniform1ui64NV"));
02118     glProgramUniform1ui64vARB =
PFNGLPROGRAMUNIFORM1UI64VARBPROC (glfwGetProcAddress ("glProgramUniform1ui64vARB"));
02119     glProgramUniform1ui64vNV =
PFNGLPROGRAMUNIFORM1UI64VNVPROC (glfwGetProcAddress ("glProgramUniform1ui64vNV"));
02120     glProgramUniform1uiEXT =
PFNGLPROGRAMUNIFORM1UIEXTPROC (glfwGetProcAddress ("glProgramUniform1uiEXT"));
02121     glProgramUniform1uiv = PFNGLPROGRAMUNIFORM1UIVPROC (glfwGetProcAddress ("glProgramUniform1uiv"));
02122     glProgramUniform1uivEXT =
PFNGLPROGRAMUNIFORM1UIVEXTPROC (glfwGetProcAddress ("glProgramUniform1uivEXT"));
02123     glProgramUniform2d = PFNGLPROGRAMUNIFORM2DPROC (glfwGetProcAddress ("glProgramUniform2d"));
02124     glProgramUniform2dEXT = PFNGLPROGRAMUNIFORM2DTEXTPROC (glfwGetProcAddress ("glProgramUniform2dEXT"));
02125     glProgramUniform2dv = PFNGLPROGRAMUNIFORM2DVPROC (glfwGetProcAddress ("glProgramUniform2dv"));
02126     glProgramUniform2dvEXT =
PFNGLPROGRAMUNIFORM2DVEXTPROC (glfwGetProcAddress ("glProgramUniform2dvEXT"));
02127     glProgramUniform2f = PFNGLPROGRAMUNIFORM2FPROC (glfwGetProcAddress ("glProgramUniform2f"));
02128     glProgramUniform2fEXT = PFNGLPROGRAMUNIFORM2FEXTPROC (glfwGetProcAddress ("glProgramUniform2fEXT"));
02129     glProgramUniform2fv = PFNGLPROGRAMUNIFORM2FVPROC (glfwGetProcAddress ("glProgramUniform2fv"));
02130     glProgramUniform2fvEXT =
PFNGLPROGRAMUNIFORM2FVEXTPROC (glfwGetProcAddress ("glProgramUniform2fvEXT"));
02131     glProgramUniform2i = PFNGLPROGRAMUNIFORM2IPROC (glfwGetProcAddress ("glProgramUniform2i"));
02132     glProgramUniform2i64ARB =
PFNGLPROGRAMUNIFORM2I64ARBPROC (glfwGetProcAddress ("glProgramUniform2i64ARB"));
02133     glProgramUniform2i64NV =
PFNGLPROGRAMUNIFORM2I64NVPROC (glfwGetProcAddress ("glProgramUniform2i64NV"));
02134     glProgramUniform2i64vARB =
PFNGLPROGRAMUNIFORM2I64VARBPROC (glfwGetProcAddress ("glProgramUniform2i64vARB"));
02135     glProgramUniform2i64vNV =
PFNGLPROGRAMUNIFORM2I64VNVPROC (glfwGetProcAddress ("glProgramUniform2i64vNV"));
02136     glProgramUniform2iEXT = PFNGLPROGRAMUNIFORM2IEXTPROC (glfwGetProcAddress ("glProgramUniform2iEXT"));
02137     glProgramUniform2iv = PFNGLPROGRAMUNIFORM2IVPROC (glfwGetProcAddress ("glProgramUniform2iv"));
02138     glProgramUniform2ivEXT =
PFNGLPROGRAMUNIFORM2IVEXTPROC (glfwGetProcAddress ("glProgramUniform2ivEXT"));
02139     glProgramUniform2ui = PFNGLPROGRAMUNIFORM2UIPROC (glfwGetProcAddress ("glProgramUniform2ui"));
02140     glProgramUniform2ui64ARB =
PFNGLPROGRAMUNIFORM2UI64ARBPROC (glfwGetProcAddress ("glProgramUniform2ui64ARB"));
02141     glProgramUniform2ui64NV =
PFNGLPROGRAMUNIFORM2UI64NVPROC (glfwGetProcAddress ("glProgramUniform2ui64NV"));
02142     glProgramUniform2ui64vARB =
PFNGLPROGRAMUNIFORM2UI64VARBPROC (glfwGetProcAddress ("glProgramUniform2ui64vARB"));
02143     glProgramUniform2ui64vNV =
PFNGLPROGRAMUNIFORM2UI64VNVPROC (glfwGetProcAddress ("glProgramUniform2ui64vNV"));
02144     glProgramUniform2uiEXT =
PFNGLPROGRAMUNIFORM2UIEXTPROC (glfwGetProcAddress ("glProgramUniform2uiEXT"));
02145     glProgramUniform2uiv = PFNGLPROGRAMUNIFORM2UIVPROC (glfwGetProcAddress ("glProgramUniform2uiv"));
02146     glProgramUniform2uivEXT =
PFNGLPROGRAMUNIFORM2UIVEXTPROC (glfwGetProcAddress ("glProgramUniform2uivEXT"));
02147     glProgramUniform3d = PFNGLPROGRAMUNIFORM3DPROC (glfwGetProcAddress ("glProgramUniform3d"));
02148     glProgramUniform3dEXT = PFNGLPROGRAMUNIFORM3DTEXTPROC (glfwGetProcAddress ("glProgramUniform3dEXT"));
02149     glProgramUniform3dv = PFNGLPROGRAMUNIFORM3DVPROC (glfwGetProcAddress ("glProgramUniform3dv"));
02150     glProgramUniform3dvEXT =
PFNGLPROGRAMUNIFORM3DVEXTPROC (glfwGetProcAddress ("glProgramUniform3dvEXT"));
02151     glProgramUniform3f = PFNGLPROGRAMUNIFORM3FPROC (glfwGetProcAddress ("glProgramUniform3f"));
02152     glProgramUniform3fEXT = PFNGLPROGRAMUNIFORM3FEXTPROC (glfwGetProcAddress ("glProgramUniform3fEXT"));
02153     glProgramUniform3fv = PFNGLPROGRAMUNIFORM3FVPROC (glfwGetProcAddress ("glProgramUniform3fv"));
02154     glProgramUniform3fvEXT =
PFNGLPROGRAMUNIFORM3FVEXTPROC (glfwGetProcAddress ("glProgramUniform3fvEXT"));
02155     glProgramUniform3i = PFNGLPROGRAMUNIFORM3IPROC (glfwGetProcAddress ("glProgramUniform3i"));
02156     glProgramUniform3i64ARB =
PFNGLPROGRAMUNIFORM3I64ARBPROC (glfwGetProcAddress ("glProgramUniform3i64ARB"));
02157     glProgramUniform3i64NV =

```

```
PFNGLPROGRAMUNIFORM3I64NVPROC (glfwGetProcAddress ("glProgramUniform3i64NV"));
02158 glProgramUniform3i64vARB =
PFNGLPROGRAMUNIFORM3I64VARBPROC (glfwGetProcAddress ("glProgramUniform3i64vARB"));
02159 glProgramUniform3i64vNV =
PFNGLPROGRAMUNIFORM3I64VNVPROC (glfwGetProcAddress ("glProgramUniform3i64vNV"));
02160 glProgramUniform3iEXT = PFNGLPROGRAMUNIFORM3IEXTPROC (glfwGetProcAddress ("glProgramUniform3iEXT"));
02161 glProgramUniform3iv = PFNGLPROGRAMUNIFORM3IVPROC (glfwGetProcAddress ("glProgramUniform3iv"));
02162 glProgramUniform3ivEXT =
PFNGLPROGRAMUNIFORM3IVEXTPROC (glfwGetProcAddress ("glProgramUniform3ivEXT"));
02163 glProgramUniform3ui = PFNGLPROGRAMUNIFORM3UIPROC (glfwGetProcAddress ("glProgramUniform3ui"));
02164 glProgramUniform3ui64ARB =
PFNGLPROGRAMUNIFORM3UI64ARBPROC (glfwGetProcAddress ("glProgramUniform3ui64ARB"));
02165 glProgramUniform3ui64NV =
PFNGLPROGRAMUNIFORM3UI64NVPROC (glfwGetProcAddress ("glProgramUniform3ui64NV"));
02166 glProgramUniform3ui64vARB =
PFNGLPROGRAMUNIFORM3UI64VARBPROC (glfwGetProcAddress ("glProgramUniform3ui64vARB"));
02167 glProgramUniform3ui64vNV =
PFNGLPROGRAMUNIFORM3UI64VNVPROC (glfwGetProcAddress ("glProgramUniform3ui64vNV"));
02168 glProgramUniform3uiEXT =
PFNGLPROGRAMUNIFORM3UIEXTPROC (glfwGetProcAddress ("glProgramUniform3uiEXT"));
02169 glProgramUniform3uiv = PFNGLPROGRAMUNIFORM3UIVPROC (glfwGetProcAddress ("glProgramUniform3uiv"));
02170 glProgramUniform3uivEXT =
PFNGLPROGRAMUNIFORM3UIVEXTPROC (glfwGetProcAddress ("glProgramUniform3uivEXT"));
02171 glProgramUniform4d = PFNGLPROGRAMUNIFORM4DPROC (glfwGetProcAddress ("glProgramUniform4d"));
02172 glProgramUniform4dEXT = PFNGLPROGRAMUNIFORM4DEXTPROC (glfwGetProcAddress ("glProgramUniform4dEXT"));
02173 glProgramUniform4dv = PFNGLPROGRAMUNIFORM4DVPROC (glfwGetProcAddress ("glProgramUniform4dv"));
02174 glProgramUniform4dvEXT =
PFNGLPROGRAMUNIFORM4DVEXTPROC (glfwGetProcAddress ("glProgramUniform4dvEXT"));
02175 glProgramUniform4f = PFNGLPROGRAMUNIFORM4FPROC (glfwGetProcAddress ("glProgramUniform4f"));
02176 glProgramUniform4fEXT = PFNGLPROGRAMUNIFORM4FEXTPROC (glfwGetProcAddress ("glProgramUniform4fEXT"));
02177 glProgramUniform4fv = PFNGLPROGRAMUNIFORM4FVPROC (glfwGetProcAddress ("glProgramUniform4fv"));
02178 glProgramUniform4fvEXT =
PFNGLPROGRAMUNIFORM4FVEXTPROC (glfwGetProcAddress ("glProgramUniform4fvEXT"));
02179 glProgramUniform4i = PFNGLPROGRAMUNIFORM4IPROC (glfwGetProcAddress ("glProgramUniform4i"));
02180 glProgramUniform4i64ARB =
PFNGLPROGRAMUNIFORM4I64ARBPROC (glfwGetProcAddress ("glProgramUniform4i64ARB"));
02181 glProgramUniform4i64NV =
PFNGLPROGRAMUNIFORM4I64NVPROC (glfwGetProcAddress ("glProgramUniform4i64NV"));
02182 glProgramUniform4i64vARB =
PFNGLPROGRAMUNIFORM4I64VARBPROC (glfwGetProcAddress ("glProgramUniform4i64vARB"));
02183 glProgramUniform4i64vNV =
PFNGLPROGRAMUNIFORM4I64VNVPROC (glfwGetProcAddress ("glProgramUniform4i64vNV"));
02184 glProgramUniform4iEXT = PFNGLPROGRAMUNIFORM4IEXTPROC (glfwGetProcAddress ("glProgramUniform4iEXT"));
02185 glProgramUniform4iv = PFNGLPROGRAMUNIFORM4IVPROC (glfwGetProcAddress ("glProgramUniform4iv"));
02186 glProgramUniform4ivEXT =
PFNGLPROGRAMUNIFORM4IVEXTPROC (glfwGetProcAddress ("glProgramUniform4ivEXT"));
02187 glProgramUniform4ui = PFNGLPROGRAMUNIFORM4UIPROC (glfwGetProcAddress ("glProgramUniform4ui"));
02188 glProgramUniform4ui64ARB =
PFNGLPROGRAMUNIFORM4UI64ARBPROC (glfwGetProcAddress ("glProgramUniform4ui64ARB"));
02189 glProgramUniform4ui64NV =
PFNGLPROGRAMUNIFORM4UI64NVPROC (glfwGetProcAddress ("glProgramUniform4ui64NV"));
02190 glProgramUniform4ui64vARB =
PFNGLPROGRAMUNIFORM4UI64VARBPROC (glfwGetProcAddress ("glProgramUniform4ui64vARB"));
02191 glProgramUniform4ui64vNV =
PFNGLPROGRAMUNIFORM4UI64VNVPROC (glfwGetProcAddress ("glProgramUniform4ui64vNV"));
02192 glProgramUniform4uiEXT =
PFNGLPROGRAMUNIFORM4UIEXTPROC (glfwGetProcAddress ("glProgramUniform4uiEXT"));
02193 glProgramUniform4uiv = PFNGLPROGRAMUNIFORM4UIVPROC (glfwGetProcAddress ("glProgramUniform4uiv"));
02194 glProgramUniform4uivEXT =
PFNGLPROGRAMUNIFORM4UIVEXTPROC (glfwGetProcAddress ("glProgramUniform4uivEXT"));
02195 glProgramUniformHandleui64ARB =
PFNGLPROGRAMUNIFORMHANDLEUI64ARBPROC (glfwGetProcAddress ("glProgramUniformHandleui64ARB"));
02196 glProgramUniformHandleui64NV =
PFNGLPROGRAMUNIFORMHANDLEUI64NVPROC (glfwGetProcAddress ("glProgramUniformHandleui64NV"));
02197 glProgramUniformHandleui64vARB =
PFNGLPROGRAMUNIFORMHANDLEUI64VARBPROC (glfwGetProcAddress ("glProgramUniformHandleui64vARB"));
02198 glProgramUniformHandleui64vNV =
PFNGLPROGRAMUNIFORMHANDLEUI64VNVPROC (glfwGetProcAddress ("glProgramUniformHandleui64vNV"));
02199 glProgramUniformMatrix2dv =
PFNGLPROGRAMUNIFORMMATRIX2DVPROC (glfwGetProcAddress ("glProgramUniformMatrix2dv"));
02200 glProgramUniformMatrix2dvEXT =
PFNGLPROGRAMUNIFORMMATRIX2DVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix2dvEXT"));
02201 glProgramUniformMatrix2fv =
PFNGLPROGRAMUNIFORMMATRIX2FVPROC (glfwGetProcAddress ("glProgramUniformMatrix2fv"));
02202 glProgramUniformMatrix2fvEXT =
PFNGLPROGRAMUNIFORMMATRIX2FVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix2fvEXT"));
02203 glProgramUniformMatrix2x3dv =
PFNGLPROGRAMUNIFORMMATRIX2X3DVPROC (glfwGetProcAddress ("glProgramUniformMatrix2x3dv"));
02204 glProgramUniformMatrix2x3dvEXT =
PFNGLPROGRAMUNIFORMMATRIX2X3DVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix2x3dvEXT"));
02205 glProgramUniformMatrix2x3fv =
PFNGLPROGRAMUNIFORMMATRIX2X3FVPROC (glfwGetProcAddress ("glProgramUniformMatrix2x3fv"));
02206 glProgramUniformMatrix2x3fvEXT =
PFNGLPROGRAMUNIFORMMATRIX2X3FVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix2x3fvEXT"));
02207 glProgramUniformMatrix2x4dv =
PFNGLPROGRAMUNIFORMMATRIX2X4DVPROC (glfwGetProcAddress ("glProgramUniformMatrix2x4dv"));
02208 glProgramUniformMatrix2x4dvEXT =
```



```

    PFNGLPROGRAMUNIFORMMATRIX2X4DVECTPROC (glfwGetProcAddress ("glProgramUniformMatrix2x4dvEXT"));
02209   glProgramUniformMatrix2x4fv =
    PFNGLPROGRAMUNIFORMMATRIX2X4FVPROC (glfwGetProcAddress ("glProgramUniformMatrix2x4fv"));
02210   glProgramUniformMatrix2x4fvEXT =
    PFNGLPROGRAMUNIFORMMATRIX2X4FVECTPROC (glfwGetProcAddress ("glProgramUniformMatrix2x4fvEXT"));
02211   glProgramUniformMatrix3dv =
    PFNGLPROGRAMUNIFORMMATRIX3DVPROC (glfwGetProcAddress ("glProgramUniformMatrix3dv"));
02212   glProgramUniformMatrix3dvEXT =
    PFNGLPROGRAMUNIFORMMATRIX3DVECTPROC (glfwGetProcAddress ("glProgramUniformMatrix3dvEXT"));
02213   glProgramUniformMatrix3fv =
    PFNGLPROGRAMUNIFORMMATRIX3FVPROC (glfwGetProcAddress ("glProgramUniformMatrix3fv"));
02214   glProgramUniformMatrix3fvEXT =
    PFNGLPROGRAMUNIFORMMATRIX3FVECTPROC (glfwGetProcAddress ("glProgramUniformMatrix3fvEXT"));
02215   glProgramUniformMatrix3x2dv =
    PFNGLPROGRAMUNIFORMMATRIX3X2DVPROC (glfwGetProcAddress ("glProgramUniformMatrix3x2dv"));
02216   glProgramUniformMatrix3x2dvEXT =
    PFNGLPROGRAMUNIFORMMATRIX3X2DVECTPROC (glfwGetProcAddress ("glProgramUniformMatrix3x2dvEXT"));
02217   glProgramUniformMatrix3x2fv =
    PFNGLPROGRAMUNIFORMMATRIX3X2FVPROC (glfwGetProcAddress ("glProgramUniformMatrix3x2fv"));
02218   glProgramUniformMatrix3x2fvEXT =
    PFNGLPROGRAMUNIFORMMATRIX3X2FVECTPROC (glfwGetProcAddress ("glProgramUniformMatrix3x2fvEXT"));
02219   glProgramUniformMatrix3x4dv =
    PFNGLPROGRAMUNIFORMMATRIX3X4DVPROC (glfwGetProcAddress ("glProgramUniformMatrix3x4dv"));
02220   glProgramUniformMatrix3x4dvEXT =
    PFNGLPROGRAMUNIFORMMATRIX3X4DVECTPROC (glfwGetProcAddress ("glProgramUniformMatrix3x4dvEXT"));
02221   glProgramUniformMatrix3x4fv =
    PFNGLPROGRAMUNIFORMMATRIX3X4FVPROC (glfwGetProcAddress ("glProgramUniformMatrix3x4fv"));
02222   glProgramUniformMatrix3x4fvEXT =
    PFNGLPROGRAMUNIFORMMATRIX3X4FVECTPROC (glfwGetProcAddress ("glProgramUniformMatrix3x4fvEXT"));
02223   glProgramUniformMatrix4dv =
    PFNGLPROGRAMUNIFORMMATRIX4DVPROC (glfwGetProcAddress ("glProgramUniformMatrix4dv"));
02224   glProgramUniformMatrix4dvEXT =
    PFNGLPROGRAMUNIFORMMATRIX4DVECTPROC (glfwGetProcAddress ("glProgramUniformMatrix4dvEXT"));
02225   glProgramUniformMatrix4fv =
    PFNGLPROGRAMUNIFORMMATRIX4FVPROC (glfwGetProcAddress ("glProgramUniformMatrix4fv"));
02226   glProgramUniformMatrix4fvEXT =
    PFNGLPROGRAMUNIFORMMATRIX4FVECTPROC (glfwGetProcAddress ("glProgramUniformMatrix4fvEXT"));
02227   glProgramUniformMatrix4x2dv =
    PFNGLPROGRAMUNIFORMMATRIX4X2DVPROC (glfwGetProcAddress ("glProgramUniformMatrix4x2dv"));
02228   glProgramUniformMatrix4x2dvEXT =
    PFNGLPROGRAMUNIFORMMATRIX4X2DVECTPROC (glfwGetProcAddress ("glProgramUniformMatrix4x2dvEXT"));
02229   glProgramUniformMatrix4x2fv =
    PFNGLPROGRAMUNIFORMMATRIX4X2FVPROC (glfwGetProcAddress ("glProgramUniformMatrix4x2fv"));
02230   glProgramUniformMatrix4x2fvEXT =
    PFNGLPROGRAMUNIFORMMATRIX4X2FVECTPROC (glfwGetProcAddress ("glProgramUniformMatrix4x2fvEXT"));
02231   glProgramUniformMatrix4x3dv =
    PFNGLPROGRAMUNIFORMMATRIX4X3DVPROC (glfwGetProcAddress ("glProgramUniformMatrix4x3dv"));
02232   glProgramUniformMatrix4x3dvEXT =
    PFNGLPROGRAMUNIFORMMATRIX4X3DVECTPROC (glfwGetProcAddress ("glProgramUniformMatrix4x3dvEXT"));
02233   glProgramUniformMatrix4x3fv =
    PFNGLPROGRAMUNIFORMMATRIX4X3FVPROC (glfwGetProcAddress ("glProgramUniformMatrix4x3fv"));
02234   glProgramUniformMatrix4x3fvEXT =
    PFNGLPROGRAMUNIFORMMATRIX4X3FVECTPROC (glfwGetProcAddress ("glProgramUniformMatrix4x3fvEXT"));
02235   glProgramUniformi64NV =
    PFNGLPROGRAMUNIFORMUI64NVPROC (glfwGetProcAddress ("glProgramUniformui64NV"));
02236   glProgramUniformui64vNV =
    PFNGLPROGRAMUNIFORMUI64VNVPROC (glfwGetProcAddress ("glProgramUniformui64vNV"));
02237   glProvokingVertex = PFNGLPROVOKINGVERTEXPROC (glfwGetProcAddress ("glProvokingVertex"));
02238   glPushClientAttribDefaultEXT =
    PFNGLPUSHCLIENTATTRIBDEFAULTEXTPROC (glfwGetProcAddress ("glPushClientAttribDefaultEXT"));
02239   glPushDebugGroup = PFNGLPUSHDEBUGGROUPPROC (glfwGetProcAddress ("glPushDebugGroup"));
02240   glPushGroupMarkerEXT = PFNGLPUSHGROUPMARKEREXTPROC (glfwGetProcAddress ("glPushGroupMarkerEXT"));
02241   glQueryCounter = PFNGLQUERYCOUNTERPROC (glfwGetProcAddress ("glQueryCounter"));
02242   glRasterSamplesEXT = PFNGLRASTERSAMPLESEXTPROC (glfwGetProcAddress ("glRasterSamplesEXT"));
02243   glReadBuffer = PFNGLREADBUFFERPROC (glfwGetProcAddress ("glReadBuffer"));
02244   glReadPixels = PFNGLREADPIXELSPROC (glfwGetProcAddress ("glReadPixels"));
02245   glReadnPixels = PFNGLREADNPIXELSPROC (glfwGetProcAddress ("glReadnPixels"));
02246   glReadnPixelsARB = PFNGLREADNPIXELSARBPROC (glfwGetProcAddress ("glReadnPixelsARB"));
02247   glReleaseShaderCompiler =
    PFNGLRELEASESHADERCOMPILERPROC (glfwGetProcAddress ("glReleaseShaderCompiler"));
02248   glRenderbufferStorage = PFNGLRENDERBUFFERSTORAGEPROC (glfwGetProcAddress ("glRenderbufferStorage"));
02249   glRenderbufferStorageMultisample =
    PFNGLRENDERBUFFERSTORAGEMULTISAMPLEPROC (glfwGetProcAddress ("glRenderbufferStorageMultisample"));
02250   glRenderbufferStorageMultisampleCoverageNV =
    PFNGLRENDERBUFFERSTORAGEMULTISAMPLECOVERAGEVPROC (glfwGetProcAddress ("glRenderbufferStorageMultisampleCoverageNV"));
02251   glResolveDepthValuesNV =
    PFNGLRESOLVEDEPTHVALUESNVPROC (glfwGetProcAddress ("glResolveDepthValuesNV"));
02252   glResumeTransformFeedback =
    PFNGLRESUMETRANSFORMFEEDBACKPROC (glfwGetProcAddress ("glResumeTransformFeedback"));
02253   glSampleCoverage = PFNGLSAMPLECOVERAGEPROC (glfwGetProcAddress ("glSampleCoverage"));
02254   glSampleMaski = PFNGLSAMPLEREMASKIPROC (glfwGetProcAddress ("glSampleMaski"));
02255   glSamplerParameterIiv = PFNGLSAMPLERPARAMETERIIVPROC (glfwGetProcAddress ("glSamplerParameterIiv"));
02256   glSamplerParameterIuiiv =
    PFNGLSAMPLERPARAMETERIUIVPROC (glfwGetProcAddress ("glSamplerParameterIuiiv"));
02257   glSamplerParameterf = PFNGLSAMPLERPARAMETERFPROC (glfwGetProcAddress ("glSamplerParameterf"));
02258   glSamplerParameterfv = PFNGLSAMPLERPARAMETERFVPROC (glfwGetProcAddress ("glSamplerParameterfv"));
02259   glSamplerParameteri = PFNGLSAMPLERPARAMETERIPROC (glfwGetProcAddress ("glSamplerParameteri"));

```

```

02260     glSamplerParameteriv = PFNGLSAMPLERPARAMETERIVPROC (glfwGetProcAddress("glSamplerParameteriv"));
02261     glScissor = PFNGLSCISSORPROC (glfwGetProcAddress("glScissor"));
02262     glScissorArrayv = PFNGLSCISSORARRAYVPROC (glfwGetProcAddress("glScissorArrayv"));
02263     glScissorIndexed = PFNGLSCISSORINDEXEDPROC (glfwGetProcAddress("glScissorIndexed"));
02264     glScissorIndexedv = PFNGLSCISSORINDEXEDVPROC (glfwGetProcAddress("glScissorIndexedv"));
02265     glSecondaryColorFormatNV =
PFNGLSECONDARYCOLORFORMATNVPROC (glfwGetProcAddress("glSecondaryColorFormatNV"));
02266     glSelectPerfMonitorCountersAMD =
PFNGLSELECTPERFMONITORCOUNTERSAMDPROC (glfwGetProcAddress("glSelectPerfMonitorCountersAMD"));
02267     glShaderBinary = PFNGLSHADERBINARYPROC (glfwGetProcAddress("glShaderBinary"));
02268     glShaderSource = PFNGLSHADERSOURCEPROC (glfwGetProcAddress("glShaderSource"));
02269     glShaderStorageBlockBinding =
PFNGLSHADERSTORAGEBLOCKBINDINGPROC (glfwGetProcAddress("glShaderStorageBlockBinding"));
02270     glSignalVkJFenceNV = PFNGLSIGNALVKFENCENVPROC (glfwGetProcAddress("glSignalVkJFenceNV"));
02271     glSignalVkSemaphoreNV = PFNGLSIGNALVKSEMAPHORENVPROC (glfwGetProcAddress("glSignalVkSemaphoreNV"));
02272     glSpecializeShaderARB = PFNGLSPECIALIZESHADERARBPROC (glfwGetProcAddress("glSpecializeShaderARB"));
02273     glStateCaptureNV = PFNGLSTATECAPTURENVPROC (glfwGetProcAddress("glStateCaptureNV"));
02274     glStencilFillPathInstancedNV =
PFNGLSTENCILFILLPATHINSTANCEDNVPROC (glfwGetProcAddress("glStencilFillPathInstancedNV"));
02275     glStencilFillPathNV = PFNGLSTENCILFILLPATHNVPROC (glfwGetProcAddress("glStencilFillPathNV"));
02276     glStencilFunc = PFNGLSTENCILFUNCPCROC (glfwGetProcAddress("glStencilFunc"));
02277     glStencilFuncSeparate = PFNGLSTENCILFUNCSEPARATEPROC (glfwGetProcAddress("glStencilFuncSeparate"));
02278     glStencilMask = PFNGLSTENCILMASKPCROC (glfwGetProcAddress("glStencilMask"));
02279     glStencilMaskSeparate = PFNGLSTENCILMASKSEPARATEPROC (glfwGetProcAddress("glStencilMaskSeparate"));
02280     glStencilOp = PFNGLSTENCILOPPROC (glfwGetProcAddress("glStencilOp"));
02281     glStencilOpSeparate = PFNGLSTENCILOPSEPARATEPROC (glfwGetProcAddress("glStencilOpSeparate"));
02282     glStencilStrokePathInstancedNV =
PFNGLSTENCILSTROKEPATHINSTANCEDNVPROC (glfwGetProcAddress("glStencilStrokePathInstancedNV"));
02283     glStencilStrokePathNV = PFNGLSTENCILSTROKEPATHNVPROC (glfwGetProcAddress("glStencilStrokePathNV"));
02284     glStencilThenCoverFillPathInstancedNV =
PFNGLSTENCILTHENCOVERFILLPATHINSTANCEDNVPROC (glfwGetProcAddress("glStencilThenCoverFillPathInstancedNV"));
02285     glStencilThenCoverFillPathNV =
PFNGLSTENCILTHENCOVERFILLPATHNVPROC (glfwGetProcAddress("glStencilThenCoverFillPathNV"));
02286     glStencilThenCoverStrokePathInstancedNV =
PFNGLSTENCILTHENCOVERSTROKEPATHINSTANCEDNVPROC (glfwGetProcAddress("glStencilThenCoverStrokePathInstancedNV"));
02287     glStencilThenCoverStrokePathNV =
PFNGLSTENCILTHENCOVERSTROKEPATHNVPROC (glfwGetProcAddress("glStencilThenCoverStrokePathNV"));
02288     glSubpixelPrecisionBiasNV =
PFNGLSUBPIXELPRECISIONBIASNVPROC (glfwGetProcAddress("glSubpixelPrecisionBiasNV"));
02289     glTexBuffer = PFNGLTEXBUFFERPROC (glfwGetProcAddress("glTexBuffer"));
02290     glTexBufferARB = PFNGLTEXBUFFERARBPROC (glfwGetProcAddress("glTexBufferARB"));
02291     glTexBufferRange = PFNGLTEXBUFFERRANGEPROC (glfwGetProcAddress("glTexBufferRange"));
02292     glTexCoordFormatNV = PFNGLTEXCOORDFORMATNVPROC (glfwGetProcAddress("glTexCoordFormatNV"));
02293     glTexImage1D = PFNGLTEXIMAGE1DPROC (glfwGetProcAddress("glTexImage1D"));
02294     glTexImage2D = PFNGLTEXIMAGE2DPROC (glfwGetProcAddress("glTexImage2D"));
02295     glTexImage2DMultisample =
PFNGLTEXIMAGE2DMULTISAMPLEPROC (glfwGetProcAddress("glTexImage2DMultisample"));
02296     glTexImage3D = PFNGLTEXIMAGE3DPROC (glfwGetProcAddress("glTexImage3D"));
02297     glTexImage3DMultisample =
PFNGLTEXIMAGE3DMULTISAMPLEPROC (glfwGetProcAddress("glTexImage3DMultisample"));
02298     glTexPageCommitmentARB =
PFNGLTEXPAGECOMMITMENTARBPROC (glfwGetProcAddress("glTexPageCommitmentARB"));
02299     glTexParameterIiv = PFNGLTEXPARAMETERIIVPROC (glfwGetProcAddress("glTexParameterIiv"));
02300     glTexParameterIuiv = PFNGLTEXPARAMETERIUIVPROC (glfwGetProcAddress("glTexParameterIuiv"));
02301     glTexParameterf = PFNGLTEXPARAMETERFPROC (glfwGetProcAddress("glTexParameterf"));
02302     glTexParameterfv = PFNGLTEXPARAMETERFVPROC (glfwGetProcAddress("glTexParameterfv"));
02303     glTexParameterI = PFNGLTEXPARAMETERIPROC (glfwGetProcAddress("glTexParameterI"));
02304     glTexParameteriv = PFNGLTEXPARAMETERIVPROC (glfwGetProcAddress("glTexParameteriv"));
02305     glTexStorage1D = PFNGLTEXSTORAGE1DPROC (glfwGetProcAddress("glTexStorage1D"));
02306     glTexStorage2D = PFNGLTEXSTORAGE2DPROC (glfwGetProcAddress("glTexStorage2D"));
02307     glTexStorage2DMultisample =
PFNGLTEXSTORAGE2DMULTISAMPLEPROC (glfwGetProcAddress("glTexStorage2DMultisample"));
02308     glTexStorage3D = PFNGLTEXSTORAGE3DPROC (glfwGetProcAddress("glTexStorage3D"));
02309     glTexStorage3DMultisample =
PFNGLTEXSTORAGE3DMULTISAMPLEPROC (glfwGetProcAddress("glTexStorage3DMultisample"));
02310     glTexSubImage1D = PFNGLTEXSUBIMAGE1DPROC (glfwGetProcAddress("glTexSubImage1D"));
02311     glTexSubImage2D = PFNGLTEXSUBIMAGE2DPROC (glfwGetProcAddress("glTexSubImage2D"));
02312     glTexSubImage3D = PFNGLTEXSUBIMAGE3DPROC (glfwGetProcAddress("glTexSubImage3D"));
02313     glTextureBarrier = PFNGLTEXTUREBARRIERPROC (glfwGetProcAddress("glTextureBarrier"));
02314     glTextureBarrierNV = PFNGLTEXTUREBARRIERNVPROC (glfwGetProcAddress("glTextureBarrierNV"));
02315     glTextureBuffer = PFNGLTEXTUREBUFFERPROC (glfwGetProcAddress("glTextureBuffer"));
02316     glTextureBufferEXT = PFNGLTEXTUREBUFFEREXTPROC (glfwGetProcAddress("glTextureBufferEXT"));
02317     glTextureBufferRange = PFNGLTEXTUREBUFFERRANGEPROC (glfwGetProcAddress("glTextureBufferRange"));
02318     glTextureBufferRangeEXT =
PFNGLTEXTUREBUFFERRANGEEXTPROC (glfwGetProcAddress("glTextureBufferRangeEXT"));
02319     glTextureImage1DEXT = PFNGLTEXTUREIMAGE1DEXTPROC (glfwGetProcAddress("glTextureImage1DEXT"));
02320     glTextureImage2DEXT = PFNGLTEXTUREIMAGE2DEXTPROC (glfwGetProcAddress("glTextureImage2DEXT"));
02321     glTextureImage3DEXT = PFNGLTEXTUREIMAGE3DEXTPROC (glfwGetProcAddress("glTextureImage3DEXT"));
02322     glTexturePageCommitmentEXT =
PFNGLTEXTUREPAGECOMMITMENTEXTPROC (glfwGetProcAddress("glTexturePageCommitmentEXT"));
02323     glTextureParameterIiv = PFNGLTEXTUREPARAMETERIIVPROC (glfwGetProcAddress("glTextureParameterIiv"));
02324     glTextureParameterIuivEXT =
PFNGLTEXTUREPARAMETERIUIVEXTPROC (glfwGetProcAddress("glTextureParameterIuivEXT"));
02325     glTextureParameterIuiv =
PFNGLTEXTUREPARAMETERIUIVPROC (glfwGetProcAddress("glTextureParameterIuiv"));
02326     glTextureParameterIuivEXT =
PFNGLTEXTUREPARAMETERIUIVEXTPROC (glfwGetProcAddress("glTextureParameterIuivEXT"));

```

```

02327     glTextureParameterf = PFNGLTEXTUREPARAMETERFPROC (glfwGetProcAddress("glTextureParameterf"));
02328     glTextureParameterfEXT =
02329     PFNGLTEXTUREPARAMETERFEXTPROC (glfwGetProcAddress("glTextureParameterfEXT"));
02329     glTextureParameterfv = PFNGLTEXTUREPARAMETERFVPROC (glfwGetProcAddress("glTextureParameterfv"));
02330     glTextureParameterfvEXT =
02331     PFNGLTEXTUREPARAMETERFVEXTPROC (glfwGetProcAddress("glTextureParameterfvEXT"));
02331     glTextureParameteri = PFNGLTEXTUREPARAMETERIPROC (glfwGetProcAddress("glTextureParameteri"));
02332     glTextureParameteriEXT =
02333     PFNGLTEXTUREPARAMETERIEXTPROC (glfwGetProcAddress("glTextureParameteriEXT"));
02333     glTextureParameteriv = PFNGLTEXTUREPARAMETERIVPROC (glfwGetProcAddress("glTextureParameteriv"));
02334     glTextureParameterivEXT =
02335     PFNGLTEXTUREPARAMETERIVEXTPROC (glfwGetProcAddress("glTextureParameterivEXT"));
02335     glTextureRenderbufferEXT =
02336     PFNGLTEXTURERENDERBUFFEREXTPROC (glfwGetProcAddress("glTextureRenderbufferEXT"));
02336     glTextureStorage1D = PFNGLTEXTURESTORAGE1DPROC (glfwGetProcAddress("glTextureStorage1D"));
02337     glTextureStorage1DEXT = PFNGLTEXTURESTORAGE1DEXTPROC (glfwGetProcAddress("glTextureStorage1DEXT"));
02338     glTextureStorage2D = PFNGLTEXTURESTORAGE2DPROC (glfwGetProcAddress("glTextureStorage2D"));
02339     glTextureStorage2DEXT = PFNGLTEXTURESTORAGE2DEXTPROC (glfwGetProcAddress("glTextureStorage2DEXT"));
02340     glTextureStorage2DMultisample =
02341     PFNGLTEXTURESTORAGE2DMULTISAMPLEPROC (glfwGetProcAddress("glTextureStorage2DMultisample"));
02341     glTextureStorage2DMultisampleEXT =
02342     PFNGLTEXTURESTORAGE2DMULTISAMPLEEXTPROC (glfwGetProcAddress("glTextureStorage2DMultisampleEXT"));
02342     glTextureStorage3D = PFNGLTEXTURESTORAGE3DPROC (glfwGetProcAddress("glTextureStorage3D"));
02343     glTextureStorage3DEXT = PFNGLTEXTURESTORAGE3DEXTPROC (glfwGetProcAddress("glTextureStorage3DEXT"));
02344     glTextureStorage3DMultisample =
02345     PFNGLTEXTURESTORAGE3DMULTISAMPLEPROC (glfwGetProcAddress("glTextureStorage3DMultisample"));
02345     glTextureStorage3DMultisampleEXT =
02346     PFNGLTEXTURESTORAGE3DMULTISAMPLEEXTPROC (glfwGetProcAddress("glTextureStorage3DMultisampleEXT"));
02346     glTextureSubImage1D = PFNGLTEXTURESUBIMAGE1DPROC (glfwGetProcAddress("glTextureSubImage1D"));
02347     glTextureSubImage1DEXT =
02348     PFNGLTEXTURESUBIMAGE1DEXTPROC (glfwGetProcAddress("glTextureSubImage1DEXT"));
02348     glTextureSubImage2D = PFNGLTEXTURESUBIMAGE2DPROC (glfwGetProcAddress("glTextureSubImage2D"));
02349     glTextureSubImage2DEXT =
02350     PFNGLTEXTURESUBIMAGE2DEXTPROC (glfwGetProcAddress("glTextureSubImage2DEXT"));
02350     glTextureSubImage3D = PFNGLTEXTURESUBIMAGE3DPROC (glfwGetProcAddress("glTextureSubImage3D"));
02351     glTextureSubImage3DEXT =
02352     PFNGLTEXTURESUBIMAGE3DEXTPROC (glfwGetProcAddress("glTextureSubImage3DEXT"));
02352     glTextureView = PFNGLTEXTUREVIEWPROC (glfwGetProcAddress("glTextureView"));
02353     glTransformFeedbackBufferBase =
02354     PFNGLTRANSFORMFEEDBACKBUFFERBASEPROC (glfwGetProcAddress("glTransformFeedbackBufferBase"));
02354     glTransformFeedbackBufferRange =
02355     PFNGLTRANSFORMFEEDBACKBUFFERRANGEPROC (glfwGetProcAddress("glTransformFeedbackBufferRange"));
02355     glTransformFeedbackVaryings =
02356     PFNGLTRANSFORMFEEDBACKVARYINGSPROC (glfwGetProcAddress("glTransformFeedbackVaryings"));
02356     glTransformPathNV = PFNGLTRANSFORMPATHNVPROC (glfwGetProcAddress("glTransformPathNV"));
02357     glUniform1d = PFNGLUNIFORM1DPROC (glfwGetProcAddress("glUniform1d"));
02358     glUniform1dv = PFNGLUNIFORM1DVPROC (glfwGetProcAddress("glUniform1dv"));
02359     glUniform1f = PFNGLUNIFORM1FPROC (glfwGetProcAddress("glUniform1f"));
02360     glUniform1fv = PFNGLUNIFORM1FVPROC (glfwGetProcAddress("glUniform1fv"));
02361     glUniform1i = PFNGLUNIFORM1IPROC (glfwGetProcAddress("glUniform1i"));
02362     glUniform1i64ARB = PFNGLUNIFORM1I64ARBPROC (glfwGetProcAddress("glUniform1i64ARB"));
02363     glUniform1i64NV = PFNGLUNIFORM1I64NVPROC (glfwGetProcAddress("glUniform1i64NV"));
02364     glUniform1i64vARB = PFNGLUNIFORM1I64VARBPROC (glfwGetProcAddress("glUniform1i64vARB"));
02365     glUniform1i64vNV = PFNGLUNIFORM1I64VNVPROC (glfwGetProcAddress("glUniform1i64vNV"));
02366     glUniform1iv = PFNGLUNIFORM1IVPROC (glfwGetProcAddress("glUniform1iv"));
02367     glUniform1ui = PFNGLUNIFORM1UIPROC (glfwGetProcAddress("glUniform1ui"));
02368     glUniform1ui64ARB = PFNGLUNIFORM1UI64ARBPROC (glfwGetProcAddress("glUniform1ui64ARB"));
02369     glUniform1ui64NV = PFNGLUNIFORM1UI64NVPROC (glfwGetProcAddress("glUniform1ui64NV"));
02370     glUniform1ui64vARB = PFNGLUNIFORM1UI64VARBPROC (glfwGetProcAddress("glUniform1ui64vARB"));
02371     glUniform1ui64vNV = PFNGLUNIFORM1UI64VNVPROC (glfwGetProcAddress("glUniform1ui64vNV"));
02372     glUniform1uiv = PFNGLUNIFORM1UIVPROC (glfwGetProcAddress("glUniform1uiv"));
02373     glUniform2d = PFNGLUNIFORM2DPROC (glfwGetProcAddress("glUniform2d"));
02374     glUniform2dv = PFNGLUNIFORM2DVPROC (glfwGetProcAddress("glUniform2dv"));
02375     glUniform2f = PFNGLUNIFORM2FPROC (glfwGetProcAddress("glUniform2f"));
02376     glUniform2fv = PFNGLUNIFORM2FVPROC (glfwGetProcAddress("glUniform2fv"));
02377     glUniform2i = PFNGLUNIFORM2IPROC (glfwGetProcAddress("glUniform2i"));
02378     glUniform2i64ARB = PFNGLUNIFORM2I64ARBPROC (glfwGetProcAddress("glUniform2i64ARB"));
02379     glUniform2i64NV = PFNGLUNIFORM2I64NVPROC (glfwGetProcAddress("glUniform2i64NV"));
02380     glUniform2i64vARB = PFNGLUNIFORM2I64VARBPROC (glfwGetProcAddress("glUniform2i64vARB"));
02381     glUniform2i64vNV = PFNGLUNIFORM2I64VNVPROC (glfwGetProcAddress("glUniform2i64vNV"));
02382     glUniform2iv = PFNGLUNIFORM2IVPROC (glfwGetProcAddress("glUniform2iv"));
02383     glUniform2ui = PFNGLUNIFORM2UIPROC (glfwGetProcAddress("glUniform2ui"));
02384     glUniform2ui64ARB = PFNGLUNIFORM2UI64ARBPROC (glfwGetProcAddress("glUniform2ui64ARB"));
02385     glUniform2ui64NV = PFNGLUNIFORM2UI64NVPROC (glfwGetProcAddress("glUniform2ui64NV"));
02386     glUniform2ui64vARB = PFNGLUNIFORM2UI64VARBPROC (glfwGetProcAddress("glUniform2ui64vARB"));
02387     glUniform2ui64vNV = PFNGLUNIFORM2UI64VNVPROC (glfwGetProcAddress("glUniform2ui64vNV"));
02388     glUniform2uiv = PFNGLUNIFORM2UIVPROC (glfwGetProcAddress("glUniform2uiv"));
02389     glUniform3d = PFNGLUNIFORM3DPROC (glfwGetProcAddress("glUniform3d"));
02390     glUniform3dv = PFNGLUNIFORM3DVPROC (glfwGetProcAddress("glUniform3dv"));
02391     glUniform3f = PFNGLUNIFORM3FPROC (glfwGetProcAddress("glUniform3f"));
02392     glUniform3fv = PFNGLUNIFORM3FVPROC (glfwGetProcAddress("glUniform3fv"));
02393     glUniform3i = PFNGLUNIFORM3IPROC (glfwGetProcAddress("glUniform3i"));
02394     glUniform3i64ARB = PFNGLUNIFORM3I64ARBPROC (glfwGetProcAddress("glUniform3i64ARB"));
02395     glUniform3i64NV = PFNGLUNIFORM3I64NVPROC (glfwGetProcAddress("glUniform3i64NV"));
02396     glUniform3i64vARB = PFNGLUNIFORM3I64VARBPROC (glfwGetProcAddress("glUniform3i64vARB"));
02397     glUniform3i64vNV = PFNGLUNIFORM3I64VNVPROC (glfwGetProcAddress("glUniform3i64vNV"));
02398     glUniform3iv = PFNGLUNIFORM3IVPROC (glfwGetProcAddress("glUniform3iv"));

```



```
02399     glUniform3ui = PFNGLUNIFORM3UIPROC (glfwGetProcAddress("glUniform3ui"));
02400     glUniform3ui64ARB = PFNGLUNIFORM3UI64ARBPROC (glfwGetProcAddress("glUniform3ui64ARB"));
02401     glUniform3ui64NV = PFNGLUNIFORM3UI64NVPROC (glfwGetProcAddress("glUniform3ui64NV"));
02402     glUniform3ui64vARB = PFNGLUNIFORM3UI64VARBPROC (glfwGetProcAddress("glUniform3ui64vARB"));
02403     glUniform3ui64vNV = PFNGLUNIFORM3UI64VNVPROC (glfwGetProcAddress("glUniform3ui64vNV"));
02404     glUniform3uiv = PFNGLUNIFORM3UIVPROC (glfwGetProcAddress("glUniform3uiv"));
02405     glUniform4d = PFNGLUNIFORM4DPROC (glfwGetProcAddress("glUniform4d"));
02406     glUniform4dv = PFNGLUNIFORM4DVPROC (glfwGetProcAddress("glUniform4dv"));
02407     glUniform4f = PFNGLUNIFORM4FPROC (glfwGetProcAddress("glUniform4f"));
02408     glUniform4fv = PFNGLUNIFORM4FVPROC (glfwGetProcAddress("glUniform4fv"));
02409     glUniform4i = PFNGLUNIFORM4IPROC (glfwGetProcAddress("glUniform4i"));
02410     glUniform4i64ARB = PFNGLUNIFORM4I64ARBPROC (glfwGetProcAddress("glUniform4i64ARB"));
02411     glUniform4i64NV = PFNGLUNIFORM4I64NVPROC (glfwGetProcAddress("glUniform4i64NV"));
02412     glUniform4i64vARB = PFNGLUNIFORM4I64VARBPROC (glfwGetProcAddress("glUniform4i64vARB"));
02413     glUniform4i64vNV = PFNGLUNIFORM4I64VNVPROC (glfwGetProcAddress("glUniform4i64vNV"));
02414     glUniform4iv = PFNGLUNIFORM4IVPROC (glfwGetProcAddress("glUniform4iv"));
02415     glUniform4ui = PFNGLUNIFORM4UIPROC (glfwGetProcAddress("glUniform4ui"));
02416     glUniform4ui64ARB = PFNGLUNIFORM4UI64ARBPROC (glfwGetProcAddress("glUniform4ui64ARB"));
02417     glUniform4ui64NV = PFNGLUNIFORM4UI64NVPROC (glfwGetProcAddress("glUniform4ui64NV"));
02418     glUniform4ui64vARB = PFNGLUNIFORM4UI64VARBPROC (glfwGetProcAddress("glUniform4ui64vARB"));
02419     glUniform4ui64vNV = PFNGLUNIFORM4UI64VNVPROC (glfwGetProcAddress("glUniform4ui64vNV"));
02420     glUniform4uiv = PFNGLUNIFORM4UIVPROC (glfwGetProcAddress("glUniform4uiv"));
02421     glUniformBlockBinding = PFNGLUNIFORMBLOCKBINDINGPROC (glfwGetProcAddress("glUniformBlockBinding"));
02422     glUniformHandleui64ARB =
PFNGLUNIFORMHANDLEUI64ARBPROC (glfwGetProcAddress("glUniformHandleui64ARB"));
02423     glUniformHandleui64NV = PFNGLUNIFORMHANDLEUI64NVPROC (glfwGetProcAddress("glUniformHandleui64NV"));
02424     glUniformHandleui64vARB =
PFNGLUNIFORMHANDLEUI64VARBPROC (glfwGetProcAddress("glUniformHandleui64vARB"));
02425     glUniformHandleui64vNV =
PFNGLUNIFORMHANDLEUI64VNVPROC (glfwGetProcAddress("glUniformHandleui64vNV"));
02426     glUniformMatrix2dv = PFNGLUNIFORMMATRIX2DVPROC (glfwGetProcAddress("glUniformMatrix2dv"));
02427     glUniformMatrix2fv = PFNGLUNIFORMMATRIX2FVPROC (glfwGetProcAddress("glUniformMatrix2fv"));
02428     glUniformMatrix2x3dv = PFNGLUNIFORMMATRIX2X3DVPROC (glfwGetProcAddress("glUniformMatrix2x3dv"));
02429     glUniformMatrix2x3fv = PFNGLUNIFORMMATRIX2X3FVPROC (glfwGetProcAddress("glUniformMatrix2x3fv"));
02430     glUniformMatrix2x4dv = PFNGLUNIFORMMATRIX2X4DVPROC (glfwGetProcAddress("glUniformMatrix2x4dv"));
02431     glUniformMatrix2x4fv = PFNGLUNIFORMMATRIX2X4FVPROC (glfwGetProcAddress("glUniformMatrix2x4fv"));
02432     glUniformMatrix3dv = PFNGLUNIFORMMATRIX3DVPROC (glfwGetProcAddress("glUniformMatrix3dv"));
02433     glUniformMatrix3fv = PFNGLUNIFORMMATRIX3FVPROC (glfwGetProcAddress("glUniformMatrix3fv"));
02434     glUniformMatrix3x2dv = PFNGLUNIFORMMATRIX3X2DVPROC (glfwGetProcAddress("glUniformMatrix3x2dv"));
02435     glUniformMatrix3x2fv = PFNGLUNIFORMMATRIX3X2FVPROC (glfwGetProcAddress("glUniformMatrix3x2fv"));
02436     glUniformMatrix3x4dv = PFNGLUNIFORMMATRIX3X4DVPROC (glfwGetProcAddress("glUniformMatrix3x4dv"));
02437     glUniformMatrix3x4fv = PFNGLUNIFORMMATRIX3X4FVPROC (glfwGetProcAddress("glUniformMatrix3x4fv"));
02438     glUniformMatrix4dv = PFNGLUNIFORMMATRIX4DVPROC (glfwGetProcAddress("glUniformMatrix4dv"));
02439     glUniformMatrix4fv = PFNGLUNIFORMMATRIX4FVPROC (glfwGetProcAddress("glUniformMatrix4fv"));
02440     glUniformMatrix4x2dv = PFNGLUNIFORMMATRIX4X2DVPROC (glfwGetProcAddress("glUniformMatrix4x2dv"));
02441     glUniformMatrix4x2fv = PFNGLUNIFORMMATRIX4X2FVPROC (glfwGetProcAddress("glUniformMatrix4x2fv"));
02442     glUniformMatrix4x3dv = PFNGLUNIFORMMATRIX4X3DVPROC (glfwGetProcAddress("glUniformMatrix4x3dv"));
02443     glUniformMatrix4x3fv = PFNGLUNIFORMMATRIX4X3FVPROC (glfwGetProcAddress("glUniformMatrix4x3fv"));
02444     glUniformSubroutinesuiv =
PFNGLUNIFORMSUBROUTINESUIVPROC (glfwGetProcAddress("glUniformSubroutinesuiv"));
02445     glUniformui64NV = PFNGLUNIFORMUI64NVPROC (glfwGetProcAddress("glUniformui64NV"));
02446     glUniformui64vNV = PFNGLUNIFORMUI64VNVPROC (glfwGetProcAddress("glUniformui64vNV"));
02447     glUnmapBuffer = PFNGLUNMAPBUFFERPROC (glfwGetProcAddress("glUnmapBuffer"));
02448     glUnmapNamedBuffer = PFNGLUNMAPNAMEDBUFFERPROC (glfwGetProcAddress("glUnmapNamedBuffer"));
02449     glUnmapNamedBufferEXT = PFNGLUNMAPNAMEDBUFFEREXTPROC (glfwGetProcAddress("glUnmapNamedBufferEXT"));
02450     glUseProgram = PFNGLUSEPROGRAMPROC (glfwGetProcAddress("glUseProgram"));
02451     glUseProgramStages = PFNGLUSEPROGRAMSTAGESPROC (glfwGetProcAddress("glUseProgramStages"));
02452     glUseProgramEXT = PFNGLUSESHADERPROGRAMEXTPROC (glfwGetProcAddress("glUseProgramEXT"));
02453     glValidateProgram = PFNGLVALIDATEPROGRAMPROC (glfwGetProcAddress("glValidateProgram"));
02454     glValidateProgramPipeline =
PFNGLVALIDATEPROGRAMPIPELINEPROC (glfwGetProcAddress("glValidateProgramPipeline"));
02455     glVertexArrayAttribBinding =
PFNGLVERTEXARRAYATTRIBBINDINGPROC (glfwGetProcAddress("glVertexArrayAttribBinding"));
02456     glVertexArrayAttribFormat =
PFNGLVERTEXARRAYATTRIBFORMATPROC (glfwGetProcAddress("glVertexArrayAttribFormat"));
02457     glVertexArrayAttribIFormat =
PFNGLVERTEXARRAYATTRIBIFORMATPROC (glfwGetProcAddress("glVertexArrayAttribIFormat"));
02458     glVertexArrayAttribLFormat =
PFNGLVERTEXARRAYATTRIBLFORMATPROC (glfwGetProcAddress("glVertexArrayAttribLFormat"));
02459     glVertexArrayBindVertexBufferEXT =
PFNGLVERTEXARRAYBINDVERTEXBUFFEREXTPROC (glfwGetProcAddress("glVertexArrayBindVertexBufferEXT"));
02460     glVertexArrayBindingDivisor =
PFNGLVERTEXARRAYBINDINGDIVISORPROC (glfwGetProcAddress("glVertexArrayBindingDivisor"));
02461     glVertexArrayColorOffsetEXT =
PFNGLVERTEXARRAYCOLOROFFSETEXTPROC (glfwGetProcAddress("glVertexArrayColorOffsetEXT"));
02462     glVertexArrayEdgeFlagOffsetEXT =
PFNGLVERTEXARRAYEDGEFLAGOFFSETEXTPROC (glfwGetProcAddress("glVertexArrayEdgeFlagOffsetEXT"));
02463     glVertexArrayElementBuffer =
PFNGLVERTEXARRAYELEMENTBUFFERPROC (glfwGetProcAddress("glVertexArrayElementBuffer"));
02464     glVertexArrayFogCoordOffsetEXT =
PFNGLVERTEXARRAYFOGCOORDOFFSETEXTPROC (glfwGetProcAddress("glVertexArrayFogCoordOffsetEXT"));
02465     glVertexArrayIndexOffsetEXT =
PFNGLVERTEXARRAYINDEXOFFSETEXTPROC (glfwGetProcAddress("glVertexArrayIndexOffsetEXT"));
02466     glVertexArrayMultiTexCoordOffsetEXT =
PFNGLVERTEXARRAYMULTITEXCOORDOFFSETEXTPROC (glfwGetProcAddress("glVertexArrayMultiTexCoordOffsetEXT"));
02467     glVertexArrayNormalOffsetEXT =
PFNGLVERTEXARRAYNORMALOFFSETEXTPROC (glfwGetProcAddress("glVertexArrayNormalOffsetEXT"));
```

```
02468     glVertexArraySecondaryColorOffsetEXT =
PFNGLVERTEXARRAYSECONDARYCOLOROFFSETEXTPROC (glfwGetProcAddress ("glVertexArraySecondaryColorOffsetEXT"));
02469     glVertexArrayTexCoordOffsetEXT =
PFNGLVERTEXARRAYTEXCOORDOFFSETEXTPROC (glfwGetProcAddress ("glVertexArrayTexCoordOffsetEXT"));
02470     glVertexArrayVertexAttribBindingEXT =
PFNGLVERTEXARRAYVERTEXATTRIBBINDINGEXTPROC (glfwGetProcAddress ("glVertexArrayVertexAttribBindingEXT"));
02471     glVertexArrayVertexAttribDivisorEXT =
PFNGLVERTEXARRAYVERTEXATTRIBDIVISOREXTPROC (glfwGetProcAddress ("glVertexArrayVertexAttribDivisorEXT"));
02472     glVertexArrayVertexAttribFormatEXT =
PFNGLVERTEXARRAYVERTEXATTRIBFORMATEXTPROC (glfwGetProcAddress ("glVertexArrayVertexAttribFormatEXT"));
02473     glVertexArrayVertexAttribIFormatEXT =
PFNGLVERTEXARRAYVERTEXATTRIBIFORMATEXTPROC (glfwGetProcAddress ("glVertexArrayVertexAttribIFormatEXT"));
02474     glVertexArrayVertexAttribIOffsetEXT =
PFNGLVERTEXARRAYVERTEXATTRIBIOFFSETEXTPROC (glfwGetProcAddress ("glVertexArrayVertexAttribIOffsetEXT"));
02475     glVertexArrayVertexAttribLFormatEXT =
PFNGLVERTEXARRAYVERTEXATTRIBLFORMATEXTPROC (glfwGetProcAddress ("glVertexArrayVertexAttribLFormatEXT"));
02476     glVertexArrayVertexAttribLOffsetEXT =
PFNGLVERTEXARRAYVERTEXATTRIBLOFFSETEXTPROC (glfwGetProcAddress ("glVertexArrayVertexAttribLOffsetEXT"));
02477     glVertexArrayVertexAttribOffsetEXT =
PFNGLVERTEXARRAYVERTEXATTRIBOFFSETEXTPROC (glfwGetProcAddress ("glVertexArrayVertexAttribOffsetEXT"));
02478     glVertexArrayVertexBindingDivisorEXT =
PFNGLVERTEXARRAYVERTEXBINDINGDIVISOREXTPROC (glfwGetProcAddress ("glVertexArrayVertexBindingDivisorEXT"));
02479     glVertexArrayVertexBuffer =
PFNGLVERTEXARRAYVERTEXBUFFERPROC (glfwGetProcAddress ("glVertexArrayVertexBuffer"));
02480     glVertexArrayVertexBuffers =
PFNGLVERTEXARRAYVERTEXBUFFERSPROC (glfwGetProcAddress ("glVertexArrayVertexBuffers"));
02481     glVertexArrayVertexOffsetEXT =
PFNGLVERTEXARRAYVERTEXOFFSETEXTPROC (glfwGetProcAddress ("glVertexArrayVertexOffsetEXT"));
02482     glVertexAttrib1d = PFNGLVERTEXATTRIB1DPROC (glfwGetProcAddress ("glVertexAttrib1d"));
02483     glVertexAttrib1dv = PFNGLVERTEXATTRIB1DVPROC (glfwGetProcAddress ("glVertexAttrib1dv"));
02484     glVertexAttrib1f = PFNGLVERTEXATTRIB1FPROC (glfwGetProcAddress ("glVertexAttrib1f"));
02485     glVertexAttrib1fv = PFNGLVERTEXATTRIB1FVPROC (glfwGetProcAddress ("glVertexAttrib1fv"));
02486     glVertexAttrib1s = PFNGLVERTEXATTRIB1SPROC (glfwGetProcAddress ("glVertexAttrib1s"));
02487     glVertexAttrib1sv = PFNGLVERTEXATTRIB1SVPROC (glfwGetProcAddress ("glVertexAttrib1sv"));
02488     glVertexAttrib2d = PFNGLVERTEXATTRIB2DPROC (glfwGetProcAddress ("glVertexAttrib2d"));
02489     glVertexAttrib2dv = PFNGLVERTEXATTRIB2DVPROC (glfwGetProcAddress ("glVertexAttrib2dv"));
02490     glVertexAttrib2f = PFNGLVERTEXATTRIB2FPROC (glfwGetProcAddress ("glVertexAttrib2f"));
02491     glVertexAttrib2fv = PFNGLVERTEXATTRIB2FVPROC (glfwGetProcAddress ("glVertexAttrib2fv"));
02492     glVertexAttrib2s = PFNGLVERTEXATTRIB2SPROC (glfwGetProcAddress ("glVertexAttrib2s"));
02493     glVertexAttrib2sv = PFNGLVERTEXATTRIB2SVPROC (glfwGetProcAddress ("glVertexAttrib2sv"));
02494     glVertexAttrib3d = PFNGLVERTEXATTRIB3DPROC (glfwGetProcAddress ("glVertexAttrib3d"));
02495     glVertexAttrib3dv = PFNGLVERTEXATTRIB3DVPROC (glfwGetProcAddress ("glVertexAttrib3dv"));
02496     glVertexAttrib3f = PFNGLVERTEXATTRIB3FPROC (glfwGetProcAddress ("glVertexAttrib3f"));
02497     glVertexAttrib3fv = PFNGLVERTEXATTRIB3FVPROC (glfwGetProcAddress ("glVertexAttrib3fv"));
02498     glVertexAttrib3s = PFNGLVERTEXATTRIB3SPROC (glfwGetProcAddress ("glVertexAttrib3s"));
02499     glVertexAttrib3sv = PFNGLVERTEXATTRIB3SVPROC (glfwGetProcAddress ("glVertexAttrib3sv"));
02500     glVertexAttrib4Nbv = PFNGLVERTEXATTRIB4NBVPROC (glfwGetProcAddress ("glVertexAttrib4Nbv"));
02501     glVertexAttrib4Nub = PFNGLVERTEXATTRIB4NUBPROC (glfwGetProcAddress ("glVertexAttrib4Nub"));
02502     glVertexAttrib4Nsv = PFNGLVERTEXATTRIB4NSVPROC (glfwGetProcAddress ("glVertexAttrib4Nsv"));
02503     glVertexAttrib4Nubv = PFNGLVERTEXATTRIB4NUBPROC (glfwGetProcAddress ("glVertexAttrib4Nubv"));
02504     glVertexAttrib4Nuiv = PFNGLVERTEXATTRIB4NUIVPROC (glfwGetProcAddress ("glVertexAttrib4Nuiv"));
02505     glVertexAttrib4Nui = PFNGLVERTEXATTRIB4NUIPROC (glfwGetProcAddress ("glVertexAttrib4Nui"));
02506     glVertexAttrib4Nusv = PFNGLVERTEXATTRIB4NUSVPROC (glfwGetProcAddress ("glVertexAttrib4Nusv"));
02507     glVertexAttrib4bv = PFNGLVERTEXATTRIB4BVPROC (glfwGetProcAddress ("glVertexAttrib4bv"));
02508     glVertexAttrib4d = PFNGLVERTEXATTRIB4DPROC (glfwGetProcAddress ("glVertexAttrib4d"));
02509     glVertexAttrib4dv = PFNGLVERTEXATTRIB4DVPROC (glfwGetProcAddress ("glVertexAttrib4dv"));
02510     glVertexAttrib4f = PFNGLVERTEXATTRIB4FPROC (glfwGetProcAddress ("glVertexAttrib4f"));
02511     glVertexAttrib4fv = PFNGLVERTEXATTRIB4FVPROC (glfwGetProcAddress ("glVertexAttrib4fv"));
02512     glVertexAttrib4iv = PFNGLVERTEXATTRIB4IVPROC (glfwGetProcAddress ("glVertexAttrib4iv"));
02513     glVertexAttrib4s = PFNGLVERTEXATTRIB4SPROC (glfwGetProcAddress ("glVertexAttrib4s"));
02514     glVertexAttrib4sv = PFNGLVERTEXATTRIB4SVPROC (glfwGetProcAddress ("glVertexAttrib4sv"));
02515     glVertexAttrib4ubv = PFNGLVERTEXATTRIB4UBVPROC (glfwGetProcAddress ("glVertexAttrib4ubv"));
02516     glVertexAttrib4uiv = PFNGLVERTEXATTRIB4UIVPROC (glfwGetProcAddress ("glVertexAttrib4uiv"));
02517     glVertexAttrib4usv = PFNGLVERTEXATTRIB4USVPROC (glfwGetProcAddress ("glVertexAttrib4usv"));
02518     glVertexAttribBinding = PFNGLVERTEXATTRIBBINDINGPROC (glfwGetProcAddress ("glVertexAttribBinding"));
02519     glVertexAttribDivisor = PFNGLVERTEXATTRIBDIVISORPROC (glfwGetProcAddress ("glVertexAttribDivisor"));
02520     glVertexAttribDivisorARB =
PFNGLVERTEXATTRIBDIVISORARBPROC (glfwGetProcAddress ("glVertexAttribDivisorARB"));
02521     glVertexAttribFormat = PFNGLVERTEXATTRIBFORMATPROC (glfwGetProcAddress ("glVertexAttribFormat"));
02522     glVertexAttribFormatNV =
PFNGLVERTEXATTRIBFORMATNVPROC (glfwGetProcAddress ("glVertexAttribFormatNV"));
02523     glVertexAttribI1i = PFNGLVERTEXATTRIBI1IPROC (glfwGetProcAddress ("glVertexAttribI1i"));
02524     glVertexAttribI1iv = PFNGLVERTEXATTRIBI1IVPROC (glfwGetProcAddress ("glVertexAttribI1iv"));
02525     glVertexAttribI1ui = PFNGLVERTEXATTRIBI1UIPROC (glfwGetProcAddress ("glVertexAttribI1ui"));
02526     glVertexAttribI1uiv = PFNGLVERTEXATTRIBI1UIVPROC (glfwGetProcAddress ("glVertexAttribI1uiv"));
02527     glVertexAttribI2i = PFNGLVERTEXATTRIBI2IPROC (glfwGetProcAddress ("glVertexAttribI2i"));
02528     glVertexAttribI2iv = PFNGLVERTEXATTRIBI2IVPROC (glfwGetProcAddress ("glVertexAttribI2iv"));
02529     glVertexAttribI2ui = PFNGLVERTEXATTRIBI2UIPROC (glfwGetProcAddress ("glVertexAttribI2ui"));
02530     glVertexAttribI2uiv = PFNGLVERTEXATTRIBI2UIVPROC (glfwGetProcAddress ("glVertexAttribI2uiv"));
02531     glVertexAttribI3i = PFNGLVERTEXATTRIBI3IPROC (glfwGetProcAddress ("glVertexAttribI3i"));
02532     glVertexAttribI3iv = PFNGLVERTEXATTRIBI3IVPROC (glfwGetProcAddress ("glVertexAttribI3iv"));
02533     glVertexAttribI3ui = PFNGLVERTEXATTRIBI3UIPROC (glfwGetProcAddress ("glVertexAttribI3ui"));
02534     glVertexAttribI3uiv = PFNGLVERTEXATTRIBI3UIVPROC (glfwGetProcAddress ("glVertexAttribI3uiv"));
02535     glVertexAttribI4bv = PFNGLVERTEXATTRIBI4BVPROC (glfwGetProcAddress ("glVertexAttribI4bv"));
02536     glVertexAttribI4i = PFNGLVERTEXATTRIBI4IPROC (glfwGetProcAddress ("glVertexAttribI4i"));
02537     glVertexAttribI4iv = PFNGLVERTEXATTRIBI4IVPROC (glfwGetProcAddress ("glVertexAttribI4iv"));
02538     glVertexAttribI4sv = PFNGLVERTEXATTRIBI4SVPROC (glfwGetProcAddress ("glVertexAttribI4sv"));
```

```

02539     glVertexAttribI4ubv = PFNGLVERTEXATTRIBI4UBVPROC (glfwGetProcAddress("glVertexAttribI4ubv"));
02540     glVertexAttribI4ui = PFNGLVERTEXATTRIBI4UIPROC (glfwGetProcAddress("glVertexAttribI4ui"));
02541     glVertexAttribI4uiv = PFNGLVERTEXATTRIBI4UIVPROC (glfwGetProcAddress("glVertexAttribI4uiv"));
02542     glVertexAttribI4usv = PFNGLVERTEXATTRIBI4USVPROC (glfwGetProcAddress("glVertexAttribI4usv"));
02543     glVertexAttribIFormat = PFNGLVERTEXATTRIBIFORMATPROC (glfwGetProcAddress("glVertexAttribIFormat"));
02544     glVertexAttribIFormatNV = PFNGLVERTEXATTRIBIFORMATNVPROC (glfwGetProcAddress("glVertexAttribIFormatNV"));
02545     glVertexAttribIPointer = PFNGLVERTEXATTRIBIPOINTERPROC (glfwGetProcAddress("glVertexAttribIPointer"));
02546     glVertexAttribL1d = PFNGLVERTEXATTRIBL1DPROC (glfwGetProcAddress("glVertexAttribL1d"));
02547     glVertexAttribL1dv = PFNGLVERTEXATTRIBL1DVPROC (glfwGetProcAddress("glVertexAttribL1dv"));
02548     glVertexAttribL1i64NV = PFNGLVERTEXATTRIBL1i64NVPROC (glfwGetProcAddress("glVertexAttribL1i64NV"));
02549     glVertexAttribL1i64vNV = PFNGLVERTEXATTRIBL1i64VNVPROC (glfwGetProcAddress("glVertexAttribL1i64vNV"));
02550     glVertexAttribL1ui64ARB = PFNGLVERTEXATTRIBL1UI64ARBPROC (glfwGetProcAddress("glVertexAttribL1ui64ARB"));
02551     glVertexAttribL1ui64NV = PFNGLVERTEXATTRIBL1UI64NVPROC (glfwGetProcAddress("glVertexAttribL1ui64NV"));
02552     glVertexAttribL1ui64vARB = PFNGLVERTEXATTRIBL1UI64VARBPROC (glfwGetProcAddress("glVertexAttribL1ui64vARB"));
02553     glVertexAttribL1ui64vNV = PFNGLVERTEXATTRIBL1UI64VNVPROC (glfwGetProcAddress("glVertexAttribL1ui64vNV"));
02554     glVertexAttribL2d = PFNGLVERTEXATTRIBL2DPROC (glfwGetProcAddress("glVertexAttribL2d"));
02555     glVertexAttribL2dv = PFNGLVERTEXATTRIBL2DVPROC (glfwGetProcAddress("glVertexAttribL2dv"));
02556     glVertexAttribL2i64NV = PFNGLVERTEXATTRIBL2i64NVPROC (glfwGetProcAddress("glVertexAttribL2i64NV"));
02557     glVertexAttribL2i64vNV = PFNGLVERTEXATTRIBL2i64VNVPROC (glfwGetProcAddress("glVertexAttribL2i64vNV"));
02558     glVertexAttribL2ui64NV = PFNGLVERTEXATTRIBL2UI64NVPROC (glfwGetProcAddress("glVertexAttribL2ui64NV"));
02559     glVertexAttribL2ui64vNV = PFNGLVERTEXATTRIBL2UI64VNVPROC (glfwGetProcAddress("glVertexAttribL2ui64vNV"));
02560     glVertexAttribL3d = PFNGLVERTEXATTRIBL3DPROC (glfwGetProcAddress("glVertexAttribL3d"));
02561     glVertexAttribL3dv = PFNGLVERTEXATTRIBL3DVPROC (glfwGetProcAddress("glVertexAttribL3dv"));
02562     glVertexAttribL3i64NV = PFNGLVERTEXATTRIBL3i64NVPROC (glfwGetProcAddress("glVertexAttribL3i64NV"));
02563     glVertexAttribL3i64vNV = PFNGLVERTEXATTRIBL3i64VNVPROC (glfwGetProcAddress("glVertexAttribL3i64vNV"));
02564     glVertexAttribL3ui64NV = PFNGLVERTEXATTRIBL3UI64NVPROC (glfwGetProcAddress("glVertexAttribL3ui64NV"));
02565     glVertexAttribL3ui64vNV = PFNGLVERTEXATTRIBL3UI64VNVPROC (glfwGetProcAddress("glVertexAttribL3ui64vNV"));
02566     glVertexAttribL4d = PFNGLVERTEXATTRIBL4DPROC (glfwGetProcAddress("glVertexAttribL4d"));
02567     glVertexAttribL4dv = PFNGLVERTEXATTRIBL4DVPROC (glfwGetProcAddress("glVertexAttribL4dv"));
02568     glVertexAttribL4i64NV = PFNGLVERTEXATTRIBL4i64NVPROC (glfwGetProcAddress("glVertexAttribL4i64NV"));
02569     glVertexAttribL4i64vNV = PFNGLVERTEXATTRIBL4i64VNVPROC (glfwGetProcAddress("glVertexAttribL4i64vNV"));
02570     glVertexAttribL4ui64NV = PFNGLVERTEXATTRIBL4UI64NVPROC (glfwGetProcAddress("glVertexAttribL4ui64NV"));
02571     glVertexAttribL4ui64vNV = PFNGLVERTEXATTRIBL4UI64VNVPROC (glfwGetProcAddress("glVertexAttribL4ui64vNV"));
02572     glVertexAttribLFormat = PFNGLVERTEXATTRIBLFORMATPROC (glfwGetProcAddress("glVertexAttribLFormat"));
02573     glVertexAttribLFormatNV = PFNGLVERTEXATTRIBLFORMATNVPROC (glfwGetProcAddress("glVertexAttribLFormatNV"));
02574     glVertexAttribLPointer = PFNGLVERTEXATTRIBLPOINTERPROC (glfwGetProcAddress("glVertexAttribLPointer"));
02575     glVertexAttribP1ui = PFNGLVERTEXATTRIBP1UIPROC (glfwGetProcAddress("glVertexAttribP1ui"));
02576     glVertexAttribP1uiv = PFNGLVERTEXATTRIBP1UIVPROC (glfwGetProcAddress("glVertexAttribP1uiv"));
02577     glVertexAttribP2ui = PFNGLVERTEXATTRIBP2UIPROC (glfwGetProcAddress("glVertexAttribP2ui"));
02578     glVertexAttribP2uiv = PFNGLVERTEXATTRIBP2UIVPROC (glfwGetProcAddress("glVertexAttribP2uiv"));
02579     glVertexAttribP3ui = PFNGLVERTEXATTRIBP3UIPROC (glfwGetProcAddress("glVertexAttribP3ui"));
02580     glVertexAttribP3uiv = PFNGLVERTEXATTRIBP3UIVPROC (glfwGetProcAddress("glVertexAttribP3uiv"));
02581     glVertexAttribP4ui = PFNGLVERTEXATTRIBP4UIPROC (glfwGetProcAddress("glVertexAttribP4ui"));
02582     glVertexAttribP4uiv = PFNGLVERTEXATTRIBP4UIVPROC (glfwGetProcAddress("glVertexAttribP4uiv"));
02583     glVertexAttribPointer = PFNGLVERTEXATTRIBPOINTERPROC (glfwGetProcAddress("glVertexAttribPointer"));
02584     glVertexBindingDivisor = PFNGLVERTEXBINDINGDIVISORPROC (glfwGetProcAddress("glVertexBindingDivisor"));
02585     glVertexFormatNV = PFNGLVERTEXFORMATNVPROC (glfwGetProcAddress("glVertexFormatNV"));
02586     glViewport = PFNGLVIEWPORTPROC (glfwGetProcAddress("glViewport"));
02587     glViewportArrayv = PFNGLVIEWPORTARRAYVPROC (glfwGetProcAddress("glViewportArrayv"));
02588     glViewportIndexedf = PFNGLVIEWPORTINDEXEDFPROC (glfwGetProcAddress("glViewportIndexedf"));
02589     glViewportIndexedfv = PFNGLVIEWPORTINDEXEDFVPROC (glfwGetProcAddress("glViewportIndexedfv"));
02590     glViewportPositionWScaleNV = PFNGLVIEWPORTPOSITIONWSCALENVPROC (glfwGetProcAddress("glViewportPositionWScaleNV"));
02591     glViewportSwizzleNV = PFNGLVIEWPORTSWIZZLENVPROC (glfwGetProcAddress("glViewportSwizzleNV"));
02592     glWaitSync = PFNGLWAITSYNCPROC (glfwGetProcAddress("glWaitSync"));
02593     glWaitVkSemaphoreNV = PFNGLWAITVKSEMAPHORENVPROC (glfwGetProcAddress("glWaitVkSemaphoreNV"));
02594     glWeightPathsNV = PFNGLWEIGHTPATHSNVPROC (glfwGetProcAddress("glWeightPathsNV"));
02595     glWindowRectanglesEXT = PFNGLWINDOWRECTANGLESEXTPROC (glfwGetProcAddress("glWindowRectanglesEXT"));
02596 #endif
02597
02598 // 使用している GPU のバッファアライメントを調べる
02599 glGetIntegerv(GL_UNIFORM_BUFFER_OFFSET_ALIGNMENT, &ggBufferAlignment);
02600 }
02601
02602 //
02603 // OpenGL のエラーをチェックする
02604 //
02605 // OpenGL の API を呼び出し直後に実行すればエラーのあるときにメッセージを表示する

```

```

02606 //
02607 //   msg エラー発生時に標準エラー出力に出力する文字列. nullptr なら何も出力しない
02608 //
02609 void gg::ggError(const std::string& name, unsigned int line)
02610 {
02611     const GLenum error{ glGetError() };
02612     if (error != GL_NO_ERROR)
02613     {
02614         if (!name.empty())
02615         {
02616             std::cerr << name;
02617             if (line > 0) std::cerr << " (" << line << ")";
02618             std::cerr << ": ";
02619         }
02620         switch (error)
02621         {
02622             case GL_INVALID_ENUM:
02623                 std::cerr << "An unacceptable value is specified for an enumerated argument" << std::endl;
02624                 break;
02625             case GL_INVALID_VALUE:
02626                 std::cerr << "A numeric argument is out of range" << std::endl;
02627                 break;
02628             case GL_INVALID_OPERATION:
02629                 std::cerr << "The specified operation is not allowed in the current state" << std::endl;
02630                 break;
02631             case GL_OUT_OF_MEMORY:
02632                 std::cerr << "There is not enough memory left to execute the command" << std::endl;
02633                 break;
02634             case GL_INVALID_FRAMEBUFFER_OPERATION:
02635                 std::cerr << "The specified operation is not allowed current frame buffer" << std::endl;
02636                 break;
02637             default:
02638                 std::cerr << "An OpenGL error has occurred: " << std::hex << std::showbase << error << std::endl;
02639                 break;
02640         }
02641     }
02642 }
02643 }
02644 }
02645 //
02646 // FBO のエラーをチェックする
02647 //
02648 //   FBO の API を呼び出し直後に実行すればエラーのあるときにメッセージを表示する
02649 //
02650 //   msg エラー発生時に標準エラー出力に出力する文字列. nullptr なら何も出力しない
02651 //
02652 //
02653 void gg::ggFBOError(const std::string& name, unsigned int line)
02654 {
02655     const auto status{ glCheckFramebufferStatus(GL_FRAMEBUFFER) };
02656     if (status != GL_FRAMEBUFFER_COMPLETE)
02657     {
02658         if (!name.empty())
02659         {
02660             std::cerr << name;
02661             if (line > 0) std::cerr << " (" << line << ")";
02662             std::cerr << ": ";
02663         }
02664         switch (status)
02665         {
02666             case GL_FRAMEBUFFER_UNDEFINED:
02667                 std::cerr << "the default framebuffer does not exist" << std::endl;
02668                 break;
02669             case GL_FRAMEBUFFER_INCOMPLETE_ATTACHMENT:
02670                 std::cerr << "Framebuffer incomplete, duplicate attachment" << std::endl;
02671                 break;
02672             case GL_FRAMEBUFFER_INCOMPLETE_MISSING_ATTACHMENT:
02673                 std::cerr << "Framebuffer incomplete, missing attachment" << std::endl;
02674                 break;
02675             case GL_FRAMEBUFFER_UNSUPPORTED:
02676                 std::cerr << "Unsupported framebuffer internal" << std::endl;
02677                 break;
02678             case GL_FRAMEBUFFER_INCOMPLETE_MULTISAMPLE:
02679                 std::cerr << "The value of GL_RENDERBUFFER_SAMPLES is not"
02680                     " the same for all attached renderbuffers or,"
02681                     " if the attached images are a mix of renderbuffers and textures,"
02682                     " the value of GL_RENDERBUFFER_SAMPLES is not zero" << std::endl;
02683                 break;
02684             #if !defined(GL_GLES_PROTOTYPES)
02685             case GL_FRAMEBUFFER_INCOMPLETE_LAYER_TARGETS:
02686                 std::cerr << "Any framebuffer attachment is layered,"
02687                     " and any populated attachment is not layered,"
02688                     " or if all populated color attachments are not from textures"
02689                     " of the same target" << std::endl;
02690                 break;
02691             #endif
02692         }
02693     }
02694 }

```

```

02693     case GL_FRAMEBUFFER_INCOMPLETE_DRAW_BUFFER:
02694         std::cerr << "Framebuffer incomplete, missing draw buffer" << std::endl;
02695         break;
02696     case GL_FRAMEBUFFER_INCOMPLETE_READ_BUFFER:
02697         std::cerr << "Framebuffer incomplete, missing read buffer" << std::endl;
02698         break;
02699 #endif
02700     default:
02701         std::cerr << "Programming error; will fail on all hardware: " << std::hex << std::showbase <<
status << std::endl;
02702         break;
02703     }
02704 }
02705 }
02706
02707 //
02708 // 変換行列：行列とベクトルの積  $c \leftarrow a \times b$ 
02709 //
02710 void gg::GgMatrix::projection(GLfloat* c, const GLfloat* a, const GLfloat* b) const
02711 {
02712     for (int i = 0; i < 4; ++i)
02713     {
02714         c[i] = a[0 + i] * b[0] + a[4 + i] * b[1] + a[8 + i] * b[2] + a[12 + i] * b[3];
02715     }
02716 }
02717
02718 //
02719 // 変換行列：行列と行列の積  $c \leftarrow a \times b$ 
02720 //
02721 void gg::GgMatrix::multiply(GLfloat* c, const GLfloat* a, const GLfloat* b) const
02722 {
02723     for (int i = 0; i < 16; ++i)
02724     {
02725         int j = i & 3, k = i & ~3;
02726
02727         c[i] = a[0 + j] * b[k + 0] + a[4 + j] * b[k + 1] + a[8 + j] * b[k + 2] + a[12 + j] * b[k + 3];
02728     }
02729 }
02730
02731 //
02732 // 変換行列：単位行列を設定する
02733 //
02734 gg::GgMatrix& gg::GgMatrix::loadIdentity()
02735 {
02736     *this =
02737     {
02738         1.0f, 0.0f, 0.0f, 0.0f,
02739         0.0f, 1.0f, 0.0f, 0.0f,
02740         0.0f, 0.0f, 1.0f, 0.0f,
02741         0.0f, 0.0f, 0.0f, 1.0f
02742     };
02743
02744     return *this;
02745 }
02746
02747 //
02748 // 変換行列：平行移動変換行列を設定する
02749 //
02750 gg::GgMatrix& gg::GgMatrix::loadTranslate(GLfloat x, GLfloat y, GLfloat z, GLfloat w)
02751 {
02752     *this =
02753     {
02754         w,    0.0f, 0.0f, 0.0f,
02755         0.0f, w,    0.0f, 0.0f,
02756         0.0f, 0.0f, w,    0.0f,
02757         x,    y,    z,    w
02758     };
02759
02760     return *this;
02761 }
02762
02763 //
02764 // 変換行列：拡大縮小変換行列を設定する
02765 //
02766 gg::GgMatrix& gg::GgMatrix::loadScale(GLfloat x, GLfloat y, GLfloat z, GLfloat w)
02767 {
02768     *this =
02769     {
02770         x,    0.0f, 0.0f, 0.0f,
02771         0.0f, y,    0.0f, 0.0f,
02772         0.0f, 0.0f, z,    0.0f,
02773         0.0f, 0.0f, 0.0f, w
02774     };
02775
02776     return *this;
02777 }
02778

```



```

02779 //
02780 // 変換行列：x 軸中心の回転変換行列を設定する
02781 //
02782 gg::GgMatrix& gg::GgMatrix::loadRotateX(GLfloat a)
02783 {
02784     const auto c{ cosf(a) };
02785     const auto s{ sinf(a) };
02786
02787     *this =
02788     {
02789         1.0f, 0.0f, 0.0f, 0.0f,
02790         0.0f, c, s, 0.0f,
02791         0.0f, -s, c, 0.0f,
02792         0.0f, 0.0f, 0.0f, 1.0f
02793     };
02794
02795     return *this;
02796 }
02797
02798 //
02799 // 変換行列：y 軸中心の回転変換行列を設定する
02800 //
02801 gg::GgMatrix& gg::GgMatrix::loadRotateY(GLfloat a)
02802 {
02803     const auto c{ cosf(a) };
02804     const auto s{ sinf(a) };
02805
02806     *this =
02807     {
02808         c, 0.0f, -s, 0.0f,
02809         0.0f, 1.0f, 0.0f, 0.0f,
02810         s, 0.0f, c, 0.0f,
02811         0.0f, 0.0f, 0.0f, 1.0f
02812     };
02813
02814     return *this;
02815 }
02816
02817 //
02818 // 変換行列：z 軸中心の回転変換行列を設定する
02819 //
02820 gg::GgMatrix& gg::GgMatrix::loadRotateZ(GLfloat a)
02821 {
02822     const auto c{ cosf(a) };
02823     const auto s{ sinf(a) };
02824
02825     *this =
02826     {
02827         c, s, 0.0f, 0.0f,
02828         -s, c, 0.0f, 0.0f,
02829         0.0f, 0.0f, 1.0f, 0.0f,
02830         0.0f, 0.0f, 0.0f, 1.0f
02831     };
02832
02833     return *this;
02834 }
02835
02836 //
02837 // 変換行列：任意軸中心の回転変換行列を設定する
02838 //
02839 gg::GgMatrix& gg::GgMatrix::loadRotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a)
02840 {
02841     const auto d{ sqrtf(x * x + y * y + z * z) };
02842
02843     if (d > 0.0f)
02844     {
02845         const auto l{ x / d }, m{ y / d }, n{ z / d };
02846         const auto l2{ l * l }, m2{ m * m }, n2{ n * n };
02847         const auto lm{ l * m }, mn{ m * n }, nl{ n * l };
02848         const auto c{ cosf(a) }, c1{ 1.0f - c };
02849         const auto s{ sinf(a) };
02850
02851         *this =
02852         {
02853             (1.0f - l2) * c + l2,
02854             lm * c1 + n * s,
02855             nl * c1 - m * s,
02856             0.0f,
02857
02858             lm * c1 - n * s,
02859             (1.0f - m2) * c + m2,
02860             mn * c1 + l * s,
02861             0.0f,
02862
02863             nl * c1 + m * s,
02864             mn * c1 - l * s,
02865             (1.0f - n2) * c + n2,

```

```

02866         0.0f,
02867
02868         0.0f,
02869         0.0f,
02870         0.0f,
02871         1.0f,
02872     };
02873 }
02874
02875 return *this;
02876 }
02877
02878 //
02879 // 変換行列：転置行列を設定する
02880 //
02881 gg::GgMatrix& gg::GgMatrix::loadTranspose(const GLfloat* marray)
02882 {
02883     *this =
02884     {
02885         marray[ 0],
02886         marray[ 4],
02887         marray[ 8],
02888         marray[12],
02889         marray[ 1],
02890         marray[ 5],
02891         marray[ 9],
02892         marray[13],
02893         marray[ 2],
02894         marray[ 6],
02895         marray[10],
02896         marray[14],
02897         marray[ 3],
02898         marray[ 7],
02899         marray[11],
02900         marray[15]
02901     };
02902
02903     return *this;
02904 }
02905
02906 //
02907 // 変換行列：逆行列を設定する
02908 //
02909 gg::GgMatrix& gg::GgMatrix::loadInvert(const GLfloat* marray)
02910 {
02911     GLfloat lu[20];
02912     GLfloat* plu[4];
02913
02914     // j 行の要素の値の絶対値の最大値を plu[j][4] に求める
02915     for (int j = 0; j < 4; ++j)
02916     {
02917         auto max{ fabs(*(plu[j] = lu + 5 * j) = *(marray++)) };
02918
02919         for (int i = 0; ++i < 4;)
02920         {
02921             auto a{ fabs(plu[j][i] = *(marray++)) };
02922             if (a > max) max = a;
02923         }
02924         if (max == 0.0f) return *this;
02925         plu[j][4] = 1.0f / max;
02926     }
02927
02928     // ピボットを考慮した LU 分解
02929     for (int j = 0; j < 4; ++j)
02930     {
02931         auto max{ fabs(plu[j][j] * plu[j][4]) };
02932         int i = j;
02933
02934         for (int k = j; ++k < 4;)
02935         {
02936             auto a{ fabs(plu[k][j] * plu[k][4]) };
02937             if (a > max)
02938             {
02939                 max = a;
02940                 i = k;
02941             }
02942         }
02943         if (i > j)
02944         {
02945             auto* t{ plu[j] };
02946             plu[j] = plu[i];
02947             plu[i] = t;
02948         }
02949         if (plu[j][j] == 0.0f) return *this;
02950         for (int k = j; ++k < 4;)
02951         {
02952             plu[k][j] /= plu[j][j];

```

```

02953         for (int i = j; ++i < 4;)
02954         {
02955             plu[k][i] -= plu[j][i] * plu[k][j];
02956         }
02957     }
02958 }
02959
02960 // LU 分解から逆行列を求める
02961 for (int k = 0; k < 4; ++k)
02962 {
02963     // array に単位行列を設定する
02964     for (int i = 0; i < 4; ++i)
02965     {
02966         (*this)[i * 4 + k] = (plu[i] == lu + k * 5) ? 1.0f : 0.0f;
02967     }
02968     // lu から逆行列を求める
02969     for (int i = 0; i < 4; ++i)
02970     {
02971         for (int j = i; ++j < 4;)
02972         {
02973             (*this)[j * 4 + k] -= (*this)[i * 4 + k] * plu[j][i];
02974         }
02975     }
02976     for (int i = 4; --i >= 0;)
02977     {
02978         for (int j = i; ++j < 4;)
02979         {
02980             (*this)[i * 4 + k] -= plu[i][j] * (*this)[j * 4 + k];
02981         }
02982         (*this)[i * 4 + k] /= plu[i][i];
02983     }
02984 }
02985
02986 return *this;
02987 }
02988
02989 //
02990 // 変換行列：法線変換行列を設定する
02991 //
02992 gg::GgMatrix& gg::GgMatrix::loadNormal(const GLfloat* marray)
02993 {
02994     *this =
02995     {
02996         marray[ 5] * marray[10] - marray[ 6] * marray[ 9],
02997         marray[ 6] * marray[ 8] - marray[ 4] * marray[10],
02998         marray[ 4] * marray[ 9] - marray[ 5] * marray[ 8],
02999         0.0f,
03000
03001         marray[ 9] * marray[ 2] - marray[10] * marray[ 1],
03002         marray[10] * marray[ 0] - marray[ 8] * marray[ 2],
03003         marray[ 8] * marray[ 1] - marray[ 9] * marray[ 0],
03004         0.0f,
03005
03006         marray[ 1] * marray[ 6] - marray[ 2] * marray[ 5],
03007         marray[ 2] * marray[ 4] - marray[ 0] * marray[ 6],
03008         marray[ 0] * marray[ 5] - marray[ 1] * marray[ 4],
03009         0.0f,
03010
03011         0.0f,
03012         0.0f,
03013         0.0f,
03014         1.0f
03015     };
03016
03017 return *this;
03018 }
03019
03020 //
03021 // 変換行列：ビュー変換行列を設定する
03022 //
03023 gg::GgMatrix& gg::GgMatrix::loadLookat(
03024     GLfloat ex, GLfloat ey, GLfloat ez,
03025     GLfloat tx, GLfloat ty, GLfloat tz,
03026     GLfloat ux, GLfloat uy, GLfloat uz
03027 )
03028 {
03029     // z 軸 = e - t
03030     const auto zx{ ex - tx };
03031     const auto zy{ ey - ty };
03032     const auto zz{ ez - tz };
03033
03034     // z 軸の長さ
03035     const auto z{ sqrtf(zx * zx + zy * zy + zz * zz) };
03036
03037     // x 軸 = u × z 軸
03038     const auto xx{ uy * zz - uz * zy };
03039     const auto xy{ uz * zx - ux * zz };

```

```

03040     const auto xz{ ux * zy - uy * zx };
03041
03042     // x 軸の長さ
03043     const auto x{ sqrtf(xx * xx + xy * xy + xz * xz) };
03044
03045     // y 軸 = z 軸 × x 軸
03046     const auto yx{ zy * xz - zz * xy };
03047     const auto yy{ zz * xx - zx * xz };
03048     const auto yz{ zx * xy - zy * xx };
03049
03050     // y 軸の長さ
03051     const auto y{ sqrtf(yx * yx + yy * yy + yz * yz) };
03052
03053     // y 軸の長さをチェック
03054     if (fabs(y) < std::numeric_limits<float>::epsilon()) return *this;
03055
03056     (*this)[ 0] = xx / x;
03057     (*this)[ 1] = yx / y;
03058     (*this)[ 2] = zx / z;
03059     (*this)[ 3] = 0.0f;
03060
03061     (*this)[ 4] = xy / x;
03062     (*this)[ 5] = yy / y;
03063     (*this)[ 6] = zy / z;
03064     (*this)[ 7] = 0.0f;
03065
03066     (*this)[ 8] = xz / x;
03067     (*this)[ 9] = yz / y;
03068     (*this)[10] = zz / z;
03069     (*this)[11] = 0.0f;
03070
03071     (*this)[12] = -(ex * (*this)[ 0] + ey * (*this)[ 4] + ez * (*this)[ 8]);
03072     (*this)[13] = -(ex * (*this)[ 1] + ey * (*this)[ 5] + ez * (*this)[ 9]);
03073     (*this)[14] = -(ex * (*this)[ 2] + ey * (*this)[ 6] + ez * (*this)[10]);
03074     (*this)[15] = 1.0f;
03075
03076     return *this;
03077 }
03078
03079 //
03080 // 変換行列：平行投影変換行列を設定する
03081 //
03082 gg::GgMatrix& gg::GgMatrix::loadOrthogonal(
03083     GLfloat left, GLfloat right,
03084     GLfloat bottom, GLfloat top,
03085     GLfloat zNear, GLfloat zFar
03086 )
03087 {
03088     const auto dx{ right - left };
03089     const auto dy{ top - bottom };
03090     const auto dz{ zFar - zNear };
03091
03092     if (dx == 0.0f || dy == 0.0f || dz == 0.0f) return *this;
03093
03094     *this =
03095     {
03096         2.0f / dx,
03097         0.0f,
03098         0.0f,
03099         0.0f,
03100
03101         0.0f,
03102         2.0f / dy,
03103         0.0f,
03104         0.0f,
03105
03106         0.0f,
03107         0.0f,
03108         -2.0f / dz,
03109         0.0f,
03110
03111         -(right + left) / dx,
03112         -(top + bottom) / dy,
03113         -(zFar + zNear) / dz,
03114         1.0f
03115     };
03116
03117     return *this;
03118 }
03119
03120 //
03121 // 変換行列：透視投影変換行列を設定する
03122 //
03123 gg::GgMatrix& gg::GgMatrix::loadFrustum(
03124     GLfloat left, GLfloat right,
03125     GLfloat bottom, GLfloat top,
03126     GLfloat zNear, GLfloat zFar

```

```

03127 )
03128 {
03129     const auto dx{ right - left };
03130     const auto dy{ top - bottom };
03131     const auto dz{ zFar - zNear };
03132
03133     if (dx == 0.0f || dy == 0.0f || dz == 0.0f) return *this;
03134
03135     *this =
03136     {
03137         2.0f * zNear / dx,
03138         0.0f,
03139         0.0f,
03140         0.0f,
03141
03142         0.0f,
03143         2.0f * zNear / dy,
03144         0.0f,
03145         0.0f,
03146
03147         (right + left) / dx,
03148         (top + bottom) / dy,
03149         -(zFar + zNear) / dz,
03150         -1.0f,
03151
03152         0.0f,
03153         0.0f,
03154         -2.0f * zFar * zNear / dz,
03155         0.0f
03156     };
03157
03158     return *this;
03159 }
03160
03161 //
03162 // 変換行列：画角から透視投影変換行列を設定する
03163 //
03164 gg::GgMatrix& gg::GgMatrix::loadPerspective(
03165     GLfloat fovy, GLfloat aspect,
03166     GLfloat zNear, GLfloat zFar
03167 )
03168 {
03169     const auto dz{ zFar - zNear };
03170
03171     if (dz == 0.0f) return *this;
03172
03173     const auto f{ 1.0f / tanf(fovy * 0.5f) };
03174
03175     *this =
03176     {
03177         f / aspect,
03178         0.0f,
03179         0.0f,
03180         0.0f,
03181
03182         0.0f,
03183         f,
03184         0.0f,
03185         0.0f,
03186
03187         0.0f,
03188         0.0f,
03189         -(zFar + zNear) / dz,
03190         -1.0f,
03191
03192         0.0f,
03193         0.0f,
03194         -2.0f * zFar * zNear / dz,
03195         0.0f
03196     };
03197
03198     return *this;
03199 }
03200
03201 //
03202 // 四元数：GgQuaternion 型の四元数 p, q の積を r に求める
03203 //
03204 void gg::GgQuaternion::multiply(GLfloat* r, const GLfloat* p, const GLfloat* q) const
03205 {
03206     r[0] = p[1] * q[2] - p[2] * q[1] + p[0] * q[3] + p[3] * q[0];
03207     r[1] = p[2] * q[0] - p[0] * q[2] + p[1] * q[3] + p[3] * q[1];
03208     r[2] = p[0] * q[1] - p[1] * q[0] + p[2] * q[3] + p[3] * q[2];
03209     r[3] = p[3] * q[3] - p[0] * q[0] - p[1] * q[1] - p[2] * q[2];
03210 }
03211
03212 //
03213 // 四元数：GgQuaternion 型の四元数 q が表す変換行列を m に求める

```

```

03214 //
03215 void gg::GgQuaternion::toMatrix(GLfloat* m, const GLfloat* q) const
03216 {
03217     const auto xx{ q[0] * q[0] * 2.0f };
03218     const auto yy{ q[1] * q[1] * 2.0f };
03219     const auto zz{ q[2] * q[2] * 2.0f };
03220     const auto xy{ q[0] * q[1] * 2.0f };
03221     const auto yz{ q[1] * q[2] * 2.0f };
03222     const auto zx{ q[2] * q[0] * 2.0f };
03223     const auto xw{ q[0] * q[3] * 2.0f };
03224     const auto yw{ q[1] * q[3] * 2.0f };
03225     const auto zw{ q[2] * q[3] * 2.0f };
03226
03227     m[ 0] = 1.0f - yy - zz;
03228     m[ 1] = xy + zw;
03229     m[ 2] = zx - yw;
03230     m[ 4] = xy - zw;
03231     m[ 5] = 1.0f - zz - xx;
03232     m[ 6] = yz + xw;
03233     m[ 8] = zx + yw;
03234     m[ 9] = yz - xw;
03235     m[10] = 1.0f - xx - yy;
03236     m[ 3] = m[ 7] = m[11] = m[12] = m[13] = m[14] = 0.0f;
03237     m[15] = 1.0f;
03238 }
03239
03240 //
03241 // 四元数：回転変換行列 a が表す四元数を q に求める
03242 //
03243 void gg::GgQuaternion::toQuaternion(GLfloat* q, const GLfloat* a) const
03244 {
03245     const auto tr{ a[0] + a[5] + a[10] + a[15] };
03246
03247     if (tr > 0.0f)
03248     {
03249         q[3] = sqrtf(tr) * 0.5f;
03250         q[0] = (a[6] - a[9]) * 0.25f / q[3];
03251         q[1] = (a[8] - a[2]) * 0.25f / q[3];
03252         q[2] = (a[1] - a[4]) * 0.25f / q[3];
03253     }
03254 }
03255
03256 //
03257 // 四元数：球面線形補間 p に q と r を t で補間した四元数を求める
03258 //
03259 void gg::GgQuaternion::slerp(GLfloat* p, const GLfloat* q, const GLfloat* r, GLfloat t) const
03260 {
03261     auto qr{ ggDot4(q, r) };
03262     GLfloat s[4]{ r[0], r[1], r[2], r[3] };
03263
03264     // 内積が負なら最短経路を通るように符号を反転する
03265     if (qr < 0.0f)
03266     {
03267         qr = -qr;
03268         s[0] = -s[0];
03269         s[1] = -s[1];
03270         s[2] = -s[2];
03271         s[3] = -s[3];
03272     }
03273
03274     // 浮動小数点誤差で 1.0f を超える場合をクランプ
03275     if (qr > 1.0f) qr = 1.0f;
03276
03277     const auto ss{ 1.0f - qr * qr };
03278
03279     if (ss <= 0.0f)
03280     {
03281         if (p != q)
03282         {
03283             p[0] = q[0];
03284             p[1] = q[1];
03285             p[2] = q[2];
03286             p[3] = q[3];
03287         }
03288     }
03289     else
03290     {
03291         const auto sp{ sqrtf(ss) };
03292         const auto ph{ acosf(qr) };
03293         const auto pt{ ph * t };
03294         const auto t1{ sinf(pt) / sp };
03295         const auto t0{ sinf(ph - pt) / sp };
03296
03297         p[0] = q[0] * t0 + s[0] * t1;
03298         p[1] = q[1] * t0 + s[1] * t1;
03299         p[2] = q[2] * t0 + s[2] * t1;
03300         p[3] = q[3] * t0 + s[3] * t1;

```

```

03301     }
03302 }
03303
03304 //
03305 // 四元数：(x, y, z) を軸とし角度 a 回転する四元数を求める
03306 //
03307 gg::GgQuaternion& gg::GgQuaternion::loadRotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a)
03308 {
03309     const auto l{ x * x + y * y + z * z };
03310     const auto w{ cosf(a * 0.5f) };
03311
03312     if (fabs(l) > std::numeric_limits<float>::epsilon())
03313     {
03314         const auto s{ sinf(a) / sqrtf(l) };
03315
03316         *this = GgQuaternion{ x * s, y * s, z * s, w };
03317     }
03318     else
03319     {
03320         *this = GgQuaternion{ 0.0f, 0.0f, 0.0f, w };
03321     }
03322
03323     return *this;
03324 }
03325
03326 //
03327 // x 軸中心に角度 a 回転する四元数を格納する
03328 //
03329 gg::GgQuaternion& gg::GgQuaternion::loadRotateX(GLfloat a)
03330 {
03331     const auto t{ a * 0.5f };
03332
03333     *this = GgQuaternion{ sinf(t), 0.0f, 0.0f, cosf(t) };
03334
03335     return *this;
03336 }
03337
03338 //
03339 // y 軸中心に角度 a 回転する四元数を格納する
03340 //
03341 gg::GgQuaternion& gg::GgQuaternion::loadRotateY(GLfloat a)
03342 {
03343     const auto t{ a * 0.5f };
03344
03345     *this = GgQuaternion{ 0.0f, sinf(t), 0.0f, cosf(t) };
03346
03347     return *this;
03348 }
03349
03350 //
03351 // z 軸中心に角度 a 回転する四元数を格納する
03352 //
03353 gg::GgQuaternion& gg::GgQuaternion::loadRotateZ(GLfloat a)
03354 {
03355     const auto t{ a * 0.5f };
03356
03357     *this = GgQuaternion{ 0.0f, 0.0f, sinf(t), cosf(t) };
03358
03359     return *this;
03360 }
03361
03362 //
03363 // 四元数：オイラー角 (heading, pitch, roll) にもとづいて四元数を求める
03364 //
03365 gg::GgQuaternion& gg::GgQuaternion::loadEuler(GLfloat heading, GLfloat pitch, GLfloat roll)
03366 {
03367     GgQuaternion h, p, r;
03368
03369     h.loadRotateY(heading);
03370     p.loadRotateX(pitch);
03371     r.loadRotateZ(roll);
03372
03373     *this = h * p * r;
03374
03375     return *this;
03376 }
03377
03378 //
03379 // 四元数：正規化して格納する
03380 //
03381 gg::GgQuaternion& gg::GgQuaternion::loadNormalize(const GLfloat* a)
03382 {
03383     *this = a;
03384
03385     ggNormalize4(data());
03386
03387     return *this;

```

```

03388 }
03389
03390 //
03391 // 四元数：共役四元数を格納する
03392 //
03393 gg::GgQuaternion& gg::GgQuaternion::loadConjugate(const GLfloat* a)
03394 {
03395     // w 要素を反転する
03396     data()[0] = a[0];
03397     data()[1] = a[1];
03398     data()[2] = a[2];
03399     data()[3] = -a[3];
03400
03401     return *this;
03402 }
03403
03404 //
03405 // 四元数：逆元を格納する
03406 //
03407 gg::GgQuaternion& gg::GgQuaternion::loadInvert(const GLfloat* a)
03408 {
03409     // ノルムの二乗を求める
03410     const auto l(ggDot4(a, a));
03411
03412     if (l > 0.0f)
03413     {
03414         // 共役四元数を求める
03415         GgQuaternion r;
03416         r.loadConjugate(a);
03417
03418         // ノルムの二乗で割る
03419         data()[0] = r[0] / l;
03420         data()[1] = r[1] / l;
03421         data()[2] = r[2] / l;
03422         data()[3] = r[3] / l;
03423     }
03424
03425     return *this;
03426 }
03427
03428 //
03429 // 簡易トラックボール処理：リセット
03430 //
03431 void gg::GgTrackball::reset(const GgQuaternion& q)
03432 {
03433     // ドラッグ中ではない
03434     drag = false;
03435
03436     // 単位クォーターニオンに初期値を与える
03437     *this = cq = q;
03438
03439     // 回転行列を初期化する
03440     GgQuaternion::getMatrix(rt);
03441 }
03442
03443 //
03444 // 簡易トラックボール処理：トラックボールする領域の設定
03445 //
03446 // Reshape コールバック (resize) の中で実行する
03447 // (w, h) ウィンドウサイズ
03448 //
03449 void gg::GgTrackball::region(GLfloat w, GLfloat h)
03450 {
03451     // マウスポインタ位置のウィンドウ内の相対的位置への換算用
03452     scale[0] = 2.0f / w;
03453     scale[1] = 2.0f / h;
03454 }
03455
03456 //
03457 // 簡易トラックボール処理：ドラッグ開始時の処理
03458 //
03459 // マウスボタンを押したときに実行する
03460 // (x, y) 現在のマウス位置
03461 //
03462 void gg::GgTrackball::begin(GLfloat x, GLfloat y)
03463 {
03464     // ドラッグ開始
03465     drag = true;
03466
03467     // ドラッグ開始点を記録する
03468     start[0] = x;
03469     start[1] = y;
03470 }
03471
03472 //
03473 // 簡易トラックボール処理：ドラッグ中の処理
03474 //

```



```

03475 // マウスのドラッグ中に実行する
03476 // (x, y) 現在のマウス位置
03477 //
03478 void gg::GgTrackball::motion(GLfloat x, GLfloat y)
03479 {
03480     // ドラッグ中でなければ何もしない
03481     if (!drag) return;
03482
03483     // マウスポインタの位置のドラッグ開始位置からの変位
03484     const auto dx{ (x - start[0]) * scale[0] };
03485     const auto dy{ (y - start[1]) * scale[1] };
03486
03487     // マウスポインタの位置のドラッグ開始位置からの距離
03488     const auto a{ hypot(dx, dy) };
03489
03490     // マウスポインタの位置がドラッグ開始位置から移動していれば
03491     if (a > 0.0)
03492     {
03493         // 現在の回転の四元数に作った四元数を掛けて合成する
03494         *this = ggRotateQuaternion(dy, dx, 0.0f, a * 3.1415926536f) * cq;
03495
03496         // 合成した四元数から回転の変換行列を求める
03497         GgQuaternion::getMatrix(rt);
03498     }
03499 }
03500
03501 //
03502 // 簡易トラックボール処理：回転角の修正
03503 //
03504 // 現在の回転角を修正する
03505 // q 修正分の回転角を表す四元数
03506 //
03507 void gg::GgTrackball::rotate(const GgQuaternion& q)
03508 {
03509     // ドラッグ中なら何もしない
03510     if (drag) return;
03511
03512     // 保存されている四元数に修正分の四元数を掛けて合成する
03513     *this = q * cq;
03514
03515     // 合成した四元数から回転の変換行列を求める
03516     GgQuaternion::getMatrix(rt);
03517
03518     // 誤差を吸収するために正規化して保存する
03519     cq = normalize();
03520 }
03521
03522 //
03523 // 簡易トラックボール処理：停止時の処理
03524 //
03525 // マウスボタンを離したときに実行する
03526 // (x, y) 現在のマウス位置
03527 //
03528 void gg::GgTrackball::end(GLfloat x, GLfloat y)
03529 {
03530     // ドラッグ終了点における回転を求める
03531     motion(x, y);
03532
03533     // 誤差を吸収するために正規化して保存する
03534     cq = normalize();
03535
03536     // ドラッグ終了
03537     drag = false;
03538 }
03539
03540 //
03541 // 配列に格納された画像の内容を TGA ファイルに保存する
03542 //
03543 // name ファイル名
03544 // buffer 画像データ
03545 // width 画像の横の画素数
03546 // height 画像の縦の画素数
03547 // depth 画像の 1 画素のバイト数
03548 // 戻り値 保存に成功すれば true, 失敗すれば false
03549 //
03550 bool gg::ggSaveTga(
03551     const std::string& name,
03552     const void* buffer,
03553     unsigned int width,
03554     unsigned int height,
03555     unsigned int depth
03556 )
03557 {
03558     // ファイルを開く
03559     std::ofstream file{ Utf8ToTChar(name), std::ios::binary };
03560
03561     // ファイルが開けなかったら戻る

```

```

03562     if (file.fail()) return false;
03563
03564     // 画像のヘッダ
03565     const unsigned char type{ static_cast<unsigned char>(depth == 0 ? 0 : depth < 3 ? 3 : 2) };
03566     const unsigned char alpha{ static_cast<unsigned char>(depth == 2 || depth == 4 ? 8 : 0) };
03567     const unsigned char header[18]
03568     {
03569         0,           // ID length
03570         0,           // Color map type (none)
03571         type,        // Image Type (2:RGB, 3:Grayscale)
03572         0, 0,        // Offset into the color map table
03573         0, 0,        // Number of color map entries
03574         0,           // Number of a color map entry bits per pixel
03575         0, 0,        // Horizontal image position
03576         0, 0,        // Vertical image position
03577         static_cast<unsigned char>(width & 0xff),
03578         static_cast<unsigned char>(width >> 8),
03579         static_cast<unsigned char>(height & 0xff),
03580         static_cast<unsigned char>(height >> 8),
03581         static_cast<unsigned char>(depth * 8),
03582         alpha        // Image descriptor
03583     };
03584
03585     // ヘッダを書き込む
03586     file.write(reinterpret_cast<const char*>(header), sizeof header);
03587
03588     // ヘッダの書き込みに失敗したら戻る
03589     if (file.bad()) return false;
03590
03591     // データを書き込む
03592     const unsigned int size{ width * height * depth };
03593     if (type == 2)
03594     {
03595         // フルカラー
03596         std::vector<char> temp(size);
03597         for (GLuint i = 0; i < size; i += depth)
03598         {
03599             temp[i + 2] = static_cast<const char*>(buffer)[i + 0];
03600             temp[i + 1] = static_cast<const char*>(buffer)[i + 1];
03601             temp[i + 0] = static_cast<const char*>(buffer)[i + 2];
03602             if (depth == 4) temp[i + 3] = static_cast<const char*>(buffer)[i + 3];
03603         }
03604         file.write(temp.data(), size);
03605     }
03606     else if (type == 3)
03607     {
03608         // グレースケール
03609         file.write(static_cast<const char*>(buffer), size);
03610     }
03611
03612     // フッタを書き込む
03613     constexpr char footer[]{ "\0\0\0\0\0\0\0\0TRUEVISION-XFILE." };
03614     file.write(footer, sizeof footer);
03615
03616     // データの書き込みに失敗していなければ true を返す
03617     return !file.bad();
03618 }
03619
03620 //
03621 // カラーバッファの内容を TGA ファイルに保存する
03622 //
03623 //     name 保存するファイル名
03624 //     戻り値 保存に成功すれば true, 失敗すれば false
03625 //
03626 bool gg::ggSaveColor(const std::string& name)
03627 {
03628     // 現在のビューポートのサイズを得る
03629     GLint viewport[4];
03630     glGetIntegerv(GL_VIEWPORT, viewport);
03631
03632     // ビューポートのサイズ分のメモリを確保する
03633     std::vector<GLubyte> buffer(viewport[2] * viewport[3] * 3);
03634
03635     // 画面表示の完了を待つ
03636     glFinish();
03637
03638     // カラーバッファを読み込む
03639     glReadPixels(viewport[0], viewport[1], viewport[2], viewport[3],
03640                 GL_RGB, GL_UNSIGNED_BYTE, buffer.data());
03641
03642     // 読み込んだデータをファイルに書き込む
03643     return ggSaveTga(name, buffer.data(), viewport[2], viewport[3], 3);
03644 }
03645
03646 //
03647 // デプスバッファの内容を TGA ファイルに保存する
03648 //

```

```

03649 // name 保存するファイル名
03650 // 戻り値 保存に成功すれば true, 失敗すれば false
03651 //
03652 bool gg::ggSaveDepth(const std::string& name)
03653 {
03654     // 現在のビューポートのサイズを得る
03655     GLint viewport[4];
03656     glGetIntegerv(GL_VIEWPORT, viewport);
03657
03658     // ビューポートのサイズ分のメモリを確保する
03659     std::vector<GLubyte> buffer(viewport[2] * viewport[3]);
03660
03661     // 画面表示の完了を待つ
03662     glFinish();
03663
03664     // デブスバッファを読み込む
03665     glReadPixels(viewport[0], viewport[1], viewport[2], viewport[3],
03666                 GL_DEPTH_COMPONENT, GL_UNSIGNED_BYTE, buffer.data());
03667
03668     // 読み込んだデータをファイルに書き込む
03669     return ggSaveTga(name, buffer.data(), viewport[2], viewport[3], 1);
03670 }
03671
03672 //
03673 // TGA ファイル (8/16/24/32bit) を読み込む
03674 //
03675 // name 読み込むファイル名
03676 // pWidth 読み込んだファイルの横の画素数の格納先のポインタ (nullptr なら格納しない)
03677 // pHeight 読み込んだファイルの縦の画素数の格納先のポインタ (nullptr なら格納しない)
03678 // pFormat 読み込んだファイルのフォーマットの格納先のポインタ (nullptr なら格納しない)
03679 // image 読み込んだ画像を格納する vector
03680 // 戻り値 読み込みに成功すれば true, 失敗すれば false
03681 //
03682 bool gg::ggReadImage(
03683     const std::string& name,
03684     std::vector<GLubyte>& image,
03685     GLsizei* pWidth,
03686     GLsizei* pHeight,
03687     GLenum* pFormat
03688 )
03689 {
03690     // ファイルを開く
03691     std::ifstream file{ Utf8ToTChar(name), std::ios::binary };
03692
03693     // ファイルが開けなかったら戻る
03694     if (file.fail()) return false;
03695
03696     // ヘッダを読み込む
03697     unsigned char header[18];
03698     file.read(reinterpret_cast<char*>(header), sizeof header);
03699
03700     // ヘッダの読み込みに失敗したら戻る
03701     if (file.bad()) return false;
03702
03703     // 深度
03704     const auto depth{ header[16] / 8 };
03705     if (pFormat)
03706     {
03707         GLenum format;
03708         switch (depth)
03709         {
03710             case 1:
03711                 format = GL_RED;
03712                 break;
03713             case 2:
03714                 format = GL_RG;
03715                 break;
03716             case 3:
03717                 format = GL_BGR;
03718                 break;
03719             case 4:
03720                 format = GL_BGRA;
03721                 break;
03722             default:
03723                 // 取り扱えないフォーマットだったら戻る
03724                 return false;
03725         }
03726         *pFormat = format;
03727     }
03728
03729     // 画像の縦横の画素数
03730     const auto width{ static_cast<GLsizei>(header[13] << 8 | header[12]) };
03731     if (pWidth) *pWidth = width;
03732     const auto height{ static_cast<GLsizei>(header[15] << 8 | header[14]) };
03733     if (pHeight) *pHeight = height;
03734
03735     // データサイズ

```

```

03736     const auto size{ width * height * depth };
03737
03738     // サイズが小さすぎたら戻る
03739     if (size < 2) return false;
03740
03741     // 読み込みに使うメモリを確保する
03742     image.resize(size);
03743
03744     // データを読み込む
03745     if (header[2] & 8)
03746     {
03747         // RLE
03748         int p{ 0 };
03749         char c;
03750         while (file.get(c))
03751         {
03752             if (c & 0x80)
03753             {
03754                 // run-length packet
03755                 const auto count{ (c & 0x7f) + 1 };
03756                 if (p + depth * count > size) break;
03757                 char temp[4];
03758                 file.read(temp, depth);
03759                 for (int i = 0; i < count; ++i)
03760                 {
03761                     for (int j = 0; j < depth; j++) image[p++] = temp[j++];
03762                 }
03763             }
03764             else
03765             {
03766                 // raw packet
03767                 const auto count{ (c + 1) * depth };
03768                 if (p + count > size) break;
03769                 file.read(reinterpret_cast<char*>(image.data() + p), count);
03770                 p += count;
03771             }
03772         }
03773     }
03774     else
03775     {
03776         // 非圧縮
03777         file.read(reinterpret_cast<char*>(image.data()), size);
03778     }
03779
03780     // 読み込みに失敗していなければ true を返す
03781     return !file.bad();
03782 }
03783
03784 //
03785 // テクスチャを作成して画像を読み込む
03786 //
03787 // image 画像データ, nullptr ならメモリの確保だけを行う
03788 // width 画像の横の画素数
03789 // height 画像の縦の画素数
03790 // format 画像データのフォーマット
03791 // type 画像のデータ型
03792 // internal テクスチャの内部フォーマット
03793 // wrap テクスチャのラッピングモード
03794 // swizzle true ならテクスチャの赤と青を入れ替える
03795 // 戻り値 テクスチャ名
03796 //
03797 GLuint gg::ggLoadTexture(
03798     const GLvoid* image,
03799     GLsizei width,
03800     GLsizei height,
03801     GLenum format,
03802     GLenum type,
03803     GLenum internal,
03804     GLenum wrap,
03805     bool swizzle
03806 )
03807 {
03808     // テクスチャを作成する
03809     GLuint texture;
03810     glGenTextures(1, &texture);
03811
03812     // 作成したテクスチャを 2D テクスチャとしてバインドする
03813     glBindTexture(GL_TEXTURE_2D, texture);
03814
03815     // アルファチャンネルがついていれば 4 バイト境界に設定する
03816     glPixelStorei(GL_UNPACK_ALIGNMENT, format == GL_RGBA || format == GL_BGRA ? 4 : 1);
03817
03818     // テクスチャを割り当てる
03819     glTexImage2D(GL_TEXTURE_2D, 0, internal, width, height, 0, format, type, image);
03820
03821     // バイリニア（ミップマップなし）、エッジでクランプ
03822     glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);

```

```

03823     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
03824     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, wrap);
03825     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, wrap);
03826
03827     if (swizzle)
03828     {
03829         // テクスチャのサンプリング時に赤と青を入れ替える
03830         glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_SWIZZLE_R, GL_BLUE);
03831         glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_SWIZZLE_B, GL_RED);
03832     }
03833
03834     // テクスチャ名を返す
03835     return texture;
03836 }
03837
03838 //
03839 // テクスチャを作成して TGA フォーマットの画像ファイルを読み込む
03840 //
03841 //     name TGA ファイル名
03842 //     pWidth 読みだした画像ファイルの横の画素数の格納先のポインタ (nullptr なら格納しない)
03843 //     pHeight 読みだした画像ファイルの縦の画素数の格納先のポインタ (nullptr なら格納しない)
03844 //     internal テクスチャの内部フォーマット
03845 //     wrap テクスチャのラッピングモード
03846 //     戻り値 テクスチャ名
03847 //
03848 GLuint gg::ggLoadImage(
03849     const std::string& name,
03850     GLsizei* pWidth,
03851     GLsizei* pHeight,
03852     GLenum internal,
03853     GLenum wrap
03854 )
03855 {
03856     // 画像データ
03857     std::vector<GLubyte> image;
03858
03859     // 画像サイズ
03860     GLsizei width, height;
03861
03862     // 画像フォーマット
03863     GLenum format;
03864
03865     // 画像を読み込む
03866     ggReadImage(name, image, &width, &height, &format);
03867
03868     // 画像が読み込めなかったら戻る
03869     if (image.empty()) return 0;
03870
03871     // internal == 0 なら内部フォーマットを読み込んだファイルに合わせる
03872     if (internal == 0) internal = format;
03873
03874     // テクスチャに読み込む (ggReadImage() で読み込んだ画像は GL_BGR / GL_BGRA)
03875     const auto tex{ ggLoadTexture(image.data(), width, height,
03876         format, GL_UNSIGNED_BYTE, internal, wrap, false) };
03877
03878     // 画像サイズを返す
03879     if (pWidth) *pWidth = width;
03880     if (pHeight) *pHeight = height;
03881
03882     // テクスチャ名を返す
03883     return tex;
03884 }
03885
03886 //
03887 // グレースケール画像 (8bit) から法線マップのデータを作成する
03888 //
03889 //     width 高さマップのグレースケール画像 hmap の横の画素数
03890 //     height 高さマップのグレースケール画像のデータ hmap の縦の画素数
03891 //     stride データの間隔
03892 //     hmap グレースケール画像のデータ
03893 //     nz 法線の z 成分の割合
03894 //     internal テクスチャの内部フォーマット
03895 //     nmap 法線マップを格納する vector
03896 //
03897 void gg::ggCreateNormalMap(
03898     const GLubyte* hmap,
03899     GLsizei width,
03900     GLsizei height,
03901     GLenum format,
03902     GLfloat nz,
03903     GLenum internal,
03904     std::vector<GgVector>& nmap
03905 )
03906 {
03907     // メモリサイズ
03908     const GLsizei size{ width * height };
03909

```

```

03910 // 法線マップのメモリを確保する
03911 nmap.resize(size);
03912
03913 // 画素のバイト数
03914 GLint stride;
03915 switch (format)
03916 {
03917 case GL_RED:
03918     stride = 1;
03919     break;
03920 case GL_RG:
03921     stride = 2;
03922     break;
03923 case GL_RGB:
03924     stride = 3;
03925     break;
03926 case GL_RGBA:
03927     stride = 4;
03928     break;
03929 default:
03930     stride = 1;
03931     break;
03932 }
03933
03934 // 法線マップの作成
03935 for (GLsizei i = 0; i < size; ++i)
03936 {
03937     const auto x{ i % width };
03938     const auto y{ i - x };
03939     const auto u0{ (y + (x - 1 + width) % width) * stride };
03940     const auto u1{ (y + (x + 1) % width) * stride };
03941     const auto v0{ ((y - width + size) % size + x) * stride };
03942     const auto v1{ ((y + width) % size + x) * stride };
03943
03944     // 隣接する画素との値の差を法線の成分に用いる
03945     nmap[i][0] = static_cast<GLfloat>(hmap[u0] - hmap[u1]);
03946     nmap[i][1] = static_cast<GLfloat>(hmap[v0] - hmap[v1]);
03947     nmap[i][2] = nz;
03948     nmap[i][3] = hmap[i * stride];
03949
03950     // 法線ベクトルを正規化する
03951     ggNormalize3(&nmap[i]);
03952 }
03953
03954 // 内部フォーマットが浮動小数点テクスチャでなければ [0,1] に正規化する
03955 if (
03956     internal != GL_RGB16F &&
03957     internal != GL_RGBA16F &&
03958     internal != GL_RGB32F &&
03959     internal != GL_RGBA32F
03960 )
03961 {
03962     for (GLsizei i = 0; i < size; ++i)
03963     {
03964         nmap[i][0] = nmap[i][0] * 0.5f + 0.5f;
03965         nmap[i][1] = nmap[i][1] * 0.5f + 0.5f;
03966         nmap[i][2] = nmap[i][2] * 0.5f + 0.5f;
03967         nmap[i][3] *= 0.0039215686f; // == 1/255
03968     }
03969 }
03970 }
03971
03972 //
03973 // TGA 画像ファイルの高さマップ読み込んで法線マップのテクスチャを作成する
03974 //
03975 // name TGA ファイル名
03976 // nz 作成した法線の z 成分の割合
03977 // pWidth 読み込んだ画像の横の画素数の格納先のポインタ (nullptr なら格納しない)
03978 // pHeight 読み込んだ画像の縦の画素数の格納先のポインタ (nullptr なら格納しない)
03979 // internal テクスチャの内部フォーマット
03980 // 戻り値 テクスチャ名
03981 //
03982 GLuint gg::ggLoadHeight(
03983     const std::string& name,
03984     GLfloat nz,
03985     GLsizei* pWidth,
03986     GLsizei* pHeight,
03987     GLenum internal
03988 )
03989 {
03990     // 画像データ
03991     std::vector<GLubyte> hmap;
03992
03993     // 画像サイズ
03994     GLsizei width, height;
03995
03996     // 画像フォーマット

```

```

03997   GLenum format;
03998
03999   // 高さマップの画像を読み込む
04000   ggReadImage(name, hmap, &width, &height, &format);
04001
04002   // 画像が読み込めなかったら戻る
04003   if (hmap.empty()) return 0;
04004
04005   // 法線マップ
04006   std::vector<GgVector> nmap;
04007
04008   // 法線マップを作成する
04009   ggCreateNormalMap(hmap.data(), width, height, format, nz, internal, nmap);
04010
04011   // 画像サイズを返す
04012   if (pWidth) *pWidth = width;
04013   if (pHeight) *pHeight = height;
04014
04015   // テクスチャを作成して返す
04016   return ggLoadTexture(nmap.data(), width, height, GL_RGBA, GL_FLOAT, internal, GL_REPEAT);
04017 }
04018
04019 //
04020 // テクスチャの赤と青を交換する
04021 //
04022 void gg::GgTexture::swapRandB(bool swap) const
04023 {
04024     const auto swizzle
04025     {
04026         swap ?
04027             std::pair<GLint, GLint>{ GL_BLUE, GL_RED } :
04028             std::pair<GLint, GLint>{ GL_RED, GL_BLUE }
04029     };
04030
04031     glBindTexture(GL_TEXTURE_2D, texture);
04032     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_SWIZZLE_R, swizzle.first);
04033     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_SWIZZLE_B, swizzle.second);
04034     glBindTexture(GL_TEXTURE_2D, 0);
04035 }
04036
04037 //
04038 // TGA フォーマットの画像ファイルを読み込んでカラーのテクスチャを作成する
04039 //
04040 //   name 読み込むファイル名
04041 //   internal glTexImage2D() に指定するテクスチャの内部フォーマット. 0 なら外部フォーマットに合わせる
04042 //   戻り値 テクスチャの作成に成功すれば true, 失敗すれば false
04043 //
04044 void gg::GgColorTexture::load(
04045     const std::string& name,
04046     GLenum internal,
04047     GLenum wrap
04048 )
04049 {
04050     // 画像データ
04051     std::vector<GLubyte> image;
04052
04053     // 画像サイズ
04054     GLsizei width, height;
04055
04056     // 画像フォーマット
04057     GLenum format;
04058
04059     // 画像を読み込む
04060     ggReadImage(name, image, &width, &height, &format);
04061
04062     // internal == 0 なら内部フォーマットを読み込んだファイルに合わせる
04063     if (internal == 0) internal = format;
04064
04065     // テクスチャを作成する (ggReadImage() で読み込んだ画像は GL_BGR / GL_BGRA)
04066     texture = std::make_shared<GgTexture>(image.data(), width, height,
04067         format, GL_UNSIGNED_BYTE, internal, wrap, false);
04068 }
04069
04070 //
04071 // TGA フォーマットの画像ファイルから高さマップ読み込んで法線マップのテクスチャを作成する
04072 //
04073 //   name 画像ファイル名
04074 //   width テクスチャとして用いる画像データの横幅
04075 //   height テクスチャとして用いる画像データの高さ
04076 //   format テクスチャとして用いる画像データのフォーマット (GL_RED, GL_RG, GL_RGB, GL_RGBA)
04077 //   nz 法線マップの z 成分の値
04078 //   internal テクスチャの内部フォーマット
04079 //
04080 void gg::GgNormalTexture::load(
04081     const std::string& name,
04082     GLfloat nz,
04083     GLenum internal

```

```

04084 )
04085 {
04086     // 高さマップ
04087     std::vector<GLubyte> hmap;
04088
04089     // 画像サイズ
04090     GLsizei width, height;
04091
04092     // 画像フォーマット
04093     GLenum format;
04094
04095     // 高さマップの画像を読み込む
04096     ggReadImage(name, hmap, &width, &height, &format);
04097
04098     // 法線マップ
04099     std::vector<GgVector> nmap;
04100
04101     // 法線マップを作成する
04102     ggCreateNormalMap(hmap.data(), width, height, format, nz, internal, nmap);
04103
04104     // テクスチャを作成する
04105     texture = std::make_shared<GgTexture>(nmap.data(), width, height, GL_RGBA, GL_FLOAT, internal,
GL_REPEAT);
04106 }
04107
04109 //
04110 // OBJ ファイルの読み込みに使うデータ型と関数
04111 //
04112 namespace gg
04113 {
04114     // GLfloat 型の 2 要素のベクトル
04115     using vec2 = std::array<GLfloat, 2>;
04116
04117     // GLfloat 型の 3 要素のベクトル
04118     using vec3 = std::array<GLfloat, 3>;
04119
04120     // 三角形データ
04121     struct fixd
04122     {
04123         GLuint p[3];           // 頂点座標番号
04124         GLuint n[3];           // 頂点法線番号
04125         GLuint t[3];           // テクスチャ座標番号
04126         bool smooth;           // スムーズシェーディングの有無
04127     };
04128
04129     // ポリゴングループ
04130     struct fgrp
04131     {
04132         GLuint nextgroup;      // 次のポリゴングループの最初の三角形番号
04133         GLuint mtlno;          // このポリゴングループの材質番号
04134
04135         // コンストラクタ
04136         fgrp(GLuint nextgroup, GLuint mtlno) :
04137             nextgroup(nextgroup),
04138             mtlno(mtlno)
04139         {
04140         }
04141     };
04142
04143     // デフォルトの材質
04144     constexpr GgSimpleShader::Material defaultMaterial
04145     {
04146         { 0.1f, 0.1f, 0.1f, 1.0f },
04147         { 0.6f, 0.6f, 0.6f, 0.0f },
04148         { 0.3f, 0.3f, 0.3f, 0.0f },
04149         60.0f
04150     };
04151
04152     // デフォルトの材質名
04153     constexpr char defaultMaterialName[] = "_default_";
04154
04155     //
04156     // Alias OBJ 形式の MTL ファイルを読み込む
04157     //
04158     // mtlpath MTL ファイルのパス名
04159     // mtl 読み込んだ材質名をキー、材質番号を値にした map
04160     // material 材質データ
04161     //
04162     static bool ggLoadMtl(const std::string& mtlpath,
04163         std::map<std::string, GLuint>& mtl,
04164         std::vector<GgSimpleShader::Material>& material)
04165     {
04166         // MTL ファイルが無ければ戻る
04167         std::ifstream mtlfile{ Utf8ToTChar(mtlpath), std::ios::binary };
04168         if (!mtlfile)
04169             return false;
04170     }

```



```

04171 #if defined(DEBUG)
04172     std::cerr << "Warning: Can't open MTL file: " << mtlpath << std::endl;
04173 #endif
04174     return false;
04175 }
04176
04177 // 一行読み込み用のバッファ
04178 std::string mtline;
04179
04180 // 材質名（ループの外に置く）
04181 std::string mtlname{ defaultMaterialName };
04182
04183 // 現在の材質番号を登録する
04184 mtl[mtlname] = static_cast<GLuint>(material.size());
04185
04186 // 現在の材質にデフォルトの材質を設定する
04187 material.emplace_back(defaultMaterial);
04188
04189 // 材質データを読み込む
04190 while (std::getline(mtlfile, mtline))
04191 {
04192     // 空行は読み飛ばす
04193     if (mtline == "") continue;
04194
04195     // 最後の文字が '\r' なら
04196     if (*(mtline.end() - 1) == '\r')
04197     {
04198         // 最後の文字を削除する
04199         mtline.erase(mtline.end() - 1, mtline.end());
04200
04201         // 空行になったら読み飛ばす
04202         if (mtline == "") continue;
04203     }
04204
04205     // 読み込んだ行を文字列ストリームにする
04206     std::istringstream mtlstr{ mtline };
04207
04208     // オペレータ
04209     std::string mtlop;
04210
04211     // 文字列ストリームから材質パラメータの種類を取り出す
04212     mtlstr >> mtlop;
04213
04214     // '#' で始まる場合はコメントとして行末まで読み飛ばす
04215     if (mtlop[0] == '#') continue;
04216
04217     if (mtlop == "newmtl")
04218     {
04219         // 新規作成する材質名を取り出す
04220         mtlstr >> mtlname;
04221
04222         // 材質名が既に存在するかどうか調べる
04223         const auto m{ mtl.find(mtlname) };
04224         if (m == mtl.end())
04225         {
04226             // 存在しないので新規作成する材質の番号をその材質名に割り当てる
04227             mtl[mtlname] = static_cast<GLuint>(material.size());
04228
04229             // 新規作成する材質にデフォルトの材質を設定しておく
04230             material.emplace_back(defaultMaterial);
04231         }
04232     }
04233     #if defined(DEBUG)
04234         std::cerr << "newmtl: " << mtlname << std::endl;
04235     #endif
04236 }
04237 else if (mtlop == "Ka")
04238 {
04239     // 環境光の反射係数を登録する
04240     mtlstr
04241     >> material.back().ambient[0]
04242     >> material.back().ambient[1]
04243     >> material.back().ambient[2];
04244 }
04245 else if (mtlop == "Kd")
04246 {
04247     // 拡散反射係数を登録する
04248     mtlstr
04249     >> material.back().diffuse[0]
04250     >> material.back().diffuse[1]
04251     >> material.back().diffuse[2];
04252 }
04253 else if (mtlop == "Ks")
04254 {
04255     // 鏡面反射係数を登録する
04256     mtlstr
04257     >> material.back().specular[0]

```

```

04258         >> material.back().specular[1]
04259         >> material.back().specular[2];
04260     }
04261     else if (mtlop == "Ns")
04262     {
04263         // 輝き係数を登録する
04264         GLfloat shininess;
04265         mtlstr >> shininess;
04266         material.back().shininess = shininess * 0.1f;
04267     }
04268     else if (mtlop == "d")
04269     {
04270         // 不透明度を登録する
04271         mtlstr >> material.back().ambient[3];
04272     }
04273 }
04274
04275 // MTL ファイルの読み込みに失敗したら戻る
04276 if (mtlfile.bad())
04277 {
04278     #if defined(DEBUG)
04279         std::cerr << "Warning: Can't read MTL file: " << mtlpath << std::endl;
04280     #endif
04281     return false;
04282 }
04283
04284 // MTL ファイルの読み込みに成功した
04285 return true;
04286 }
04287
04288 //
04289 // Alias OBJ 形式のファイルを解析する
04290 //
04291 // name Alias OBJ 形式のファイルのファイル名
04292 // group 同じ材質を割り当てるポリゴングループ
04293 // mtl 読み込んだ材質名をキーにした map
04294 // pos 頂点の位置
04295 // norm 頂点の法線
04296 // tex 頂点のテクスチャ座標
04297 // face 三角形のデータ
04298 //
04299 static bool ggParseObj(
04300     const std::string& name,
04301     std::vector<fgrp>& group,
04302     std::vector<GgSimpleShader::Material>& material,
04303     std::vector<vec3>& pos,
04304     std::vector<vec3>& norm,
04305     std::vector<vec2>& tex,
04306     std::vector<fidx>& face,
04307     bool normalize
04308 )
04309 {
04310     // ファイルパスからディレクトリ名を取り出す
04311     const std::string path{ name };
04312     const size_t base{ path.find_last_of("/\\") };
04313     const std::string dirname{ (base == std::string::npos) ? "" : path.substr(0, base + 1) };
04314
04315     // OBJ ファイルを読み込む
04316     std::ifstream file{ Utf8ToTChar(path) };
04317
04318     // 読み込みに失敗したら戻る
04319     if (file.fail())
04320     {
04321         #if defined(DEBUG)
04322             std::cerr << "Error: Can't open OBJ file: " << path << std::endl;
04323         #endif
04324         return false;
04325     }
04326
04327     // ポリゴングループの最初の三角形番号
04328     GLsizei startgroup{ static_cast<GLsizei>(group.size()) };
04329
04330     // スムーズシェーディングのスイッチ
04331     bool smooth{ false };
04332
04333     // 材質のテーブル
04334     std::map<std::string, GLuint> mtl;
04335
04336     // 現在の材質名 (ループの外で宣言する)
04337     std::string mtlname;
04338
04339     // 座標値の最小値・最大値
04340     vec3 bmin{ FLT_MAX }, bmax{ -FLT_MAX };
04341
04342     // 一行読み込み用のバッファ
04343     std::string line;
04344

```

```

04345 // データの読み込み
04346 while (std::getline(file, line))
04347 {
04348     // 空行は読み飛ばす
04349     if (line == "") continue;
04350
04351     // 最後の文字が '\r' なら
04352     if (*(line.end() - 1) == '\r')
04353     {
04354         // 最後の文字を削除する
04355         line.erase(line.end() - 1, line.end());
04356
04357         // 空行になったら読み飛ばす
04358         if (line == "") continue;
04359     }
04360
04361     // 一行を文字列ストリームに入れる
04362     std::istringstream str(line);
04363
04364     // 最初のトークンを命令 (op) とみなす
04365     std::string op;
04366     str >> op;
04367
04368     if (op[0] == '#') continue;
04369
04370     if (op == "v")
04371     {
04372         // 頂点位置
04373         vec3 v;
04374
04375         // 頂点位置はスペースで区切られている
04376         str >> v[0] >> v[1] >> v[2];
04377
04378         // 頂点位置を記録する
04379         pos.emplace_back(v);
04380
04381         // 頂点位置の最小値と最大値を求める (AABB)
04382         for (int i = 0; i < 3; ++i)
04383         {
04384             bmin[i] = std::min(bmin[i], v[i]);
04385             bmax[i] = std::max(bmax[i], v[i]);
04386         }
04387     }
04388     else if (op == "vt")
04389     {
04390         // テクスチャ座標
04391         vec2 t;
04392
04393         // 頂点位置はスペースで区切られている
04394         str >> t[0] >> t[1];
04395
04396         // テクスチャ座標を記録する
04397         tex.emplace_back(t);
04398     }
04399     else if (op == "vn")
04400     {
04401         // 頂点法線
04402         vec3 n;
04403
04404         // 頂点法線はスペースで区切られている
04405         str >> n[0] >> n[1] >> n[2];
04406
04407         // 頂点法線を記録する
04408         norm.emplace_back(n);
04409     }
04410     else if (op == "f")
04411     {
04412         // 三角形データ
04413         fidx f;
04414
04415         // スムースシェーディング
04416         f.smooth = smooth;
04417
04418         // 三頂点のそれぞれについて
04419         for (int i = 0; i < 3; ++i)
04420         {
04421             // 1項目取り出す
04422             std::string s;
04423             str >> s;
04424
04425             // 文字の位置
04426             auto c{ s.begin() };
04427
04428             // テクスチャ座標と法線の番号は未定義を表す 0 にしておく
04429             f.p[i] = f.t[i] = f.n[i] = 0;
04430
04431             // 項目の最初の要素は頂点座標番号

```

```

04432         while (c != s.end() && isdigit(*c)) f.p[i] = f.p[i] * 10 + *c++ - '0';
04433         if (c == s.end() || *c++ != '/') continue;
04434
04435         // 二つ目の項目はテクスチャ座標
04436         while (c != s.end() && isdigit(*c)) f.t[i] = f.t[i] * 10 + *c++ - '0';
04437         if (c == s.end() || *c++ != '/') continue;
04438
04439         // 三つ目の項目は法線番号
04440         while (c != s.end() && isdigit(*c)) f.n[i] = f.n[i] * 10 + *c++ - '0';
04441     }
04442
04443     // 三角形データを登録する
04444     face.emplace_back(f);
04445 }
04446 else if (op == "s")
04447 {
04448     // '1' だったらスムーズシェーディング有効
04449     std::string s;
04450     str >> s;
04451     smooth = s == "1";
04452 }
04453 else if (op == "usemtl")
04454 {
04455     // 次のポリゴングループの最初の三角形番号
04456     const GLsizei nextgroup(static_cast<GLsizei>(face.size()));
04457
04458     // ポリゴングループに三角形が存在すれば
04459     if (nextgroup > startgroup)
04460     {
04461         // ポリゴングループの三角形数と材質番号を記録する
04462         group.emplace_back(nextgroup, mtl[mtlname]);
04463
04464         // 次のポリゴングループの開始番号を保存しておく
04465         startgroup = nextgroup;
04466     }
04467
04468     // 次に usemtl が来るまで材質名を保持する
04469     str >> mtlname;
04470
04471     // 材質の存在チェック
04472     if (mtl.find(mtlname) == mtl.end())
04473     {
04474         #if defined(DEBUG)
04475             std::cerr << "Warning: Undefined material: " << mtlname << std::endl;
04476         #endif
04477
04478         // デフォルトの材質を割り当てておく
04479         mtlname = defaultMaterialName;
04480     }
04481     #if defined(DEBUG)
04482     else std::cerr << "usemtl: " << mtlname << std::endl;
04483     #endif
04484 }
04485 else if (op == "mtllib")
04486 {
04487     // MTL ファイルのパス名を作る
04488     str >> std::ws;
04489     std::string mtlpath;
04490     std::getline(str, mtlpath);
04491
04492     // MTL ファイルを読み込む
04493     ggLoadMtl(dirname + mtlpath, mtl, material);
04494 }
04495 }
04496
04497 // OBJ ファイルの読み込みに失敗したら戻る
04498 if (file.bad())
04499 {
04500     #if defined(DEBUG)
04501         std::cerr << "Error: Can't read OBJ file: " << path << std::endl;
04502     #endif
04503     return false;
04504 }
04505
04506 // 最後のポリゴングループの次の三角形番号
04507 const GLsizei nextgroup(static_cast<GLsizei>(face.size()));
04508 if (nextgroup > startgroup)
04509 {
04510     // 最後のポリゴングループの三角形数と材質を記録する
04511     group.emplace_back(nextgroup, static_cast<GLuint>(mtl[mtlname]));
04512 }
04513
04514 // スムーズシェーディングしない三角形の頂点を追加する
04515 for (auto& f : face)
04516 {
04517     if (!f.smooth)
04518     {

```

```

04519         // 三頂点のそれぞれについて
04520         for (int i = 0; i < 3; ++i)
04521         {
04522             // 新しい頂点座標を生成する (std::array の要素は emplace_back できない)
04523             pos.emplace_back(pos[f.p[i] - 1]);
04524             f.p[i] = static_cast<GLuint>(pos.size());
04525
04526             if (f.t[i] > 0)
04527             {
04528                 // 新しいテクスチャ座標を生成する
04529                 tex.emplace_back(tex[f.t[i] - 1]);
04530                 f.t[i] = static_cast<GLuint>(tex.size());
04531             }
04532
04533             if (f.n[i] > 0)
04534             {
04535                 // 新しい法線を生成する
04536                 norm.emplace_back(norm[f.n[i] - 1]);
04537                 f.n[i] = static_cast<GLuint>(norm.size());
04538             }
04539         }
04540     }
04541 }
04542
04543 // 法線データがなければ算出しておく
04544 if (norm.empty())
04545 {
04546     // 法線データ数の初期値は頂点数と同じでスムーズシェーディングのために初期値は 0
04547     norm.resize(pos.size(), { 0.0f, 0.0f, 0.0f });
04548
04549     // 面の法線の算出と頂点法線の算出
04550     for (auto& f : face)
04551     {
04552         // 頂点座標番号
04553         const auto v0{ f.p[0] - 1 };
04554         const auto v1{ f.p[1] - 1 };
04555         const auto v2{ f.p[2] - 1 };
04556
04557         // v1 - v0, v2 - v0 を求める
04558         const GLfloat d1[]{ pos[v1][0] - pos[v0][0], pos[v1][1] - pos[v0][1], pos[v1][2] - pos[v0][2] };
04559         const GLfloat d2[]{ pos[v2][0] - pos[v0][0], pos[v2][1] - pos[v0][1], pos[v2][2] - pos[v0][2] };
04560
04561         // 外積により面法線を求める
04562         vec3 n;
04563         ggCross(n.data(), d1, d2);
04564
04565         if (f.smooth)
04566         {
04567             // スムーズシェーディングを行うときは
04568             for (int i = 0; i < 3; ++i)
04569             {
04570                 // 面法線を頂点法線に積算する
04571                 norm[v0][i] += n[i];
04572                 norm[v1][i] += n[i];
04573                 norm[v2][i] += n[i];
04574
04575                 // 面の各頂点の法線番号は頂点番号と同じにする
04576                 f.n[i] = f.p[i];
04577             }
04578         }
04579         else
04580         {
04581             // 面法線を最初の頂点に保存する
04582             norm[v0] = n;
04583             f.n[0] = f.p[0];
04584
04585             // 2 頂点追加
04586             for (int i = 1; i < 3; ++i)
04587             {
04588                 norm.emplace_back(n);
04589                 f.n[i] = static_cast<GLuint>(norm.size());
04590             }
04591         }
04592     }
04593
04594     // 頂点の法線ベクトルを正規化する
04595     for (auto& n : norm) ggNormalize3(n.data());
04596 }
04597
04598 // 図形の正規化
04599 if (normalize)
04600 {
04601     // 図形の大きさ
04602     const auto sx{ bmax[0] - bmin[0] };
04603     const auto sy{ bmax[1] - bmin[1] };

```

```

04604     const auto sz{ bmax[2] - bmin[2] };
04605
04606     // 図形のスケール
04607     GLfloat s{ sx };
04608     if (sy > s) s = sy;
04609     if (sz > s) s = sz;
04610     const auto scale{ s != 0.0f ? 2.0f / s : 1.0f };
04611
04612     // 図形の中心位置
04613     const auto cx{ (bmax[0] + bmin[0]) * 0.5f };
04614     const auto cy{ (bmax[1] + bmin[1]) * 0.5f };
04615     const auto cz{ (bmax[2] + bmin[2]) * 0.5f };
04616
04617     // 図形の大きさと位置を正規化する
04618     for (auto& p : pos)
04619     {
04620         p[0] = (p[0] - cx) * scale;
04621         p[1] = (p[1] - cy) * scale;
04622         p[2] = (p[2] - cz) * scale;
04623     }
04624 }
04625
04626 #if defined(DEBUG)
04627     std::cerr
04628     << "[" << name << "]"\\n(Parsed) Group: " << group.size() << ", Material: " << mtl.size()
04629     << ", Pos: " << pos.size() << ", Norm: " << norm.size() << ", Tex: " << tex.size()
04630     << ", Face: " << face.size() << "\\n";
04631 #endif
04632
04633     // OBJ ファイルの読み込み成功
04634     return true;
04635 }
04636 }
04637
04638 //
04639 // 三角形分割された Alias OBJ 形式のファイルと MTL ファイルを読み込む (Arrays 形式)
04640 //
04641 //
04642 //     name 読み込むOBJ ファイル名
04643 //     group 読み込んだデータの各ポリゴングループの最初の三角形番号と三角形数
04644 //     material 読み込んだデータのポリゴングループごとの材質
04645 //     vert 読み込んだデータの頂点属性
04646 //     normalize true ならサイズを正規化する
04647 //     戻り値 読み込みに成功したら true
04648 //
04649 bool gg::ggLoadSimpleObj(const std::string& name,
04650     std::vector<std::array<GLuint, 3>>& group,
04651     std::vector<GgSimpleShader::Material>& material,
04652     std::vector<GgVertex>& vert,
04653     bool normalize)
04654 {
04655     // 読み込み用の一時記憶領域
04656     std::vector<fgrp> tgroup;
04657     std::vector<vec3> tpos;
04658     std::vector<vec3> tnorm;
04659     std::vector<vec2> ttex;
04660     std::vector<fidx> tface;
04661
04662     // OBJ ファイルを解析する
04663     if (!ggParseObj(name, tgroup, material, tpos, tnorm, ttex, tface, normalize)) return false;
04664
04665     // 頂点属性データのメモリを確保する
04666     vert.reserve(vert.size() + tface.size() * 3);
04667
04668     // ポリゴングループデータのメモリを確保する
04669     group.reserve(group.size() + tgroup.size());
04670     material.reserve(material.size() + tgroup.size());
04671
04672     // ポリゴングループの最初の三角形番号
04673     GLuint startgroup{ 0 };
04674
04675     // ポリゴングループデータの作成
04676     for (auto& g : tgroup)
04677     {
04678         // このポリゴングループの最初の頂点番号と頂点数・材質番号
04679         std::array<GLuint, 3> v;
04680
04681         // このポリゴングループの最初の頂点番号を保存する
04682         v[0] = static_cast<GLuint>(vert.size());
04683
04684         // 三角形ごとの頂点データの作成
04685         for (GLuint j = startgroup; j < g.nextgroup; ++j)
04686         {
04687             // 処理対象の三角形
04688             auto& f = tface[j];
04689
04690             // 三頂点のそれぞれについて
04691             for (int i = 0; i < 3; ++i)

```

```

04692     {
04693         // テクスチャ座標
04694         vec2 tex{ 0.0f, 0.0f };
04695         if (f.t[i] > 0) tex = ttex[f.t[i] - 1];
04696
04697         // 頂点法線
04698         vec3 norm{ 0.0f, 0.0f, 0.0f };
04699         if (f.n[i] > 0) norm = tnorm[f.n[i] - 1];
04700
04701         // 頂点属性の追加
04702         vert.emplace_back(tpos[f.p[i] - 1].data(), norm.data());
04703     }
04704 }
04705
04706 // このポリゴングループの頂点数を保存する
04707 v[1] = static_cast<GLuint>(vert.size()) - v[0];
04708 v[2] = g.mtlno;
04709
04710 // このポリゴングループの最初の頂点番号と頂点数・材質番号を登録する
04711 group.emplace_back(v);
04712
04713 // 次のポリゴングループの最初の三角形番号を求める
04714 startgroup = g.nextgroup;
04715 }
04716
04717 #if defined(DEBUG)
04718     std::cerr
04719     << "(Stored) Group: " << group.size() << ", Material: " << material.size()
04720     << ", Vertex: " << vert.size() << "\n";
04721 #endif
04722
04723 // OBJ ファイルの読み込み成功
04724 return true;
04725 }
04726
04727 //
04728 // 三角形分割された Alias OBJ 形式のファイルと MTL ファイルを読み込む (Elements 形式)
04729 //
04730 //     name 読み込むOBJ ファイル名
04731 //     group 読み込んだデータの各ポリゴングループの最初の三角形番号と三角形数
04732 //     material 読み込んだデータのポリゴングループごとの材質
04733 //     vert 読み込んだデータの頂点属性
04734 //     face 読み込んだデータの三角形の頂点インデックス
04735 //     normalize true ならサイズを正規化する
04736 //     戻り値 読み込みに成功したら true
04737 //
04738 bool gg::ggLoadSimpleObj(const std::string& name,
04739     std::vector<std::array<GLuint, 3>>& group,
04740     std::vector<GgSimpleShader::Material>& material,
04741     std::vector<GgVertex>& vert,
04742     std::vector<GLuint>& face,
04743     bool normalize)
04744 {
04745     // 読み込み用の一時記憶領域
04746     std::vector<fgrp> tgroup;
04747     std::vector<vec3> tpos;
04748     std::vector<vec3> tnorm;
04749     std::vector<vec2> ttex;
04750     std::vector<fidx> tface;
04751
04752     // OBJ ファイルを解析する
04753     if (!ggParseObj(name, tgroup, material, tpos, tnorm, ttex, tface, normalize)) return false;
04754
04755     // 頂点属性データの最初の頂点番号
04756     const auto vertbase{ static_cast<GLuint>(vert.size()) };
04757
04758     // 頂点属性データのメモリを確保する
04759     vert.resize(vertbase + tpos.size());
04760
04761     // 三角形データのメモリを確保する
04762     face.reserve(face.size() + tface.size());
04763
04764     // ポリゴングループデータのメモリを確保する
04765     group.reserve(group.size() + tgroup.size());
04766     material.reserve(material.size() + tgroup.size());
04767
04768     // ポリゴングループの最初の三角形番号
04769     GLuint startgroup{ 0 };
04770
04771     // ポリゴングループデータの作成
04772     for (auto& g : tgroup)
04773     {
04774         // このポリゴングループの最初の頂点番号
04775         const auto first{ static_cast<GLuint>(face.size()) };
04776
04777         // 三角形ごとの頂点データの作成
04778         for (GLuint j = startgroup; j < g.nextgroup; ++j)

```

```

04779     {
04780         // 処理対象の三角形
04781         auto& f{ tface[j] };
04782
04783         // 三頂点のそれぞれについて
04784         for (int i = 0; i < 3; ++i)
04785         {
04786             // 追加する三角形データの頂点番号
04787             const auto q{ f.p[i] - 1 + vertbase };
04788
04789             // 三角形データの追加
04790             face.emplace_back(q);
04791
04792             // テクスチャ座標番号
04793             vec2 tex{ 0.0f, 0.0f };
04794             if (f.t[i] > 0) tex = ttex[f.t[i] - 1];
04795
04796             // 頂点法線番号
04797             vec3 norm{ 0.0f, 0.0f, 0.0f };
04798             if (f.n[i] > 0) norm = tnorm[f.n[i] - 1];
04799
04800             // 頂点の格納
04801             vert[q] = GgVertex(tpos[f.p[i] - 1].data(), norm.data());
04802         }
04803     }
04804
04805     // このポリゴングループの最初の三角形番号と三角形数・材質番号を登録する
04806     group.emplace_back(std::array<GLuint, 3>{ first, static_cast<GLuint>(face.size()) - first, g.mtlno
});
04807
04808     // 次のポリゴングループの最初の三角形番号を求める
04809     startgroup = g.nextgroup;
04810 }
04811
04812 #if defined(DEBUG)
04813     std::cerr
04814     << "(Stored) Group: " << group.size() << ", Material: " << material.size()
04815     << ", Vertex: " << vert.size() << ", Face: " << face.size() << "\n";
04816 #endif
04817
04818     // OBJ ファイルの読み込み成功
04819     return true;
04820 }
04821
04822 //
04823 // シェーダオブジェクトのコンパイル結果を表示する
04824 //
04825 static GLboolean printShaderInfoLog(GLuint shader, const std::string& str)
04826 {
04827     // コンパイル結果を取得する
04828     GLint status;
04829     glGetShaderiv(shader, GL_COMPILE_STATUS, &status);
04830     #if defined(DEBUG)
04831     if (status == GL_FALSE) std::cerr << "Compile Error in " << str << std::endl;
04832     #endif
04833
04834     // シェーダのコンパイル時のログの長さを取得する
04835     GLsizei bufSize;
04836     glGetShaderiv(shader, GL_INFO_LOG_LENGTH, &bufSize);
04837
04838     if (bufSize > 1)
04839     {
04840         // シェーダのコンパイル時のログの内容を取得する
04841         std::vector<GLchar> infoLog(bufSize);
04842         GLsizei length;
04843         glGetShaderInfoLog(shader, bufSize, &length, infoLog.data());
04844         #if defined(DEBUG)
04845         std::cerr << infoLog.data();
04846         #endif
04847     }
04848
04849     // コンパイル結果を返す
04850     return static_cast<GLboolean>(status);
04851 }
04852
04853 //
04854 // プログラムオブジェクトのリンク結果を表示する
04855 //
04856 static GLboolean printProgramInfoLog(GLuint program)
04857 {
04858     // リンク結果を取得する
04859     GLint status;
04860     glGetProgramiv(program, GL_LINK_STATUS, &status);
04861     #if defined(DEBUG)
04862     if (status == GL_FALSE) std::cerr << "Link Error." << std::endl;
04863     #endif
04864 }

```



```

04865 // シェーダのリンク時のログの長さを取得する
04866 GLsizei bufSize;
04867 glGetProgramiv(program, GL_INFO_LOG_LENGTH, &bufSize);
04868
04869 // シェーダのリンク時のログの内容を取得する
04870 if (bufSize > 1)
04871 {
04872     std::vector<GLchar> infoLog(bufSize);
04873     GLsizei length;
04874     glGetProgramInfoLog(program, bufSize, &length, infoLog.data());
04875 #if defined(DEBUG)
04876     std::cerr << infoLog.data() << std::endl;
04877 #endif
04878 }
04879
04880 // リンク結果を返す
04881 return static_cast<GLboolean>(status);
04882 }
04883
04884 //
04885 // シェーダのソースプログラムの文字列を読み込んでプログラムオブジェクトを作成する
04886 //
04887 // vsrc パーテックスシェーダのソースプログラムの文字列
04888 // fsrc フラグメントシェーダのソースプログラムの文字列 (nullptr なら不使用)
04889 // gsrc ジオメトリシェーダのソースプログラムの文字列 (nullptr なら不使用)
04890 // nvarying フィードバックする varying 変数の数 (0 なら不使用)
04891 // varyings フィードバックする varying 変数のリスト (nullptr なら不使用)
04892 // vtext パーテックスシェーダのコンパイル時のメッセージに追加する文字列
04893 // ftext フラグメントシェーダのコンパイル時のメッセージに追加する文字列
04894 // gtext ジオメトリシェーダのコンパイル時のメッセージに追加する文字列
04895 // 戻り値 プログラムオブジェクトのプログラム名 (作成できなければ 0)
04896 //
04897 GLuint gg::ggCreateShader(
04898     const std::string& vsrc,
04899     const std::string& fsrc,
04900     const std::string& gsrc,
04901     GLint nvarying,
04902     const char* const* varyings,
04903     const std::string& vtext,
04904     const std::string& ftext,
04905     const std::string& gtext)
04906 {
04907     // シェーダプログラムの作成
04908     const auto program{ glCreateProgram() };
04909
04910     if (program > 0)
04911     {
04912         bool status = true;
04913
04914         if (!vsrc.empty())
04915         {
04916             // パーテックスシェーダのシェーダオブジェクトを作成する
04917             const auto vertShader{ glCreateShader(GL_VERTEX_SHADER) };
04918             const auto* vsrcp{ vsrc.c_str() };
04919             glShaderSource(vertShader, 1, &vsrcp, nullptr);
04920             glCompileShader(vertShader);
04921
04922             // パーテックスシェーダのシェーダオブジェクトをプログラムオブジェクトに組み込む
04923             if (printShaderInfoLog(vertShader, vtext))
04924                 glAttachShader(program, vertShader);
04925             else
04926                 status = false;
04927             glDeleteShader(vertShader);
04928         }
04929
04930         if (!fsrc.empty())
04931         {
04932             // フラグメントシェーダのシェーダオブジェクトを作成する
04933             const auto fragShader{ glCreateShader(GL_FRAGMENT_SHADER) };
04934             const auto* fsrcp{ fsrc.c_str() };
04935             glShaderSource(fragShader, 1, &fsrcp, nullptr);
04936             glCompileShader(fragShader);
04937
04938             // フラグメントシェーダのシェーダオブジェクトをプログラムオブジェクトに組み込む
04939             if (printShaderInfoLog(fragShader, ftext))
04940                 glAttachShader(program, fragShader);
04941             else
04942                 status = false;
04943             glDeleteShader(fragShader);
04944         }
04945
04946         if (!gsrc.empty())
04947         {
04948             #if defined(GL_GLES_PROTOTYPES)
04949             # if defined(DEBUG)
04950                 std::cerr << gtext << ": The geometry shader is not supported." << std::endl;
04951                 status = false;

```

```

04952 # endif
04953 #else
04954     // ジオメトリシェーダのシェーダオブジェクトを作成する
04955     const auto geomShader{ glCreateShader(GL_GEOMETRY_SHADER) };
04956     const auto* gsrcp{ gsrc.c_str() };
04957     glShaderSource(geomShader, 1, &gsrcp, nullptr);
04958     glCompileShader(geomShader);
04959
04960     // ジオメトリシェーダのシェーダオブジェクトをプログラムオブジェクトに組み込む
04961     if (printShaderInfoLog(geomShader, gtext))
04962         glAttachShader(program, geomShader);
04963     else
04964         status = false;
04965     glDeleteShader(geomShader);
04966 #endif
04967 }
04968
04969 // 全てのシェーダオブジェクトのコンパイルに成功したら
04970 if (status)
04971 {
04972     // feedback に使う varying 変数を指定する
04973     if (nvarying > 0)
04974         glTransformFeedbackVaryings(program, nvarying, varyings, GL_SEPARATE_ATTRIBS);
04975
04976     // シェーダプログラムをリンクする
04977     glLinkProgram(program);
04978
04979     // リンクに成功したらプログラムオブジェクト名を返す
04980     if (printProgramInfoLog(program) != GL_FALSE) return program;
04981 }
04982 }
04983
04984 // プログラムオブジェクトが作成できなかった
04985 glDeleteProgram(program);
04986 return 0;
04987 }
04988
04989 //
04990 // シェーダのソースファイルを読み込んだ vector を返す
04991 //
04992 // name ソースファイル名
04993 // src 読み込んだソースファイルの文字列
04994 // 戻り値 読み込みの成功すれば true, 失敗したら false
04995 //
04996 static bool readShaderSource(const std::string& name, std::string& src)
04997 {
04998     // ファイル名が nullptr ならそのまま戻る
04999     if (name.empty()) return true;
05000
05001     // ソースファイルを開く
05002     std::ifstream file{ Utf8ToTChar(name), std::ios::binary };
05003     if (file.fail())
05004     {
05005         // ファイルが開けなければエラーで戻る
05006         #if defined(DEBUG)
05007             std::cerr << "Error: Can't open source file: " << name << std::endl;
05008         #endif
05009         return false;
05010     }
05011
05012     // ファイル全体を文字列として読み込む
05013     std::istreambuf_iterator<char> it{ file };
05014     std::istreambuf_iterator<char> last;
05015     src = std::string(it, last);
05016
05017     // ファイルがうまく読み込めなければ戻る
05018     if (file.bad())
05019     {
05020         #if defined(DEBUG)
05021             std::cerr << "Error: Could not read source file: " << name << std::endl;
05022         #endif
05023         return false;
05024     }
05025
05026     // ファイルを閉じて戻る
05027     return true;
05028 }
05029
05030 //
05031 // シェーダのソースファイルを読み込んでプログラムオブジェクトを作成する
05032 //
05033 // vert バーテックスシェーダのソースファイル名
05034 // frag フラグメントシェーダのソースファイル名 (nullptr なら不使用)
05035 // geom ジオメトリシェーダのソースファイル名 (nullptr なら不使用)
05036 // nvarying フィードバックする varying 変数の数 (0 なら不使用)
05037 // varyings フィードバックする varying 変数のリスト (nullptr なら不使用)
05038 // 戻り値 シェーダプログラムのプログラム名 (作成できなければ 0)

```

```

05039 //
05040 GLuint gg::ggLoadShader(
05041     const std::string& vert,
05042     const std::string& frag,
05043     const std::string& geom,
05044     GLint nvarying,
05045     const char* const* varyings
05046 )
05047 {
05048     // 読み込んだシェーダのソースプログラム
05049     std::string vsrc, fsrc, gsrc;
05050
05051     // 読み込んだ結果
05052     bool status;
05053
05054     // シェーダのソースファイルを読み込む
05055     status = readShaderSource(vert, vsrc);
05056     status = readShaderSource(frag, fsrc) && status;
05057     status = readShaderSource(geom, gsrc) && status;
05058
05059     // 全てのソースファイルが読み込めていなかったらエラー
05060     if (!status) return 0;
05061
05062     // プログラムオブジェクトを作成する
05063     return ggCreateShader(vsrc, fsrc, gsrc, nvarying, varyings, vert, frag, geom);
05064 }
05065
05066 #if !defined(__APPLE__) && !defined(GL_GLES_PROTOTYPES)
05067 //
05068 // コンピュートシェーダのソースプログラムの文字列を読み込んでプログラムオブジェクトを作成する
05069 //
05070 //     csrc コンピュートシェーダのソースプログラムの文字列
05071 //     戻り値 プログラムオブジェクトのプログラム名 (作成できなければ 0)
05072 //
05073 GLuint gg::ggCreateComputeShader(const std::string& csrc, const std::string& ctext)
05074 {
05075     // シェーダプログラムの作成
05076     const auto program{ glCreateProgram() };
05077
05078     if (program > 0)
05079     {
05080         // ソースプログラムの文字列が空だったら 0 を返す
05081         if (csrc.empty()) return 0;
05082
05083         // コンピュートシェーダのシェーダオブジェクトを作成する
05084         const auto compShader{ glCreateShader(GL_COMPUTE_SHADER) };
05085         const auto* csrcp{ csrc.c_str() };
05086         glShaderSource(compShader, 1, &csrcp, nullptr);
05087         glCompileShader(compShader);
05088
05089         // コンピュートシェーダのシェーダオブジェクトをプログラムオブジェクトに組み込む
05090         if (printShaderInfoLog(compShader, ctext)) glAttachShader(program, compShader);
05091         glDeleteShader(compShader);
05092
05093         // シェーダプログラムをリンクする
05094         glLinkProgram(program);
05095
05096         // プログラムオブジェクトが作成できなければ 0 を返す
05097         if (printProgramInfoLog(program) == GL_FALSE)
05098         {
05099             glDeleteProgram(program);
05100             return 0;
05101         }
05102     }
05103
05104     // プログラムオブジェクトを返す
05105     return program;
05106 }
05107
05108 //
05109 // コンピュートシェーダのソースファイルを読み込んでプログラムオブジェクトを作成する
05110 //
05111 //     comp コンピュートシェーダのソースファイル名
05112 //     戻り値 プログラムオブジェクトのプログラム名 (作成できなければ 0)
05113 //
05114 GLuint gg::ggLoadComputeShader(const std::string& comp)
05115 {
05116     // シェーダのソースファイルを読み込む
05117     std::string csrc;
05118     if (readShaderSource(comp, csrc))
05119     {
05120         // プログラムオブジェクトを作成する
05121         return ggCreateComputeShader(csrc.data(), comp);
05122     }
05123
05124     // プログラムオブジェクト作成失敗
05125     return 0;

```

```

05126 }
05127 #endif
05128
05129 //
05130 // 点：データ作成
05131 //
05132 void gg::GgPoints::load(const GgVector* pos, GLsizei count, GLenum usage)
05133 {
05134     // 頂点バッファオブジェクトを作成する
05135     position = std::make_shared<GgBuffer<GgVector>>(GL_ARRAY_BUFFER, pos,
05136         static_cast<GLsizei>(sizeof(GgVector)), count, usage);
05137     // このバッファオブジェクトは index == 0 の in 変数から入力する
05138     glVertexAttribPointer(0, static_cast<GLint>(pos->size()), GL_FLOAT, GL_FALSE, 0, 0);
05139     glEnableVertexAttribArray(0);
05140 }
05141
05142 //
05143 // 点：描画
05144 //
05145 void gg::GgPoints::draw(GLint first, GLsizei count) const
05146 {
05147     // 頂点配列オブジェクトを指定する
05148     GgShape::draw(first, count);
05149     // 図形を描画する
05150     glDrawArrays(getMode(), first, count > 0 ? count : getCount() - first);
05151 }
05152
05153 //
05154 // 三角形：データ作成
05155 //
05156 void gg::GgTriangles::load(const GgVertex* vert, GLsizei count, GLenum usage)
05157 {
05158     // 頂点バッファオブジェクトを作成する
05159     vertex = std::make_shared<GgBuffer<GgVertex>>(GL_ARRAY_BUFFER, vert,
05160         static_cast<GLsizei>(sizeof(GgVertex)), count, usage);
05161     // 頂点の位置は index == 0 の in 変数から入力する
05162     glVertexAttribPointer(0, static_cast<GLint>(vert->position.size()), GL_FLOAT, GL_FALSE,
05163         sizeof(GgVertex), static_cast<const char*>(0) + offsetof(GgVertex, position));
05164     glEnableVertexAttribArray(0);
05165     // 頂点の法線は index == 1 の in 変数から入力する
05166     glVertexAttribPointer(1, static_cast<GLint>(vert->normal.size()), GL_FLOAT, GL_FALSE,
05167         sizeof(GgVertex), static_cast<const char*>(0) + offsetof(GgVertex, normal));
05168     glEnableVertexAttribArray(1);
05169 }
05170
05171 //
05172 // 三角形：描画
05173 //
05174 void gg::GgTriangles::draw(GLint first, GLsizei count) const
05175 {
05176     // 頂点配列オブジェクトを指定する
05177     GgShape::draw(first, count);
05178     // 図形を描画する
05179     glDrawArrays(getMode(), first, count > 0 ? count : getCount() - first);
05180 }
05181
05182 //
05183 // オブジェクト：描画
05184 //
05185 void gg::GgElements::draw(GLint first, GLsizei count) const
05186 {
05187     // 頂点配列オブジェクトを指定する
05188     GgShape::draw(first, count);
05189     // 図形を描画する
05190     glDrawElements(getMode(), count > 0 ? count : getIndexCount() - first,
05191         GL_UNSIGNED_INT, static_cast<GLuint*>(0) + first);
05192 }
05193
05194 //
05195 // 点群を立方体状に生成する
05196 //
05197 std::shared_ptr<gg::GgPoints> gg::ggPointsCube(GLsizei count, GLfloat length, GLfloat cx, GLfloat cy,
05198     GLfloat cz)
05199 {
05200     // メモリを確保する
05201     std::vector<GgVector> pos;
05202     pos.reserve(count);
05203     // 点を生成する
05204     for (GLsizei v = 0; v < count; ++v)
05205     {

```

```

05210     const GgVector p
05211     {
05212         (static_cast<GLfloat>(rand()) / static_cast<GLfloat>(RAND_MAX) - 0.5f) * length + cx,
05213         (static_cast<GLfloat>(rand()) / static_cast<GLfloat>(RAND_MAX) - 0.5f) * length + cy,
05214         (static_cast<GLfloat>(rand()) / static_cast<GLfloat>(RAND_MAX) - 0.5f) * length + cz,
05215         1.0f
05216     };
05217
05218     pos.emplace_back(p);
05219 }
05220
05221 // 点データの GgPoints オブジェクトを作成して返す
05222 return std::make_shared<gg::GgPoints>(pos.data(), static_cast<GLsizei>(pos.size()), GL_POINTS);
05223 }
05224
05225 //
05226 // 点群を球状に生成する
05227 //
05228 std::shared_ptr<gg::GgPoints> gg::ggPointsSphere(GLsizei count, GLfloat radius,
05229 GLfloat cx, GLfloat cy, GLfloat cz)
05230 {
05231     // メモリを確保する
05232     std::vector<GgVector> pos;
05233     pos.reserve(count);
05234
05235     // 点を生成する
05236     for (GLsizei v = 0; v < count; ++v)
05237     {
05238         const auto r{ radius * static_cast<GLfloat>(rand()) / static_cast<GLfloat>(RAND_MAX) };
05239         const auto t{ 6.28318530718f * static_cast<GLfloat>(rand()) / (static_cast<GLfloat>(RAND_MAX) +
05240 1.0f) };
05241         const auto cp{ 2.0f * static_cast<GLfloat>(rand()) / static_cast<GLfloat>(RAND_MAX) - 1.0f };
05242         const auto sp{ sqrtf(1.0f - cp * cp) };
05243         const auto ct{ cosf(t) };
05244         const auto st{ sinf(t) };
05245         const GgVector p{ r * sp * ct + cx, r * sp * st + cy, r * cp + cz, 1.0f };
05246
05247         pos.emplace_back(p);
05248     }
05249
05250 // 点データの GgPoints オブジェクトを作成して返す
05251 return std::make_shared<gg::GgPoints>(pos.data(), static_cast<GLsizei>(pos.size()), GL_POINTS);
05252 }
05253 //
05254 // 矩形状に 2 枚の三角形を生成する
05255 //
05256 std::shared_ptr<gg::GgTriangles> gg::ggRectangle(GLfloat width, GLfloat height)
05257 {
05258     // 頂点属性
05259     std::array<gg::GgVertex, 4> vert;
05260
05261     // 頂点位置と法線を求める
05262     for (int v = 0; v < 4; ++v)
05263     {
05264         const auto x{ ((v & 1) * 2 - 1) * width };
05265         const auto y{ ((v & 2) - 1) * height };
05266
05267         vert[v] = gg::GgVertex{ x, y, 0.0f, 0.0f, 0.0f, 1.0f };
05268     }
05269
05270 // 矩形の GgTriangles オブジェクトを作成する
05271 return std::make_shared<gg::GgTriangles>(vert.data(), static_cast<GLsizei>(vert.size()),
GL_TRIANGLE_STRIP);
05272 }
05273 //
05274 // 楕円状に三角形を生成する
05275 //
05276 std::shared_ptr<gg::GgTriangles> gg::ggEllipse(GLfloat width, GLfloat height, GLuint slices)
05277 {
05278     // 楕円のスケール
05279     constexpr GLfloat scale{ 0.5f };
05280
05281     // 作業用のメモリ
05282     std::vector<gg::GgVertex> vert;
05283
05284     // 頂点位置と法線を求める
05285     for (GLuint v = 0; v < slices; ++v)
05286     {
05287         const auto t{ 6.28318530717f * static_cast<GLfloat>(v) / static_cast<GLfloat>(slices) };
05288         const auto x{ cosf(t) * width * scale };
05289         const auto y{ sinf(t) * height * scale };
05290
05291         vert.emplace_back(x, y, 0.0f, 0.0f, 0.0f, 1.0f);
05292     }
05293
05294 }

```

```

05295 // GgTriangles オブジェクトを作成する
05296 return std::make_shared<gg::GgTriangles>(vert.data(), static_cast<GLsizei>(vert.size()),
GL_TRIANGLE_FAN);
05297 }
05298
05299 //
05300 // Wavefront OBJ ファイルを読み込む (Arrays 形式)
05301 //
05302 std::shared_ptr<gg::GgTriangles> gg::ggArraysObj(const std::string& name, bool normalize)
05303 {
05304     std::vector<std::array<GLuint, 3>> group;
05305     std::vector<gg::GgSimpleShader::Material> material;
05306     std::vector<gg::GgVertex> vert;
05307
05308     // ファイルを読み込む
05309     if (!ggLoadSimpleObj(name, group, material, vert, normalize)) return nullptr;
05310
05311     // GgTriangles オブジェクトを作成する
05312     return std::make_shared<gg::GgTriangles>(vert.data(), static_cast<GLsizei>(vert.size()),
GL_TRIANGLES);
05313 }
05314
05315 //
05316 // Wavefront OBJ ファイルを読み込む (Elements 形式)
05317 //
05318 std::shared_ptr<gg::GgElements> gg::ggElementsObj(const std::string& name, bool normalize)
05319 {
05320     std::vector<std::array<GLuint, 3>> group;
05321     std::vector<gg::GgSimpleShader::Material> material;
05322     std::vector<gg::GgVertex> vert;
05323     std::vector<GLuint> face;
05324
05325     // ファイルを読み込む
05326     if (!ggLoadSimpleObj(name, group, material, vert, face, normalize)) return nullptr;
05327
05328     // GgElements オブジェクトを作成する
05329     return std::make_shared<gg::GgElements>(vert.data(), static_cast<GLsizei>(vert.size()),
face.data(), static_cast<GLsizei>(face.size()), GL_TRIANGLES);
05330 }
05331
05332 //
05333 // メッシュ形状を作成する (Elements 形式)
05334 //
05335 std::shared_ptr<gg::GgElements> gg::ggElementsMesh(GLuint slices, GLuint stacks, const
GLfloat (*pos)[3], const GLfloat (*norm)[3])
05336 {
05337     // 頂点属性
05338     std::vector<gg::GgVertex> vert((slices + 1) * (stacks + 1));
05339
05340     // 頂点の法線を求める
05341     for (GLuint j = 0; j <= stacks; ++j)
05342     {
05343         for (GLuint i = 0; i <= slices; ++i)
05344         {
05345             // 処理対象の頂点番号
05346             const auto k{ j * (slices + 1) + i };
05347
05348             // 頂点の法線
05349             gg::GgVector tnorm;
05350             tnorm[3] = 0.0f;
05351
05352             if (norm)
05353             {
05354                 tnorm[0] = norm[k][0];
05355                 tnorm[1] = norm[k][1];
05356                 tnorm[2] = norm[k][2];
05357             }
05358             else
05359             {
05360                 // 処理対象の頂点の周囲の頂点番号
05361                 const auto kim{ i > 0 ? k - 1 : k };
05362                 const auto kip{ i < slices ? k + 1 : k };
05363                 const auto kjm{ j > 0 ? k - slices - 1 : k };
05364                 const auto kjp{ j < stacks ? k + slices + 1 : k };
05365
05366                 // 接線ベクトル
05367                 const std::array<GLfloat, 3> t
05368                 {
05369                     pos[kip][0] - pos[kim][0],
05370                     pos[kip][1] - pos[kim][1],
05371                     pos[kip][2] - pos[kim][2]
05372                 };
05373
05374                 // 従接線ベクトル
05375                 const std::array<GLfloat, 3> b
05376                 {
05377                     pos[kjp][0] - pos[kjm][0],

```

```

05379         pos[kjp][1] - pos[kjm][1],
05380         pos[kjp][2] - pos[kjm][2]
05381     };
05382
05383     // 法線
05384     tnorm[0] = t[1] * b[2] - t[2] * b[1];
05385     tnorm[1] = t[2] * b[0] - t[0] * b[2];
05386     tnorm[2] = t[0] * b[1] - t[1] * b[0];
05387
05388     // 法線の正規化
05389     ggNormalize3(tnorm);
05390 }
05391
05392 // 頂点の位置
05393 const gg::GgVector tpos{ pos[k][0], pos[k][1], pos[k][2], 1.0f };
05394
05395 // 頂点属性の保存
05396 vert[k] = gg::GgVertex{ tpos, tnorm };
05397 }
05398 }
05399
05400 // 頂点のインデックス (三角形データ)
05401 std::vector<GLuint> face;
05402
05403 // 頂点のインデックスを求める
05404 for (GLuint j = 0; j < stacks; ++j)
05405 {
05406     for (GLuint i = 0; i < slices; ++i)
05407     {
05408         // 処理対象のマス
05409         const auto k{ (slices + 1) * j + i };
05410
05411         // マスの上半分の三角形
05412         face.emplace_back(k);
05413         face.emplace_back(k + slices + 2);
05414         face.emplace_back(k + 1);
05415
05416         // マスの下半分の三角形
05417         face.emplace_back(k);
05418         face.emplace_back(k + slices + 1);
05419         face.emplace_back(k + slices + 2);
05420     }
05421 }
05422
05423 // GgElements オブジェクトを作成する
05424 return std::make_shared<GgElements>(vert.data(), static_cast<GLsizei>(vert.size()),
05425     face.data(), static_cast<GLsizei>(face.size()), GL_TRIANGLES);
05426 }
05427
05428 //
05429 // 球状に三角形データを生成する (Elements 形式)
05430 //
05431 std::shared_ptr<gg::GgElements> gg::ggElementsSphere(GLfloat radius, int slices, int stacks)
05432 {
05433     // 頂点の位置と法線
05434     std::vector<GLfloat> p, n;
05435
05436     // 頂点の位置と法線を求める
05437     for (int j = 0; j <= stacks; ++j)
05438     {
05439         const auto t{ static_cast<GLfloat>(j) / static_cast<GLfloat>(stacks) };
05440         const auto ph{ 3.1415926536f * t };
05441         const auto y{ cosf(ph) };
05442         const auto r{ sinf(ph) };
05443
05444         for (int i = 0; i <= slices; ++i)
05445         {
05446             const auto s{ static_cast<GLfloat>(i) / static_cast<GLfloat>(slices) };
05447             const auto th{ -2.0f * 3.1415926536f * s };
05448             const auto x{ r * cosf(th) };
05449             const auto z{ r * sinf(th) };
05450
05451             // 頂点の座標値
05452             p.emplace_back(x * radius);
05453             p.emplace_back(y * radius);
05454             p.emplace_back(z * radius);
05455
05456             // 頂点の法線
05457             n.emplace_back(x);
05458             n.emplace_back(y);
05459             n.emplace_back(z);
05460         }
05461     }
05462
05463     // GgElements オブジェクトを作成する
05464     return ggElementsMesh(slices, stacks, reinterpret_cast<GLfloat(*)[3]>(p.data()),
05465         reinterpret_cast<GLfloat(*)[3]>(n.data()));

```

```

05466 }
05467
05468 //
05469 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の強度の環境光成分を設定する
05470 //
05471 //   r 光源の強度の環境光成分の赤成分
05472 //   g 光源の強度の環境光成分の緑成分
05473 //   b 光源の強度の環境光成分の青成分
05474 //   a 光源の強度の環境光成分の不透明度
05475 //   first 値を設定する光源データの最初の番号
05476 //   count 値を設定する光源データの数
05477 //
05478 void gg::GgSimpleShader::LightBuffer::loadAmbient(GLfloat r, GLfloat g, GLfloat b, GLfloat a,
05479 GLint first, GLsizei count) const
05480 {
05481     // データを格納するバッファオブジェクトの先頭のポインタ
05482     const auto start{ static_cast<char*>(map(first, count)) };
05483     for (GLsizei i = 0; i < count; ++i)
05484     {
05485         // バッファオブジェクトの i 番目のブロックのポインタ
05486         const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05487
05488         // 光源の環境光成分を設定する
05489         light->ambient[0] = r;
05490         light->ambient[1] = g;
05491         light->ambient[2] = b;
05492         light->ambient[3] = a;
05493     }
05494     unmap();
05495 }
05496
05497 //
05498 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の強度の環境光成分を設定する
05499 //
05500 //   ambient 光源の強度の環境光成分
05501 //   first 値を設定する光源データの最初の番号
05502 //   count 値を設定する光源データの数
05503 //
05504 void gg::GgSimpleShader::LightBuffer::loadAmbient(const GgVector& ambient,
05505 GLint first, GLsizei count) const
05506 {
05507     // データを格納するバッファオブジェクトの先頭のポインタ
05508     const auto start{ static_cast<char*>(map(first, count)) };
05509     for (GLsizei i = 0; i < count; ++i)
05510     {
05511         // バッファオブジェクトの i 番目のブロックのポインタ
05512         const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05513
05514         // 光源の強度の環境光成分を設定する
05515         light->ambient = ambient;
05516     }
05517     unmap();
05518 }
05519
05520 //
05521 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の強度の拡散反射光成分を設定する
05522 //
05523 //   r 光源の強度の拡散反射光成分の赤成分
05524 //   g 光源の強度の拡散反射光成分の緑成分
05525 //   b 光源の強度の拡散反射光成分の青成分
05526 //   a 光源の強度の拡散反射光成分の不透明度
05527 //   first 値を設定する光源データの最初の番号
05528 //   count 値を設定する光源データの数
05529 //
05530 void gg::GgSimpleShader::LightBuffer::loadDiffuse(GLfloat r, GLfloat g, GLfloat b, GLfloat a,
05531 GLint first, GLsizei count) const
05532 {
05533     // データを格納するバッファオブジェクトの先頭のポインタ
05534     const auto start{ static_cast<char*>(map(first, count)) };
05535     for (GLsizei i = 0; i < count; ++i)
05536     {
05537         // バッファオブジェクトの i 番目のブロックのポインタ
05538         const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05539
05540         // 光源の拡散反射光成分を設定する
05541         light->diffuse[0] = r;
05542         light->diffuse[1] = g;
05543         light->diffuse[2] = b;
05544         light->diffuse[3] = a;
05545     }
05546     unmap();
05547 }
05548
05549 //
05550 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の強度の拡散反射光成分を設定する
05551 //
05552 //   ambient 光源の強度の拡散反射光成分

```



```

05553 // first 値を設定する光源データの最初の番号
05554 // count 値を設定する光源データの数
05555 //
05556 void gg::GgSimpleShader::LightBuffer::loadDiffuse(const GgVector& diffuse,
05557 GLint first, GLsizei count) const
05558 {
05559 // データを格納するバッファオブジェクトの先頭のポインタ
05560 const auto start{ static_cast<char*>(map(first, count)) };
05561 for (GLsizei i = 0; i < count; ++i)
05562 {
05563 // バッファオブジェクトの i 番目のブロックのポインタ
05564 const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05565 // 光源の強度の拡散反射光成分を設定する
05566 light->diffuse = diffuse;
05567 }
05568 }
05569 unmap();
05570 }
05571 //
05572 //
05573 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の強度の鏡面反射光成分を設定する
05574 //
05575 // r 光源の強度の鏡面反射光成分の赤成分
05576 // g 光源の強度の鏡面反射光成分の緑成分
05577 // b 光源の強度の鏡面反射光成分の青成分
05578 // a 光源の強度の鏡面反射光成分の不透明度
05579 // first 値を設定する光源データの最初の番号
05580 // count 値を設定する光源データの数
05581 //
05582 void gg::GgSimpleShader::LightBuffer::loadSpecular(GLfloat r, GLfloat g, GLfloat b, GLfloat a,
05583 GLint first, GLsizei count) const
05584 {
05585 // データを格納するバッファオブジェクトの先頭のポインタ
05586 const auto start{ static_cast<char*>(map(first, count)) };
05587 for (GLsizei i = 0; i < count; ++i)
05588 {
05589 // バッファオブジェクトの i 番目のブロックのポインタ
05590 const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05591 // 光源の鏡面反射光成分を設定する
05592 light->specular[0] = r;
05593 light->specular[1] = g;
05594 light->specular[2] = b;
05595 light->specular[3] = a;
05596 }
05597 }
05598 unmap();
05599 }
05600 //
05601 //
05602 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の強度の鏡面反射光成分を設定する
05603 //
05604 // ambient 光源の強度の鏡面反射光成分
05605 // first 値を設定する光源データの最初の番号
05606 // count 値を設定する光源データの数
05607 //
05608 void gg::GgSimpleShader::LightBuffer::loadSpecular(const GgVector& specular,
05609 GLint first, GLsizei count) const
05610 {
05611 // データを格納するバッファオブジェクトの先頭のポインタ
05612 const auto start{ static_cast<char*>(map(first, count)) };
05613 for (GLsizei i = 0; i < count; ++i)
05614 {
05615 // バッファオブジェクトの i 番目のブロックのポインタ
05616 const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05617 // 光源の強度の鏡面反射光成分を設定する
05618 light->specular = specular;
05619 }
05620 }
05621 unmap();
05622 }
05623 //
05624 //
05625 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の色を設定するが位置は変更しない
05626 //
05627 // material 光源の特性の GgSimpleShader::Light 構造体
05628 // first 値を設定する光源データの最初の番号
05629 // count 値を設定する光源データの数
05630 //
05631 void gg::GgSimpleShader::LightBuffer::loadColor(const Light& color,
05632 GLint first, GLsizei count) const
05633 {
05634 // データを格納するバッファオブジェクトの先頭のポインタ
05635 const auto start{ static_cast<char*>(map(first, count)) };
05636 for (GLsizei i = 0; i < count; ++i)
05637 {
05638 // バッファオブジェクトの i 番目のブロックのポインタ
05639 const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };

```

```

05640
05641 // 光源の色を設定する
05642 light->ambient = color.ambient;
05643 light->diffuse = color.diffuse;
05644 light->specular = color.specular;
05645 }
05646 unmap();
05647 }
05648
05649 //
05650 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の位置を設定する
05651 //
05652 // x 光源の位置の x 座標
05653 // y 光源の位置の y 座標
05654 // z 光源の位置の z 座標
05655 // w 光源の位置の w 座標
05656 // first 値を設定する光源データの最初の番号
05657 // count 値を設定する光源データの数
05658 //
05659 void gg::GgSimpleShader::LightBuffer::loadPosition(GLfloat x, GLfloat y, GLfloat z, GLfloat w,
05660 GLint first, GLsizei count) const
05661 {
05662 // データを格納するバッファオブジェクトの先頭のポインタ
05663 const auto start{ static_cast<char*>(map(first, count)) };
05664 for (GLsizei i = 0; i < count; ++i)
05665 {
05666 // バッファオブジェクトの i 番目のブロックのポインタ
05667 const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05668
05669 // 光源の位置を設定する
05670 light->position[0] = x;
05671 light->position[1] = y;
05672 light->position[2] = z;
05673 light->position[3] = w;
05674 }
05675 unmap();
05676 }
05677
05678 //
05679 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の位置を設定する
05680 //
05681 // position 光源の位置
05682 // first 値を設定する光源データの最初の番号
05683 // count 値を設定する光源データの数
05684 //
05685 void gg::GgSimpleShader::LightBuffer::loadPosition(const GgVector& position,
05686 GLint first, GLsizei count) const
05687 {
05688 // データを格納するバッファオブジェクトの先頭のポインタ
05689 const auto start{ static_cast<char*>(map(first, count)) };
05690 for (GLsizei i = 0; i < count; ++i)
05691 {
05692 // バッファオブジェクトの i 番目のブロックのポインタ
05693 const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05694
05695 // 光源の位置を設定する
05696 light->position = position;
05697 }
05698 unmap();
05699 }
05700
05701 //
05702 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：環境光に対する反射係数を設定する
05703 //
05704 // r 環境光に対する反射係数の赤成分
05705 // g 環境光に対する反射係数の緑成分
05706 // b 環境光に対する反射係数の青成分
05707 // a 環境光に対する反射係数の不透明度
05708 // first 値を設定する材質データの最初の番号
05709 // count 値を設定する材質データの数
05710 //
05711 void gg::GgSimpleShader::MaterialBuffer::loadAmbient(GLfloat r, GLfloat g, GLfloat b, GLfloat a,
05712 GLint first, GLsizei count) const
05713 {
05714 // データを格納するバッファオブジェクトの先頭のポインタ
05715 const auto start{ static_cast<char*>(map(first, count)) };
05716 for (GLsizei i = 0; i < count; ++i)
05717 {
05718 // バッファオブジェクトの i 番目のブロックのポインタ
05719 const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05720
05721 // 環境光に対する反射係数を設定する
05722 material->ambient[0] = r;
05723 material->ambient[1] = g;
05724 material->ambient[2] = b;
05725 material->ambient[3] = a;
05726 }

```

```

05727     unmap();
05728 }
05729
05730 //
05731 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：環境光に対する反射係数を設定する
05732 //
05733 // ambient 環境光に対する反射係数
05734 // first 値を設定する光源データの最初の番号
05735 // count 値を設定する光源データの数
05736 //
05737 void gg::GgSimpleShader::MaterialBuffer::loadAmbient(const GgVector& ambient,
05738 GLint first, GLsizei count) const
05739 {
05740     // データを格納するバッファオブジェクトの先頭のポインタ
05741     const auto start{ static_cast<char*>(map(first, count)) };
05742     for (GLsizei i = 0; i < count; ++i)
05743     {
05744         // バッファオブジェクトの i 番目のブロックのポインタ
05745         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05746
05747         // 光源の強度の環境光成分を設定する
05748         material->ambient = ambient;
05749     }
05750     unmap();
05751 }
05752
05753 //
05754 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：拡散反射係数を設定する
05755 //
05756 // r 拡散反射係数の赤成分
05757 // g 拡散反射係数の緑成分
05758 // b 拡散反射係数の青成分
05759 // a 拡散反射係数の不透明度
05760 // first 値を設定する材質データの最初の番号
05761 // count 値を設定する材質データの数
05762 //
05763 void gg::GgSimpleShader::MaterialBuffer::loadDiffuse(GLfloat r, GLfloat g, GLfloat b, GLfloat a,
05764 GLint first, GLsizei count) const
05765 {
05766     // データを格納するバッファオブジェクトの先頭のポインタ
05767     const auto start{ static_cast<char*>(map(first, count)) };
05768     for (GLsizei i = 0; i < count; ++i)
05769     {
05770         // バッファオブジェクトの i 番目のブロックのポインタ
05771         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05772
05773         // 拡散反射係数を設定する
05774         material->diffuse[0] = r;
05775         material->diffuse[1] = g;
05776         material->diffuse[2] = b;
05777         material->diffuse[3] = a;
05778     }
05779     unmap();
05780 }
05781
05782 //
05783 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：拡散反射係数を設定する
05784 //
05785 // diffuse 拡散反射係数
05786 // first 値を設定する光源データの最初の番号
05787 // count 値を設定する光源データの数
05788 //
05789 void gg::GgSimpleShader::MaterialBuffer::loadDiffuse(const GgVector& diffuse,
05790 GLint first, GLsizei count) const
05791 {
05792     // データを格納するバッファオブジェクトの先頭のポインタ
05793     const auto start{ static_cast<char*>(map(first, count)) };
05794     for (GLsizei i = 0; i < count; ++i)
05795     {
05796         // バッファオブジェクトの i 番目のブロックのポインタ
05797         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05798
05799         // 光源の強度の環境光成分を設定する
05800         material->diffuse = diffuse;
05801     }
05802     unmap();
05803 }
05804
05805 //
05806 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：環境光に対する反射係数と拡散反射係数を設定する
05807 //
05808 // r 環境光に対する反射係数と拡散反射係数の赤成分
05809 // g 環境光に対する反射係数と拡散反射係数の緑成分
05810 // b 環境光に対する反射係数と拡散反射係数の青成分
05811 // a 環境光に対する反射係数と拡散反射係数の不透明度
05812 // first 値を設定する材質データの最初の番号
05813 // count 値を設定する材質データの数

```

```

05814 //
05815 void gg::GgSimpleShader::MaterialBuffer::loadAmbientAndDiffuse(GLfloat r, GLfloat g, GLfloat b,
    GLfloat a,
05816 GLint first, GLsizei count) const
05817 {
05818     // データを格納するバッファオブジェクトの先頭のポインタ
05819     const auto start{ static_cast<char*>(map(first, count)) };
05820     for (GLsizei i = 0; i < count; ++i)
05821     {
05822         // バッファオブジェクトの i 番目のブロックのポインタ
05823         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05824
05825         // 環境光に対する反射係数と拡散反射係数を設定する
05826         material->ambient[0] = material->diffuse[0] = r;
05827         material->ambient[1] = material->diffuse[1] = g;
05828         material->ambient[2] = material->diffuse[2] = b;
05829         material->ambient[3] = material->diffuse[3] = a;
05830     }
05831     unmap();
05832 }
05833
05834 //
05835 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：環境光に対する反射係数と拡散反射係数を設定する
05836 //
05837 // color 環境光に対する反射係数と拡散反射係数
05838 // first 値を設定する光源データの最初の番号
05839 // count 値を設定する光源データの数
05840 //
05841 void gg::GgSimpleShader::MaterialBuffer::loadAmbientAndDiffuse(const GgVector& color,
05842 GLint first, GLsizei count) const
05843 {
05844     // データを格納するバッファオブジェクトの先頭のポインタ
05845     const auto start{ static_cast<char*>(map(first, count)) };
05846     for (GLsizei i = 0; i < count; ++i)
05847     {
05848         // バッファオブジェクトの i 番目のブロックのポインタ
05849         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05850
05851         // 光源の強度の環境光成分を設定する
05852         material->ambient = material->diffuse = color;
05853     }
05854     unmap();
05855 }
05856
05857 //
05858 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：環境光に対する反射係数と拡散反射係数を設定する
05859 //
05860 // color 環境光に対する反射係数と拡散反射係数を格納した GLfloat 型の 4 要素の配列
05861 // first 値を設定する材質データの最初の番号
05862 // count 値を設定する材質データの数
05863 //
05864 void gg::GgSimpleShader::MaterialBuffer::loadAmbientAndDiffuse(const GLfloat* color,
05865 GLint first, GLsizei count) const
05866 {
05867     // ambient 要素のバイトオフセット
05868     constexpr auto ambientOffset{ offsetof(Material, ambient) };
05869
05870     // ambient 要素のバイト数
05871     constexpr auto ambientSize{ sizeof(Material::diffuse) };
05872
05873     // diffuse 要素のバイトオフセット
05874     constexpr auto diffuseOffset{ offsetof(Material, diffuse) };
05875
05876     // diffuse 要素のバイト数
05877     constexpr auto diffuseSize{ sizeof(Material::diffuse) };
05878
05879     // 元のデータの先頭
05880     const auto* source{ reinterpret_cast<const char*>(color) };
05881
05882     // first 番目のブロックから count 個の ambient 要素と diffuse 要素に値を設定する
05883     bind();
05884     for (GLsizei i = 0; i < count; ++i)
05885     {
05886         // 格納先
05887         const auto destination{ getStride() * (first + i) };
05888
05889         // first + i 番目のブロックの ambient 要素に値を設定する
05890         glBufferSubData(getTarget(), destination + ambientOffset, ambientSize, source + i * ambientSize);
05891
05892         // first + i 番目のブロックの diffuse 要素に値を設定する
05893         glBufferSubData(getTarget(), destination + diffuseOffset, diffuseSize, source + i * diffuseSize);
05894     }
05895 }
05896
05897 //
05898 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：鏡面反射係数を設定する
05899 //

```

```

05900 //   r 鏡面反射係数の赤成分
05901 //   g 鏡面反射係数の緑成分
05902 //   b 鏡面反射係数の青成分
05903 //   a 鏡面反射係数の不透明度
05904 //   first 値を設定する材質データの最初の番号
05905 //   count 値を設定する材質データの数
05906 //
05907 void gg::GgSimpleShader::MaterialBuffer::loadSpecular(GLfloat r, GLfloat g, GLfloat b, GLfloat a,
05908   GLint first, GLsizei count) const
05909 {
05910   // データを格納するバッファオブジェクトの先頭のポインタ
05911   const auto start{ static_cast<char*>(map(first, count)) };
05912   for (GLsizei i = 0; i < count; ++i)
05913   {
05914     // バッファオブジェクトの i 番目のブロックのポインタ
05915     const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05916
05917     // 鏡面反射係数を設定する
05918     material->specular[0] = r;
05919     material->specular[1] = g;
05920     material->specular[2] = b;
05921     material->specular[3] = a;
05922   }
05923   unmap();
05924 }
05925 //
05926 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：鏡面反射係数を設定する
05927 //
05928 //   specular 鏡面反射係数
05929 //   first 値を設定する光源データの最初の番号
05930 //   count 値を設定する光源データの数
05931 //
05932 //
05933 void gg::GgSimpleShader::MaterialBuffer::loadSpecular(const GgVector& specular,
05934   GLint first, GLsizei count) const
05935 {
05936   // データを格納するバッファオブジェクトの先頭のポインタ
05937   const auto start{ static_cast<char*>(map(first, count)) };
05938   for (GLsizei i = 0; i < count; ++i)
05939   {
05940     // バッファオブジェクトの i 番目のブロックのポインタ
05941     const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05942
05943     // 光源の強度の環境光成分を設定する
05944     material->specular = specular;
05945   }
05946   unmap();
05947 }
05948 //
05949 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：輝き係数を設定する
05950 //
05951 //   shininess 輝き係数
05952 //   first 値を設定する材質データの最初の番号
05953 //   count 値を設定する材質データの数
05954 //
05955 //
05956 void gg::GgSimpleShader::MaterialBuffer::loadShininess(GLfloat shininess,
05957   GLint first, GLsizei count) const
05958 {
05959   // データを格納するバッファオブジェクトの先頭のポインタ
05960   const auto start{ static_cast<char*>(map(first, count)) };
05961   for (GLsizei i = 0; i < count; ++i)
05962   {
05963     // バッファオブジェクトの i 番目のブロックのポインタ
05964     const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05965     material->shininess = shininess;
05966   }
05967   unmap();
05968 }
05969 //
05970 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：輝き係数を設定する
05971 //
05972 //   shininess 輝き係数
05973 //   first 値を設定する材質データの最初の番号
05974 //   count 値を設定する材質データの数
05975 //
05976 //
05977 void gg::GgSimpleShader::MaterialBuffer::loadShininess(const GLfloat* shininess,
05978   GLint first, GLsizei count) const
05979 {
05980   // データを格納するバッファオブジェクトの先頭のポインタ
05981   const auto start{ static_cast<char*>(map(first, count)) };
05982   for (GLsizei i = 0; i < count; ++i)
05983   {
05984     // バッファオブジェクトの i 番目のブロックのポインタ
05985     const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05986     material->shininess = shininess[i];

```

```

05987     }
05988     unmap();
05989 }
05990
05991 //
05992 // 三角形に単純な陰影付けを行うシェーダ：シェーダのソースファイルの読み込み
05993 //
05994 bool gg::GgSimpleShader::load(const std::string& vert, const std::string& frag, const std::string&
    geom,
05995     GLint nvarying, const char* const* varyings)
05996 {
05997     if (!GgPointShader::load(vert, frag, geom, nvarying, varyings)) return false;
05998
05999     mnLoc = glGetUniformLocation(get(), "mn");
06000     const auto lightIndex{ glGetUniformLocation(get(), "Light") };
06001     glUniformBlockBinding(get(), lightIndex, LightBindingPoint);
06002     const auto materialIndex{ glGetUniformLocation(get(), "Material") };
06003     glUniformBlockBinding(get(), materialIndex, MaterialBindingPoint);
06004
06005     return true;
06006 }
06007
06008 //
06009 // Wavefront OBJ 形式のデータ：コンストラクタ
06010 //
06011 gg::GgSimpleObj::GgSimpleObj(const std::string& name, bool normalize)
06012 {
06013     // 作業用のメモリ
06014     std::vector<GgSimpleShader::Material> mat;
06015     std::vector<GgVertex> vert;
06016     std::vector<GLuint> face;
06017
06018     // グループのデータのメモリを確保する
06019     group = std::make_shared<std::vector<std::array<GLuint, 3>>>();
06020
06021     // ファイルを読み込む
06022     if (ggLoadSimpleObj(name, *group, mat, vert, face, normalize))
06023     {
06024         // 頂点バッファオブジェクトを作成する
06025         data = std::make_shared<GgElements>(vert.data(), static_cast<GLsizei>(vert.size()),
06026             face.data(), static_cast<GLsizei>(face.size()), GL_TRIANGLES);
06027
06028         // 描画するオブジェクトを切り替えるために頂点配列オブジェクトを閉じておく
06029         glBindVertexArray(0);
06030
06031         // 材質データを設定する
06032         material = std::make_shared<GgSimpleShader::MaterialBuffer>(mat.data(),
06033             static_cast<GLsizei>(mat.size()));
06034     }
06035 }
06036
06037 // Wavefront OBJ 形式のデータ：図形の描画
06038 //
06039 void gg::GgSimpleObj::draw(GLint first, GLsizei count) const
06040 {
06041     // 保持しているグループの数
06042     const auto ng{ static_cast<GLsizei>(group->size()) };
06043
06044     // 描画する最後のグループの次
06045     auto last{ count <= 0 ? ng : first + count };
06046     if (last > ng) last = ng;
06047
06048     for (GLsizei i = first; i < last; ++i)
06049     {
06050         // グループのデータ
06051         const auto& g{ (*group)[i] };
06052
06053         // 材質を設定する
06054         material->select(g[2]);
06055
06056         // 図形を描画する
06057         data->draw(g[0], g[1]);
06058     }
06059 }

```

10.7 gg.h ファイル

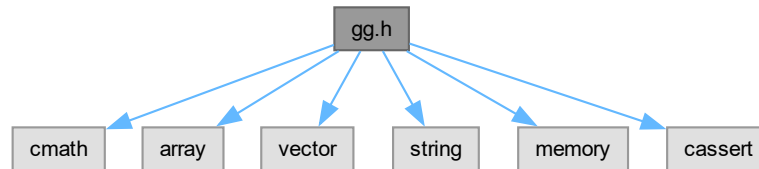
```

#include <cmath>
#include <array>
#include <vector>
#include <string>

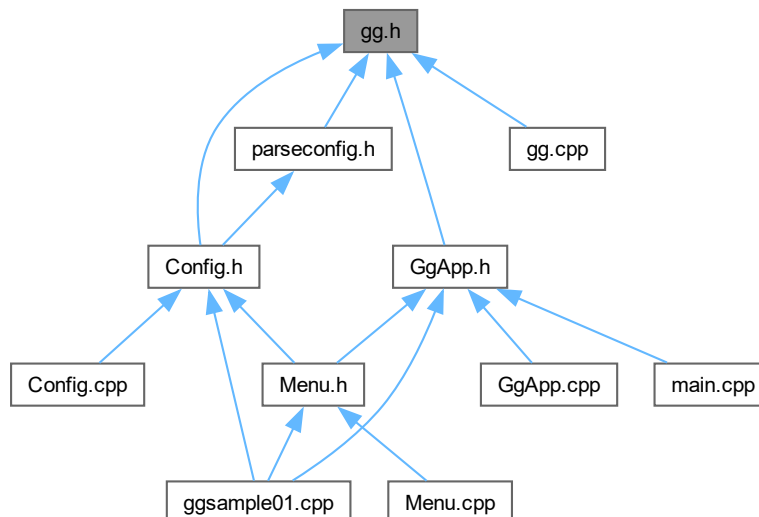
```

```
#include <memory>
#include <cassert>
```

gg.h の依存先関係図:



被依存関係図:



クラス

- class `gg::GgVector`
- class `gg::GgMatrix`
- class `gg::GgQuaternion`
- class `gg::GgTrackball`
- class `gg::GgTexture`
- class `gg::GgColorTexture`
- class `gg::GgNormalTexture`
- class `gg::GgBuffer< T >`
- class `gg::GgUniformBuffer< T >`
- class `gg::GgVertexArray`
- class `gg::GgShape`
- class `gg::GgPoints`
- struct `gg::GgVertex`
- class `gg::GgTriangles`

- class `gg::GgElements`
- class `gg::GgShader`
- class `gg::GgPointShader`
- class `gg::GgSimpleShader`
- struct `gg::GgSimpleShader::Light`
- class `gg::GgSimpleShader::LightBuffer`
- struct `gg::GgSimpleShader::Material`
- class `gg::GgSimpleShader::MaterialBuffer`
- class `gg::GgSimpleObj`

名前空間

- namespace `gg`

マクロ定義

- `#define ggError()`
- `#define ggFBOError()`

型定義

- using `pathString` = `std::string`
- using `pathChar` = `char`

列挙型

- enum `gg::BindingPoints` { `gg::LightBindingPoint` = 0 , `gg::MaterialBindingPoint` }

関数

- `pathString Utf8ToTChar` (const `std::string` &string)
- `std::string TCharToUtf8` (const `pathString` &cstring)
- void `gg::ggInit` ()
- void `gg::ggError` (const `std::string` &name="", unsigned int line=0)
- void `gg::ggFBOError` (const `std::string` &name="", unsigned int line=0)
- void `gg::ggCross` (GLfloat *c, const GLfloat *a, const GLfloat *b)
- GLfloat `gg::ggDot3` (const GLfloat *a, const GLfloat *b)
- GLfloat `gg::ggLength3` (const GLfloat *a)
- GLfloat `gg::ggDistance3` (const GLfloat *a, const GLfloat *b)
- void `gg::ggNormalize3` (const GLfloat *a, GLfloat *b)
- void `gg::ggNormalize3` (GLfloat *a)
- GLfloat `gg::ggDot4` (const GLfloat *a, const GLfloat *b)
- GLfloat `gg::ggLength4` (const GLfloat *a)
- GLfloat `gg::ggDistance4` (const GLfloat *a, const GLfloat *b)
- void `gg::ggNormalize4` (const GLfloat *a, GLfloat *b)
- void `gg::ggNormalize4` (GLfloat *a)
- const `GgVector` & `gg::operator+` (const `GgVector` &v)
- `GgVector` `gg::operator+` (GLfloat a, const `GgVector` &b)
- const `GgVector` `gg::operator-` (const `GgVector` &v)
- `GgVector` `gg::operator-` (GLfloat a, const `GgVector` &b)
- `GgVector` `gg::operator*` (GLfloat a, const `GgVector` &b)
- `GgVector` `gg::operator/` (GLfloat a, const `GgVector` &b)
- `GgVector` `gg::ggCross` (const `GgVector` &a, const `GgVector` &b)
- GLfloat `gg::ggDot3` (const `GgVector` &a, const `GgVector` &b)
- GLfloat `gg::ggLength3` (const `GgVector` &a)
- GLfloat `gg::ggDistance3` (const `GgVector` &a, const `GgVector` &b)

- `GgVector gg::ggNormalize3` (const `GgVector` &a)
- `void gg::ggNormalize3` (`GgVector` *a)
- `GLfloat gg::ggDot4` (const `GgVector` &a, const `GgVector` &b)
- `GLfloat gg::ggLength4` (const `GgVector` &a)
- `GLfloat gg::ggDistance4` (const `GgVector` &a, const `GgVector` &b)
- `GgVector gg::ggNormalize4` (const `GgVector` &a)
- `void gg::ggNormalize4` (`GgVector` *a)
- `GgMatrix gg::ggIdentity` ()
- `GgMatrix gg::ggTranslate` (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f)
- `GgMatrix gg::ggTranslate` (const GLfloat *t)
- `GgMatrix gg::ggTranslate` (const `GgVector` &t)
- `GgMatrix gg::ggScale` (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f)
- `GgMatrix gg::ggScale` (const GLfloat *s)
- `GgMatrix gg::ggScale` (const `GgVector` &s)
- `GgMatrix gg::ggRotateX` (GLfloat a)
- `GgMatrix gg::ggRotateY` (GLfloat a)
- `GgMatrix gg::ggRotateZ` (GLfloat a)
- `GgMatrix gg::ggRotate` (GLfloat x, GLfloat y, GLfloat z, GLfloat a)
- `GgMatrix gg::ggRotate` (const GLfloat *r, GLfloat a)
- `GgMatrix gg::ggRotate` (const `GgVector` &r, GLfloat a)
- `GgMatrix gg::ggRotate` (const GLfloat *r)
- `GgMatrix gg::ggRotate` (const `GgVector` &r)
- `GgMatrix gg::ggLookat` (GLfloat ex, GLfloat ey, GLfloat ez, GLfloat tx, GLfloat ty, GLfloat tz, GLfloat ux, GLfloat uy, GLfloat uz)
- `GgMatrix gg::ggLookat` (const GLfloat *e, const GLfloat *t, const GLfloat *u)
- `GgMatrix gg::ggLookat` (const `GgVector` &e, const `GgVector` &t, const `GgVector` &u)
- `GgMatrix gg::ggOrthogonal` (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar)
- `GgMatrix gg::ggFrustum` (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar)
- `GgMatrix gg::ggPerspective` (GLfloat fovy, GLfloat aspect, GLfloat zNear, GLfloat zFar)
- `GgMatrix gg::ggTranspose` (const `GgMatrix` &m)
- `GgMatrix gg::ggInvert` (const `GgMatrix` &m)
- `GgMatrix gg::ggNormal` (const `GgMatrix` &m)
- `GgQuaternion gg::ggIdentityQuaternion` ()
- `GgQuaternion gg::ggMatrixQuaternion` (const GLfloat *a)
- `GgQuaternion gg::ggMatrixQuaternion` (const `GgMatrix` &m)
- `GgMatrix gg::ggQuaternionMatrix` (const `GgQuaternion` &q)
- `GgMatrix gg::ggQuaternionTransposeMatrix` (const `GgQuaternion` &q)
- `GgQuaternion gg::ggRotateQuaternion` (GLfloat x, GLfloat y, GLfloat z, GLfloat a)
- `GgQuaternion gg::ggRotateQuaternion` (const GLfloat *v, GLfloat a)
- `GgQuaternion gg::ggRotateQuaternion` (const GLfloat *v)
- `GgQuaternion gg::ggEulerQuaternion` (GLfloat heading, GLfloat pitch, GLfloat roll)
- `GgQuaternion gg::ggEulerQuaternion` (const GLfloat *e)
- `GgQuaternion gg::ggEulerQuaternion` (const `GgVector` &e)
- `GgQuaternion gg::ggSlerp` (const GLfloat *a, const GLfloat *b, GLfloat t)
- `GgQuaternion gg::ggSlerp` (const `GgQuaternion` &q, const `GgQuaternion` &r, GLfloat t)
- `GgQuaternion gg::ggSlerp` (const `GgQuaternion` &q, const GLfloat *a, GLfloat t)
- `GgQuaternion gg::ggSlerp` (const GLfloat *a, const `GgQuaternion` &q, GLfloat t)
- `GLfloat gg::ggNorm` (const `GgQuaternion` &q)
- `GgQuaternion gg::ggNormalize` (const `GgQuaternion` &q)
- `GgQuaternion gg::ggConjugate` (const `GgQuaternion` &q)
- `GgQuaternion gg::ggInvert` (const `GgQuaternion` &q)
- `bool gg::ggSaveTga` (const std::string &name, const void *buffer, unsigned int width, unsigned int height, unsigned int depth)
- `bool gg::ggSaveColor` (const std::string &name)

- bool [gg::ggSaveDepth](#) (const std::string &name)
- bool [gg::ggReadImage](#) (const std::string &name, std::vector< GLubyte > &image, GLsizei *pWidth, GLsizei *pHeight, GLenum *pFormat)
- GLuint [gg::ggLoadTexture](#) (const GLvoid *image, GLsizei width, GLsizei height, GLenum format=GL_↵RGB, GLenum type=GL_UNSIGNED_BYTE, GLenum internal=GL_RGB, GLenum wrap=GL_CLAMP_TO_↵EDGE, bool swizzle=false)
- GLuint [gg::ggLoadImage](#) (const std::string &name, GLsizei *pWidth=nullptr, GLsizei *pHeight=nullptr, GLenum internal=GL_RGBA, GLenum wrap=GL_CLAMP_TO_EDGE)
- void [gg::ggCreateNormalMap](#) (const GLubyte *hmap, GLsizei width, GLsizei height, GLenum format, GLfloat nz, GLenum internal, std::vector< [GgVector](#) > &nmap)
- GLuint [gg::ggLoadHeight](#) (const std::string &name, GLfloat nz, GLsizei *pWidth=nullptr, GLsizei *p↵Height=nullptr, GLenum internal=GL_RGBA)
- GLuint [gg::ggCreateShader](#) (const std::string &vsr, const std::string &fsr="", const std::string &gsr="", GLint nvarying=0, const char *const *varyings=nullptr, const std::string &vtext="vertex shader", const std↵::string &ftext="fragment shader", const std::string >ext="geometry shader")
- GLuint [gg::ggLoadShader](#) (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- GLuint [gg::ggLoadShader](#) (const std::array< std::string, 3 > &files, GLint nvarying=0, const char *const *varyings=nullptr)
- GLuint [gg::ggCreateComputeShader](#) (const std::string &csrc, const std::string &ctext="compute shader")
- GLuint [gg::ggLoadComputeShader](#) (const std::string &comp)
- std::shared_ptr< [GgPoints](#) > [gg::ggPointsCube](#) (GLsizei countv, GLfloat length=1.0f, GLfloat cx=0.0f, GLfloat cy=0.0f, GLfloat cz=0.0f)
- std::shared_ptr< [GgPoints](#) > [gg::ggPointsSphere](#) (GLsizei countv, GLfloat radius=0.5f, GLfloat cx=0.0f, GLfloat cy=0.0f, GLfloat cz=0.0f)
- std::shared_ptr< [GgTriangles](#) > [gg::ggRectangle](#) (GLfloat width=1.0f, GLfloat height=1.0f)
- std::shared_ptr< [GgTriangles](#) > [gg::ggEllipse](#) (GLfloat width=1.0f, GLfloat height=1.0f, GLuint slices=16)
- std::shared_ptr< [GgTriangles](#) > [gg::ggArraysObj](#) (const std::string &name, bool normalize=false)
- std::shared_ptr< [GgElements](#) > [gg::ggElementsObj](#) (const std::string &name, bool normalize=false)
- std::shared_ptr< [GgElements](#) > [gg::ggElementsMesh](#) (GLuint slices, GLuint stacks, const GLfloat(*pos)[3], const GLfloat(*norm)[3]=nullptr)
- std::shared_ptr< [GgElements](#) > [gg::ggElementsSphere](#) (GLfloat radius=1.0f, int slices=16, int stacks=8)
- bool [gg::ggLoadSimpleObj](#) (const std::string &name, std::vector< std::array< GLuint, 3 > > &group, std↵::vector< [GgSimpleShader::Material](#) > &material, std::vector< [GgVertex](#) > &vert, bool normalize=false)
- bool [gg::ggLoadSimpleObj](#) (const std::string &name, std::vector< std::array< GLuint, 3 > > &group, std↵::vector< [GgSimpleShader::Material](#) > &material, std::vector< [GgVertex](#) > &vert, std::vector< GLuint > &face, bool normalize=false)

変数

- GLint [gg::ggBufferAlignment](#)
使用している GPU のバッファアライメント。

10.7.1 詳解

ゲームグラフィックス特論の宿題用補助プログラム GLFW3 版の宣言。

著者

Kohe Tokoi

日付

July 17, 2025

[gg.h](#) に定義があります。

10.7.2 マクロ定義詳解

10.7.2.1 ggError

```
#define ggError()
```

OpenGL のエラーの発生を検知したときにソースファイル名と行番号を表示する。

覚え書き

このマクロを置いた場所（より前）で OpenGL のエラーが発生していた時に、このマクロを置いたソースファイル名と行番号を出力する。リリースビルド時には無視される。

gg.h の 1398 行目に定義があります。

10.7.2.2 ggFBOError

```
#define ggFBOError()
```

FBO のエラーの発生を検知したときにソースファイル名と行番号を表示する。

覚え書き

このマクロを置いた場所（より前）で FBO のエラーが発生していた時に、このマクロを置いたソースファイル名と行番号を出力する。リリースビルド時には無視される。

gg.h の 1425 行目に定義があります。

10.7.3 型定義詳解

10.7.3.1 pathChar

```
using pathChar = char
```

gg.h の 78 行目に定義があります。

10.7.3.2 pathString

```
using pathString = std::string
```

gg.h の 77 行目に定義があります。

10.7.4 関数詳解

10.7.4.1 TCharToUtf8()

```
std::string TCharToUtf8 (  
    const pathString & cstring) [inline]
```

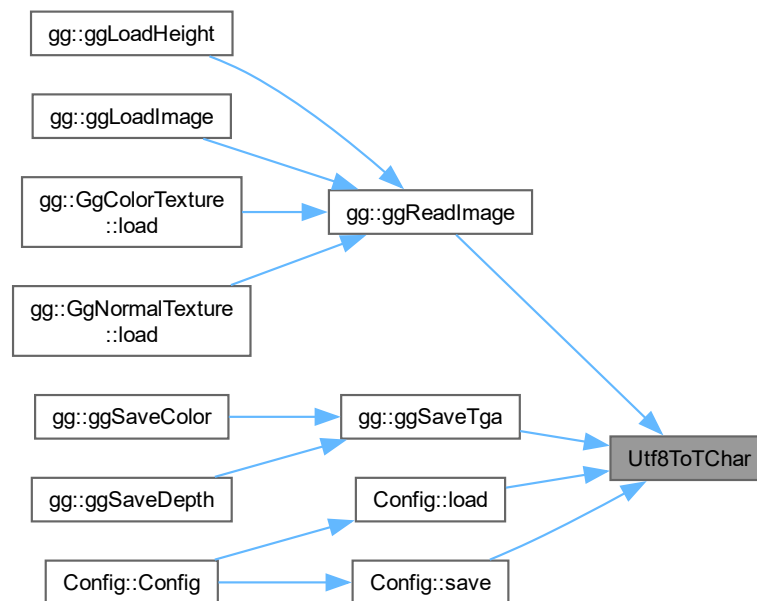
gg.h の 80 行目に定義があります。

10.7.4.2 Utf8ToTChar()

```
pathString Utf8ToTChar (  
    const std::string & string) [inline]
```

gg.h の 79 行目に定義があります。

被呼び出し関係図:



10.8 gg.h

[詳解]

```

00001 #pragma once
00002
00026
00034
00035 // 標準ライブラリ
00036 #include <cmath>
00037 #include <array>
00038 #include <vector>
00039 #include <string>
00040 #include <memory>
00041 #include <cassert>
00042
00043 // Windows (Visual Studio) のとき
00044 #if defined(_MSC_VER)
00045 // 非推奨の警告を出さない
00046 # pragma warning(disable:4996)
00047 // 数学ライブラリの定数を使う
00048 # define _USE_MATH_DEFINES
00049 // MIN() / MAX マクロは使わない
00050 # define NOMINMAX
00051 // ファイルパスの文字コード
00052 #include <atlstr.h>
00053 using pathString = CString;
00054 using pathChar = wchar_t;
00055 extern pathString Utf8ToTChar(const std::string& string);
00056 extern std::string TCharToUtf8(const pathString& cstring);
00057 // デバッグビルドかどうか調べる
00058 # if defined(DEBUG)
00059 #   if !defined(DEBUG)
00060 #     define DEBUG
00061 #   endif
00062 #   define GLFW3_CONFIGURATION "Debug"
00063 # else
00064 #   if !defined(NDEBUG)
00065 #     define NDEBUG
00066 #   endif
00067 #   define GLFW3_CONFIGURATION "Release"
00068 # endif
00069 // プラットフォームを調べる
00070 # if defined(_WIN64)

```

```

00071 #     define GLFW3_PLATFORM "x64"
00072 #     else
00073 #     define GLFW3_PLATFORM "Win32"
00074 #     endif
00075 #else
00076 // ファイルパスの文字コード
00077 using pathString = std::string;
00078 using pathChar = char;
00079 inline pathString Utf8ToTChar(const std::string& string) { return string; }
00080 inline std::string TCharToUtf8(const pathString& cstring) { return cstring; }
00081 #endif
00082
00083
00084
00085 // macOS で "OpenGL deprecated" の警告を出さない
00086 #if defined(__APPLE__)
00087 #     define GL_SILENCE_DEPRECATED
00088 #endif
00089
00090 // フレームワークに GLFW 3 を使う
00091 #if defined(IMGUI_IMPL_OPENGL_ES2)
00092 #     define GLFW_INCLUDE_ES2
00093 #elif defined(IMGUI_IMPL_OPENGL_ES3)
00094 #     define GLFW_INCLUDE_ES3
00095 #else
00096 #     define GLFW_INCLUDE_GLCOREARB
00097 #endif
00098 #include <GLFW/glfw3.h>
00099 #if !defined(__gl_h__)
00100 #     define __gl_h__
00101 #endif
00102 #if !defined(__GL_H__)
00103 #     define __GL_H__
00104 #endif
00105
00106 // OpenGL 3.2 の API のエントリポイント
00107 #if !defined(GL3_PROTOTYPES) && !defined(GL_GLES_PROTOTYPES)
00108 extern PFNGLACTIVEPROGRAMEXTPROC glActiveProgramEXT;
00109 extern PFNGLACTIVESHADERPROGRAMPROC glActiveShaderProgram;
00110 extern PFNGLACTIVETEXTUREPROC glActiveTexture;
00111 extern PFNGLAPPLYFRAMEBUFFERATTACHMENTCMAAINTELPROC glApplyFramebufferAttachmentCMAAINTEL;
00112 extern PFNGLATTACHSHADERPROC glAttachShader;
00113 extern PFNGLBEGINCONDITIONALRENDERNVPROC glBeginConditionalRenderNV;
00114 extern PFNGLBEGINCONDITIONALRENDERPROC glBeginConditionalRender;
00115 extern PFNGLBEGINPERFMONITORAMDPROC glBeginPerfMonitorAMD;
00116 extern PFNGLBEGINPERFQUERYINTELPROC glBeginPerfQueryINTEL;
00117 extern PFNGLBEGINQUERYINDEXEDPROC glBeginQueryIndexed;
00118 extern PFNGLBEGINQUERYPROC glBeginQuery;
00119 extern PFNGLBEGINTRANSFORMFEEDBACKPROC glBeginTransformFeedback;
00120 extern PFNGLBINDATTRIBLOCATIONPROC glBindAttribLocation;
00121 extern PFNGLBINDBUFFERBASEPROC glBindBufferBase;
00122 extern PFNGLBINDBUFFERPROC glBindBuffer;
00123 extern PFNGLBINDBUFFERRANGEPROC glBindBufferRange;
00124 extern PFNGLBINDBUFFERSBASEPROC glBindBuffersBase;
00125 extern PFNGLBINDBUFFERSRANGEPROC glBindBuffersRange;
00126 extern PFNGLBINDFRAGDATALOCATIONINDEXEDPROC glBindFragDataLocationIndexed;
00127 extern PFNGLBINDFRAGDATALOCATIONPROC glBindFragDataLocation;
00128 extern PFNGLBINDFRAMEBUFFERPROC glBindFramebuffer;
00129 extern PFNGLBINDIMAGETEXTUREPROC glBindImageTexture;
00130 extern PFNGLBINDIMAGETEXTURESPROC glBindImageTextures;
00131 extern PFNGLBINDMULTITEXTUREEXTPROC glBindMultiTextureEXT;
00132 extern PFNGLBINDPROGRAMPIPELINEPROC glBindProgramPipeline;
00133 extern PFNGLBINDRENDERBUFFERPROC glBindRenderbuffer;
00134 extern PFNGLBINDSAMPLERPROC glBindSampler;
00135 extern PFNGLBINDSAMPLERSPROC glBindSamplers;
00136 extern PFNGLBINDTEXTUREPROC glBindTexture;
00137 extern PFNGLBINDTEXTURESPROC glBindTextures;
00138 extern PFNGLBINDTEXTUREUNITPROC glBindTextureUnit;
00139 extern PFNGLBINDTRANSFORMFEEDBACKPROC glBindTransformFeedback;
00140 extern PFNGLBINDVERTEXARRAYPROC glBindVertexArray;
00141 extern PFNGLBINDVERTEXBUFFERPROC glBindVertexBuffer;
00142 extern PFNGLBINDVERTEXBUFFERSPROC glBindVertexBuffers;
00143 extern PFNGLBLENDBARRIERKHRPROC glBlendBarrierKHR;
00144 extern PFNGLBLENDBARRIERNVPROC glBlendBarrierNV;
00145 extern PFNGLBLENDCOLORPROC glBlendColor;
00146 extern PFNGLBLEND EQUATIONIARBPROC glBlendEquationiARB;
00147 extern PFNGLBLEND EQUATIONIPROC glBlendEquationi;
00148 extern PFNGLBLEND EQUATIONPROC glBlendEquation;
00149 extern PFNGLBLEND EQUATIONSEPARATEIARBPROC glBlendEquationSeparateiARB;
00150 extern PFNGLBLEND EQUATIONSEPARATEIPROC glBlendEquationSeparatei;
00151 extern PFNGLBLEND EQUATIONSEPARATEPROC glBlendEquationSeparate;
00152 extern PFNGLBLENDFUNCTIONIARBPROC glBlendFunciARB;
00153 extern PFNGLBLENDFUNCTIONPROC glBlendFunci;
00154 extern PFNGLBLENDFUNCTIONPROC glBlendFunc;
00155 extern PFNGLBLENDFUNCTIONSEPARATEIARBPROC glBlendFuncSeparateiARB;
00156 extern PFNGLBLENDFUNCTIONSEPARATEIPROC glBlendFuncSeparatei;
00157 extern PFNGLBLENDFUNCTIONSEPARATEPROC glBlendFuncSeparate;
00158 extern PFNGLBLENDPARAMETERINVPROC glBlendParameteriNV;

```

```
00159 extern PFNGLBLITFRAMEBUFFERPROC glBlitFramebuffer;
00160 extern PFNGLBLITNAMEDFRAMEBUFFERPROC glBlitNamedFramebuffer;
00161 extern PFNGLBUFFERADDRESSRANGENVPROC glBufferAddressRangeNV;
00162 extern PFNGLBUFFERDATAPROC glBufferData;
00163 extern PFNGLBUFFERPAGECOMMITMENTARBPROC glBufferPageCommitmentARB;
00164 extern PFNGLBUFFERSTORAGEPROC glBufferStorage;
00165 extern PFNGLBUFFERSUBDATAPROC glBufferSubData;
00166 extern PFNGLCALLCOMMANDLISTNVPROC glCallCommandListNV;
00167 extern PFNGLCHECKFRAMEBUFFERSTATUSPROC glCheckFramebufferStatus;
00168 extern PFNGLCHECKNAMEDFRAMEBUFFERSTATUSEXTPROC glCheckNamedFramebufferStatusEXT;
00169 extern PFNGLCHECKNAMEDFRAMEBUFFERSTATUSPROC glCheckNamedFramebufferStatus;
00170 extern PFNGLCLAMPColorPROC glClampColor;
00171 extern PFNGLCLEARBUFFERDATAPROC glClearBufferData;
00172 extern PFNGLCLEARBUFFERFIPROC glClearBufferfi;
00173 extern PFNGLCLEARBUFFERFVPROC glClearBufferfv;
00174 extern PFNGLCLEARBUFFERIVPROC glClearBufferiv;
00175 extern PFNGLCLEARBUFFERSUBDATAPROC glClearBufferSubData;
00176 extern PFNGLCLEARBUFFERUIVPROC glClearBufferuiv;
00177 extern PFNGLCLEARColorPROC glClearColor;
00178 extern PFNGLCLEARDEPTHfPROC glClearDepthf;
00179 extern PFNGLCLEARDEPTHdPROC glClearDepthd;
00180 extern PFNGLCLEARNAMEDBUFFERDATAEXTPROC glClearNamedBufferDataEXT;
00181 extern PFNGLCLEARNAMEDBUFFERDATAPROC glClearNamedBufferData;
00182 extern PFNGLCLEARNAMEDBUFFERSUBDATAEXTPROC glClearNamedBufferSubDataEXT;
00183 extern PFNGLCLEARNAMEDBUFFERSUBDATAPROC glClearNamedBufferSubData;
00184 extern PFNGLCLEARNAMEDFRAMEBUFFERFIPROC glClearNamedFramebufferfi;
00185 extern PFNGLCLEARNAMEDFRAMEBUFFERFVPROC glClearNamedFramebufferfv;
00186 extern PFNGLCLEARNAMEDFRAMEBUFFERIVPROC glClearNamedFramebufferiv;
00187 extern PFNGLCLEARNAMEDFRAMEBUFFERUIVPROC glClearNamedFramebufferuiv;
00188 extern PFNGLCLEARPROC glClear;
00189 extern PFNGLCLEARSTENCILPROC glClearStencil;
00190 extern PFNGLCLEARTEXIMAGEPROC glClearTexImage;
00191 extern PFNGLCLEARTEXSUBIMAGEPROC glClearTexSubImage;
00192 extern PFNGLCLIENTATTRIBDEFAULTPROC glClientAttribDefaultEXT;
00193 extern PFNGLCLIENTWAITSYNCPROC glClientWaitSync;
00194 extern PFNGLCLIPCONTROLPROC glClipControl;
00195 extern PFNGLCOLORFORMATNVPROC glColorFormatNV;
00196 extern PFNGLCOLORMASKIPROC glColorMaski;
00197 extern PFNGLCOLORMASKPROC glColorMask;
00198 extern PFNGLCOMMANDLISTSEGMENTSNVPROC glCommandListSegmentsNV;
00199 extern PFNGLCOMPILECOMMANDLISTNVPROC glCompileCommandListNV;
00200 extern PFNGLCOMPILESHADERINCLUDEARBPROC glCompileShaderIncludeARB;
00201 extern PFNGLCOMPILESHADERPROC glCompileShader;
00202 extern PFNGLCOMPRESSEDMULTITEXIMAGE1DExTPROC glCompressedMultiTexImage1DExT;
00203 extern PFNGLCOMPRESSEDMULTITEXIMAGE2DExTPROC glCompressedMultiTexImage2DExT;
00204 extern PFNGLCOMPRESSEDMULTITEXIMAGE3DExTPROC glCompressedMultiTexImage3DExT;
00205 extern PFNGLCOMPRESSEDMULTITEXSUBIMAGE1DExTPROC glCompressedMultiTexSubImage1DExT;
00206 extern PFNGLCOMPRESSEDMULTITEXSUBIMAGE2DExTPROC glCompressedMultiTexSubImage2DExT;
00207 extern PFNGLCOMPRESSEDMULTITEXSUBIMAGE3DExTPROC glCompressedMultiTexSubImage3DExT;
00208 extern PFNGLCOMPRESSEDTEXIMAGE1DPROC glCompressedTexImage1D;
00209 extern PFNGLCOMPRESSEDTEXIMAGE2DPROC glCompressedTexImage2D;
00210 extern PFNGLCOMPRESSEDTEXIMAGE3DPROC glCompressedTexImage3D;
00211 extern PFNGLCOMPRESSEDTEXSUBIMAGE1DPROC glCompressedTexSubImage1D;
00212 extern PFNGLCOMPRESSEDTEXSUBIMAGE2DPROC glCompressedTexSubImage2D;
00213 extern PFNGLCOMPRESSEDTEXSUBIMAGE3DPROC glCompressedTexSubImage3D;
00214 extern PFNGLCOMPRESSEDTEXTUREIMAGE1DExTPROC glCompressedTextureImage1DExT;
00215 extern PFNGLCOMPRESSEDTEXTUREIMAGE2DExTPROC glCompressedTextureImage2DExT;
00216 extern PFNGLCOMPRESSEDTEXTUREIMAGE3DExTPROC glCompressedTextureImage3DExT;
00217 extern PFNGLCOMPRESSEDTEXTURESUBIMAGE1DExTPROC glCompressedTextureSubImage1DExT;
00218 extern PFNGLCOMPRESSEDTEXTURESUBIMAGE1DPROC glCompressedTextureSubImage1D;
00219 extern PFNGLCOMPRESSEDTEXTURESUBIMAGE2DExTPROC glCompressedTextureSubImage2DExT;
00220 extern PFNGLCOMPRESSEDTEXTURESUBIMAGE2DPROC glCompressedTextureSubImage2D;
00221 extern PFNGLCOMPRESSEDTEXTURESUBIMAGE3DExTPROC glCompressedTextureSubImage3DExT;
00222 extern PFNGLCOMPRESSEDTEXTURESUBIMAGE3DPROC glCompressedTextureSubImage3D;
00223 extern PFNGLCONSERVATIVERASTERPARAMETERFNVPROC glConservativeRasterParameterfNV;
00224 extern PFNGLCONSERVATIVERASTERPARAMETERINVPROC glConservativeRasterParameteriNV;
00225 extern PFNGLCOPYBUFFERSUBDATAPROC glCopyBufferSubData;
00226 extern PFNGLCOPYIMAGESUBDATAPROC glCopyImageSubData;
00227 extern PFNGLCOPYMULTITEXIMAGE1DExTPROC glCopyMultiTexImage1DExT;
00228 extern PFNGLCOPYMULTITEXIMAGE2DExTPROC glCopyMultiTexImage2DExT;
00229 extern PFNGLCOPYMULTITEXSUBIMAGE1DExTPROC glCopyMultiTexSubImage1DExT;
00230 extern PFNGLCOPYMULTITEXSUBIMAGE2DExTPROC glCopyMultiTexSubImage2DExT;
00231 extern PFNGLCOPYMULTITEXSUBIMAGE3DExTPROC glCopyMultiTexSubImage3DExT;
00232 extern PFNGLCOPYNAMEDBUFFERSUBDATAPROC glCopyNamedBufferSubData;
00233 extern PFNGLCOPYPATHNVPROC glCopyPathNV;
00234 extern PFNGLCOPYTEXIMAGE1DPROC glCopyTexImage1D;
00235 extern PFNGLCOPYTEXIMAGE2DPROC glCopyTexImage2D;
00236 extern PFNGLCOPYTEXSUBIMAGE1DPROC glCopyTexSubImage1D;
00237 extern PFNGLCOPYTEXSUBIMAGE2DPROC glCopyTexSubImage2D;
00238 extern PFNGLCOPYTEXSUBIMAGE3DPROC glCopyTexSubImage3D;
00239 extern PFNGLCOPYTEXTUREIMAGE1DExTPROC glCopyTextureImage1DExT;
00240 extern PFNGLCOPYTEXTUREIMAGE2DExTPROC glCopyTextureImage2DExT;
00241 extern PFNGLCOPYTEXTURESUBIMAGE1DExTPROC glCopyTextureSubImage1DExT;
00242 extern PFNGLCOPYTEXTURESUBIMAGE1DPROC glCopyTextureSubImage1D;
00243 extern PFNGLCOPYTEXTURESUBIMAGE2DExTPROC glCopyTextureSubImage2DExT;
00244 extern PFNGLCOPYTEXTURESUBIMAGE2DPROC glCopyTextureSubImage2D;
00245 extern PFNGLCOPYTEXTURESUBIMAGE3DExTPROC glCopyTextureSubImage3DExT;
```



```
00246 extern PFNGLCOPYTEXTURESUBIMAGE3DPROC glCopyTextureSubImage3D;
00247 extern PFNGLCOVERAGEMODULATIONNVPROC glCoverageModulationNV;
00248 extern PFNGLCOVERAGEMODULATIONTABLENVPROC glCoverageModulationTableNV;
00249 extern PFNGLCOVERFILLPATHINSTANCEDNVPROC glCoverFillPathInstancedNV;
00250 extern PFNGLCOVERFILLPATHNVPROC glCoverFillPathNV;
00251 extern PFNGLCOVERSTROKEPATHINSTANCEDNVPROC glCoverStrokePathInstancedNV;
00252 extern PFNGLCOVERSTROKEPATHNVPROC glCoverStrokePathNV;
00253 extern PFNGLCREATEBUFFERSPROC glCreateBuffers;
00254 extern PFNGLCREATecomMANDLISTSNVPROC glCreateCommandListsNV;
00255 extern PFNGLCREATEFRAMEBUFFERSPROC glCreateFramebuffers;
00256 extern PFNGLCREATEPERFQUERYINTELPROC glCreatePerfQueryINTEL;
00257 extern PFNGLCREATEPROGRAMPIPELINESPROC glCreateProgramPipelines;
00258 extern PFNGLCREATEPROGRAMPROC glCreateProgram;
00259 extern PFNGLCREATEQUERIESPROC glCreateQueries;
00260 extern PFNGLCREATERENDERBUFFERSPROC glCreateRenderbuffers;
00261 extern PFNGLCREATESAMPLERSPROC glCreateSamplers;
00262 extern PFNGLCREATESHADERPROC glCreateShader;
00263 extern PFNGLCREATESHADERPROGRAMEXTPROC glCreateShaderProgramEXT;
00264 extern PFNGLCREATESHADERPROGRAMVPROC glCreateShaderProgramv;
00265 extern PFNGLCREATESTATESNVPROC glCreateStatesNV;
00266 extern PFNGLCREATESYNCFROMCLEVENTARBPROC glCreateSyncFromCLeventARB;
00267 extern PFNGLCREATETEXTURESPROC glCreateTextures;
00268 extern PFNGLCREATETRANSFORMFEEDBACKSPROC glCreateTransformFeedbacks;
00269 extern PFNGLCREATEVERTEXARRAYSPROC glCreateVertexArrays;
00270 extern PFNGLCULLFACEPROC glCullFace;
00271 extern PFNGLDEBUGMESSAGECALLBACKARBPROC glDebugMessageCallbackARB;
00272 extern PFNGLDEBUGMESSAGECALLBACKPROC glDebugMessageCallback;
00273 extern PFNGLDEBUGMESSAGECONTROLARBPROC glDebugMessageControlARB;
00274 extern PFNGLDEBUGMESSAGECONTROLPROC glDebugMessageControl;
00275 extern PFNGLDEBUGMESSAGEINSERTARBPROC glDebugMessageInsertARB;
00276 extern PFNGLDEBUGMESSAGEINSERTPROC glDebugMessageInsert;
00277 extern PFNGLDELETEBUFFERSPROC glDeleteBuffers;
00278 extern PFNGLDELETecomMANDLISTSNVPROC glDeleteCommandListsNV;
00279 extern PFNGLDELETEFRAMEBUFFERSPROC glDeleteFramebuffers;
00280 extern PFNGLDELETENAMEDSTRINGARBPROC glDeleteNamedStringARB;
00281 extern PFNGLDELETEPATHSNVPROC glDeletePathsNV;
00282 extern PFNGLDELETEPERFMONITORSAMDPROC glDeletePerfMonitorsAMD;
00283 extern PFNGLDELETEPERFQUERYINTELPROC glDeletePerfQueryINTEL;
00284 extern PFNGLDELETEPROGRAMPIPELINESPROC glDeleteProgramPipelines;
00285 extern PFNGLDELETEPROGRAMPROC glDeleteProgram;
00286 extern PFNGLDELETEQUERIESPROC glDeleteQueries;
00287 extern PFNGLDELETERENDERBUFFERSPROC glDeleteRenderbuffers;
00288 extern PFNGLDELETESAMPLERSPROC glDeleteSamplers;
00289 extern PFNGLDELETESHADERPROC glDeleteShader;
00290 extern PFNGLDELETETESTATESNVPROC glDeleteStatesNV;
00291 extern PFNGLDELETESYNCPROC glDeleteSync;
00292 extern PFNGLDELETETEXTURESPROC glDeleteTextures;
00293 extern PFNGLDELETETRANSFORMFEEDBACKSPROC glDeleteTransformFeedbacks;
00294 extern PFNGLDELETEVERTEXARRAYSPROC glDeleteVertexArrays;
00295 extern PFNGLDEPTHFUNCPROC glDepthFunc;
00296 extern PFNGLDEPTHMASKPROC glDepthMask;
00297 extern PFNGLDEPTHRANGEARRAYVPROC glDepthRangeArrayv;
00298 extern PFNGLDEPTHRANGEFPROC glDepthRangef;
00299 extern PFNGLDEPTHRANGEINDEXEDPROC glDepthRangeIndexed;
00300 extern PFNGLDEPTHRANGEPROC glDepthRange;
00301 extern PFNGLDETACHSHADERPROC glDetachShader;
00302 extern PFNGLDISABLECLIENTSTATEIEXTPROC glDisableClientStateiEXT;
00303 extern PFNGLDISABLECLIENTSTATEINDEXEDEXTPROC glDisableClientStateIndexedEXT;
00304 extern PFNGLDISABLEINDEXEDEXTPROC glDisableIndexedEXT;
00305 extern PFNGLDISABLEIPROC glDisablei;
00306 extern PFNGLDISABLEPROC glDisable;
00307 extern PFNGLDISABLEVERTEXARRAYATTRIBEXTPROC glDisableVertexArrayAttribEXT;
00308 extern PFNGLDISABLEVERTEXARRAYATTRIBPROC glDisableVertexArrayAttrib;
00309 extern PFNGLDISABLEVERTEXARRAYEXTPROC glDisableVertexArrayEXT;
00310 extern PFNGLDISABLEVERTEXATTRIBARRAYPROC glDisableVertexAttribArray;
00311 extern PFNGLDISPATCHCOMPUTEGROUPSIZEARBPROC glDispatchComputeGroupSizeARB;
00312 extern PFNGLDISPATCHCOMPUTEINDIRECTPROC glDispatchComputeIndirect;
00313 extern PFNGLDISPATCHCOMPUTEPROC glDispatchCompute;
00314 extern PFNGLDRAWARRAYSINDIRECTPROC glDrawArraysIndirect;
00315 extern PFNGLDRAWARRAYSINSTANCEDARBPROC glDrawArraysInstancedARB;
00316 extern PFNGLDRAWARRAYSINSTANCEDBASEINSTANCEPROC glDrawArraysInstancedBaseInstance;
00317 extern PFNGLDRAWARRAYSINSTANCEDEXTPROC glDrawArraysInstancedEXT;
00318 extern PFNGLDRAWARRAYSINSTANCEDPROC glDrawArraysInstanced;
00319 extern PFNGLDRAWARRAYSPROC glDrawArrays;
00320 extern PFNGLDRAWBUFFERPROC glDrawBuffer;
00321 extern PFNGLDRAWBUFFERSPROC glDrawBuffers;
00322 extern PFNGLDRAWCOMMANDSADDRESSNVPROC glDrawCommandsAddressNV;
00323 extern PFNGLDRAWCOMMANDSNVPROC glDrawCommandsNV;
00324 extern PFNGLDRAWCOMMANDSSTATESADDRESSNVPROC glDrawCommandsStatesAddressNV;
00325 extern PFNGLDRAWCOMMANDSSTATESNVPROC glDrawCommandsStatesNV;
00326 extern PFNGLDRAWELEMENTSBASEVERTEXPROC glDrawElementsBaseVertex;
00327 extern PFNGLDRAWELEMENTSINDIRECTPROC glDrawElementsIndirect;
00328 extern PFNGLDRAWELEMENTSINSTANCEDARBPROC glDrawElementsInstancedARB;
00329 extern PFNGLDRAWELEMENTSINSTANCEDBASEINSTANCEPROC glDrawElementsInstancedBaseInstance;
00330 extern PFNGLDRAWELEMENTSINSTANCEDBASEVERTEXBASEINSTANCEPROC
    glDrawElementsInstancedBaseVertexBaseInstance;
00331 extern PFNGLDRAWELEMENTSINSTANCEDBASEVERTEXPROC glDrawElementsInstancedBaseVertex;
```

```

00332 extern PFNGLDRAWELEMENTSINSTANCEDEXTPROC glDrawElementsInstancedEXT;
00333 extern PFNGLDRAWELEMENTSINSTANCEDPROC glDrawElementsInstanced;
00334 extern PFNGLDRAWELEMENTSPROC glDrawElements;
00335 extern PFNGLDRAWRANGEELEMENTSBASEVERTEXPROC glDrawRangeElementsBaseVertex;
00336 extern PFNGLDRAWRANGEELEMENTSPROC glDrawRangeElements;
00337 extern PFNGLDRAWTRANSFORMFEEDBACKINSTANCEDPROC glDrawTransformFeedbackInstanced;
00338 extern PFNGLDRAWTRANSFORMFEEDBACKPROC glDrawTransformFeedback;
00339 extern PFNGLDRAWTRANSFORMFEEDBACKSTREAMINSTANCEDPROC glDrawTransformFeedbackStreamInstanced;
00340 extern PFNGLDRAWTRANSFORMFEEDBACKSTREAMPROC glDrawTransformFeedbackStream;
00341 extern PFNGLDRAWVKIMAGENVPROC glDrawVkImageNV;
00342 extern PFNGLEDGEFLAGFORMATNVPROC glEdgeFlagFormatNV;
00343 extern PFNGLENABLECLIENTSTATEIEXTPROC glEnableClientStateiEXT;
00344 extern PFNGLENABLECLIENTSTATEINDEXEDEXTPROC glEnableClientStateIndexedEXT;
00345 extern PFNGLENABLEINDEXEDEXTPROC glEnableIndexedEXT;
00346 extern PFNGLENABLEIPROC glEnablei;
00347 extern PFNGLENABLEPROC glEnable;
00348 extern PFNGLENABLEVERTEXARRAYATTRIBEXTPROC glEnableVertexArrayAttribEXT;
00349 extern PFNGLENABLEVERTEXARRAYATTRIBPROC glEnableVertexArrayAttrib;
00350 extern PFNGLENABLEVERTEXARRAYEXTPROC glEnableVertexArrayEXT;
00351 extern PFNGLENABLEVERTEXATTRIBARRAYPROC glEnableVertexAttribArray;
00352 extern PFNGLENDCONDITIONALRENDERNVPROC glEndConditionalRenderNV;
00353 extern PFNGLENDCONDITIONALRENDERPROC glEndConditionalRender;
00354 extern PFNGLENDPERFMONITORAMDPROC glEndPerfMonitorAMD;
00355 extern PFNGLENDPERFQUERYINTELPROC glEndPerfQueryINTEL;
00356 extern PFNGLENDQUERYINDEXEDPROC glEndQueryIndexed;
00357 extern PFNGLENDQUERYPROC glEndQuery;
00358 extern PFNGLENDTRANSFORMFEEDBACKPROC glEndTransformFeedback;
00359 extern PFNGLEVALUATEDEPTHVALUESARBPROC glEvaluateDepthValuesARB;
00360 extern PFNGLFENCESYNCPROC glFenceSync;
00361 extern PFNGLFINISHPROC glFinish;
00362 extern PFNGLFLUSHMAPPEDBUFFERRANGEPROC glFlushMappedBufferRange;
00363 extern PFNGLFLUSHMAPPEDNAMEDBUFFERRANGEEXTPROC glFlushMappedNamedBufferRangeEXT;
00364 extern PFNGLFLUSHMAPPEDNAMEDBUFFERRANGEPROC glFlushMappedNamedBufferRange;
00365 extern PFNGLFLUSHPROC glFlush;
00366 extern PFNGLFOGCOORDFORMATNVPROC glFogCoordFormatNV;
00367 extern PFNGLFRAGMENTCOVERAGECOLORNVPROC glFragmentCoverageColorNV;
00368 extern PFNGLFRAMEBUFFERDRAWBUFFEREXTPROC glFramebufferDrawBufferEXT;
00369 extern PFNGLFRAMEBUFFERDRAWBUFFERSEXTPROC glFramebufferDrawBuffersEXT;
00370 extern PFNGLFRAMEBUFFERPARAMETERIPROC glFramebufferParameteri;
00371 extern PFNGLFRAMEBUFFERREADBUFFEREXTPROC glFramebufferReadBufferEXT;
00372 extern PFNGLFRAMEBUFFERRENDERBUFFERPROC glFramebufferRenderbuffer;
00373 extern PFNGLFRAMEBUFFERSAMPLELOCATIONSFVARBPROC glFramebufferSampleLocationsfvARB;
00374 extern PFNGLFRAMEBUFFERSAMPLELOCATIONSFVNVPROC glFramebufferSampleLocationsfvNV;
00375 extern PFNGLFRAMEBUFFERTEXTURE1DPROC glFramebufferTexture1D;
00376 extern PFNGLFRAMEBUFFERTEXTURE2DPROC glFramebufferTexture2D;
00377 extern PFNGLFRAMEBUFFERTEXTURE3DPROC glFramebufferTexture3D;
00378 extern PFNGLFRAMEBUFFERTEXTUREARBPROC glFramebufferTextureARB;
00379 extern PFNGLFRAMEBUFFERTEXTUREFACEARBPROC glFramebufferTextureFaceARB;
00380 extern PFNGLFRAMEBUFFERTEXTURELAYERARBPROC glFramebufferTextureLayerARB;
00381 extern PFNGLFRAMEBUFFERTEXTURELAYERPROC glFramebufferTextureLayer;
00382 extern PFNGLFRAMEBUFFERTEXTUREMULTIVIEWOVRPROC glFramebufferTextureMultiviewOVR;
00383 extern PFNGLFRAMEBUFFERTEXTUREPROC glFramebufferTexture;
00384 extern PFNGLFRONTFACEPROC glFrontFace;
00385 extern PFNGLGENBUFFERSPROC glGenBuffers;
00386 extern PFNGLGENERATEMIPMAPPROC glGenerateMipmap;
00387 extern PFNGLGENERATEMULTITEXMIPMAPEXTPROC glGenerateMultiTexMipmapEXT;
00388 extern PFNGLGENERATETEXTUREMIPMAPEXTPROC glGenerateTextureMipmapEXT;
00389 extern PFNGLGENERATETEXTUREMIPMAPPROC glGenerateTextureMipmap;
00390 extern PFNGLGENFRAMEBUFFERSPROC glGenFramebuffers;
00391 extern PFNGLGENPATHSNVPROC glGenPathsNV;
00392 extern PFNGLGENPERFMONITORSAMDPROC glGenPerfMonitorsAMD;
00393 extern PFNGLGENPROGRAMPIPELINESPROC glGenProgramPipelines;
00394 extern PFNGLGENQUERIESPROC glGenQueries;
00395 extern PFNGLGENRENDERBUFFERSPROC glGenRenderbuffers;
00396 extern PFNGLGENSAMPLERSPROC glGenSamplers;
00397 extern PFNGLGENTEXTURESPROC glGenTextures;
00398 extern PFNGLGENTRANSFORMFEEDBACKSPROC glGenTransformFeedbacks;
00399 extern PFNGLGENVERTEXARRAYSPROC glGenVertexArrays;
00400 extern PFNGLGETACTIVEATOMICCOUNTERBUFFERIVPROC glGetActiveAtomicCounterBufferiv;
00401 extern PFNGLGETACTIVEATTRIBPROC glGetActiveAttrib;
00402 extern PFNGLGETACTIVESUBROUTINENAMEPROC glGetActiveSubroutineName;
00403 extern PFNGLGETACTIVESUBROUTINEUNIFORMIVPROC glGetActiveSubroutineUniformiv;
00404 extern PFNGLGETACTIVESUBROUTINEUNIFORMNAMEPROC glGetActiveSubroutineUniformName;
00405 extern PFNGLGETACTIVEUNIFORMBLOCKIVPROC glGetActiveUniformBlockiv;
00406 extern PFNGLGETACTIVEUNIFORMBLOCKNAMEPROC glGetActiveUniformBlockName;
00407 extern PFNGLGETACTIVEUNIFORMNAMEPROC glGetActiveUniformName;
00408 extern PFNGLGETACTIVEUNIFORMPROC glGetActiveUniform;
00409 extern PFNGLGETACTIVEUNIFORMSIVPROC glGetActiveUniformsiv;
00410 extern PFNGLGETATTACHEDSHADERSPROC glGetAttachedShaders;
00411 extern PFNGLGETATTRIBLOCATIONPROC glGetAttribLocation;
00412 extern PFNGLGETBOOLEANINDEXEDVEXTPROC glGetBooleanIndexedvEXT;
00413 extern PFNGLGETBOOLEANI_VPROC glGetBooleani_v;
00414 extern PFNGLGETBOOLEANVPROC glGetBooleanv;
00415 extern PFNGLGETBUFFERPARAMETERI64VPROC glGetBufferParameteri64v;
00416 extern PFNGLGETBUFFERPARAMETERIVPROC glGetBufferParameteriv;
00417 extern PFNGLGETBUFFERPARAMETERUI64VNVPROC glGetBufferParameterui64vNV;
00418 extern PFNGLGETBUFFERPOINTERVPROC glGetBufferPointerv;

```



```
00419 extern PFNGLGETBUFFERSUBDATAPROC glGetBufferSubData;
00420 extern PFNGLGETCOMMANDHEADERNVPROC glGetCommandHeaderNV;
00421 extern PFNGLGETCOMPRESSEDMULTITEXIMAGEEXTPROC glGetCompressedMultiTexImageEXT;
00422 extern PFNGLGETCOMPRESSEDTEXIMAGEPROC glGetCompressedTexImage;
00423 extern PFNGLGETCOMPRESSEDTEXTUREIMAGEEXTPROC glGetCompressedTextureImageEXT;
00424 extern PFNGLGETCOMPRESSEDTEXTUREIMAGEPROC glGetCompressedTextureImage;
00425 extern PFNGLGETCOMPRESSEDTEXTURESUBIMAGEPROC glGetCompressedTextureSubImage;
00426 extern PFNGLGETCOVERAGEMODULATIONTABLENVPROC glGetCoverageModulationTableNV;
00427 extern PFNGLGETDEBUGMESSAGELOGARBPROC glGetDebugMessageLogARB;
00428 extern PFNGLGETDEBUGMESSAGELOGPROC glGetDebugMessageLog;
00429 extern PFNGLGETDOUBLEINDEXEDVEXTPROC glGetDoubleIndexedvEXT;
00430 extern PFNGLGETDOUBLEI_VEXTPROC glGetDoublei_vEXT;
00431 extern PFNGLGETDOUBLEI_VPROC glGetDoublei_v;
00432 extern PFNGLGETDOUBLEVPROC glGetDoublev;
00433 extern PFNGLGETERRORPROC glGetError;
00434 extern PFNGLGETFIRSTPERFQUERYIDINTELPROC glGetFirstPerfQueryIdINTEL;
00435 extern PFNGLGETFLOATINDEXEDVEXTPROC glGetFloatIndexedvEXT;
00436 extern PFNGLGETFLOATI_VEXTPROC glGetFloati_vEXT;
00437 extern PFNGLGETFLOATI_VPROC glGetFloati_v;
00438 extern PFNGLGETFLOATVPROC glGetFloatv;
00439 extern PFNGLGETFRAGDATAINDEXPROC glGetFragDataIndex;
00440 extern PFNGLGETFRAGDATALOCATIONPROC glGetFragDataLocation;
00441 extern PFNGLGETFRAMEBUFFERATTACHMENTPARAMETERIVPROC glGetFramebufferAttachmentParameteriv;
00442 extern PFNGLGETFRAMEBUFFERPARAMETERIVEXTPROC glGetFramebufferParameterivEXT;
00443 extern PFNGLGETFRAMEBUFFERPARAMETERIVPROC glGetFramebufferParameteriv;
00444 extern PFNGLGETGRAPHICSRESETSTATUSARBPROC glGetGraphicsResetStatusARB;
00445 extern PFNGLGETGRAPHICSRESETSTATUSPROC glGetGraphicsResetStatus;
00446 extern PFNGLGETIMAGEHANDLEARBPROC glGetImageHandleARB;
00447 extern PFNGLGETIMAGEHANDLENVPROC glGetImageHandleNV;
00448 extern PFNGLGETINTEGER64I_VPROC glGetInteger64i_v;
00449 extern PFNGLGETINTEGER64VPROC glGetInteger64v;
00450 extern PFNGLGETINTEGERINDEXEDVEXTPROC glGetIntegerIndexedvEXT;
00451 extern PFNGLGETINTEGERI_VPROC glGetIntegeri_v;
00452 extern PFNGLGETINTEGERUI64I_VNVPROC glGetIntegerui64i_vNV;
00453 extern PFNGLGETINTEGERUI64VNVPROC glGetIntegerui64vNV;
00454 extern PFNGLGETINTEGERVPROC glGetIntegeriv;
00455 extern PFNGLGETINTERNALFORMATI64VPROC glGetInternalformati64v;
00456 extern PFNGLGETINTERNALFORMATIVPROC glGetInternalformativ;
00457 extern PFNGLGETINTERNALFORMATSAMPLEIVNVPROC glGetInternalformatSampleivNV;
00458 extern PFNGLGETMULTISAMPLEFVPROC glGetMultisamplefv;
00459 extern PFNGLGETMULTITEXENVFVEXTPROC glGetMultiTexEnvfvEXT;
00460 extern PFNGLGETMULTITEXENVIVEXTPROC glGetMultiTexEnvivEXT;
00461 extern PFNGLGETMULTITEXGENDVEXTPROC glGetMultiTexGenfvEXT;
00462 extern PFNGLGETMULTITEXGENFVEXTPROC glGetMultiTexGenfvEXT;
00463 extern PFNGLGETMULTITEXGENIVEXTPROC glGetMultiTexGenivEXT;
00464 extern PFNGLGETMULTITEXIMAGEEXTPROC glGetMultiTexImageEXT;
00465 extern PFNGLGETMULTITEXLEVELPARAMETERFVEXTPROC glGetMultiTexLevelParameterfvEXT;
00466 extern PFNGLGETMULTITEXLEVELPARAMETERIVEXTPROC glGetMultiTexLevelParameterivEXT;
00467 extern PFNGLGETMULTITEXPARAMETERFVEXTPROC glGetMultiTexParameterfvEXT;
00468 extern PFNGLGETMULTITEXPARAMETERIIVEXTPROC glGetMultiTexParameteriivEXT;
00469 extern PFNGLGETMULTITEXPARAMETERIUIVEXTPROC glGetMultiTexParameteriuiEXT;
00470 extern PFNGLGETMULTITEXPARAMETERIVEXTPROC glGetMultiTexParameterivEXT;
00471 extern PFNGLGETNAMEDBUFFERPARAMETERI64VPROC glGetNamedBufferParameteri64v;
00472 extern PFNGLGETNAMEDBUFFERPARAMETERIVEXTPROC glGetNamedBufferParameterivEXT;
00473 extern PFNGLGETNAMEDBUFFERPARAMETERIVPROC glGetNamedBufferParameteriv;
00474 extern PFNGLGETNAMEDBUFFERPARAMETERUI64VNVPROC glGetNamedBufferParameterui64vNV;
00475 extern PFNGLGETNAMEDBUFFERPOINTINTERVEXTPROC glGetNamedBufferPointervEXT;
00476 extern PFNGLGETNAMEDBUFFERPOINTINTERVPROC glGetNamedBufferPointerv;
00477 extern PFNGLGETNAMEDBUFFERSUBDATAEXTPROC glGetNamedBufferSubDataEXT;
00478 extern PFNGLGETNAMEDBUFFERSUBDATAPROC glGetNamedBufferSubData;
00479 extern PFNGLGETNAMEDFRAMEBUFFERATTACHMENTPARAMETERIVEXTPROC
glGetNamedFramebufferAttachmentParameterivEXT;
00480 extern PFNGLGETNAMEDFRAMEBUFFERATTACHMENTPARAMETERIVPROC glGetNamedFramebufferAttachmentParameteriv;
00481 extern PFNGLGETNAMEDFRAMEBUFFERPARAMETERIVEXTPROC glGetNamedFramebufferParameterivEXT;
00482 extern PFNGLGETNAMEDFRAMEBUFFERPARAMETERIVPROC glGetNamedFramebufferParameteriv;
00483 extern PFNGLGETNAMEDPROGRAMIVEXTPROC glGetNamedProgramivEXT;
00484 extern PFNGLGETNAMEDPROGRAMLOCALPARAMETERDVEXTPROC glGetNamedProgramLocalParameterdvEXT;
00485 extern PFNGLGETNAMEDPROGRAMLOCALPARAMETERFVEXTPROC glGetNamedProgramLocalParameterfvEXT;
00486 extern PFNGLGETNAMEDPROGRAMLOCALPARAMETERIIVEXTPROC glGetNamedProgramLocalParameteriivEXT;
00487 extern PFNGLGETNAMEDPROGRAMLOCALPARAMETERIUIVEXTPROC glGetNamedProgramLocalParameteriuiEXT;
00488 extern PFNGLGETNAMEDPROGRAMSTRINGEXTPROC glGetNamedProgramStringEXT;
00489 extern PFNGLGETNAMEDRENDERBUFFERPARAMETERIVEXTPROC glGetNamedRenderbufferParameterivEXT;
00490 extern PFNGLGETNAMEDRENDERBUFFERPARAMETERIVPROC glGetNamedRenderbufferParameteriv;
00491 extern PFNGLGETNAMEDSTRINGARBPROC glGetNamedStringARB;
00492 extern PFNGLGETNAMEDSTRINGIARBPROC glGetNamedStringivARB;
00493 extern PFNGLGETNCOMPRESSEDTEXIMAGEARBPROC glGetnCompressedTexImageARB;
00494 extern PFNGLGETNCOMPRESSEDTEXIMAGEPROC glGetnCompressedTexImage;
00495 extern PFNGLGETNEXTPERFQUERYIDINTELPROC glGetNextPerfQueryIdINTEL;
00496 extern PFNGLGETNTEXIMAGEARBPROC glGetnTexImageARB;
00497 extern PFNGLGETNTEXIMAGEPROC glGetnTexImage;
00498 extern PFNGLGETNUNIFORMDVARBPROC glGetnUniformdvARB;
00499 extern PFNGLGETNUNIFORMDVPROC glGetnUniformdv;
00500 extern PFNGLGETNUNIFORMFVARBPROC glGetnUniformfvARB;
00501 extern PFNGLGETNUNIFORMFVPROC glGetnUniformfv;
00502 extern PFNGLGETNUNIFORMI64VARBPROC glGetnUniformi64vARB;
00503 extern PFNGLGETNUNIFORMIVARBPROC glGetnUniformivARB;
00504 extern PFNGLGETNUNIFORMIVPROC glGetnUniformiv;
```

```

00505 extern PFNGLGETNUNIFORMUI64VARBPROC glGetnUniformui64vARB;
00506 extern PFNGLGETNUNIFORMUIVARBPROC glGetnUniformuiVARB;
00507 extern PFNGLGETNUNIFORMUIVPROC glGetnUniformiv;
00508 extern PFNGLGETOBJECTLABELEXTPROC glGetObjectLabelEXT;
00509 extern PFNGLGETOBJECTLABELPROC glGetObjectLabel;
00510 extern PFNGLGETOBJECTPTRLABELPROC glGetObjectPtrLabel;
00511 extern PFNGLGETPATHCOMMANDSNVPROC glGetPathCommandsNV;
00512 extern PFNGLGETPATHCOORDSNVPROC glGetPathCoordsNV;
00513 extern PFNGLGETPATHDASHARRAYNVPROC glGetPathDashArrayNV;
00514 extern PFNGLGETPATHLENGTHNVPROC glGetPathLengthNV;
00515 extern PFNGLGETPATHMETRICRANGENVPROC glGetPathMetricRangeNV;
00516 extern PFNGLGETPATHMETRICSNVPROC glGetPathMetricsNV;
00517 extern PFNGLGETPATHPARAMETERFVNVPROC glGetPathParameterfvNV;
00518 extern PFNGLGETPATHPARAMETERIVNVPROC glGetPathParameterivNV;
00519 extern PFNGLGETPATHSPACINGNVPROC glGetPathSpacingNV;
00520 extern PFNGLGETPERFCOUNTERINFOINTELPROC glGetPerfCounterInfoINTEL;
00521 extern PFNGLGETPERFMONITORCOUNTERDATAAMDPROC glGetPerfMonitorCounterDataAMD;
00522 extern PFNGLGETPERFMONITORCOUNTERINFOAMDPROC glGetPerfMonitorCounterInfoAMD;
00523 extern PFNGLGETPERFMONITORCOUNTERSAMDPROC glGetPerfMonitorCountersAMD;
00524 extern PFNGLGETPERFMONITORCOUNTERSTRINGAMDPROC glGetPerfMonitorCounterStringAMD;
00525 extern PFNGLGETPERFMONITORGROUPSAMDPROC glGetPerfMonitorGroupsAMD;
00526 extern PFNGLGETPERFMONITORGROUPSTRINGAMDPROC glGetPerfMonitorGroupStringAMD;
00527 extern PFNGLGETPERFQUERYDATAINTELPROC glGetPerfQueryDataINTEL;
00528 extern PFNGLGETPERFQUERYIDBYNAMEINTELPROC glGetPerfQueryIdByNameINTEL;
00529 extern PFNGLGETPERFQUERYINFOINTELPROC glGetPerfQueryInfoINTEL;
00530 extern PFNGLGETPOINTERINDEXEDVEXTPROC glGetPointerIndexedvEXT;
00531 extern PFNGLGETPOINTERI_VEXTPROC glGetPointeri_vEXT;
00532 extern PFNGLGETPOINTERVPROC glGetPointeriv;
00533 extern PFNGLGETPROGRAMBINARYPROC glGetProgramBinary;
00534 extern PFNGLGETPROGRAMINFOLOGPROC glGetProgramInfoLog;
00535 extern PFNGLGETPROGRAMINTERFACEIVPROC glGetProgramInterfaceiv;
00536 extern PFNGLGETPROGRAMIVPROC glGetProgramiv;
00537 extern PFNGLGETPROGRAMPIPELINEINFOLOGPROC glGetProgramPipelineInfoLog;
00538 extern PFNGLGETPROGRAMPIPELINEIVPROC glGetProgramPipelineiv;
00539 extern PFNGLGETPROGRAMRESOURCEFVNVPROC glGetProgramResourcefvNV;
00540 extern PFNGLGETPROGRAMRESOURCEINDEXPROC glGetProgramResourceIndex;
00541 extern PFNGLGETPROGRAMRESOURCEIVPROC glGetProgramResourceiv;
00542 extern PFNGLGETPROGRAMRESOURCELOCATIONINDEXPROC glGetProgramResourceLocationIndex;
00543 extern PFNGLGETPROGRAMRESOURCELOCATIONPROC glGetProgramResourceLocation;
00544 extern PFNGLGETPROGRAMRESOURCENAMEPROC glGetProgramResourceName;
00545 extern PFNGLGETPROGRAMSTAGEIVPROC glGetProgramStageiv;
00546 extern PFNGLGETQUERYBUFFEROBJECTI64VPROC glGetQueryBufferObjecti64v;
00547 extern PFNGLGETQUERYBUFFEROBJECTIVPROC glGetQueryBufferObjectiv;
00548 extern PFNGLGETQUERYBUFFEROBJECTUI64VPROC glGetQueryBufferObjectui64v;
00549 extern PFNGLGETQUERYBUFFEROBJECTUIVPROC glGetQueryBufferObjectuiv;
00550 extern PFNGLGETQUERYINDEXEDIVPROC glGetQueryIndexediv;
00551 extern PFNGLGETQUERYIVPROC glGetQueryiv;
00552 extern PFNGLGETQUERYOBJECTI64VPROC glGetQueryObjecti64v;
00553 extern PFNGLGETQUERYOBJECTIVPROC glGetQueryObjectiv;
00554 extern PFNGLGETQUERYOBJECTUI64VPROC glGetQueryObjectui64v;
00555 extern PFNGLGETQUERYOBJECTUIVPROC glGetQueryObjectuiv;
00556 extern PFNGLGETRENDERBUFFERPARAMETERIVPROC glGetRenderbufferParameteriv;
00557 extern PFNGLGETSAMPLERPARAMETERFVPROC glGetSamplerParameterfv;
00558 extern PFNGLGETSAMPLERPARAMETERIIVPROC glGetSamplerParameterIiv;
00559 extern PFNGLGETSAMPLERPARAMETERIUIVPROC glGetSamplerParameterIuiv;
00560 extern PFNGLGETSAMPLERPARAMETERIVPROC glGetSamplerParameteriv;
00561 extern PFNGLGETSHADERINFOLOGPROC glGetShaderInfoLog;
00562 extern PFNGLGETSHADERIVPROC glGetShaderiv;
00563 extern PFNGLGETSHADERPRECISIONFORMATPROC glGetShaderPrecisionFormat;
00564 extern PFNGLGETSHADERSOURCEPROC glGetShaderSource;
00565 extern PFNGLGETSTAGEINDEXNVPROC glGetStageIndexNV;
00566 extern PFNGLGETSTRINGIPROC glGetStringi;
00567 extern PFNGLGETSTRINGPROC glGetString;
00568 extern PFNGLGETSUBROUTINEINDEXPROC glGetSubroutineIndex;
00569 extern PFNGLGETSUBROUTINEUNIFORMLOCATIONPROC glGetSubroutineUniformLocation;
00570 extern PFNGLGETSYNClVPROC glGetSynciv;
00571 extern PFNGLGETTEXIMAGEPROC glGetTexImage;
00572 extern PFNGLGETTEXLEVELPARAMETERFVPROC glGetTexLevelParameterfv;
00573 extern PFNGLGETTEXLEVELPARAMETERIVPROC glGetTexLevelParameteriv;
00574 extern PFNGLGETTEXPARAMETERFVPROC glGetTexParameterfv;
00575 extern PFNGLGETTEXPARAMETERIIVPROC glGetTexParameterIiv;
00576 extern PFNGLGETTEXPARAMETERIUIVPROC glGetTexParameterIuiv;
00577 extern PFNGLGETTEXPARAMETERIVPROC glGetTexParameteriv;
00578 extern PFNGLGETTEXTUREHANDLEARBPROC glGetTextureHandleARB;
00579 extern PFNGLGETTEXTUREHANDLENVPROC glGetTextureHandleNV;
00580 extern PFNGLGETTEXTUREIMAGEEXTPROC glGetTextureImageEXT;
00581 extern PFNGLGETTEXTUREIMAGEPROC glGetTextureImage;
00582 extern PFNGLGETTEXTURELEVELPARAMETERFVEXTPROC glGetTextureLevelParameterfvEXT;
00583 extern PFNGLGETTEXTURELEVELPARAMETERFVPROC glGetTextureLevelParameterfv;
00584 extern PFNGLGETTEXTURELEVELPARAMETERIVEXTPROC glGetTextureLevelParameterivEXT;
00585 extern PFNGLGETTEXTURELEVELPARAMETERIVPROC glGetTextureLevelParameteriv;
00586 extern PFNGLGETTEXTUREPARAMETERFVEXTPROC glGetTextureParameterfvEXT;
00587 extern PFNGLGETTEXTUREPARAMETERFVPROC glGetTextureParameterfv;
00588 extern PFNGLGETTEXTUREPARAMETERIIVEXTPROC glGetTextureParameterIivEXT;
00589 extern PFNGLGETTEXTUREPARAMETERIIVPROC glGetTextureParameterIiv;
00590 extern PFNGLGETTEXTUREPARAMETERIUIVEXTPROC glGetTextureParameterIuivEXT;
00591 extern PFNGLGETTEXTUREPARAMETERIUIVPROC glGetTextureParameterIuiv;

```

```

00592 extern PFNGLGETTEXTUREPARAMETERIVEXTPROC glGetTextureParameterivEXT;
00593 extern PFNGLGETTEXTUREPARAMETERIVPROC glGetTextureParameteriv;
00594 extern PFNGLGETTEXTURESAMPLERHANDLEARBPROC glGetTextureSamplerHandleARB;
00595 extern PFNGLGETTEXTURESAMPLERHANDLENVPROC glGetTextureSamplerHandleNV;
00596 extern PFNGLGETTEXTURESUBIMAGEPROC glGetTextureSubImage;
00597 extern PFNGLGETTRANSFORMFEEDBACKI64VPROC glGetTransformFeedbacki64_v;
00598 extern PFNGLGETTRANSFORMFEEDBACKIVPROC glGetTransformFeedbackiv;
00599 extern PFNGLGETTRANSFORMFEEDBACKI_VPROC glGetTransformFeedbacki_v;
00600 extern PFNGLGETTRANSFORMFEEDBACKVARYINGPROC glGetTransformFeedbackVarying;
00601 extern PFNGLGETUNIFORMBLOCKINDEXPROC glGetUniformBlockIndex;
00602 extern PFNGLGETUNIFORMDVPROC glGetUniformdv;
00603 extern PFNGLGETUNIFORMFVPROC glGetUniformfv;
00604 extern PFNGLGETUNIFORMI64VARBPROC glGetUniformi64vARB;
00605 extern PFNGLGETUNIFORMI64VNVPROC glGetUniformi64vNV;
00606 extern PFNGLGETUNIFORMINDICESPROC glGetUniformIndices;
00607 extern PFNGLGETUNIFORMIVPROC glGetUniformiv;
00608 extern PFNGLGETUNIFORMLOCATIONPROC glGetUniformLocation;
00609 extern PFNGLGETUNIFORMSUBROUTINEUIVPROC glGetUniformSubroutineuiv;
00610 extern PFNGLGETUNIFORMUI64VARBPROC glGetUniformui64vARB;
00611 extern PFNGLGETUNIFORMUI64VNVPROC glGetUniformui64vNV;
00612 extern PFNGLGETUNIFORMUIVPROC glGetUniformuiv;
00613 extern PFNGLGETVERTEXARRAYINDEXED64IVPROC glGetVertexArrayIndexed64iv;
00614 extern PFNGLGETVERTEXARRAYINDEXEDIVPROC glGetVertexArrayIndexediv;
00615 extern PFNGLGETVERTEXARRAYINTEGERI_VEXTPROC glGetVertexArrayIntegeri_vEXT;
00616 extern PFNGLGETVERTEXARRAYINTEGERVEXTPROC glGetVertexArrayInteger_vEXT;
00617 extern PFNGLGETVERTEXARRAYIVPROC glGetVertexArrayiv;
00618 extern PFNGLGETVERTEXARRAYPOINTERI_VEXTPROC glGetVertexArrayPointeri_vEXT;
00619 extern PFNGLGETVERTEXARRAYPOINTERVEXTPROC glGetVertexArrayPointervEXT;
00620 extern PFNGLGETVERTEXATTRIBDVPROC glGetVertexAttribdv;
00621 extern PFNGLGETVERTEXATTRIBFVPROC glGetVertexAttribfv;
00622 extern PFNGLGETVERTEXATTRIBIIVPROC glGetVertexAttribIiv;
00623 extern PFNGLGETVERTEXATTRIBIUIVPROC glGetVertexAttribIuiv;
00624 extern PFNGLGETVERTEXATTRIBIVPROC glGetVertexAttribiv;
00625 extern PFNGLGETVERTEXATTRIBLDVPROC glGetVertexAttribldv;
00626 extern PFNGLGETVERTEXATTRIBLI64VNVPROC glGetVertexAttribLi64vNV;
00627 extern PFNGLGETVERTEXATTRIBLUI64VARBPROC glGetVertexAttribLui64vARB;
00628 extern PFNGLGETVERTEXATTRIBLUI64VNVPROC glGetVertexAttribLui64vNV;
00629 extern PFNGLGETVERTEXATTRIBPOINTERVPROC glGetVertexAttribPointerv;
00630 extern PFNGLGETVKPROCADDRNVPROC glGetVkProcAddrNV;
00631 extern PFNGLHINTPROC glHint;
00632 extern PFNGLINDEXFORMATNVPROC glIndexFormatNV;
00633 extern PFNGLINSERTEVENTMARKEREXTPROC glInsertEventMarkerEXT;
00634 extern PFNGLINTERPOLATEPATHSNVPROC glInterpolatePathsNV;
00635 extern PFNGLINVALIDATEBUFFERDATAPROC glInvalidateBufferData;
00636 extern PFNGLINVALIDATEBUFFERSUBDATAPROC glInvalidateBufferSubData;
00637 extern PFNGLINVALIDATEFRAMEBUFFERPROC glInvalidateFramebuffer;
00638 extern PFNGLINVALIDATENAMEDFRAMEBUFFERDATAPROC glInvalidateNamedFramebufferData;
00639 extern PFNGLINVALIDATENAMEDFRAMEBUFFERSUBDATAPROC glInvalidateNamedFramebufferSubData;
00640 extern PFNGLINVALIDATESUBFRAMEBUFFERPROC glInvalidateSubFramebuffer;
00641 extern PFNGLINVALIDATETEXIMAGEPROC glInvalidateTexImage;
00642 extern PFNGLINVALIDATETEXSUBIMAGEPROC glInvalidateTexSubImage;
00643 extern PFNGLISBUFFERPROC glIsBuffer;
00644 extern PFNGLISBUFFERRESIDENTNVPROC glIsBufferResidentNV;
00645 extern PFNGLISCOMMANDLISTNVPROC glIsCommandListNV;
00646 extern PFNGLISENABLEDINDEXEDEXTPROC glIsEnabledIndexedEXT;
00647 extern PFNGLISENABLEDIPROC glIsEnabledi;
00648 extern PFNGLISENABLEDPROC glIsEnabled;
00649 extern PFNGLISFRAMEBUFFERPROC glIsFramebuffer;
00650 extern PFNGLISIMAGEHANDLERESIDENTARBPROC glIsImageHandleResidentARB;
00651 extern PFNGLISIMAGEHANDLERESIDENTNVPROC glIsImageHandleResidentNV;
00652 extern PFNGLISNAMEDBUFFERRESIDENTNVPROC glIsNamedBufferResidentNV;
00653 extern PFNGLISNAMEDSTRINGARBPROC glIsNamedStringARB;
00654 extern PFNGLISPATHTNVPROC glIsPathNV;
00655 extern PFNGLISPOINTINFILLPATHNVPROC glIsPointInFillPathNV;
00656 extern PFNGLISPOINTINSTROKEPATHNVPROC glIsPointInStrokePathNV;
00657 extern PFNGLISPROGRAMPIPELINEPROC glIsProgramPipeline;
00658 extern PFNGLISPROGRAMPROC glIsProgram;
00659 extern PFNGLISQUERYPROC glIsQuery;
00660 extern PFNGLISRENDERBUFFERPROC glIsRenderbuffer;
00661 extern PFNGLISSAMPLERPROC glIsSampler;
00662 extern PFNGLISSHADERPROC glIsShader;
00663 extern PFNGLISSTATENVPROC glIsStateNV;
00664 extern PFNGLISSYNCPROC glIsSync;
00665 extern PFNGLISTEXTUREHANDLERESIDENTARBPROC glIsTextureHandleResidentARB;
00666 extern PFNGLISTEXTUREHANDLERESIDENTNVPROC glIsTextureHandleResidentNV;
00667 extern PFNGLISTEXTUREPROC glIsTexture;
00668 extern PFNGLISTRANSFORMFEEDBACKPROC glIsTransformFeedback;
00669 extern PFNGLISVERTEXARRAYPROC glIsVertexArray;
00670 extern PFNGLLABELOBJECTEXTPROC glLabelObjectEXT;
00671 extern PFNGLLINEWIDTHPROC glLineWidth;
00672 extern PFNGLLINKPROGRAMPROC glLinkProgram;
00673 extern PFNGLLISTDRAWCOMMANDSSTATESCIENTNVPROC glListDrawCommandsStatesClientNV;
00674 extern PFNGLLOGICOPPROC glLogicOp;
00675 extern PFNGLMAKEBUFFERNONRESIDENTNVPROC glMakeBufferNonResidentNV;
00676 extern PFNGLMAKEBUFFERRESIDENTNVPROC glMakeBufferResidentNV;
00677 extern PFNGLMAKEIMAGEHANDLENONRESIDENTARBPROC glMakeImageHandleNonResidentARB;
00678 extern PFNGLMAKEIMAGEHANDLENONRESIDENTNVPROC glMakeImageHandleNonResidentNV;

```

```
00679 extern PFNGLMAKEIMAGEHANDLERESIDENTARBPROC glMakeImageHandleResidentARB;
00680 extern PFNGLMAKEIMAGEHANDLERESIDENTNVPROC glMakeImageHandleResidentNV;
00681 extern PFNGLMAKENAMEDBUFFERNONRESIDENTNVPROC glMakeNamedBufferNonResidentNV;
00682 extern PFNGLMAKENAMEDBUFFERRESIDENTNVPROC glMakeNamedBufferResidentNV;
00683 extern PFNGLMAKETEXTUREHANDLENONRESIDENTARBPROC glMakeTextureHandleNonResidentARB;
00684 extern PFNGLMAKETEXTUREHANDLENONRESIDENTNVPROC glMakeTextureHandleNonResidentNV;
00685 extern PFNGLMAKETEXTUREHANDLERESIDENTARBPROC glMakeTextureHandleResidentARB;
00686 extern PFNGLMAKETEXTUREHANDLERESIDENTNVPROC glMakeTextureHandleResidentNV;
00687 extern PFNGLMAPBUFFERPROC glMapBuffer;
00688 extern PFNGLMAPBUFFERRANGEPROC glMapBufferRange;
00689 extern PFNGLMAPNAMEDBUFFEREXTPROC glMapNamedBufferEXT;
00690 extern PFNGLMAPNAMEDBUFFERPROC glMapNamedBuffer;
00691 extern PFNGLMAPNAMEDBUFFERRANGEEXTPROC glMapNamedBufferRangeEXT;
00692 extern PFNGLMAPNAMEDBUFFERRANGEPROC glMapNamedBufferRange;
00693 extern PFNGLMATRIXFRUSTUMEXTPROC glMatrixFrustumEXT;
00694 extern PFNGLMATRIXLOAD3X2FNVPROC glMatrixLoad3x2fNV;
00695 extern PFNGLMATRIXLOAD3X3FNVPROC glMatrixLoad3x3fNV;
00696 extern PFNGLMATRIXLOADDEXTPROC glMatrixLoadEXT;
00697 extern PFNGLMATRIXLOADFEXTPROC glMatrixLoadfEXT;
00698 extern PFNGLMATRIXLOADIDENTITYEXTPROC glMatrixLoadIdentityEXT;
00699 extern PFNGLMATRIXLOADTRANSPOSE3X3FNVPROC glMatrixLoadTranspose3x3fNV;
00700 extern PFNGLMATRIXLOADTRANSPOSEDEXTPROC glMatrixLoadTransposedEXT;
00701 extern PFNGLMATRIXLOADTRANSPOSEFEXTPROC glMatrixLoadTransposefEXT;
00702 extern PFNGLMATRIXMULT3X2FNVPROC glMatrixMult3x2fNV;
00703 extern PFNGLMATRIXMULT3X3FNVPROC glMatrixMult3x3fNV;
00704 extern PFNGLMATRIXMULTDEXTPROC glMatrixMultEXT;
00705 extern PFNGLMATRIXMULTFEXTPROC glMatrixMultfEXT;
00706 extern PFNGLMATRIXMULTTRANSPOSE3X3FNVPROC glMatrixMultTranspose3x3fNV;
00707 extern PFNGLMATRIXMULTTRANSPOSEDEXTPROC glMatrixMultTransposedEXT;
00708 extern PFNGLMATRIXMULTTRANSPOSEFEXTPROC glMatrixMultTransposefEXT;
00709 extern PFNGLMATRIXORTHOEXTPROC glMatrixOrthoEXT;
00710 extern PFNGLMATRIXPOPEXTPROC glMatrixPopEXT;
00711 extern PFNGLMATRIXPUSHEXTPROC glMatrixPushEXT;
00712 extern PFNGLMATRIXROTATEDEXTPROC glMatrixRotatedEXT;
00713 extern PFNGLMATRIXROTATEFEXTPROC glMatrixRotatefEXT;
00714 extern PFNGLMATRIXSCALEDEXTPROC glMatrixScaledEXT;
00715 extern PFNGLMATRIXSCALEFEXTPROC glMatrixScalefEXT;
00716 extern PFNGLMATRIXTRANSLATEDEXTPROC glMatrixTranslatedEXT;
00717 extern PFNGLMATRIXTRANSLATEFEXTPROC glMatrixTranslatefEXT;
00718 extern PFNGLMAXSHADERCOMPILERTHREADSARBPROC glMaxShaderCompilerThreadsARB;
00719 extern PFNGLMEMORYBARRIERBYREGIONPROC glMemoryBarrierByRegion;
00720 extern PFNGLMEMORYBARRIERPROC glMemoryBarrier;
00721 extern PFNGLMINSAMPLESHADINGARBPROC glMinSampleShadingARB;
00722 extern PFNGLMINSAMPLESHADINGPROC glMinSampleShading;
00723 extern PFNGLMULTIDRAWARRAYSINDIRECTBINDLESSCOUNTNVPROC glMultiDrawArraysIndirectBindlessCountNV;
00724 extern PFNGLMULTIDRAWARRAYSINDIRECTBINDLESSNVPROC glMultiDrawArraysIndirectBindlessNV;
00725 extern PFNGLMULTIDRAWARRAYSINDIRECTCOUNTARBPROC glMultiDrawArraysIndirectCountARB;
00726 extern PFNGLMULTIDRAWARRAYSINDIRECTPROC glMultiDrawArraysIndirect;
00727 extern PFNGLMULTIDRAWARRAYSPROC glMultiDrawArrays;
00728 extern PFNGLMULTIDRAWELEMENTSBASEVERTEXPROC glMultiDrawElementsBaseVertex;
00729 extern PFNGLMULTIDRAWELEMENTSINDIRECTBINDLESSCOUNTNVPROC glMultiDrawElementsIndirectBindlessCountNV;
00730 extern PFNGLMULTIDRAWELEMENTSINDIRECTBINDLESSNVPROC glMultiDrawElementsIndirectBindlessNV;
00731 extern PFNGLMULTIDRAWELEMENTSINDIRECTCOUNTARBPROC glMultiDrawElementsIndirectCountARB;
00732 extern PFNGLMULTIDRAWELEMENTSINDIRECTPROC glMultiDrawElementsIndirect;
00733 extern PFNGLMULTIDRAWELEMENTSPROC glMultiDrawElements;
00734 extern PFNGLMULTITEXBUFFEREXTPROC glMultiTexBufferEXT;
00735 extern PFNGLMULTITEXCOORDPOINTEREXTPROC glMultiTexCoordPointerEXT;
00736 extern PFNGLMULTITEXENVFEXTPROC glMultiTexEnvfEXT;
00737 extern PFNGLMULTITEXENVFVEXTPROC glMultiTexEnvfvEXT;
00738 extern PFNGLMULTITEXENVIEXTPROC glMultiTexEnviEXT;
00739 extern PFNGLMULTITEXENVIVEXTPROC glMultiTexEnvivEXT;
00740 extern PFNGLMULTITEXGENDEXTPROC glMultiTexGendEXT;
00741 extern PFNGLMULTITEXGENDVEXTPROC glMultiTexGendvEXT;
00742 extern PFNGLMULTITEXGENFEXTPROC glMultiTexGenfEXT;
00743 extern PFNGLMULTITEXGENFVEXTPROC glMultiTexGenfvEXT;
00744 extern PFNGLMULTITEXGENIEXTPROC glMultiTexGeniEXT;
00745 extern PFNGLMULTITEXGENIVEXTPROC glMultiTexGenivEXT;
00746 extern PFNGLMULTITEXIMAGE1DEXTPROC glMultiTexImage1DEXT;
00747 extern PFNGLMULTITEXIMAGE2DEXTPROC glMultiTexImage2DEXT;
00748 extern PFNGLMULTITEXIMAGE3DEXTPROC glMultiTexImage3DEXT;
00749 extern PFNGLMULTITEXPARAMETERFEXTPROC glMultiTexParameterfEXT;
00750 extern PFNGLMULTITEXPARAMETERFVEXTPROC glMultiTexParameterfvEXT;
00751 extern PFNGLMULTITEXPARAMETERIEXTPROC glMultiTexParameteriEXT;
00752 extern PFNGLMULTITEXPARAMETERIIVEXTPROC glMultiTexParameterIivEXT;
00753 extern PFNGLMULTITEXPARAMETERIUIVEXTPROC glMultiTexParameterIuivEXT;
00754 extern PFNGLMULTITEXPARAMETERIVEXTPROC glMultiTexParameterivEXT;
00755 extern PFNGLMULTITEXRENDERBUFFEREXTPROC glMultiTexRenderbufferEXT;
00756 extern PFNGLMULTITEXSUBIMAGE1DEXTPROC glMultiTexSubImage1DEXT;
00757 extern PFNGLMULTITEXSUBIMAGE2DEXTPROC glMultiTexSubImage2DEXT;
00758 extern PFNGLMULTITEXSUBIMAGE3DEXTPROC glMultiTexSubImage3DEXT;
00759 extern PFNGLNAMEDBUFFERDATAEXTPROC glNamedBufferDataEXT;
00760 extern PFNGLNAMEDBUFFERDATAPROC glNamedBufferData;
00761 extern PFNGLNAMEDBUFFERPAGECOMMITMENTARBPROC glNamedBufferPageCommitmentARB;
00762 extern PFNGLNAMEDBUFFERPAGECOMMITMENTEXTPROC glNamedBufferPageCommitmentEXT;
00763 extern PFNGLNAMEDBUFFERSTORAGEEXTPROC glNamedBufferStorageEXT;
00764 extern PFNGLNAMEDBUFFERSTORAGEPROC glNamedBufferStorage;
00765 extern PFNGLNAMEDBUFFERSUBDATAEXTPROC glNamedBufferSubDataEXT;
```



```
00766 extern PFNGLNAMEDBUFFERSUBDATAPROC glNamedBufferSubData;
00767 extern PFNGLNAMEDCOPYBUFFERSUBDATAEXTPROC glNamedCopyBufferSubDataEXT;
00768 extern PFNGLNAMEDFRAMEBUFFERDRAWBUFFERPROC glNamedFramebufferDrawBuffer;
00769 extern PFNGLNAMEDFRAMEBUFFERDRAWBUFFERSPROC glNamedFramebufferDrawBuffers;
00770 extern PFNGLNAMEDFRAMEBUFFERPARAMETERIEXTPROC glNamedFramebufferParameteriEXT;
00771 extern PFNGLNAMEDFRAMEBUFFERPARAMETERIPROC glNamedFramebufferParameteri;
00772 extern PFNGLNAMEDFRAMEBUFFERREADBUFFERPROC glNamedFramebufferReadBuffer;
00773 extern PFNGLNAMEDFRAMEBUFFERRENDERBUFFEREXTPROC glNamedFramebufferRenderbufferEXT;
00774 extern PFNGLNAMEDFRAMEBUFFERRENDERBUFFERPROC glNamedFramebufferRenderbuffer;
00775 extern PFNGLNAMEDFRAMEBUFFERSAMPLELOCATIONSFVARBPROC glNamedFramebufferSampleLocationsfvARB;
00776 extern PFNGLNAMEDFRAMEBUFFERSAMPLELOCATIONSFVNVPROC glNamedFramebufferSampleLocationsfvNV;
00777 extern PFNGLNAMEDFRAMEBUFFERTEXTURE1DEXTPROC glNamedFramebufferTexture1DEXT;
00778 extern PFNGLNAMEDFRAMEBUFFERTEXTURE2DEXTPROC glNamedFramebufferTexture2DEXT;
00779 extern PFNGLNAMEDFRAMEBUFFERTEXTURE3DEXTPROC glNamedFramebufferTexture3DEXT;
00780 extern PFNGLNAMEDFRAMEBUFFERTEXTUREEXTPROC glNamedFramebufferTextureEXT;
00781 extern PFNGLNAMEDFRAMEBUFFERTEXTUREFACEEXTPROC glNamedFramebufferTextureFaceEXT;
00782 extern PFNGLNAMEDFRAMEBUFFERTEXTURELAYEREXTPROC glNamedFramebufferTextureLayerEXT;
00783 extern PFNGLNAMEDFRAMEBUFFERTEXTURELAYERPROC glNamedFramebufferTextureLayer;
00784 extern PFNGLNAMEDFRAMEBUFFERTEXTUREPROC glNamedFramebufferTexture;
00785 extern PFNGLNAMEDPROGRAMLOCALPARAMETER4DEXTPROC glNamedProgramLocalParameter4dEXT;
00786 extern PFNGLNAMEDPROGRAMLOCALPARAMETER4DVEXTPROC glNamedProgramLocalParameter4dvEXT;
00787 extern PFNGLNAMEDPROGRAMLOCALPARAMETER4FEXTPROC glNamedProgramLocalParameter4fEXT;
00788 extern PFNGLNAMEDPROGRAMLOCALPARAMETER4FVEXTPROC glNamedProgramLocalParameter4fvEXT;
00789 extern PFNGLNAMEDPROGRAMLOCALPARAMETERI4IEXTPROC glNamedProgramLocalParameterI4iEXT;
00790 extern PFNGLNAMEDPROGRAMLOCALPARAMETERI4IVEXTPROC glNamedProgramLocalParameterI4ivEXT;
00791 extern PFNGLNAMEDPROGRAMLOCALPARAMETERI4UIEXTPROC glNamedProgramLocalParameterI4uiEXT;
00792 extern PFNGLNAMEDPROGRAMLOCALPARAMETERI4UIVEXTPROC glNamedProgramLocalParameterI4uivEXT;
00793 extern PFNGLNAMEDPROGRAMLOCALPARAMETERS4FVEXTPROC glNamedProgramLocalParameters4fvEXT;
00794 extern PFNGLNAMEDPROGRAMLOCALPARAMETERSI4IVEXTPROC glNamedProgramLocalParametersI4ivEXT;
00795 extern PFNGLNAMEDPROGRAMLOCALPARAMETERSI4UIVEXTPROC glNamedProgramLocalParametersI4uivEXT;
00796 extern PFNGLNAMEDPROGRAMSTRINGEXTPROC glNamedProgramStringEXT;
00797 extern PFNGLNAMEDRENDERBUFFERSTORAGEEXTPROC glNamedRenderbufferStorageEXT;
00798 extern PFNGLNAMEDRENDERBUFFERSTORAGEMULTISAMPLECOVERAGEEXTPROC
    glNamedRenderbufferStorageMultisampleCoverageEXT;
00799 extern PFNGLNAMEDRENDERBUFFERSTORAGEMULTISAMPLEEXTPROC glNamedRenderbufferStorageMultisampleEXT;
00800 extern PFNGLNAMEDRENDERBUFFERSTORAGEMULTISAMPLEPROC glNamedRenderbufferStorageMultisample;
00801 extern PFNGLNAMEDRENDERBUFFERSTORAGEPROC glNamedRenderbufferStorage;
00802 extern PFNGLNAMEDSTRINGARBPROC glNamedStringARB;
00803 extern PFNGLNORMALFORMATNVPROC glNormalFormatNV;
00804 extern PFNGLOBJECTLABELPROC glObjectLabel;
00805 extern PFNGLOBJECTPTRLABELPROC glObjectPtrLabel;
00806 extern PFNGLPATCHPARAMETERFVPROC glPatchParameterfv;
00807 extern PFNGLPATCHPARAMETERIPROC glPatchParameteri;
00808 extern PFNGLPATHCOMMANDSNVPROC glPathCommandsNV;
00809 extern PFNGLPATHCOORDSNVPROC glPathCoordsNV;
00810 extern PFNGLPATHCOVERDEPTHFUNCNVPROC glPathCoverDepthFuncNV;
00811 extern PFNGLPATHDASHARRAYNVPROC glPathDashArrayNV;
00812 extern PFNGLPATHGLYPHINDEXARRAYNVPROC glPathGlyphIndexArrayNV;
00813 extern PFNGLPATHGLYPHINDEXRANGENVPROC glPathGlyphIndexRangeNV;
00814 extern PFNGLPATHGLYPHRANGENVPROC glPathGlyphRangeNV;
00815 extern PFNGLPATHGLYPHSNVPROC glPathGlyphsNV;
00816 extern PFNGLPATHMEMORYGLYPHINDEXARRAYNVPROC glPathMemoryGlyphIndexArrayNV;
00817 extern PFNGLPATHPARAMETERFNVPROC glPathParameterfNV;
00818 extern PFNGLPATHPARAMETERFVNVPROC glPathParameterfvNV;
00819 extern PFNGLPATHPARAMETERINVPROC glPathParameteriNV;
00820 extern PFNGLPATHPARAMETERIVNVPROC glPathParameterivNV;
00821 extern PFNGLPATHSTENCILDEPTHOFFSETNVPROC glPathStencilDepthOffsetNV;
00822 extern PFNGLPATHSTENCILFUNCNVPROC glPathStencilFuncNV;
00823 extern PFNGLPATHSTRINGNVPROC glPathStringNV;
00824 extern PFNGLPATHSUBCOMMANDSNVPROC glPathSubCommandsNV;
00825 extern PFNGLPATHSUBCOORDSNVPROC glPathSubCoordsNV;
00826 extern PFNGLPAUSETRANSFORMFEEDBACKPROC glPauseTransformFeedback;
00827 extern PFNGLPIXELSTOREFPROC glPixelStoref;
00828 extern PFNGLPIXELSTOREIPROC glPixelStorei;
00829 extern PFNGLPOINTALONGPATHNVPROC glPointAlongPathNV;
00830 extern PFNGLPOINTPARAMETERFPROC glPointParameterf;
00831 extern PFNGLPOINTPARAMETERFVPROC glPointParameterfv;
00832 extern PFNGLPOINTPARAMETERIPROC glPointParameteri;
00833 extern PFNGLPOINTPARAMETERIVPROC glPointParameteriv;
00834 extern PFNGLPOINTSIZEPROC glPointSize;
00835 extern PFNGLPOLYGONMODEPROC glPolygonMode;
00836 extern PFNGLPOLYGONOFFSETCLAMPEXTPROC glPolygonOffsetClampEXT;
00837 extern PFNGLPOLYGONOFFSETPROC glPolygonOffset;
00838 extern PFNGLPOPDEBUGGROUPPROC glPopDebugGroup;
00839 extern PFNGLPOPDEBUGGROUPMARKEREXTPROC glPopDebugGroupMarkerEXT;
00840 extern PFNGLPRIMITIVEBOUNDINGBOXARBPROC glPrimitiveBoundingBoxARB;
00841 extern PFNGLPRIMITIVERESTARTINDEXPROC glPrimitiveRestartIndex;
00842 extern PFNGLPROGRAMBINARYPROC glProgramBinary;
00843 extern PFNGLPROGRAMPARAMETERIARBPROC glProgramParameteriARB;
00844 extern PFNGLPROGRAMPARAMETERIPROC glProgramParameteri;
00845 extern PFNGLPROGRAMPATHFRAGMENTINPUTGENNVPROC glProgramPathFragmentInputGenNV;
00846 extern PFNGLPROGRAMUNIFORM1DEXTPROC glProgramUniform1dEXT;
00847 extern PFNGLPROGRAMUNIFORM1DPROC glProgramUniform1d;
00848 extern PFNGLPROGRAMUNIFORM1DVEXTPROC glProgramUniform1dvEXT;
00849 extern PFNGLPROGRAMUNIFORM1DVPROC glProgramUniform1dv;
00850 extern PFNGLPROGRAMUNIFORM1FEXTPROC glProgramUniform1fEXT;
00851 extern PFNGLPROGRAMUNIFORM1FPROC glProgramUniform1f;
```

```
00852 extern PFNGLPROGRAMUNIFORM1FVEXTPROC glProgramUniform1fvEXT;
00853 extern PFNGLPROGRAMUNIFORM1FVPROC glProgramUniform1fv;
00854 extern PFNGLPROGRAMUNIFORM1I64ARBPROC glProgramUniform1i64ARB;
00855 extern PFNGLPROGRAMUNIFORM1I64NVPROC glProgramUniform1i64NV;
00856 extern PFNGLPROGRAMUNIFORM1I64VARBPROC glProgramUniform1i64vARB;
00857 extern PFNGLPROGRAMUNIFORM1I64VNVPROC glProgramUniform1i64vNV;
00858 extern PFNGLPROGRAMUNIFORM1IEXTPROC glProgramUniform1iEXT;
00859 extern PFNGLPROGRAMUNIFORM1IPROC glProgramUniform1i;
00860 extern PFNGLPROGRAMUNIFORM1IVEXTPROC glProgramUniform1ivEXT;
00861 extern PFNGLPROGRAMUNIFORM1IVPROC glProgramUniform1iv;
00862 extern PFNGLPROGRAMUNIFORM1UI64ARBPROC glProgramUniform1ui64ARB;
00863 extern PFNGLPROGRAMUNIFORM1UI64NVPROC glProgramUniform1ui64NV;
00864 extern PFNGLPROGRAMUNIFORM1UI64VARBPROC glProgramUniform1ui64vARB;
00865 extern PFNGLPROGRAMUNIFORM1UI64VNVPROC glProgramUniform1ui64vNV;
00866 extern PFNGLPROGRAMUNIFORM1UIEXTPROC glProgramUniform1uiEXT;
00867 extern PFNGLPROGRAMUNIFORM1UIPROC glProgramUniform1ui;
00868 extern PFNGLPROGRAMUNIFORM1UIVEXTPROC glProgramUniform1uivEXT;
00869 extern PFNGLPROGRAMUNIFORM1UIVPROC glProgramUniform1uiv;
00870 extern PFNGLPROGRAMUNIFORM2DEXTPROC glProgramUniform2dEXT;
00871 extern PFNGLPROGRAMUNIFORM2DPROC glProgramUniform2d;
00872 extern PFNGLPROGRAMUNIFORM2DVEXTPROC glProgramUniform2dvEXT;
00873 extern PFNGLPROGRAMUNIFORM2DVPROC glProgramUniform2dv;
00874 extern PFNGLPROGRAMUNIFORM2FEXTPROC glProgramUniform2fEXT;
00875 extern PFNGLPROGRAMUNIFORM2FPROC glProgramUniform2f;
00876 extern PFNGLPROGRAMUNIFORM2FVEXTPROC glProgramUniform2fvEXT;
00877 extern PFNGLPROGRAMUNIFORM2FVPROC glProgramUniform2fv;
00878 extern PFNGLPROGRAMUNIFORM2I64ARBPROC glProgramUniform2i64ARB;
00879 extern PFNGLPROGRAMUNIFORM2I64NVPROC glProgramUniform2i64NV;
00880 extern PFNGLPROGRAMUNIFORM2I64VARBPROC glProgramUniform2i64vARB;
00881 extern PFNGLPROGRAMUNIFORM2I64VNVPROC glProgramUniform2i64vNV;
00882 extern PFNGLPROGRAMUNIFORM2IEXTPROC glProgramUniform2iEXT;
00883 extern PFNGLPROGRAMUNIFORM2IPROC glProgramUniform2i;
00884 extern PFNGLPROGRAMUNIFORM2IVEXTPROC glProgramUniform2ivEXT;
00885 extern PFNGLPROGRAMUNIFORM2IVPROC glProgramUniform2iv;
00886 extern PFNGLPROGRAMUNIFORM2UI64ARBPROC glProgramUniform2ui64ARB;
00887 extern PFNGLPROGRAMUNIFORM2UI64NVPROC glProgramUniform2ui64NV;
00888 extern PFNGLPROGRAMUNIFORM2UI64VARBPROC glProgramUniform2ui64vARB;
00889 extern PFNGLPROGRAMUNIFORM2UI64VNVPROC glProgramUniform2ui64vNV;
00890 extern PFNGLPROGRAMUNIFORM2UIEXTPROC glProgramUniform2uiEXT;
00891 extern PFNGLPROGRAMUNIFORM2UIPROC glProgramUniform2ui;
00892 extern PFNGLPROGRAMUNIFORM2UIVEXTPROC glProgramUniform2uivEXT;
00893 extern PFNGLPROGRAMUNIFORM2UIVPROC glProgramUniform2uiv;
00894 extern PFNGLPROGRAMUNIFORM3DEXTPROC glProgramUniform3dEXT;
00895 extern PFNGLPROGRAMUNIFORM3DPROC glProgramUniform3d;
00896 extern PFNGLPROGRAMUNIFORM3DVEXTPROC glProgramUniform3dvEXT;
00897 extern PFNGLPROGRAMUNIFORM3DVPROC glProgramUniform3dv;
00898 extern PFNGLPROGRAMUNIFORM3FEXTPROC glProgramUniform3fEXT;
00899 extern PFNGLPROGRAMUNIFORM3FPROC glProgramUniform3f;
00900 extern PFNGLPROGRAMUNIFORM3FVEXTPROC glProgramUniform3fvEXT;
00901 extern PFNGLPROGRAMUNIFORM3FVPROC glProgramUniform3fv;
00902 extern PFNGLPROGRAMUNIFORM3I64ARBPROC glProgramUniform3i64ARB;
00903 extern PFNGLPROGRAMUNIFORM3I64NVPROC glProgramUniform3i64NV;
00904 extern PFNGLPROGRAMUNIFORM3I64VARBPROC glProgramUniform3i64vARB;
00905 extern PFNGLPROGRAMUNIFORM3I64VNVPROC glProgramUniform3i64vNV;
00906 extern PFNGLPROGRAMUNIFORM3IEXTPROC glProgramUniform3iEXT;
00907 extern PFNGLPROGRAMUNIFORM3IPROC glProgramUniform3i;
00908 extern PFNGLPROGRAMUNIFORM3IVEXTPROC glProgramUniform3ivEXT;
00909 extern PFNGLPROGRAMUNIFORM3IVPROC glProgramUniform3iv;
00910 extern PFNGLPROGRAMUNIFORM3UI64ARBPROC glProgramUniform3ui64ARB;
00911 extern PFNGLPROGRAMUNIFORM3UI64NVPROC glProgramUniform3ui64NV;
00912 extern PFNGLPROGRAMUNIFORM3UI64VARBPROC glProgramUniform3ui64vARB;
00913 extern PFNGLPROGRAMUNIFORM3UI64VNVPROC glProgramUniform3ui64vNV;
00914 extern PFNGLPROGRAMUNIFORM3UIEXTPROC glProgramUniform3uiEXT;
00915 extern PFNGLPROGRAMUNIFORM3UIPROC glProgramUniform3ui;
00916 extern PFNGLPROGRAMUNIFORM3UIVEXTPROC glProgramUniform3uivEXT;
00917 extern PFNGLPROGRAMUNIFORM3UIVPROC glProgramUniform3uiv;
00918 extern PFNGLPROGRAMUNIFORM4DEXTPROC glProgramUniform4dEXT;
00919 extern PFNGLPROGRAMUNIFORM4DPROC glProgramUniform4d;
00920 extern PFNGLPROGRAMUNIFORM4DVEXTPROC glProgramUniform4dvEXT;
00921 extern PFNGLPROGRAMUNIFORM4DVPROC glProgramUniform4dv;
00922 extern PFNGLPROGRAMUNIFORM4FEXTPROC glProgramUniform4fEXT;
00923 extern PFNGLPROGRAMUNIFORM4FPROC glProgramUniform4f;
00924 extern PFNGLPROGRAMUNIFORM4FVEXTPROC glProgramUniform4fvEXT;
00925 extern PFNGLPROGRAMUNIFORM4FVPROC glProgramUniform4fv;
00926 extern PFNGLPROGRAMUNIFORM4I64ARBPROC glProgramUniform4i64ARB;
00927 extern PFNGLPROGRAMUNIFORM4I64NVPROC glProgramUniform4i64NV;
00928 extern PFNGLPROGRAMUNIFORM4I64VARBPROC glProgramUniform4i64vARB;
00929 extern PFNGLPROGRAMUNIFORM4I64VNVPROC glProgramUniform4i64vNV;
00930 extern PFNGLPROGRAMUNIFORM4IEXTPROC glProgramUniform4iEXT;
00931 extern PFNGLPROGRAMUNIFORM4IPROC glProgramUniform4i;
00932 extern PFNGLPROGRAMUNIFORM4IVEXTPROC glProgramUniform4ivEXT;
00933 extern PFNGLPROGRAMUNIFORM4IVPROC glProgramUniform4iv;
00934 extern PFNGLPROGRAMUNIFORM4UI64ARBPROC glProgramUniform4ui64ARB;
00935 extern PFNGLPROGRAMUNIFORM4UI64NVPROC glProgramUniform4ui64NV;
00936 extern PFNGLPROGRAMUNIFORM4UI64VARBPROC glProgramUniform4ui64vARB;
00937 extern PFNGLPROGRAMUNIFORM4UI64VNVPROC glProgramUniform4ui64vNV;
00938 extern PFNGLPROGRAMUNIFORM4UIEXTPROC glProgramUniform4uiEXT;
```

```
00939 extern PFNGLPROGRAMUNIFORM4UIPROC glProgramUniform4ui;
00940 extern PFNGLPROGRAMUNIFORM4UIVPROC glProgramUniform4uivEXT;
00941 extern PFNGLPROGRAMUNIFORM4UIVPROC glProgramUniform4uiv;
00942 extern PFNGLPROGRAMUNIFORMHANDLEUI64ARBPROC glProgramUniformHandleui64ARB;
00943 extern PFNGLPROGRAMUNIFORMHANDLEUI64NVPROC glProgramUniformHandleui64NV;
00944 extern PFNGLPROGRAMUNIFORMHANDLEUI64VARBPROC glProgramUniformHandleui64vARB;
00945 extern PFNGLPROGRAMUNIFORMHANDLEUI64VNVPROC glProgramUniformHandleui64vNV;
00946 extern PFNGLPROGRAMUNIFORMMATRIX2DVEXTPROC glProgramUniformMatrix2dvEXT;
00947 extern PFNGLPROGRAMUNIFORMMATRIX2DVPROC glProgramUniformMatrix2dv;
00948 extern PFNGLPROGRAMUNIFORMMATRIX2FVEXTPROC glProgramUniformMatrix2fvEXT;
00949 extern PFNGLPROGRAMUNIFORMMATRIX2FVPROC glProgramUniformMatrix2fv;
00950 extern PFNGLPROGRAMUNIFORMMATRIX2X3DVEXTPROC glProgramUniformMatrix2x3dvEXT;
00951 extern PFNGLPROGRAMUNIFORMMATRIX2X3DVPROC glProgramUniformMatrix2x3dv;
00952 extern PFNGLPROGRAMUNIFORMMATRIX2X3FVEXTPROC glProgramUniformMatrix2x3fvEXT;
00953 extern PFNGLPROGRAMUNIFORMMATRIX2X3FVPROC glProgramUniformMatrix2x3fv;
00954 extern PFNGLPROGRAMUNIFORMMATRIX2X4DVEXTPROC glProgramUniformMatrix2x4dvEXT;
00955 extern PFNGLPROGRAMUNIFORMMATRIX2X4DVPROC glProgramUniformMatrix2x4dv;
00956 extern PFNGLPROGRAMUNIFORMMATRIX2X4FVEXTPROC glProgramUniformMatrix2x4fvEXT;
00957 extern PFNGLPROGRAMUNIFORMMATRIX2X4FVPROC glProgramUniformMatrix2x4fv;
00958 extern PFNGLPROGRAMUNIFORMMATRIX3DVEXTPROC glProgramUniformMatrix3dvEXT;
00959 extern PFNGLPROGRAMUNIFORMMATRIX3DVPROC glProgramUniformMatrix3dv;
00960 extern PFNGLPROGRAMUNIFORMMATRIX3FVEXTPROC glProgramUniformMatrix3fvEXT;
00961 extern PFNGLPROGRAMUNIFORMMATRIX3FVPROC glProgramUniformMatrix3fv;
00962 extern PFNGLPROGRAMUNIFORMMATRIX3X2DVEXTPROC glProgramUniformMatrix3x2dvEXT;
00963 extern PFNGLPROGRAMUNIFORMMATRIX3X2DVPROC glProgramUniformMatrix3x2dv;
00964 extern PFNGLPROGRAMUNIFORMMATRIX3X2FVEXTPROC glProgramUniformMatrix3x2fvEXT;
00965 extern PFNGLPROGRAMUNIFORMMATRIX3X2FVPROC glProgramUniformMatrix3x2fv;
00966 extern PFNGLPROGRAMUNIFORMMATRIX3X4DVEXTPROC glProgramUniformMatrix3x4dvEXT;
00967 extern PFNGLPROGRAMUNIFORMMATRIX3X4DVPROC glProgramUniformMatrix3x4dv;
00968 extern PFNGLPROGRAMUNIFORMMATRIX3X4FVEXTPROC glProgramUniformMatrix3x4fvEXT;
00969 extern PFNGLPROGRAMUNIFORMMATRIX3X4FVPROC glProgramUniformMatrix3x4fv;
00970 extern PFNGLPROGRAMUNIFORMMATRIX4DVEXTPROC glProgramUniformMatrix4dvEXT;
00971 extern PFNGLPROGRAMUNIFORMMATRIX4DVPROC glProgramUniformMatrix4dv;
00972 extern PFNGLPROGRAMUNIFORMMATRIX4FVEXTPROC glProgramUniformMatrix4fvEXT;
00973 extern PFNGLPROGRAMUNIFORMMATRIX4FVPROC glProgramUniformMatrix4fv;
00974 extern PFNGLPROGRAMUNIFORMMATRIX4X2DVEXTPROC glProgramUniformMatrix4x2dvEXT;
00975 extern PFNGLPROGRAMUNIFORMMATRIX4X2DVPROC glProgramUniformMatrix4x2dv;
00976 extern PFNGLPROGRAMUNIFORMMATRIX4X2FVEXTPROC glProgramUniformMatrix4x2fvEXT;
00977 extern PFNGLPROGRAMUNIFORMMATRIX4X2FVPROC glProgramUniformMatrix4x2fv;
00978 extern PFNGLPROGRAMUNIFORMMATRIX4X3DVEXTPROC glProgramUniformMatrix4x3dvEXT;
00979 extern PFNGLPROGRAMUNIFORMMATRIX4X3DVPROC glProgramUniformMatrix4x3dv;
00980 extern PFNGLPROGRAMUNIFORMMATRIX4X3FVEXTPROC glProgramUniformMatrix4x3fvEXT;
00981 extern PFNGLPROGRAMUNIFORMMATRIX4X3FVPROC glProgramUniformMatrix4x3fv;
00982 extern PFNGLPROGRAMUNIFORMUI64NVPROC glProgramUniformui64NV;
00983 extern PFNGLPROGRAMUNIFORMUI64VNVPROC glProgramUniformui64vNV;
00984 extern PFNGLPROVOKINGVERTEXPROC glProvokingVertex;
00985 extern PFNGLPUSHCLIENTATTRIBDEFAULTTEXTPROC glPushClientAttribDefaultEXT;
00986 extern PFNGLPUSHDEBUGGROUPPROC glPushDebugGroup;
00987 extern PFNGLPUSHGROUPMARKEREXTPROC glPushGroupMarkerEXT;
00988 extern PFNGLQUERYCOUNTERPROC glQueryCounter;
00989 extern PFNGLRASTERSAMPLESEXTPROC glRasterSamplesEXT;
00990 extern PFNGLREADBUFFERPROC glReadBuffer;
00991 extern PFNGLREADNPIXELSARBPROC glReadnPixelsARB;
00992 extern PFNGLREADNPIXELSPROC glReadnPixels;
00993 extern PFNGLREADPIXELSPROC glReadPixels;
00994 extern PFNGLRELEASESHADERCOMPILERPROC glReleaseShaderCompiler;
00995 extern PFNGLRENDERBUFFERSTORAGEMULTISAMPLECOVERAGENVPROC glRenderbufferStorageMultisampleCoverageNV;
00996 extern PFNGLRENDERBUFFERSTORAGEMULTISAMPLEPROC glRenderbufferStorageMultisample;
00997 extern PFNGLRENDERBUFFERSTORAGEPROC glRenderbufferStorage;
00998 extern PFNGLRESOLVEDEPTHVALUESNVPROC glResolveDepthValuesNV;
00999 extern PFNGLRESUMETRANSFORMFEEDBACKPROC glResumeTransformFeedback;
01000 extern PFNGLSAMPLECOVERAGEPROC glSampleCoverage;
01001 extern PFNGLSAMPLEMASKIPROC glSampleMaski;
01002 extern PFNGLSAMPLERPARAMETERFPROC glSamplerParameterf;
01003 extern PFNGLSAMPLERPARAMETERFVPROC glSamplerParameterfv;
01004 extern PFNGLSAMPLERPARAMETERIIVPROC glSamplerParameterIiv;
01005 extern PFNGLSAMPLERPARAMETERIPROC glSamplerParameteri;
01006 extern PFNGLSAMPLERPARAMETERIUIVPROC glSamplerParameterIuiv;
01007 extern PFNGLSAMPLERPARAMETERIVPROC glSamplerParameteriv;
01008 extern PFNGLSCISSORARRAYVPROC glScissorArrayv;
01009 extern PFNGLSCISSORINDEXEDPROC glScissorIndexed;
01010 extern PFNGLSCISSORINDEXEDVPROC glScissorIndexedv;
01011 extern PFNGLSCISSORPROC glScissor;
01012 extern PFNGLSECONDARYCOLORFORMATNVPROC glSecondaryColorFormatNV;
01013 extern PFNGLSELECTPERFMONITORCOUNTERSAMDPROC glSelectPerfMonitorCountersAMD;
01014 extern PFNGLSHADERBINARYPROC glShaderBinary;
01015 extern PFNGLSHADERSOURCEPROC glShaderSource;
01016 extern PFNGLSHADERSTORAGEBLOCKBINDINGPROC glShaderStorageBlockBinding;
01017 extern PFNGLSIGNALVKFENCENVPROC glSignalVkFenceNV;
01018 extern PFNGLSIGNALVKSEMAPHORENVPROC glSignalVkSemaphoreNV;
01019 extern PFNGLSPECIALIZESHADERARBPROC glSpecializeShaderARB;
01020 extern PFNGLSTATECAPTURENVPROC glStateCaptureNV;
01021 extern PFNGLSTENCILFILLPATHINSTANCEDNVPROC glStencilFillPathInstancedNV;
01022 extern PFNGLSTENCILFILLPATHNVPROC glStencilFillPathNV;
01023 extern PFNGLSTENCILFUNCPROC glStencilFunc;
01024 extern PFNGLSTENCILFUNCSEPARATEPROC glStencilFuncSeparate;
01025 extern PFNGLSTENCILMASKPROC glStencilMask;
```

```

01026 extern PFNGLSTENCILMASKSEPARATEPROC glStencilMaskSeparate;
01027 extern PFNGLSTENCILOPPROC glStencilOp;
01028 extern PFNGLSTENCILOPSEPARATEPROC glStencilOpSeparate;
01029 extern PFNGLSTENCILSTROKEPATHINSTANCEDNVPROC glStencilStrokePathInstancedNV;
01030 extern PFNGLSTENCILSTROKEPATHNVPROC glStencilStrokePathNV;
01031 extern PFNGLSTENCILTHENCOVERFILLPATHINSTANCEDNVPROC glStencilThenCoverFillPathInstancedNV;
01032 extern PFNGLSTENCILTHENCOVERFILLPATHNVPROC glStencilThenCoverFillPathNV;
01033 extern PFNGLSTENCILTHENCOVERSTROKEPATHINSTANCEDNVPROC glStencilThenCoverStrokePathInstancedNV;
01034 extern PFNGLSTENCILTHENCOVERSTROKEPATHNVPROC glStencilThenCoverStrokePathNV;
01035 extern PFNGLSUBPIXELPRECISIONBIASNVPROC glSubpixelPrecisionBiasNV;
01036 extern PFNGLTEXBUFFERARBPROC glTexBufferARB;
01037 extern PFNGLTEXBUFFERPROC glTexBuffer;
01038 extern PFNGLTEXBUFFERRANGEPROC glTexBufferRange;
01039 extern PFNGLTEXCOORDFORMATNVPROC glTexCoordFormatNV;
01040 extern PFNGLTEXIMAGE1DPROC glTexImage1D;
01041 extern PFNGLTEXIMAGE2DMULTISAMPLEPROC glTexImage2DMultisample;
01042 extern PFNGLTEXIMAGE2DPROC glTexImage2D;
01043 extern PFNGLTEXIMAGE3DMULTISAMPLEPROC glTexImage3DMultisample;
01044 extern PFNGLTEXIMAGE3DPROC glTexImage3D;
01045 extern PFNGLTEXPAGECOMMITMENTARBPROC glTexPageCommitmentARB;
01046 extern PFNGLTEXPARAMETERFPROC glTexParameterf;
01047 extern PFNGLTEXPARAMETERFVPROC glTexParameterfv;
01048 extern PFNGLTEXPARAMETERIIVPROC glTexParameterIiv;
01049 extern PFNGLTEXPARAMETERIPROC glTexParameteri;
01050 extern PFNGLTEXPARAMETERIUIVPROC glTexParameterIuiv;
01051 extern PFNGLTEXPARAMETERIVPROC glTexParameteriv;
01052 extern PFNGLTEXSTORAGE1DPROC glTexStorage1D;
01053 extern PFNGLTEXSTORAGE2DMULTISAMPLEPROC glTexStorage2DMultisample;
01054 extern PFNGLTEXSTORAGE2DPROC glTexStorage2D;
01055 extern PFNGLTEXSTORAGE3DMULTISAMPLEPROC glTexStorage3DMultisample;
01056 extern PFNGLTEXSTORAGE3DPROC glTexStorage3D;
01057 extern PFNGLTEXSUBIMAGE1DPROC glTexSubImage1D;
01058 extern PFNGLTEXSUBIMAGE2DPROC glTexSubImage2D;
01059 extern PFNGLTEXSUBIMAGE3DPROC glTexSubImage3D;
01060 extern PFNGLTEXTUREBARRIERNVPROC glTextureBarrierNV;
01061 extern PFNGLTEXTUREBARRIERPROC glTextureBarrier;
01062 extern PFNGLTEXTUREBUFFEREXTPROC glTextureBufferEXT;
01063 extern PFNGLTEXTUREBUFFERPROC glTextureBuffer;
01064 extern PFNGLTEXTUREBUFFERRANGEEXTPROC glTextureBufferRangeEXT;
01065 extern PFNGLTEXTUREBUFFERRANGEPROC glTextureBufferRange;
01066 extern PFNGLTEXTUREIMAGE1DEXTPROC glTextureImage1DEXT;
01067 extern PFNGLTEXTUREIMAGE2DEXTPROC glTextureImage2DEXT;
01068 extern PFNGLTEXTUREIMAGE3DEXTPROC glTextureImage3DEXT;
01069 extern PFNGLTEXTUREPAGECOMMITMENTEXTPROC glTexturePageCommitmentEXT;
01070 extern PFNGLTEXTUREPARAMETERFEXTPROC glTextureParameterfEXT;
01071 extern PFNGLTEXTUREPARAMETERFPROC glTextureParameterf;
01072 extern PFNGLTEXTUREPARAMETERFVEXTPROC glTextureParameterfvEXT;
01073 extern PFNGLTEXTUREPARAMETERFVPROC glTextureParameterfv;
01074 extern PFNGLTEXTUREPARAMETERIEXTPROC glTextureParameteriEXT;
01075 extern PFNGLTEXTUREPARAMETERIIVEXTPROC glTextureParameterIivEXT;
01076 extern PFNGLTEXTUREPARAMETERIIVPROC glTextureParameterIiv;
01077 extern PFNGLTEXTUREPARAMETERIPROC glTextureParameteri;
01078 extern PFNGLTEXTUREPARAMETERIUIVEXTPROC glTextureParameterIuivEXT;
01079 extern PFNGLTEXTUREPARAMETERIUIVPROC glTextureParameterIuiv;
01080 extern PFNGLTEXTUREPARAMETERIVEXTPROC glTextureParameterivEXT;
01081 extern PFNGLTEXTUREPARAMETERIVPROC glTextureParameteriv;
01082 extern PFNGLTEXTURERENDERBUFFEREXTPROC glTextureRenderbufferEXT;
01083 extern PFNGLTEXTURESTORAGE1DEXTPROC glTextureStorage1DEXT;
01084 extern PFNGLTEXTURESTORAGE1DPROC glTextureStorage1D;
01085 extern PFNGLTEXTURESTORAGE2DEXTPROC glTextureStorage2DEXT;
01086 extern PFNGLTEXTURESTORAGE2DMULTISAMPLEEXTPROC glTextureStorage2DMultisampleEXT;
01087 extern PFNGLTEXTURESTORAGE2DMULTISAMPLEPROC glTextureStorage2DMultisample;
01088 extern PFNGLTEXTURESTORAGE2DPROC glTextureStorage2D;
01089 extern PFNGLTEXTURESTORAGE3DEXTPROC glTextureStorage3DEXT;
01090 extern PFNGLTEXTURESTORAGE3DMULTISAMPLEEXTPROC glTextureStorage3DMultisampleEXT;
01091 extern PFNGLTEXTURESTORAGE3DMULTISAMPLEPROC glTextureStorage3DMultisample;
01092 extern PFNGLTEXTURESTORAGE3DPROC glTextureStorage3D;
01093 extern PFNGLTEXTURESUBIMAGE1DEXTPROC glTextureSubImage1DEXT;
01094 extern PFNGLTEXTURESUBIMAGE1DPROC glTextureSubImage1D;
01095 extern PFNGLTEXTURESUBIMAGE2DEXTPROC glTextureSubImage2DEXT;
01096 extern PFNGLTEXTURESUBIMAGE2DPROC glTextureSubImage2D;
01097 extern PFNGLTEXTURESUBIMAGE3DEXTPROC glTextureSubImage3DEXT;
01098 extern PFNGLTEXTURESUBIMAGE3DPROC glTextureSubImage3D;
01099 extern PFNGLTEXTUREVIEWPROC glTextureView;
01100 extern PFNGLTRANSFORMFEEDBACKBUFFERBASEPROC glTransformFeedbackBufferBase;
01101 extern PFNGLTRANSFORMFEEDBACKBUFFERRANGEPROC glTransformFeedbackBufferRange;
01102 extern PFNGLTRANSFORMFEEDBACKVARYINGSPROC glTransformFeedbackVaryings;
01103 extern PFNGLTRANSFORMPATHNVPROC glTransformPathNV;
01104 extern PFNGLUNIFORM1DPROC glUniform1d;
01105 extern PFNGLUNIFORM1DVPROC glUniform1dv;
01106 extern PFNGLUNIFORM1FPROC glUniform1f;
01107 extern PFNGLUNIFORM1FVPROC glUniform1fv;
01108 extern PFNGLUNIFORM1I64ARBPROC glUniform1i64ARB;
01109 extern PFNGLUNIFORM1I64NVPROC glUniform1i64NV;
01110 extern PFNGLUNIFORM1I64VARBPROC glUniform1i64vARB;
01111 extern PFNGLUNIFORM1I64VNVPROC glUniform1i64vNV;
01112 extern PFNGLUNIFORM1IPROC glUniform1i;

```



```

01113 extern PFNGLUNIFORM1IVPROC glUniform1iv;
01114 extern PFNGLUNIFORM1UI64ARBPROC glUniform1ui64ARB;
01115 extern PFNGLUNIFORM1UI64NVPROC glUniform1ui64NV;
01116 extern PFNGLUNIFORM1UI64VARBPROC glUniform1ui64vARB;
01117 extern PFNGLUNIFORM1UI64VNVPROC glUniform1ui64vNV;
01118 extern PFNGLUNIFORM1UIPROC glUniform1ui;
01119 extern PFNGLUNIFORM1UIVPROC glUniform1uiv;
01120 extern PFNGLUNIFORM2DPROC glUniform2d;
01121 extern PFNGLUNIFORM2DVPROC glUniform2dv;
01122 extern PFNGLUNIFORM2FPROC glUniform2f;
01123 extern PFNGLUNIFORM2FVPROC glUniform2fv;
01124 extern PFNGLUNIFORM2I64ARBPROC glUniform2i64ARB;
01125 extern PFNGLUNIFORM2I64NVPROC glUniform2i64NV;
01126 extern PFNGLUNIFORM2I64VARBPROC glUniform2i64vARB;
01127 extern PFNGLUNIFORM2I64VNVPROC glUniform2i64vNV;
01128 extern PFNGLUNIFORM2IPROC glUniform2i;
01129 extern PFNGLUNIFORM2IVPROC glUniform2iv;
01130 extern PFNGLUNIFORM2UI64ARBPROC glUniform2ui64ARB;
01131 extern PFNGLUNIFORM2UI64NVPROC glUniform2ui64NV;
01132 extern PFNGLUNIFORM2UI64VARBPROC glUniform2ui64vARB;
01133 extern PFNGLUNIFORM2UI64VNVPROC glUniform2ui64vNV;
01134 extern PFNGLUNIFORM2UIPROC glUniform2ui;
01135 extern PFNGLUNIFORM2UIVPROC glUniform2uiv;
01136 extern PFNGLUNIFORM3DPROC glUniform3d;
01137 extern PFNGLUNIFORM3DVPROC glUniform3dv;
01138 extern PFNGLUNIFORM3FPROC glUniform3f;
01139 extern PFNGLUNIFORM3FVPROC glUniform3fv;
01140 extern PFNGLUNIFORM3I64ARBPROC glUniform3i64ARB;
01141 extern PFNGLUNIFORM3I64NVPROC glUniform3i64NV;
01142 extern PFNGLUNIFORM3I64VARBPROC glUniform3i64vARB;
01143 extern PFNGLUNIFORM3I64VNVPROC glUniform3i64vNV;
01144 extern PFNGLUNIFORM3IPROC glUniform3i;
01145 extern PFNGLUNIFORM3IVPROC glUniform3iv;
01146 extern PFNGLUNIFORM3UI64ARBPROC glUniform3ui64ARB;
01147 extern PFNGLUNIFORM3UI64NVPROC glUniform3ui64NV;
01148 extern PFNGLUNIFORM3UI64VARBPROC glUniform3ui64vARB;
01149 extern PFNGLUNIFORM3UI64VNVPROC glUniform3ui64vNV;
01150 extern PFNGLUNIFORM3UIPROC glUniform3ui;
01151 extern PFNGLUNIFORM3UIVPROC glUniform3uiv;
01152 extern PFNGLUNIFORM4DPROC glUniform4d;
01153 extern PFNGLUNIFORM4DVPROC glUniform4dv;
01154 extern PFNGLUNIFORM4FPROC glUniform4f;
01155 extern PFNGLUNIFORM4FVPROC glUniform4fv;
01156 extern PFNGLUNIFORM4I64ARBPROC glUniform4i64ARB;
01157 extern PFNGLUNIFORM4I64NVPROC glUniform4i64NV;
01158 extern PFNGLUNIFORM4I64VARBPROC glUniform4i64vARB;
01159 extern PFNGLUNIFORM4I64VNVPROC glUniform4i64vNV;
01160 extern PFNGLUNIFORM4IPROC glUniform4i;
01161 extern PFNGLUNIFORM4IVPROC glUniform4iv;
01162 extern PFNGLUNIFORM4UI64ARBPROC glUniform4ui64ARB;
01163 extern PFNGLUNIFORM4UI64NVPROC glUniform4ui64NV;
01164 extern PFNGLUNIFORM4UI64VARBPROC glUniform4ui64vARB;
01165 extern PFNGLUNIFORM4UI64VNVPROC glUniform4ui64vNV;
01166 extern PFNGLUNIFORM4UIPROC glUniform4ui;
01167 extern PFNGLUNIFORM4UIVPROC glUniform4uiv;
01168 extern PFNGLUNIFORMBLOCKBINDINGPROC glUniformBlockBinding;
01169 extern PFNGLUNIFORMHANDLEUI64ARBPROC glUniformHandleui64ARB;
01170 extern PFNGLUNIFORMHANDLEUI64NVPROC glUniformHandleui64NV;
01171 extern PFNGLUNIFORMHANDLEUI64VARBPROC glUniformHandleui64vARB;
01172 extern PFNGLUNIFORMHANDLEUI64VNVPROC glUniformHandleui64vNV;
01173 extern PFNGLUNIFORMMATRIX2DVPROC glUniformMatrix2dv;
01174 extern PFNGLUNIFORMMATRIX2FVPROC glUniformMatrix2fv;
01175 extern PFNGLUNIFORMMATRIX2X3DVPROC glUniformMatrix2x3dv;
01176 extern PFNGLUNIFORMMATRIX2X3FVPROC glUniformMatrix2x3fv;
01177 extern PFNGLUNIFORMMATRIX2X4DVPROC glUniformMatrix2x4dv;
01178 extern PFNGLUNIFORMMATRIX2X4FVPROC glUniformMatrix2x4fv;
01179 extern PFNGLUNIFORMMATRIX3DVPROC glUniformMatrix3dv;
01180 extern PFNGLUNIFORMMATRIX3FVPROC glUniformMatrix3fv;
01181 extern PFNGLUNIFORMMATRIX3X2DVPROC glUniformMatrix3x2dv;
01182 extern PFNGLUNIFORMMATRIX3X2FVPROC glUniformMatrix3x2fv;
01183 extern PFNGLUNIFORMMATRIX3X4DVPROC glUniformMatrix3x4dv;
01184 extern PFNGLUNIFORMMATRIX3X4FVPROC glUniformMatrix3x4fv;
01185 extern PFNGLUNIFORMMATRIX4DVPROC glUniformMatrix4dv;
01186 extern PFNGLUNIFORMMATRIX4FVPROC glUniformMatrix4fv;
01187 extern PFNGLUNIFORMMATRIX4X2DVPROC glUniformMatrix4x2dv;
01188 extern PFNGLUNIFORMMATRIX4X2FVPROC glUniformMatrix4x2fv;
01189 extern PFNGLUNIFORMMATRIX4X3DVPROC glUniformMatrix4x3dv;
01190 extern PFNGLUNIFORMMATRIX4X3FVPROC glUniformMatrix4x3fv;
01191 extern PFNGLUNIFORMSUBROUTINESUIVPROC glUniformSubroutinesuiv;
01192 extern PFNGLUNIFORMUI64NVPROC glUniformui64NV;
01193 extern PFNGLUNIFORMUI64VNVPROC glUniformui64vNV;
01194 extern PFNGLUNMAPBUFFERPROC glUnmapBuffer;
01195 extern PFNGLUNMAPNAMEDBUFFEREXTPROC glUnmapNamedBufferEXT;
01196 extern PFNGLUNMAPNAMEDBUFFERPROC glUnmapNamedBuffer;
01197 extern PFNGLUSEPROGRAMPROC glUseProgram;
01198 extern PFNGLUSEPROGRAMSTAGESPROC glUseProgramStages;
01199 extern PFNGLUSESHADERPROGRAMEXTPROC glUseProgramEXT;

```

```

01200 extern PFNGLVALIDATEPROGRAMPIPELINEPROC glValidateProgramPipeline;
01201 extern PFNGLVALIDATEPROGRAMPROC glValidateProgram;
01202 extern PFNGLVERTEXARRAYATTRIBBINDINGPROC glVertexArrayAttribBinding;
01203 extern PFNGLVERTEXARRAYATTRIBFORMATPROC glVertexArrayAttribFormat;
01204 extern PFNGLVERTEXARRAYATTRIBIFORMATPROC glVertexArrayAttribIFormat;
01205 extern PFNGLVERTEXARRAYATTRIBLFORMATPROC glVertexArrayAttribLFormat;
01206 extern PFNGLVERTEXARRAYBINDINGDIVISORPROC glVertexArrayBindingDivisor;
01207 extern PFNGLVERTEXARRAYBINDVERTEXBUFFEREXTPROC glVertexArrayBindVertexBufferEXT;
01208 extern PFNGLVERTEXARRAYCOLOROFFSETEXTPROC glVertexArrayColorOffsetEXT;
01209 extern PFNGLVERTEXARRAYEDGEFLAGOFFSETEXTPROC glVertexArrayEdgeFlagOffsetEXT;
01210 extern PFNGLVERTEXARRAYELEMENTBUFFERPROC glVertexArrayElementBuffer;
01211 extern PFNGLVERTEXARRAYFOGCOORDOFFSETEXTPROC glVertexArrayFogCoordOffsetEXT;
01212 extern PFNGLVERTEXARRAYINDEXOFFSETEXTPROC glVertexArrayIndexOffsetEXT;
01213 extern PFNGLVERTEXARRAYMULTITEXCOORDOFFSETEXTPROC glVertexArrayMultiTexCoordOffsetEXT;
01214 extern PFNGLVERTEXARRAYNORMALOFFSETEXTPROC glVertexArrayNormalOffsetEXT;
01215 extern PFNGLVERTEXARRAYSECONDARYCOLOROFFSETEXTPROC glVertexArraySecondaryColorOffsetEXT;
01216 extern PFNGLVERTEXARRAYTEXCOORDOFFSETEXTPROC glVertexArrayTexCoordOffsetEXT;
01217 extern PFNGLVERTEXARRAYVERTEXATTRIBBINDINGEXTPROC glVertexArrayVertexAttribBindingEXT;
01218 extern PFNGLVERTEXARRAYVERTEXATTRIBDIVISOREXTPROC glVertexArrayVertexAttribDivisorEXT;
01219 extern PFNGLVERTEXARRAYVERTEXATTRIBFORMATEXTPROC glVertexArrayVertexAttribFormatEXT;
01220 extern PFNGLVERTEXARRAYVERTEXATTRIBIFORMATEXTPROC glVertexArrayVertexAttribIFormatEXT;
01221 extern PFNGLVERTEXARRAYVERTEXATTRIBIOFFSETEXTPROC glVertexArrayVertexAttribIOffsetEXT;
01222 extern PFNGLVERTEXARRAYVERTEXATTRIBLFORMATEXTPROC glVertexArrayVertexAttribLFormatEXT;
01223 extern PFNGLVERTEXARRAYVERTEXATTRIBLOFFSETEXTPROC glVertexArrayVertexAttribLOffsetEXT;
01224 extern PFNGLVERTEXARRAYVERTEXATTRIBOFFSETEXTPROC glVertexArrayVertexAttribOffsetEXT;
01225 extern PFNGLVERTEXARRAYVERTEXBINDINGDIVISOREXTPROC glVertexArrayVertexBindingDivisorEXT;
01226 extern PFNGLVERTEXARRAYVERTEXBUFFERPROC glVertexArrayVertexBuffer;
01227 extern PFNGLVERTEXARRAYVERTEXBUFFERSPROC glVertexArrayVertexBuffers;
01228 extern PFNGLVERTEXARRAYVERTEXOFFSETEXTPROC glVertexArrayVertexOffsetEXT;
01229 extern PFNGLVERTEXATTRIB1DPROC glVertexAttrib1d;
01230 extern PFNGLVERTEXATTRIB1DVPROC glVertexAttrib1dv;
01231 extern PFNGLVERTEXATTRIB1FPROC glVertexAttrib1f;
01232 extern PFNGLVERTEXATTRIB1FVPROC glVertexAttrib1fv;
01233 extern PFNGLVERTEXATTRIB1SPROC glVertexAttrib1s;
01234 extern PFNGLVERTEXATTRIB1SVPROC glVertexAttrib1sv;
01235 extern PFNGLVERTEXATTRIB2DPROC glVertexAttrib2d;
01236 extern PFNGLVERTEXATTRIB2DVPROC glVertexAttrib2dv;
01237 extern PFNGLVERTEXATTRIB2FPROC glVertexAttrib2f;
01238 extern PFNGLVERTEXATTRIB2FVPROC glVertexAttrib2fv;
01239 extern PFNGLVERTEXATTRIB2SPROC glVertexAttrib2s;
01240 extern PFNGLVERTEXATTRIB2SVPROC glVertexAttrib2sv;
01241 extern PFNGLVERTEXATTRIB3DPROC glVertexAttrib3d;
01242 extern PFNGLVERTEXATTRIB3DVPROC glVertexAttrib3dv;
01243 extern PFNGLVERTEXATTRIB3FPROC glVertexAttrib3f;
01244 extern PFNGLVERTEXATTRIB3FVPROC glVertexAttrib3fv;
01245 extern PFNGLVERTEXATTRIB3SPROC glVertexAttrib3s;
01246 extern PFNGLVERTEXATTRIB3SVPROC glVertexAttrib3sv;
01247 extern PFNGLVERTEXATTRIB4BVPROC glVertexAttrib4bv;
01248 extern PFNGLVERTEXATTRIB4DPROC glVertexAttrib4d;
01249 extern PFNGLVERTEXATTRIB4DVPROC glVertexAttrib4dv;
01250 extern PFNGLVERTEXATTRIB4FPROC glVertexAttrib4f;
01251 extern PFNGLVERTEXATTRIB4FVPROC glVertexAttrib4fv;
01252 extern PFNGLVERTEXATTRIB4IVPROC glVertexAttrib4iv;
01253 extern PFNGLVERTEXATTRIB4NBVPROC glVertexAttrib4Nbv;
01254 extern PFNGLVERTEXATTRIB4NIVPROC glVertexAttrib4Niv;
01255 extern PFNGLVERTEXATTRIB4NSVPROC glVertexAttrib4Nsv;
01256 extern PFNGLVERTEXATTRIB4NUBPROC glVertexAttrib4Nub;
01257 extern PFNGLVERTEXATTRIB4NUBVPROC glVertexAttrib4Nubv;
01258 extern PFNGLVERTEXATTRIB4NUIVPROC glVertexAttrib4Nuiv;
01259 extern PFNGLVERTEXATTRIB4NUSVPROC glVertexAttrib4Nusv;
01260 extern PFNGLVERTEXATTRIB4SPROC glVertexAttrib4s;
01261 extern PFNGLVERTEXATTRIB4SVPROC glVertexAttrib4sv;
01262 extern PFNGLVERTEXATTRIB4UBVPROC glVertexAttrib4ubv;
01263 extern PFNGLVERTEXATTRIB4UIVPROC glVertexAttrib4uiv;
01264 extern PFNGLVERTEXATTRIB4USVPROC glVertexAttrib4usv;
01265 extern PFNGLVERTEXATTRIBBINDINGPROC glVertexAttribBinding;
01266 extern PFNGLVERTEXATTRIBDIVISORARBPROC glVertexAttribDivisorARB;
01267 extern PFNGLVERTEXATTRIBDIVISORPROC glVertexAttribDivisor;
01268 extern PFNGLVERTEXATTRIBFORMATNVPROC glVertexAttribFormatNV;
01269 extern PFNGLVERTEXATTRIBFORMATPROC glVertexAttribFormat;
01270 extern PFNGLVERTEXATTRIBI1IPROC glVertexAttribI1i;
01271 extern PFNGLVERTEXATTRIBI1IVPROC glVertexAttribI1iv;
01272 extern PFNGLVERTEXATTRIBI1UIPROC glVertexAttribI1ui;
01273 extern PFNGLVERTEXATTRIBI1UIVPROC glVertexAttribI1uiv;
01274 extern PFNGLVERTEXATTRIBI2IPROC glVertexAttribI2i;
01275 extern PFNGLVERTEXATTRIBI2IVPROC glVertexAttribI2iv;
01276 extern PFNGLVERTEXATTRIBI2UIPROC glVertexAttribI2ui;
01277 extern PFNGLVERTEXATTRIBI2UIVPROC glVertexAttribI2uiv;
01278 extern PFNGLVERTEXATTRIBI3IPROC glVertexAttribI3i;
01279 extern PFNGLVERTEXATTRIBI3IVPROC glVertexAttribI3iv;
01280 extern PFNGLVERTEXATTRIBI3UIPROC glVertexAttribI3ui;
01281 extern PFNGLVERTEXATTRIBI3UIVPROC glVertexAttribI3uiv;
01282 extern PFNGLVERTEXATTRIBI4BVPROC glVertexAttribI4bv;
01283 extern PFNGLVERTEXATTRIBI4IPROC glVertexAttribI4i;
01284 extern PFNGLVERTEXATTRIBI4IVPROC glVertexAttribI4iv;
01285 extern PFNGLVERTEXATTRIBI4SVPROC glVertexAttribI4sv;
01286 extern PFNGLVERTEXATTRIBI4UBVPROC glVertexAttribI4ubv;

```

```

01287 extern PFNGLVERTEXATTRIBI4UIPROC glVertexAttribI4ui;
01288 extern PFNGLVERTEXATTRIBI4UIVPROC glVertexAttribI4uiv;
01289 extern PFNGLVERTEXATTRIBI4USVPROC glVertexAttribI4usv;
01290 extern PFNGLVERTEXATTRIBIFORMATNVPROC glVertexAttribIFormatNV;
01291 extern PFNGLVERTEXATTRIBIFORMATPROC glVertexAttribIFormat;
01292 extern PFNGLVERTEXATTRIBIPOINTERPROC glVertexAttribIPointer;
01293 extern PFNGLVERTEXATTRIBL1DPROC glVertexAttribL1d;
01294 extern PFNGLVERTEXATTRIBL1DVPROC glVertexAttribL1dv;
01295 extern PFNGLVERTEXATTRIBL1I64NVPROC glVertexAttribL1i64NV;
01296 extern PFNGLVERTEXATTRIBL1I64VNVPROC glVertexAttribL1i64vNV;
01297 extern PFNGLVERTEXATTRIBL1UI64ARBPROC glVertexAttribL1ui64ARB;
01298 extern PFNGLVERTEXATTRIBL1UI64NVPROC glVertexAttribL1ui64NV;
01299 extern PFNGLVERTEXATTRIBL1UI64VARBPROC glVertexAttribL1ui64vARB;
01300 extern PFNGLVERTEXATTRIBL1UI64VNVPROC glVertexAttribL1ui64vNV;
01301 extern PFNGLVERTEXATTRIBL2DPROC glVertexAttribL2d;
01302 extern PFNGLVERTEXATTRIBL2DVPROC glVertexAttribL2dv;
01303 extern PFNGLVERTEXATTRIBL2I64NVPROC glVertexAttribL2i64NV;
01304 extern PFNGLVERTEXATTRIBL2I64VNVPROC glVertexAttribL2i64vNV;
01305 extern PFNGLVERTEXATTRIBL2UI64NVPROC glVertexAttribL2ui64NV;
01306 extern PFNGLVERTEXATTRIBL2UI64VNVPROC glVertexAttribL2ui64vNV;
01307 extern PFNGLVERTEXATTRIBL3DPROC glVertexAttribL3d;
01308 extern PFNGLVERTEXATTRIBL3DVPROC glVertexAttribL3dv;
01309 extern PFNGLVERTEXATTRIBL3I64NVPROC glVertexAttribL3i64NV;
01310 extern PFNGLVERTEXATTRIBL3I64VNVPROC glVertexAttribL3i64vNV;
01311 extern PFNGLVERTEXATTRIBL3UI64NVPROC glVertexAttribL3ui64NV;
01312 extern PFNGLVERTEXATTRIBL3UI64VNVPROC glVertexAttribL3ui64vNV;
01313 extern PFNGLVERTEXATTRIBL4DPROC glVertexAttribL4d;
01314 extern PFNGLVERTEXATTRIBL4DVPROC glVertexAttribL4dv;
01315 extern PFNGLVERTEXATTRIBL4I64NVPROC glVertexAttribL4i64NV;
01316 extern PFNGLVERTEXATTRIBL4I64VNVPROC glVertexAttribL4i64vNV;
01317 extern PFNGLVERTEXATTRIBL4UI64NVPROC glVertexAttribL4ui64NV;
01318 extern PFNGLVERTEXATTRIBL4UI64VNVPROC glVertexAttribL4ui64vNV;
01319 extern PFNGLVERTEXATTRIBLFORMATNVPROC glVertexAttribLFormatNV;
01320 extern PFNGLVERTEXATTRIBLFORMATPROC glVertexAttribLFormat;
01321 extern PFNGLVERTEXATTRIBLPOINTERPROC glVertexAttribLPointer;
01322 extern PFNGLVERTEXATTRIBP1UIPROC glVertexAttribP1ui;
01323 extern PFNGLVERTEXATTRIBP1UIVPROC glVertexAttribP1uiv;
01324 extern PFNGLVERTEXATTRIBP2UIPROC glVertexAttribP2ui;
01325 extern PFNGLVERTEXATTRIBP2UIVPROC glVertexAttribP2uiv;
01326 extern PFNGLVERTEXATTRIBP3UIPROC glVertexAttribP3ui;
01327 extern PFNGLVERTEXATTRIBP3UIVPROC glVertexAttribP3uiv;
01328 extern PFNGLVERTEXATTRIBP4UIPROC glVertexAttribP4ui;
01329 extern PFNGLVERTEXATTRIBP4UIVPROC glVertexAttribP4uiv;
01330 extern PFNGLVERTEXATTRIBPOINTERPROC glVertexAttribPointer;
01331 extern PFNGLVERTEXBINDINGDIVISORPROC glVertexBindingDivisor;
01332 extern PFNGLVERTEXFORMATNVPROC glVertexFormatNV;
01333 extern PFNGLVIEWPORTARRAYVPROC glViewportArrayv;
01334 extern PFNGLVIEWPORTINDEXEDFPROC glViewportIndexdf;
01335 extern PFNGLVIEWPORTINDEXEDFVPROC glViewportIndexdfv;
01336 extern PFNGLVIEWPORTPOSITIONWSCALENVPROC glViewportPositionWScaleNV;
01337 extern PFNGLVIEWPORTPROC glViewport;
01338 extern PFNGLVIEWPORTSWIZZLENVPROC glViewportSwizzleNV;
01339 extern PFNGLWAITSYNCPROC glWaitSync;
01340 extern PFNGLWAITVKSEMAPHORENVPROC glWaitVkSemaphoreNV;
01341 extern PFNGLWEIGHTPATHSNVPROC glWeightPathsNV;
01342 extern PFNGLWINDOWRECTANGLESEXTPROC glWindowRectanglesEXT;
01343 #endif
01344
01345
01346
01350 namespace gg
01351 {
01352     enum BindingPoints
01353     {
01354         LightBindingPoint = 0,
01355         MaterialBindingPoint,
01356     };
01357
01364     extern GLint ggBufferAlignment;
01365
01372     extern void ggInit();
01373
01383     extern void _ggError(const std::string& name = "", unsigned int line = 0);
01384
01395     #if defined(DEBUG)
01396     #   define ggError() gg::_ggError(__FILE__, __LINE__)
01397     #else
01398     #   define ggError()
01399     #endif
01400
01410     extern void _ggFBOError(const std::string& name = "", unsigned int line = 0);
01411
01422     #if defined(DEBUG)
01423     #   define ggFBOError() gg::_ggFBOError(__FILE__, __LINE__)
01424     #else
01425     #   define ggFBOError()
01426     #endif
01427

```

```

01435 inline void ggCross(GLfloat* c, const GLfloat* a, const GLfloat* b)
01436 {
01437     c[0] = a[1] * b[2] - a[2] * b[1];
01438     c[1] = a[2] * b[0] - a[0] * b[2];
01439     c[2] = a[0] * b[1] - a[1] * b[0];
01440 }
01441
01449 inline GLfloat ggDot3(const GLfloat* a, const GLfloat* b)
01450 {
01451     return a[0] * b[0] + a[1] * b[1] + a[2] * b[2];
01452 }
01453
01460 inline GLfloat ggLength3(const GLfloat* a)
01461 {
01462     return sqrtf(ggDot3(a, a));
01463 }
01464
01472 inline GLfloat ggDistance3(const GLfloat* a, const GLfloat* b)
01473 {
01474     const GLfloat c[] { a[0] - b[0], a[1] - b[1], a[2] - b[2], 0.0f };
01475     return ggLength3(c);
01476 }
01477
01484 inline void ggNormalize3(const GLfloat* a, GLfloat* b)
01485 {
01486     const GLfloat l { ggLength3(a) };
01487     assert(l > 0.0f);
01488     b[0] = a[0] / l;
01489     b[1] = a[1] / l;
01490     b[2] = a[2] / l;
01491 }
01492
01498 inline void ggNormalize3(GLfloat* a)
01499 {
01500     const GLfloat l { ggLength3(a) };
01501     assert(l > 0.0f);
01502     a[0] /= l;
01503     a[1] /= l;
01504     a[2] /= l;
01505 }
01506
01514 inline GLfloat ggDot4(const GLfloat* a, const GLfloat* b)
01515 {
01516     return a[0] * b[0] + a[1] * b[1] + a[2] * b[2] + a[3] * b[3];
01517 }
01518
01525 inline GLfloat ggLength4(const GLfloat* a)
01526 {
01527     return sqrtf(ggDot4(a, a));
01528 }
01529
01537 inline GLfloat ggDistance4(const GLfloat* a, const GLfloat* b)
01538 {
01539     const GLfloat c[] { a[0] - b[0], a[1] - b[1], a[2] - b[2], a[3] - b[3] };
01540     return ggLength4(c);
01541 }
01542
01549 inline void ggNormalize4(const GLfloat* a, GLfloat* b)
01550 {
01551     const GLfloat l { ggLength4(a) };
01552     assert(l > 0.0f);
01553     b[0] = a[0] / l;
01554     b[1] = a[1] / l;
01555     b[2] = a[2] / l;
01556     b[3] = a[3] / l;
01557 }
01558
01564 inline void ggNormalize4(GLfloat* a)
01565 {
01566     const GLfloat l { ggLength4(a) };
01567     assert(l > 0.0f);
01568     a[0] /= l;
01569     a[1] /= l;
01570     a[2] /= l;
01571     a[3] /= l;
01572 }
01573
01577 class GgVector : public std::array<GLfloat, 4>
01578 {
01579 public:
01580
01584     GgVector() = default;
01585
01594     constexpr GgVector(GLfloat v0, GLfloat v1, GLfloat v2, GLfloat v3) :
01595         std::array<GLfloat, 4>{ v0, v1, v2, v3 }
01596     {
01597     }

```

```

01598
01604     constexpr GgVector(const GLfloat* a) :
01605         GgVector{ a[0], a[1], a[2], a[3] }
01606     {
01607     }
01608
01614     constexpr GgVector(GLfloat c) :
01615         GgVector{ c, c, c, c }
01616     {
01617     }
01618
01624     GgVector(const GgVector& v) = default;
01625
01631     GgVector(GgVector&& v) = default;
01632
01636     ~GgVector() = default;
01637
01641     GgVector& operator=(const GgVector& v) = default;
01642
01646     GgVector& operator=(GgVector&& v) = default;
01647
01654     constexpr GgVector operator+(const GgVector& v) const noexcept
01655     {
01656         return GgVector{ (*this)[0] + v[0], (*this)[1] + v[1], (*this)[2] + v[2], (*this)[3] + v[3] };
01657     }
01658
01665     constexpr GgVector operator+(GLfloat c) const noexcept
01666     {
01667         return GgVector{ (*this)[0] + c, (*this)[1] + c, (*this)[2] + c, (*this)[3] + c };
01668     }
01669
01676     constexpr GgVector& operator+=(const GgVector& v) noexcept
01677     {
01678         (*this)[0] += v[0];
01679         (*this)[1] += v[1];
01680         (*this)[2] += v[2];
01681         (*this)[3] += v[3];
01682         return *this;
01683     }
01684
01691     constexpr GgVector& operator+=(GLfloat c) noexcept
01692     {
01693         (*this)[0] += c;
01694         (*this)[1] += c;
01695         (*this)[2] += c;
01696         (*this)[3] += c;
01697         return *this;
01698     }
01699
01706     constexpr GgVector operator-(const GgVector& v) const noexcept
01707     {
01708         return GgVector{ (*this)[0] - v[0], (*this)[1] - v[1], (*this)[2] - v[2], (*this)[3] - v[3] };
01709     }
01710
01717     constexpr GgVector operator-(GLfloat c) const noexcept
01718     {
01719         return GgVector{ (*this)[0] - c, (*this)[1] - c, (*this)[2] - c, (*this)[3] - c };
01720     }
01721
01728     constexpr GgVector& operator-=(const GgVector& v) noexcept
01729     {
01730         (*this)[0] -= v[0];
01731         (*this)[1] -= v[1];
01732         (*this)[2] -= v[2];
01733         (*this)[3] -= v[3];
01734         return *this;
01735     }
01736
01743     constexpr GgVector& operator-=(GLfloat c) noexcept
01744     {
01745         (*this)[0] -= c;
01746         (*this)[1] -= c;
01747         (*this)[2] -= c;
01748         (*this)[3] -= c;
01749         return *this;
01750     }
01751
01758     constexpr GgVector operator*(const GgVector& v) const noexcept
01759     {
01760         return GgVector{ (*this)[0] * v[0], (*this)[1] * v[1], (*this)[2] * v[2], (*this)[3] * v[3] };
01761     }
01762
01769     constexpr GgVector operator*(GLfloat c) const noexcept
01770     {
01771         return GgVector{ (*this)[0] * c, (*this)[1] * c, (*this)[2] * c, (*this)[3] * c };
01772     }
01773

```

```

01780 constexpr GgVector& operator*=(const GgVector& v) noexcept
01781 {
01782     (*this)[0] *= v[0];
01783     (*this)[1] *= v[1];
01784     (*this)[2] *= v[2];
01785     (*this)[3] *= v[3];
01786     return *this;
01787 }
01788
01795 constexpr GgVector& operator*=(GLfloat c) noexcept
01796 {
01797     (*this)[0] *= c;
01798     (*this)[1] *= c;
01799     (*this)[2] *= c;
01800     (*this)[3] *= c;
01801     return *this;
01802 }
01803
01810 constexpr GgVector operator/(const GgVector& v) const noexcept
01811 {
01812     return GgVector{ (*this)[0] / v[0], (*this)[1] / v[1], (*this)[2] / v[2], (*this)[3] / v[3] };
01813 }
01814
01821 constexpr GgVector operator/(GLfloat c) const noexcept
01822 {
01823     return GgVector{ (*this)[0] / c, (*this)[1] / c, (*this)[2] / c, (*this)[3] / c };
01824 }
01825
01832 constexpr GgVector& operator/=(const GgVector& v) noexcept
01833 {
01834     (*this)[0] /= v[0];
01835     (*this)[1] /= v[1];
01836     (*this)[2] /= v[2];
01837     (*this)[3] /= v[3];
01838     return *this;
01839 }
01840
01847 constexpr GgVector& operator/=(GLfloat c) noexcept
01848 {
01849     (*this)[0] /= c;
01850     (*this)[1] /= c;
01851     (*this)[2] /= c;
01852     (*this)[3] /= c;
01853     return *this;
01854 }
01855
01862 inline GLfloat dot3(const GgVector& v) const
01863 {
01864     return ggDot3(data(), v.data());
01865 }
01866
01872 inline GLfloat length3() const
01873 {
01874     return ggLength3(data());
01875 }
01876
01883 inline GLfloat distance3(const GgVector& v) const
01884 {
01885     return ggDistance3(data(), v.data());
01886 }
01887
01893 inline GgVector normalize3() const
01894 {
01895     GgVector b;
01896     ggNormalize3(data(), b.data());
01897     return b;
01898 }
01899
01906 inline GLfloat dot4(const GgVector& v) const
01907 {
01908     return ggDot4(data(), v.data());
01909 }
01910
01916 inline GLfloat length4() const
01917 {
01918     return ggLength4(data());
01919 }
01920
01927 inline GLfloat distance4(const GgVector& v) const
01928 {
01929     return ggDistance4(data(), v.data());
01930 }
01931
01937 inline GgVector normalize4() const
01938 {
01939     GgVector b;
01940     ggNormalize4(data(), b.data());

```

```

01941     return b;
01942 }
01943 };
01944
01951 inline const GgVector& operator+(const GgVector& v)
01952 {
01953     return v;
01954 }
01955
01963 inline GgVector operator+(GLfloat a, const GgVector& b)
01964 {
01965     return GgVector{ a + b[0], a + b[1], a + b[2], a + b[3] };
01966 }
01967
01974 inline const GgVector operator-(const GgVector& v)
01975 {
01976     return GgVector{ -v[0], -v[1], -v[2], -v[3] };
01977 }
01978
01986 inline GgVector operator-(GLfloat a, const GgVector& b)
01987 {
01988     return GgVector{ a - b[0], a - b[1], a - b[2], a - b[3] };
01989 }
01990
01998 inline GgVector operator*(GLfloat a, const GgVector& b)
01999 {
02000     return GgVector{ a * b[0], a * b[1], a * b[2], a * b[3] };
02001 }
02002
02010 inline GgVector operator/(GLfloat a, const GgVector& b)
02011 {
02012     return GgVector{ a / b[0], a / b[1], a / b[2], a / b[3] };
02013 }
02014
02025 inline GgVector ggCross(const GgVector& a, const GgVector& b)
02026 {
02027     GgVector c;
02028     ggCross(c.data(), a.data(), b.data());
02029     return c;
02030 }
02031
02039 inline GLfloat ggDot3(const GgVector& a, const GgVector& b)
02040 {
02041     return ggDot3(a.data(), b.data());
02042 }
02043
02050 inline GLfloat ggLength3(const GgVector& a)
02051 {
02052     return ggLength3(a.data());
02053 }
02054
02062 inline GLfloat ggDistance3(const GgVector& a, const GgVector& b)
02063 {
02064     return ggDistance3(a.data(), b.data());
02065 }
02066
02076 inline GgVector ggNormalize3(const GgVector& a)
02077 {
02078     GgVector b;
02079     ggNormalize3(a.data(), b.data());
02080     return b;
02081 }
02082
02091 inline void ggNormalize3(GgVector* a)
02092 {
02093     ggNormalize3(a->data());
02094 }
02095
02103 inline GLfloat ggDot4(const GgVector& a, const GgVector& b)
02104 {
02105     return ggDot4(a.data(), b.data());
02106 }
02107
02114 inline GLfloat ggLength4(const GgVector& a)
02115 {
02116     return ggLength4(a.data());
02117 }
02118
02126 inline GLfloat ggDistance4(const GgVector& a, const GgVector& b)
02127 {
02128     return ggDistance4(a.data(), b.data());
02129 }
02130
02137 inline GgVector ggNormalize4(const GgVector& a)
02138 {
02139     GgVector b;
02140     ggNormalize4(a.data(), b.data());

```

```

02141     return b;
02142 }
02143
02149 inline void ggNormalize4(GgVector* a)
02150 {
02151     ggNormalize4(a->data());
02152 }
02153
02157 class GgMatrix : public std::array<GLfloat, 16>
02158 {
02159     // 行列 a とベクトル b の積をベクトル c に格納する
02160     void projection(GLfloat* c, const GLfloat* a, const GLfloat* b) const;
02161
02162     // 行列 a と行列 b の積を行列 c に格納する
02163     void multiply(GLfloat* c, const GLfloat* a, const GLfloat* b) const;
02164
02165 public:
02166
02170     GgMatrix() = default;
02171
02192     constexpr GgMatrix(
02193         GLfloat m00, GLfloat m01, GLfloat m02, GLfloat m03,
02194         GLfloat m10, GLfloat m11, GLfloat m12, GLfloat m13,
02195         GLfloat m20, GLfloat m21, GLfloat m22, GLfloat m23,
02196         GLfloat m30, GLfloat m31, GLfloat m32, GLfloat m33
02197     ) :
02198         std::array<GLfloat, 16>{ m00, m01, m02, m03, m10, m11, m12, m13, m20, m21, m22, m23, m30, m31,
02199         m32, m33 }
02200     {
02201     }
02202
02207     constexpr GgMatrix(const GLfloat* a) :
02208         GgMatrix{ a[0], a[1], a[2], a[3], a[4], a[5], a[6], a[7], a[8], a[9], a[10], a[11], a[12],
02209         a[13], a[14], a[15] }
02210     {
02211     }
02217     constexpr GgMatrix(GLfloat c) :
02218         GgMatrix{ c, c, c, c, c, c, c, c, c, c, c, c, c, c, c, c }
02219     {
02220     }
02221
02227     GgMatrix(const GgMatrix& m) = default;
02228
02234     GgMatrix(GgMatrix&& m) = default;
02235
02239     ~GgMatrix() = default;
02240
02244     GgMatrix& operator=(const GgMatrix& m) = default;
02245
02249     GgMatrix& operator=(GgMatrix&& m) = default;
02250
02257     GgMatrix& operator=(const GLfloat* a)
02258     {
02259         *this = GgMatrix(a);
02260         return *this;
02261     }
02262
02269     GgMatrix operator+(const GLfloat* a) const
02270     {
02271         GgMatrix t;
02272         for (std::size_t i = 0; i < size(); ++i) t[i] = data()[i] + a[i];
02273         return t;
02274     }
02275
02282     GgMatrix operator+(const GgMatrix& m) const
02283     {
02284         return operator+(m.data());
02285     }
02286
02293     GgMatrix& operator+=(const GLfloat* a)
02294     {
02295         for (std::size_t i = 0; i < size(); ++i) data()[i] += a[i];
02296         return *this;
02297     }
02298
02305     GgMatrix& operator+=(const GgMatrix& m)
02306     {
02307         return operator+=(m.data());
02308     }
02309
02316     GgMatrix operator-(const GLfloat* a) const
02317     {
02318         GgMatrix t;
02319         for (std::size_t i = 0; i < size(); ++i) t[i] = data()[i] - a[i];
02320         return t;
02321     }

```



```

02322
02329 GgMatrix operator-(const GgMatrix& m) const
02330 {
02331     return operator-(m.data());
02332 }
02333
02340 GgMatrix& operator--(const GLfloat* a)
02341 {
02342     for (std::size_t i = 0; i < size(); ++i) data()[i] -= a[i];
02343     return *this;
02344 }
02345
02352 GgMatrix& operator--(const GgMatrix& m)
02353 {
02354     return operator--(m.data());
02355 }
02356
02363 GgMatrix operator*(const GLfloat* a) const
02364 {
02365     GgMatrix t;
02366     multiply(t.data(), data(), a);
02367     return t;
02368 }
02369
02376 GgMatrix operator*(const GgMatrix& m) const
02377 {
02378     return operator*(m.data());
02379 }
02380
02387 GgMatrix& operator*=(const GLfloat* a)
02388 {
02389     *this = operator*(a);
02390     return *this;
02391 }
02392
02399 GgMatrix& operator*=(const GgMatrix& m)
02400 {
02401     return operator*=(m.data());
02402 }
02403
02410 GgMatrix operator/(const GLfloat* a) const
02411 {
02412     GgMatrix i, t;
02413     i.loadInvert(a);
02414     multiply(t.data(), data(), i.data());
02415     return t;
02416 }
02417
02424 GgMatrix operator/(const GgMatrix& m) const
02425 {
02426     return operator/(m.data());
02427 }
02428
02435 GgMatrix& operator/=(const GLfloat* a)
02436 {
02437     *this = operator/(a);
02438     return *this;
02439 }
02440
02447 GgMatrix& operator/=(const GgMatrix& m)
02448 {
02449     return operator/=(m.data());
02450 }
02451
02455 GgMatrix& loadIdentity();
02456
02466 GgMatrix& loadTranslate(GLfloat x, GLfloat y, GLfloat z, GLfloat w = 1.0f);
02467
02474 GgMatrix& loadTranslate(const GLfloat* t)
02475 {
02476     return loadTranslate(t[0], t[1], t[2]);
02477 }
02478
02485 GgMatrix& loadTranslate(const GgVector& t)
02486 {
02487     return loadTranslate(t[0], t[1], t[2], t[3]);
02488 }
02489
02499 GgMatrix& loadScale(GLfloat x, GLfloat y, GLfloat z, GLfloat w = 1.0f);
02500
02507 GgMatrix& loadScale(const GLfloat* s)
02508 {
02509     return loadScale(s[0], s[1], s[2]);
02510 }
02511
02518 GgMatrix& loadScale(const GgVector& s)
02519 {

```

```

02520     return loadScale(s[0], s[1], s[2], s[3]);
02521 }
02522
02529 GgMatrix& loadRotateX(GLfloat a);
02530
02537 GgMatrix& loadRotateY(GLfloat a);
02538
02545 GgMatrix& loadRotateZ(GLfloat a);
02546
02556 GgMatrix& loadRotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a);
02557
02565 GgMatrix& loadRotate(const GLfloat* r, GLfloat a)
02566 {
02567     return loadRotate(r[0], r[1], r[2], a);
02568 }
02569
02577 GgMatrix& loadRotate(const GgVector& r, GLfloat a)
02578 {
02579     return loadRotate(r[0], r[1], r[2], a);
02580 }
02581
02588 GgMatrix& loadRotate(const GLfloat* r)
02589 {
02590     return loadRotate(r[0], r[1], r[2], r[3]);
02591 }
02592
02599 GgMatrix& loadRotate(const GgVector& r)
02600 {
02601     return loadRotate(r[0], r[1], r[2], r[3]);
02602 }
02603
02618 GgMatrix& loadLookat(GLfloat ex, GLfloat ey, GLfloat ez,
02619                     GLfloat tx, GLfloat ty, GLfloat tz,
02620                     GLfloat ux, GLfloat uy, GLfloat uz);
02621
02630 GgMatrix& loadLookat(const GLfloat* e, const GLfloat* t, const GLfloat* u)
02631 {
02632     return loadLookat(e[0], e[1], e[2], t[0], t[1], t[2], u[0], u[1], u[2]);
02633 }
02634
02643 GgMatrix& loadLookat(const GgVector& e, const GgVector& t, const GgVector& u)
02644 {
02645     return loadLookat(e[0], e[1], e[2], t[0], t[1], t[2], u[0], u[1], u[2]);
02646 }
02647
02659 GgMatrix& loadOrthogonal(GLfloat left, GLfloat right,
02660                         GLfloat bottom, GLfloat top,
02661                         GLfloat zNear, GLfloat zFar);
02662
02674 GgMatrix& loadFrustum(GLfloat left, GLfloat right,
02675                      GLfloat bottom, GLfloat top,
02676                      GLfloat zNear, GLfloat zFar);
02677
02687 GgMatrix& loadPerspective(GLfloat fovy, GLfloat aspect,
02688                          GLfloat zNear, GLfloat zFar);
02689
02696 GgMatrix& loadTranspose(const GLfloat* a);
02697
02704 GgMatrix& loadTranspose(const GgMatrix& m)
02705 {
02706     return loadTranspose(m.data());
02707 }
02708
02715 GgMatrix& loadInvert(const GLfloat* a);
02716
02723 GgMatrix& loadInvert(const GgMatrix& m)
02724 {
02725     return loadInvert(m.data());
02726 }
02727
02734 GgMatrix& loadNormal(const GLfloat* a);
02735
02742 GgMatrix& loadNormal(const GgMatrix& m)
02743 {
02744     return loadNormal(m.data());
02745 }
02746
02756 GgMatrix translate(GLfloat x, GLfloat y, GLfloat z, GLfloat w = 1.0f) const
02757 {
02758     GgMatrix m;
02759     return operator*(m.loadTranslate(x, y, z, w));
02760 }
02761
02768 GgMatrix translate(const GLfloat* t) const
02769 {
02770     return translate(t[0], t[1], t[2]);
02771 }

```

```

02772
02779 GgMatrix translate(const GgVector& t) const
02780 {
02781     return translate(t[0], t[1], t[2], t[3]);
02782 }
02783
02793 GgMatrix scale(GLfloat x, GLfloat y, GLfloat z, GLfloat w = 1.0f) const
02794 {
02795     GgMatrix m;
02796     return operator*(m.loadScale(x, y, z, w));
02797 }
02798
02805 GgMatrix scale(const GLfloat* s) const
02806 {
02807     return scale(s[0], s[1], s[2]);
02808 }
02809
02816 GgMatrix scale(const GgVector& s) const
02817 {
02818     return scale(s[0], s[1], s[2], s[3]);
02819 }
02820
02827 GgMatrix rotateX(GLfloat a) const
02828 {
02829     GgMatrix m;
02830     return operator*(m.loadRotateX(a));
02831 }
02832
02839 GgMatrix rotateY(GLfloat a) const
02840 {
02841     GgMatrix m;
02842     return operator*(m.loadRotateY(a));
02843 }
02844
02851 GgMatrix rotateZ(GLfloat a) const
02852 {
02853     GgMatrix m;
02854     return operator*(m.loadRotateZ(a));
02855 }
02856
02866 GgMatrix rotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a) const
02867 {
02868     GgMatrix m;
02869     return operator*(m.loadRotate(x, y, z, a));
02870 }
02871
02879 GgMatrix rotate(const GLfloat* r, GLfloat a) const
02880 {
02881     return rotate(r[0], r[1], r[2], a);
02882 }
02883
02891 GgMatrix rotate(const GgVector& r, GLfloat a) const
02892 {
02893     return rotate(r[0], r[1], r[2], a);
02894 }
02895
02902 GgMatrix rotate(const GLfloat* r) const
02903 {
02904     return rotate(r[0], r[1], r[2], r[3]);
02905 }
02906
02913 GgMatrix rotate(const GgVector& r) const
02914 {
02915     return rotate(r[0], r[1], r[2], r[3]);
02916 }
02917
02932 GgMatrix lookat(
02933     GLfloat ex, GLfloat ey, GLfloat ez,
02934     GLfloat tx, GLfloat ty, GLfloat tz,
02935     GLfloat ux, GLfloat uy, GLfloat uz
02936 ) const
02937 {
02938     GgMatrix m;
02939     return operator*(m.loadLookat(ex, ey, ez, tx, ty, tz, ux, uy, uz));
02940 }
02941
02950 GgMatrix lookat(const GLfloat* e, const GLfloat* t, const GLfloat* u) const
02951 {
02952     return lookat(e[0], e[1], e[2], t[0], t[1], t[2], u[0], u[1], u[2]);
02953 }
02954
02963 GgMatrix lookat(const GgVector& e, const GgVector& t, const GgVector& u) const
02964 {
02965     return lookat(e[0], e[1], e[2], t[0], t[1], t[2], u[0], u[1], u[2]);
02966 }
02967
02979 GgMatrix orthogonal(

```

```

02980     GLfloat left, GLfloat right,
02981     GLfloat bottom, GLfloat top,
02982     GLfloat zNear, GLfloat zFar
02983 ) const
02984 {
02985     GgMatrix m;
02986     return operator*(m.loadOrthogonal(left, right, bottom, top, zNear, zFar));
02987 }
02988
03000 GgMatrix frustum(
03001     GLfloat left, GLfloat right,
03002     GLfloat bottom, GLfloat top,
03003     GLfloat zNear, GLfloat zFar
03004 ) const
03005 {
03006     GgMatrix m;
03007     return operator*(m.loadFrustum(left, right, bottom, top, zNear, zFar));
03008 }
03009
03019 GgMatrix perspective(
03020     GLfloat fovy, GLfloat aspect,
03021     GLfloat zNear, GLfloat zFar
03022 ) const
03023 {
03024     GgMatrix m;
03025     return operator*(m.loadPerspective(fovy, aspect, zNear, zFar));
03026 }
03027
03033 GgMatrix transpose() const
03034 {
03035     GgMatrix t;
03036     return t.loadTranspose(*this);
03037 }
03038
03044 GgMatrix invert() const
03045 {
03046     GgMatrix t;
03047     return t.loadInvert(*this);
03048 }
03049
03055 GgMatrix normal() const
03056 {
03057     GgMatrix t;
03058     return t.loadNormal(*this);
03059 }
03060
03067 void projection(GLfloat* c, const GLfloat* v) const
03068 {
03069     projection(c, data(), v);
03070 }
03071
03078 void projection(GLfloat* c, const GgVector& v) const
03079 {
03080     projection(c, v.data());
03081 }
03082
03089 void projection(GgVector& c, const GLfloat* v) const
03090 {
03091     projection(c.data(), v);
03092 }
03093
03100 void projection(GgVector& c, const GgVector& v) const
03101 {
03102     projection(c.data(), v.data());
03103 }
03104
03111 GgVector operator*(const GgVector& v) const
03112 {
03113     GgVector c;
03114     projection(c, v);
03115     return c;
03116 }
03117
03123 const GLfloat* get() const
03124 {
03125     return data();
03126 }
03127
03133 void get(GLfloat* a) const
03134 {
03135     std::copy(data(), data() + size(), a);
03136 }
03137 };
03138
03144 inline GgMatrix ggIdentity()
03145 {
03146     GgMatrix t;

```

```

03147     return t.loadIdentity();
03148 }
03149
03159 inline GgMatrix ggTranslate(GLfloat x, GLfloat y, GLfloat z, GLfloat w = 1.0f)
03160 {
03161     GgMatrix m;
03162     return m.loadTranslate(x, y, z, w);
03163 }
03164
03171 inline GgMatrix ggTranslate(const GLfloat* t)
03172 {
03173     GgMatrix m;
03174     return m.loadTranslate(t[0], t[1], t[2]);
03175 }
03176
03183 inline GgMatrix ggTranslate(const GgVector& t)
03184 {
03185     GgMatrix m;
03186     return m.loadTranslate(t[0], t[1], t[2], t[3]);
03187 }
03188
03198 inline GgMatrix ggScale(GLfloat x, GLfloat y, GLfloat z, GLfloat w = 1.0f)
03199 {
03200     GgMatrix m;
03201     return m.loadScale(x, y, z, w);
03202 }
03203
03210 inline GgMatrix ggScale(const GLfloat* s)
03211 {
03212     GgMatrix m;
03213     return m.loadScale(s[0], s[1], s[2]);
03214 }
03215
03222 inline GgMatrix ggScale(const GgVector& s)
03223 {
03224     GgMatrix m;
03225     return m.loadScale(s[0], s[1], s[2], s[3]);
03226 }
03227
03234 inline GgMatrix ggRotateX(GLfloat a)
03235 {
03236     GgMatrix m;
03237     return m.loadRotateX(a);
03238 }
03239
03246 inline GgMatrix ggRotateY(GLfloat a)
03247 {
03248     GgMatrix m;
03249     return m.loadRotateY(a);
03250 }
03251
03258 inline GgMatrix ggRotateZ(GLfloat a)
03259 {
03260     GgMatrix m;
03261     return m.loadRotateZ(a);
03262 }
03263
03273 inline GgMatrix ggRotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a)
03274 {
03275     GgMatrix m;
03276     return m.loadRotate(x, y, z, a);
03277 }
03278
03286 inline GgMatrix ggRotate(const GLfloat* r, GLfloat a)
03287 {
03288     GgMatrix m;
03289     return m.loadRotate(r[0], r[1], r[2], a);
03290 }
03291
03299 inline GgMatrix ggRotate(const GgVector& r, GLfloat a)
03300 {
03301     GgMatrix m;
03302     return m.loadRotate(r[0], r[1], r[2], a);
03303 }
03304
03311 inline GgMatrix ggRotate(const GLfloat* r)
03312 {
03313     GgMatrix m;
03314     return m.loadRotate(r[0], r[1], r[2], r[3]);
03315 }
03316
03323 inline GgMatrix ggRotate(const GgVector& r)
03324 {
03325     GgMatrix m;
03326     return m.loadRotate(r[0], r[1], r[2], r[3]);
03327 }
03328

```

```

03343 inline GgMatrix ggLookat(
03344     GLfloat ex, GLfloat ey, GLfloat ez,    // 視点の位置
03345     GLfloat tx, GLfloat ty, GLfloat tz,    // 目標点の位置
03346     GLfloat ux, GLfloat uy, GLfloat uz,    // 上方向のベクトル
03347 )
03348 {
03349     GgMatrix m;
03350     return m.loadLookat(ex, ey, ez, tx, ty, tz, ux, uy, uz);
03351 }
03352
03361 inline GgMatrix ggLookat(
03362     const GLfloat* e,    // 視点の位置
03363     const GLfloat* t,    // 目標点の位置
03364     const GLfloat* u,    // 上方向のベクトル
03365 )
03366 {
03367     GgMatrix m;
03368     return m.loadLookat(e[0], e[1], e[2], t[0], t[1], t[2], u[0], u[1], u[2]);
03369 }
03370
03379 inline GgMatrix ggLookat(
03380     const GgVector& e,    // 視点の位置
03381     const GgVector& t,    // 目標点の位置
03382     const GgVector& u,    // 上方向のベクトル
03383 )
03384 {
03385     GgMatrix m;
03386     return m.loadLookat(e[0], e[1], e[2], t[0], t[1], t[2], u[0], u[1], u[2]);
03387 }
03388
03400 inline GgMatrix ggOrthogonal(GLfloat left, GLfloat right,
03401     GLfloat bottom, GLfloat top,
03402     GLfloat zNear, GLfloat zFar)
03403 {
03404     GgMatrix m;
03405     return m.loadOrthogonal(left, right, bottom, top, zNear, zFar);
03406 }
03407
03419 inline GgMatrix ggFrustum(
03420     GLfloat left, GLfloat right,
03421     GLfloat bottom, GLfloat top,
03422     GLfloat zNear, GLfloat zFar
03423 )
03424 {
03425     GgMatrix m;
03426     return m.loadFrustum(left, right, bottom, top, zNear, zFar);
03427 }
03428
03438 inline GgMatrix ggPerspective(
03439     GLfloat fovy, GLfloat aspect,
03440     GLfloat zNear, GLfloat zFar
03441 )
03442 {
03443     GgMatrix m;
03444     return m.loadPerspective(fovy, aspect, zNear, zFar);
03445 }
03446
03453 inline GgMatrix ggTranspose(const GgMatrix& m)
03454 {
03455     return m.transpose();
03456 }
03457
03464 inline GgMatrix ggInvert(const GgMatrix& m)
03465 {
03466     return m.invert();
03467 }
03468
03475 inline GgMatrix ggNormal(const GgMatrix& m)
03476 {
03477     return m.normal();
03478 }
03479
03483 class GgQuaternion : public GgVector
03484 {
03485     // GgQuaternion 型の四元数 p と四元数 q の積を四元数 r に求める
03486     void multiply(GLfloat* r, const GLfloat* p, const GLfloat* q) const;
03487
03488     // GgQuaternion 型の四元数 q が表す回転の変換行列を m に求める
03489     void toMatrix(GLfloat* m, const GLfloat* q) const;
03490
03491     // 回転の変換行列 m が表す四元数を q に求める
03492     void toQuaternion(GLfloat* q, const GLfloat* m) const;
03493
03494     // 球面線形補間 q と r を t で補間した四元数を p に求める
03495     void slerp(GLfloat* p, const GLfloat* q, const GLfloat* r, GLfloat t) const;
03496
03497 public:

```

```

03498
03502     GgQuaternion() = default;
03503
03512     constexpr GgQuaternion(GLfloat x, GLfloat y, GLfloat z, GLfloat w) :
03513         GgVector{ x, y, z, w }
03514     {
03515     }
03516
03522     constexpr GgQuaternion(GLfloat c) :
03523         GgQuaternion{ c, c, c, c }
03524     {
03525     }
03526
03532     GgQuaternion(const GLfloat* a) :
03533         GgQuaternion{ a[0], a[1], a[2], a[3] }
03534     {
03535     }
03536
03542     GgQuaternion(const GgVector& v) :
03543         GgQuaternion{ v[0], v[1], v[2], v[3] }
03544     {
03545     }
03546
03552     GgQuaternion(const GgQuaternion& q) = default;
03553
03559     GgQuaternion(GgQuaternion&& q) = default;
03560
03564     ~GgQuaternion() = default;
03565
03572     GgQuaternion& operator=(const GgQuaternion& q) = default;
03573
03580     GgQuaternion& operator=(GgQuaternion&& q) = default;
03581
03587     GLfloat norm() const
03588     {
03589         return ggLength4(*this);
03590     }
03591
03601     GgQuaternion& loadAdd(GLfloat x, GLfloat y, GLfloat z, GLfloat w)
03602     {
03603         (*this)[0] += x;
03604         (*this)[1] += y;
03605         (*this)[2] += z;
03606         (*this)[3] += w;
03607
03608         return *this;
03609     }
03610
03617     GgQuaternion& loadAdd(const GLfloat* a)
03618     {
03619         return loadAdd(a[0], a[1], a[2], a[3]);
03620     }
03621
03628     GgQuaternion& loadAdd(const GgVector& v)
03629     {
03630         return loadAdd(v[0], v[1], v[2], v[3]);
03631     }
03632
03639     GgQuaternion& loadAdd(const GgQuaternion& q)
03640     {
03641         return loadAdd(q[0], q[1], q[2], q[3]);
03642     }
03643
03653     GgQuaternion& loadSubtract(GLfloat x, GLfloat y, GLfloat z, GLfloat w)
03654     {
03655         (*this)[0] -= x;
03656         (*this)[1] -= y;
03657         (*this)[2] -= z;
03658         (*this)[3] -= w;
03659
03660         return *this;
03661     }
03662
03669     GgQuaternion& loadSubtract(const GLfloat* a)
03670     {
03671         return loadSubtract(a[0], a[1], a[2], a[3]);
03672     }
03673
03680     GgQuaternion& loadSubtract(const GgVector& v)
03681     {
03682         return loadSubtract(v[0], v[1], v[2], v[3]);
03683     }
03684
03691     GgQuaternion& loadSubtract(const GgQuaternion& q)
03692     {
03693         return loadSubtract(q[0], q[1], q[2], q[3]);
03694     }

```

```

03695
03705 GgQuaternion& loadMultiply(GLfloat x, GLfloat y, GLfloat z, GLfloat w)
03706 {
03707     const GLfloat a[] { x, y, z, w };
03708     return loadMultiply(a);
03709 }
03710
03717 GgQuaternion& loadMultiply(const GLfloat* a)
03718 {
03719     return *this = multiply(a);
03720 }
03721
03728 GgQuaternion& loadMultiply(const GgVector& v)
03729 {
03730     return loadMultiply(v.data());
03731 }
03732
03739 GgQuaternion& loadMultiply(const GgQuaternion& q)
03740 {
03741     return loadMultiply(q.data());
03742 }
03743
03753 GgQuaternion& loadDivide(GLfloat x, GLfloat y, GLfloat z, GLfloat w)
03754 {
03755     const GLfloat a[] { x, y, z, w };
03756     return loadDivide(a);
03757 }
03758
03765 GgQuaternion& loadDivide(const GLfloat* a)
03766 {
03767     return *this = divide(a);
03768 }
03769
03776 GgQuaternion& loadDivide(const GgVector& v)
03777 {
03778     return loadDivide(v.data());
03779 }
03780
03787 GgQuaternion& loadDivide(const GgQuaternion& q)
03788 {
03789     return loadDivide(q.data());
03790 }
03791
03801 GgQuaternion add(GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
03802 {
03803     GgQuaternion s { *this };
03804     return s.loadAdd(x, y, z, w);
03805 }
03806
03813 GgQuaternion add(const GLfloat* a) const
03814 {
03815     return add(a[0], a[1], a[2], a[3]);
03816 }
03817
03824 GgQuaternion add(const GgVector& v) const
03825 {
03826     return add(v[0], v[1], v[2], v[3]);
03827 }
03828
03835 GgQuaternion add(const GgQuaternion& q) const
03836 {
03837     return add(q[0], q[1], q[2], q[3]);
03838 }
03839
03849 GgQuaternion subtract(GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
03850 {
03851     GgQuaternion s { *this };
03852     return s.loadSubtract(x, y, z, w);
03853 }
03854
03861 GgQuaternion subtract(const GLfloat* a) const
03862 {
03863     return subtract(a[0], a[1], a[2], a[3]);
03864 }
03865
03872 GgQuaternion subtract(const GgVector& v) const
03873 {
03874     return subtract(v[0], v[1], v[2], v[3]);
03875 }
03876
03883 GgQuaternion subtract(const GgQuaternion& q) const
03884 {
03885     return subtract(q[0], q[1], q[2], q[3]);
03886 }
03887
03897 GgQuaternion multiply(GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
03898 {

```



```

03899     const GLfloat a[] { x, y, z, w };
03900     return multiply(a);
03901 }
03902
03909 GgQuaternion multiply(const GLfloat* a) const
03910 {
03911     GgQuaternion s;
03912     multiply(s.data(), data(), a);
03913     return s;
03914 }
03915
03922 GgQuaternion multiply(const GgVector& v) const
03923 {
03924     return multiply(v.data());
03925 }
03926
03933 GgQuaternion multiply(const GgQuaternion& q) const
03934 {
03935     return multiply(q.data());
03936 }
03937
03947 GgQuaternion divide(GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
03948 {
03949     const GLfloat a[] { x, y, z, w };
03950     return divide(a);
03951 }
03952
03959 GgQuaternion divide(const GLfloat* a) const
03960 {
03961     GgQuaternion s, ia;
03962     ia.loadInvert(a);
03963     multiply(s.data(), data(), ia.data());
03964     return s;
03965 }
03966
03973 GgQuaternion divide(const GgVector& v) const
03974 {
03975     return divide(v.data());
03976 }
03977
03984 GgQuaternion divide(const GgQuaternion& q) const
03985 {
03986     return divide(q.data());
03987 }
03988
03989 // 演算子
03990 GgQuaternion& operator=(const GLfloat* a)
03991 {
03992     return *this = GgQuaternion(a);
03993 }
03994 GgQuaternion& operator=(const GgVector& v)
03995 {
03996     return *this = GgQuaternion(v);
03997 }
03998 GgQuaternion& operator+=(const GLfloat* a)
03999 {
04000     return loadAdd(a);
04001 }
04002 GgQuaternion& operator+=(const GgVector& v)
04003 {
04004     return loadAdd(v);
04005 }
04006 GgQuaternion& operator+=(const GgQuaternion& q)
04007 {
04008     return loadAdd(q);
04009 }
04010 GgQuaternion& operator-=(const GLfloat* a)
04011 {
04012     return loadSubtract(a);
04013 }
04014 GgQuaternion& operator-=(const GgVector& v)
04015 {
04016     return loadSubtract(v);
04017 }
04018 GgQuaternion& operator-=(const GgQuaternion& q)
04019 {
04020     return operator-=(q.data());
04021 }
04022 GgQuaternion& operator*=(const GLfloat* a)
04023 {
04024     return loadMultiply(a);
04025 }
04026 GgQuaternion& operator*=(const GgVector& v)
04027 {
04028     return loadMultiply(v);
04029 }
04030 GgQuaternion& operator*=(const GgQuaternion& q)

```

```

04031     {
04032         return operator*=(q.data());
04033     }
04034     GgQuaternion& operator/=(const GLfloat* a)
04035     {
04036         return loadDivide(a);
04037     }
04038     GgQuaternion& operator/=(const GgVector& v)
04039     {
04040         return loadDivide(v);
04041     }
04042     GgQuaternion& operator/=(const GgQuaternion& q)
04043     {
04044         return operator/=(q.data());
04045     }
04046     GgQuaternion operator+(const GLfloat* a) const
04047     {
04048         return add(a);
04049     }
04050     GgQuaternion operator+(const GgVector& v) const
04051     {
04052         return add(v);
04053     }
04054     GgQuaternion operator+(const GgQuaternion& q) const
04055     {
04056         return operator+(q.data());
04057     }
04058     GgQuaternion operator-(const GLfloat* a) const
04059     {
04060         return subtract(a);
04061     }
04062     GgQuaternion operator-(const GgVector& v) const
04063     {
04064         return subtract(v);
04065     }
04066     GgQuaternion operator-(const GgQuaternion& q) const
04067     {
04068         return operator-(q.data());
04069     }
04070     GgQuaternion operator*(const GLfloat* a) const
04071     {
04072         return multiply(a);
04073     }
04074     GgQuaternion operator*(const GgVector& v) const
04075     {
04076         return multiply(v);
04077     }
04078     GgQuaternion operator*(const GgQuaternion& q) const
04079     {
04080         return operator*(q.data());
04081     }
04082     GgQuaternion operator/(const GLfloat* a) const
04083     {
04084         return divide(a);
04085     }
04086     GgQuaternion operator/(const GgVector& v) const
04087     {
04088         return divide(v);
04089     }
04090     GgQuaternion operator/(const GgQuaternion& q) const
04091     {
04092         return operator/(q.data());
04093     }
04094
04101     GgQuaternion& loadMatrix(const GLfloat* a)
04102     {
04103         toQuaternion(data(), a);
04104         return *this;
04105     }
04106
04113     GgQuaternion& loadMatrix(const GgMatrix& m)
04114     {
04115         return loadMatrix(m.get());
04116     }
04117
04123     GgQuaternion& loadIdentity()
04124     {
04125         return *this = GgQuaternion(0.0f, 0.0f, 0.0f, 1.0f);
04126     }
04127
04137     GgQuaternion& loadRotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a);
04138
04146     GgQuaternion& loadRotate(const GLfloat* v, GLfloat a)
04147     {
04148         return loadRotate(v[0], v[1], v[2], a);
04149     }
04150

```

```

04157     GgQuaternion& loadRotate(const GLfloat* v)
04158     {
04159         return loadRotate(v[0], v[1], v[2], v[3]);
04160     }
04161
04168     GgQuaternion& loadRotateX(GLfloat a);
04169
04176     GgQuaternion& loadRotateY(GLfloat a);
04177
04184     GgQuaternion& loadRotateZ(GLfloat a);
04185
04195     GgQuaternion rotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a) const
04196     {
04197         GgQuaternion q;
04198         return multiply(q.loadRotate(x, y, z, a));
04199     }
04200
04208     GgQuaternion rotate(const GLfloat* v, GLfloat a) const
04209     {
04210         return rotate(v[0], v[1], v[2], a);
04211     }
04212
04219     GgQuaternion rotate(const GLfloat* v) const
04220     {
04221         return rotate(v[0], v[1], v[2], v[3]);
04222     }
04223
04230     GgQuaternion rotateX(GLfloat a) const
04231     {
04232         return rotate(1.0f, 0.0f, 0.0f, a);
04233     }
04234
04241     GgQuaternion rotateY(GLfloat a) const
04242     {
04243         return rotate(0.0f, 1.0f, 0.0f, a);
04244     }
04245
04252     GgQuaternion rotateZ(GLfloat a) const
04253     {
04254         return rotate(0.0f, 0.0f, 1.0f, a);
04255     }
04256
04265     GgQuaternion& loadEuler(GLfloat heading, GLfloat pitch, GLfloat roll);
04266
04273     GgQuaternion& loadEuler(const GLfloat* e)
04274     {
04275         return loadEuler(e[0], e[1], e[2]);
04276     }
04277
04286     GgQuaternion euler(GLfloat heading, GLfloat pitch, GLfloat roll) const
04287     {
04288         GgQuaternion r;
04289         return multiply(r.loadEuler(heading, pitch, roll));
04290     }
04291
04298     GgQuaternion euler(const GLfloat* e) const
04299     {
04300         return euler(e[0], e[1], e[2]);
04301     }
04302
04312     GgQuaternion euler(const GgVector& e) const
04313     {
04314         return euler(e[0], e[1], e[2]);
04315     }
04316
04325     GgQuaternion& loadSlerp(const GLfloat* a, const GLfloat* b, GLfloat t)
04326     {
04327         slerp(data(), a, b, t);
04328         return *this;
04329     }
04330
04339     GgQuaternion& loadSlerp(const GgQuaternion& q, const GgQuaternion& r, GLfloat t)
04340     {
04341         return loadSlerp(q.data(), r.data(), t);
04342     }
04343
04352     GgQuaternion& loadSlerp(const GgQuaternion& q, const GLfloat* a, GLfloat t)
04353     {
04354         return loadSlerp(q.data(), a, t);
04355     }
04356
04365     GgQuaternion& loadSlerp(const GLfloat* a, const GgQuaternion& q, GLfloat t)
04366     {
04367         return loadSlerp(a, q.data(), t);
04368     }
04369
04376     GgQuaternion& loadNormalize(const GLfloat* a);

```

```

04377
04384 GgQuaternion& loadNormalize(const GgQuaternion& q)
04385 {
04386     return loadNormalize(q.data());
04387 }
04388
04395 GgQuaternion& loadConjugate(const GLfloat* a);
04396
04403 GgQuaternion& loadConjugate(const GgQuaternion& q)
04404 {
04405     return loadConjugate(q.data());
04406 }
04407
04414 GgQuaternion& loadInvert(const GLfloat* a);
04415
04422 GgQuaternion& loadInvert(const GgQuaternion& q)
04423 {
04424     return loadInvert(q.data());
04425 }
04426
04434 GgQuaternion slerp(GLfloat* a, GLfloat t) const
04435 {
04436     GgQuaternion p;
04437     slerp(p.data(), data(), a, t);
04438     return p;
04439 }
04440
04448 GgQuaternion slerp(const GgQuaternion& q, GLfloat t) const
04449 {
04450     GgQuaternion p;
04451     slerp(p.data(), data(), q.data(), t);
04452     return p;
04453 }
04454
04460 GgQuaternion normalize() const
04461 {
04462     GgQuaternion q;
04463     q.loadNormalize(data());
04464     return q;
04465 }
04466
04472 GgQuaternion conjugate() const
04473 {
04474     GgQuaternion q;
04475     q.loadConjugate(data());
04476     return q;
04477 }
04478
04484 GgQuaternion invert() const
04485 {
04486     GgQuaternion q;
04487     q.loadInvert(data());
04488     return q;
04489 }
04490
04496 void get(GLfloat* a) const
04497 {
04498     a[0] = data()[0];
04499     a[1] = data()[1];
04500     a[2] = data()[2];
04501     a[3] = data()[3];
04502 }
04503
04509 void getMatrix(GLfloat* a) const
04510 {
04511     toMatrix(a, data());
04512 }
04513
04519 void getMatrix(GgMatrix& m) const
04520 {
04521     getMatrix(m.data());
04522 }
04523
04529 GgMatrix getMatrix() const
04530 {
04531     GgMatrix m;
04532     getMatrix(m);
04533     return m;
04534 }
04535
04541 void getConjugateMatrix(GLfloat* a) const
04542 {
04543     GgQuaternion c;
04544     c.loadConjugate(data());
04545     toMatrix(a, c.data());
04546 }
04547

```

```

04553     void getConjugateMatrix(GgMatrix& m) const
04554     {
04555         getConjugateMatrix(m.data());
04556     }
04557
04563     GgMatrix getConjugateMatrix() const
04564     {
04565         GgMatrix m;
04566         getConjugateMatrix(m);
04567         return m;
04568     }
04569 };
04570
04576     inline GgQuaternion ggIdentityQuaternion()
04577     {
04578         GgQuaternion q;
04579         return q.loadIdentity();
04580     }
04581
04588     inline GgQuaternion ggMatrixQuaternion(const GLfloat* a)
04589     {
04590         GgQuaternion q;
04591         return q.loadMatrix(a);
04592     }
04593
04600     inline GgQuaternion ggMatrixQuaternion(const GgMatrix& m)
04601     {
04602         return ggMatrixQuaternion(m.get());
04603     }
04604
04611     inline GgMatrix ggQuaternionMatrix(const GgQuaternion& q)
04612     {
04613         GLfloat m[16];
04614         q.getMatrix(m);
04615         return GgMatrix{ m };
04616     }
04617
04624     inline GgMatrix ggQuaternionTransposeMatrix(const GgQuaternion& q)
04625     {
04626         GLfloat m[16];
04627         q.getMatrix(m);
04628         GgMatrix t;
04629         return t.loadTranspose(m);
04630     }
04631
04641     inline GgQuaternion ggRotateQuaternion(GLfloat x, GLfloat y, GLfloat z, GLfloat a)
04642     {
04643         GgQuaternion q;
04644         return q.loadRotate(x, y, z, a);
04645     }
04646
04654     inline GgQuaternion ggRotateQuaternion(const GLfloat* v, GLfloat a)
04655     {
04656         return ggRotateQuaternion(v[0], v[1], v[2], a);
04657     }
04658
04665     inline GgQuaternion ggRotateQuaternion(const GLfloat* v)
04666     {
04667         return ggRotateQuaternion(v[0], v[1], v[2], v[3]);
04668     }
04669
04678     inline GgQuaternion ggEulerQuaternion(GLfloat heading, GLfloat pitch, GLfloat roll)
04679     {
04680         GgQuaternion q;
04681         return q.loadEuler(heading, pitch, roll);
04682     }
04683
04690     inline GgQuaternion ggEulerQuaternion(const GLfloat* e)
04691     {
04692         return ggEulerQuaternion(e[0], e[1], e[2]);
04693     }
04694
04701     inline GgQuaternion ggEulerQuaternion(const GgVector& e)
04702     {
04703         return ggEulerQuaternion(e[0], e[1], e[2]);
04704     }
04705
04714     inline GgQuaternion ggSlerp(const GLfloat* a, const GLfloat* b, GLfloat t)
04715     {
04716         GgQuaternion r;
04717         return r.loadSlerp(a, b, t);
04718     }
04719
04728     inline GgQuaternion ggSlerp(const GgQuaternion& q, const GgQuaternion& r, GLfloat t)
04729     {
04730         return ggSlerp(q.data(), r.data(), t);
04731     }

```

```

04732
04741 inline GgQuaternion ggSlerp(const GgQuaternion& q, const GLfloat* a, GLfloat t)
04742 {
04743     return ggSlerp(q.data(), a, t);
04744 }
04745
04754 inline GgQuaternion ggSlerp(const GLfloat* a, const GgQuaternion& q, GLfloat t)
04755 {
04756     return ggSlerp(a, q.data(), t);
04757 }
04758
04765 inline GLfloat ggNorm(const GgQuaternion& q)
04766 {
04767     return q.norm();
04768 }
04769
04776 inline GgQuaternion ggNormalize(const GgQuaternion& q)
04777 {
04778     return q.normalize();
04779 }
04780
04787 inline GgQuaternion ggConjugate(const GgQuaternion& q)
04788 {
04789     return q.conjugate();
04790 }
04791
04798 inline GgQuaternion ggInvert(const GgQuaternion& q)
04799 {
04800     return q.invert();
04801 }
04802
04806 class GgTrackball : public GgQuaternion
04807 {
04808     bool drag;           // ドラッグ中か否か
04809     GLfloat start[2];    // ドラッグ開始位置
04810     GLfloat scale[2];    // マウスの絶対位置→ウィンドウ内での相対位置の換算係数
04811     GgQuaternion cq;     // 回転の初期値 (四元数)
04812     GgMatrix rt;         // 回転の変換行列
04813
04814 public:
04815
04821     GgTrackball(const GgQuaternion& q = ggIdentityQuaternion())
04822     {
04823         reset(q);
04824     }
04825
04829     ~GgTrackball() = default;
04830
04836     GgTrackball& operator=(const GgQuaternion& q)
04837     {
04838         GgQuaternion::operator=(q);
04839         return *this;
04840     }
04841
04851     void region(GLfloat w, GLfloat h);
04852
04862     void region(int w, int h)
04863     {
04864         region(static_cast<GLfloat>(w), static_cast<GLfloat>(h));
04865     }
04866
04876     void begin(GLfloat x, GLfloat y);
04877
04887     void motion(GLfloat x, GLfloat y);
04888
04894     void rotate(const GgQuaternion& q);
04895
04905     void end(GLfloat x, GLfloat y);
04906
04912     void reset(const GgQuaternion& q = ggIdentityQuaternion());
04913
04919     const GLfloat* getStart() const
04920     {
04921         return start;
04922     }
04923
04927     const GLfloat& getStart(int direction) const
04928     {
04929         return start[direction];
04930     }
04931
04937     void getStart(GLfloat* position) const
04938     {
04939         position[0] = start[0];
04940         position[1] = start[1];
04941     }
04942

```

```

04948     const GLfloat* getScale() const
04949     {
04950         return scale;
04951     }
04952
04958     const GLfloat getScale(int direction) const
04959     {
04960         return scale[direction];
04961     }
04962
04968     void getScale(GLfloat* factor) const
04969     {
04970         factor[0] = scale[0];
04971         factor[1] = scale[1];
04972     }
04973
04979     const GgQuaternion& getQuaternion() const
04980     {
04981         return *this;
04982     }
04983
04989     const GgMatrix& getMatrix() const
04990     {
04991         return rt;
04992     }
04993
04999     const GLfloat* get() const
05000     {
05001         return rt.get();
05002     }
05003 };
05004
05015 extern bool ggSaveTga(
05016     const std::string& name,
05017     const void* buffer,
05018     unsigned int width,
05019     unsigned int height,
05020     unsigned int depth
05021 );
05022
05029 extern bool ggSaveColor(const std::string& name);
05030
05037 extern bool ggSaveDepth(const std::string& name);
05038
05049 extern bool ggReadImage(
05050     const std::string& name,
05051     std::vector<GLubyte>& image,
05052     GLsizei* pWidth,
05053     GLsizei* pHeight,
05054     GLenum* pFormat
05055 );
05056
05070 extern GLuint ggLoadTexture(
05071     const GLvoid* image,
05072     GLsizei width,
05073     GLsizei height,
05074     GLenum format = GL_RGB,
05075     GLenum type = GL_UNSIGNED_BYTE,
05076     GLenum internal = GL_RGB,
05077     GLenum wrap = GL_CLAMP_TO_EDGE,
05078     bool swizzle = false
05079 );
05080
05091 extern GLuint ggLoadImage(
05092     const std::string& name,
05093     GLsizei* pWidth = nullptr,
05094     GLsizei* pHeight = nullptr,
05095     GLenum internal = GL_RGBA,
05096     GLenum wrap = GL_CLAMP_TO_EDGE
05097 );
05098
05110 extern void ggCreateNormalMap(
05111     const GLubyte* hmap,
05112     GLsizei width,
05113     GLsizei height,
05114     GLenum format,
05115     GLfloat nz,
05116     GLenum internal,
05117     std::vector<GgVector>& nmap
05118 );
05119
05130 extern GLuint ggLoadHeight(
05131     const std::string& name,
05132     GLfloat nz,
05133     GLsizei* pWidth = nullptr,
05134     GLsizei* pHeight = nullptr,
05135     GLenum internal = GL_RGBA

```

```

05136     );
05137
05151     extern GLuint ggCreateShader(
05152         const std::string& vsrc,
05153         const std::string& fsrc = "",
05154         const std::string& gsrc = "",
05155         GLint nvarying = 0,
05156         const char* const* varyings = nullptr,
05157         const std::string& vtext = "vertex shader",
05158         const std::string& ftext = "fragment shader",
05159         const std::string& gtext = "geometry shader");
05160
05171     extern GLuint ggLoadShader(
05172         const std::string& vert,
05173         const std::string& frag = "",
05174         const std::string& geom = "",
05175         GLint nvarying = 0,
05176         const char* const* varyings = nullptr
05177     );
05178
05187     inline GLuint ggLoadShader(
05188         const std::array<std::string, 3>& files,
05189         GLint nvarying = 0,
05190         const char* const* varyings = nullptr
05191     )
05192     {
05193         return ggLoadShader(files[0], files[1], files[2], nvarying, varyings);
05194     }
05195
05196 #if !defined(__APPLE__)
05204     extern GLuint ggCreateComputeShader(
05205         const std::string& csrc,
05206         const std::string& ctext = "compute shader"
05207     );
05208
05215     extern GLuint ggLoadComputeShader(const std::string& comp);
05216 #endif
05217
05224     class GgTexture
05225     {
05226         // テクスチャ名
05227         GLuint texture;
05228
05229         // テクスチャの縦横の画素数
05230         GLsizei size[2];
05231
05232     public:
05233
05246         GgTexture(
05247             const GLvoid* image,
05248             GLsizei width,
05249             GLsizei height,
05250             GLenum format = GL_RGB,
05251             GLenum type = GL_UNSIGNED_BYTE,
05252             GLenum internal = GL_RGBA,
05253             GLenum wrap = GL_CLAMP_TO_EDGE,
05254             bool swizzle = false
05255         ) :
05256             texture{ ggLoadTexture(image, width, height, format, type, internal, wrap, swizzle) },
05257             size{ width, height }
05258         {
05259         }
05260
05266         GgTexture(const GgTexture& texture) = delete;
05267
05273         GgTexture(GgTexture&& texture) = default;
05274
05278         virtual ~GgTexture()
05279         {
05280             glBindTexture(GL_TEXTURE_2D, 0);
05281             glDeleteTextures(1, &texture);
05282         }
05283
05290         GgTexture& operator=(const GgTexture& texture) = delete;
05291
05298         GgTexture& operator=(GgTexture&& texture) = default;
05299
05303         void bind() const
05304         {
05305             glBindTexture(GL_TEXTURE_2D, texture);
05306         }
05307
05311         void unbind() const
05312         {
05313             glBindTexture(GL_TEXTURE_2D, 0);
05314         }
05315

```



```

05321     void swapRandB(bool swizzle) const;
05322
05328     const GLsizei& getWidth() const
05329     {
05330         return size[0];
05331     }
05332
05338     const GLsizei& getHeight() const
05339     {
05340         return size[1];
05341     }
05342
05348     void getSize(GLsizei* size) const
05349     {
05350         size[0] = getWidth();
05351         size[1] = getHeight();
05352     }
05353
05359     const GLsizei* getSize() const
05360     {
05361         return size;
05362     }
05363
05369     const GLuint& getTexture() const
05370     {
05371         return texture;
05372     }
05373 };
05374
05381 class GgColorTexture
05382 {
05383     // テクスチャ
05384     std::shared_ptr<GgTexture> texture;
05385
05386 public:
05387
05391     GgColorTexture()
05392     {
05393     }
05394
05407     GgColorTexture(
05408         const GLvoid* image,
05409         GLsizei width,
05410         GLsizei height,
05411         GLenum format = GL_RGB,
05412         GLenum type = GL_UNSIGNED_BYTE,
05413         GLenum internal = GL_RGB,
05414         GLenum wrap = GL_CLAMP_TO_EDGE,
05415         bool swizzle = true
05416     )
05417     {
05418         load(image, width, height, format, type, internal, wrap, swizzle);
05419     }
05420
05428     GgColorTexture(
05429         const std::string& name,
05430         GLenum internal = 0,
05431         GLenum wrap = GL_CLAMP_TO_EDGE
05432     )
05433     {
05434         load(name, internal, wrap);
05435     }
05436
05440     virtual ~GgColorTexture()
05441     {
05442     }
05443
05456     void load(
05457         const GLvoid* image,
05458         GLsizei width,
05459         GLsizei height,
05460         GLenum format = GL_RGB,
05461         GLenum type = GL_UNSIGNED_BYTE,
05462         GLenum internal = GL_RGB,
05463         GLenum wrap = GL_CLAMP_TO_EDGE,
05464         bool swizzle = true
05465     )
05466     {
05467         // テクスチャを作成する
05468         texture = std::make_shared<GgTexture>(image, width, height, format, type, internal, wrap,
05469         swizzle);
05470     }
05471
05478     void load(
05479         const std::string& name,
05480         GLenum internal = 0,
05481         GLenum wrap = GL_CLAMP_TO_EDGE

```

```

05482     );
05483
05489     explicit operator bool() const noexcept
05490     {
05491         return texture != nullptr;
05492     }
05493
05499     bool operator!() const noexcept
05500     {
05501         return !static_cast<bool>(*this);
05502     }
05503
05509     const GgTexture* get() const
05510     {
05511         return texture.get();
05512     }
05513
05519     GLuint getTexture() const
05520     {
05521         return texture ? texture->getTexture() : 0;
05522     }
05523
05529     GLsizei getWidth() const
05530     {
05531         return texture ? texture->getWidth() : 0;
05532     }
05533
05539     GLsizei getHeight() const
05540     {
05541         return texture ? texture->getHeight() : 0;
05542     }
05543
05549     void getSize(GLsizei* size) const
05550     {
05551         if (texture) texture->getSize(size);
05552     }
05553
05559     const GLsizei* getSize() const
05560     {
05561         return texture ? texture->getSize() : nullptr;
05562     }
05563
05567     void bind() const
05568     {
05569         if (texture) texture->bind();
05570     }
05571
05575     void unbind() const
05576     {
05577         if (texture) texture->unbind();
05578     }
05579 };
05580
05587 class GgNormalTexture
05588 {
05589     // テクスチャ
05590     std::shared_ptr<GgTexture> texture;
05591
05592 public:
05593
05597     GgNormalTexture()
05598     {
05599     }
05600
05611     GgNormalTexture(
05612         const GLubyte* image,
05613         GLsizei width,
05614         GLsizei height,
05615         GLenum format = GL_RED,
05616         GLfloat nz = 1.0f,
05617         GLenum internal = GL_RGBA
05618     )
05619     {
05620         // 法線マップのテクスチャを作成する
05621         load(image, width, height, format, nz, internal);
05622     }
05623
05631     GgNormalTexture(
05632         const std::string& name,
05633         GLfloat nz = 1.0f,
05634         GLenum internal = GL_RGBA
05635     )
05636     {
05637         // 法線マップのテクスチャを作成する
05638         load(name, nz, internal);
05639     }
05640

```

```

05644     virtual ~GgNormalTexture()
05645     {
05646     }
05647
05658     void load(
05659         const GLubyte* hmap,
05660         GLsizei width,
05661         GLsizei height,
05662         GLenum format = GL_RED,
05663         GLfloat nz = 1.0f,
05664         GLenum internal = GL_RGBA
05665     )
05666     {
05667         // 法線マップ
05668         std::vector<GgVector> nmap;
05669
05670         // 法線マップを作成する
05671         ggCreateNormalMap(hmap, width, height, format, nz, internal, nmap);
05672
05673         // テクスチャを作成する
05674         texture = std::make_shared<GgTexture>(nmap.data(), width, height, GL_RGBA, GL_FLOAT, internal,
GL_REPEAT);
05675     }
05676
05684     void load(
05685         const std::string& name,
05686         GLfloat nz = 1.0f,
05687         GLenum internal = GL_RGBA
05688     );
05689
05695     explicit operator bool() const noexcept
05696     {
05697         return texture != nullptr;
05698     }
05699
05705     bool operator!() const noexcept
05706     {
05707         return !static_cast<bool>(*this);
05708     }
05709
05715     const GgTexture* get() const
05716     {
05717         return texture.get();
05718     }
05719
05725     GLuint getTexture() const
05726     {
05727         return texture ? texture->getTexture() : 0;
05728     }
05729
05735     GLsizei getWidth() const
05736     {
05737         return texture ? texture->getWidth() : 0;
05738     }
05739
05745     GLsizei getHeight() const
05746     {
05747         return texture ? texture->getHeight() : 0;
05748     }
05749
05755     void getSize(GLsizei* size) const
05756     {
05757         if (texture) texture->getSize(size);
05758     }
05759
05765     const GLsizei* getSize() const
05766     {
05767         return texture ? texture->getSize() : nullptr;
05768     }
05769
05773     void bind() const
05774     {
05775         if (texture) texture->bind();
05776     }
05777
05781     void unbind() const
05782     {
05783         if (texture) texture->unbind();
05784     }
05785 };
05786
05793     template <typename T>
05794     class GgBuffer
05795     {
05796     // ターゲット
05797         const GLenum target;
05798     }

```

```

05799 // バッファオブジェクトのアライメントを考慮したデータの間隔
05800 const GLsizei stride;
05801
05802 // データの数
05803 const GLsizei count;
05804
05805 // バッファオブジェクト
05806 const GLuint buffer;
05807
05808 public:
05809
05819 GgBuffer(
05820     GLenum target,
05821     const T* data,
05822     GLsizei stride,
05823     GLsizei count,
05824     GLenum usage
05825 ) :
05826     target{ target },
05827     stride{ stride },
05828     count{ count },
05829     buffer{ [] { GLuint buffer; glGenBuffers(1, &buffer); return buffer; } () }
05830 {
05831     // バッファオブジェクトのメモリを確保してデータを転送する
05832     glBindBuffer(target, buffer);
05833     glBufferData(target, getStride() * count, data, usage);
05834 }
05835
05841 GgBuffer(const GgBuffer<T>& buffer) = delete;
05842
05848 GgBuffer(GgBuffer<T>&& buffer) = default;
05849
05853 virtual ~GgBuffer()
05854 {
05855     // バッファオブジェクトを削除する
05856     glBindBuffer(target, 0);
05857     glDeleteBuffers(1, &buffer);
05858 }
05859
05866 GgBuffer<T>& operator=(const GgBuffer<T>& buffer) = delete;
05874 GgBuffer<T>& operator=(GgBuffer<T>&& buffer) = default;
05875
05881 const GLuint& getTarget() const
05882 {
05883     return target;
05884 }
05885
05891 GLsizeiptr getStride() const
05892 {
05893     return static_cast<GLsizeiptr>(stride);
05894 }
05895
05901 const GLsizei& getCount() const
05902 {
05903     return count;
05904 }
05905
05911 const GLuint& getBuffer() const
05912 {
05913     return buffer;
05914 }
05915
05919 void bind() const
05920 {
05921     glBindBuffer(target, buffer);
05922 }
05923
05927 void unbind() const
05928 {
05929     glBindBuffer(target, 0);
05930 }
05931
05937 void* map() const
05938 {
05939     glBindBuffer(target, buffer);
05940 #if defined(GL_GLES_PROTOTYPES)
05941     return glMapBufferRange(target, 0, getStride() * count, GL_MAP_WRITE_BIT);
05942 #else
05943     return glMapBuffer(target, GL_WRITE_ONLY);
05944 #endif
05945 }
05946
05954 void* map(GLint first, GLsizei count) const
05955 {
05956     // count が 0 なら全データをマップする
05957     if (count == 0) count = getCount();

```

```

05958         if (first + count > getCount()) count = getCount() - first;
05959
05960         glBindBuffer(target, buffer);
05961         return glMapBufferRange(target, getStride() * first, getStride() * count, GL_MAP_WRITE_BIT);
05962     }
05963
05964     void unmap() const
05965     {
05966         glUnmapBuffer(target);
05967     }
05968
05969     void send(const T* data, GLint first, GLsizei count) const
05970     {
05971         // count が 0 なら全データを転送する
05972         if (count == 0) count = getCount();
05973         if (first + count > getCount()) count = getCount() - first;
05974
05975         // データを既存のバッファオブジェクトに転送する
05976         glBindBuffer(target, buffer);
05977         glBufferSubData(target, getStride() * first, getStride() * count, data);
05978     }
05979
05980     void read(T* data, GLint first, GLsizei count) const
05981     {
05982         // count が 0 なら全データを抽出する
05983         if (count == 0) count = getCount();
05984         if (first + count > getCount()) count = getCount() - first;
05985
05986         // データをバッファオブジェクトから抽出する
05987         glBindBuffer(target, buffer);
05988         #if defined(GL_GLES_PROTOTYPES)
05989             const GLsizeiptr begin{ getStride() * first };
05990             const GLsizeiptr range{ getStride() * count };
05991             T* const source{ glMapBufferRange(target, begin, range, GL_MAP_READ_BIT) };
05992             std::copy(source, source + count, data);
05993             glUnmapBuffer(target);
05994         #else
05995             glGetBufferSubData(target, getStride() * first, getStride() * count, data);
05996         #endif
05997     }
05998
06000     void copy(GLuint src_buffer, GLint src_first = 0, GLint dst_first = 0, GLsizei count = 0) const
06001     {
06002         // count が 0 なら全データを複製する
06003         if (count == 0) count = getCount();
06004         if (src_first + count > getCount()) count = getCount() - src_first;
06005         if (dst_first + count > getCount()) count = getCount() - dst_first;
06006
06007         // データの間隔
06008         const GLsizeiptr stride{ getStride() };
06009
06010         glBindBuffer(GL_COPY_READ_BUFFER, src_buffer);
06011         glBindBuffer(GL_COPY_WRITE_BUFFER, buffer);
06012         glCopyBufferSubData(GL_COPY_READ_BUFFER, GL_COPY_WRITE_BUFFER,
06013             stride * src_first, stride * dst_first, stride * count);
06014         glBindBuffer(GL_COPY_WRITE_BUFFER, 0);
06015         glBindBuffer(GL_COPY_READ_BUFFER, 0);
06016     }
06017 };
06018
06019 template <typename T>
06020 class GgUniformBuffer
06021 {
06022     // ユニフォームバッファオブジェクト
06023     std::shared_ptr<GgBuffer<T>> uniform;
06024
06025 public:
06026     GgUniformBuffer()
06027     {
06028     }
06029
06030     GgUniformBuffer(const T* data, GLsizei count, GLenum usage = GL_STATIC_DRAW)
06031     {
06032         load(data, count, usage);
06033     }
06034
06035     GgUniformBuffer(const T& data, GLsizei count, GLenum usage = GL_STATIC_DRAW)
06036     {
06037         load(data, count, usage);
06038     }
06039
06040     virtual ~GgUniformBuffer()
06041     {
06042     }
06043
06044     const GLuint& getTarget() const

```

```

06101     {
06102         return uniform->getTarget();
06103     }
06104
06110     GLsizeiptr getStride() const
06111     {
06112         return uniform->getStride();
06113     }
06114
06120     const GLsizei& getCount() const
06121     {
06122         return uniform->getCount();
06123     }
06124
06130     const GLuint& getBuffer() const
06131     {
06132         return uniform->getBuffer();
06133     }
06134
06138     void bind() const
06139     {
06140         uniform->bind();
06141     }
06142
06146     void unbind() const
06147     {
06148         uniform->unbind();
06149     }
06150
06156     void* map() const
06157     {
06158         return uniform->map();
06159     }
06160
06168     void* map(GLint first, GLsizei count) const
06169     {
06170         return uniform->map(first, count);
06171     }
06172
06176     void unmap() const
06177     {
06178         uniform->unmap();
06179     }
06180
06188     void load(const T* data, GLsizei count, GLenum usage = GL_STATIC_DRAW)
06189     {
06190         // バッファオブジェクト上のデータの間の隔
06191         const GLsizei stride{ (((static_cast<GLint>(sizeof(T)) - 1) / ggBufferAlignment) + 1) *
ggBufferAlignment } };
06192
06193         // ユニフォームバッファオブジェクトを確保する
06194         uniform = std::make_shared<GgBuffer<T>>(GL_UNIFORM_BUFFER, nullptr, stride, count, usage);
06195
06196         // 確保したユニフォームバッファオブジェクトにデータを転送する
06197         if (data) send(data, 0, sizeof(T), 0, count);
06198     }
06199
06207     void load(const T& data, GLsizei count, GLenum usage = GL_STATIC_DRAW)
06208     {
06209         // バッファオブジェクト上のデータの間の隔
06210         const GLsizei stride{ (((static_cast<GLint>(sizeof(T)) - 1) / ggBufferAlignment) + 1) *
ggBufferAlignment } };
06211
06212         // ユニフォームバッファオブジェクトを確保する
06213         uniform = std::make_shared<GgBuffer<T>>(GL_UNIFORM_BUFFER, nullptr, stride, count, usage);
06214
06215         // 確保したユニフォームバッファオブジェクトにデータを転送する
06216         fill(&data, 0, sizeof(T), 0, count);
06217     }
06218
06228     void send(
06229         const GLvoid* data,
06230         GLint offset = 0,
06231         GLsizei size = sizeof(T),
06232         GLint first = 0,
06233         GLsizei count = 0
06234     ) const
06235     {
06236         // count が 0 なら全データを転送する
06237         if (count == 0) count = getCount();
06238         if (first + count > getCount()) count = getCount() - first;
06239
06240         // 転送元のデータの先頭
06241         const char* source{ reinterpret_cast<const char*>(data) };
06242
06243         // ターゲット
06244         const GLuint target{ getTarget() };

```

```

06245
06246 // データの間隔
06247 const GLsizeiptr stride{ getStride() };
06248
06249 // first 番目のブロックから count 個の各ブロックの先頭から offset バイトの位置にデータを転送する
06250 bind();
06251 for (GLsizei i = 0; i < count; ++i)
06252 {
06253     glBufferSubData(target, stride * (first + i) + offset, size, source + size * i);
06254 }
06255 }
06256
06266 void fill(
06267     const GLvoid* data,
06268     GLint offset = 0,
06269     GLsizei size = sizeof(T),
06270     GLint first = 0,
06271     GLsizei count = 0
06272 ) const
06273 {
06274     // count が 0 なら全データを転送する
06275     if (count == 0) count = getCount();
06276     if (first + count > getCount()) count = getCount() - first;
06277
06278     // ターゲット
06279     const GLuint target{ getTarget() };
06280
06281     // データの間隔
06282     const GLsizeiptr stride{ getStride() };
06283
06284     // first 番目のブロックから count 個の各ブロックの先頭から offset バイトの位置にデータを転送する
06285     bind();
06286     for (GLsizei i = 0; i < count; ++i)
06287     {
06288         glBufferSubData(target, stride * (first + i) + offset, size, data);
06289     }
06290 }
06291
06301 void read(
06302     GLvoid* data,
06303     GLint offset = 0,
06304     GLsizei size = sizeof(T),
06305     GLint first = 0,
06306     GLsizei count = 0
06307 ) const
06308 {
06309     // count が 0 なら全データを転送する
06310     if (count == 0) count = getCount();
06311     if (first + count > getCount()) count = getCount() - first;
06312
06313     // 抽出先のデータの先頭
06314     char* const destination{ reinterpret_cast<char*>(data) };
06315
06316     // ターゲット
06317     const GLuint target{ getTarget() };
06318
06319     // データの間隔
06320     const GLsizeiptr stride{ getStride() };
06321
06322     // データをユニフォームバッファオブジェクトから抽出する
06323     bind();
06324     #if defined(GL_GLES_PROTOTYPES)
06325     const char* const source{ glMapBufferRange(target, stride * first, stride * count,
06326         GL_MAP_READ_BIT) };
06327     for (GLsizei i = 0; i < count; ++i)
06328     {
06329         const char* const begin{ source + stride * i + offset };
06330         const char* const end{ begin + size };
06331         std::copy(begin, end, destination + sizeof(T) * i);
06332     }
06333     glUnmapBuffer(target);
06334     #else
06335     for (GLsizei i = 0; i < count; ++i)
06336     {
06337         glGetBufferSubData(target, stride * (first + i) + offset, size, destination + sizeof(T) * i);
06338     }
06339     #endif
06340 }
06349 void copy(
06350     GLuint src_buffer,
06351     GLint src_first = 0,
06352     GLint dst_first = 0,
06353     GLsizei count = 0
06354 ) const
06355 {
06356     // count が 0 なら全データを複写する

```

```

06357     if (count == 0) count = getCount();
06358     if (src_first + count > getCount()) count = getCount() - src_first;
06359     if (dst_first + count > getCount()) count = getCount() - dst_first;
06360
06361     // データの間隔
06362     const GLsizeiptr stride{ getStride() };
06363
06364     // ユニフォームバッファオブジェクトではブロックごとに転送する
06365     glBindBuffer(GL_COPY_READ_BUFFER, src_buffer);
06366     glBindBuffer(GL_COPY_WRITE_BUFFER, getBuffer());
06367     for (GLsizei i = 0; i < count; ++i)
06368     {
06369         glCopyBufferSubData(GL_COPY_READ_BUFFER, GL_COPY_WRITE_BUFFER,
06370             stride * (src_first + i), stride * (dst_first + i), sizeof(T));
06371     }
06372     glBindBuffer(GL_COPY_WRITE_BUFFER, 0);
06373     glBindBuffer(GL_COPY_READ_BUFFER, 0);
06374 }
06375 };
06376
06380 class GgVertexArray
06381 {
06382     // 頂点配列オブジェクト
06383     const GLuint vao;
06384
06385 public:
06386
06392     GgVertexArray(GLenum mode = 0) :
06393         vao{ [] { GLuint vao; glGenVertexArrays(1, &vao); return vao; } () }
06394     {
06395         glBindVertexArray(vao);
06396     }
06397
06403     GgVertexArray(const GgVertexArray& array) = delete;
06404
06410     GgVertexArray(GgVertexArray&& array) = default;
06411
06415     virtual ~GgVertexArray()
06416     {
06417         glBindVertexArray(0);
06418         glDeleteVertexArrays(1, &vao);
06419     }
06420
06427     GgVertexArray& operator=(const GgVertexArray& array) = delete;
06428
06434     const GLuint& get() const
06435     {
06436         return vao;
06437     }
06438
06442     void bind() const
06443     {
06444         glBindVertexArray(vao);
06445     }
06446 };
06447
06455 class GgShape
06456 {
06457     // 頂点配列オブジェクト
06458     std::shared_ptr<GgVertexArray> object;
06459
06460     // 基本図形の種類
06461     GLenum mode;
06462
06463 public:
06464
06470     GgShape(GLenum mode = 0) :
06471         object{ std::make_shared<GgVertexArray>() },
06472         mode{ mode }
06473     {
06474     }
06475
06479     virtual ~GgShape()
06480     {
06481     }
06482
06488     virtual explicit operator bool() const noexcept
06489     {
06490         return object.get() != nullptr;
06491     }
06492
06498     virtual bool operator!() const noexcept
06499     {
06500         return !static_cast<bool>(*this);
06501     }
06502
06508     const GLuint& get() const

```



```

06509     {
06510         return object->get();
06511     }
06512
06518     void setMode(GLenum mode)
06519     {
06520         this->mode = mode;
06521     }
06522
06528     const GLenum& getMode() const
06529     {
06530         return this->mode;
06531     }
06532
06539     virtual void draw(GLint first = 0, GLsizei count = 0) const
06540     {
06541         object->bind();
06542     }
06543 };
06544
06548     class GgPoints
06549     : public GgShape
06550     {
06551         // 頂点バッファオブジェクト
06552         std::shared_ptr<GgBuffer<GgVector>> position;
06553     public:
06554
06559         GgPoints(GLenum mode = GL_POINTS) :
06560             GgShape(mode)
06561         {
06562         }
06563
06572         GgPoints(
06573             const GgVector* pos,
06574             GLsizei countv,
06575             GLenum mode = GL_POINTS,
06576             GLenum usage = GL_STATIC_DRAW
06577         ) :
06578             GgPoints(mode)
06579         {
06580             load(pos, countv, usage);
06581         }
06582
06586         virtual ~GgPoints()
06587         {
06588         }
06589
06595         explicit operator bool() const noexcept
06596         {
06597             return position.get() != nullptr;
06598         }
06599
06605         bool operator!() const noexcept
06606         {
06607             return !static_cast<bool>(*this);
06608         }
06609
06615         const GLsizei& getCount() const
06616         {
06617             return position->getCount();
06618         }
06619
06625         const GLuint& getBuffer() const
06626         {
06627             return position->getBuffer();
06628         }
06629
06637         void send(const GgVector* pos, GLint first = 0, GLsizei count = 0) const
06638         {
06639             position->send(pos, first, count);
06640         }
06641
06649         void load(const GgVector* pos, GLsizei count, GLenum usage = GL_STATIC_DRAW);
06650
06657         virtual void draw(GLint first = 0, GLsizei count = 0) const;
06658     };
06659
06663     struct GgVertex
06664     {
06666         GgVector position;
06667
06669         GgVector normal;
06670
06674         GgVertex()
06675         {
06676         }

```

```

06677
06684 GgVertex(const GgVector& pos, const GgVector& norm) :
06685     position(pos),
06686     normal(norm)
06687 {
06688 }
06689
06700 GgVertex(
06701     GLfloat px, GLfloat py, GLfloat pz,
06702     GLfloat nx, GLfloat ny, GLfloat nz
06703 ) :
06704     position{ px, py, pz, 1.0f },
06705     normal{ nx, ny, nz, 0.0f }
06706 {
06707 }
06708
06715 GgVertex(const GLfloat* pos, const GLfloat* norm) :
06716     GgVertex(pos[0], pos[1], pos[2], norm[0], norm[1], norm[2])
06717 {
06718 }
06719 };
06720
06724 class GgTriangles
06725 : public GgShape
06726 {
06727     // 頂点属性
06728     std::shared_ptr<GgBuffer<GgVertex>> vertex;
06729
06730 public:
06731
06737 GgTriangles(GLenum mode = GL_TRIANGLES) :
06738     GgShape(mode)
06739 {
06740 }
06741
06750 GgTriangles(
06751     const GgVertex* vert,
06752     GLsizei count,
06753     GLenum mode = GL_TRIANGLES,
06754     GLenum usage = GL_STATIC_DRAW
06755 ) :
06756     GgTriangles(mode)
06757 {
06758     load(vert, count, usage);
06759 }
06760
06764 virtual ~GgTriangles()
06765 {
06766 }
06767
06773 const GLsizei& getCount() const
06774 {
06775     return vertex->getCount();
06776 }
06777
06783 const GLuint& getBuffer() const
06784 {
06785     return vertex->getBuffer();
06786 }
06787
06795 void send(const GgVertex* vert, GLint first = 0, GLsizei count = 0) const
06796 {
06797     vertex->send(vert, first, count);
06798 }
06799
06807 void load(const GgVertex* vert, GLsizei count, GLenum usage = GL_STATIC_DRAW);
06808
06815 virtual void draw(GLint first = 0, GLsizei count = 0) const;
06816 };
06817
06821 class GgElements
06822 : public GgTriangles
06823 {
06824     // インデックスを格納する頂点バッファオブジェクト
06825     std::shared_ptr<GgBuffer<GLuint>> index;
06826
06827 public:
06828
06834 GgElements(GLenum mode = GL_TRIANGLES) :
06835     GgTriangles(mode)
06836 {
06837 }
06838
06849 GgElements(
06850     const GgVertex* vert,
06851     GLsizei countv,
06852     const GLuint* face,

```

```

06853     GLsizei countf,
06854     GLenum mode = GL_TRIANGLES,
06855     GLenum usage = GL_STATIC_DRAW
06856 ) :
06857     GgElements(mode)
06858 {
06859     load(vert, countv, face, countf, usage);
06860 }
06861
06862 virtual ~GgElements()
06863 {
06864 }
06865
06866 const GLsizei& getIndexCount() const
06867 {
06868     return index->getCount();
06869 }
06870
06871 const GLuint& getIndexBuffer() const
06872 {
06873     return index->getBuffer();
06874 }
06875
06876 void send(
06877     const GgVertex* vert,
06878     GLuint firstv,
06879     GLsizei countv,
06880     const GLuint* face = nullptr,
06881     GLuint firstf = 0,
06882     GLsizei countf = 0
06883 ) const
06884 {
06885     GgTriangles::send(vert, firstv, countv);
06886     if (face != nullptr && countf > 0) index->send(face, firstf, countf);
06887 }
06888
06889 void load(
06890     const GgVertex* vert,
06891     GLsizei countv,
06892     const GLuint* face,
06893     GLsizei countf,
06894     GLenum usage = GL_STATIC_DRAW
06895 )
06896 {
06897     // 頂点バッファオブジェクトを作成する
06898     GgTriangles::load(vert, countv, usage);
06899
06900     // インデックスの頂点バッファオブジェクトを作成する
06901     index = std::make_shared<GgBuffer<GLuint>>(GL_ELEMENT_ARRAY_BUFFER, face,
06902         static_cast<GLsizei>(sizeof(GLuint)), countf, usage);
06903 }
06904
06905 virtual void draw(GLint first = 0, GLsizei count = 0) const;
06906 };
06907
06908 extern std::shared_ptr<GgPoints> ggPointsCube(
06909     GLsizei countv,
06910     GLfloat length = 1.0f,
06911     GLfloat cx = 0.0f,
06912     GLfloat cy = 0.0f,
06913     GLfloat cz = 0.0f
06914 );
06915
06916 extern std::shared_ptr<GgPoints> ggPointsSphere(
06917     GLsizei countv,
06918     GLfloat radius = 0.5f,
06919     GLfloat cx = 0.0f,
06920     GLfloat cy = 0.0f,
06921     GLfloat cz = 0.0f
06922 );
06923
06924 extern std::shared_ptr<GgTriangles> ggRectangle(
06925     GLfloat width = 1.0f,
06926     GLfloat height = 1.0f
06927 );
06928
06929 extern std::shared_ptr<GgTriangles> ggEllipse(
06930     GLfloat width = 1.0f,
06931     GLfloat height = 1.0f,
06932     GLuint slices = 16
06933 );
06934
06935 extern std::shared_ptr<GgTriangles> ggArraysObj(
06936     const std::string& name,
06937     bool normalize = false
06938 );
06939
06940
06941

```

```

07033 extern std::shared_ptr<GgElements> ggElementsObj(
07034     const std::string& name,
07035     bool normalize = false
07036 );
07037
07050 extern std::shared_ptr<GgElements> ggElementsMesh(
07051     GLuint slices,
07052     GLuint stacks,
07053     const GLfloat(*pos)[3],
07054     const GLfloat(*norm)[3] = nullptr
07055 );
07056
07067 extern std::shared_ptr<GgElements> ggElementsSphere(
07068     GLfloat radius = 1.0f,
07069     int slices = 16,
07070     int stacks = 8
07071 );
07072
07079 class GgShader
07080 {
07081     // プログラム名
07082     const GLuint program;
07083
07084 public:
07085
07095     GgShader(
07096         const std::string& vert,
07097         const std::string& frag = "",
07098         const std::string& geom = "",
07099         int nvarying = 0,
07100         const char* const* varyings = nullptr
07101     ) :
07102         program(ggLoadShader(vert, frag, geom, nvarying, varyings))
07103     {
07104     }
07105
07113     GgShader(
07114         const std::array<std::string, 3>& files,
07115         int nvarying = 0,
07116         const char* const* varyings = nullptr
07117     ) :
07118         GgShader(files[0], files[1], files[2], nvarying, varyings)
07119     {
07120     }
07121
07127     GgShader(const GgShader& shader) = delete;
07128
07134     GgShader(GgShader&& shader) = default;
07135
07139     virtual ~GgShader()
07140     {
07141         // 参照しているオブジェクトが一つだけならシェーダを削除する
07142         glUseProgram(0);
07143         glDeleteProgram(program);
07144     }
07145
07152     GgShader& operator=(const GgShader& shader) = delete;
07153
07157     void use() const
07158     {
07159         glUseProgram(program);
07160     }
07161
07165     void unuse() const
07166     {
07167         glUseProgram(0);
07168     }
07169
07175     GLuint get() const
07176     {
07177         return program;
07178     }
07179 };
07180
07184 class GgPointShader
07185 {
07186     // シェーダー
07187     std::shared_ptr<GgShader> shader;
07188
07189     // 投影変換行列の uniform 変数の場所
07190     GLint mpLoc;
07191
07192     // モデルビュー変換行列の uniform 変数の場所
07193     GLint mvLoc;
07194
07195 public:
07196

```

```

07200     GgPointShader() :
07201         mpLoc{ -1 },
07202         mvLoc{ -1 }
07203     {
07204     }
07205
07215     GgPointShader(
07216         const std::string& vert,
07217         const std::string& frag = "",
07218         const std::string& geom = "",
07219         GLint nvarying = 0,
07220         const char* const* varyings = nullptr
07221     ) :
07222         GgPointShader()
07223     {
07224         load(vert, frag, geom, nvarying, varyings);
07225     }
07226
07234     GgPointShader(
07235         const std::array<std::string, 3>& files,
07236         int nvarying = 0,
07237         const char* const* varyings = nullptr
07238     ) :
07239         GgPointShader(files[0], files[1], files[2], nvarying, varyings)
07240     {
07241     }
07242
07246     virtual ~GgPointShader()
07247     {
07248     }
07249
07260     bool load(
07261         const std::string& vert,
07262         const std::string& frag = "",
07263         const std::string& geom = "",
07264         GLint nvarying = 0,
07265         const char* const* varyings = nullptr
07266     )
07267     {
07268         // シェーダを作成する
07269         shader = std::make_shared<GgShader>(vert, frag, geom, nvarying, varyings);
07270
07271         // プログラム名を取り出す
07272         const GLuint program(shader->get());
07273
07274         // プログラムオブジェクトが作成できていなければ戻る
07275         if (program == 0) return false;
07276
07277         // 変換行列の uniform 変数の場所
07278         mpLoc = glGetUniformLocation(program, "mp");
07279         mvLoc = glGetUniformLocation(program, "mv");
07280
07281         // プログラムオブジェクトの作成に成功した
07282         return true;
07283     }
07284
07293     bool load(
07294         const std::array<std::string, 3>& files,
07295         GLint nvarying = 0,
07296         const char* const* varyings = nullptr
07297     )
07298     {
07299         return load(files[0], files[1], files[2], nvarying, varyings);
07300     }
07301
07307     virtual void loadProjectionMatrix(const GLfloat* mp) const
07308     {
07309         glUniformMatrix4fv(mpLoc, 1, GL_FALSE, mp);
07310     }
07311
07317     virtual void loadProjectionMatrix(const GgMatrix& mp) const
07318     {
07319         loadProjectionMatrix(mp.get());
07320     }
07321
07327     virtual void loadModelviewMatrix(const GLfloat* mv) const
07328     {
07329         glUniformMatrix4fv(mvLoc, 1, GL_FALSE, mv);
07330     }
07331
07337     virtual void loadModelviewMatrix(const GgMatrix& mv) const
07338     {
07339         loadModelviewMatrix(mv.get());
07340     }
07341
07348     virtual void loadMatrix(const GLfloat* mp, const GLfloat* mv) const
07349     {

```

```

07350         loadProjectionMatrix(mp);
07351         loadModelviewMatrix(mv);
07352     }
07353
07360     virtual void loadMatrix(const GgMatrix& mp, const GgMatrix& mv) const
07361     {
07362         loadMatrix(mp.get(), mv.get());
07363     }
07364
07368     virtual void use() const
07369     {
07370         shader->use();
07371     }
07372
07378     void use(const GLfloat* mp) const
07379     {
07380         use();
07381         loadProjectionMatrix(mp);
07382     }
07383
07389     void use(const GgMatrix& mp) const
07390     {
07391         use(mp.get());
07392     }
07393
07400     void use(const GLfloat* mp, const GLfloat* mv) const
07401     {
07402         use(mp);
07403         loadModelviewMatrix(mv);
07404     }
07405
07412     void use(const GgMatrix& mp, const GgMatrix& mv) const
07413     {
07414         use(mp.get(), mv.get());
07415     }
07416
07420     void unuse() const
07421     {
07422         shader->unuse();
07423     }
07424
07430     GLuint get() const
07431     {
07432         return shader->get();
07433     }
07434 };
07435
07439 class GgSimpleShader
07440 : public GgPointShader
07441 {
07442     // モデルビュー変換の法線変換行列の uniform 変数の場所
07443     GLint mnLoc;
07444
07445 public:
07446
07450     GgSimpleShader() :
07451         GgPointShader(),
07452         mnLoc{ -1 }
07453     {
07454     }
07455
07465     GgSimpleShader(
07466         const std::string& vert,
07467         const std::string& frag = "",
07468         const std::string& geom = "",
07469         GLint nvarying = 0,
07470         const char* const* varyings = nullptr
07471     )
07472     {
07473         load(vert, frag, geom, nvarying, varyings);
07474     }
07475
07483     GgSimpleShader(
07484         const std::array<std::string, 3>& files,
07485         GLint nvarying = 0,
07486         const char* const* varyings = nullptr
07487     ) :
07488         GgSimpleShader(files[0], files[1], files[2], nvarying, varyings)
07489     {
07490     }
07491
07495     GgSimpleShader(const GgSimpleShader& o) :
07496         GgPointShader(o),
07497         mnLoc{ o.mnLoc }
07498     {
07499     }
07500

```

```

07504     virtual ~GgSimpleShader()
07505     {
07506     }
07507
07514     GgSimpleShader& operator=(const GgSimpleShader& o)
07515     {
07516         if (&o != this)
07517         {
07518             GgPointShader::operator=(o);
07519             mnLoc = o.mnLoc;
07520         }
07521
07522         return *this;
07523     }
07524
07535     bool load(
07536         const std::string& vert,
07537         const std::string& frag = "",
07538         const std::string& geom = "",
07539         GLint nvarying = 0,
07540         const char* const* varyings = nullptr
07541     );
07542
07551     bool load(
07552         const std::array<std::string, 3>& files,
07553         GLint nvarying = 0,
07554         const char* const* varyings = nullptr
07555     )
07556     {
07557         return load(files[0], files[1], files[2], nvarying, varyings);
07558     }
07559
07566     virtual void loadModelviewMatrix(const GLfloat* mv, const GLfloat* mn) const
07567     {
07568         GgPointShader::loadModelviewMatrix(mv);
07569         glUniformMatrix4fv(mnLoc, 1, GL_FALSE, mn);
07570     }
07571
07578     virtual void loadModelviewMatrix(const GgMatrix& mv, const GgMatrix& mn) const
07579     {
07580         loadModelviewMatrix(mv.get(), mn.get());
07581     }
07582
07588     virtual void loadModelviewMatrix(const GLfloat* mv) const
07589     {
07590         loadModelviewMatrix(mv, GgMatrix(mv).normal().get());
07591     }
07592
07598     virtual void loadModelviewMatrix(const GgMatrix& mv) const
07599     {
07600         loadModelviewMatrix(mv.get());
07601     }
07602
07610     virtual void loadMatrix(const GLfloat* mp, const GLfloat* mv, const GLfloat* mn) const
07611     {
07612         GgPointShader::loadMatrix(mp, mv);
07613         glUniformMatrix4fv(mnLoc, 1, GL_FALSE, mn);
07614     }
07615
07623     virtual void loadMatrix(const GgMatrix& mp, const GgMatrix& mv, const GgMatrix& mn) const
07624     {
07625         loadMatrix(mp.get(), mv.get(), mn.get());
07626     }
07627
07634     virtual void loadMatrix(const GLfloat* mp, const GLfloat* mv) const
07635     {
07636         loadMatrix(mp, mv, GgMatrix(mv).normal());
07637     }
07638
07645     virtual void loadMatrix(const GgMatrix& mp, const GgMatrix& mv) const
07646     {
07647         loadMatrix(mp, mv, mv.normal());
07648     }
07649
07653     struct Light
07654     {
07655         GgVector ambient;
07656         GgVector diffuse;
07657         GgVector specular;
07658         GgVector position;
07659     };
07660
07664     class LightBuffer
07665     : public GgUniformBuffer<Light>
07666     {
07667     public:
07668

```

```

07676     LightBuffer(
07677         const Light* light = nullptr,
07678         GLsizei count = 1,
07679         GLenum usage = GL_STATIC_DRAW
07680     ) :
07681     {
07682     {
07683     }
07684
07692     LightBuffer(
07693         const Light& light,
07694         GLsizei count = 1,
07695         GLenum usage = GL_STATIC_DRAW
07696     ) :
07697     {
07698     {
07699     }
07700
07711     LightBuffer(
07712         GgVector ambient,
07713         GgVector diffuse,
07714         GgVector specular,
07715         GgVector position,
07716         GLsizei count = 1,
07717         GLenum usage = GL_STATIC_DRAW
07718     ) :
07719     {
07720     {
07721     }
07722
07726     virtual ~LightBuffer()
07727     {
07728     }
07729
07740     void loadAmbient(
07741         GLfloat r, GLfloat g, GLfloat b, GLfloat a = 1.0f,
07742         GLint first = 0, GLsizei count = 1
07743     ) const;
07744
07752     void loadAmbient(const GgVector& ambient, GLint first = 0, GLsizei count = 1) const;
07753
07761     void loadAmbient(const GLfloat* ambient, GLint first = 0, GLsizei count = 1) const
07762     {
07763         // first 番目のブロックから count 個の ambient 要素に値を設定する
07764         send(ambient, offsetof(Light, ambient), sizeof(Light::ambient), first, count);
07765     }
07766
07777     void loadDiffuse(
07778         GLfloat r, GLfloat g, GLfloat b, GLfloat a = 1.0f,
07779         GLint first = 0, GLsizei count = 1
07780     ) const;
07781
07789     void loadDiffuse(const GgVector& diffuse, GLint first = 0, GLsizei count = 1) const;
07790
07798     void loadDiffuse(const GLfloat* diffuse, GLint first = 0, GLsizei count = 1) const
07799     {
07800         // first 番目のブロックから count 個の diffuse 要素に値を設定する
07801         send(diffuse, offsetof(Light, diffuse), sizeof(Light::diffuse), first, count);
07802     }
07803
07814     void loadSpecular(
07815         GLfloat r, GLfloat g, GLfloat b, GLfloat a = 1.0f,
07816         GLint first = 0, GLsizei count = 1
07817     ) const;
07818
07826     void loadSpecular(const GgVector& specular, GLint first = 0, GLsizei count = 1) const;
07827
07835     void loadSpecular(const GLfloat* specular, GLint first = 0, GLsizei count = 1) const
07836     {
07837         // first 番目のブロックから count 個の specular 要素に値を設定する
07838         send(specular, offsetof(Light, specular), sizeof(Light::specular), first, count);
07839     }
07840
07848     void loadColor(const Light& color, GLint first = 0, GLsizei count = 1) const;
07849
07860     void loadPosition(
07861         GLfloat x, GLfloat y, GLfloat z, GLfloat w = 1.0f,
07862         GLint first = 0, GLsizei count = 1
07863     ) const;
07864
07872     void loadPosition(const GgVector& position, GLint first = 0, GLsizei count = 1) const;
07873
07881     void loadPosition(const GLfloat* position, GLint first = 0, GLsizei count = 1) const
07882     {
07883         // first 番目のブロックから count 個の position 要素に値を設定する
07884         send(position, offsetof(Light, position), sizeof(Light::position), first, count);
07885     }

```



```

07886
07894 void loadPosition(const GgVector* position, GLint first = 0, GLsizei count = 1) const
07895 {
07896     loadPosition(position->data(), first, count);
07897 }
07898
07906 void load(const Light* light, GLint first = 0, GLsizei count = 1) const
07907 {
07908     send(light, 0, sizeof(Light), first, count);
07909 }
07910
07918 void load(const Light& light, GLint first = 0, GLsizei count = 1) const
07919 {
07920     load(&light, first, count);
07921 }
07922
07928 void select(GLint i = 0) const
07929 {
07930     // バッファオブジェクトの i 番目のブロックの位置
07931     const GLintptr offset(static_cast<GLintptr>(getStride()) * i);
07932     glBindBufferRange(getTarget(), LightBindingPoint, getBuffer(), offset, sizeof(Light));
07933 }
07934 };
07935
07939 struct Material
07940 {
07941     GgVector ambient;
07942     GgVector diffuse;
07943     GgVector specular;
07944     GLfloat shininess;
07945 };
07946
07950 class MaterialBuffer
07951 : public GgUniformBuffer<Material>
07952 {
07953 public:
07954
07962     MaterialBuffer(
07963         const Material* material = nullptr,
07964         GLsizei count = 1,
07965         GLenum usage = GL_STATIC_DRAW
07966     ) :
07967         GgUniformBuffer<Material>(material, count, usage)
07968     {
07969     }
07970
07978     MaterialBuffer(
07979         const Material& material,
07980         GLsizei count = 1,
07981         GLenum usage = GL_STATIC_DRAW
07982     ) :
07983         GgUniformBuffer<Material>(material, count, usage)
07984     {
07985     }
07986
07997     MaterialBuffer(
07998         GgVector ambient,
07999         GgVector diffuse,
08000         GgVector specular,
08001         GLfloat shininess,
08002         GLsizei count = 1,
08003         GLenum usage = GL_STATIC_DRAW
08004     ) :
08005         GgUniformBuffer<Material>(Material{ ambient, diffuse, specular, shininess }, count, usage)
08006     {
08007     }
08008
08012     virtual ~MaterialBuffer()
08013     {
08014     }
08015
08026     void loadAmbient(
08027         GLfloat r, GLfloat g, GLfloat b, GLfloat a = 1.0f,
08028         GLint first = 0, GLsizei count = 1
08029     ) const;
08030
08038     void loadAmbient(const GgVector& ambient, GLint first = 0, GLsizei count = 1) const;
08039
08047     void loadAmbient(const GLfloat* ambient, GLint first = 0, GLsizei count = 1) const
08048     {
08049         // first 番目のブロックから count 個のブロックの ambient 要素に値を設定する
08050         send(ambient, offsetof(Material, ambient), sizeof(Material::ambient), first, count);
08051     }
08052
08063     void loadDiffuse(
08064         GLfloat r, GLfloat g, GLfloat b, GLfloat a = 1.0f,
08065         GLint first = 0, GLsizei count = 1

```

```

08066     ) const;
08067
08075 void loadDiffuse(const GgVector& diffuse, GLint first = 0, GLsizei count = 1) const;
08076
08084 void loadDiffuse(const GLfloat* diffuse, GLint first = 0, GLsizei count = 1) const
08085 {
08086     // first 番目のブロックから count 個の diffuse 要素に値を設定する
08087     send(diffuse, offsetof(Material, diffuse), sizeof(Material::diffuse), first, count);
08088 }
08089
08100 void loadAmbientAndDiffuse(
08101     GLfloat r, GLfloat g, GLfloat b, GLfloat a = 1.0f,
08102     GLint first = 0, GLsizei count = 1
08103 ) const;
08104
08112 void loadAmbientAndDiffuse(const GgVector& color, GLint first = 0, GLsizei count = 1) const;
08113
08121 void loadAmbientAndDiffuse(const GLfloat* color, GLint first = 0, GLsizei count = 1) const;
08122
08133 void loadSpecular(
08134     GLfloat r, GLfloat g, GLfloat b, GLfloat a = 1.0f,
08135     GLint first = 0, GLsizei count = 1
08136 ) const;
08137
08145 void loadSpecular(const GgVector& specular, GLint first = 0, GLsizei count = 1) const;
08146
08154 void loadSpecular(const GLfloat* specular, GLint first = 0, GLsizei count = 1) const
08155 {
08156     // first 番目のブロックから count 個の specular 要素に値を設定する
08157     send(specular, offsetof(Material, specular), sizeof(Material::specular), first, count);
08158 }
08159
08167 void loadShininess(GLfloat shininess, GLint first = 0, GLsizei count = 1) const;
08168
08176 void loadShininess(const GLfloat* shininess, GLint first = 0, GLsizei count = 1) const;
08177
08185 void load(const Material* material, GLint first = 0, GLsizei count = 1) const
08186 {
08187     send(material, 0, sizeof(Material), first, count);
08188 }
08189
08197 void load(const Material& material, GLint first = 0, GLsizei count = 1) const
08198 {
08199     load(&material, first, count);
08200 }
08201
08207 void select(GLint i = 0) const
08208 {
08209     // バッファオブジェクトの i 番目のブロックの位置
08210     const GLintptr offset{ static_cast<GLintptr>(getStride()) * i };
08211     glBindBufferRange(getTarget(), MaterialBindingPoint, getBuffer(), offset, sizeof(Material));
08212 }
08213 };
08214
08218 void use() const
08219 {
08220     // プログラムオブジェクトは基底クラスで指定する
08221     GgPointShader::use();
08222 }
08223
08231 void use(const GLfloat* mp, const GLfloat* mv, const GLfloat* mn) const
08232 {
08233     // プログラムオブジェクトを指定する
08234     use();
08235
08236     // 変換行列を設定する
08237     loadMatrix(mp, mv, mn);
08238 }
08239
08247 void use(const GgMatrix& mp, const GgMatrix& mv, const GgMatrix& mn) const
08248 {
08249     use(mp.get(), mv.get(), mn.get());
08250 }
08251
08258 void use(const GLfloat* mp, const GLfloat* mv) const
08259 {
08260     use(mp, mv, GgMatrix(mv).normal().get());
08261 }
08262
08269 void use(const GgMatrix& mp, const GgMatrix& mv) const
08270 {
08271     use(mp, mv, mv.normal());
08272 }
08273
08280 void use(const LightBuffer* light, GLint i = 0) const
08281 {
08282     // プログラムオブジェクトを指定する

```

```

08283     use();
08284
08285     // 光源を設定する
08286     light->select(i);
08287 }
08288
08295 void use(const LightBuffer& light, GLint i = 0) const
08296 {
08297     use(&light, i);
08298 }
08299
08309 void use(
08310     const GLfloat* mp,
08311     const GLfloat* mv,
08312     const GLfloat* mn,
08313     const LightBuffer* light,
08314     GLint i = 0
08315 ) const
08316 {
08317     // 光源を指定してプログラムオブジェクトを指定する
08318     use(light, i);
08319
08320     // 変換行列を設定する
08321     loadMatrix(mp, mv, mn);
08322 }
08323
08333 void use(
08334     const GgMatrix& mp,
08335     const GgMatrix& mv,
08336     const GgMatrix& mn,
08337     const LightBuffer& light,
08338     GLint i = 0
08339 ) const
08340 {
08341     use(mp.get(), mv.get(), mn.get(), &light, i);
08342 }
08343
08352 void use(
08353     const GLfloat* mp,
08354     const GLfloat* mv,
08355     const LightBuffer* light,
08356     GLint i = 0
08357 ) const
08358 {
08359     use(mp, mv, GgMatrix(mv).normal().get(), light, i);
08360 }
08361
08370 void use(
08371     const GgMatrix& mp,
08372     const GgMatrix& mv,
08373     const LightBuffer& light,
08374     GLint i = 0
08375 ) const
08376 {
08377     use(mp, mv, mv.normal(), light, i);
08378 }
08379
08387 void use(const GLfloat* mp, const LightBuffer* light, GLint i = 0) const
08388 {
08389     // 光源を指定してプログラムオブジェクトを指定する
08390     use(light, i);
08391
08392     // 投影変換行列を設定する
08393     loadProjectionMatrix(mp);
08394 }
08395
08403 void use(const GgMatrix& mp, const LightBuffer& light, GLint i = 0) const
08404 {
08405     // 光源を指定してプログラムオブジェクトを指定する
08406     use(mp.get(), &light, i);
08407 }
08408 };
08409
08420 extern bool ggLoadSimpleObj(
08421     const std::string& name,
08422     std::vector<std::array<GLuint, 3>>& group,
08423     std::vector<GgSimpleShader::Material>& material,
08424     std::vector<GgVertex>& vert,
08425     bool normalize = false
08426 );
08427
08439 extern bool ggLoadSimpleObj(
08440     const std::string& name,
08441     std::vector<std::array<GLuint, 3>>& group,
08442     std::vector<GgSimpleShader::Material>& material,
08443     std::vector<GgVertex>& vert,
08444     std::vector<GLuint>& face,

```

```

08445     bool normalize = false
08446 );
08447
08451 class GgSimpleObj
08452 {
08453     // 同じ材質を割り当てるポリゴングループごとの三角形数
08454     std::shared_ptr<std::vector<std::array<GLuint, 3>>> group;
08455
08456     // ポリゴングループごとの材質のユニフォームバッファ
08457     std::shared_ptr<GgSimpleShader::MaterialBuffer> material;
08458
08459     // この図形の形状データ
08460     std::shared_ptr<GgElements> data;
08461
08462 public:
08463
08470     GgSimpleObj(const std::string& name, bool normalize = false);
08471
08474     virtual ~GgSimpleObj()
08475     {
08476     }
08477
08483     explicit operator bool() const noexcept
08484     {
08485         return data.get() != nullptr;
08486     }
08487
08493     bool operator!() const noexcept
08494     {
08495         return !static_cast<bool>(*this);
08496     }
08497
08503     const GgTriangles* get() const
08504     {
08505         return data.get();
08506     }
08507
08514     virtual void draw(GLint first = 0, GLsizei count = 0) const;
08515 };
08516 }

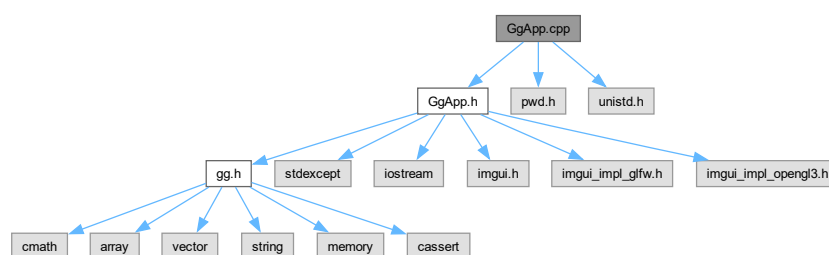
```

10.9 GgApp.cpp ファイル

```

#include "GgApp.h"
#include <pwd.h>
#include <unistd.h>
GgApp.cpp の依存先関係図:

```



10.9.1 詳解

ゲームグラフィックス特論宿題アプリケーションクラスの実装.

著者

Kohe Tokoi

日付

July 27, 2025

[GgApp.cpp](#) に定義があります。

10.10 GgApp.cpp

[詳解]

```

00001 /*
00002
00003 ゲームグラフィックス特論用補助プログラム GLFW3 版
00004
00005 Copyright (c) 2011-2025 Kohe Tokoi. All Rights Reserved.
00006
00007 Permission is hereby granted, free of charge, to any person obtaining a copy
00008 of this software and associated documentation files (the "Software"), to deal
00009 in the Software without restriction, including without limitation the rights
00010 to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00011 copies or substantial portions of the Software.
00012
00013 The above copyright notice and this permission notice shall be included in
00014 all copies or substantial portions of the Software.
00015
00016 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00017 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
00018 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL
00019 KOHE TOKOI BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN
00020 AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN
00021 CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
00022
00023 */
00024
00032 #include "GgApp.h"
00033 using namespace gg;
00034
00035 //
00036 // GLFW のエラー表示
00037 //
00038 static void glfwErrorCallback(int error, const char* description)
00039 {
00040     #if defined(__aarch64__)
00041         if (error == 65544) return;
00042     #endif
00043     throw std::runtime_error(description);
00044 }
00045
00046 //
00047 // GgApp クラスのコンストラクタ
00048 //
00049 GgApp::GgApp(int major, int minor)
00050 {
00051     // GLFW のエラー処理関数を登録する
00052     glfwSetErrorCallback(glfwErrorCallback);
00053
00054     // GLFW を初期化する
00055     if (glfwInit() == GLFW_FALSE) throw std::runtime_error("Can't initialize GLFW");
00056
00057     // OpenGL の major 番号が指定されていれば
00058     if (major > 0)
00059     {
00060         // OpenGL のバージョンを指定する
00061         glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, major);
00062         glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, minor);
00063
00064         #if defined(GL_GLES_PROTOTYPES)
00065             // OpenGL ES 3 のコンテキストを指定する
00066             glfwWindowHint(GLFW_CLIENT_API, GLFW_OPENGL_ES_API);
00067             glfwWindowHint(GLFW_CONTEXT_CREATION_API, GLFW_EGL_CONTEXT_API);
00068         #else
00069             // OpenGL Version 3.2 以降なら
00070             if (major * 10 + minor >= 32)
00071             {
00072                 // Core Profile を選択する (macOS の都合)
00073                 glfwWindowHint(GLFW_OPENGL_FORWARD_COMPAT, GL_TRUE);
00074                 glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
00075             }
00076         #endif
00077     }
00078
00079     #if defined(IMGUI_VERSION)
00080         // ImGui のバージョンをチェックする
00081         ImGui::CheckVersion();

```

```

00082
00083 // ImGui のコンテキストを作成する
00084 ImGui::CreateContext();
00085 #endif
00086 }
00087
00088 //
00089 // デストラクタ
00090 //
00091 GgApp::~GgApp()
00092 {
00093     #if defined(IMGUI_VERSION)
00094         // Shutdown Platform/Renderer bindings
00095         ImGui_ImplOpenGL3_Shutdown();
00096         ImGui_ImplGlfw_Shutdown();
00097         ImGui::DestroyContext();
00098     #endif
00099
00100     // プログラム終了時に GLFW を終了する
00101     glfwTerminate();
00102 }
00103
00104 //
00105 // マウスや矢印キーによる平行移動量を初期化する
00106 //
00107 void GgApp::Window::HumanInterface::resetTranslation()
00108 {
00109     // 平行移動量を初期化する
00110     for (auto& t : translation)
00111     {
00112         std::fill(t.begin(), t.end(), GgVector{ 0.0f, 0.0f, 0.0f, 1.0f });
00113     }
00114
00115     // 矢印キーの設定値を初期化する
00116     std::fill(arrow.begin(), arrow.end(), std::array<int, 2>{ 0, 0 });
00117
00118     // マウスホイールの回転量を初期化する
00119     std::fill(wheel.begin(), wheel.end(), 0.0f);
00120 }
00121
00122 //
00123 // 平行移動量と回転量を更新する (X, Y のみ, Z は wheel() で計算する)
00124 //
00125 void GgApp::Window::HumanInterface::calcTranslation(int button, const std::array<GLfloat, 3>&
velocity)
00126 {
00127     // マウスの相対変位
00128     assert(button >= GLFW_MOUSE_BUTTON_1 && button < GLFW_MOUSE_BUTTON_1 + GG_BUTTON_COUNT);
00129     const auto dx{ (mouse[0] - rotation[button].getStart(0)) * rotation[button].getScale(0) };
00130     const auto dy{ (rotation[button].getStart(1) - mouse[1]) * rotation[button].getScale(1) };
00131
00132     // 平行移動量
00133     auto& t{ translation[button] };
00134
00135     // 平行移動量の更新
00136     t[1][0] = dx * velocity[0] + t[0][0];
00137     t[1][1] = dy * velocity[1] + t[0][1];
00138
00139     // 回転量の更新
00140     rotation[button].motion(mouse[0], mouse[1]);
00141 }
00142
00143 //
00144 // ウィンドウのサイズ変更時の処理
00145 //
00146 void GgApp::Window::resize(GLFWwindow* window, int width, int height)
00147 {
00148     // このインスタンスの this ポインタを得る
00149     auto* const instance{ static_cast<Window*>(glfwGetWindowUserPointer(window)) };
00150
00151     if (instance)
00152     {
00153         // ウィンドウのサイズを保存する
00154         instance->size[0] = width;
00155         instance->size[1] = height;
00156
00157         // トラックボール処理の範囲を設定する
00158         for (auto& current_if : instance->interfaceData)
00159         {
00160             for (auto& t : current_if.rotation)
00161             {
00162                 t.region(width, height);
00163             }
00164         }
00165
00166         // ビューポートを更新する
00167         instance->updateViewport();

```

```

00168
00169 // ユーザー定義のコールバック関数の呼び出し
00170 if (instance->resizeFunc) (*instance->resizeFunc)(instance, width, height);
00171 }
00172 }
00173
00174 //
00175 // キーボードをタイプした時の処理
00176 //
00177 void GgApp::Window::keyboard(GLFWwindow* window, int key, int scancode, int action, int mods)
00178 {
00179 #if defined(IMGUI_VERSION)
00180 // ImGui がキーボードを使うときはキーボードの処理を行わない
00181 if (ImGui::GetIO().WantCaptureKeyboard) return;
00182 #endif
00183
00184 // このインスタンスの this ポインタを得る
00185 auto* const instance{ static_cast<Window*>(glfwGetWindowUserPointer(window)) };
00186
00187 if (instance && action)
00188 {
00189 // ユーザー定義のコールバック関数の呼び出し
00190 if (instance->keyboardFunc) (*instance->keyboardFunc)(instance, key, scancode, action, mods);
00191
00192 // 対象のユーザインタフェース
00193 auto& current_if{ instance->interfaceData[instance->interfaceNo] };
00194
00195 switch (key)
00196 {
00197 case GLFW_KEY_HOME:
00198
00199 // トラックボールを初期化する
00200 instance->resetRotation();
00201 [[fallthrough]];
00202
00203 case GLFW_KEY_END:
00204
00205 // 平行移動量を初期化する
00206 instance->resetTranslation();
00207 break;
00208
00209 case GLFW_KEY_UP:
00210
00211 if (mods & GLFW_MOD_SHIFT)
00212 current_if.arrow[1][1]++;
00213 else if (mods & GLFW_MOD_CONTROL)
00214 current_if.arrow[2][1]++;
00215 else if (mods & GLFW_MOD_ALT)
00216 current_if.arrow[3][1]++;
00217 else
00218 current_if.arrow[0][1]++;
00219 break;
00220
00221 case GLFW_KEY_DOWN:
00222
00223 if (mods & GLFW_MOD_SHIFT)
00224 current_if.arrow[1][1]--;
00225 else if (mods & GLFW_MOD_CONTROL)
00226 current_if.arrow[2][1]--;
00227 else if (mods & GLFW_MOD_ALT)
00228 current_if.arrow[3][1]--;
00229 else
00230 current_if.arrow[0][1]--;
00231 break;
00232
00233 case GLFW_KEY_RIGHT:
00234
00235 if (mods & GLFW_MOD_SHIFT)
00236 current_if.arrow[1][0]++;
00237 else if (mods & GLFW_MOD_CONTROL)
00238 current_if.arrow[2][0]++;
00239 else if (mods & GLFW_MOD_ALT)
00240 current_if.arrow[3][0]++;
00241 else
00242 current_if.arrow[0][0]++;
00243 break;
00244
00245 case GLFW_KEY_LEFT:
00246
00247 if (mods & GLFW_MOD_SHIFT)
00248 current_if.arrow[1][0]--;
00249 else if (mods & GLFW_MOD_CONTROL)
00250 current_if.arrow[2][0]--;
00251 else if (mods & GLFW_MOD_ALT)
00252 current_if.arrow[3][0]--;
00253 else
00254 current_if.arrow[0][0]--;

```

```

00255         break;
00256
00257     default:
00258         break;
00259     }
00260
00261     current_if.lastKey = key;
00262 }
00263 }
00264
00265 //
00266 // マウスボタンを操作したときの処理
00267 //
00268 void GgApp::Window::mouse(GLFWwindow* window, int button, int action, int mods)
00269 {
00270     #if defined(IMGUI_VERSION)
00271         // ImGui がマウスを使うときは Window クラスのマウス位置を更新しない
00272         if (ImGui::GetIO().WantCaptureMouse) return;
00273     #endif
00274
00275     // このインスタンスの this ポインタを得る
00276     auto* const instance{ static_cast<Window*>(glfwGetWindowUserPointer(window)) };
00277
00278     // マウスボタンの状態を記録する
00279     assert(button >= GLFW_MOUSE_BUTTON_1 && button < GLFW_MOUSE_BUTTON_1 + GG.BUTTON_COUNT);
00280     instance->status[button] = action != GLFW_RELEASE;
00281
00282     if (instance)
00283     {
00284         // ユーザー定義のコールバック関数の呼び出し
00285         if (instance->mouseFunc) (*instance->mouseFunc)(instance, button, action, mods);
00286
00287         // 対象のユーザインタフェース
00288         auto& current_if{ instance->interfaceData[instance->interfaceNo] };
00289
00290         // マウスの現在位置を得る
00291         const auto x{ current_if.mouse[0] };
00292         const auto y{ current_if.mouse[1] };
00293
00294         if (x < 0 || x >= instance->size[0] || y < 0 || y >= instance->size[1]) return;
00295
00296         if (action)
00297         {
00298             // ドラッグ開始
00299             current_if.rotation[button].begin(x, y);
00300         }
00301         else
00302         {
00303             // ドラッグ終了
00304             current_if.translation[button][0] = current_if.translation[button][1];
00305             current_if.rotation[button].end(x, y);
00306         }
00307     }
00308 }
00309
00310 //
00311 // マウスホイールを操作した時の処理
00312 //
00313 void GgApp::Window::wheel(GLFWwindow* window, double x, double y)
00314 {
00315     #if defined(IMGUI_VERSION)
00316         // ImGui がマウスを使うときは Window クラスのマウス位置を更新しない
00317         if (ImGui::GetIO().WantCaptureMouse) return;
00318     #endif
00319
00320     // このインスタンスの this ポインタを得る
00321     auto* const instance{ static_cast<Window*>(glfwGetWindowUserPointer(window)) };
00322
00323     if (instance)
00324     {
00325         // ユーザー定義のコールバック関数の呼び出し
00326         if (instance->wheelFunc) (*instance->wheelFunc)(instance, x, y);
00327
00328         // 対象のユーザインタフェース
00329         auto& current_if{ instance->interfaceData[instance->interfaceNo] };
00330
00331         // マウスホイールの回転量の保存
00332         current_if.wheel[0] += static_cast<GLfloat>(x);
00333         current_if.wheel[1] += static_cast<GLfloat>(y);
00334
00335         // マウスによる平行移動量の z 値の更新
00336         const auto z{ current_if.wheel[1] * instance->velocity[2] };
00337         for (auto& t : current_if.translation) t[1][2] = z;
00338     }
00339 }
00340
00341 //

```



```

00342 // Window クラスのコンストラクタ
00343 //
00344 GgApp::Window::Window(const std::string& title, int width, int height, int fullscreen, GLFWwindow*
share) :
00345     size{ width, height },
00346     fboSize{ width, height }
00347 {
00348     // ディスプレイの情報
00349     GLFWmonitor* monitor{ nullptr };
00350
00351     // フルスクリーン表示
00352     if (fullscreen > 0)
00353     {
00354         // 接続されているモニタの数を数える
00355         int mcount;
00356         auto** const monitors{ glfwGetMonitors(&mcount) };
00357
00358         // セカンダリモニタがあればそれを使う
00359         if (fullscreen > mcount) fullscreen = mcount;
00360         monitor = monitors[fullscreen - 1];
00361
00362         // モニタのモードを調べる
00363         const auto* mode{ glfwGetVideoMode(monitor) };
00364
00365         // ウィンドウのサイズをディスプレイのサイズにする
00366         width = mode->width;
00367         height = mode->height;
00368     }
00369
00370     // GLFW のウィンドウを作成する
00371     window = glfwCreateWindow(width, height, title.c_str(), monitor, share);
00372
00373     // ウィンドウが作成できなければエラー
00374     if (!window) throw std::runtime_error("Unable to open the GLFW window.");
00375
00376     // 現在のウィンドウを処理対象にする
00377     glfwMakeContextCurrent(window);
00378
00379     // ゲームグラフィックス特論の都合による初期化を行う
00380     ggInit();
00381
00382     // このインスタンスの this ポインタを記録しておく
00383     glfwSetWindowUserPointer(window, this);
00384
00385     // キーボードを操作した時の処理を登録する
00386     glfwSetKeyCallback(window, keyboard);
00387
00388     // マウスボタンを操作したときの処理を登録する
00389     glfwSetMouseButtonCallback(window, mouse);
00390
00391     // マウスホイール操作時に呼び出す処理を登録する
00392     glfwSetScrollCallback(window, wheel);
00393
00394     // ウィンドウのサイズ変更時に呼び出す処理を登録する
00395     glfwSetFramebufferSizeCallback(window, resize);
00396
00397     // 垂直同期タイミングに合わせる
00398     glfwSwapInterval(1);
00399
00400     // 実際のフレームバッファのサイズを取得する
00401     glfwGetFramebufferSize(window, &width, &height);
00402
00403     // ビューポートと投影変換行列を初期化する
00404     resize(window, width, height);
00405
00406 #if defined(IMGUI_VERSION)
00407     // 最初のウィンドウを開いたとき
00408     static bool firstTime{ true };
00409     if (firstTime)
00410     {
00411         // Setup Platform/Renderer bindings
00412         ImGui_ImplGlfw_InitForOpenGL(window, true);
00413         ImGui_ImplOpenGL3_Init(nullptr);
00414
00415         // 実行済みであることを記録する
00416         firstTime = false;
00417     }
00418 #endif
00419 }
00420
00421 //
00422 // Window クラスのムーブコンストラクタ
00423 //
00424 GgApp::Window::Window(Window&& w) noexcept :
00425     window{ w.window },
00426     size{ w.size },
00427     fboSize{ w.fboSize },

```

```

00428     interfaceData{ std::move(w.interfaceData) },
00429     interfaceNo{ w.interfaceNo },
00430     userPointer{ w.userPointer },
00431     resizeFunc{ w.resizeFunc },
00432     keyboardFunc{ w.keyboardFunc },
00433     mouseFunc{ w.mouseFunc },
00434     wheelFunc{ w.wheelFunc }
00435 {
00436     w.window = nullptr;
00437     if (window)
00438     {
00439         glfwSetWindowUserPointer(window, this);
00440     }
00441 }
00442
00443 //
00444 // Window クラスのムーブ代入演算子
00445 //
00446 GgApp::Window& GgApp::Window::operator=(Window&& w) noexcept
00447 {
00448     if (&w != this)
00449     {
00450         if (window)
00451         {
00452             glfwDestroyWindow(window);
00453         }
00454         window = w.window;
00455         size = w.size;
00456         fboSize = w.fboSize;
00457         interfaceData = std::move(w.interfaceData);
00458         interfaceNo = w.interfaceNo;
00459         userPointer = w.userPointer;
00460         resizeFunc = w.resizeFunc;
00461         keyboardFunc = w.keyboardFunc;
00462         mouseFunc = w.mouseFunc;
00463         wheelFunc = w.wheelFunc;
00464
00465         w.window = nullptr;
00466         if (window)
00467         {
00468             glfwSetWindowUserPointer(window, this);
00469         }
00470     }
00471     return *this;
00472 }
00473
00474 //
00475 // イベントを取得してループを継続すべきかどうか調べる
00476 //
00477 GgApp::Window::operator bool()
00478 {
00479     // イベントを取り出す
00480     glfwPollEvents();
00481
00482     // ウィンドウを閉じるべきなら false を返す
00483     if (shouldClose()) return false;
00484
00485     // 対象のユーザインタフェース
00486     auto& current_if{ interfaceData[interfaceNo] };
00487
00488 #if defined(ImGui_VERSION)
00489     // ImGui の新規フレームを作成する
00490     ImGui::ImplOpenGL3.NewFrame();
00491     ImGui::ImplGlfw.NewFrame();
00492     ImGui::NewFrame();
00493
00494     // ImGui の状態を取り出す
00495     const ImGuiIO& io{ ImGui::GetIO() };
00496
00497     // ImGui がマウスを使うときは Window クラスのマウス位置を更新しない
00498     if (io.WantCaptureMouse) return true;
00499
00500     // マウスの位置を更新する
00501     current_if.mouse = std::array<GLfloat, 2>{ io.MousePos.x, io.MousePos.y };
00502 #else
00503     // マウスの現在位置を調べる
00504     double x, y;
00505     glfwGetCursorPos(window, &x, &y);
00506
00507     // マウスの位置を更新する
00508     current_if.mouse = std::array<GLfloat, 2>{ static_cast<GLfloat>(x), static_cast<GLfloat>(y) };
00509 #endif
00510
00511     // マウスドラッグ
00512     for (int button = GLFW_MOUSE_BUTTON_1; button < GLFW_MOUSE_BUTTON_1 + GG.BUTTON_COUNT; ++button)
00513     {
00514         // マウスボタンを押していたら

```

```

00515     if (status[button])
00516     {
00517         // 現在位置と平行移動量を更新する
00518         current_if.calcTranslation(button, velocity);
00519     }
00520 }
00521
00522 return true;
00523 }
00524
00525 //
00526 // カラーバッファを入れ替える
00527 //
00528 void GgApp::Window::swapBuffers() const
00529 {
00530 #if defined(IMGUI_VERSION)
00531     // ImGui の描画データがあればフレームをレンダリングする
00532     ImGui::Render();
00533     ImDrawData* data{ ImGui::GetDrawData() };
00534     if (data) ImGui::ImplOpenGL3.RenderDrawData(data);
00535 #endif
00536
00537     // エラーチェック
00538     ggError();
00539
00540     // カラーバッファを入れ替える
00541     glfwSwapBuffers(window);
00542 }
00543
00544 //
00545 // ビューポートのサイズを更新する
00546 //
00547 void GgApp::Window::updateViewport()
00548 {
00549     // フレームバッファの大きさを求める
00550     glfwGetFramebufferSize(window, &fboSize[0], &fboSize[1]);
00551
00552 #if defined(IMGUI_VERSION)
00553     // フレームバッファの高さからメニューバーの高さを減じる
00554     fboSize[1] -= menubarHeight;
00555 #endif
00556
00557     // ウィンドウの縦横比を保存する
00558     aspect = static_cast<GLfloat>(fboSize[0]) / static_cast<GLfloat>(fboSize[1]);
00559
00560     // ビューポートを設定する
00561     restoreViewport();
00562 }
00563
00564 #if defined(GG_USE_OPENXR)
00565
00566 //
00567 // コンストラクタ
00568 //
00569 GgApp::OpenXR::OpenXR() :
00570     instance{ XR_NULL_HANDLE },
00571     systemId{ XR_NULL_SYSTEM_ID },
00572     session{ XR_NULL_HANDLE },
00573     sessionState{ XR_SESSION_STATE_UNKNOWN },
00574     appSpace{ XR_NULL_HANDLE },
00575     referenceSpaceType{ XR_REFERENCE_SPACE_TYPE_STAGE },
00576     openxrFbo{ 0, 0 },
00577     openxrDepth{ 0, 0 },
00578     frameState{ XR_TYPE_FRAME_STATE },
00579     isSessionRunning{ false },
00580     window{ nullptr },
00581     actionSet{ XR_NULL_HANDLE },
00582     aimPoseAction{ XR_NULL_HANDLE },
00583     gripPoseAction{ XR_NULL_HANDLE },
00584     triggerAction{ XR_NULL_HANDLE },
00585     gripAction{ XR_NULL_HANDLE },
00586     thumbstickAction{ XR_NULL_HANDLE },
00587     thumbstickClickAction{ XR_NULL_HANDLE },
00588     primaryButtonAction{ XR_NULL_HANDLE },
00589     secondaryButtonAction{ XR_NULL_HANDLE },
00590     menuButtonAction{ XR_NULL_HANDLE },
00591     hapticAction{ XR_NULL_HANDLE }
00592 {
00593     currentIndex[0] = 0;
00594     currentIndex[1] = 0;
00595     for (int i = 0; i < Hand::Count; ++i)
00596     {
00597         aimSpace[i] = XR_NULL_HANDLE;
00598         gripSpace[i] = XR_NULL_HANDLE;
00599         handSubactionPath[i] = XR_NULL_PATH;
00600         controllerStates[i] = ControllerState{};
00601     }

```

```

00602 }
00603
00604 //
00605 // アクションシステムを初期化する
00606 //
00607 void GgApp::OpenXR::initActions()
00608 {
00609     // アクションセットの作成
00610     XrActionSetCreateInfo actionSetInfo{ XR_TYPE_ACTION_SET_CREATE_INFO };
00611     strcpy(actionSetInfo.actionSetName, "gameplay");
00612     strcpy(actionSetInfo.localizedActionSetName, "Gameplay");
00613     actionSetInfo.priority = 0;
00614     if (XR_FAILED(xrCreateActionSet(instance, &actionSetInfo, &actionSet))) return;
00615
00616     // サブアクションパスの取得
00617     xrStringToPath(instance, "/user/hand/left", &handSubactionPath[Hand::Left]);
00618     xrStringToPath(instance, "/user/hand/right", &handSubactionPath[Hand::Right]);
00619
00620     // アクション作成ヘルパー
00621     auto createAction = [this](const char* name, const char* localizedName, XrActionType type, XrAction&
        action, bool subactions = true)
00622     {
00623         XrActionCreateInfo createInfo{ XR_TYPE_ACTION_CREATE_INFO };
00624         strcpy(createInfo.actionName, name);
00625         strcpy(createInfo.localizedActionName, localizedName);
00626         createInfo.actionType = type;
00627         if (subactions)
00628         {
00629             createInfo.countSubactionPaths = Hand::Count;
00630             createInfo.subactionPaths = handSubactionPath;
00631         }
00632         xrCreateAction(actionSet, &createInfo, &action);
00633     };
00634
00635     createAction("aim-pose", "Aim Pose", XR_ACTION_TYPE_POSE_INPUT, aimPoseAction);
00636     createAction("grip-pose", "Grip Pose", XR_ACTION_TYPE_POSE_INPUT, gripPoseAction);
00637     createAction("trigger", "Trigger", XR_ACTION_TYPE_FLOAT_INPUT, triggerAction);
00638     createAction("grip", "Grip", XR_ACTION_TYPE_FLOAT_INPUT, gripAction);
00639     createAction("thumbstick", "Thumbstick", XR_ACTION_TYPE_VECTOR2F_INPUT, thumbstickAction);
00640     createAction("thumbstick.click", "Thumbstick Click", XR_ACTION_TYPE_BOOLEAN_INPUT,
        thumbstickClickAction);
00641     createAction("primary.button", "Primary Button", XR_ACTION_TYPE_BOOLEAN_INPUT, primaryButtonAction);
00642     createAction("secondary.button", "Secondary Button", XR_ACTION_TYPE_BOOLEAN_INPUT,
        secondaryButtonAction);
00643     createAction("menu.button", "Menu Button", XR_ACTION_TYPE_BOOLEAN_INPUT, menuButtonAction);
00644     createAction("haptic", "Haptic Vibration", XR_ACTION_TYPE_VIBRATION_OUTPUT, hapticAction);
00645
00646     // アクションスペースの作成
00647     for (int i = 0; i < Hand::Count; ++i)
00648     {
00649         XrActionSpaceCreateInfo spaceInfo{ XR_TYPE_ACTION_SPACE_CREATE_INFO };
00650         spaceInfo.poseInActionSpace.orientation.w = 1.0f;
00651         spaceInfo.subactionPath = handSubactionPath[i];
00652
00653         spaceInfo.action = aimPoseAction;
00654         xrCreateActionSpace(session, &spaceInfo, &aimSpace[i]);
00655
00656         spaceInfo.action = gripPoseAction;
00657         xrCreateActionSpace(session, &spaceInfo, &gripSpace[i]);
00658     }
00659
00660     // パス文字列から XrPath を取得するヘルパー
00661     auto getPath = [this](const char* pathStr) -> XrPath
00662     {
00663         XrPath path{ XR_NULL_PATH };
00664         xrStringToPath(instance, pathStr, &path);
00665         return path;
00666     };
00667
00668     // バインディング設定ヘルパー
00669     auto suggestBindings = [this, &getPath](const char* profileStr, const
        std::vector<std::pair<XrAction, const char*>>& bindings)
00670     {
00671         XrPath profilePath = getPath(profileStr);
00672         std::vector<XrActionSuggestedBinding> suggestedBindings;
00673         suggestedBindings.reserve(bindings.size());
00674         for (const auto& [action, pathStr] : bindings)
00675         {
00676             suggestedBindings.push_back({ action, getPath(pathStr) });
00677         }
00678         XrInteractionProfileSuggestedBinding profileSuggestedBindings{
            XR_TYPE_INTERACTION_PROFILE_SUGGESTED_BINDING };
00679         profileSuggestedBindings.interactionProfile = profilePath;
00680         profileSuggestedBindings.suggestedBindings = suggestedBindings.data();
00681         profileSuggestedBindings.countSuggestedBindings = static_cast<uint32_t>(suggestedBindings.size());
00682         xrSuggestInteractionProfileBindings(instance, &profileSuggestedBindings);
00683     };

```

```

00684
00685 // Oculus Touch コントローラーのバインディング
00686 suggestBindings("/interaction_profiles/oculus/touch_controller", {
00687     { aimPoseAction, "/user/hand/left/input/aim/pose" },
00688     { aimPoseAction, "/user/hand/right/input/aim/pose" },
00689     { gripPoseAction, "/user/hand/left/input/grip/pose" },
00690     { gripPoseAction, "/user/hand/right/input/grip/pose" },
00691     { triggerAction, "/user/hand/left/input/trigger/value" },
00692     { triggerAction, "/user/hand/right/input/trigger/value" },
00693     { gripAction, "/user/hand/left/input/squeeze/value" },
00694     { gripAction, "/user/hand/right/input/squeeze/value" },
00695     { thumbstickAction, "/user/hand/left/input/thumbstick" },
00696     { thumbstickAction, "/user/hand/right/input/thumbstick" },
00697     { thumbstickClickAction, "/user/hand/left/input/thumbstick/click" },
00698     { thumbstickClickAction, "/user/hand/right/input/thumbstick/click" },
00699     { primaryButtonAction, "/user/hand/left/input/x/click" },
00700     { primaryButtonAction, "/user/hand/right/input/a/click" },
00701     { secondaryButtonAction, "/user/hand/left/input/y/click" },
00702     { secondaryButtonAction, "/user/hand/right/input/b/click" },
00703     { menuButtonAction, "/user/hand/left/input/menu/click" },
00704     { hapticAction, "/user/hand/left/output/haptic" },
00705     { hapticAction, "/user/hand/right/output/haptic" }
00706 });
00707
00708 // Simple Controller のバインディング (フォールバック用)
00709 suggestBindings("/interaction_profiles/khr/simple_controller", {
00710     { aimPoseAction, "/user/hand/left/input/aim/pose" },
00711     { aimPoseAction, "/user/hand/right/input/aim/pose" },
00712     { gripPoseAction, "/user/hand/left/input/grip/pose" },
00713     { gripPoseAction, "/user/hand/right/input/grip/pose" },
00714     { triggerAction, "/user/hand/left/input/select/click" },
00715     { triggerAction, "/user/hand/right/input/select/click" },
00716     { menuButtonAction, "/user/hand/left/input/menu/click" },
00717     { menuButtonAction, "/user/hand/right/input/menu/click" },
00718     { hapticAction, "/user/hand/left/output/haptic" },
00719     { hapticAction, "/user/hand/right/output/haptic" }
00720 });
00721
00722 // HTC Vive コントローラーのバインディング
00723 suggestBindings("/interaction_profiles/htc/vive_controller", {
00724     { aimPoseAction, "/user/hand/left/input/aim/pose" },
00725     { aimPoseAction, "/user/hand/right/input/aim/pose" },
00726     { gripPoseAction, "/user/hand/left/input/grip/pose" },
00727     { gripPoseAction, "/user/hand/right/input/grip/pose" },
00728     { triggerAction, "/user/hand/left/input/trigger/value" },
00729     { triggerAction, "/user/hand/right/input/trigger/value" },
00730     { gripAction, "/user/hand/left/input/squeeze/click" },
00731     { gripAction, "/user/hand/right/input/squeeze/click" },
00732     { thumbstickAction, "/user/hand/left/input/trackpad" },
00733     { thumbstickAction, "/user/hand/right/input/trackpad" },
00734     { thumbstickClickAction, "/user/hand/left/input/trackpad/click" },
00735     { thumbstickClickAction, "/user/hand/right/input/trackpad/click" },
00736     { menuButtonAction, "/user/hand/left/input/menu/click" },
00737     { menuButtonAction, "/user/hand/right/input/menu/click" },
00738     { hapticAction, "/user/hand/left/output/haptic" },
00739     { hapticAction, "/user/hand/right/output/haptic" }
00740 });
00741
00742 // Valve Index コントローラーのバインディング
00743 suggestBindings("/interaction_profiles/valve/index_controller", {
00744     { aimPoseAction, "/user/hand/left/input/aim/pose" },
00745     { aimPoseAction, "/user/hand/right/input/aim/pose" },
00746     { gripPoseAction, "/user/hand/left/input/grip/pose" },
00747     { gripPoseAction, "/user/hand/right/input/grip/pose" },
00748     { triggerAction, "/user/hand/left/input/trigger/value" },
00749     { triggerAction, "/user/hand/right/input/trigger/value" },
00750     { gripAction, "/user/hand/left/input/squeeze/value" },
00751     { gripAction, "/user/hand/right/input/squeeze/value" },
00752     { thumbstickAction, "/user/hand/left/input/thumbstick" },
00753     { thumbstickAction, "/user/hand/right/input/thumbstick" },
00754     { thumbstickClickAction, "/user/hand/left/input/thumbstick/click" },
00755     { thumbstickClickAction, "/user/hand/right/input/thumbstick/click" },
00756     { primaryButtonAction, "/user/hand/left/input/a/click" },
00757     { primaryButtonAction, "/user/hand/right/input/a/click" },
00758     { secondaryButtonAction, "/user/hand/left/input/b/click" },
00759     { secondaryButtonAction, "/user/hand/right/input/b/click" },
00760     { hapticAction, "/user/hand/left/output/haptic" },
00761     { hapticAction, "/user/hand/right/output/haptic" }
00762 });
00763
00764 // セッションにアクションセットをアタッチ
00765 XrSessionActionSetsAttachInfo attachInfo{ XR_TYPE_SESSION_ACTION_SETS_ATTACH_INFO };
00766 attachInfo.countActionSets = 1;
00767 attachInfo.actionSets = &actionSet;
00768 xrAttachSessionActionSets(session, &attachInfo);
00769 }
00770

```

```

00771 //
00772 // アクション状態を更新する
00773 //
00774 void GgApp::OpenXR::pollActions()
00775 {
00776     if (!isSessionRunning || sessionState < XR_SESSION_STATE_VISIBLE || actionSet == XR_NULL_HANDLE)
00777         return;
00778     XrActiveActionSet activeActionSet{ actionSet, XR_NULL_PATH };
00779     XrActionsSyncInfo syncInfo{ XR_TYPE_ACTIONS_SYNC_INFO };
00780     syncInfo.countActiveActionSets = 1;
00781     syncInfo.activeActionSets = &activeActionSet;
00782     if (XR_FAILED(xrSyncActions(session, &syncInfo))) return;
00783
00784     for (int i = 0; i < Hand::Count; ++i)
00785     {
00786         auto& state = controllerStates[i];
00787         XrPath subaction = handSubactionPath[i];
00788
00789         // グリップポーズの取得
00790         XrActionStateGetInfo getInfo{ XR_TYPE_ACTION_STATE_GET_INFO };
00791         getInfo.subactionPath = subaction;
00792
00793         XrActionStatePose gripPoseState{ XR_TYPE_ACTION_STATE_POSE };
00794         getInfo.action = gripPoseAction;
00795         xrGetActionStatePose(session, &getInfo, &gripPoseState);
00796         state.isTracked = gripPoseState.isActive;
00797
00798         if (state.isTracked && gripSpace[i] != XR_NULL_HANDLE)
00799         {
00800             XrSpaceLocation location{ XR_TYPE_SPACE_LOCATION };
00801             xrLocateSpace(gripSpace[i], appSpace, frameState.predictedDisplayTime, &location);
00802             if (location.locationFlags & XR_SPACE_LOCATION_POSITION_VALID_BIT)
00803             {
00804                 state.gripPose = location.pose;
00805             }
00806         }
00807
00808         // エイムポーズの取得
00809         if (state.isTracked && aimSpace[i] != XR_NULL_HANDLE)
00810         {
00811             XrSpaceLocation location{ XR_TYPE_SPACE_LOCATION };
00812             xrLocateSpace(aimSpace[i], appSpace, frameState.predictedDisplayTime, &location);
00813             if (location.locationFlags & XR_SPACE_LOCATION_POSITION_VALID_BIT)
00814             {
00815                 state.aimPose = location.pose;
00816             }
00817         }
00818
00819         // トリガー
00820         XrActionStateFloat triggerState{ XR_TYPE_ACTION_STATE_FLOAT };
00821         getInfo.action = triggerAction;
00822         if (XR_SUCCEEDED(xrGetActionStateFloat(session, &getInfo, &triggerState)) &&
00823             triggerState.isActive)
00824             state.trigger = triggerState.currentState;
00825         else
00826             state.trigger = 0.0f;
00827
00828         // グリップ
00829         XrActionStateFloat gripState{ XR_TYPE_ACTION_STATE_FLOAT };
00830         getInfo.action = gripAction;
00831         if (XR_SUCCEEDED(xrGetActionStateFloat(session, &getInfo, &gripState)) && gripState.isActive)
00832             state.grip = gripState.currentState;
00833         else
00834             state.grip = 0.0f;
00835
00836         // スティック
00837         XrActionStateVector2f thumbstickState{ XR_TYPE_ACTION_STATE_VECTOR2F };
00838         getInfo.action = thumbstickAction;
00839         if (XR_SUCCEEDED(xrGetActionStateVector2f(session, &getInfo, &thumbstickState)) &&
00840             thumbstickState.isActive)
00841             state.thumbstick = { thumbstickState.currentState.x, thumbstickState.currentState.y };
00842         else
00843             state.thumbstick = { 0.0f, 0.0f };
00844
00845         // スティッククリック
00846         XrActionStateBoolean clickState{ XR_TYPE_ACTION_STATE_BOOLEAN };
00847         getInfo.action = thumbstickClickAction;
00848         if (XR_SUCCEEDED(xrGetActionStateBoolean(session, &getInfo, &clickState)) && clickState.isActive)
00849             state.thumbstickClick = clickState.currentState != XR_FALSE;
00850         else
00851             state.thumbstickClick = false;
00852
00853         // プライマリボタン
00854         XrActionStateBoolean primState{ XR_TYPE_ACTION_STATE_BOOLEAN };
00855         getInfo.action = primaryButtonAction;
00856         if (XR_SUCCEEDED(xrGetActionStateBoolean(session, &getInfo, &primState)) && primState.isActive)

```

```

00855     state.primaryButton = primState.currentState != XR_FALSE;
00856 else
00857     state.primaryButton = false;
00858
00859 // セカンダリボタン
00860 XrActionStateBoolean secState{ XR_TYPE_ACTION_STATE_BOOLEAN };
00861 getInfo.action = secondaryButtonAction;
00862 if (XR_SUCCEEDED(xrGetActionStateBoolean(session, &getInfo, &secState)) && secState.isActive)
00863     state.secondaryButton = secState.currentState != XR_FALSE;
00864 else
00865     state.secondaryButton = false;
00866
00867 // メニューボタン
00868 XrActionStateBoolean menuState{ XR_TYPE_ACTION_STATE_BOOLEAN };
00869 getInfo.action = menuButtonAction;
00870 if (XR_SUCCEEDED(xrGetActionStateBoolean(session, &getInfo, &menuState)) && menuState.isActive)
00871     state.menuButton = menuState.currentState != XR_FALSE;
00872 else
00873     state.menuButton = false;
00874 }
00875 }
00876
00877 //
00878 // OpenXR のセッションを作成する
00879 //
00880 GgApp::OpenXR& GgApp::OpenXR::initialize(const Window& window, XrReferenceSpaceType spaceType)
00881 {
00882     static OpenXR openxr;
00883
00884     if (openxr.instance != XR_NULL_HANDLE) return openxr;
00885
00886     openxr.window = &window;
00887     openxr.referenceSpaceType = spaceType;
00888
00889     // XrInstance の作成
00890     XrInstanceCreateInfo createInfo{ XR_TYPE_INSTANCE_CREATE_INFO };
00891     strcpy(createInfo.applicationInfo.applicationName, "ggsample01");
00892     createInfo.applicationInfo.applicationVersion = 1;
00893     strcpy(createInfo.applicationInfo.engineName, "GgApp");
00894     createInfo.applicationInfo.engineVersion = 1;
00895     createInfo.applicationInfo.apiVersion = XR_CURRENT_API_VERSION;
00896
00897     const char* extensions[] = {
00898         XR_KHR_OPENGL_ENABLE_EXTENSION_NAME
00899     };
00900     createInfo.enabledExtensionCount = 1;
00901     createInfo.enabledExtensionNames = extensions;
00902
00903     if (XR_FAILED(xrCreateInstance(&createInfo, &openxr.instance)))
00904         throw std::runtime_error("Can't create OpenXR instance");
00905
00906     // Get System Id
00907     XrSystemGetInfo systemInfo{ XR_TYPE_SYSTEM_GET_INFO };
00908     systemInfo.formFactor = XR_FORM_FACTOR_HEAD_MOUNTED_DISPLAY;
00909     if (XR_FAILED(xrGetSystem(openxr.instance, &systemInfo, &openxr.systemId)))
00910         throw std::runtime_error("Can't get OpenXR system");
00911
00912     // Require OpenGL graphics binding
00913     PFN_xrGetOpenGLGraphicsRequirementsKHR pfnGetOpenGLGraphicsRequirementsKHR = nullptr;
00914     xrGetInstanceProcAddr(openxr.instance, "xrGetOpenGLGraphicsRequirementsKHR",
00915         (PFN_xrVoidFunction*)&pfnGetOpenGLGraphicsRequirementsKHR);
00916
00917     XrGraphicsRequirementsOpenGDKHR graphicsRequirements{ XR_TYPE_GRAPHICS_REQUIREMENTS_OPENGLKHR };
00918     if (pfnGetOpenGLGraphicsRequirementsKHR)
00919     {
00919         pfnGetOpenGLGraphicsRequirementsKHR(openxr.instance, openxr.systemId, &graphicsRequirements);
00920     }
00921
00922 #if defined(_WIN32)
00923     XrGraphicsBindingOpenGLWin32KHR graphicsBinding{ XR_TYPE_GRAPHICS_BINDING_OPENGL_WIN32_KHR };
00924     graphicsBinding.hDC = wglGetCurrentDC();
00925     graphicsBinding.hGLRC = wglGetCurrentContext();
00926 #else
00927     XrGraphicsBindingOpenGLXlibKHR graphicsBinding{ XR_TYPE_GRAPHICS_BINDING_OPENGL_XLIB_KHR };
00928     graphicsBinding.xDisplay = glXGetCurrentDisplay();
00929     graphicsBinding.visualid = 0;
00930     graphicsBinding.glxFBCfg = nullptr;
00931     graphicsBinding.glXDrawable = glXGetCurrentDrawable();
00932     graphicsBinding.glXContext = glXGetCurrentContext();
00933 #endif
00934
00935     // Create session
00936     XrSessionCreateInfo sessionCreateInfo{ XR_TYPE_SESSION_CREATE_INFO };
00937     sessionCreateInfo.next = &graphicsBinding;
00938     sessionCreateInfo.systemId = openxr.systemId;
00939     if (XR_FAILED(xrCreateSession(openxr.instance, &sessionCreateInfo, &openxr.session)))
00940         throw std::runtime_error("Can't create OpenXR session");

```



```

00941
00942 // Create Reference Space (要求されたスペースタイプを作成、失敗した場合は LOCAL にフォールバック)
00943 XrReferenceSpaceCreateInfo spaceCreateInfo{ XR_TYPE_REFERENCE_SPACE_CREATE_INFO };
00944 spaceCreateInfo.referenceSpaceType = openxr.referenceSpaceType;
00945 spaceCreateInfo.poseInReferenceSpace.orientation.w = 1.0f;
00946 if (XR_FAILED(xrCreateReferenceSpace(openxr.session, &spaceCreateInfo, &openxr.appSpace)))
00947 {
00948     openxr.referenceSpaceType = XR_REFERENCE_SPACE_TYPE_LOCAL;
00949     spaceCreateInfo.referenceSpaceType = XR_REFERENCE_SPACE_TYPE_LOCAL;
00950     if (XR_FAILED(xrCreateReferenceSpace(openxr.session, &spaceCreateInfo, &openxr.appSpace)))
00951         throw std::runtime_error("Can't create OpenXR reference space");
00952 }
00953
00954 // アクションシステムの初期化
00955 openxr.initActions();
00956
00957 // Enumerate views and create swapchains
00958 uint32_t viewCount = 0;
00959 xrEnumerateViewConfigurationViews(openxr.instance, openxr.systemId,
XR_VIEW_CONFIGURATION_TYPE_PRIMARY_STEREO, 0, &viewCount, nullptr);
00960 openxr.views.resize(viewCount, { XR_TYPE_VIEW_CONFIGURATION_VIEW });
00961 xrEnumerateViewConfigurationViews(openxr.instance, openxr.systemId,
XR_VIEW_CONFIGURATION_TYPE_PRIMARY_STEREO, viewCount, &viewCount, openxr.views.data());
00962
00963 openxr.viewStates.resize(viewCount, { XR_TYPE_VIEW });
00964
00965 glGenFramebuffers(2, openxr.openxrFbo);
00966 glGenTextures(2, openxr.openxrDepth);
00967
00968 for (uint32_t i = 0; i < viewCount; ++i)
00969 {
00970     XrSwapchainCreateInfo swapchainCreateInfo{ XR_TYPE_SWAPCHAIN_CREATE_INFO };
00971     swapchainCreateInfo.arraySize = 1;
00972     swapchainCreateInfo.format = GL_SRGB8_ALPHA8;
00973     swapchainCreateInfo.width = openxr.views[i].recommendedImageRectWidth;
00974     swapchainCreateInfo.height = openxr.views[i].recommendedImageRectHeight;
00975     swapchainCreateInfo.mipCount = 1;
00976     swapchainCreateInfo.faceCount = 1;
00977     swapchainCreateInfo.sampleCount = 1;
00978     swapchainCreateInfo.usageFlags = XR_SWAPCHAIN_USAGE_COLOR_ATTACHMENT_BIT;
00979
00980     XrSwapchain swapchain;
00981     xrCreateSwapchain(openxr.session, &swapchainCreateInfo, &swapchain);
00982     openxr.swapchains.push_back(swapchain);
00983
00984     uint32_t imageCount;
00985     xrEnumerateSwapchainImages(swapchain, 0, &imageCount, nullptr);
00986     std::vector<XrSwapchainImageOpenGLKhr> images(imageCount, { XR_TYPE_SWAPCHAIN_IMAGE_OPENGL_KHR });
00987     xrEnumerateSwapchainImages(swapchain, imageCount, &imageCount,
(XrSwapchainImageBaseHeader*)images.data());
00988     openxr.swapchainImages.push_back(std::move(images));
00989
00990     glBindTexture(GL_TEXTURE_2D, openxr.openxrDepth[i]);
00991     glTexImage2D(GL_TEXTURE_2D, 0, GL_DEPTH_COMPONENT_32F, swapchainCreateInfo.width,
swapchainCreateInfo.height, 0, GL_DEPTH_COMPONENT, GL_FLOAT, NULL);
00992 }
00993
00994 // OpenXR にレンダリングするときは sRGB カラースペースを使う
00995 glEnable(GL_FRAMEBUFFER_SRGB);
00996 glDrawBuffer(GL_FRONT);
00997 glfSwapInterval(0);
00998
00999 return openxr;
01000 }
01001
01002 //
01003 // OpenXR のセッションを破棄する
01004 //
01005 void GgApp::OpenXR::terminate()
01006 {
01007     for (int i = 0; i < Hand::Count; ++i)
01008     {
01009         if (aimSpace[i] != XR_NULL_HANDLE) { xrDestroySpace(aimSpace[i]); aimSpace[i] = XR_NULL_HANDLE; }
01010         if (gripSpace[i] != XR_NULL_HANDLE) { xrDestroySpace(gripSpace[i]); gripSpace[i] = XR_NULL_HANDLE; }
01011     }
01012
01013     if (actionSet != XR_NULL_HANDLE)
01014     {
01015         if (aimPoseAction != XR_NULL_HANDLE) xrDestroyAction(aimPoseAction);
01016         if (gripPoseAction != XR_NULL_HANDLE) xrDestroyAction(gripPoseAction);
01017         if (triggerAction != XR_NULL_HANDLE) xrDestroyAction(triggerAction);
01018         if (gripAction != XR_NULL_HANDLE) xrDestroyAction(gripAction);
01019         if (thumbstickAction != XR_NULL_HANDLE) xrDestroyAction(thumbstickAction);
01020         if (thumbstickClickAction != XR_NULL_HANDLE) xrDestroyAction(thumbstickClickAction);
01021         if (primaryButtonAction != XR_NULL_HANDLE) xrDestroyAction(primaryButtonAction);
01022         if (secondaryButtonAction != XR_NULL_HANDLE) xrDestroyAction(secondaryButtonAction);
01023         if (menuButtonAction != XR_NULL_HANDLE) xrDestroyAction(menuButtonAction);

```



```

01024     if (hapticAction != XR_NULL_HANDLE) xrDestroyAction(hapticAction);
01025     xrDestroyActionSet(actionSet);
01026     actionSet = XR_NULL_HANDLE;
01027 }
01028
01029 if (session != XR_NULL_HANDLE)
01030 {
01031     for (auto swapchain : swapchains) xrDestroySwapchain(swapchain);
01032     swapchains.clear();
01033     swapchainImages.clear();
01034     if (appSpace != XR_NULL_HANDLE) { xrDestroySpace(appSpace); appSpace = XR_NULL_HANDLE; }
01035     xrDestroySession(session);
01036     session = XR_NULL_HANDLE;
01037 }
01038 if (instance != XR_NULL_HANDLE)
01039 {
01040     xrDestroyInstance(instance);
01041     instance = XR_NULL_HANDLE;
01042 }
01043 glDeleteFramebuffers(2, openxrFbo);
01044 glDeleteTextures(2, openxrDepth);
01045
01046 glDisable(GL_FRAMEBUFFER_SRGB);
01047 glDrawBuffer(GL_BACK);
01048 glfwSwapInterval(1);
01049 }
01050
01051 //
01052 // OpenXR による描画開始
01053 //
01054 bool GgApp::OpenXR::begin()
01055 {
01056     XrEventDataBuffer eventData{ XR_TYPE_EVENT_DATA_BUFFER };
01057     while (xrPollEvent(instance, &eventData) == XR_SUCCESS)
01058     {
01059         if (eventData.type == XR_TYPE_EVENT_DATA_SESSION_STATE_CHANGED)
01060         {
01061             auto* stateChanged = reinterpret_cast<XrEventDataSessionStateChanged*>(&eventData);
01062             sessionState = stateChanged->state;
01063             if (sessionState == XR_SESSION_STATE_READY)
01064             {
01065                 XrSessionBeginInfo beginInfo{ XR_TYPE_SESSION_BEGIN_INFO };
01066                 beginInfo.primaryViewConfigurationType = XR_VIEW_CONFIGURATION_TYPE_PRIMARY_STEREO;
01067                 xrBeginSession(session, &beginInfo);
01068                 isSessionRunning = true;
01069             }
01070             else if (sessionState == XR_SESSION_STATE_STOPPING)
01071             {
01072                 xrEndSession(session);
01073                 isSessionRunning = false;
01074             }
01075             else if (sessionState == XR_SESSION_STATE_EXITING || sessionState == XR_SESSION_STATE_LOSS_PENDING)
01076             {
01077                 if (window) window->setClose(GLFW_TRUE);
01078             }
01079         }
01080         eventData.type = XR_TYPE_EVENT_DATA_BUFFER;
01081     }
01082
01083     if (!isSessionRunning || sessionState < XR_SESSION_STATE_VISIBLE) return false;
01084
01085     XrFrameWaitInfo waitInfo{ XR_TYPE_FRAME_WAIT_INFO };
01086     frameState = { XR_TYPE_FRAME_STATE };
01087     xrWaitFrame(session, &waitInfo, &frameState);
01088
01089     XrFrameBeginInfo beginInfo{ XR_TYPE_FRAME_BEGIN_INFO };
01090     xrBeginFrame(session, &beginInfo);
01091
01092     if (frameState.shouldRender)
01093     {
01094         XrViewLocateInfo viewLocateInfo{ XR_TYPE_VIEW_LOCATE_INFO };
01095         viewLocateInfo.viewConfigurationType = XR_VIEW_CONFIGURATION_TYPE_PRIMARY_STEREO;
01096         viewLocateInfo.displayTime = frameState.predictedDisplayTime;
01097         viewLocateInfo.space = appSpace;
01098
01099         XrViewState viewState{ XR_TYPE_VIEW_STATE };
01100         uint32_t viewCount;
01101         xrLocateViews(session, &viewLocateInfo, &viewState, static_cast<uint32_t>(viewStates.size()),
01102             &viewCount, viewStates.data());
01103
01104         pollActions();
01105         return true;
01106     }
01107     return false;
01108 }
01109

```

```

01110 //
01111 // 描画対象の目を指定してフレームバッファとビューポートを設定する
01112 //
01113 void GgApp::OpenXR::select(int eye)
01114 {
01115     XrSwapchainImageAcquireInfo acquireInfo{ XR_TYPE_SWAPCHAIN_IMAGE_ACQUIRE_INFO };
01116     xrAcquireSwapchainImage(swapchains[eye], &acquireInfo, &currentImageIndex[eye]);
01117
01118     XrSwapchainImageWaitInfo waitInfo{ XR_TYPE_SWAPCHAIN_IMAGE_WAIT_INFO };
01119     waitInfo.timeout = XR_INFINITE_DURATION;
01120     xrWaitSwapchainImage(swapchains[eye], &waitInfo);
01121
01122     GLuint texId = swapchainImages[eye][currentImageIndex[eye]].image;
01123     glBindFramebuffer(GL_FRAMEBUFFER, openxrFbo[eye]);
01124     glFramebufferTexture2D(GL_FRAMEBUFFER, GL_COLOR_ATTACHMENT0, GL_TEXTURE_2D, texId, 0);
01125     glFramebufferTexture2D(GL_FRAMEBUFFER, GL_DEPTH_ATTACHMENT, GL_TEXTURE_2D, openxrDepth[eye], 0);
01126
01127     glViewport(0, 0, views[eye].recommendedImageRectWidth, views[eye].recommendedImageRectHeight);
01128 }
01129
01130 //
01131 // 描画対象の目を指定する (旧 LibOVR 仕様互換)
01132 //
01133 void GgApp::OpenXR::select(int eye, GLfloat* screen, GLfloat* position, GLfloat* orientation)
01134 {
01135     select(eye);
01136
01137     const auto& pose = viewStates[eye].pose;
01138     const auto& fov = viewStates[eye].fov;
01139
01140     screen[0] = tanf(fov.angleLeft);
01141     screen[1] = tanf(fov.angleRight);
01142     screen[2] = tanf(fov.angleDown);
01143     screen[3] = tanf(fov.angleUp);
01144
01145     position[0] = pose.position.x;
01146     position[1] = pose.position.y;
01147     position[2] = pose.position.z;
01148
01149     orientation[0] = pose.orientation.x;
01150     orientation[1] = pose.orientation.y;
01151     orientation[2] = pose.orientation.z;
01152     orientation[3] = pose.orientation.w;
01153 }
01154
01155 //
01156 // 描画した目のスワップチェーンイメージを解放する
01157 //
01158 void GgApp::OpenXR::commit(int eye)
01159 {
01160     glBindFramebuffer(GL_FRAMEBUFFER, openxrFbo[eye]);
01161     glFramebufferTexture2D(GL_FRAMEBUFFER, GL_COLOR_ATTACHMENT0, GL_TEXTURE_2D, 0, 0);
01162
01163     XrSwapchainImageReleaseInfo releaseInfo{ XR_TYPE_SWAPCHAIN_IMAGE_RELEASE_INFO };
01164     xrReleaseSwapchainImage(swapchains[eye], &releaseInfo);
01165 }
01166
01167 //
01168 // フレームを転送して HMD に表示する
01169 //
01170 bool GgApp::OpenXR::submit(bool mirror)
01171 {
01172     if (frameState.shouldRender)
01173     {
01174         XrCompositionLayerProjectionView projectionViews[2];
01175         for (int eye = 0; eye < 2; ++eye)
01176         {
01177             projectionViews[eye].type = XR_TYPE_COMPOSITION_LAYER_PROJECTION_VIEW;
01178             projectionViews[eye].next = nullptr;
01179             projectionViews[eye].pose = viewStates[eye].pose;
01180             projectionViews[eye].fov = viewStates[eye].fov;
01181             projectionViews[eye].subImage.swapchain = swapchains[eye];
01182             projectionViews[eye].subImage.imageRect.offset = { 0, 0 };
01183             projectionViews[eye].subImage.imageRect.extent = {
01184                 static_cast<int32_t>(views[eye].recommendedImageRectWidth),
01185                 static_cast<int32_t>(views[eye].recommendedImageRectHeight)
01186             };
01187             projectionViews[eye].subImage.imageArrayIndex = 0;
01188         }
01189
01190         XrCompositionLayerProjection layer{ XR_TYPE_COMPOSITION_LAYER_PROJECTION };
01191         layer.space = appSpace;
01192         layer.viewCount = 2;
01193         layer.views = projectionViews;
01194
01195         const XrCompositionLayerBaseHeader* layers =
01196             reinterpret_cast<XrCompositionLayerBaseHeader*>(&layer);

```

```

01196
01197     XrFrameEndInfo endInfo{ XR_TYPE_FRAME_END_INFO };
01198     endInfo.displayTime = frameState.predictedDisplayTime;
01199     endInfo.environmentBlendMode = XR_ENVIRONMENT_BLEND_MODE_OPAQUE;
01200     endInfo.layerCount = 1;
01201     endInfo.layers = &layers;
01202
01203     xrEndFrame(session, &endInfo);
01204
01205     if (mirror && window)
01206     {
01207         GLsizei size[2];
01208         window->getSize(size);
01209         glBindFramebuffer(GL_READ_FRAMEBUFFER, openxrFbo[0]);
01210         glBindFramebuffer(GL_DRAW_FRAMEBUFFER, 0);
01211         glBlitFramebuffer(0, 0, views[0].recommendedImageRectWidth, views[0].recommendedImageRectHeight,
01212             0, 0, size[0], size[1], GL_COLOR_BUFFER_BIT, GL_NEAREST);
01213         glBindFramebuffer(GL_READ_FRAMEBUFFER, 0);
01214         glFlush();
01215     }
01216 }
01217 else
01218 {
01219     XrFrameEndInfo endInfo{ XR_TYPE_FRAME_END_INFO };
01220     endInfo.displayTime = frameState.predictedDisplayTime;
01221     endInfo.environmentBlendMode = XR_ENVIRONMENT_BLEND_MODE_OPAQUE;
01222     endInfo.layerCount = 0;
01223     endInfo.layers = nullptr;
01224
01225     xrEndFrame(session, &endInfo);
01226 }
01227
01228 return true;
01229 }
01230
01231 //
01232 // 指定した目の透視投影変換行列を取得する
01233 //
01234 GgMatrix GgApp::OpenXR::getProjectionMatrix(int eye, GLfloat zNear, GLfloat zFar) const
01235 {
01236     assert(eye >= 0 && eye < static_cast<int>(viewStates.size()));
01237     const auto& fov{ viewStates[eye].fov };
01238     const GLfloat l{ tanf(fov.angleLeft) * zNear };
01239     const GLfloat r{ tanf(fov.angleRight) * zNear };
01240     const GLfloat b{ tanf(fov.angleDown) * zNear };
01241     const GLfloat t{ tanf(fov.angleUp) * zNear };
01242     return ggFrustum(l, r, b, t, zNear, zFar);
01243 }
01244
01245 //
01246 // 指定した目のビュー変換行列を取得する
01247 //
01248 GgMatrix GgApp::OpenXR::getViewMatrix(int eye) const
01249 {
01250     assert(eye >= 0 && eye < static_cast<int>(viewStates.size()));
01251     const auto& pose{ viewStates[eye].pose };
01252     const GgQuaternion q{ pose.orientation.x, pose.orientation.y, pose.orientation.z, pose.orientation.w };
01253     return q.getConjugateMatrix() * ggTranslate(-pose.position.x, -pose.position.y, -pose.position.z);
01254 }
01255
01256 //
01257 // 指定した目の姿勢行列を取得する
01258 //
01259 GgMatrix GgApp::OpenXR::getPoseMatrix(int eye) const
01260 {
01261     assert(eye >= 0 && eye < static_cast<int>(viewStates.size()));
01262     const auto& pose{ viewStates[eye].pose };
01263     const GgQuaternion q{ pose.orientation.x, pose.orientation.y, pose.orientation.z, pose.orientation.w };
01264     return ggTranslate(pose.position.x, pose.position.y, pose.position.z) * q.getMatrix();
01265 }
01266
01267 //
01268 // 指定した目の視点位置を取得する
01269 //
01270 GgVector GgApp::OpenXR::getPosition(int eye) const
01271 {
01272     assert(eye >= 0 && eye < static_cast<int>(viewStates.size()));
01273     const auto& pos{ viewStates[eye].pose.position };
01274     return GgVector{ pos.x, pos.y, pos.z, 1.0f };
01275 }
01276
01277 //
01278 // 指定した目の視線方向の回転四元数を取得する
01279 //
01280 GgQuaternion GgApp::OpenXR::getOrientation(int eye) const

```

```

01281 {
01282     assert(eye >= 0 && eye < static_cast<int>(viewStates.size()));
01283     const auto& ori{ viewStates[eye].pose.orientation };
01284     return GgQuaternion{ ori.x, ori.y, ori.z, ori.w };
01285 }
01286
01287 //
01288 // 指定した目の視野角情報 (XrFovf) を取得する
01289 //
01290 const XrFovf& GgApp::OpenXR::getFov(int eye) const
01291 {
01292     assert(eye >= 0 && eye < static_cast<int>(viewStates.size()));
01293     return viewStates[eye].fov;
01294 }
01295
01296 //
01297 // 指定した目の姿勢情報 (XrPosef) を取得する
01298 //
01299 const XrPosef& GgApp::OpenXR::getPose(int eye) const
01300 {
01301     assert(eye >= 0 && eye < static_cast<int>(viewStates.size()));
01302     return viewStates[eye].pose;
01303 }
01304
01305 //
01306 // レンダリング推奨解像度の横幅を取得する
01307 //
01308 GLsizei GgApp::OpenXR::getWidth(int eye) const
01309 {
01310     assert(eye >= 0 && eye < static_cast<int>(views.size()));
01311     return views[eye].recommendedImageRectWidth;
01312 }
01313
01314 //
01315 // レンダリング推奨解像度の高さを取得する
01316 //
01317 GLsizei GgApp::OpenXR::getHeight(int eye) const
01318 {
01319     assert(eye >= 0 && eye < static_cast<int>(views.size()));
01320     return views[eye].recommendedImageRectHeight;
01321 }
01322
01323 //
01324 // アスペクト比を取得する
01325 //
01326 GLfloat GgApp::OpenXR::getAspect(int eye) const
01327 {
01328     assert(eye >= 0 && eye < static_cast<int>(views.size()));
01329     return static_cast<GLfloat>(views[eye].recommendedImageRectWidth) /
01330         static_cast<GLfloat>(views[eye].recommendedImageRectHeight);
01331 }
01332
01333 //
01334 // ビューの総数を取得する
01335 //
01336 uint32_t GgApp::OpenXR::getViewCount() const
01337 {
01338     return static_cast<uint32_t>(views.size());
01339 }
01340
01341 //
01342 // 現在の参照空間タイプを取得する
01343 //
01344 XrReferenceSpaceType GgApp::OpenXR::getReferenceSpaceType() const
01345 {
01346     return referenceSpaceType;
01347 }
01348
01349 //
01350 // コントローラーがトラッキングされているか取得する
01351 //
01352 bool GgApp::OpenXR::isTracked(int hand) const
01353 {
01354     assert(hand >= 0 && hand < Hand::Count);
01355     return controllerStates[hand].isTracked;
01356 }
01357
01358 //
01359 // コントローラーのグリップ変換行列を取得する
01360 //
01361 GgMatrix GgApp::OpenXR::getGripMatrix(int hand) const
01362 {
01363     assert(hand >= 0 && hand < Hand::Count);
01364     const auto& pose = controllerStates[hand].gripPose;
01365     const GgQuaternion q{ pose.orientation.x, pose.orientation.y, pose.orientation.z, pose.orientation.w };
01366     return ggTranslate(pose.position.x, pose.position.y, pose.position.z) * q.getMatrix();
01367 }

```

```

01366 }
01367
01368 //
01369 // コントローラーのエイム変換行列を取得する
01370 //
01371 GgMatrix GgApp::OpenXR::getAimMatrix(int hand) const
01372 {
01373     assert(hand >= 0 && hand < Hand::Count);
01374     const auto& pose = controllerStates[hand].aimPose;
01375     const GgQuaternion q{ pose.orientation.x, pose.orientation.y, pose.orientation.z, pose.orientation.w
01376 };
01377     return ggTranslate(pose.position.x, pose.position.y, pose.position.z) * q.getMatrix();
01378 }
01379 //
01380 // コントローラーのグリップ位置を取得する
01381 //
01382 GgVector GgApp::OpenXR::getGripPosition(int hand) const
01383 {
01384     assert(hand >= 0 && hand < Hand::Count);
01385     const auto& pos = controllerStates[hand].gripPose.position;
01386     return GgVector{ pos.x, pos.y, pos.z, 1.0f };
01387 }
01388 //
01389 // コントローラーのグリップ回転四元数を取得する
01390 //
01391 //
01392 GgQuaternion GgApp::OpenXR::getGripOrientation(int hand) const
01393 {
01394     assert(hand >= 0 && hand < Hand::Count);
01395     const auto& ori = controllerStates[hand].gripPose.orientation;
01396     return GgQuaternion{ ori.x, ori.y, ori.z, ori.w };
01397 }
01398 //
01399 // コントローラーのエイム位置を取得する
01400 //
01401 //
01402 GgVector GgApp::OpenXR::getAimPosition(int hand) const
01403 {
01404     assert(hand >= 0 && hand < Hand::Count);
01405     const auto& pos = controllerStates[hand].aimPose.position;
01406     return GgVector{ pos.x, pos.y, pos.z, 1.0f };
01407 }
01408 //
01409 // コントローラーのエイム回転四元数を取得する
01410 //
01411 //
01412 GgQuaternion GgApp::OpenXR::getAimOrientation(int hand) const
01413 {
01414     assert(hand >= 0 && hand < Hand::Count);
01415     const auto& ori = controllerStates[hand].aimPose.orientation;
01416     return GgQuaternion{ ori.x, ori.y, ori.z, ori.w };
01417 }
01418 //
01419 // トリガーの押し込み量を取得する
01420 //
01421 //
01422 float GgApp::OpenXR::getTrigger(int hand) const
01423 {
01424     assert(hand >= 0 && hand < Hand::Count);
01425     return controllerStates[hand].trigger;
01426 }
01427 //
01428 // グリップの押し込み量を取得する
01429 //
01430 //
01431 float GgApp::OpenXR::getGrip(int hand) const
01432 {
01433     assert(hand >= 0 && hand < Hand::Count);
01434     return controllerStates[hand].grip;
01435 }
01436 //
01437 // アナログスティックの入力値を取得する
01438 //
01439 //
01440 std::array<float, 2> GgApp::OpenXR::getThumbstick(int hand) const
01441 {
01442     assert(hand >= 0 && hand < Hand::Count);
01443     return controllerStates[hand].thumbstick;
01444 }
01445 //
01446 // アナログスティックのクリック状態を取得する
01447 //
01448 //
01449 bool GgApp::OpenXR::getThumbstickClick(int hand) const
01450 {
01451     assert(hand >= 0 && hand < Hand::Count);

```

```

01452     return controllerStates[hand].thumbstickClick;
01453 }
01454
01455 //
01456 // プライマリボタンの押下状態を取得する
01457 //
01458 bool GgApp::OpenXR::getPrimaryButton(int hand) const
01459 {
01460     assert(hand >= 0 && hand < Hand::Count);
01461     return controllerStates[hand].primaryButton;
01462 }
01463
01464 //
01465 // セカンダリボタンの押下状態を取得する
01466 //
01467 bool GgApp::OpenXR::getSecondaryButton(int hand) const
01468 {
01469     assert(hand >= 0 && hand < Hand::Count);
01470     return controllerStates[hand].secondaryButton;
01471 }
01472
01473 //
01474 // メニューボタンの押下状態を取得する
01475 //
01476 bool GgApp::OpenXR::getMenuButton(int hand) const
01477 {
01478     assert(hand >= 0 && hand < Hand::Count);
01479     return controllerStates[hand].menuButton;
01480 }
01481
01482 //
01483 // コントローラーに振動を出力する
01484 //
01485 void GgApp::OpenXR::applyHapticVibration(int hand, float durationSeconds, float frequency, float
    amplitude)
01486 {
01487     assert(hand >= 0 && hand < Hand::Count);
01488     if (session == XR_NULL_HANDLE || hapticAction == XR_NULL_HANDLE) return;
01489
01490     XrHapticVibration vibration{ XR_TYPE_HAPTIC_VIBRATION };
01491     vibration.duration = static_cast<XrDuration>(durationSeconds * 1e9f);
01492     vibration.frequency = frequency;
01493     vibration.amplitude = amplitude;
01494
01495     XrHapticActionInfo actionInfo{ XR_TYPE_HAPTIC_ACTION_INFO };
01496     actionInfo.action = hapticAction;
01497     actionInfo.subactionPath = handSubactionPath[hand];
01498
01499     xrApplyHapticFeedback(session, &actionInfo, reinterpret_cast<const
        XrHapticBaseHeader*>(&vibration));
01500 }
01501
01502 #endif
01503
01504 #if defined(_WIN32)
01505 #   if !defined(_INC_WINDOWS) && !defined(_WINDOWS_)
01506 #       include <windows.h>
01507 #   endif
01508 #else
01509 #   include <pwd.h>
01510 #   include <unistd.h>
01511 #endif
01512 //
01513 // ユーザ名を得る
01514 //
01515 std::string GgApp::getUsername()
01516 {
01517     // 環境変数からユーザ名を得る
01518     const char* user{
01519         #if defined(_WIN32)
01520             std::getenv("USERNAME")
01521         #else
01522             std::getenv("USER")
01523         #endif
01524     };
01525
01526     // 環境変数からユーザ名が得られたらそれを返す
01527     if (user) return user;
01528
01529     #if defined(_WIN32)
01530         // Win32 API を使ってユーザ名を得る
01531         char username[256];
01532         DWORD size{ sizeof(username) };
01533         if (GetUserNameA(username, &size)) return std::string(username);
01534     #else
01535         struct passwd* pw{ getpwuid(getuid()) };
01536         if (pw) return std::string(pw->pw_name);
01537     #endif

```

```

01537 #endif
01538
01539 // ユーザ名が得られなかった
01540 return "unknown";
01541 }

```

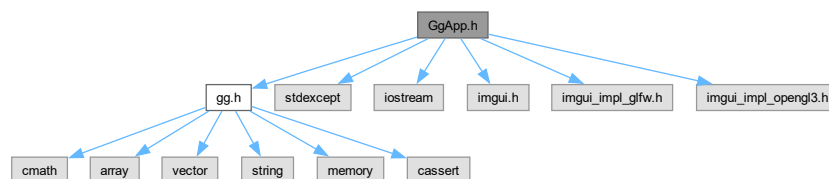
10.11 GgApp.h ファイル

```

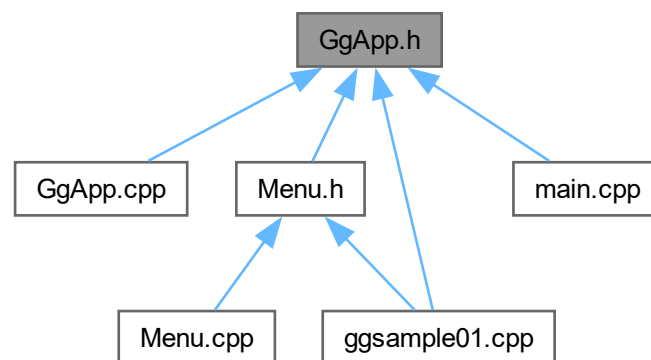
#include "gg.h"
#include <stdexcept>
#include <iostream>
#include "imgui.h"
#include "imgui_impl_glfw.h"
#include "imgui_impl_opengl3.h"

```

GgApp.h の依存先関係図:



被依存関係図:



クラス

- class [GgApp](#)
- class [GgApp::Window](#)

マクロ定義

- #define [GG.USE_IMGUI](#)
- #define [GG.BUTTON_COUNT](#) 3
- #define [GG.INTERFACE_COUNT](#) 5

10.11.1 詳解

ゲームグラフィックス特論宿題アプリケーションクラスの定義。

著者

Kohe Tokoi

日付

July 17, 2025

[GgApp.h](#) に定義があります。

10.11.2 マクロ定義詳解

10.11.2.1 GG_BUTTON_COUNT

```
#define GG_BUTTON_COUNT 3
```

[GgApp.h](#) の 45 行目に定義があります。

10.11.2.2 GG_INTERFACE_COUNT

```
#define GG_INTERFACE_COUNT 5
```

[GgApp.h](#) の 50 行目に定義があります。

10.11.2.3 GG_USE_IMGUI

```
#define GG_USE_IMGUI
```

[GgApp.h](#) の 37 行目に定義があります。

10.12 GgApp.h

[[詳解](#)]

```
00001 #pragma once
00002
00003 /*
00004
00005 ゲームグラフィックス特論用補助プログラム GLFW3 版
00006
00007 Copyright (c) 2011-2025 Kohe Tokoi. All Rights Reserved.
00008
00009 Permission is hereby granted, free of charge, to any person obtaining a copy
00010 of this software and associated documentation files (the "Software"), to deal
00011 in the Software without restriction, including without limitation the rights
00012 to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00013 copies or substantial portions of the Software.
00014
00015 The above copyright notice and this permission notice shall be included in
00016 all copies or substantial portions of the Software.
00017
00018 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00019 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
00020 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL
00021 KOHE TOKOI BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN
00022 AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN
00023 CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
00024
00025 */
00026
00034
00035 // Dear ImGui を使うなら
00036 #if !defined(GG_USE_IMGUI) && !defined(GG_NO_IMGUI)
00037 # define GG_USE_IMGUI
00038 #endif
00039
00040 // OpenXR を使うなら
00041 // #define GG_USE_OPENXR
00042
00043 // 使用するマウスのボタン数
00044 #if !defined(GG_BUTTON_COUNT)
00045 # define GG_BUTTON_COUNT 3
```



```

00046 #endif
00047
00048 // 使用するユーザインタフェースの数
00049 #if !defined(GG_INTERFACE_COUNT)
00050 # define GG_INTERFACE_COUNT 5
00051 #endif
00052
00053 // 補助プログラム
00054 #include "gg.h"
00055
00056 // 標準ライブラリ
00057 #include <stdexcept>
00058 #include <iostream>
00059
00060 // ImGui の組み込み
00061 #if defined(GG_USE_IMGUI)
00062 # include "imgui.h"
00063 # include "imgui_impl_glfw.h"
00064 # include "imgui_impl_opengl3.h"
00065 #endif
00066
00067 // OpenXR ライブラリの組み込み
00068 #if defined(GG_USE_OPENXR)
00069 # if defined(_WIN32)
00070 #   define XR_USE_PLATFORM_WIN32
00071 #   define XR_USE_GRAPHICS_API_OPENGL
00072 #   define GLFW_EXPOSE_NATIVE_WIN32
00073 #   define GLFW_EXPOSE_NATIVE_WGL
00074 #   include <GLFW/glfw3native.h>
00075 #   include <windows.h>
00076 #   include <unknwn.h>
00077 # else
00078 #   define XR_USE_PLATFORM_XLIB
00079 #   define XR_USE_GRAPHICS_API_OPENGL
00080 #   define GLFW_EXPOSE_NATIVE_X11
00081 #   define GLFW_EXPOSE_NATIVE_GLX
00082 #   include <GLFW/glfw3native.h>
00083 # endif
00084 # include <openxr/openxr.h>
00085 # include <openxr/openxr_platform.h>
00086 # include <vector>
00087 # include <cstring>
00088 #endif
00089
00093 class GgApp
00094 {
00095 public:
00096
00103 GgApp(int major = 0, int minor = 1);
00104
00110 GgApp(const GgApp& w) = delete;
00111
00117 GgApp(GgApp&& w) = default;
00118
00122 virtual ~GgApp();
00123
00130 GgApp& operator=(const GgApp& w) = delete;
00131
00138 GgApp& operator=(GgApp&& w) = default;
00139
00147 int main(int argc, const char* const* argv);
00148
00155 class Window
00156 {
00157 // ウィンドウの識別子
00158 GLFWwindow* window{ nullptr };
00159
00160 // ビューポートの横幅と高さ
00161 std::array<GLsizei, 2> size;
00162
00163 // フレームバッファの横幅と高さ
00164 std::array<GLsizei, 2> fboSize;
00165
00166 #if defined(IMGUI_VERSION)
00167 // メニューバーの高さ
00168 GLsizei menuBarHeight{ 0 };
00169 #endif
00170
00171 // ビューポートの縦横比
00172 GLfloat aspect{ 1.0f };
00173
00174 // マウスの移動速度[X/Y/Z]
00175 std::array<GLfloat, 3> velocity{ 1.0f, 1.0f, 0.1f };
00176
00177 // マウスボタンの状態
00178 std::array<bool, GG_BUTTON_COUNT> status{};
00179

```

```

00180 // ユーザインタフェースのデータ構造
00181 struct HumanInterface
00182 {
00183     // 最後にタイプしたキー
00184     int lastKey{ 0 };
00185
00186     // 矢印キー
00187     std::array<std::array<int, 2>, 4> arrow{};
00188
00189     // マウスの現在位置
00190     std::array<GLfloat, 2> mouse{};
00191
00192     // マウスホイールの回転量
00193     std::array<GLfloat, 2> wheel{};
00194
00195     // 平行移動量[ボタン][直前/更新][X/Y/Z]
00196     std::array<std::array<gg::GgVector, 2>, GG.BUTTON_COUNT> translation{};
00197
00198     // トラックボール
00199     std::array<gg::GgTrackball, GG.BUTTON_COUNT> rotation;
00200
00201     // コンストラクタ
00202     HumanInterface()
00203     {
00204         resetTranslation();
00205     }
00206
00207     //
00208     // マウスや矢印キーによる平行移動量を初期化する
00209     //
00210     void resetTranslation();
00211
00212     //
00213     // 平行移動量と回転量を更新する (X, Y のみ, Z は wheel() で計算する)
00214     //
00215     void calcTranslation(int button, const std::array<GLfloat, 3>& velocity);
00216 };
00217
00218 // ヒューマンインタフェースデバイスのデータ
00219 std::array<HumanInterface, GG.INTERFACE_COUNT> interfaceData;
00220
00221 // ヒューマンインタフェースデバイスの番号
00222 int interfaceNo{ 0 };
00223
00224 //
00225 // ユーザー定義のコールバック関数へのポインタ
00226 //
00227 void* userPointer{ nullptr };
00228 void (*resizeFunc)(const Window* window, int width, int height){ nullptr };
00229 void (*keyboardFunc)(const Window* window, int key, int scancode, int action, int mods){ nullptr };
00230 };
00231 void (*mouseFunc)(const Window* window, int button, int action, int mods){ nullptr };
00232 void (*wheelFunc)(const Window* window, double x, double y){ nullptr };
00233
00234 //
00235 // ウィンドウのサイズ変更時の処理
00236 //
00237 static void resize(GLFWwindow* window, int width, int height);
00238
00239 //
00240 // キーボードをタイプした時の処理
00241 //
00242 static void keyboard(GLFWwindow* window, int key, int scancode, int action, int mods);
00243
00244 //
00245 // マウスボタンを操作したときの処理
00246 //
00247 static void mouse(GLFWwindow* window, int button, int action, int mods);
00248
00249 //
00250 // マウスホイールを操作した時の処理
00251 //
00252 static void wheel(GLFWwindow* window, double x, double y);
00253 public:
00254
00255     Window(const std::string& title = "GLFW Window", int width = 640, int height = 480,
00256            int fullscreen = 0, GLFWwindow* share = nullptr);
00257
00258     Window(const Window& w) = delete;
00259
00260     Window(Window&& w) noexcept;
00261
00262     virtual ~Window()
00263     {
00264         // ウィンドウが作成されていない場合は戻る
00265         if (!window) return;
00266     }

```

```

00288
00289     // ウィンドウを破棄する
00290     glfwDestroyWindow(window);
00291 }
00292
00299 Window& operator=(const Window& w) = delete;
00300
00307 Window& operator=(Window&& w) noexcept;
00308
00314 auto* get() const
00315 {
00316     return window;
00317 }
00318
00324 void setClose(int flag = GLFW_TRUE) const
00325 {
00326     glfwSetWindowShouldClose(window, flag);
00327 }
00328
00334 bool shouldClose() const
00335 {
00336     // ウィンドウを閉じるべきなら true を返す
00337     return glfwWindowShouldClose(window) != GLFW_FALSE;
00338 }
00339
00345 explicit operator bool();
00346
00350 void swapBuffers() const;
00351
00355 void restoreViewport() const
00356 {
00357     if (!glfwGetWindowAttrib(window, GLFW_ICONIFIED)) glViewport(0, 0, fboSize[0], fboSize[1]);
00358 }
00359
00363 void updateViewport();
00364
00365 #if defined(IMGUI_VERSION)
00371 void setMenuBarHeight(GLsizei height)
00372 {
00373     // メニューバーの高さを保存する
00374     menubarHeight = height;
00375
00376     // ビューポートを復帰する
00377     updateViewport();
00378 }
00379 #endif
00380
00386 auto getWidth() const
00387 {
00388     return size[0];
00389 }
00390
00396 auto getHeight() const
00397 {
00398     return size[1];
00399 }
00400
00406 auto getFboWidth() const
00407 {
00408     return fboSize[0];
00409 }
00410
00416 auto getFboHeight() const
00417 {
00418     return fboSize[1];
00419 }
00420
00426 const auto& getSize() const
00427 {
00428     return size;
00429 }
00430
00436 void getSize(GLsizei* size) const
00437 {
00438     size[0] = getWidth();
00439     size[1] = getHeight();
00440 }
00441
00447 const auto& getFboSize() const
00448 {
00449     return fboSize;
00450 }
00451
00457 void getFboSize(GLsizei* fboSize) const
00458 {
00459     fboSize[0] = getFboWidth();
00460     fboSize[1] = getFboHeight();

```

```

00461     }
00462
00468     auto getAspect() const
00469     {
00470         return aspect;
00471     }
00472
00478     bool getKey(int key) const
00479     {
00480 #if defined(ImGui_VERSION)
00481         // ImGui がキーボードを使うときはキーボードの処理を行わない
00482         if (ImGui::GetIO().WantCaptureKeyboard) return false;
00483 #endif
00484
00485         return glfwGetKey(window, key) != GLFW_RELEASE;
00486     }
00487
00493     void selectInterface(int no)
00494     {
00495         assert(static_cast<size_t>(no) < interfaceData.size());
00496         interfaceNo = no;
00497     }
00498
00506     void setVelocity(GLfloat vx, GLfloat vy, GLfloat vz = 0.1f)
00507     {
00508         velocity = std::array<GLfloat, 3>{ vx, vy, vz };
00509     }
00510
00516     int getLastKey()
00517     {
00518         auto& current_if{ interfaceData[interfaceNo] };
00519         const int key{ current_if.lastKey };
00520         current_if.lastKey = 0;
00521         return key;
00522     }
00523
00531     auto getArrow(int direction = 0, int mods = 0) const
00532     {
00533         const auto& current_if{ interfaceData[interfaceNo] };
00534         return static_cast<GLfloat>(current_if.arrow[mods & 3][direction & 1]);
00535     }
00536
00543     auto getArrowX(int mods = 0) const
00544     {
00545         return getArrow(0, mods);
00546     }
00547
00554     auto getArrowY(int mods = 0) const
00555     {
00556         return getArrow(1, mods);
00557     }
00558
00565     void getArrow(GLfloat* arrow, int mods = 0) const
00566     {
00567         arrow[0] = getArrowX(mods);
00568         arrow[1] = getArrowY(mods);
00569     }
00570
00576     auto getShiftArrowX() const
00577     {
00578         return getArrow(0, 1);
00579     }
00580
00586     auto getShiftArrowY() const
00587     {
00588         return getArrow(1, 1);
00589     }
00590
00596     void getShiftArrow(GLfloat* shift_arrow) const
00597     {
00598         shift_arrow[0] = getShiftArrowX();
00599         shift_arrow[1] = getShiftArrowY();
00600     }
00601
00607     auto getControlArrowX() const
00608     {
00609         return getArrow(0, 2);
00610     }
00611
00617     auto getControlArrowY() const
00618     {
00619         return getArrow(1, 2);
00620     }
00621
00627     void getControlArrow(GLfloat* control_arrow) const
00628     {
00629         control_arrow[0] = getControlArrowX();

```

```

00630     control_arrow[1] = getControlArrowY();
00631 }
00632
00638 auto getAltArrowX() const
00639 {
00640     return getArrow(0, 3);
00641 }
00642
00648 auto getAltArrowY() const
00649 {
00650     return getArrow(1, 3);
00651 }
00652
00658 void getAltArrow(GLfloat* alt_arrow) const
00659 {
00660     alt_arrow[0] = getAltArrowX();
00661     alt_arrow[1] = getAltArrowY();
00662 }
00663
00669 const auto* getMouse() const
00670 {
00671     const auto& current_if{ interfaceData[interfaceNo] };
00672     return current_if.mouse.data();
00673 }
00674
00680 void getMouse(GLfloat* position) const
00681 {
00682     const auto& current_if{ interfaceData[interfaceNo] };
00683     position[0] = current_if.mouse[0];
00684     position[1] = current_if.mouse[1];
00685 }
00686
00693 auto getMouse(int direction) const
00694 {
00695     const auto& current_if{ interfaceData[interfaceNo] };
00696     return current_if.mouse[direction & 1];
00697 }
00698
00704 auto getMouseX() const
00705 {
00706     const auto& current_if{ interfaceData[interfaceNo] };
00707     return current_if.mouse[0];
00708 }
00709
00715 auto getMouseY() const
00716 {
00717     const auto& current_if{ interfaceData[interfaceNo] };
00718     return current_if.mouse[1];
00719 }
00720
00726 const auto* getWheel() const
00727 {
00728     const auto& current_if{ interfaceData[interfaceNo] };
00729     return current_if.wheel.data();
00730 }
00731
00737 void getWheel(GLfloat* rotation) const
00738 {
00739     const auto& current_if{ interfaceData[interfaceNo] };
00740     rotation[0] = current_if.wheel[0];
00741     rotation[1] = current_if.wheel[1];
00742 }
00743
00750 auto getWheel(int direction) const
00751 {
00752     const auto& current_if{ interfaceData[interfaceNo] };
00753     return current_if.wheel[direction & 1];
00754 }
00755
00761 auto getWheelX() const
00762 {
00763     const auto& current_if{ interfaceData[interfaceNo] };
00764     return current_if.wheel[0];
00765 }
00766
00772 auto getWheelY() const
00773 {
00774     const auto& current_if{ interfaceData[interfaceNo] };
00775     return current_if.wheel[1];
00776 }
00777
00784 const auto& getTranslation(int button = GLFW_MOUSE_BUTTON_1) const
00785 {
00786     const auto& current_if{ interfaceData[interfaceNo] };
00787     assert(button >= GLFW_MOUSE_BUTTON_1 && button < GLFW_MOUSE_BUTTON_1 + GG_BUTTON_COUNT);
00788     return current_if.translation[button][1];
00789 }

```

```

00790
00797 auto getTranslationMatrix(int button = GLFW_MOUSE_BUTTON_1) const
00798 {
00799     const auto& current_if{ interfaceData[interfaceNo] };
00800     assert(button >= GLFW_MOUSE_BUTTON_1 && button < GLFW_MOUSE_BUTTON_1 + GG_BUTTON_COUNT);
00801     const auto& t{ current_if.translation[button][1] };
00802
00803     gg::GgMatrix m
00804     {
00805         1.0f, 0.0f, 0.0f, 0.0f,
00806         0.0f, 1.0f, 0.0f, 0.0f,
00807         0.0f, 0.0f, 1.0f, 0.0f,
00808         t[0], t[1], t[2], 1.0f
00809     };
00810
00811     return m;
00812 }
00813
00820 auto getScrollMatrix(int button = GLFW_MOUSE_BUTTON_1) const
00821 {
00822     const auto& current_if{ interfaceData[interfaceNo] };
00823     assert(button >= GLFW_MOUSE_BUTTON_1 && button < GLFW_MOUSE_BUTTON_1 + GG_BUTTON_COUNT);
00824     const auto& t{ current_if.translation[button][1] };
00825
00826     gg::GgMatrix m
00827     {
00828         t[2] + 1.0f, 0.0f, 0.0f, 0.0f,
00829         0.0f, t[2] + 1.0f, 0.0f, 0.0f,
00830         0.0f, 0.0f, 1.0f, 0.0f,
00831         t[0], t[1], 0.0f, 1.0f
00832     };
00833
00834     return m;
00835 }
00836
00843 auto getRotation(int button = GLFW_MOUSE_BUTTON_1) const
00844 {
00845     const auto& current_if{ interfaceData[interfaceNo] };
00846     assert(button >= GLFW_MOUSE_BUTTON_1 && button < GLFW_MOUSE_BUTTON_1 + GG_BUTTON_COUNT);
00847     return current_if.rotation[button].getQuaternion();
00848 }
00849
00856 auto getRotationMatrix(int button = GLFW_MOUSE_BUTTON_1) const
00857 {
00858     const auto& current_if{ interfaceData[interfaceNo] };
00859     assert(button >= GLFW_MOUSE_BUTTON_1 && button < GLFW_MOUSE_BUTTON_1 + GG_BUTTON_COUNT);
00860     return current_if.rotation[button].getMatrix();
00861 }
00862
00866 void resetRotation()
00867 {
00868     // トラックボールを初期化する
00869     for (auto& tb : interfaceData[interfaceNo].rotation) tb.reset();
00870 }
00871
00875 void resetTranslation()
00876 {
00877     // 現在のインターフェースの平行移動量を初期化する
00878     interfaceData[interfaceNo].resetTranslation();
00879 }
00880
00884 void reset()
00885 {
00886     // トラックボール処理を初期化する
00887     resetRotation();
00888
00889     // 平行移動量を初期化する
00890     resetTranslation();
00891 }
00892
00898 void* getUserPointer() const
00899 {
00900     return userPointer;
00901 }
00902
00908 void setUserPointer(void* pointer)
00909 {
00910     userPointer = pointer;
00911 }
00912
00918 void setResizeFunc(void (*func)(const Window* window, int width, int height))
00919 {
00920     resizeFunc = func;
00921 }
00922
00928 void setKeyboardFunc(void (*func)(const Window* window, int key, int scancode, int action, int
mods))

```

```

00929     {
00930         keyboardFunc = func;
00931     }
00932
00933 void setMouseFunc(void (*func)(const Window* window, int button, int action, int mods))
00934 {
00935     mouseFunc = func;
00936 }
00937
00938 void setWheelFunc(void (*func)(const Window* window, double x, double y))
00939 {
00940     wheelFunc = func;
00941 }
00942
00943 };
00944
00945 #if defined(GG_USE_OPENXR)
00946 class OpenXR
00947 {
00948 public:
00949     enum Hand
00950     {
00951         Left = 0,
00952         Right = 1,
00953         Count = 2
00954     };
00955 private:
00956     // OpenXR のインスタンスとシステム
00957     XrInstance instance{ XR_NULL_HANDLE };
00958     XrSystemId systemId{ XR_NULL_SYSTEM_ID };
00959
00960     // OpenXR のセッション
00961     XrSession session{ XR_NULL_HANDLE };
00962     XrSessionState sessionState{ XR_SESSION_STATE_UNKNOWN };
00963
00964     // OpenXR の参照空間
00965     XrSpace appSpace{ XR_NULL_HANDLE };
00966     XrReferenceSpaceType referenceSpaceType{ XR_REFERENCE_SPACE_TYPE_STAGE };
00967
00968     // ビュー設定とステート
00969     std::vector<XrViewConfigurationView> views;
00970     std::vector<XrView> viewStates;
00971
00972     // OpenXR のスワップチェーン
00973     std::vector<XrSwapchain> swapchains;
00974     std::vector<std::vector<XrSwapchainImageOpenGLKHR>> swapchainImages;
00975
00976     // OpenXR へのレンダリングに使う FBO とデプステクスチャ
00977     GLuint openxrFbo[2]{ 0, 0 };
00978     GLuint openxrDepth[2]{ 0, 0 };
00979
00980     // フレームの同期状態
00981     XrFrameState frameState{ XR_TYPE_FRAME_STATE };
00982     bool isSessionRunning{ false };
00983
00984     // 各フレームで取得したスワップチェーンイメージのインデックス
00985     uint32_t currentIndex[2]{ 0, 0 };
00986
00987     // OpenXR のミラー表示を行うウィンドウ
00988     const Window* window{ nullptr };
00989
00990     // アクションセット
00991     XrActionSet actionSet{ XR_NULL_HANDLE };
00992
00993     // アクション定義
00994     XrAction aimPoseAction{ XR_NULL_HANDLE };
00995     XrAction gripPoseAction{ XR_NULL_HANDLE };
00996     XrAction triggerAction{ XR_NULL_HANDLE };
00997     XrAction gripAction{ XR_NULL_HANDLE };
00998     XrAction thumbstickAction{ XR_NULL_HANDLE };
00999     XrAction thumbstickClickAction{ XR_NULL_HANDLE };
01000     XrAction primaryButtonAction{ XR_NULL_HANDLE };
01001     XrAction secondaryButtonAction{ XR_NULL_HANDLE };
01002     XrAction menuButtonAction{ XR_NULL_HANDLE };
01003     XrAction hapticAction{ XR_NULL_HANDLE };
01004
01005     // コントローラーのスペース
01006     XrSpace aimSpace[Hand::Count]{ XR_NULL_HANDLE, XR_NULL_HANDLE };
01007     XrSpace gripSpace[Hand::Count]{ XR_NULL_HANDLE, XR_NULL_HANDLE };
01008
01009     // コントローラーの状態キャッシュ
01010     struct ControllerState
01011     {
01012         bool isTracked{ false };
01013         XrPosef gripPose{ { 0.0f, 0.0f, 0.0f, 1.0f }, { 0.0f, 0.0f, 0.0f } };
01014     };

```

```

01035     XrPosef aimPose{ { 0.0f, 0.0f, 0.0f, 1.0f }, { 0.0f, 0.0f, 0.0f } };
01036     float trigger{ 0.0f };
01037     float grip{ 0.0f };
01038     std::array<float, 2> thumbstick{ 0.0f, 0.0f };
01039     bool thumbstickClick{ false };
01040     bool primaryButton{ false };
01041     bool secondaryButton{ false };
01042     bool menuButton{ false };
01043 };
01044 ControllerState controllerStates[Hand::Count];
01045
01046 // サポートするバスの文字列
01047 XrPath handSubactionPath[Hand::Count]{ XR_NULL_PATH, XR_NULL_PATH };
01048
01049 //
01050 // アクションシステムを初期化する
01051 //
01052 void initActions();
01053
01054 //
01055 // アクション状態を更新する
01056 //
01057 void pollActions();
01058
01059 //
01060 // コンストラクタ
01061 //
01062 OpenXR();
01063
01064 //
01065 // デストラクタ
01066 //
01067 virtual ~OpenXR() = default;
01068
01069 public:
01070
01071 // シングルトンなのでコピー・ムーブ禁止
01072 OpenXR(const OpenXR&) = delete;
01073 OpenXR& operator=(const OpenXR&) = delete;
01074 OpenXR(OpenXR&&) = delete;
01075 OpenXR& operator=(OpenXR&&) = delete;
01076
01083 static OpenXR& initialize(const Window& window, XrReferenceSpaceType spaceType =
XR_REFERENCE_SPACE_TYPE_STAGE);
01084
01088 void terminate();
01089
01095 bool begin();
01096
01102 void select(int eye);
01103
01112 void select(int eye, GLfloat* screen, GLfloat* position, GLfloat* orientation);
01113
01119 void commit(int eye);
01120
01127 bool submit(bool mirror = true);
01128
01137 gg::GgMatrix getProjectionMatrix(int eye, GLfloat zNear = 0.1f, GLfloat zFar = 100.0f) const;
01138
01145 gg::GgMatrix getViewMatrix(int eye) const;
01146
01153 gg::GgMatrix getPoseMatrix(int eye) const;
01154
01161 gg::GgVector getPosition(int eye) const;
01162
01169 gg::GgQuaternion getOrientation(int eye) const;
01170
01177 const XrFovf& getFov(int eye) const;
01178
01185 const XrPosef& getPose(int eye) const;
01186
01193 GLsizei getWidth(int eye = 0) const;
01194
01201 GLsizei getHeight(int eye = 0) const;
01202
01209 GLfloat getAspect(int eye = 0) const;
01210
01216 uint32_t getViewCount() const;
01217
01223 XrReferenceSpaceType getReferenceSpaceType() const;
01224
01231 bool isTracked(int hand) const;
01232
01239 gg::GgMatrix getGripMatrix(int hand) const;
01240
01247 gg::GgMatrix getAimMatrix(int hand) const;
01248

```



```

01255     gg::GgVector getGripPosition(int hand) const;
01256
01263     gg::GgQuaternion getGripOrientation(int hand) const;
01264
01271     gg::GgVector getAimPosition(int hand) const;
01272
01279     gg::GgQuaternion getAimOrientation(int hand) const;
01280
01287     float getTrigger(int hand) const;
01288
01295     float getGrip(int hand) const;
01296
01303     std::array<float, 2> getThumbstick(int hand) const;
01304
01311     bool getThumbstickClick(int hand) const;
01312
01319     bool getPrimaryButton(int hand) const;
01320
01327     bool getSecondaryButton(int hand) const;
01328
01335     bool getMenuButton(int hand = Hand::Left) const;
01336
01345     void applyHapticVibration(int hand, float durationSeconds = 0.1f, float frequency =
XR_FREQUENCY_UNSPECIFIED, float amplitude = 0.5f);
01346 };
01347 #endif
01348
01354     static std::string getUsername();
01355 };

```

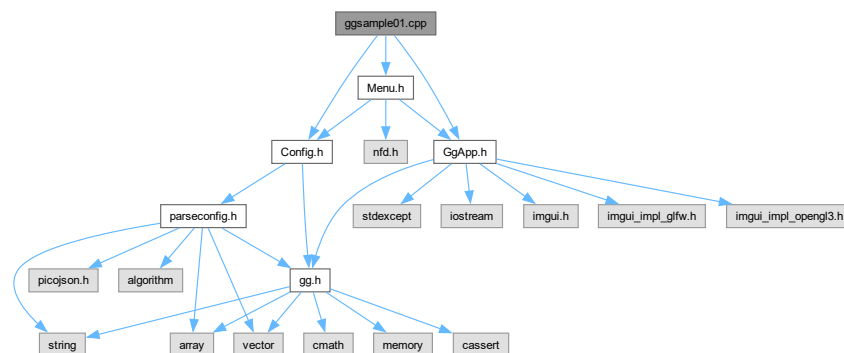
10.13 ggsample01.cpp ファイル

```

#include "GgApp.h"
#include "Config.h"
#include "Menu.h"

```

ggsample01.cpp の依存先関係図:



マクロ定義

- #define PROJECT_NAME "ggsample01"
- #define CONFIG_FILE PROJECT_NAME "_config.json"

10.13.1 マクロ定義詳解

10.13.1.1 CONFIG_FILE

```

#define CONFIG_FILE PROJECT_NAME "_config.json"

```

ggsample01.cpp の 15 行目に定義があります。

10.13.1.2 PROJECT_NAME

```

#define PROJECT_NAME "ggsample01"

```

ggsample01.cpp の 10 行目に定義があります。

10.14 ggsample01.cpp

[詳解]

```

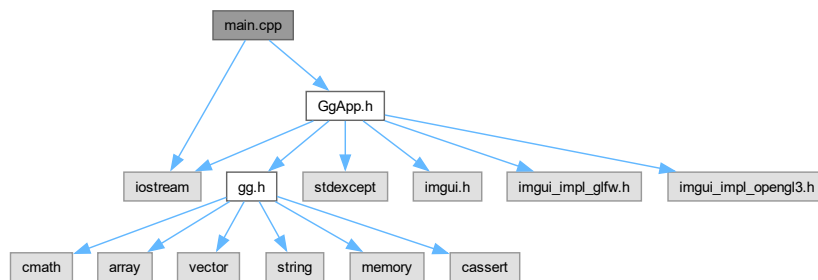
00001 //
00002 // ゲームグラフィックス特論宿題アプリケーション
00003 //
00004
00005 // 補助プログラムのラッパー
00006 #include "GgApp.h"
00007
00008 // プロジェクト名
00009 #if !defined(PROJECT_NAME)
00010 # define PROJECT_NAME "ggsample01"
00011 #endif
00012
00013 // 構成ファイル名
00014 #if !defined(CONFIG_FILE)
00015 # define CONFIG_FILE PROJECT_NAME "_config.json"
00016 #endif
00017
00018 // 構成データ
00019 #include "Config.h"
00020
00021 // メニューの描画
00022 #include "Menu.h"
00023
00024 // 補助プログラム
00025 using namespace gg;
00026
00027 //
00028 // アプリケーション本体
00029 //
00030 int GgApp::main(int argc, const char* const* argv)
00031 {
00032     // 設定を読み込む
00033     const Config config{ CONFIG_FILE };
00034
00035     // ウィンドウを作成する
00036     Window window{ argc > 1 ? argv[1] : getUsername(), config.getWidth(), config.getHeight() };
00037
00038     // メニューを初期化する
00039     Menu menu{ config };
00040
00041     // 背景色を設定する
00042     glClearColor(0.1f, 0.2f, 0.3f, 0.0f);
00043
00044     // 隠面消去処理を設定する
00045     glEnable(GL_DEPTH_TEST);
00046     glEnable(GL_CULL_FACE);
00047
00048     // ビュー変換行列を設定する
00049     const GgMatrix mv{ ggLookat(0.0f, 0.0f, 5.0f, 0.0f, 0.0f, 0.0f, 0.0f, 1.0f, 0.0f) };
00050
00051     // ウィンドウが開いている間繰り返す
00052     while (window)
00053     {
00054         // ウィンドウを消去する
00055         glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
00056
00057         // メニューを表示する
00058         menu.draw();
00059
00060         // オブジェクトの回転の変換行列を設定する
00061         const auto& mr{ window.getRotationMatrix(0) };
00062
00063         // 視点の平行移動の変換行列を設定する
00064         const auto& mt{ window.getTranslationMatrix(1) };
00065
00066         // 投影変換行列を設定する
00067         const GgMatrix&& mp{ ggPerspective(0.5f, window.getAspect(), 1.0f, 15.0f) };
00068
00069         // シェーダを指定する
00070         menu.getShader().use(mp, mt * mv * mr, menu.getLight());
00071
00072         // シーンを描画する
00073         menu.getModel().draw();
00074
00075         // カラーバッファを入れ替えてイベントを取り出す
00076         window.swapBuffers();
00077     }
00078
00079     return 0;

```

```
00080 }
```

10.15 main.cpp ファイル

```
#include "GgApp.h"
#include <iostream>
main.cpp の依存先関係図:
```



マクロ定義

- `#define HEADER_STR` "ゲームグラフィックス特論"

関数

- `int main` (int argc, const char *const *argv)

10.15.1 詳解

ゲームグラフィックス特論宿題アプリケーション

著者

Kohe Tokoi

日付

February 20, 2024

`main.cpp` に定義があります。

10.15.2 マクロ定義詳解

10.15.2.1 HEADER_STR

```
#define HEADER_STR "ゲームグラフィックス特論"
```

`main.cpp` の 18 行目に定義があります。

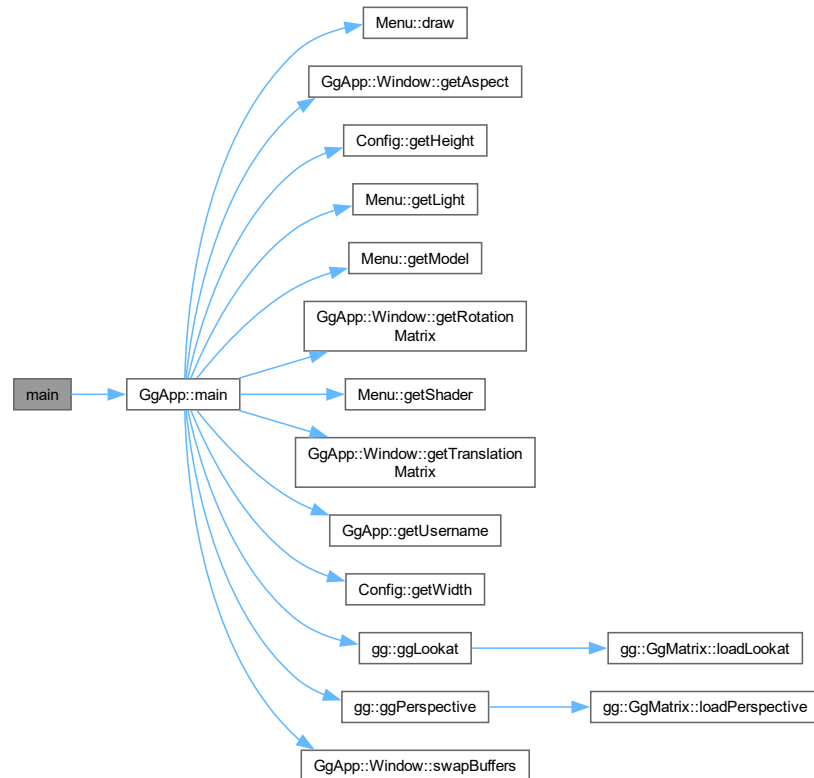
10.15.3 関数詳解

10.15.3.1 main()

```
int main (
    int argc,
    const char *const * argv)
```

`main.cpp` の 23 行目に定義があります。

呼び出し関係図:



10.16 main.cpp

[詳解]

```

00001
00008 #include "GgApp.h"
00009
00010 // MessageBox の準備
00011 #if defined(_MSC_VER)
00012 # include <atlstr.h>
00013 #elif defined(__APPLE__)
00014 # include <CoreFoundation/CoreFoundation.h>
00015 #else
00016 # include <iostream>
00017 #endif
00018 #define HEADER_STR "ゲームグラフィックス特論"
00019
00020 //
00021 // メインプログラム
00022 //
00023 int main(int argc, const char* const* argv) try
00024 {
00025     #if defined(__APPLE__)
00026         // バンドル内の Resources ディレクトリのパスを取得
00027         CFBundleRef mainBundle{ CFBundleGetMainBundle() };
00028         CFURLRef resourcesURL{ CFBundleCopyResourcesDirectoryURL(mainBundle) };
00029         char path[PATH_MAX];
00030         if (CFURLGetFileSystemRepresentation(resourcesURL, TRUE, (UInt8 *)path, PATH_MAX))
00031         {
00032             // カレントディレクトリを Resources に変更
00033             chdir(path);
00034         }
00035         CFRelease(resourcesURL);
00036     #endif
00037
00038     // アプリケーションのオブジェクトを生成する
00039     #if defined(GL_GLES_PROTOTYPES)
  
```

```

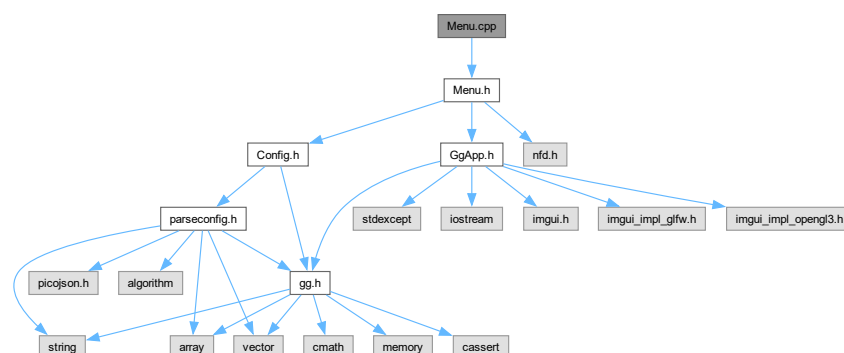
00040     GgApp app(3, 1);
00041 #else
00042     GgApp app(4, 1);
00043 #endif
00044
00045     // アプリケーションを実行する
00046     return app.main(argc, argv);
00047 }
00048 catch (const std::runtime_error &e)
00049 {
00050     // エラーメッセージを表示する
00051     #if defined(_MSC_VER)
00052         MessageBox(NULL, CString(e.what()), TEXT(HEADER_STR), MB_ICONERROR);
00053     #elif defined(__APPLE__)
00054         // the following code is copied from
00055         // http://blog.jorgearimany.com/2010/05/messagebox-from-windows-to-mac.html
00056         // convert the strings from char* to CFStringRef
00057         CFStringRef msg_ref = CFStringCreateWithCString(NULL, e.what(), kCFStringEncodingUTF8);
00058         // result code from the message box
00059         CFOptionFlags result;
00060         // launch the message box
00061         CFUserNotificationDisplayAlert(
00062             0, // no timeout
00063             kCFUserNotificationNoteAlertLevel, // change it depending message_type flags ( MB_ICONASTERISK....
00064             etc.)
00065             NULL, // icon url, use default, you can change it depending
00066             message_type flags
00067             NULL, // not used
00068             NULL, // localization of strings
00069             CFSTR(HEADER_STR), // header text
00070             msg_ref, // message text
00071             NULL, // default "ok" text in button
00072             NULL, // alternate button title
00073             NULL, // other button title, null--> no other button
00074             &result // response flags
00075         );
00076         // Clean up the strings
00077         CFRelease(msg_ref);
00078     #else
00079         std::cerr << HEADER_STR << ": " << e.what() << '\n';
00080     #endif
00081
00082     // プログラムを終了する
00083     return EXIT_FAILURE;
00084 }

```

10.17 Menu.cpp ファイル

```
#include "Menu.h"
```

Menu.cpp の依存先関係図:



10.17.1 詳解

メニューの描画クラスの実装

著者

Kohe Tokoi

日付

November 15, 2022

[Menu.cpp](#) に定義があります。

10.18 Menu.cpp

[詳解]

```

00001
00008 #include "Menu.h"
00009
00010 // 補助プログラム
00011 using namespace gg;
00012
00013 //
00014 // コンストラクタ
00015 //
00016 Menu::Menu(const Config& config) :
00017     defaults{ config },
00018     settings{ config },
00019     light{ std::make_unique<GgSimpleShader::LightBuffer>(config.light) },
00020     shader{ std::make_unique<GgSimpleShader>(config.shader) },
00021     model{ std::make_unique<GgSimpleObj>(config.model, true) }
00022 {
00023     #if defined(IMGUI_VERSION)
00024         //
00025         // ImGui の初期設定
00026         //
00027
00028         // ファイルダイアログ (Native File Dialog Extended) を初期化する
00029         NFD_Init();
00030
00031         // Dear ImGui の入力デバイス
00032         //io.ConfigFlags |= ImGuiConfigFlags_NavEnableKeyboard; // キーボードコントロールを使う
00033         //io.ConfigFlags |= ImGuiConfigFlags_NavEnableGamepad;   // ゲームパッドを使う
00034
00035         // Dear ImGui のスタイル
00036         //ImGui::StyleColorsDark(); // 暗めのスタイル
00037         //ImGui::StyleColorsClassic(); // 以前のスタイル
00038
00039         // 日本語を表示できるメニューフォントを読み込む
00040         if (!ImGui::GetIO().Fonts->AddFontFromFileTTF(config.menuFont.c_str(), config.menuFontSize, nullptr,
00041             ImGui::GetIO().Fonts->GetGlyphRangesJapanese()))
00042         {
00043             // メニューフォントが読み込めなかったらエラーにする
00044             throw std::runtime_error("Cannot find any menu fonts.");
00045         }
00046     #endif
00047 }
00048
00049 //
00050 // デストラクタ
00051 //
00052 Menu::~Menu()
00053 {
00054     #if defined(IMGUI_VERSION)
00055         // ファイルダイアログ (Native File Dialog Extended) を終了する
00056         NFD_Quit();
00057     #endif
00058 }
00059
00060 //
00061 // ファイルパスを取得する
00062 //
00063 bool Menu::getFilePath(std::string& path, const nfdfilteritem_t* filter)
00064 {
00065     // ファイルダイアログから得るパス
00066     nfdchar_t* filepath{ nullptr };
00067
00068     // ファイルダイアログを開く
00069     if (NFD_OpenDialog(&filepath, filter, 1, nullptr) == NFD_OKAY)
00070     {
00071         path = TCharToUtf8(filepath);
00072         NFD_FreePath(filepath);
00073         return true;
00074     }

```

```

00075
00076     return false;
00077 }
00078
00079 //
00080 // 描画する
00081 //
00082 void Menu::draw()
00083 {
00084     #if defined(IMGUI_VERSION)
00085         //
00086         // ImGui によるユーザインタフェース
00087         //
00088
00089         // メニューの表示位置を決定する
00090         ImGui::SetNextWindowPos(ImVec2(4, 4), ImGuiCond_Once);
00091         ImGui::SetNextWindowSize(ImVec2(320, 98), ImGuiCond_Once);
00092
00093         // メニュー表示開始
00094         ImGui::Begin(u8"コントロールパネル");
00095
00096         // 光源
00097         if (ImGui::SliderFloat3(u8"光源位置", settings.light.position.data(), -10.0f, 10.0f, "%.2f"))
00098             light->loadPosition(settings.light.position);
00099         if (ImGui::ColorEdit3(u8"光源色", settings.light.diffuse.data(), ImGuiColorEditFlags_Float))
00100             light->loadDiffuse(settings.light.diffuse);
00101
00102         // メニュー表示終了
00103         ImGui::End();
00104     #endif
00105 }

```

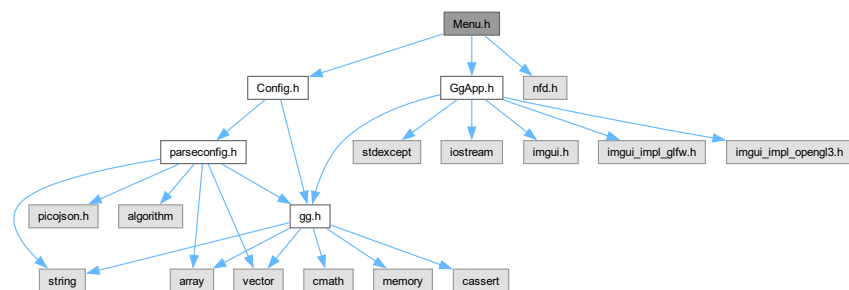
10.19 Menu.h ファイル

```

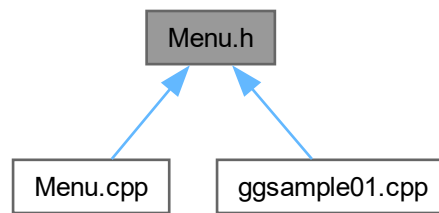
#include "GgApp.h"
#include "Config.h"
#include "nfd.h"

```

Menu.h の依存先関係図:



被依存関係図:



クラス

- class [Menu](#)

10.19.1 詳解

メニューの描画クラスの定義

著者

Kohe Tokoi

日付

August 15, 2025

[Menu.h](#) に定義があります。

10.20 Menu.h

[詳解]

```
00001 #pragma once
00002
00010
00011 // 宿題用補助プログラムのラッパー
00012 #include "GgApp.h"
00013
00014 // 構成データ
00015 #include "Config.h"
00016
00017 // ファイルダイアログ
00018 #include "nfd.h"
00019
00023 class Menu
00024 {
00025     // 図形の描画クラスから参照する
00026     friend class Scene;
00027
00028     // オリジナルの構成データ
00029     const Config& defaults;
00030
00031     // 構成データのコピー
00032     Config settings;
00033
00034     // 光源データ
00035     std::unique_ptr<const gg::GgSimpleShader::LightBuffer> light;
00036
00037     // シェーダ
00038     std::unique_ptr<const gg::GgSimpleShader> shader;
00039
00040     // CAD データ
00041     std::unique_ptr<const gg::GgSimpleObj> model;
```



```

00042
00043 // ファイルパスを取得する
00044 bool getFilePath(std::string& path, const nfdfilteritem_t* filter);
00045
00046 public:
00047
00051 Menu(const Config& config);
00052
00056 Menu(const Menu& menu) = delete;
00057
00061 Menu(Menu&& menu) = default;
00062
00066 virtual ~Menu();
00067
00071 Menu& operator=(const Menu& menu) = delete;
00072
00076 const auto& getLight() const
00077 {
00078     return *light;
00079 }
00080
00084 const auto& getShader() const
00085 {
00086     return *shader;
00087 }
00088
00092 const auto& getModel() const
00093 {
00094     return *model;
00095 }
00096
00100 void draw();
00101 };

```

10.21 OPENXR.md ファイル

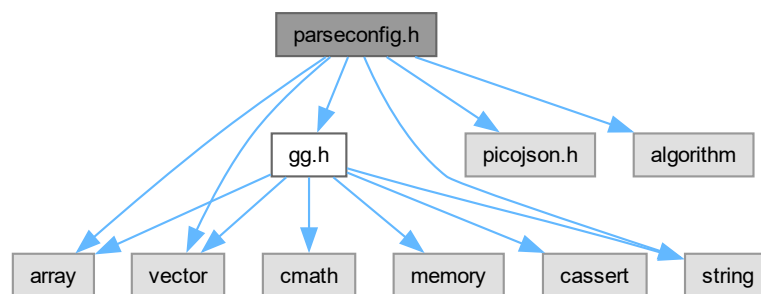
10.22 parseconfig.h ファイル

```

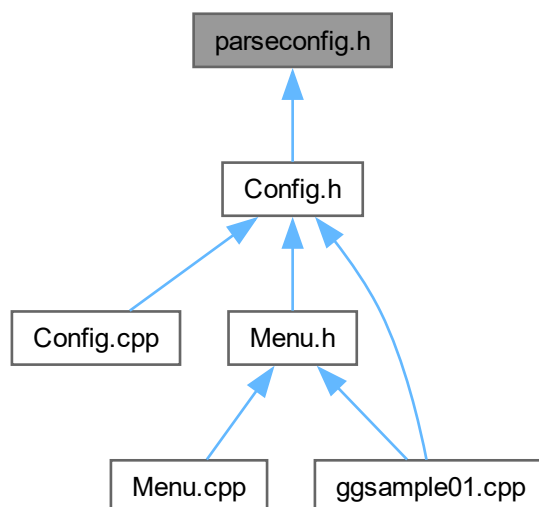
#include "gg.h"
#include "picojson.h"
#include <string>
#include <vector>
#include <array>
#include <algorithm>

```

parseconfig.h の依存先関係図:



被依存関係図:



関数

- `template<typename T>`
`bool getValue (const picojson::object &object, const std::string &key, T &scalar)`
- `bool getValue (const picojson::object &object, const std::string &key, bool &boolean)`
- `template<typename T, std::size_t U>`
`bool getValue (const picojson::object &object, const std::string &key, std::array< T, U > &vector)`
- `template<typename T>`
`bool getValue (const picojson::object &object, const std::string &key, std::vector< T > &vector)`
- `bool getVector (const picojson::object &object, const std::string &key, gg::GgVector &vector)`
- `bool getString (const picojson::object &object, const std::string &key, std::string &string)`
- `template<std::size_t U>`
`bool getString (const picojson::object &object, const std::string &key, std::array< std::string, U > &strings)`
- `bool getString (const picojson::object &object, const std::string &key, std::vector< std::string > &strings)`
- `template<typename T>`
`void setValue (picojson::object &object, const std::string &key, const T &scalar)`
- `void setValue (picojson::object &object, const std::string &key, bool boolean)`
- `template<typename T, std::size_t U>`
`void setValue (picojson::object &object, const std::string &key, const std::array< T, U > &vector)`
- `template<typename T>`
`void setValue (picojson::object &object, const std::string &key, const std::vector< T > &vector)`
- `void setVector (picojson::object &object, const std::string &key, const gg::GgVector &vector)`
- `void setString (picojson::object &object, const std::string &key, const std::string &string)`
- `template<std::size_t U>`
`void setString (picojson::object &object, const std::string &key, const std::array< std::string, U > &strings)`
- `void setString (picojson::object &object, const std::string &key, const std::vector< std::string > &strings)`

10.22.1 詳解

構成ファイルの読み取り補助

著者

Kohe Tokoi

日付

November 15, 2022

[parseconfig.h](#) に定義があります。

10.22.2 関数詳解

10.22.2.1 getString() [1/3]

```
template<std::size_t U>
bool getString (
    const picojson::object & object,
    const std::string & key,
    std::array< std::string, U > & strings)
構成ファイルの JSON オブジェクトから文字列の固定長配列を取得する
```

テンプレート引数

<i>U</i>	構成ファイルから取得する配列の要素の文字列の数
----------	-------------------------

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	取得する JSON オブジェクトのキー
<i>strings</i>	取得した文字列の配列を格納する変数

戻り値

取得に成功すれば true

[parseconfig.h](#) の 243 行目に定義があります。

10.22.2.2 getString() [2/3]

```
bool getString (
    const picojson::object & object,
    const std::string & key,
    std::string & string) [inline]
構成ファイルの JSON オブジェクトから文字列を取得する
```

引数

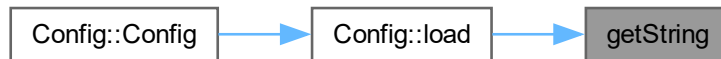
<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	取得する JSON オブジェクトのキー
<i>string</i>	取得した文字列を格納する変数

戻り値

取得に成功すれば `true`

[parseconfig.h](#) の 218 行目に定義があります。

被呼び出し関係図:



10.22.2.3 getString() [3/3]

```
bool getString (  
    const picojson::object & object,  
    const std::string & key,  
    std::vector< std::string > & strings) [inline]
```

構成ファイルの JSON オブジェクトから文字列の可変長配列を取得する

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	取得する JSON オブジェクトのキー
<i>strings</i>	取得した文字列の配列を格納する変数

戻り値

取得に成功すれば `true`

[parseconfig.h](#) の 277 行目に定義があります。

10.22.2.4 getValue() [1/4]

```
bool getValue (  
    const picojson::object & object,  
    const std::string & key,  
    bool & boolean) [inline]
```

構成ファイルの JSON オブジェクトから真偽値を取得する

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	取得する JSON オブジェクトのキー
<i>boolean</i>	取得した JSON オブジェクトの真偽値を格納する変数

戻り値

取得に成功すれば `true`

[parseconfig.h](#) の 66 行目に定義があります。

10.22.2.5 getValue() [2/4]

```
template<typename T, std::size_t U>
bool getValue (
    const picojson::object & object,
    const std::string & key,
    std::array< T, U > & vector)
```

構成ファイルの JSON オブジェクトから数値の固定長配列を取得する

テンプレート引数

<i>T</i>	構成ファイルから取得する配列の要素の数値のデータ型
<i>U</i>	構成ファイルから取得する配列の要素の数値の数

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	取得する JSON オブジェクトのキー
<i>vector</i>	取得した数値の配列を格納する変数

戻り値

取得に成功すれば true

[parseconfig.h](#) の 101 行目に定義があります。

10.22.2.6 getValue() [3/4]

```
template<typename T>
bool getValue (
    const picojson::object & object,
    const std::string & key,
    std::vector< T > & vector)
```

構成ファイルの JSON オブジェクトから数値の可変長配列を取得する

テンプレート引数

<i>T</i>	構成ファイルから取得する配列の要素の数値のデータ型
----------	---------------------------

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	取得する JSON オブジェクトのキー
<i>vector</i>	取得した数値の配列を格納する変数

戻り値

取得に成功すれば true

[parseconfig.h](#) の 143 行目に定義があります。

10.22.2.7 getValue() [4/4]

```
template<typename T>
bool getValue (
    const picojson::object & object,
    const std::string & key,
    T & scalar)
```

構成ファイルの JSON オブジェクトから数値を取得する

テンプレート引数

<i>T</i>	構成ファイルから取得する数値のデータ型
----------	---------------------

引数

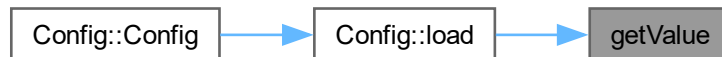
<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	取得する JSON オブジェクトのキー
<i>scalar</i>	取得した JSON オブジェクトの数値を格納する変数

戻り値

取得に成功すれば true

[parseconfig.h](#) の 33 行目に定義があります。

被呼び出し関係図:



10.22.2.8 getVector()

```
bool getVector (
    const picojson::object & object,
    const std::string & key,
    gg::GgVector & vector) [inline]
```

構成ファイルの JSON オブジェクトから 4 要素の数値のベクトルを取得する

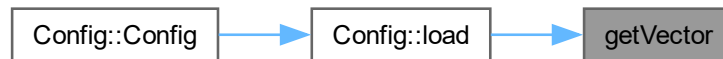
引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	取得する JSON オブジェクトのキー
<i>vector</i>	取得した数値のベクトルを格納する変数

戻り値

取得に成功すれば `true`

[parseconfig.h](#) の 184 行目に定義があります。
被呼び出し関係図:



10.22.2.9 setString() [1/3]

```

template<std::size_t U>
void setString (
    picojson::object & object,
    const std::string & key,
    const std::array< std::string, U > & strings)
  
```

構成ファイルの JSON オブジェクトに文字列の固定長配列を設定する

テンプレート引数

<i>U</i>	構成ファイルに設定する配列の要素の文字列の数
----------	------------------------

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	設定する JSON オブジェクトのキー
<i>strings</i>	設定する文字列の配列

[parseconfig.h](#) の 436 行目に定義があります。

10.22.2.10 setString() [2/3]

```

void setString (
    picojson::object & object,
    const std::string & key,
    const std::string & string) [inline]
  
```

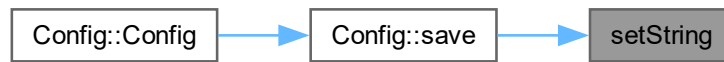
構成ファイルの JSON オブジェクトに文字列を設定する

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	設定する JSON オブジェクトのキー
<i>string</i>	設定する文字列

[parseconfig.h](#) の 421 行目に定義があります。

被呼び出し関係図:



10.22.2.11 setString() [3/3]

```
void setString (
    picojson::object & object,
    const std::string & key,
    const std::vector< std::string > & strings) [inline]
```

構成ファイルの JSON オブジェクトに文字列の可変長配列を設定する

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	設定する JSON オブジェクトのキー
<i>strings</i>	設定する文字列の配列

[parseconfig.h](#) の 462 行目に定義があります。

10.22.2.12 setValue() [1/4]

```
void setValue (
    picojson::object & object,
    const std::string & key,
    bool boolean) [inline]
```

構成ファイルの JSON オブジェクトに真偽値を設定する

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	設定する JSON オブジェクトのキー
<i>boolean</i>	設定する真偽値

[parseconfig.h](#) の 326 行目に定義があります。

10.22.2.13 setValue() [2/4]

```
template<typename T, std::size_t U>
void setValue (
    picojson::object & object,
    const std::string & key,
    const std::array< T, U > & vector)
```

構成ファイルの JSON オブジェクトに数値の固定長配列を設定する

テンプレート引数

<i>T</i>	構成ファイルに設定する配列の要素の数値のデータ型
<i>U</i>	構成ファイルに設定する配列の要素の数値の数

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	設定する JSON オブジェクトのキー
<i>vector</i>	設定する数値の配列

[parseconfig.h](#) の 342 行目に定義があります。

10.22.2.14 setValue() [3/4]

```
template<typename T>
void setValue (
    picojson::object & object,
    const std::string & key,
    const std::vector< T > & vector)
```

構成ファイルの JSON オブジェクトに数値の可変長配列を設定する

テンプレート引数

<i>T</i>	構成ファイルに設定する配列の要素の数値のデータ型
----------	--------------------------

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	設定する JSON オブジェクトのキー
<i>vector</i>	設定する数値の配列

[parseconfig.h](#) の 369 行目に定義があります。

10.22.2.15 setValue() [4/4]

```
template<typename T>
void setValue (
    picojson::object & object,
    const std::string & key,
    const T & scalar)
```

構成ファイルの JSON オブジェクトに数値を設定する

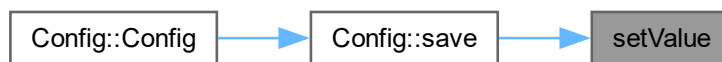
テンプレート引数

<i>T</i>	構成ファイルに設定する数値のデータ型
----------	--------------------

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	設定する JSON オブジェクトのキー
<i>scalar</i>	設定する数値

[parseconfig.h](#) の 312 行目に定義があります。
被呼び出し関係図:



10.22.2.16 setVector()

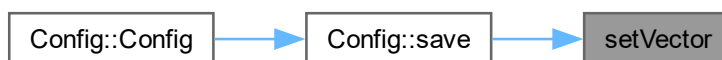
```
void setVector (
    picojson::object & object,
    const std::string & key,
    const gg::GgVector & vector) [inline]
```

構成ファイルの JSON オブジェクトに 4 要素の数値のベクトルを設定する

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	設定する JSON オブジェクトのキー
<i>vector</i>	設定するベクトル

[parseconfig.h](#) の 395 行目に定義があります。
被呼び出し関係図:



10.23 parseconfig.h

[詳解]

```
00001 #pragma once
00002
00010
00011 // 補助プログラム
00012 #include "gg.h"
00013
```

```

00014 // JSON
00015 #include "picojson.h"
00016
00017 // 標準ライブラリ
00018 #include <string>
00019 #include <vector>
00020 #include <array>
00021 #include <algorithm>
00022
00023 template <typename T>
00033 bool getValue(const picojson::object& object,
00034               const std::string& key, T& scalar)
00035 {
00036     // key に一致するオブジェクトを探す
00037     const auto value{ object.find(key) };
00038
00039     // オブジェクトが無かったら戻る
00040     if (value == object.end()) return false;
00041
00042     // 数値または真偽値として格納する
00043     if (value->second.is<double>())
00044     {
00045         scalar = static_cast<T>(value->second.get<double>());
00046         return true;
00047     }
00048     if (value->second.is<bool>())
00049     {
00050         scalar = static_cast<T>(value->second.get<bool>());
00051         return true;
00052     }
00053     return false;
00054 }
00055
00056 inline
00066 bool getValue(const picojson::object& object,
00067               const std::string& key, bool& boolean)
00068 {
00069     // key に一致するオブジェクトを探す
00070     const auto value{ object.find(key) };
00071
00072     // オブジェクトが無かったら戻る
00073     if (value == object.end()) return false;
00074
00075     // 真偽値または数値として格納する
00076     if (value->second.is<bool>())
00077     {
00078         boolean = value->second.get<bool>();
00079         return true;
00080     }
00081     if (value->second.is<double>())
00082     {
00083         boolean = value->second.get<double>() != 0.0;
00084         return true;
00085     }
00086     return false;
00087 }
00088
00089 template <typename T, std::size_t U>
00101 bool getValue(const picojson::object& object,
00102               const std::string& key, std::array<T, U>& vector)
00103 {
00104     // key に一致するオブジェクトを探す
00105     const auto value{ object.find(key) };
00106
00107     // オブジェクトが無いか配列でなかったら戻る
00108     if (value == object.end() || !value->second.is<picojson::array>()) return false;
00109
00110     // 配列を取り出す
00111     const auto& array{ value->second.get<picojson::array>() };
00112
00113     // 配列の要素数とデータの格納先の要素数の少ない方の数
00114     const auto n{ std::min(U, array.size()) };
00115
00116     // 配列の要素について
00117     for (std::size_t i = 0; i < n; ++i)
00118     {
00119         // 要素が数値なら格納する
00120         if (array[i].is<double>())
00121         {
00122             vector[i] = static_cast<T>(array[i].get<double>());
00123         }
00124         else if (array[i].is<bool>())
00125         {
00126             vector[i] = static_cast<T>(array[i].get<bool>());
00127         }
00128     }

```

```

00128     }
00129
00130     return true;
00131 }
00132
00142 template <typename T>
00143 bool getValue(const picojson::object& object,
00144              const std::string& key, std::vector<T>& vector)
00145 {
00146     // key に一致するオブジェクトを探す
00147     const auto value{ object.find(key) };
00148
00149     // オブジェクトが無いか配列でなかったら戻る
00150     if (value == object.end() || !value->second.is<picojson::array>()) return false;
00151
00152     // 配列を取り出す
00153     const auto& array{ value->second.get<picojson::array>() };
00154
00155     // メモリを確保してクリア
00156     vector.clear();
00157     vector.reserve(array.size());
00158
00159     // 配列の要素について
00160     for (const auto& element : array)
00161     {
00162         if (element.is<double>())
00163         {
00164             vector.push_back(static_cast<T>(element.get<double>()));
00165         }
00166         else if (element.is<bool>())
00167         {
00168             vector.push_back(static_cast<T>(element.get<bool>()));
00169         }
00170     }
00171
00172     return true;
00173 }
00174
00183 inline
00184 bool getVector(const picojson::object& object,
00185               const std::string& key, gg::GgVector& vector)
00186 {
00187     // key に一致するオブジェクトを探す
00188     const auto value{ object.find(key) };
00189
00190     // オブジェクトが無いか配列でなかったら戻る
00191     if (value == object.end() || !value->second.is<picojson::array>()) return false;
00192
00193     // 配列を取り出す
00194     const auto& array{ value->second.get<picojson::array>() };
00195
00196     // 配列の要素数とデータの格納先の要素数の少ない方の数
00197     const auto n{ std::min(vector.size(), array.size()) };
00198
00199     // 配列の要素について
00200     for (std::size_t i = 0; i < n; ++i)
00201     {
00202         // 要素が数値なら格納する
00203         if (array[i].is<double>()) vector[i] = static_cast<GLfloat>(array[i].get<double>());
00204     }
00205
00206     return true;
00207 }
00208
00217 inline
00218 bool getString(const picojson::object& object,
00219               const std::string& key, std::string& string)
00220 {
00221     // key に一致するオブジェクトを探す
00222     const auto value{ object.find(key) };
00223
00224     // オブジェクトが無いか文字列でなかったら戻る
00225     if (value == object.end() || !value->second.is<std::string>()) return false;
00226
00227     // 文字列として格納する
00228     string = value->second.get<std::string>();
00229
00230     return true;
00231 }
00232
00242 template <std::size_t U>
00243 bool getString(const picojson::object& object,
00244               const std::string& key, std::array<std::string, U>& strings)
00245 {
00246     // key に一致するオブジェクトを探す
00247     const auto value{ object.find(key) };
00248

```

```

00249 // オブジェクトが無いか配列でなかったら戻る
00250 if (value == object.end() || !value->second.is<picojson::array>()) return false;
00251
00252 // 配列を取り出す
00253 const auto& array{ value->second.get<picojson::array>() };
00254
00255 // 配列の要素数とデータの格納先の要素数の少ない方の数
00256 const auto n{ std::min(U, array.size()) };
00257
00258 // 配列の要素について
00259 for (std::size_t i = 0; i < n; ++i)
00260 {
00261     // 要素が文字列なら文字列として格納する
00262     strings[i] = array[i].is<std::string>() ? array[i].get<std::string>() : "";
00263 }
00264
00265 return true;
00266 }
00267
00276 inline
00277 bool getString(const picojson::object& object,
00278               const std::string& key, std::vector<std::string>& strings)
00279 {
00280     // key に一致するオブジェクトを探す
00281     const auto value{ object.find(key) };
00282
00283     // オブジェクトが無いか配列でなかったら戻る
00284     if (value == object.end() || !value->second.is<picojson::array>()) return false;
00285
00286     // 配列を取り出す
00287     const auto& array{ value->second.get<picojson::array>() };
00288
00289     // メモリを確保してクリア
00290     strings.clear();
00291     strings.reserve(array.size());
00292
00293     // 配列のすべての要素について
00294     for (const auto& element : array)
00295     {
00296         // 要素が文字列なら文字列として格納する
00297         strings.emplace_back(element.is<std::string>() ? element.get<std::string>() : "");
00298     }
00299
00300     return true;
00301 }
00302
00311 template <typename T>
00312 void setValue(picojson::object& object,
00313              const std::string& key, const T& scalar)
00314 {
00315     object[key] = picojson::value(static_cast<double>(scalar));
00316 }
00317
00325 inline
00326 void setValue(picojson::object& object,
00327              const std::string& key, bool boolean)
00328 {
00329     object[key] = picojson::value(boolean);
00330 }
00331
00341 template <typename T, std::size_t U>
00342 void setValue(picojson::object& object,
00343              const std::string& key, const std::array<T, U>& vector)
00344 {
00345     // picojson の配列
00346     picojson::array array;
00347     array.reserve(vector.size());
00348
00349     // 配列のすべての要素について
00350     for (const auto& element : vector)
00351     {
00352         // 要素を picojson::array に追加する
00353         array.emplace_back(picojson::value(static_cast<double>(element)));
00354     }
00355
00356     // オブジェクトに設定する
00357     object[key] = picojson::value(std::move(array));
00358 }
00359
00368 template <typename T>
00369 void setValue(picojson::object& object,
00370              const std::string& key, const std::vector<T>& vector)
00371 {
00372     // picojson の配列
00373     picojson::array array;
00374     array.reserve(vector.size());
00375

```

```

00376 // 配列のすべての要素について
00377 for (const auto& element : vector)
00378 {
00379     // 要素を picojson::array に追加する
00380     array.emplace_back(picojson::value(static_cast<double>(element)));
00381 }
00382
00383 // オブジェクトに設定する
00384 object[key] = picojson::value(std::move(array));
00385 }
00386
00394 inline
00395 void setVector(picojson::object& object,
00396               const std::string& key, const gg::GgVector& vector)
00397 {
00398     // picojson の配列
00399     picojson::array array;
00400     array.reserve(vector.size());
00401
00402     // ベクトルのすべての要素について
00403     for (const auto& element : vector)
00404     {
00405         // 要素を picojson::array に追加する
00406         array.emplace_back(picojson::value(static_cast<double>(element)));
00407     }
00408
00409     // オブジェクトに設定する
00410     object[key] = picojson::value(std::move(array));
00411 }
00412
00420 inline
00421 void setString(picojson::object& object,
00422               const std::string& key, const std::string& string)
00423 {
00424     object[key] = picojson::value(string);
00425 }
00426
00435 template <std::size_t U>
00436 void setString(picojson::object& object,
00437               const std::string& key, const std::array<std::string, U>& strings)
00438 {
00439     // picojson の配列
00440     picojson::array array;
00441     array.reserve(strings.size());
00442
00443     // 配列のすべての要素について
00444     for (const auto& string : strings)
00445     {
00446         // 要素を picojson::array に追加する
00447         array.emplace_back(picojson::value(string));
00448     }
00449
00450     // オブジェクトに設定する
00451     object[key] = picojson::value(std::move(array));
00452 }
00453
00461 inline
00462 void setString(picojson::object& object,
00463               const std::string& key, const std::vector<std::string>& strings)
00464 {
00465     // picojson の配列
00466     picojson::array array;
00467     array.reserve(strings.size());
00468
00469     // 配列のすべての要素について
00470     for (const auto& string : strings)
00471     {
00472         // 要素を picojson::array に追加する
00473         array.emplace_back(picojson::value(string));
00474     }
00475
00476     // オブジェクトに設定する
00477     object[key] = picojson::value(std::move(array));
00478 }

```

10.24 README.md ファイル

Index

- .ggError
 - gg, [28](#)
- .ggFBOError
 - gg, [28](#)
- ~Config
 - Config, [90](#)
- ~GgApp
 - GgApp, [95](#)
- ~GgBuffer
 - gg::GgBuffer< T >, [101](#)
- ~GgColorTexture
 - gg::GgColorTexture, [110](#)
- ~GgElements
 - gg::GgElements, [116](#)
- ~GgMatrix
 - gg::GgMatrix, [124](#)
- ~GgNormalTexture
 - gg::GgNormalTexture, [177](#)
- ~GgPointShader
 - gg::GgPointShader, [188](#)
- ~GgPoints
 - gg::GgPoints, [183](#)
- ~GgQuaternion
 - gg::GgQuaternion, [204](#)
- ~GgShader
 - gg::GgShader, [266](#)
- ~GgShape
 - gg::GgShape, [268](#)
- ~GgSimpleObj
 - gg::GgSimpleObj, [271](#)
- ~GgSimpleShader
 - gg::GgSimpleShader, [277](#)
- ~GgTexture
 - gg::GgTexture, [295](#)
- ~GgTrackball
 - gg::GgTrackball, [303](#)
- ~GgTriangles
 - gg::GgTriangles, [312](#)
- ~GgUniformBuffer
 - gg::GgUniformBuffer< T >, [316](#)
- ~GgVector
 - gg::GgVector, [331](#)
- ~GgVertexArray
 - gg::GgVertexArray, [348](#)
- ~LightBuffer
 - gg::GgSimpleShader::LightBuffer, [354](#)
- ~MaterialBuffer
 - gg::GgSimpleShader::MaterialBuffer, [372](#)
- ~Menu
 - Menu, [385](#)
- ~Window
 - GgApp::Window, [391](#)
- add
 - gg::GgQuaternion, [204](#), [205](#)
- ambient
 - gg::GgSimpleShader::Light, [350](#)
 - gg::GgSimpleShader::Material, [367](#)
- begin
 - gg::GgTrackball, [303](#)
- bind
 - gg::GgBuffer< T >, [101](#)
 - gg::GgColorTexture, [110](#)
 - gg::GgNormalTexture, [177](#)
 - gg::GgTexture, [295](#)
 - gg::GgUniformBuffer< T >, [317](#)
 - gg::GgVertexArray, [348](#)
- BindingPoints
 - gg, [28](#)
- Config, [89](#)
 - ~Config, [90](#)
 - Config, [89](#)
 - getHeight, [90](#)
 - getWidth, [91](#)
 - load, [91](#)
 - Menu, [93](#)
 - save, [92](#)
- Config.cpp, [415](#)
 - defaultLight, [416](#)
- Config.h, [418](#)
- CONFIG_FILE
 - ggsample01.cpp, [587](#)
- conjugate
 - gg::GgQuaternion, [207](#)
- copy
 - gg::GgBuffer< T >, [101](#)
 - gg::GgUniformBuffer< T >, [317](#)
- defaultLight
 - Config.cpp, [416](#)
- diffuse
 - gg::GgSimpleShader::Light, [350](#)
 - gg::GgSimpleShader::Material, [367](#)
- distance3
 - gg::GgVector, [331](#)
- distance4
 - gg::GgVector, [332](#)

- divide
 - gg::GgQuaternion, 208, 209
- dot3
 - gg::GgVector, 332
- dot4
 - gg::GgVector, 333
- draw
 - gg::GgElements, 116
 - gg::GgPoints, 183
 - gg::GgShape, 268
 - gg::GgSimpleObj, 272
 - gg::GgTriangles, 312
 - Menu, 386
- end
 - gg::GgTrackball, 303
- euler
 - gg::GgQuaternion, 210, 211
- fill
 - gg::GgUniformBuffer< T >, 318
- frustum
 - gg::GgMatrix, 124
- get
 - gg::GgColorTexture, 110
 - gg::GgMatrix, 125, 126
 - gg::GgNormalTexture, 177
 - gg::GgPointShader, 188
 - gg::GgQuaternion, 212
 - gg::GgShader, 266
 - gg::GgShape, 269
 - gg::GgSimpleObj, 272
 - gg::GgTrackball, 304
 - gg::GgVertexArray, 348
 - GgApp::Window, 391
- getAltArrow
 - GgApp::Window, 391
- getAltArrowX
 - GgApp::Window, 391
- getAltArrowY
 - GgApp::Window, 392
- getArrow
 - GgApp::Window, 393
- getArrowX
 - GgApp::Window, 394
- getArrowY
 - GgApp::Window, 395
- getAspect
 - GgApp::Window, 395
- getBuffer
 - gg::GgBuffer< T >, 102
 - gg::GgPoints, 183
 - gg::GgTriangles, 313
 - gg::GgUniformBuffer< T >, 319
- getConjugateMatrix
 - gg::GgQuaternion, 213, 214
- getControlArrow
 - GgApp::Window, 396
- getControlArrowX
 - GgApp::Window, 396
- getControlArrowY
 - GgApp::Window, 397
- getCount
 - gg::GgBuffer< T >, 102
 - gg::GgPoints, 184
 - gg::GgTriangles, 313
 - gg::GgUniformBuffer< T >, 320
- getFboHeight
 - GgApp::Window, 398
- getFboSize
 - GgApp::Window, 398
- getFboWidth
 - GgApp::Window, 399
- getHeight
 - Config, 90
 - gg::GgColorTexture, 110
 - gg::GgNormalTexture, 178
 - gg::GgTexture, 295
 - GgApp::Window, 399
- getIndexBuffer
 - gg::GgElements, 116
- getIndexCount
 - gg::GgElements, 117
- getKey
 - GgApp::Window, 399
- getLastKey
 - GgApp::Window, 400
- getLight
 - Menu, 386
- getMatrix
 - gg::GgQuaternion, 214, 215
 - gg::GgTrackball, 304
- getMode
 - gg::GgShape, 269
- getModel
 - Menu, 386
- getMouse
 - GgApp::Window, 400
- getMouseX
 - GgApp::Window, 400
- getMouseY
 - GgApp::Window, 401
- getQuaternion
 - gg::GgTrackball, 304
- getRotation
 - GgApp::Window, 401
- getRotationMatrix
 - GgApp::Window, 401
- getScale
 - gg::GgTrackball, 304, 305
- getScrollMatrix
 - GgApp::Window, 402
- getShader
 - Menu, 386
- getShiftArrow
 - GgApp::Window, 402

- getShiftArrowX
 - GgApp::Window, 402
- getShiftArrowY
 - GgApp::Window, 403
- getSize
 - gg::GgColorTexture, 110
 - gg::GgNormalTexture, 178
 - gg::GgTexture, 295
 - GgApp::Window, 404
- getStart
 - gg::GgTrackball, 305
- getStride
 - gg::GgBuffer< T >, 102
 - gg::GgUniformBuffer< T >, 320
- getString
 - parseconfig.h, 597, 598
- getTarget
 - gg::GgBuffer< T >, 103
 - gg::GgUniformBuffer< T >, 321
- getTexture
 - gg::GgColorTexture, 111
 - gg::GgNormalTexture, 178
 - gg::GgTexture, 296
- getTranslation
 - GgApp::Window, 404
- getTranslationMatrix
 - GgApp::Window, 405
- getUsername
 - GgApp, 95
- getUserPointer
 - GgApp::Window, 405
- getValue
 - parseconfig.h, 598, 599
- getVector
 - parseconfig.h, 600
- getWheel
 - GgApp::Window, 405, 406
- getWheelX
 - GgApp::Window, 406
- getWheelY
 - GgApp::Window, 406
- getWidth
 - Config, 91
 - gg::GgColorTexture, 111
 - gg::GgNormalTexture, 178
 - gg::GgTexture, 296
 - GgApp::Window, 406
- gg, 25
 - _ggError, 28
 - _ggFBOError, 28
 - BindingPoints, 28
 - ggArraysObj, 28
 - ggBufferAlignment, 87
 - ggConjugate, 29
 - ggCreateComputeShader, 29
 - ggCreateNormalMap, 30
 - ggCreateShader, 31
 - ggCross, 32
 - ggDistance3, 33
 - ggDistance4, 34, 35
 - ggDot3, 35, 36
 - ggDot4, 37
 - ggElementsMesh, 38
 - ggElementsObj, 39
 - ggElementsSphere, 40
 - ggEllipse, 40
 - ggEulerQuaternion, 42, 43
 - ggFrustum, 44
 - ggIdentity, 44
 - ggIdentityQuaternion, 45
 - ggInit, 45
 - ggInvert, 46
 - ggLength3, 47
 - ggLength4, 48, 49
 - ggLoadComputeShader, 49
 - ggLoadHeight, 50
 - ggLoadImage, 51
 - ggLoadShader, 51, 52
 - ggLoadSimpleObj, 53, 54
 - ggLoadTexture, 54
 - ggLookat, 55, 56
 - ggMatrixQuaternion, 57, 58
 - ggNorm, 59
 - ggNormal, 59
 - ggNormalize, 59
 - ggNormalize3, 60–62
 - ggNormalize4, 62–64
 - ggOrthogonal, 64
 - ggPerspective, 65
 - ggPointsCube, 66
 - ggPointsSphere, 66
 - ggQuaternionMatrix, 67
 - ggQuaternionTransposeMatrix, 67
 - ggReadImage, 68
 - ggRectangle, 69
 - ggRotate, 69–71
 - ggRotateQuaternion, 72, 73
 - ggRotateX, 74
 - ggRotateY, 74
 - ggRotateZ, 75
 - ggSaveColor, 75
 - ggSaveDepth, 76
 - ggSaveTga, 76
 - ggScale, 77, 78
 - ggSlerp, 79, 80
 - ggTranslate, 81, 82
 - ggTranspose, 84
 - LightBindingPoint, 28
 - MaterialBindingPoint, 28
 - operator+, 86
 - operator-, 86, 87
 - operator/, 87
 - operator*, 84
- gg.cpp, 420
- gg.h, 496
- ggError, 501

- gg::GgFBOError, 501
- pathChar, 501
- pathString, 501
- TCharToUtf8, 501
- Utf8ToTChar, 501
- gg::GgBuffer< T >, 98
 - ~GgBuffer, 101
 - bind, 101
 - copy, 101
 - getBuffer, 102
 - getCount, 102
 - getStride, 102
 - getTarget, 103
 - GgBuffer, 99, 100
 - map, 103, 104
 - operator=, 104, 106
 - read, 106
 - send, 107
 - unbind, 107
 - unmap, 108
- gg::GgColorTexture, 108
 - ~GgColorTexture, 110
 - bind, 110
 - get, 110
 - getHeight, 110
 - getSize, 110
 - getTexture, 111
 - getWidth, 111
 - GgColorTexture, 108, 109
 - load, 111, 112
 - operator bool, 112
 - operator!, 113
 - unbind, 113
- gg::GgElements, 113
 - ~GgElements, 116
 - draw, 116
 - getIndexBuffer, 116
 - getIndexCount, 117
 - GgElements, 115
 - load, 117
 - send, 118
- gg::GgMatrix, 119
 - ~GgMatrix, 124
 - frustum, 124
 - get, 125, 126
 - GgMatrix, 121–124
 - invert, 127
 - loadFrustum, 127
 - loadIdentity, 128
 - loadInvert, 128, 129
 - loadLookat, 129, 131, 132
 - loadNormal, 133
 - loadOrthogonal, 134
 - loadPerspective, 135
 - loadRotate, 135, 137–139
 - loadRotateX, 140
 - loadRotateY, 141
 - loadRotateZ, 141
 - loadScale, 142, 143
 - loadTranslate, 144, 145
 - loadTranspose, 146
 - lookat, 147, 148
 - normal, 149
 - operator+, 153, 155
 - operator+=, 155, 156
 - operator-, 157
 - operator-=, 158
 - operator/, 160, 161
 - operator/=: 161, 162
 - operator=, 162, 163
 - operator*, 150, 151
 - operator*=, 152, 153
 - orthogonal, 164
 - perspective, 164
 - projection, 165, 166
 - rotate, 166–168
 - rotateX, 169
 - rotateY, 170
 - rotateZ, 170
 - scale, 171, 172
 - translate, 173, 174
 - transpose, 175
- gg::GgNormalTexture, 176
 - ~GgNormalTexture, 177
 - bind, 177
 - get, 177
 - getHeight, 178
 - getSize, 178
 - getTexture, 178
 - getWidth, 178
 - GgNormalTexture, 176, 177
 - load, 178, 179
 - operator bool, 180
 - operator!, 180
 - unbind, 180
- gg::GgPoints, 180
 - ~GgPoints, 183
 - draw, 183
 - getBuffer, 183
 - getCount, 184
 - GgPoints, 182
 - load, 184
 - operator bool, 184
 - operator!, 185
 - send, 185
- gg::GgPointShader, 185
 - ~GgPointShader, 188
 - get, 188
 - GgPointShader, 186–188
 - load, 189
 - loadMatrix, 190, 191
 - loadModelviewMatrix, 192
 - loadProjectionMatrix, 193
 - unuse, 194
 - use, 194–197
- gg::GgQuaternion, 197

- ~GgQuaternion, 204
- add, 204, 205
- conjugate, 207
- divide, 208, 209
- euler, 210, 211
- get, 212
- getConjugateMatrix, 213, 214
- getMatrix, 214, 215
- GgQuaternion, 201–203
- invert, 216
- loadAdd, 217, 218
- loadConjugate, 219, 220
- loadDivide, 220–222
- loadEuler, 223
- loadIdentity, 224
- loadInvert, 225
- loadMatrix, 226, 227
- loadMultiply, 227, 228, 230
- loadNormalize, 231
- loadRotate, 232, 234
- loadRotateX, 235
- loadRotateY, 235
- loadRotateZ, 236
- loadSlerp, 237–239
- loadSubtract, 240, 241
- multiply, 242–244
- norm, 244
- normalize, 245
- operator+, 248
- operator+=, 249
- operator-, 250
- operator=, 251
- operator/, 252
- operator/=, 253, 254
- operator=, 254–256
- operator*, 246
- operator*=, 247
- rotate, 256, 257
- rotateX, 258
- rotateY, 259
- rotateZ, 259
- slerp, 260
- subtract, 261, 262
- gg::GgShader, 263
 - ~GgShader, 266
 - get, 266
 - GgShader, 264–266
 - operator=, 266
 - unuse, 267
 - use, 267
- gg::GgShape, 267
 - ~GgShape, 268
 - draw, 268
 - get, 269
 - getMode, 269
 - GgShape, 268
 - operator bool, 270
 - operator!, 270
 - setMode, 270
- gg::GgSimpleObj, 271
 - ~GgSimpleObj, 271
 - draw, 272
 - get, 272
 - GgSimpleObj, 271
 - operator bool, 272
 - operator!, 272
- gg::GgSimpleShader, 273
 - ~GgSimpleShader, 277
 - GgSimpleShader, 274–276
 - load, 277
 - loadMatrix, 278–280
 - loadModelviewMatrix, 281, 282
 - operator=, 283
 - use, 284–291
- gg::GgSimpleShader::Light, 349
 - ambient, 350
 - diffuse, 350
 - position, 350
 - specular, 350
- gg::GgSimpleShader::LightBuffer, 350
 - ~LightBuffer, 354
 - LightBuffer, 352, 353
 - load, 354, 355
 - loadAmbient, 355, 356
 - loadColor, 357
 - loadDiffuse, 358, 359
 - loadPosition, 360–362
 - loadSpecular, 363, 364
 - select, 365
- gg::GgSimpleShader::Material, 366
 - ambient, 367
 - diffuse, 367
 - shininess, 367
 - specular, 368
- gg::GgSimpleShader::MaterialBuffer, 368
 - ~MaterialBuffer, 372
 - load, 372, 373
 - loadAmbient, 373–375
 - loadAmbientAndDiffuse, 375–377
 - loadDiffuse, 378, 379
 - loadShininess, 380
 - loadSpecular, 381, 382
 - MaterialBuffer, 370, 371
 - select, 383
- gg::GgTexture, 292
 - ~GgTexture, 295
 - bind, 295
 - getHeight, 295
 - getSize, 295
 - getTexture, 296
 - getWidth, 296
 - GgTexture, 293, 294
 - operator=, 296, 297
 - swapRandB, 297
 - unbind, 298
- gg::GgTrackball, 298

- ~GgTrackball, 303
- begin, 303
- end, 303
- get, 304
- getMatrix, 304
- getQuaternion, 304
- getScale, 304, 305
- getStart, 305
- GgTrackball, 302
- motion, 306
- operator=, 306
- region, 307
- reset, 308
- rotate, 309
- gg::GgTriangles, 309
 - ~GgTriangles, 312
 - draw, 312
 - getBuffer, 313
 - getCount, 313
 - GgTriangles, 311
 - load, 313
 - send, 314
- gg::GgUniformBuffer< T >, 314
 - ~GgUniformBuffer, 316
 - bind, 317
 - copy, 317
 - fill, 318
 - getBuffer, 319
 - getCount, 320
 - getStride, 320
 - getTarget, 321
 - GgUniformBuffer, 315, 316
 - load, 321, 322
 - map, 323
 - read, 324
 - send, 325
 - unbind, 326
 - unmap, 326
- gg::GgVector, 327
 - ~GgVector, 331
 - distance3, 331
 - distance4, 332
 - dot3, 332
 - dot4, 333
 - GgVector, 328–331
 - length3, 333
 - length4, 334
 - normalize3, 334
 - normalize4, 335
 - operator+, 337
 - operator+=, 338
 - operator-, 339
 - operator-=, 340
 - operator/, 341
 - operator/=, 342
 - operator=, 343
 - operator*, 335
 - operator*=, 336
- gg::GgVertex, 344
 - GgVertex, 344, 345
 - normal, 346
 - position, 346
- gg::GgVertexArray, 346
 - ~GgVertexArray, 348
 - bind, 348
 - get, 348
 - GgVertexArray, 347, 348
 - operator=, 348
- GG_BUTTON_COUNT
 - GgApp.h, 578
- GG_INTERFACE_COUNT
 - GgApp.h, 578
- GG_USE_IMGUI
 - GgApp.h, 578
- GgApp, 93
 - ~GgApp, 95
 - getUsername, 95
 - GgApp, 94, 95
 - main, 96
 - operator=, 97, 98
- GgApp.cpp, 558
- GgApp.h, 577
 - GG_BUTTON_COUNT, 578
 - GG_INTERFACE_COUNT, 578
 - GG_USE_IMGUI, 578
- GgApp::Window, 387
 - ~Window, 391
 - get, 391
 - getAltArrow, 391
 - getAltArrowX, 391
 - getAltArrowY, 392
 - getArrow, 393
 - getArrowX, 394
 - getArrowY, 395
 - getAspect, 395
 - getControlArrow, 396
 - getControlArrowX, 396
 - getControlArrowY, 397
 - getFboHeight, 398
 - getFboSize, 398
 - getFboWidth, 399
 - getHeight, 399
 - getKey, 399
 - getLastKey, 400
 - getMouse, 400
 - getMouseX, 400
 - getMouseY, 401
 - getRotation, 401
 - getRotationMatrix, 401
 - getScrollMatrix, 402
 - getShiftArrow, 402
 - getShiftArrowX, 402
 - getShiftArrowY, 403
 - getSize, 404
 - getTranslation, 404
 - getTranslationMatrix, 405

- getUserPointer, 405
- getWheel, 405, 406
- getWheelX, 406
- getWheelY, 406
- getWidth, 406
- operator bool, 407
- operator=, 407, 408
- reset, 408
- resetRotation, 408
- resetTranslation, 409
- restoreViewport, 409
- selectInterface, 409
- setClose, 410
- setKeyboardFunc, 410
- setMouseFunc, 410
- setResizeFunc, 411
- setUserPointer, 411
- setVelocity, 411
- setWheelFunc, 412
- shouldClose, 412
- swapBuffers, 412
- updateViewport, 413
- Window, 389, 390
- ggArraysObj
 - gg, 28
- GgBuffer
 - gg::GgBuffer< T >, 99, 100
- ggBufferAlignment
 - gg, 87
- GgColorTexture
 - gg::GgColorTexture, 108, 109
- ggConjugate
 - gg, 29
- ggCreateComputeShader
 - gg, 29
- ggCreateNormalMap
 - gg, 30
- ggCreateShader
 - gg, 31
- ggCross
 - gg, 32
- ggDistance3
 - gg, 33
- ggDistance4
 - gg, 34, 35
- ggDot3
 - gg, 35, 36
- ggDot4
 - gg, 37
- GgElements
 - gg::GgElements, 115
- ggElementsMesh
 - gg, 38
- ggElementsObj
 - gg, 39
- ggElementsSphere
 - gg, 40
- ggEllipse
 - gg, 40
- ggError
 - gg.h, 501
- ggEulerQuaternion
 - gg, 42, 43
- ggFBOError
 - gg.h, 501
- ggFrustum
 - gg, 44
- ggIdentity
 - gg, 44
- ggIdentityQuaternion
 - gg, 45
- ggInit
 - gg, 45
- ggInvert
 - gg, 46
- ggLength3
 - gg, 47
- ggLength4
 - gg, 48, 49
- ggLoadComputeShader
 - gg, 49
- ggLoadHeight
 - gg, 50
- ggLoadImage
 - gg, 51
- ggLoadShader
 - gg, 51, 52
- ggLoadSimpleObj
 - gg, 53, 54
- ggLoadTexture
 - gg, 54
- ggLookat
 - gg, 55, 56
- GgMatrix
 - gg::GgMatrix, 121–124
- ggMatrixQuaternion
 - gg, 57, 58
- ggNorm
 - gg, 59
- ggNormal
 - gg, 59
- ggNormalize
 - gg, 59
- ggNormalize3
 - gg, 60–62
- ggNormalize4
 - gg, 62–64
- GgNormalTexture
 - gg::GgNormalTexture, 176, 177
- ggOrthogonal
 - gg, 64
- ggPerspective
 - gg, 65
- GgPoints
 - gg::GgPoints, 182
- ggPointsCube

- gg, 66
- GgPointShader
 - gg::GgPointShader, 186–188
- ggPointsSphere
 - gg, 66
- GgQuaternion
 - gg::GgQuaternion, 201–203
- ggQuaternionMatrix
 - gg, 67
- ggQuaternionTransposeMatrix
 - gg, 67
- ggReadImage
 - gg, 68
- ggRectangle
 - gg, 69
- ggRotate
 - gg, 69–71
- ggRotateQuaternion
 - gg, 72, 73
- ggRotateX
 - gg, 74
- ggRotateY
 - gg, 74
- ggRotateZ
 - gg, 75
- ggsample01, 13
- ggsample01.cpp, 587
 - CONFIG_FILE, 587
 - PROJECT_NAME, 587
- ggSaveColor
 - gg, 75
- ggSaveDepth
 - gg, 76
- ggSaveTga
 - gg, 76
- ggScale
 - gg, 77, 78
- GgShader
 - gg::GgShader, 264–266
- GgShape
 - gg::GgShape, 268
- GgSimpleObj
 - gg::GgSimpleObj, 271
- GgSimpleShader
 - gg::GgSimpleShader, 274–276
- ggSlerp
 - gg, 79, 80
- GgTexture
 - gg::GgTexture, 293, 294
- GgTrackball
 - gg::GgTrackball, 302
- ggTranslate
 - gg, 81, 82
- ggTranspose
 - gg, 84
- GgTriangles
 - gg::GgTriangles, 311
- GgUniformBuffer
 - gg::GgUniformBuffer< T >, 315, 316
- GgVector
 - gg::GgVector, 328–331
- GgVertex
 - gg::GgVertex, 344, 345
- GgVertexArray
 - gg::GgVertexArray, 347, 348
- HEADER_STR
 - main.cpp, 589
- invert
 - gg::GgMatrix, 127
 - gg::GgQuaternion, 216
- length3
 - gg::GgVector, 333
- length4
 - gg::GgVector, 334
- LightBindingPoint
 - gg, 28
- LightBuffer
 - gg::GgSimpleShader::LightBuffer, 352, 353
- load
 - Config, 91
 - gg::GgColorTexture, 111, 112
 - gg::GgElements, 117
 - gg::GgNormalTexture, 178, 179
 - gg::GgPoints, 184
 - gg::GgPointShader, 189
 - gg::GgSimpleShader, 277
 - gg::GgSimpleShader::LightBuffer, 354, 355
 - gg::GgSimpleShader::MaterialBuffer, 372, 373
 - gg::GgTriangles, 313
 - gg::GgUniformBuffer< T >, 321, 322
- loadAdd
 - gg::GgQuaternion, 217, 218
- loadAmbient
 - gg::GgSimpleShader::LightBuffer, 355, 356
 - gg::GgSimpleShader::MaterialBuffer, 373–375
- loadAmbientAndDiffuse
 - gg::GgSimpleShader::MaterialBuffer, 375–377
- loadColor
 - gg::GgSimpleShader::LightBuffer, 357
- loadConjugate
 - gg::GgQuaternion, 219, 220
- loadDiffuse
 - gg::GgSimpleShader::LightBuffer, 358, 359
 - gg::GgSimpleShader::MaterialBuffer, 378, 379
- loadDivide
 - gg::GgQuaternion, 220–222
- loadEuler
 - gg::GgQuaternion, 223
- loadFrustum
 - gg::GgMatrix, 127
- loadIdentity
 - gg::GgMatrix, 128
 - gg::GgQuaternion, 224
- loadInvert

- gg::GgMatrix, [128](#), [129](#)
- gg::GgQuaternion, [225](#)
- loadLookat
 - gg::GgMatrix, [129](#), [131](#), [132](#)
- loadMatrix
 - gg::GgPointShader, [190](#), [191](#)
 - gg::GgQuaternion, [226](#), [227](#)
 - gg::GgSimpleShader, [278–280](#)
- loadModelviewMatrix
 - gg::GgPointShader, [192](#)
 - gg::GgSimpleShader, [281](#), [282](#)
- loadMultiply
 - gg::GgQuaternion, [227](#), [228](#), [230](#)
- loadNormal
 - gg::GgMatrix, [133](#)
- loadNormalize
 - gg::GgQuaternion, [231](#)
- loadOrthogonal
 - gg::GgMatrix, [134](#)
- loadPerspective
 - gg::GgMatrix, [135](#)
- loadPosition
 - gg::GgSimpleShader::LightBuffer, [360–362](#)
- loadProjectionMatrix
 - gg::GgPointShader, [193](#)
- loadRotate
 - gg::GgMatrix, [135](#), [137–139](#)
 - gg::GgQuaternion, [232](#), [234](#)
- loadRotateX
 - gg::GgMatrix, [140](#)
 - gg::GgQuaternion, [235](#)
- loadRotateY
 - gg::GgMatrix, [141](#)
 - gg::GgQuaternion, [235](#)
- loadRotateZ
 - gg::GgMatrix, [141](#)
 - gg::GgQuaternion, [236](#)
- loadScale
 - gg::GgMatrix, [142](#), [143](#)
- loadShininess
 - gg::GgSimpleShader::MaterialBuffer, [380](#)
- loadSlerp
 - gg::GgQuaternion, [237–239](#)
- loadSpecular
 - gg::GgSimpleShader::LightBuffer, [363](#), [364](#)
 - gg::GgSimpleShader::MaterialBuffer, [381](#), [382](#)
- loadSubtract
 - gg::GgQuaternion, [240](#), [241](#)
- loadTranslate
 - gg::GgMatrix, [144](#), [145](#)
- loadTranspose
 - gg::GgMatrix, [146](#)
- lookat
 - gg::GgMatrix, [147](#), [148](#)
- main
 - GgApp, [96](#)
 - main.cpp, [589](#)
- main.cpp, [589](#)
- HEADER_STR, [589](#)
- main, [589](#)
- map
 - gg::GgBuffer< T >, [103](#), [104](#)
 - gg::GgUniformBuffer< T >, [323](#)
- MaterialBindingPoint
 - gg, [28](#)
- MaterialBuffer
 - gg::GgSimpleShader::MaterialBuffer, [370](#), [371](#)
- Menu, [384](#)
 - ~Menu, [385](#)
 - Config, [93](#)
 - draw, [386](#)
 - getLight, [386](#)
 - getModel, [386](#)
 - getShader, [386](#)
 - Menu, [384](#), [385](#)
 - operator=, [387](#)
 - Scene, [387](#)
- Menu.cpp, [591](#)
- Menu.h, [593](#)
- motion
 - gg::GgTrackball, [306](#)
- multiply
 - gg::GgQuaternion, [242–244](#)
- norm
 - gg::GgQuaternion, [244](#)
- normal
 - gg::GgMatrix, [149](#)
 - gg::GgVertex, [346](#)
- normalize
 - gg::GgQuaternion, [245](#)
- normalize3
 - gg::GgVector, [334](#)
- normalize4
 - gg::GgVector, [335](#)
- OpenXR ラッパ (<tt>GgApp::OpenXR</tt>) マ
ニユアル & 使用方法解説, [3](#)
- OPENXR.md, [595](#)
- operator bool
 - gg::GgColorTexture, [112](#)
 - gg::GgNormalTexture, [180](#)
 - gg::GgPoints, [184](#)
 - gg::GgShape, [270](#)
 - gg::GgSimpleObj, [272](#)
 - GgApp::Window, [407](#)
- operator!
 - gg::GgColorTexture, [113](#)
 - gg::GgNormalTexture, [180](#)
 - gg::GgPoints, [185](#)
 - gg::GgShape, [270](#)
 - gg::GgSimpleObj, [272](#)
- operator+
 - gg, [86](#)
 - gg::GgMatrix, [153](#), [155](#)
 - gg::GgQuaternion, [248](#)
 - gg::GgVector, [337](#)

- operator+=
 - gg::GgMatrix, 155, 156
 - gg::GgQuaternion, 249
 - gg::GgVector, 338
- operator-
 - gg, 86, 87
 - gg::GgMatrix, 157
 - gg::GgQuaternion, 250
 - gg::GgVector, 339
- operator-=
 - gg::GgMatrix, 158
 - gg::GgQuaternion, 251
 - gg::GgVector, 340
- operator/
 - gg, 87
 - gg::GgMatrix, 160, 161
 - gg::GgQuaternion, 252
 - gg::GgVector, 341
- operator/=
 - gg::GgMatrix, 161, 162
 - gg::GgQuaternion, 253, 254
 - gg::GgVector, 342
- operator=
 - gg::GgBuffer< T >, 104, 106
 - gg::GgMatrix, 162, 163
 - gg::GgQuaternion, 254–256
 - gg::GgShader, 266
 - gg::GgSimpleShader, 283
 - gg::GgTexture, 296, 297
 - gg::GgTrackball, 306
 - gg::GgVector, 343
 - gg::GgVertexArray, 348
 - GgApp, 97, 98
 - GgApp::Window, 407, 408
 - Menu, 387
- operator*
 - gg, 84
 - gg::GgMatrix, 150, 151
 - gg::GgQuaternion, 246
 - gg::GgVector, 335
- operator*=
 - gg::GgMatrix, 152, 153
 - gg::GgQuaternion, 247
 - gg::GgVector, 336
- orthogonal
 - gg::GgMatrix, 164
- parseconfig.h, 595
 - getString, 597, 598
 - getValue, 598, 599
 - getVector, 600
 - setString, 601, 602
 - setValue, 602, 603
 - setVector, 604
- pathChar
 - gg.h, 501
- pathString
 - gg.h, 501
- perspective
 - gg::GgMatrix, 164
- position
 - gg::GgSimpleShader::Light, 350
 - gg::GgVertex, 346
- PROJECT_NAME
 - ggsample01.cpp, 587
- projection
 - gg::GgMatrix, 165, 166
- read
 - gg::GgBuffer< T >, 106
 - gg::GgUniformBuffer< T >, 324
- README.md, 608
- region
 - gg::GgTrackball, 307
- reset
 - gg::GgTrackball, 308
 - GgApp::Window, 408
- resetRotation
 - GgApp::Window, 408
- resetTranslation
 - GgApp::Window, 409
- restoreViewport
 - GgApp::Window, 409
- rotate
 - gg::GgMatrix, 166–168
 - gg::GgQuaternion, 256, 257
 - gg::GgTrackball, 309
- rotateX
 - gg::GgMatrix, 169
 - gg::GgQuaternion, 258
- rotateY
 - gg::GgMatrix, 170
 - gg::GgQuaternion, 259
- rotateZ
 - gg::GgMatrix, 170
 - gg::GgQuaternion, 259
- save
 - Config, 92
- scale
 - gg::GgMatrix, 171, 172
- Scene
 - Menu, 387
- select
 - gg::GgSimpleShader::LightBuffer, 365
 - gg::GgSimpleShader::MaterialBuffer, 383
- selectInterface
 - GgApp::Window, 409
- send
 - gg::GgBuffer< T >, 107
 - gg::GgElements, 118
 - gg::GgPoints, 185
 - gg::GgTriangles, 314
 - gg::GgUniformBuffer< T >, 325
- setClose
 - GgApp::Window, 410
- setKeyboardFunc
 - GgApp::Window, 410

- setMode
 - gg::GgShape, 270
- setMouseFunc
 - GgApp::Window, 410
- setResizeFunc
 - GgApp::Window, 411
- setString
 - parseconfig.h, 601, 602
- setUserPointer
 - GgApp::Window, 411
- setValue
 - parseconfig.h, 602, 603
- setVector
 - parseconfig.h, 604
- setVelocity
 - GgApp::Window, 411
- setWheelFunc
 - GgApp::Window, 412
- shininess
 - gg::GgSimpleShader::Material, 367
- shouldClose
 - GgApp::Window, 412
- slerp
 - gg::GgQuaternion, 260
- specular
 - gg::GgSimpleShader::Light, 350
 - gg::GgSimpleShader::Material, 368
- subtract
 - gg::GgQuaternion, 261, 262
- swapBuffers
 - GgApp::Window, 412
- swapRandB
 - gg::GgTexture, 297
- TCharToUtf8
 - gg.h, 501
- translate
 - gg::GgMatrix, 173, 174
- transpose
 - gg::GgMatrix, 175
- unbind
 - gg::GgBuffer< T >, 107
 - gg::GgColorTexture, 113
 - gg::GgNormalTexture, 180
 - gg::GgTexture, 298
 - gg::GgUniformBuffer< T >, 326
- unmap
 - gg::GgBuffer< T >, 108
 - gg::GgUniformBuffer< T >, 326
- unuse
 - gg::GgPointShader, 194
 - gg::GgShader, 267
- updateViewport
 - GgApp::Window, 413
- use
 - gg::GgPointShader, 194–197
 - gg::GgShader, 267
 - gg::GgSimpleShader, 284–291
- Utf8ToTChar
 - gg.h, 501
- Window
 - GgApp::Window, 389, 390
- ゲームグラフィックス特論の宿題用補助プログラム GLFW3 版., 1